MindVideo AI

بايثون 3.14

من الصفر إلى الاحتراف

دليل عملي متكامل لبناء عقلية المبرمج وإتقان اللغة
من أول سطر كود حتى نشر أول منتج حقيقي

تعلم
طبق
ابني مشاريع
انشر منتجك

م/أحمد صالح المغز

2026

بايثون 3.14
من الصفر إلى الاحتراف

دليل عملي متكامل لبناء عقلية المبرمج وإتقان اللغة — من أول سطر كود حتى نشر أول منتج حقيقي.

م/ أحمد صالح المغز

الإصدار الأول · 2026

مقدمة الكتاب

> *"أفضل وقت لتبدأ كان قبل عشر سنوات. ثاني أفضل وقت هو الآن."*

هذا الكتاب وُلد من ملاحظة بسيطة تكرّرت أمامي مئات المرات: معظم من يتعلّم البرمجة لا يفشل لأن اللغة صعبة، بل لأن أول كتاب وضعه أمامه كان مكتوبًا لمن يعرف البرمجة أصلًا.

لاحظ الفرق. كتاب يقول لك: «المتغيّر هو مساحة مُسمّاة في الذاكرة تُستخدم لتخزين قيمة». جملة صحيحة تمامًا، وميتة تمامًا بالنسبة لشخص لم يكتب سطرًا في حياته. لن يفهم منها شيئًا، وسيظن أن المشكلة فيه.

هذا الكتاب مكتوب بالاتجاه المعاكس. يبدأ من السؤال «لماذا نحتاج هذا الشيء أصلًا؟»، ثم يبني الفهم بالتشبيه والتجربة، ثم يصل إلى التعريف الرسمي — بعد أن يكون قد صار بديهيًا.

لمن كُتب هذا الكتاب؟

ثلاثة أنواع من القرّاء سيجدون فيه ما يبحثون عنه:

- **المبتدئ التام** الذي لم يكتب سطر كود في حياته، ولا يعرف الفرق بين المترجم والمفسّر، ولا يهمّه أن يعرف الآن. هذا الكتاب يفترض أنك تبدأ من الصفر المطلق، ولا يفترض أي خلفية رياضية أو هندسية.
- **القادم من لغة أخرى** الذي يعرف كيف يفكّر برمجيًا، لكنه يريد أن يتعلّم *أسلوب* بايثون الحديث — لا مجرد ترجمة أفكاره القديمة إلى صيغة جديدة.
- **الممارس الذي يريد ترتيب معرفته** بعد سنوات من التعلّم المتقطّع من هنا وهناك، ويريد مرجعًا واحدًا متماسكًا يعود إليه.

ما الذي يميّز هذا الكتاب؟

أولًا: الترتيب مقصود. الفصول ليست كومة مواضيع، بل سلسلة اعتماديات. كل فصل يستخدم ما قبله فقط، ولا يفترض ما بعده. لن تجد نفسك في منتصف مثال يستخدم مفهومًا لم تصل إليه بعد.

ثانيًا: كل مثال يعمل فعلًا. لا توجد أمثلة «توضيحية» ناقصة يصعب تشغيلها. الأكواد في هذا الكتاب مكتوبة لتُنسخ وتُشغَّل، ومجمّعة لك في مجلد مرافق منظّم حسب الفصول، مع سكربت يشغّلها كلها ويتحقق منها.

ثالثًا: الأخطاء جزء من الدرس. كل فصل يحتوي على قسم صريح للأخطاء الشائعة، لا لتتجنّبها فقط، بل لتفهم *لماذا* تحدث. من يفهم سبب الخطأ لا يكرّره؛ ومن يحفظ الحل فقط سيكرّره بصيغة أخرى.

رابعًا: بايثون الحديثة، لا بايثون الأمس. الكتاب مبنيّ على Python 3.14+ ويستخدم الأدوات التي يعتمدها المهندسون اليوم فعلًا: `uv` بدل `pip` التقليدي، `ruff` و`black` للتنسيق، `pytest` للاختبار، Type Hints كأسلوب كتابة

افتراضي، و`match-case` بدل سلاسل `if` الطويلة. لن تتعلم هنا أسلوبًا عفا عليه الزمن ثم تضطر إلى إعادة تعلّمه.

خامسًا: الطريق إلى ما بعد الأساسيات. الجزء الخامس من الكتاب يأخذك في جولة عملية داخل خمسة مسارات مهنية — الويب، تحليل البيانات، الأتمتة، الذكاء الاصطناعي، وأدوات سطر الأوامر — كل مسار بمشروع حقيقي مكتمل. لن تخرج خبيرًا في أي منها، لكنك ستخرج وأنت تعرف *طعم* كل مسار، وقرارك بشأن تخصّصك سيكون مبنيًا على تجربة لا على مقال.

كيف تستفيد منه إلى أقصى حد؟

قاعدة واحدة تلخّص كل شيء: **لا تقرأ الكتاب، نفّذه.**

البرمجة مهارة حركية بقدر ما هي معرفة. قراءة فصل عن الحلقات تشبه قراءة فصل عن السباحة؛ ستفهمه، وستنساه، وستغرق. اكتب الكود بيدك — لا تنسخه — واكسرْه عن قصد، واقرأ رسالة الخطأ، وأصلحه. هذه الدورة القصيرة (اكتب ← شغّل ← افهم الخطأ ← أصلح) هي ما يبني المهارة فعليًا.

ولهذا السبب، كل فصل ينتهي بثلاثة أشياء لا تجاوزها: **تمارين متدرجة**، ثم **اختبار فهم سريع**، ثم **ملخص مرجعي**. إن لم تحلّ التمارين، فأنت لم تنهِ الفصل بعد — أنت فقط قرأته.

ملاحظة على المصطلحات

ستجد في الكتاب مزيجًا مقصودًا بين العربية والإنجليزية: نشرح المفهوم بالعربية، ثم نُثبّت المصطلح الإنجليزي بين قوسين. هذا ليس ترفًا. في عملك الحقيقي — التوثيق، رسائل الأخطاء، الاجتماعات، مقابلات العمل — ستقابل المصطلح الإنجليزي وحده. تعلّمه من الآن، ولا تنتظر حتى تحتاج إليه.

شكر وتقدير

هذا الكتاب يقف على أكتاف مجتمع بايثون كله: من كتبوا التوثيق الرسمي مجانًا، ومن أجابوا عن ملايين الأسئلة في المنتديات، ومن بنوا المكتبات التي تجعل هذه اللغة قادرة على إدارة أنظمة يعتمد عليها مليارات البشر يوميًا. إن كنت ستتعلّم من عملهم، فردّ الجميل لاحقًا: أجب عن سؤال مبتدئ، أو اكتب توثيقًا واضحًا لكودك.

الآن، اقلب الصفحة. رحلتك تبدأ من الفصل الأول — وقبل أن تكتب أي كود.

كيف تقرأ هذا الكتاب

قبل أن تبدأ، خُذ دقيقتين لتفهم كيف بُني هذا الكتاب. القراءة الصحيحة توفّر عليك أسابيع.

تشريح الفصل الواحد

كل فصل — بلا استثناء — مبنيّ على التسلسل نفسه. عندما تعتاد عليه، ستعرف تلقائيًا أين أنت وماذا ينتظرك:

1. **مقدمة تحفيزية** — لماذا يستحق هذا الموضوع وقتك أصلًا.
2. **الشرح النظري** — الفكرة من أولها، بالتشبيه قبل المصطلح.
3. **أمثلة متدرجة** — من سطر واحد إلى برنامج كامل، خطوة بخطوة.
4. 💡 **نصائح الخبراء** — ما يعرفه من مارس العمل فعليًا ولم يُكتب في الكتب.
5. ⚠️ **تحذيرات وأخطاء شائعة** — الفخاخ التي يسقط فيها الجميع، ولماذا هي فخاخ.
6. 🔄 **مقارنات** — بين الطرق المختلفة، ومتى تختار كل واحدة.
7. **تمارين متدرجة** — ثلاثة مستويات: 🟢 مبتدئ، 🟡 متوسط، 🔴 تحدٍّ.
8. **مشروع تطبيقي** — يجمع كل ما تعلّمته في الفصل في شيء واحد يعمل.
9. 📌 **ملخص مرجعي** — جدول سريع تعود إليه لاحقًا بدل إعادة قراءة الفصل.

دليل الرموز

خلال الكتاب ستقابل هذه الصناديق الملوّنة. كل واحد له وظيفة محددة، لا مجرد زينة:

> 💡 **نصيحة خبرة** — معلومة من الممارسة العملية، توفّر عليك وقتًا أو ألمًا.
>
> ⚠️ **تحذير** — خطأ يقع فيه المبتدئون بكثرة. اقرأه ببطء، فغالبًا ستقع فيه أنت أيضًا.
>
> ✅ **أفضل ممارسة** — الطريقة التي يكتب بها المحترفون هذا النوع من الكود.
>
> 🔄 **مقارنة** — جدول أو شرح يقارن بديلين، ليساعدك على الاختيار لا الحفظ.
>
> 🎯 **هدف أو تمرين** — مهمة تنفيذية. لا تتجاوزها.
>
> 📌 **ملاحظة** — تفصيل إضافي مفيد، لكنه ليس جوهريًا للمتابعة.

كذلك ستجد صناديق قابلة للفتح مثل هذا:

🔍 مثال: اضغط للكشف عن التفاصيل

هذه صناديق «اكشف الحل» — تحتوي على حلول التمارين وتلميحاتها وتفاصيل إضافية. في النسخة التفاعلية (HTML) تُفتح بالنقر. في نسخة PDF تظهر مفتوحة بالكامل حتى لا يفوتك شيء أثناء الطباعة.

أكواد الكتاب

كل مثال برمجي في الكتاب موجود في مجلد `code` المرافق، منظّمًا حسب الفصول:

```
code/
├── chapters/
│   ├── ch01/ … ch26/      # أمثلة كل فصل بالترتيب
│   └── appendix-a/ …      # أمثلة الملاحق
├── run_all.py             # يشغّل كل الأمثلة ويتحقق منها
├── MANIFEST.json          # فهرس: كل ملف وقسمه وموضوعه
└── requirements.txt       # المكتبات الخارجية
```

اسم كل ملف يتبع النمط `رقم-الفصل` + `رقم-المثال` + `اسم_الدالة`، وفي أعلى كل ملف تجد عنوان القسم الذي جاء منه. لتشغيل مثال واحد:

Terminal

```
python code/chapters/ch03/03-004-variable_name.py
```

ولتشغيل كل الأمثلة والتحقق من سلامتها دفعة واحدة:

Terminal

```
python code/run_all.py
```

⚠ **ملاحظة مهمة:** بعض الأمثلة في الكتاب مكتوبة *بقصد* لتُظهر خطأً — مثل مثال يوضّح ماذا يحدث عند جمع نص برقم. هذه الأمثلة ستفشل عند التشغيل، وهذا هو المطلوب منها. سكربت `run_all.py` يصنّف كل مثال تلقائيًا: ناجح، أو مقتطف يعتمد على مثال سابق، أو تفاعلي يطلب إدخالًا، أو خطأ مقصود — فلا تختلط عليك الأمور.

ثلاثة مسارات قراءة

لست مضطرًا لقراءة الكتاب بالترتيب من الغلاف إلى الغلاف. اختر المسار الذي يناسب وضعك:

المسار الأول — المبتدئ التام (الافتراضي).
اقرأ كل شيء بالترتيب، دون استثناء، وحلّ كل تمرين. المدة الواقعية: 8–12 أسبوعًا بمعدل ساعة يوميًا. لا تتخطَّ الفصول 1–10 مهما بدت بسيطة؛ هي الأساس الذي سيُحاسبك عليه كل ما بعدها.

المسار الثاني — القادم من لغة أخرى.
ابدأ بالفصل الثالث مباشرة، واقرأ بسرعة الفصول 3–7 (ستجدها مألوفة)، ثم توقّف عند الفصل الثامن والتاسع (هياكل البيانات في بايثون لها تفاصيل تختلف عن غيرها)، واقرأ الفصول 10–18 بتركيز كامل. هذا هو جوهر ما يميّز بايثون عن غيرها.

المسار الثالث — المرجع السريع.
إن كنت تعرف بايثون وتريد مرجعًا: ابدأ من جدول المحتويات، واقرأ ما تحتاجه فقط، ثم استخدم الملحق ج (الجدول المرجعي الشامل) والملحق أ (قاموس المصطلحات) كمرجع دائم.

نصائح عملية قبل البدء

- **جهّز بيئتك أولًا.** لا تقرأ الفصل الثالث قبل أن تُثبّت بايثون وتشغّل أول ملف. الفصل الثاني هو بوابتك.
- **اكتب بيدك، لا تنسخ.** النسخ يخلق وهم الفهم. الكتابة تُجبرك على مواجهة التفاصيل.
- **لا تُطارد الكمال.** لن تفهم 100% من أي فصل في المرة الأولى، وهذا طبيعي ومقصود. الفهم البرمجي حلزوني: تعود للمفهوم لاحقًا بفهم أعمق.
- **خصص وقتًا للتمارين أكبر من وقت القراءة.** النسبة الصحية: 40% قراءة، 60% تطبيق.
- **عند التعطّل، لا تستسلم سريعًا ولا تُطل.** القاعدة العملية: 20 دقيقة محاولة جادة، ثم ابحث عن تلميح، ثم اسأل.

🎯 **التزام صغير، أثر كبير:** اكتب في ورقة «سأنهي الفصل الأول بحلول [تاريخ]» وعلّقها أمامك. الالتزام المكتوب يضاعف احتمال الإكمال — هذه ليست نصيحة تحفيزية، بل نتيجة تتكرر في كل دراسة عن التعلّم.

خريطة الكتاب

الكتاب مقسّم إلى ستة أجزاء متتابعة وثلاثة ملاحق مرجعية. كل جزء يبني على ما قبله، وكل جزء ينتهي بقدرة جديدة تستطيع استخدامها فورًا.

01

ما قبل الكود

قبل أن تلمس لوحة المفاتيح، تحتاج إلى عقلية جديدة وورشة عمل جاهزة. هذان الفصلان يبنيان الأساس الذهني والتقني الذي يستند إليه كل ما بعدهما.

الفصول 1–2

02

الأساسيات المتينة

المتغيّرات، النصوص، الأرقام، القرار، التكرار، هياكل البيانات، والدوال. ثمانية فصول تُتقن فيها مفردات اللغة التي ستكتب بها كل برامجك.

الفصول 3–10

03

البرمجة الوظيفية والكائنية

الانتقال من «كود يعمل» إلى «كود مُهندَس»: معالجة الأخطاء، الدوال المتقدمة، والبرمجة الكائنية بأساسها وتعمّقها.

الفصول 11–14

04

بايثون الحديثة

المهارات التي تفصل المبرمج العادي عن المحترف: تلميحات الأنواع، الملفات وتنسيقات البيانات، البرمجة غير المتزامنة، وإدارة المشاريع بأدوات العصر.

الفصول 15–18

05

جسور التخصص

جولة عملية في خمسة مسارات مهنية: الويب، تحليل البيانات، الأتمتة، الذكاء الاصطناعي، وأدوات سطر الأوامر — كل مسار بمشروع حقيقي مكتمل.

الفصول 19–23

06

الاحتراف والنشر

كيف تكتب كودًا يقرأه الآخرون بسرور، كيف تثق فيه باختبارات آلية، وكيف تُحوّله من ملف على جهازك إلى منتج حقيقي.

الفصول 24–26

07

مراجع دائمة

ثلاثة ملاحق للرجوع إليها مرارًا: قاموس المصطلحات، حلول جميع التمارين، وجدول مرجعي سريع يلخّص الكتاب كله.

الـفـصـول 1–3

ما ستبنيه بنهاية الكتاب

الكتاب لا يعلّمك مفردات اللغة فقط، بل يجعلك تبني أشياء حقيقية. هذه قائمة بما ستنجزه فعليًا — كل واحد منها مشروع كامل يعمل:

- **بطاقة تعريف تفاعلية** تأخذ بيانات المستخدم وتحسبها وتطبعها بتنسيق احترافي.
- **مولّد كلمات مرور** يبني كلمات قوية بقواعد قابلة للتخصيص.
- **حاسبة علمية** من الطرفية تتعامل مع العمليات والأولويات بأمان.
- **لعبة تخمين الأرقام** بقواعد تحقق وحلقات تحكم كاملة.
- **قائمة مهام (To-Do)** تُدار من الطرفية بكل عمليات الإضافة والحذف والبحث.
- **نظام جرد مخزون** يتعامل مع القواميس والمجموعات بكفاءة.
- **مكتبة دوال رياضية** خاصة بك، موثّقة ومُختبرة.
- **نظام إدارة حسابات بنكية** مبني على البرمجة الكائنية.
- **نظام حجز مكتبة كامل** يجمع الوراثة وتعدد الأشكال والـ Dataclasses.
- **محلّل ملفات بيانات** يقرأ JSON وCSV ويستخرج منه رؤى.
- **واجهة API كاملة لإدارة المهام** بـ FastAPI مع تحقق واختبار.
- **تحليل بيانات مبيعات حقيقية** بالرسوم البيانية.
- **بوت كشط بيانات** يجمع المعلومات من الويب ويحفظها منظّمة.
- **نموذج تصنيف أول** بـ scikit-learn.
- **أداة سطر أوامر احترافية** لتنظيم الملفات تستخدمها يوميًا.
- **حزمة بايثون قابلة للنشر** مع اختباراتها وتوثيقها.

ما لا يغطيه هذا الكتاب

الصدق جزء من الاحتراف. هذا الكتاب لا يحاول أن يكون موسوعة، ولن يجعلك خبيرًا في:

- **تطوير الويب المتقدّم** (قواعد البيانات الموزّعة، المصادقة، النشر السحابي على نطاق واسع).
- **التعلّم العميق** (PyTorch، الشبكات العصبية، ضبط النماذج الكبيرة) — الجزء الخاص بالذكاء الاصطناعي هو نافذة أولى فقط.
- **هندسة البيانات الضخمة** (Spark، تدفّقات المعالجة، مستودعات البيانات).
- **تطوير تطبيقات الموبايل** — بايثون ليست الخيار الأول هنا، والكتاب يقول ذلك بصراحة.

هذه ليست ثغرات في الكتاب، بل حدود مقصودة. كل واحد منها يستحق كتابًا كاملًا، والملحق الأخير في نهاية رحلتك سيدلّك على الطريق الصحيح لكل مسار.

عن هذه النسخة

المؤلف	م/ أحمد صالح المغز
العنوان	بايثون 3.14 — من الصفر إلى الاحتراف
الإصدار	الإصدار الأول · 2026
لغة البرمجة	Python 3.14 أو أحدث
المحتوى	26 فصلًا في 6 أجزاء، و3 ملاحق، و430 قسمًا
الأمثلة	794 مثالًا برمجيًا، مجمّعة في مجلد `code` المرافق
الصيغ	PDF (للطباعة والقراءة)، HTML (تفاعلي)، Markdown (قابل للتعديل والنشر)
الخطوط	Cairo · IBM Plex Sans Arabic · JetBrains Mono

هذا الكتاب مرجع تعليمي. الأمثلة والأكواد مرخّصة للاستخدام والتعديل في مشاريعك الشخصية والتجارية دون إذن مسبق. يُمنع إعادة نشر النص نفسه تجاريًا دون اتفاق.

صُمّم وهُندس هذا الكتاب آليًا من مصدر Markdown واحد، لتبقى الصيغ الثلاث متزامنة دائمًا.

جدول المحتويات

26 فصلًا · 3 ملاحق · 6 أجزاء

الجزء الأول — ما قبل الكود

1 كيف يفكّر المبرمج؟ 15

2 تجهيز ورشة العمل 27

الجزء الثاني — الأساسيات المتينة

3 المتغيرات - أول لبنة 46

4 التعامل مع النصوص (Strings) 67

5 الأرقام والعمليات الحسابية 87

6 اتخاذ القرار (Control Flow) 107

7 التكرار (Loops) 130

8 هياكل البيانات - القوائم (Lists) 151

9 القواميس والمجموعات (Dict & Set) 176

10 الدوال - تنظيم الكود 201

الجزء الثالث — البرمجة الوظيفية والكائنية

11 معالجة الأخطاء (Error Handling) 231

12 الدوال المتقدمة (Lambda, Decorators, Generators, Closures) 258

13 البرمجة الكائنية - الأساس (Class, Object, __init__, self) 290

14 البرمجة الكائنية 320

الجزء الرابع — بايثون الحديثة

15 Type Hints — نحو برمجة أكثر وضوحًا ودقة (باستخدام mypy) 352

16 الملفات وتنسيقات البيانات 392

17 البرمجة غير المتزامنة - مقدمة عملية (async/await) 429

18 إدارة المشاريع كمحترف 458

الجزء الخامس — جسور التخصص

19 مسار تطوير الويب - بناء واجهة API لإدارة المهام (FastAPI) 493
20 مسار تحليل البيانات (pandas, numpy, matplotlib) 518
21 مسار الأتمتة — استخراج البيانات من الويب (Web Scraping) 556
22 مسار الذكاء الاصطناعي — نافذة أولى (scikit-learn) 586
23 مسار بناء أدوات سطر الأوامر (CLI) 610

الجزء السادس — الاحتراف والنشر

24 كتابة كود نظيف (Clean Code) 639
25 الاختبار (Testing) باستخدام pytest 667
26 من الكود إلى المنتج (Packaging, Deployment، وخارطة الطريق) 698

الملاحق — مراجع دائمة

1 قاموس المصطلحات البرمجية (عربي-إنجليزي) 730
2 حلول جميع التمارين 745
3 جدول مرجعي سريع (Cheat Sheet) شامل 780

الجزء الأول · فصلان

01

ما قبل الكود

قبل أن تلمس لوحة المفاتيح، تحتاج إلى عقلية جديدة وورشة عمل جاهزة. هذان الفصلان يبنيان الأساس الذهني والتقني الذي يستند إليه كل ما بعدهما.

1 · كيف يفكّر المبرمج؟

2 · تجهيز ورشة العمل

فصل 01 / 26 · الجزء الأول

01

كيف يفكّر المبرمج؟

9 دقيقة قراءة | 11 قسمًا | 1 مثالًا | 1,847 كلمة

في هذا الفصل

1.1 مقدمة: لماذا هذا الفصل قبل أي سطر كود؟
1.2 ما هي البرمجة فعليًا؟
1.3 الخوارزمية: الوصفة قبل الكود
1.4 كيف «يفهم» الكمبيوتر الكود الذي تكتبه؟
1.5 لماذا Python تحديدًا؟ (مقارنة صادقة)
1.6 عقلية حل المشكلات: المهارة الحقيقية
1.7 أخطاء ذهنية شائعة عند المبتدئين (تجنّبها من اليوم الأول)
1.8 بناء «عضلة» حل المشكلات - تمارين عملية
1.9 مشروع تطبيقي صغير للفصل: «مصمم الخوارزميات»
1.10 اختبر فهمك (Quiz سريع)
1.11 ملخص الفصل (Cheat Sheet)

> *«البرمجة ليست عن تعلّم لغة، بل عن تعلّم طريقة تفكير جديدة تمامًا.»*

1.1 📖 مقدمة: لماذا هذا الفصل قبل أي سطر كود؟

أغلب الكتب التعليمية ترتكب خطأ جوهريًا: تقذف بالمبتدئ مباشرة في بحر الأقواس والـ syntax قبل أن يفهم **ماذا يحدث فعليًا** عندما «يبرمج». النتيجة؟ يحفظ القارئ الأوامر دون أن يفهمها، فينهار عند أول مشكلة لم يرها من قبل.

هذا الفصل مختلف. لن تكتب فيه سطر كود واحد بلغة Python، لكنك ستخرج منه بشيء أهم: **عقلية المبرمج**. وهذه العقلية هي ما سيبقى معك حتى لو تعلمت غدًا لغة برمجة أخرى غير Python بالكامل.

1.2 ما هي البرمجة فعليًا؟

لنبدأ بتعريف بسيط وصادق:

> ***البرمجة = إعطاء تعليمات دقيقة جدًا لجهاز «غبي» حرفيًا، لكنه سريع جدًا في التنفيذ.***

الكلمة المفتاحية هنا هي **«دقيقة»**. الكمبيوتر لا يفهم السياق، ولا يخمّن قصدك، ولا يقول «أعتقد أنك تقصد كذا». هو ينفّذ **بالضبط** ما كتبته، لا أكثر ولا أقل.

🍳 تشبيه وصفة الطبخ

تخيل أنك تكتب وصفة لشخص لم يدخل مطبخًا في حياته، ولا يملك أي حس سليم، لكنه ينفذ التعليمات بدقة روبوتية:

وصفة سيئة (غامضة):

```
اصنع بيضة مقلية
```

الشخص الحرفي لا يعرف: هل يكسر البيضة أولاً؟ هل يشغّل الموقد قبل أم بعد وضع المقلاة؟ كم من الوقت يقليها؟

وصفة جيدة (دقيقة كالكود):

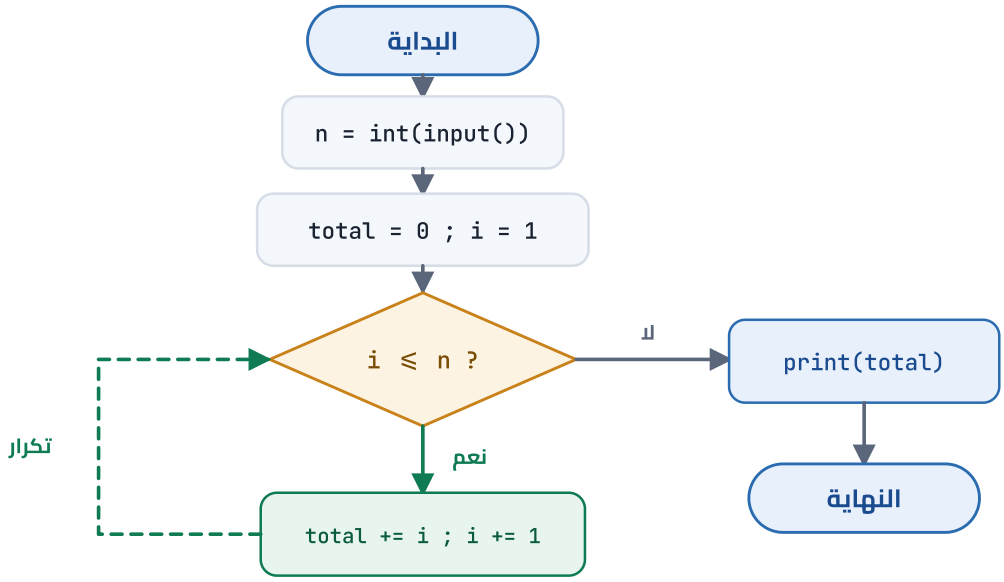
```
1. أحضر مقلاة من الخزانة
2. ضع المقلاة على الموقد
3. شغّل الموقد على حرارة متوسطة
4. أحضر بيضة واحدة من الثلاجة
5. اكسر البيضة في المقلاة
6. انتظر 3 دقائق
7. أطفئ الموقد
8. انقل البيضة إلى طبق
```

هذا بالضبط ما تفعله عندما تكتب كودًا. أنت لا «تطلب» من الكمبيوتر شيئًا، أنت **تصف له كل خطوة** بترتيب منطقي صارم.

💡 نصيحة الخبراء

> *أفضل المبرمجين ليسوا من يحفظون أكبر عدد من الأوامر، بل من يستطيعون تفكيك مشكلة كبيرة ومعقدة إلى سلسلة من خطوات صغيرة وبسيطة جدًا. هذه المهارة تسمى **Decomposition** (التفكيك)، وهي أهم مهارة ستطورها في هذا الكتاب.*

1.3 الخوارزمية: الوصفة قبل الكود



تُنفَّذ الخطوات بالترتيب، ويتكرّر الجسم طالما بقي الشرط صحيحًا.

شكل 1.1 — خوارزمية مجموع الأعداد من 1 إلى n

الكلمة التقنية لـ«الوصفة» هذه هي **Algorithm (خوارزمية)**.

> ***الخوارزمية*** = *سلسلة محددة ومنتهية من الخطوات لحل مشكلة معينة.*

لاحظ ثلاث خصائص مهمة في هذا التعريف:

الخاصية	المعنى	مثال مخالف (خطأ)
محددة	كل خطوة واضحة بلا غموض	«أضف القليل من الملح» ❌
منتهية	لها بداية ونهاية واضحتان	حلقة لا تتوقف أبدًا ❌
قابلة للتنفيذ	يمكن فعلاً القيام بها	«فكّر بعمق في الحل» ❌

🎯 تمرين ذهني (بدون كمبيوتر بعد)

قبل أن نكتب أي كود، دعنا نتدرب على **التفكير الخوارزمي**. اكتب خوارزمية (بالعربية، كنقاط مرقّمة) لحل هذه المشكلة:

> ***المشكلة:*** *لديك 3 أرقام، وتريد معرفة أي رقم منها هو الأكبر.*

حل مقترح (اضغط للكشف)

```
1. سمِّ الأرقام: رقم_أ، رقم_ب، رقم_ج
2. افترض أن رقم_أ هو الأكبر مبدئيًا، وسمِّه "الأكبر_الحالي"
3. قارن رقم_ب بـ "الأكبر_الحالي":
   - إذا كان رقم_ب أكبر، اجعل "الأكبر_الحالي" = رقم_ب
4. قارن رقم_ج بـ "الأكبر_الحالي":
   - إذا كان رقم_ج أكبر، اجعل "الأكبر_الحالي" = رقم_ج
5. أعلن أن "الأكبر_الحالي" هو الإجابة النهائية
```

لاحظ: هذا بالضبط ما سنكتبه بلغة Python لاحقًا، لكن الآن بصيغة بشرية. هذه الصيغة الوسطى تُسمى **Pseudocode** (كود زائف) وسنستخدمها كثيرًا في هذا الكتاب قبل كل مشكلة جديدة.

1.4 كيف «يفهم» الكمبيوتر الكود الذي تكتبه؟

لا يُترجَم الكود إلى لغة الآلة مباشرة، بل إلى شفرة بايتية تُنفّذ على آلة افتراضية.

شكل 1.2 — رحلة الكود من المصدر إلى التنفيذ

هذا سؤال يتجنبه كثير من الكتب، لكن فهمه يزيل غموضًا كبيرًا من ذهن المبتدئ.

الكمبيوتر في الحقيقة **لا يفهم** Python إطلاقًا. المعالج (CPU) يفهم لغة واحدة فقط: **اللغة الثنائية** (0 و 1)، تسمى **Machine Code**.

السؤال إذن: كيف يتحول الكود الذي نكتبه بكلمات شبه إنجليزية (`print` , `if` , `for`) إلى لغة الأصفار والآحاد؟

🔄 مترجم (Compiler) مقابل مفسّر (Interpreter)

هناك طريقتان رئيسيتان لحل هذه المشكلة في عالم البرمجة:

الطريقة الأولى - الترجمة الكاملة مسبقًا (Compiled Languages)
مثل لغة C++. يأخذ برنامج يسمى **Compiler** كل الكود دفعة واحدة، ويترجمه بالكامل إلى ملف تنفيذي بلغة الآلة، **قبل** أي تشغيل فعلي. تشبّه هذه العملية بترجمة كتاب كامل إلى لغة أخرى قبل طباعته.

الطريقة الثانية - الترجمة الفورية أثناء التشغيل (Interpreted Languages)
وهذه طريقة Python. برنامج يسمى **Interpreter** يقرأ الكود **سطرًا بسطر**، ويترجم وينفّذ كل سطر فورًا قبل الانتقال للسطر التالي. يشبه هذا مترجمًا فوريًا يترجم كلام متحدث مباشرة أثناء إلقائه الخطاب.

🔄 جدول مقارنة: لماذا هذا يهمك عمليًا؟

الجانب	لغات مُفسَّرة (Python)	لغات مُترجَمة (C++, Rust)
السرعة في التطوير	✅ سريعة جدًا - اكتب وشغّل فورًا	❌ أبطأ - يجب الترجمة أولاً
سرعة التنفيذ	❌ أبطأ نسبيًا	✅ أسرع بكثير
اكتشاف الأخطاء	عند الوصول للسطر الخاطئ فقط	✅ قبل التشغيل بالكامل
الأفضل لـ	التعلم، السكربتات، النماذج الأولية	الأنظمة الحساسة للأداء

> 💡 **لماذا هذا مهم لك كمبتدئ؟**
> لأن Python ستسامحك! إن كان لديك خطأ في السطر 50، فستُنفَّذ الأسطر من 1 إلى 49 بنجاح تام قبل أن تتوقف عند الخطأ. هذا رائع للتعلم التدريجي، لكنه يعني أيضًا أن اختبار الكود بشكل كامل قبل الاعتماد عليه أمر ضروري (سنتعلم هذا في فصل الاختبارات لاحقًا).

⚙️ ما يحدث فعليًا خلف الكواليس في Python

بدقة أكبر قليلاً (دون إغراق تقني):

```
الكود المصدري (ملف .py)
        ↓
   [يقرأه Interpreter المُفسِّر]
        ↓
يُحوَّل إلى Bytecode (صيغة وسيطة، ليست صفر وواحد مباشرة)
        ↓
   [تنفّذه Python Virtual Machine (PVM)]
        ↓
   النتيجة الفعلية على شاشتك
```

> ⚠️ **تحذير من الخلط الشائع:**
> كثير من المبتدئين يظنون أن Python «بطيئة لأنها سيئة». الحقيقة أدق: Python تضحي ببعض السرعة التنفيذية مقابل سهولة هائلة في الكتابة والقراءة وتقليل الأخطاء. هذه مقايضة (trade-off) مقصودة وذكية، وليست عيبًا. ولهذا بالضبط تجد Python تدير أنظمة عملاقة مثل Instagram وNetflix رغم «بطئها» النظري.

1.5 لماذا Python تحديدًا؟ (مقارنة صادقة)

لن أعطيك دعاية مجانية لـ Python. دعنا نكون موضوعيين تمامًا.

✅ نقاط قوة Python الحقيقية

1. **قابلية القراءة (Readability)**: صُممت لتُقرأ كالإنجليزية شبه الطبيعية
2. **منحنى تعلّم لطيف**: من أسهل اللغات للبدء بها فعليًا
3. **مجتمع ومكتبات هائلة**: تقريبًا أي مشكلة لها حل جاهز (مكتبة)
4. **شمولية استثنائية**: نفس اللغة تُستخدم في الويب، تحليل البيانات، الذكاء الاصطناعي، الأتمتة، الألعاب
5. **طلب سوق عمل ضخم** ومستمر في النمو

❌ حدود Python الحقيقية (بصدق تام)

1. **أبطأ تنفيذيًا** من لغات مثل C++, Rust, Go في المهام الحسابية الثقيلة جدًا
2. **ليست الخيار الأمثل** لتطبيقات الموبايل الأصلية (native mobile apps)
3. **استهلاك ذاكرة أعلى** نسبيًا مقارنة باللغات المُترجمة

🔄 مقارنة سريعة مع لغات أخرى شهيرة

اللغة	متى تُفضَّل عن Python
JavaScript	تطوير واجهات المواقع (Frontend) - Python لا تعمل في المتصفح
C++/Rust	أنظمة تشغيل، محركات ألعاب، أداء حرج قصوى
Java/Kotlin	تطبيقات أندرويد الأصلية
Swift	تطبيقات iOS الأصلية

🎯 **الخلاصة الصادقة:** Python ليست «الأفضل مطلقًا» (لا توجد لغة كذلك)، لكنها **الأفضل كنقطة بداية** لأنها تعلّمك التفكير البرمجي الصحيح بأقل قدر من الحواجز التقنية، ثم تفتح لك أبوابًا شبه لا نهائية بعد إتقانها.

1.6 عقلية حل المشكلات: المهارة الحقيقية

إليك سرًا يغفل عنه كثيرون: **90% من البرمجة الاحترافية ليست كتابة كود، بل التفكير قبل الكتابة.**

🧩 منهجية حل أي مشكلة برمجية (ستستخدمها طوال الكتاب)

عندما تواجه أي مشكلة، مهما بدت بسيطة أو معقدة، اتبع هذه الخطوات دائمًا:

```
1. افهم المشكلة تمامًا
   → ما هي المُدخلات (Inputs)؟
   → ما هي المُخرجات المطلوبة (Outputs)؟

2. فكّر بأمثلة يدوية بسيطة
   → جرّب حل المشكلة بيدك على ورقة بمثال صغير

3. اكتب الخوارزمية (Pseudocode)
   → بخطوات واضحة بلغتك الخاصة، بدون قلق من الـ syntax

4. حوّل الخوارزمية إلى كود حقيقي
   → هذه أسهل خطوة فعليًا إن أتقنت ما قبلها!

5. اختبر الكود بحالات متعددة
   → حالة عادية، حالة حدّية (edge case)، حالة خاطئة
```

🎯 مثال تطبيقي كامل على المنهجية

لنطبق هذه الخطوات على مشكلة واقعية:

> ***المشكلة:*** *تريد برنامجًا يخبرك إن كانت سنة معينة «سنة كبيسة» (366 يومًا) أم لا.*

الخطوة 1: فهم المشكلة

- المُدخل: رقم سنة (مثل 2024)
- المُخرج: نعم أو لا

الخطوة 2: جرّب بأمثلة يدوية

- هل تعرف قاعدة السنة الكبيسة؟ (قابلة للقسمة على 4، لكن ليس على 100 إلا إذا كانت قابلة للقسمة على 400)
- 2024 ÷ 4 = 506 بدون باقي → كبيسة ✅
- 1900 ÷ 4 = بدون باقي، لكن ÷ 100 أيضًا بدون باقي، و÷400 بها باقي → **ليست** كبيسة ❌
- 2000 ÷ 400 = بدون باقي → كبيسة ✅

الخطوة 3: الخوارزمية (Pseudocode)

```
1. خذ السنة كمُدخل
2. إذا كانت السنة قابلة للقسمة على 400 → كبيسة
3. وإلا، إذا كانت قابلة للقسمة على 100 → ليست كبيسة
4. وإلا، إذا كانت قابلة للقسمة على 4 → كبيسة
5. وإلا → ليست كبيسة
```

الخطوة 4: الكود الحقيقي (نظرة تشويقية فقط - سنتعلم هذا بالتفصيل لاحقًا)

Python

```python
def is_leap_year(year: int) -> bool:
    """تحديد ما إذا كانت السنة كبيسة أم لا."""
    if year % 400 == 0:
        return True
    elif year % 100 == 0:
        return False
    elif year % 4 == 0:
        return True
    else:
        return False

# اختبار الدالة
print(is_leap_year(2024))  # True
print(is_leap_year(1900))  # False
print(is_leap_year(2000))  # True
```

الخطوة 5: اختبار بحالات متعددة

- حالة عادية: 2023 → ليست كبيسة ✅
- حالة حدّية: 2000 (قرن لكن قابل للقسمة على 400) ✅
- حالة حدّية أخرى: 1900 (قرن غير قابل للقسمة على 400) ✅

> **لاحظ شيئًا مهمًا جدًا**: لم تفهم كل سطر في الكود أعلاه، وهذا طبيعي تمامًا! الهدف من هذا المثال ليس تعليمك `def` أو `%` الآن، بل إثبات أن **المنهجية** (الخطوات 1-3) هي الجزء الصعب والمهم فعليًا، وأن كتابة الكود النهائي (الخطوة 4) تصبح سهلة نسبيًا بمجرد أن تفكر بوضوح.

1.7 أخطاء ذهنية شائعة عند المبتدئين (تجنّبها من اليوم الأول)

⚠ الخطأ 1: «يجب أن أحفظ كل شيء عن ظهر قلب»

الحقيقة: حتى المبرمجون المحترفون ذوو خبرة 20 عامًا يبحثون في جوجل يوميًا. المهارة الحقيقية هي **فهم المفاهيم**، لا حفظ الأوامر. الحفظ يأتي تلقائيًا بالممارسة المتكررة.

⚠ الخطأ 2: «إذا ظهر لي خطأ (Error)، فأنا فاشل»

الحقيقة: الأخطاء (Errors) هي **أفضل أصدقائك** في رحلة التعلم. كل رسالة خطأ هي معلم يشرح لك بالضبط ماذا حدث. سنتعلم في الفصل القادم كيف «نقرأ» رسائل الأخطاء بدل الخوف منها.

⚠ الخطأ 3: «يجب أن أفهم كل التفاصيل قبل الانتقال للموضوع التالي»

الحقيقة: التعلم البرمجي **حلزوني** وليس خطيًا. ستتعلم مفهومًا بسيطًا، تستخدمه بشكل سطحي، ثم تعود له لاحقًا بفهم أعمق بعد أن ترى تطبيقات أكثر. لا تعلّق نفسك في فصل واحد لأسابيع.

⚠ الخطأ 4: «النسخ من مصادر أخرى حرام أو غش»

الحقيقة: كل المبرمجين المحترفين يبحثون ويرجعون لمصادر (التوثيق الرسمي، Stack Overflow). الفرق بين المبتدئ والمحترف ليس «عدم البحث»، بل **فهم ما تنسخه** قبل استخدامه.

1.8 بناء «عضلة» حل المشكلات - تمارين عملية

الآن حان دورك. هذه التمارين **لا تحتاج كتابة كود** - فقط ورقة وقلم، أو حتى تفكير ذهني. الهدف تدريب عقلك على التفكير الخوارزمي قبل الانتقال للكود الفعلي في الفصول القادمة.

🟢 تمرين 1 (مستوى مبتدئ)

اكتب خوارزمية (خطوات مرقّمة بالعربية) لتحضير كوب شاي، بافتراض أن من ينفّذها لا يعرف شيئًا إطلاقًا عن المطبخ.

🟡 تمرين 2 (مستوى متوسط)

لديك قائمة من 5 أسماء طلاب ودرجاتهم في امتحان. اكتب خوارزمية لإيجاد اسم الطالب صاحب أعلى درجة.

> 💡 تلميح: فكّر كيف حللنا مثال «أكبر رقم بين 3 أرقام» سابقًا، ثم وسّع نفس المنطق.

🔴 تمرين 3 (تحدي)

اكتب خوارزمية للتحقق مما إذا كانت كلمة معينة (مثل «مدام» أو "level") هي **Palindrome** (تُقرأ نفسها من اليمين واليسار).

<details>
<summary>🔍 تلميح للتمرين 3 (اضغط إذا احتجت مساعدة)</summary>

فكّر في مقارنة الحرف الأول بالحرف الأخير، ثم الثاني بما قبل الأخير، وهكذا... متى تتوقف هذه المقارنة؟ وماذا لو وجدت عدم تطابق في منتصف العملية؟

</details>

1.9 مشروع تطبيقي صغير للفصل: «مصمم الخوارزميات»

بما أننا لم نكتب كودًا بعد، مشروع هذا الفصل مختلف عن باقي الفصول: هو **تمرين تفكير منهجي كامل**.

📋 المهمة

اختر **موقفًا واقعيًا** من حياتك اليومية (مثال: «كيف أختار ماذا ألبس حسب حالة الطقس»، أو «كيف أرتب أولويات مهامي اليومية»)، ثم:

1. حدد **المُدخلات** بدقة (ما المعلومات التي تحتاجها لاتخاذ القرار؟)
2. حدد **المُخرج** المطلوب (ما القرار النهائي؟)
3. اكتب **خوارزمية كاملة** بخطوات مرقّمة وواضحة تمامًا
4. اختبر خوارزميتك ذهنيًا بـ 3 حالات مختلفة (تأكد أنها تعمل منطقيًا في كل حالة)

> 🎯 **لماذا هذا المشروع مهم؟** لأنه يحاكي **بالضبط** ما ستفعله في كل مشروع برمجي طوال حياتك المهنية: تحويل مشكلة واقعية غامضة إلى خطوات منطقية دقيقة، **قبل** لمس لوحة المفاتيح لكتابة كود فعلي. المبرمجون الذين يتخطون هذه الخطوة هم من يقضون ساعات في «تصحيح» كود مبني على تفكير غير واضح من الأساس.

1.10 اختبر فهمك (Quiz سريع)

أجب عن هذه الأسئلة ذهنيًا أو كتابيًا للتأكد من استيعابك قبل الانتقال للفصل التالي:

1. ما الفرق الجوهري بين لغة مُفسَّرة ولغة مُترجَمة؟

2. لماذا تُعتبر «دقة التعليمات» أهم خاصية في أي خوارزمية؟
3. اذكر خاصيتين من خصائص Python تجعلها مناسبة للمبتدئين، وخاصية واحدة تُعتبر نقطة ضعف حقيقية لها.
4. رتّب خطوات منهجية حل المشكلات البرمجية الخمس بالترتيب الصحيح (دون الرجوع للنص).
5. لماذا يُعتبر ظهور رسالة خطأ (Error) أمرًا إيجابيًا للمتعلم، لا سلبيًا؟

<details>
<summary>✅ الإجابات النموذجية</summary>

1. اللغة المُفسَّرة تُترجَم وتُنفَّذ سطرًا بسطر أثناء التشغيل (مثل Python)، بينما المُترجَمة تُترجَم بالكامل إلى لغة الآلة *قبل* أي تشغيل (مثل C++).
2. لأن الكمبيوتر ينفّذ حرفيًا ما يُكتب له دون تخمين النية أو السياق؛ أي غموض يؤدي لنتائج خاطئة أو توقف البرنامج.
3. نقاط القوة: قابلية القراءة العالية، منحنى تعلّم لطيف، مجتمع ومكتبات ضخمة. نقطة الضعف: أبطأ تنفيذيًا من اللغات المُترجَمة في المهام الحسابية الثقيلة.
4. فهم المشكلة ← التجربة بأمثلة يدوية ← كتابة الخوارزمية (Pseudocode) ← كتابة الكود الفعلي ← الاختبار بحالات متعددة.
5. لأن رسالة الخطأ تخبرك بدقة أين ولماذا فشل الكود، مما يسرّع عملية التعلم ويحوّل الفشل إلى معلومة بنّاءة بدل أن يكون نهاية مسدودة.

</details>

1.11 📌 ملخص الفصل (Cheat Sheet)

المفهوم	التعريف المختصر
Algorithm (خوارزمية)	سلسلة محددة ومنتهية من الخطوات لحل مشكلة
Pseudocode	وصف خطوات الحل بلغة بشرية بسيطة قبل كتابة الكود الفعلي
Interpreter (مُفسِّر)	برنامج يترجم وينفّذ الكود سطرًا بسطر أثناء التشغيل (طريقة Python)
Compiler (مُترجِم)	برنامج يترجم كل الكود دفعة واحدة قبل أي تشغيل
Decomposition (تفكيك)	تقسيم مشكلة كبيرة معقدة إلى خطوات صغيرة وبسيطة
Edge Case (حالة حدّية)	حالة استثنائية أو نادرة يجب اختبار الكود ضدها

🎯 ماذا بعد؟

في **الفصل القادم**، سننتقل أخيرًا للجانب العملي: تثبيت Python 3.14 فعليًا على جهازك، تجهيز بيئة تطوير احترافية بـ VS Code، وكتابة وتشغيل أول سطر كود حقيقي - مع تعلّم كيفية **قراءة رسائل الخطأ** بثقة بدل الخوف منها منذ اللحظة الأولى.

فصل 02 / 26 · الجزء الأول

02

تجهيز ورشة العمل

11 دقيقة قراءة | 14 قسمًا | 7 مثالًا | 2,289 كلمة

في هذا الفصل

2.1 مقدمة: لماذا هذا الفصل حاسم؟
2.2 فهم المكوّنات الأساسية قبل التثبيت
2.3 تثبيت Python 3.14
2.4 أداة uv: إدارة حديثة وفائقة السرعة
2.5 تجهيز محرر الأكواد: Visual Studio Code
2.6 أول تواصل مع Python: الـ REPL
2.7 كتابة وتشغيل أول ملف كود حقيقي
2.8 فن قراءة رسائل الخطأ (مهارة تُغيّر رحلتك بالكامل)
2.9 لمحة عن البيئات الافتراضية (Virtual Environments)
2.10 أخطاء التهيئة الشائعة وحلولها السريعة
2.11 تمارين عملية للفصل
2.12 مشروع تطبيقي صغير: «بطاقة التعريف الذكية»
2.13 اختبر فهمك (Quiz سريع)
2.14 ملخص الفصل (Cheat Sheet)

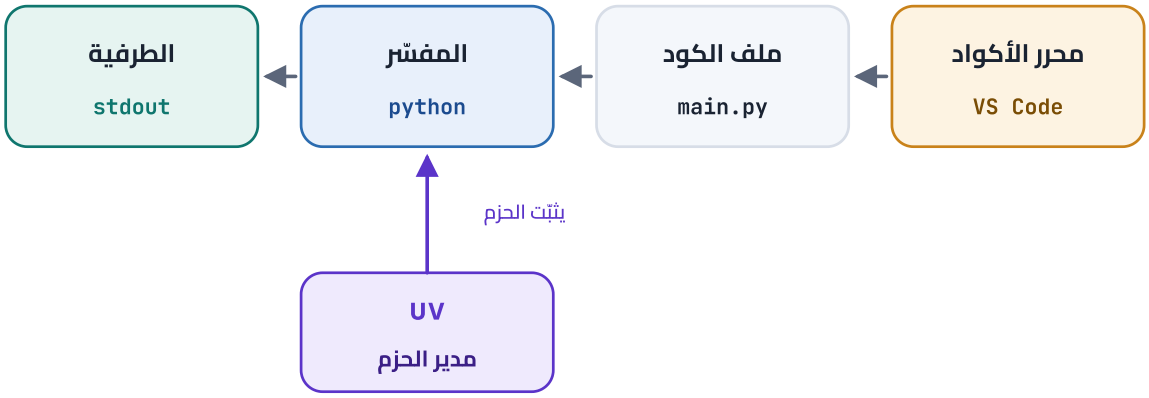
> *«النجار الماهر يعتني بأدواته قبل أن يصنع أي قطعة خشب. المبرمج الماهر يعتني ببيئة عمله قبل أن يكتب أي سطر كود.»*

2.1 📖 مقدمة: لماذا هذا الفصل حاسم؟

كثير من المبتدئين يواجهون **أول إحباط حقيقي** في رحلتهم البرمجية ليس بسبب صعوبة الكود، بل بسبب مشاكل التثبيت والإعداد. رسالة خطأ غامضة عند التثبيت، أو برنامج تحرير نصوص غير مناسب، كافية لإقناع الكثيرين أن «البرمجة ليست لهم» - وهذا خطأ فادح وغير صحيح إطلاقًا.

في هذا الفصل، سنبني بيئة عمل **احترافية وحديثة** خطوة بخطوة، بنفس الأدوات التي يستخدمها المبرمجون المحترفون في الشركات العالمية الكبرى اليوم. لن نستخدم طرقًا قديمة «كانت تُستخدم قبل 10 سنوات» - بل أحدث الممارسات المعتمدة في 2025.

2.2 فهم المكوّنات الأساسية قبل التثبيت



لكل أداة دور واحد: المحرر يكتب، والمفسّر ينفّذ، والطرفية تعرض، و uv يجهّز الحزم.

شكل 2.1 — أدوات ورشة العمل: من المحرر إلى الطرفية

قبل أن تضغط أي زر تثبيت، من الضروري أن تفهم **ماذا** تُثبّت ولماذا. هذا الفهم سيوفر عليك ساعات من الحيرة لاحقًا.

🧩 المكوّنات الثلاثة التي سنحتاجها

المكوّن	الوظيفة	مثال تشبيهي
Python Interpreter	«المحرك» الذي يفهم وينفّذ كودك	محرك السيارة
محرر أكواد (VS Code)	البيئة التي تكتب فيها الكود براحة	المكتب ولوحة القيادة
مدير الحزم (uv)	أداة لتثبيت وإدارة مكتبات إضافية	ميكانيكي يحضر لك قطع غيار

💡 **نصيحة مهمة:** لا تخلط بين «لغة Python» و"Python ". VS Code هي اللغة نفسها، بينما VS Code مجرد برنامج تحرير نصوص ذكي يساعدك على كتابتها بسهولة أكبر. يمكنك تقنيًا كتابة بايثون بأي محرر نصوص بسيط (حتى Notepad!)، لكن هذا يشبه قيادة سيارة بدون مقود مساعد - ممكن، لكن غير عملي إطلاقًا.

2.3 تثبيت Python 3.14

⚠️ تحذير قبل البدء: مشكلة الإصدارات المتعددة

أحد أكثر الأخطاء شيوعًا بين المبتدئين هو انتهاء المطاف بعدة إصدارات من Python مُثبّتة على نفس الجهاز دون علمهم، مما يسبب ارتباكًا لاحقًا («لماذا الكود يعمل هنا ولا يعمل هناك؟»). سنتجنب هذه المشكلة من البداية باتباع خطوات دقيقة.

🪟 التثبيت على Windows

الخطوة 1: التحقق المسبق

قبل التثبيت، تحقق إن كان لديك بالفعل إصدار قديم:

1. اضغط `Win + R`
2. اكتب `cmd` واضغط Enter
3. في النافذة السوداء التي تظهر، اكتب:

```
Terminal
```

```
python --version
```

- إذا ظهرت رسالة مثل `Python 3.9.0` أو أي رقم أقل من 3.12، ستحتاج للتحديث
- إذا ظهرت رسالة خطأ (`is not recognized`)، فهذا يعني Python غير مثبّت، وهذا جيد - ستبدأ نظيفًا

الخطوة 2: تحميل Python 3.14

1. اذهب إلى الموقع الرسمي: `python.org/downloads`
2. حمّل أحدث إصدار من سلسلة 3.14.x (تجنب أي موقع آخر غير الرسمي)
3. شغّل ملف التثبيت الذي حمّلته

الخطوة 3: خطوات التثبيت (الأهم!)

عند فتح برنامج التثبيت، ستظهر لك نافذة بها خيارات. هذه الخطوة حاسمة:

```
☑ Add python.exe to PATH    ← هذا الخيار يجب تفعيله!
☐ Install launcher for all users
```

> ⚠️ **تحذير حرج جدًا:** خيار "Add python.exe to PATH" **يجب** أن يكون مُفعّلاً (✅). هذا الخيار يسمح لجهازك بالتعرّف على أمر `python` من أي مكان في الطرفية. تجاهل هذه الخطوة هو **السبب رقم واحد** لمشاكل «Python غير معروف» التي يواجهها المبتدئون.

بعد تفعيل الخيار، اضغط **"Install Now"** وانتظر اكتمال التثبيت.

الخطوة 4: التحقق من نجاح التثبيت

أغلق نافذة cmd القديمة وافتح واحدة **جديدة** (مهم: نافذة جديدة لتحديث الإعدادات)، ثم اكتب:

```
Terminal
python --version
```

يجب أن تظهر:

```
Python 3.14.0
```

🍎 التثبيت على macOS

الخطوة 1: فحص الإصدار المُدمج

macOS يأتي أحيانًا بإصدار قديم من Python مُدمج في النظام. **لا تستخدمه أبدًا** للتطوير - هو مخصص لعمليات النظام الداخلية فقط.

افتح تطبيق **Terminal** (ابحث عنه في Spotlight بـ `Cmd + Space`)، واكتب:

```
Terminal
python3 --version
```

الخطوة 2: التثبيت باستخدام Homebrew (الطريقة الموصى بها عالميًا)

إذا لم يكن لديك Homebrew (مدير الحزم الأشهر لـ macOS)، ثبّته أولاً:

```
Terminal
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

بعدها، ثبّت Python:

```
Terminal
brew install python@3.14
```

الخطوة 3: التحقق

```
Terminal
python3.14 --version
```

> **نصيحة**: على macOS، يُستخدم الأمر `python3` بدلاً من `python` غالبًا، لأن `python` وحدها قد تشير للإصدار القديم المُدمج بالنظام. سنتعامل مع هذا التفصيل بوضوح أكثر في قسم بيئات العمل الافتراضية لاحقًا.

🐧 التثبيت على Linux (Ubuntu/Debian كمثال)

معظم توزيعات Linux تأتي بـ Python مُثبّتًا مسبقًا، لكن غالبًا إصدار قديم.

```
Terminal
# تحديث قائمة الحزم
sudo apt update

# تثبيت الأدوات اللازمة للبناء
sudo apt install software-properties-common -y

# إضافة مستودع يحتوي أحدث إصدارات Python
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update

# تثبيت Python 3.14
sudo apt install python3.14 -y
```

التحقق:

```
Terminal
python3.14 --version
```

2.4 أداة uv: إدارة حديثة وفائقة السرعة

🔄 لماذا uv بدلاً من pip التقليدي؟

لسنوات، كان `pip` هو الأداة الوحيدة لتثبيت مكتبات Python الإضافية. لكن في عامي 2023-2024، ظهرت أداة ثورية اسمها **uv** (من شركة Astral، صانعة أداة Ruff الشهيرة) غيّرت قواعد اللعبة بالكامل.

الخاصية	pip (التقليدية)	uv (الحديثة)
السرعة	بطيئة نسبيًا	⚡ أسرع بـ 10-100 مرة
إدارة البيئات الافتراضية	تحتاج أداة منفصلة (venv)	✅ مدمجة بالكامل
إدارة إصدارات Python	غير مدعومة	✅ مدعومة مباشرة
حل التعارضات بين الحزم	بدائي	✅ ذكي ومتقدم
الاعتماد العالمي 2025	يتراجع تدريجيًا	📈 يتزايد بسرعة

> 💡 **قرار تعليمي مهم**: سنعتمد في هذا الكتاب بالكامل على `uv` بدلاً من `pip` التقليدية، لأنها أصبحت المعيار الحديث الموصى به من قبل خبراء المجتمع، وستوفر عليك أخطاء ومشاكل كثيرة كان يعاني منها المبتدئون سابقًا مع الأدوات القديمة.

📥 تثبيت uv

على Windows (PowerShell):

```PowerShell
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

على macOS / Linux:

```Terminal
curl -LsSf https://astral.sh/uv/install.sh | sh
```

التحقق من التثبيت (على أي نظام):

أغلق الطرفية وافتح واحدة جديدة، ثم:

```
Terminal
```

```
uv --version
```

يجب أن تظهر نتيجة مثل:

```
uv 0.5.x
```

✅ **ميزة إضافية رائعة:** بإمكان `uv` تثبيت إصدارات Python نفسها! مثلاً، يمكنك كتابة `uv python install 3.14` وستُثبَّت تلقائيًا دون الحاجة للذهاب لموقع Python الرسمي يدويًا. سنستخدم هذه الميزة بكثرة لاحقًا.

2.5 تجهيز محرر الأكواد: Visual Studio Code

🤔 لماذا VS Code تحديدًا؟

يوجد عشرات المحررات والبيئات (PyCharm, Sublime Text, Vim...)، لكن VS Code يبقى الخيار الأمثل للمبتدئين والمحترفين على حد سواء للأسباب التالية:

- ✅ **مجاني بالكامل** ومفتوح المصدر
- ✅ **خفيف** نسبيًا على موارد الجهاز
- ✅ **دعم استثنائي لـ Python** عبر الإضافات الرسمية
- ✅ **الأكثر استخدامًا عالميًا** (حسب استطلاعات Stack Overflow السنوية)
- ✅ يعمل بنفس الطريقة على Windows/Mac/Linux

📥 خطوات التثبيت

1. اذهب إلى `code.visualstudio.com`
2. حمّل النسخة المناسبة لنظامك
3. ثبّته باتباع التعليمات الافتراضية (بدون تعقيدات)

🧩 الإضافات الأساسية (لا تُثبّت أكثر من هذا في البداية!)

أحد الأخطاء الشائعة بين المبتدئين هو تثبيت عشرات الإضافات ظنًا أنها ستجعلهم «محترفين أسرع». النتيجة غالبًا جهاز بطيء وإرباك غير ضروري. إليك فقط **ما تحتاجه فعليًا**:

افتح VS Code، اذهب لأيقونة الإضافات (Extensions) في الشريط الجانبي (أو `Ctrl+Shift+X`)، وابحث عن:

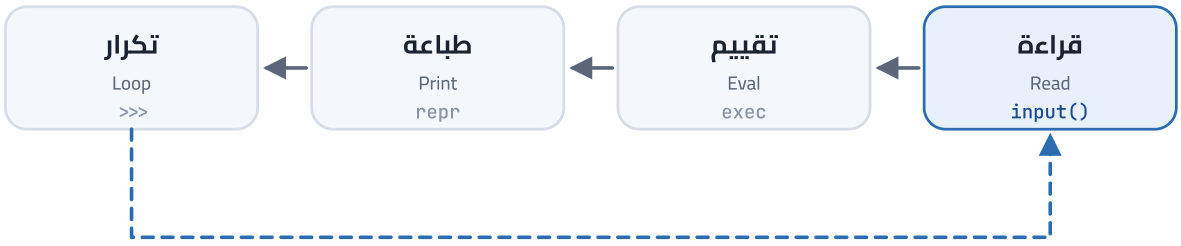
الإضافة	الوظيفة	لماذا ضرورية؟
Python (من Microsoft)	الدعم الأساسي الكامل للغة	التلوين، الإكمال التلقائي، تشغيل الكود
Pylance (من Microsoft)	تحليل ذكي للكود	يكتشف الأخطاء قبل التشغيل
Ruff (من Astral)	فحص وتنسيق تلقائي للكود	يُعلّمك الأسلوب الصحيح من البداية
Error Lens	إبراز الأخطاء مباشرة في السطر	رؤية فورية للمشاكل

⚠ **تحذير:** عند تثبيت إضافة "Python"، ستُثبَّت "Pylance" تلقائيًا معها غالبًا كجزء من الحزمة - لا تقلق إن وجدتهما مرتبطتين.

إعداد سريع بعد التثبيت

افتح VS Code، واضغط `Ctrl + ,` (أو `Cmd + ,` على Mac) لفتح الإعدادات، وابحث عن `Format On Save`، وفعّله (✅). هذا سيجعل VS Code ينسّق كودك تلقائيًا بأسلوب احترافي معياري كل مرة تحفظ فيها الملف - عادة ستشكر نفسك عليها لاحقًا.

2.6 أول تواصل مع Python: الـ REPL



بيئة تفاعلية: سطر واحد يُنفَّذ فورًا، فترى النتيجة قبل أن تكتب التالي.

شكل 2.2 — دورة REPL: قراءة ثم تقييم ثم طباعة

ما هو REPL؟

قبل أن نكتب ملفات كود حقيقية، دعنا نتعرّف على طريقة تفاعلية رائعة لتجربة Python فورًا: الـ **REPL** (اختصار لـ Read-Eval-Print-Loop).

فكّر في REPL كـ«محادثة مباشرة» مع Python: تكتب سطرًا، يُنفَّذ فورًا، وترى النتيجة مباشرة.

🎮 جرّب بنفسك

افتح الطرفية (Terminal في VS Code، أو cmd/Terminal في نظامك)، واكتب:

```
Terminal
python
```

(أو `python3` على Mac/Linux إن لزم الأمر)

ستظهر لك واجهة مثل هذه:

```
Python 3.14.0 (main, ...)
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

الرمز `>>>` يعني أن Python **جاهزة وتنتظر** أوامرك. جرّب كتابة:

```
Python
>>> 2 + 2
4
```

```
Python
>>> "مرحبا" + " " + "بالعالم"
'مرحبا بالعالم'
```

```
Python
>>> print("أنا أتعلم Python!")
أنا أتعلم Python!
```

> **نصيحة عملية:** استخدم REPL دائمًا لتجربة سطر كود سريع أو اختبار فكرة صغيرة، بدل إنشاء ملف كامل لكل تجربة. أغلب المبرمجين المحترفين يستخدمونه يوميًا لاختبارات سريعة.

للخروج من REPL في أي وقت:

```
Python
>>> exit()
```

2.7 كتابة وتشغيل أول ملف كود حقيقي

الـ REPL رائع للتجارب السريعة، لكن المشاريع الحقيقية تُكتب في **ملفات** تنتهي بامتداد `.py`.

📂 الخطوة 1: تنظيم مجلد العمل

قبل أي شيء، أنشئ مجلدًا مخصصًا لتمارين الكتاب:

```
Terminal

# أنشئ مجلدًا جديدًا
mkdir python-journey
cd python-journey
```

📝 الخطوة 2: فتح المجلد في VS Code

في الطرفية، داخل المجلد الذي أنشأته:

```
Terminal

code .
```

(النقطة `.` تعني «افتح هذا المجلد الحالي في VS Code»)

✍️ الخطوة 3: إنشاء أول ملف

في VS Code، أنشئ ملفًا جديدًا باسم `hello.py`. لكن بدلاً من مجرد `Hello World` التقليدي المُعاد استخدامه آلاف المرات، سنكتب شيئًا تفاعليًا بسيطًا يعطيك إحساسًا حقيقيًا بقوة البرمجة من اليوم الأول:

```python
Python  hello.py

# أول برنامج تفاعلي حقيقي

name = input("ما اسمك؟ ")
age = input("كم عمرك؟ ")

print(f"مرحبًا {name}! 👋")
print(f"عمرك {age} سنة.")

age_in_days = int(age) * 365
print(f"هذا يعني أنك عشت تقريبًا {age_in_days} يومًا على هذه الأرض!")
```

▶️ الخطوة 4: تشغيل الكود

في VS Code، يوجد عدة طرق لتشغيل الملف:

الطريقة 1 (الأسهل): اضغط الزر المثلث ▶️ في الزاوية العلوية اليمنى

الطريقة 2 (الاحترافية - تعلّمها من الآن): استخدم الطرفية المدمجة في VS Code (`Ctrl+``) واكتب:

```
Terminal
```

```
python hello.py
```

ستظهر النتيجة في الطرفية أسفل الشاشة، وسيطلب منك البرنامج إدخال اسمك وعمرك تفاعليًا.

✅ **لماذا الطريقة الثانية أفضل للتعلّم؟** لأنها تعكس بالضبط كيف يعمل المبرمجون المحترفون - عبر سطر الأوامر مباشرة - وستحتاج هذه المهارة بشدة لاحقًا عند العمل بمشاريع حقيقية خارج VS Code.

2.8 فن قراءة رسائل الخطأ (مهارة تُغيّر رحلتك بالكامل)

شكل 2.3 — تشريح رسالة الخطأ (Traceback)

🎯 لماذا هذا القسم هو الأهم في الفصل؟

تذكّر ما قلناه في الفصل الأول: الأخطاء ليست أعداءك، بل معلّموك. لكن لتستفيد منها، يجب أن تتعلّم **كيف تقرأها**. دعنا نرتكب خطأ مقصودًا ونحلّله معًا.

🔬 تجربة: خطأ مقصود

عدّل ملف `hello.py` وأضف هذا السطر الخاطئ عمدًا في النهاية:

```python
print("النتيجة النهائية: " + age_in_days)
```

شغّل الملف وستظهر رسالة خطأ تشبه هذه:

```
Traceback (most recent call last):
  File "hello.py", line 10, in <module>
    print("النتيجة النهائية: " + age_in_days)
                                  ~~~~~~~~~~~~~~~~~~~~~~~~^~~~~~~~~~~~~~
TypeError: can only concatenate str (not "int") to str
```

🧩 تشريح رسالة الخطأ خطوة بخطوة

هذه الرسالة تبدو مرعبة لأول وهلة، لكنها في الحقيقة **خريطة دقيقة** تخبرك بكل شيء تحتاجه:

```
Traceback (most recent call last):
  ↑ هذا فقط يعني "إليك تتبّع مسار الخطأ"

File "hello.py", line 10, in <module>
  ↑ اسم الملف ورقم السطر بالضبط حيث حدثت المشكلة

print("النتيجة النهائية: " + age_in_days)
  ↑ السطر الفعلي الذي سبّب الخطأ (معروض لك مباشرة!)

TypeError: can only concatenate str (not "int") to str
  ↑ شرح دقيق: "لا يمكن دمج نوع الخطأ + شرح دقيق: "لا يمكن دمج str مع int"
```

🔑 القاعدة الذهبية لقراءة أي خطأ في Python

```
1. اقرأ السطر الأخير أولاً (نوع الخطأ + الشرح) - هذا 80% من الحل
2. انظر لرقم السطر المذكور - اذهب إليه مباشرة في كودك
3. اسأل نفسك: "ماذا يحاول هذا السطر فعله، ولماذا يرفضه Python؟"
```

في مثالنا: `age_in_days` هو رقم (int)، بينما النص قبله هو str. Python لا تسمح بدمج (concatenate) نوعين مختلفين بـ `+` مباشرة. الحل:

```python
print("النتيجة النهائية: " + str(age_in_days))
```

> 💡 **لا تقلق إن لم تفهم `str` و `int` بعمق الآن** - سنشرحهما بالتفصيل الكامل في الفصل القادم. الهدف هنا فقط تعلّم **منهجية** قراءة الخطأ، لا حل هذا الخطأ تحديدًا.

📚 أنواع الأخطاء الأكثر شيوعًا للمبتدئين (مرجع سريع)

نوع الخطأ	متى يظهر	مثال شائع
`SyntaxError`	خطأ في قواعد كتابة الكود نفسه	نسيان `:` بعد `if`
`NameError`	استخدام متغير غير موجود	خطأ إملائي في اسم متغير
`TypeError`	خلط أنواع بيانات غير متوافقة	جمع نص مع رقم مباشرة
`IndentationError`	مشكلة في المسافات البادئة	عدم محاذاة الأسطر بشكل صحيح
`ZeroDivisionError`	قسمة أي رقم على صفر	`10 / 0`

> ⚠️ **تحذير من عادة سيئة شائعة جدًا:** كثير من المبتدئين عند رؤية خطأ طويل، يصابون بالذعر وينسخون **الرسالة بأكملها** إلى جوجل أو ChatGPT دون قراءتها أولاً. تدرّب على قراءة السطر الأخير بنفسك أولاً - ستندهش كم مرة ستفهم المشكلة بنفسك خلال ثوانٍ فقط.

2.9 لمحة عن البيئات الافتراضية (Virtual Environments)

🤔 لماذا نحتاج هذا المفهوم من البداية؟

تخيل أنك تعمل على مشروعين مختلفين: الأول يحتاج نسخة قديمة من مكتبة معينة، والثاني يحتاج أحدث نسخة من نفس المكتبة. إذا ثبّت كل شيء «عالميًا» على جهازك، ستحدث تعارضات حتمًا.

البيئة الافتراضية (Virtual Environment) هي حل بسيط وعبقري: «فقاعة» معزولة خاصة بكل مشروع، تحتوي نسخة Python والمكتبات الخاصة به فقط، دون التأثير على بقية مشاريعك أو نظامك.

> 💡 سنتعمّق في هذا المفهوم بالتفصيل الكامل في فصل لاحق مخصص لإدارة المشاريع الاحترافية. الآن، فقط تذكّر: **كل مشروع حقيقي يحتاج بيئته الافتراضية الخاصة** - هذه قاعدة ذهبية لا تُخالف أبدًا في الممارسة الاحترافية.

🎮 تجربة سريعة بـ uv

داخل مجلد مشروعك، جرّب:

```
Terminal
```

```
# إنشاء بيئة افتراضية جديدة
uv venv

# تفعيلها (الأوامر تختلف حسب نظامك)
# على Windows:
.venv\Scripts\activate

# على Mac/Linux:
source .venv/bin/activate
```

بعد التفعيل، ستلاحظ تغيّرًا بسيطًا في الطرفية (عادة اسم البيئة يظهر بين قوسين)، وهذا يعني أنك الآن «داخل» بيئة معزولة. سنستخدم هذا بكثرة بدءًا من مشاريعنا الأولى في الفصول القادمة.

2.10 أخطاء التهيئة الشائعة وحلولها السريعة

⚠ المشكلة 1: `'python' is not recognized as an internal or external command`

السبب: لم يتم تفعيل خيار "Add to PATH" أثناء التثبيت (على Windows).

الحل: أعد تثبيت Python، وتأكد من تفعيل ذلك الخيار هذه المرة، أو أضف المسار يدويًا لمتغيرات البيئة (خطوة متقدمة، البحث عنها بـ"Add Python to PATH manually" يعطيك حلولاً مفصلة).

⚠ المشكلة 2: VS Code لا «يرى» Python المُثبَّت

السبب: VS Code لم يكتشف تلقائيًا أين Python مُثبَّت.

الحل: اضغط `Ctrl+Shift+P` (أو `Cmd+Shift+P`)، اكتب `Python: Select Interpreter`، واختر الإصدار الصحيح من القائمة الظاهرة.

⚠ المشكلة 3: الأحرف العربية تظهر كرموز غريبة

السبب: مشكلة في ترميز الملف (Encoding).

الحل: تأكد في VS Code (أسفل يمين الشاشة) أن الترميز هو `UTF-8`. إذا لم يكن كذلك، اضغط عليه واختر `Reopen with Encoding` ثم `UTF-8`.

2.11 تمارين عملية للفصل

🟢 تمرين 1 (مستوى مبتدئ)

تأكد من نجاح تثبيت uv، Python، وVS Code بتنفيذ الأوامر التالية والتقاط لقطة شاشة (أو كتابة النتيجة) لكل منها:

```
Terminal
```

```
python --version
uv --version
code --version
```

🟡 تمرين 2 (مستوى متوسط)

عدّل ملف `hello.py` ليسأل المستخدم أيضًا عن **مدينته**، ثم يطبع جملة كاملة تجمع كل المعلومات (الاسم، العمر، المدينة) في رسالة ترحيبية واحدة منسّقة بشكل جميل باستخدام f-strings.

🔴 تمرين 3 (تحدي)

تعمّد كتابة 3 أنواع مختلفة من الأخطاء (مثلاً: `SyntaxError` بنسيان قوس، `NameError` باستخدام متغير غير معرّف، و `TypeError` بجمع نص مع رقم)، ثم اكتب تحليلك الخاص لكل رسالة خطأ ظهرت، موضحًا: ما السبب؟ وأين بالضبط في الكود حدثت المشكلة؟

2.12 مشروع تطبيقي صغير: «بطاقة التعريف الذكية»

📋 المهمة

بالاعتماد على كل ما تعلمته في هذا الفصل (كتابة ملف، استخدام `input`، `print`، f-strings)، ابنِ برنامجًا يُسمى `id_card.py` يقوم بـ:

1. يسأل المستخدم عن: الاسم الكامل، سنة الميلاد، المدينة، والهواية المفضلة
2. يحسب العمر التقريبي (بافتراض أن السنة الحالية 2025)
3. يطبع «بطاقة تعريف» منسّقة بشكل جميل، تشمل كل المعلومات، باستخدام رموز تعبيرية (emojis) وحدود نصية بسيطة (مثل `=====`) لتبدو احترافية

💻 مثال على الشكل المتوقع للنتيجة

```
==================================
🪪  بطاقة التعريف الشخصية
==================================
👤 الاسم: أحمد محمد
🎂 العمر التقريبي: 28 سنة
🏙️ المدينة: الرياض
🎯 الهواية: القراءة
==================================
```

> 🎯 لا تقلق إن استغرقت وقتًا في هذا التمرين - هو مصمم عمدًا ليدفعك لتطبيق كل ما تعلمته بحرية، دون إجابة واحدة «صحيحة». جرّب، أخطئ، اقرأ رسائل الأخطاء، وصحّح. هذه هي حلقة التعلم الحقيقية.

2.13 اختبر فهمك (Quiz سريع)

1. ما الفرق الجوهري بين REPL وكتابة كود في ملف `.py` ؟ ومتى تستخدم كل طريقة؟
2. لماذا نحتاج لتفعيل خيار "Add Python to PATH" عند التثبيت على Windows تحديدًا؟
3. اذكر الخطوات الثلاث لقراءة أي رسالة خطأ في Python بمنهجية صحيحة.
4. ما الفرق بين `pip` و `uv` ؟ ولماذا اعتمدنا `uv` في هذا الكتاب؟
5. ما هي البيئة الافتراضية (Virtual Environment) باختصار، ولماذا هي مهمة؟

✅ الإجابات النموذجية

1. REPL يُنفّذ كل سطر فورًا ومناسب للتجارب السريعة، بينما ملفات `.py` تُحفظ وتُشغَّل كاملة دفعة واحدة، وهي الطريقة المناسبة للمشاريع والبرامج الحقيقية.
2. لأن هذا الخيار يسمح لنظام التشغيل بالتعرّف على أمر `python` من أي مكان في الطرفية دون كتابة المسار الكامل يدويًا في كل مرة.
3. أولاً: اقرأ السطر الأخير (نوع الخطأ وشرحه). ثانيًا: حدد رقم السطر المذكور في كودك. ثالثًا: اسأل نفسك ماذا كان هذا السطر يحاول فعله ولماذا رفضته Python.
4. `pip` هي الأداة التقليدية لإدارة الحزم، بينما `uv` أداة حديثة أسرع بكثير، وتدير أيضًا إصدارات Python والبيئات الافتراضية بشكل مدمج. اعتمدناها لأنها الاتجاه الحديث الموصى به عالميًا.
5. هي «فقاعة» معزولة خاصة بكل مشروع، تحتوي نسخة Python ومكتباته الخاصة فقط، لتجنب التعارضات بين متطلبات المشاريع المختلفة على نفس الجهاز.

2.14 📌 ملخص الفصل (Cheat Sheet)

الأمر/المفهوم	الوظيفة
`python --version`	التحقق من إصدار Python المُثبَّت
`uv venv`	إنشاء بيئة افتراضية جديدة
`uv --version`	التحقق من تثبيت uv
`python filename.py`	تشغيل ملف كود بايثون
`code .`	فتح المجلد الحالي في VS Code
REPL	وضع تفاعلي لتجربة أكواد سريعة (`>>>`)
`Traceback`	تتبّع مسار حدوث الخطأ في الكود
Virtual Environment	بيئة معزولة لكل مشروع لتجنب تعارض المكتبات

🎯 ماذا بعد؟

الآن بيئتك جاهزة بالكامل، وتعلّمت كيف «تتحدث» مع Python وتفهم ردودها (حتى عند الأخطاء). في **الفصل الثالث**، سنبدأ رحلتنا الفعلية في عالم اللغة: **المتغيرات** - اللبنة الأساسية الأولى لأي برنامج، مع فهم عميق لما يحدث فعليًا في ذاكرة الكمبيوتر عند كل سطر كود تكتبه.

الجزء الثاني · 8 فصول

02

الأساسيات المتينة

المتغيّرات، النصوص، الأرقام، القرار، التكرار، هياكل البيانات، والدوال. ثمانية فصول تُتقن فيها مفردات اللغة التي ستكتب بها كل برامجك.

3 · المتغيرات
4 · التعامل مع النصوص
5 · الأرقام والعمليات الحسابية
6 · اتخاذ القرار
7 · التكرار
8 · هياكل البيانات
9 · القواميس والمجموعات
10 · الدوال

فصل 03 / 26 · الجزء الثاني

03

المتغيرات - أول لبنة

12 دقيقة قراءة | 17 قسمًا | 36 مثالًا | 2,308 كلمة

في هذا الفصل

3.1 مقدمة: لماذا المتغيرات هي نقطة البداية الحقيقية؟
3.2 ما هو المتغير؟ (الفهم الصحيح، لا المبسّط الخاطئ)
3.3 إنشاء المتغيرات: عملية الإسناد (Assignment)
3.4 قواعد وأعراف تسمية المتغيرات
3.5 الأنواع الأساسية للبيانات (Data Types)
3.6 Python لغة ديناميكية التنميط (Dynamically Typed)
3.7 Type Hints: الكتابة الواضحة من اليوم الأول
3.8 تحويل الأنواع (Type Casting)
3.9 الإسناد المتعدد وتبديل القيم (تقنيات Python الأنيقة)
3.10 الثوابت (Constants): عُرف لا قاعدة صارمة
3.11 جدول مقارنة شامل: أنواع البيانات الأساسية
3.12 أخطاء شائعة يرتكبها المبتدئون (تجنّبها الآن)
3.13 أفضل الممارسات: ملخص شامل
3.14 تمارين عملية للفصل
3.15 مشروع تطبيقي: «بطاقة التعريف الذكية - النسخة المُطوَّرة»
3.16 اختبر فهمك (Quiz سريع)
3.17 ملخص الفصل (Cheat Sheet)

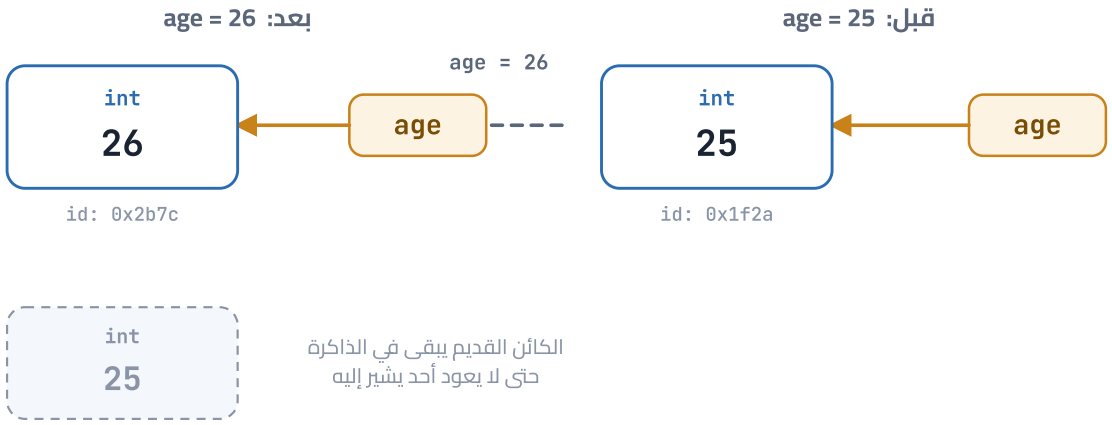
> *«المتغيرات هي الذاكرة التي تمنح برنامجك القدرة على 'التذكّر'. بدونها، كل برنامج سينسى كل شيء بمجرد الانتقال للسطر التالي.»*

3.1 📖 مقدمة: لماذا المتغيرات هي نقطة البداية الحقيقية؟

في الفصل السابق، كتبت برنامجك الأول الذي يسأل عن اسمك ويطبع ترحيبًا. لكن هل تساءلت: كيف «تذكّر» البرنامج اسمك بين سطر `input()` وسطر `print()`؟ الإجابة: **المتغيرات**.

المتغيرات هي أبسط وأهم مفهوم في أي لغة برمجة على الإطلاق. بدونها، لا يمكن للبرنامج تخزين أي معلومة، ولا معالجتها، ولا استخدامها لاحقًا. كل شيء ستتعلمه لاحقًا - من القوائم إلى الدوال إلى البرمجة الكائنية - مبني فوق هذا المفهوم الأساسي.

3.2 ما هو المتغير؟ (الفهم الصحيح، لا المبسّط الخاطئ)



المتغيّر ليس صندوقًا، بل اسم مربوط بكائن — والإسناد الجديد يعيد ربطه بكائن آخر.

شكل 3.1 — المتغيّر ملصق مربوط بكائن، والإسناد يعيد الربط

التشبيه الشائع (ولماذا هو غير دقيق تمامًا في Python)

أغلب الكتب تشرح المتغير كـ«صندوق» تضع بداخله قيمة:

```
يحتوي 25 ← صندوق باسم "age"]
```

هذا التشبيه مفيد للبداية، لكنه **غير دقيق** بخصوص كيفية عمل Python فعليًا، وسيُسبب لك التباسًا لاحقًا عند تعلّم القوائم والكائنات. لذلك، سنتعلّم النموذج الصحيح من البداية.

النموذج الصحيح: المتغيرات كـ«ملصقات» (Labels)

في Python، الصورة الأدق هي:

> ***المتغير ليس صندوقًا يحتوي قيمة، بل «ملصق» (اسم) يُشير إلى مكان في الذاكرة تُخزَّن فيه القيمة.***

تخيّل الأمر هكذا:

```python
age = 25
```

ما يحدث فعليًا في الذاكرة:

```
الذاكرة:
┌──────────────┐
│  كائن ←  │  25 القيمة (Object) مُخزَّن في مكان ما بالذاكرة
└──────────────┘

      ↑
      │ (يُشير إليه)
      │
   الملصق "age"
```

الاسم `age` هو مجرد **ملصق** يُشير إلى حيث يوجد الرقم 25 في الذاكرة. هذا الفرق يبدو بسيطًا الآن، لكنه سيكون حاسمًا لفهم سلوكيات معينة لاحقًا (خصوصًا عند التعامل مع القوائم في الفصل السابع).

🔬 إثبات عملي: دالة `id()`

يمكنك فعليًا «رؤية» أين تُخزَّن القيمة في الذاكرة باستخدام دالة `id()` المُدمجة:

Python

```python
age = 25
print(id(age))  # 140712834567280 :سيطبع رقمًا طويلاً مثل
```

هذا الرقم هو عنوان الذاكرة (مبسّط) حيث يوجد الكائن `25` . جرّب هذا:

Python

```python
age = 25
age_copy = age

print(id(age))       # 140712834567280
print(id(age_copy))  # 140712834567280  ← نفس العنوان بالضبط!
```

> **ملاحظة مهمة:** كلا المتغيرين `age` و `age_copy` يُشيران لنفس الكائن `25` في الذاكرة. لم يتم «نسخ» الرقم فعليًا، بل تم إنشاء ملصق جديد يُشير لنفس المكان. هذا السلوك طبيعي تمامًا للأرقام الصغيرة والقيم البسيطة، وسنتعمّق في تبعاته لاحقًا مع أنواع بيانات أكثر تعقيدًا.

3.3 إنشاء المتغيرات: عملية الإسناد (Assignment)

✍️ الصيغة الأساسية

```python
variable_name = value
```

علامة `=` هنا تُسمى **عامل الإسناد (Assignment Operator)**، وهي **لا تعني** «يساوي» بالمعنى الرياضي إطلاقًا، بل تعني: **«اجعل هذا الاسم يُشير إلى تلك القيمة»**.

```python
name = "سارة"
age = 30
height = 1.65
is_student = False
```

⚠️ تحذير حرج: الخلط بين `=` و `==`

هذا من أكثر الأخطاء شيوعًا بين المبتدئين على الإطلاق، ويستمر معهم أحيانًا لأسابيع قبل أن يستوعبوه تمامًا:

الرمز	الاسم	الوظيفة	مثال
`=`	Assignment (إسناد)	**تخزين** قيمة في متغير	`age = 25`
`==`	Equality (مقارنة)	**مقارنة** إن كانت قيمتان متساويتين	`age == 25`

```python
age = 25          # ✅ صحيح: تخزين القيمة 25 في age
print(age == 25)  # ✅ صحيح: مقارنة - هل age تساوي 25؟ → True

age = 25
print(age = 25)  # ❌ خطأ! SyntaxError - لا يمكن استخدام = داخل print()
```

🎯 **قاعدة ذهبية للتذكّر:** استخدم `=` عندما تريد «**تخزين**» قيمة، واستخدم `==` عندما تريد «**السؤال**» إن كانت قيمتان متساويتين. سنتعمّق في `==` أكثر في فصل التحكم بسير البرنامج (الفصل السادس).

3.4 قواعد وأعراف تسمية المتغيرات

📏 القواعد الإلزامية (يجب اتباعها، وإلا سيحدث خطأ)

Python تفرض قواعد صارمة لأسماء المتغيرات:

```python
# ✅ أسماء صحيحة
name = "أحمد"
age2 = 25
_private_value = 100
userName = "test"

# ❌ أسماء غير صحيحة (ستُسبب SyntaxError)
2age = 25          # لا يمكن البدء برقم
user-name = "x"    # الشرطة - غير مسموحة
class = "test"     # "class" كلمة محجوزة في Python
```

القواعد الإلزامية:

1. يجب أن يبدأ الاسم بحرف أو شرطة سفلية `_` (لا يبدأ برقم أبدًا)
2. يمكن أن يحتوي على حروف، أرقام، وشرطات سفلية فقط (لا مسافات، لا رموز خاصة)
3. حساس لحالة الأحرف (`Name` و `name` متغيران مختلفان تمامًا)
4. لا يمكن استخدام **الكلمات المحجوزة** (Reserved Keywords) مثل `if` , `for` , `class` , `True` , `None` ...

🎨 الأعراف الاحترافية (Convention) - ليست إلزامية، لكن اتبعها دائمًا

هنا يأتي الفرق بين كود «يعمل» وكود «احترافي». المجتمع العالمي لمبرمجي Python يتبع معيارًا موثقًا رسميًا يُسمى **PEP 8**، وأهم قاعدة فيه لتسمية المتغيرات:

> ***استخدم snake_case دائمًا:*** *كل الحروف صغيرة، والكلمات مفصولة بشرطة سفلية.*

```python
# ✅ الأسلوب الاحترافي الموصى به (snake_case)
first_name = "محمد"
total_price = 150.5
is_logged_in = True

# ❌ أساليب تعمل لكنها غير احترافية في Python
firstName = "محمد"      # camelCase - شائع في JavaScript وليس Python
FirstName = "محمد"      # PascalCase - يُستخدم للـ Classes فقط لاحقًا
FIRST_NAME = "محمد"     # كله كبير - يُستخدم للثوابت (Constants) فقط
```

🔄 مقارنة سريعة مع لغات أخرى

اللغة	العُرف الشائع	مثال
Python	snake_case	`user_name`
JavaScript	camelCase	`userName`
Java	camelCase	`userName`

⚠️ **تحذير للمبرمجين القادمين من لغات أخرى:** إذا كنت تعرف JavaScript أو Java مسبقًا، ستميل عفويًا لكتابة `camelCase`. تجنّب هذا في Python - الكود الذي لا يتبع snake_case يُعتبر «كودًا غير أصيل» (not Pythonic) في أي مراجعة كود احترافية.

👄 الأسماء ذات المعنى: فرق بين مبرمج مبتدئ ومحترف

```python
# ❌ أسماء غامضة (تعمل لكنها سيئة)
x = 25
y = "محمد"
z = x * 12

# ✅ أسماء واضحة ومعبّرة (احترافية)
age = 25
name = "محمد"
age_in_months = age * 12
```

💡 **نصيحة الخبراء:** الكود يُقرأ أكثر بكثير مما يُكتب. قد تعود لكودك بعد 6 أشهر ولن تتذكّر ماذا كان يعني `x`. استثمر ثانيتين إضافيتين الآن في اختيار اسم واضح، ستوفر عليك دقائق (أو ساعات) من الحيرة لاحقًا.

3.5 الأنواع الأساسية للبيانات (Data Types)

Python تحتوي أنواعًا كثيرة من البيانات، لكن في هذا الفصل سنتعرّف على **الأنواع الأساسية (Primitive Types)** التي ستستخدمها يوميًا.

🔢 الأعداد الصحيحة (Integer - `int`)

أي رقم صحيح، موجب أو سالب، بدون فاصلة عشرية:

```python
age = 25
temperature = -5
year = 2025

print(type(age))  # <class 'int'>
```

💡 **ميزة فريدة في Python**: على عكس لغات أخرى كثيرة، أعداد `int` في Python ليس لها حد أقصى للحجم (محدودة فقط بذاكرة جهازك). يمكنك فعليًا تخزين أرقام بمئات الخانات دون أي مشاكل:

```python
huge_number = 12345678901234567890123456789O
print(huge_number * 2)  # يعمل بدون أي مشاكل!
```

🔢 الأعداد العشرية (Float)

أي رقم يحتوي فاصلة عشرية:

```python
height = 1.75
price = 99.99
pi_approx = 3.14159

print(type(height))  # <class 'float'>
```

⚠️ **تحذير مهم - فخ الدقة العشرية**: ستُفاجأ لاحقًا (في الفصل الخامس) بأن Python (وكل لغات البرمجة تقريبًا) تعاني من مشكلة دقة في الأعداد العشرية:

```python
print(0.1 + 0.2)  # 0.30000000000000004  ← ليس 0.3 تمامًا!
```

هذا **ليس خطأ** في Python، بل طبيعة كيفية تخزين الكمبيوترات للأعداد العشرية في الذاكرة (نظام ثنائي). سنتعلّم كيف نتعامل مع هذا بذكاء في الفصل الخامس.

📝 النصوص (String - `str`)

أي سلسلة من الحروف، محاطة بعلامات اقتباس:

```python
name = "سارة"
city = 'الرياض'  # علامات مفردة أو مزدوجة - كلاهما صحيح
message = "اليوم هو يوم جميل!"

print(type(name))  # <class 'str'>
```

💡 **نصيحة اتساق:** Python لا تفرّق بين `'single quotes'` و `"double quotes"` وظيفيًا، لكن اختر أسلوبًا واحدًا والتزم به في كل مشروعك للاتساق. المعيار الأكثر شيوعًا عالميًا هو علامات الاقتباس المزدوجة `"..."`.

✅ القيم المنطقية (Boolean - `bool`)

قيمتان فقط: `True` أو `False` (لاحظ حرف كبير إلزامي في البداية):

```python
is_student = True
is_raining = False

print(type(is_student))  # <class 'bool'>
```

⚠️ **خطأ شائع جدًا:** كتابة `true` أو `false` بحروف صغيرة (كما في JavaScript أو Java) ستُسبب `NameError` في Python:

```python
is_active = true  # ❌ NameError: name 'true' is not defined
is_active = True  # ✅ صحيح
```

🚫 القيمة الفارغة (`None`)

تمثّل «لا شيء» أو «غياب القيمة» - مفهوم مهم جدًا سنستخدمه كثيرًا لاحقًا:

```python
middle_name = None  # الشخص ليس لديه اسم أوسط

print(type(middle_name))  # <class 'NoneType'>
```

💡 **متى تُستخدم `None`؟** عندما تريد التعبير عن «لا توجد قيمة بعد» أو «فارغ عمدًا»، على عكس الصفر `0` أو النص الفارغ `""` اللذين هما قيم فعلية موجودة. سنرى استخدامات عملية قوية لهذا في فصول الدوال والبرمجة الكائنية.

Python `3.6` لغة ديناميكية التنميط (Dynamically Typed)

🔄 الفهم الجوهري

في لغات مثل C++ أو Java، يجب عليك **تحديد نوع المتغير صراحة** عند إنشائه، ولا يمكن تغييره لاحقًا:

```java
// في Java (ليست Python!)
int age = 25;
age = "خطأ! لا يمكن تغيير النوع";  // ❌ خمسة وعشرون"
```

لكن في Python، النوع **يُحدَّد تلقائيًا** حسب القيمة، ويمكن أن **يتغيّر** في أي وقت:

```python
age = 25          # الآن age من نوع int
print(type(age))  # <class 'int'>

age = "مسموح تمامًا في Python!"  # ✅ "خمسة وعشرون
print(type(age))      # <class 'str'>
```

🔄 مقارنة: ثابت التنميط مقابل ديناميكي التنميط

الخاصية	Statically Typed (Java, C++)	Dynamically Typed (Python)
تحديد النوع	إلزامي وصريح عند الإنشاء	تلقائي حسب القيمة
تغيير النوع لاحقًا	❌ غير مسموح	✅ مسموح
اكتشاف أخطاء الأنواع	قبل التشغيل (compile time)	أثناء التشغيل فقط
سرعة الكتابة	أبطأ (تفاصيل أكثر)	⚡ أسرع وأكثر مرونة
احتمالية أخطاء مخفية	أقل	⚠️ أعلى نسبيًا

> ⚠️ **تحذير احترافي مهم:** مرونة Python هذه سلاح ذو حدين. في مشاريع صغيرة، هي ميزة رائعة تُسرّع التطوير. لكن في مشاريع كبيرة بفرق متعددة، قد يُسبب هذا أخطاءً مخفية يصعب اكتشافها (مثل تمرير نص لدالة تتوقع رقمًا دون أي تحذير فوري). هذا بالضبط سبب ابتكار **Type Hints** التي سنتعلّمها في القسم القادم - وهي حل Python الأنيق لهذه المشكلة.

Type Hints `3.7` : الكتابة الواضحة من اليوم الأول

🤔 لماذا نتعلّم هذا بهذا التبكير؟

معظم الكتب التعليمية تؤجّل هذا المفهوم لفصول متقدمة جدًا، أو تتجاهله كليًا. لكننا سنكسر هذا التقليد، لأن **Type Hints** أصبحت جزءًا أساسيًا من أسلوب كتابة Python الحديث والاحترافي، ومن الأفضل بناء العادة الصحيحة من البداية بدل تعلّم «الطريقة القديمة» ثم «تصحيحها» لاحقًا.

✍️ الصيغة الأساسية

```python
variable_name: type = value
```

مقارنة بين الأسلوبين:

```python
# بدون Type Hint (يعمل تمامًا، لكن أقل وضوحًا)
age = 25
name = "محمد"

# مع Type Hint (نفس الوظيفة، لكن أوضح للقارئ والأدوات)
age: int = 25
name: str = "محمد"
is_active: bool = True
height: float = 1.75
```

🎯 ما الفائدة الفعلية؟

Type Hints **لا تُغيّر** سلوك الكود إطلاقًا وقت التشغيل (Python ستتجاهلها عمليًا)، لكنها توفر فوائد ضخمة:

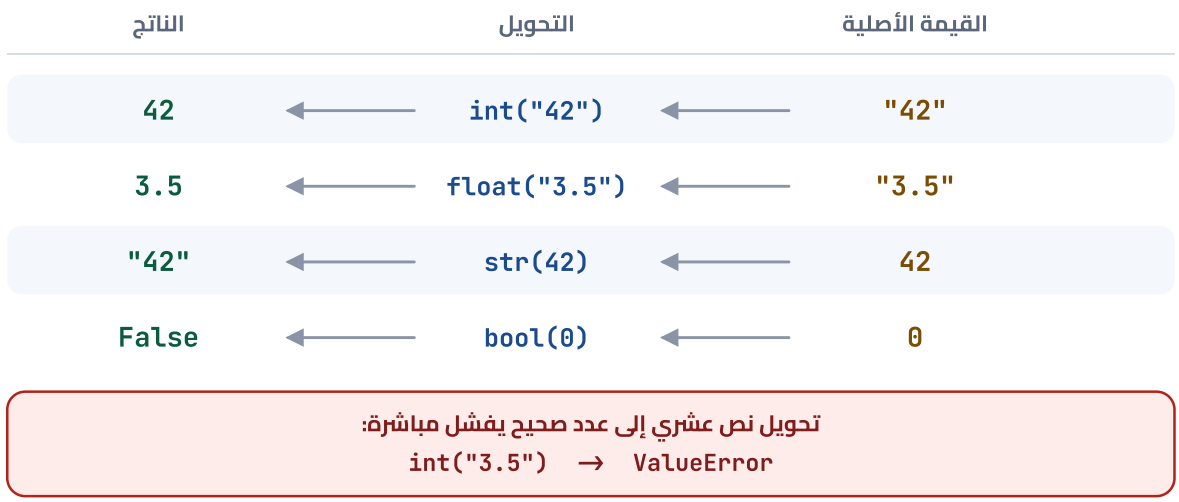
1. **توثيق ذاتي للكود**: أي شخص (حتى أنت بعد 6 أشهر) يفهم نوع البيانات المتوقع فورًا
2. **اكتشاف أخطاء مبكرًا**: محررات مثل VS Code (مع Pylance) تُنبّهك فورًا إن حاولت استخدام نوع خاطئ
3. **إكمال تلقائي أذكى**: المحرر يقترح عليك الدوال والخصائص الصحيحة بناءً على النوع المُعلن

🔬 تجربة عملية في VS Code

```python
age: int = 25
age = "خمسة وعشرون"  # لن يُسبب خطأ وقت التشغيل، لكن VS Code تحته خط تحذير أحمر سيُظهر!
```

> 💡 **نصيحة عملية**: من الآن فصاعدًا في هذا الكتاب، سنستخدم Type Hints في كل مثال تقريبًا - ليس لأنها إلزامية، بل لأنها **أفضل ممارسة عالمية حديثة** ستجعل كودك أوضح واحترافيًا من البداية، وستجنّبك إعادة تعلّم عادات لاحقًا.
>
> ⚠️ **توضيح مهم**: Type Hints **اختيارية بالكامل** في Python (على عكس لغات مثل Java حيث هي إلزامية). يمكنك كتابة برنامج كامل بدونها وسيعمل تمامًا. لكن في عالم العمل الاحترافي اليوم (خصوصًا في الشركات الكبرى)، أصبحت شبه معيار إلزامي لأي كود جاد.

3.8 تحويل الأنواع (Type Casting)



شكل 3.2 — التحويل الصريح بين الأنواع الأساسية

أحيانًا تحتاج لتحويل قيمة من نوع لآخر صراحة. Python توفر دوال مُدمجة لهذا:

```python
# تحويل من str إلى int
age_text: str = "25"
age_number: int = int(age_text)
print(age_number + 5)  # 30

# تحويل من int إلى str
score: int = 95
score_text: str = str(score)
print("درجتك: " + score_text)  # درجتك: 95

# تحويل من str إلى float
price_text: str = "19.99"
price: float = float(price_text)

# تحويل إلى bool
print(bool(0))      # False
print(bool(1))      # True
print(bool(""))     # False (نص فارغ = False)
print(bool("لا"))   # True  (أي نص غير فارغ = True، حتى لو كان "لا"!)
```

⚠️ الفخ الأشهر على الإطلاق للمبتدئين: `input()` تُعيد دائمًا نصًا

تذكر برنامجك في الفصل الثاني؟ هذا الفخ بالضبط وقعت فيه (أو كاد):

```python
age = input("كم عمرك؟ ")  # المستخدم يكتب: 25
print(type(age))            # <class 'str'> ← ليس int!

# محاولة استخدامه كرقم مباشرة:
age_next_year = age + 1  # ❌ TypeError: can only concatenate str (not "int") to str
```

الحل الصحيح:

```python
age: str = input("كم عمرك؟ ")
age_number: int = int(age)  # التحويل الصريح إلى int
age_next_year: int = age_number + 1
print(age_next_year)  # يعمل بنجاح الآن
```

🎯 **قاعدة ذهبية يجب حفظها:** دالة `input()` **تُعيد دائمًا نصًا (str)**، بغض النظر عمّا يكتبه المستخدم - حتى لو كتب رقمًا! يجب عليك دائمًا تحويله صراحة (`int()` أو `float()`) إن أردت استخدامه كرقم في عمليات حسابية.

⚠️ تحذير: محاولة تحويل نص غير رقمي

```python
age = int("خمسة وعشرون")  # ❌ ValueError: invalid literal for int()
```

سنتعلّم في الفصل الحادي عشر (معالجة الأخطاء) كيف «نمسك» هذا النوع من الأخطاء بأناقة قبل أن يُوقف البرنامج بالكامل.

3.9 الإسناد المتعدد وتبديل القيم (تقنيات Python الأنيقة)

🎁 الإسناد المتعدد

Python تسمح بإسناد عدة متغيرات في سطر واحد بأناقة:

```python
# الطريقة التقليدية
x = 5
y = 10
z = 15

# طريقة Python الأنيقة (Multiple Assignment)
x, y, z = 5, 10, 15
```

🎁 إسناد نفس القيمة لعدة متغيرات

```python
a = b = c = 0
print(a, b, c)  # 0 0 0
```

🔄 تبديل قيم متغيرين (دون متغير مؤقت!)

في أغلب اللغات، تبديل قيمتين يحتاج متغيرًا ثالثًا مؤقتًا:

```python
# الطريقة التقليدية (في لغات أخرى، أو حتى في Python)
temp = x
x = y
y = temp
```

لكن Python توفر طريقة أنيقة جدًا (سحرية بعض الشيء للمبتدئين):

```python
x, y = 5, 10
x, y = y, x  # تبديل القيم في سطر واحد!

print(x, y)  # 10 5
```

> 💡 **لماذا هذا مهم؟** هذا مثال ممتاز على فلسفة Python: «**القابلية للقراءة مهمة**» (Readability counts - من PEP 20، فلسفة Python الرسمية). بدلاً من 3 أسطر مع متغير مؤقت مُربك، سطر واحد واضح يُعبّر عن النية مباشرة.

3.10 الثوابت (Constants): عُرف لا قاعدة صارمة

🤔 هل توجد ثوابت حقيقية في Python؟

بصراحة تامة: **لا**. على عكس لغات مثل Java أو C++ التي لديها كلمة `const` تمنع تغيير القيمة فعليًا، Python **لا تملك آلية حقيقية** لمنع تغيير أي متغير.

لكن المجتمع البرمجي اتفق على **عُرف (Convention)**: أي متغير بأحرف كبيرة بالكامل (`UPPER_CASE`) يُعتبر «ثابتًا» **بالاتفاق**، ولا يجب تغييره، رغم أن Python تقنيًا لن تمنعك:

```python
PI: float = 3.14159
MAX_USERS: int = 100
APP_NAME: str = "نظام إدارة المكتبة"

# تقنيًا يمكنك فعل هذا، لكنه يُعتبر خطأ في الممارسة الاحترافية:
PI = 3.14  # ⚠️ ليس خطأ تقنيًا، لكنه مخالفة صارخة للعُرف المتفق عليه
```

✅ **أفضل الممارسات**: استخدم `UPPER_CASE` لأي قيمة تعرف أنها لن (ويجب ألا) تتغيّر طوال تشغيل البرنامج (مثل Pi، الحد الأقصى لشيء ما، اسم التطبيق). هذا يُرسل إشارة واضحة لأي مبرمج آخر (أو لك مستقبلًا) أن هذه القيمة «مقدّسة» ولا تُعدَّل.

3.11 جدول مقارنة شامل: أنواع البيانات الأساسية

النوع	الكلمة المفتاحية	مثال	هل قابل للتعديل (Mutable)؟
عدد صحيح	`int`	`25`	❌ لا (Immutable)
عدد عشري	`float`	`3.14`	❌ لا (Immutable)
نص	`str`	`"مرحبا"`	❌ لا (Immutable)
منطقي	`bool`	`True`	❌ لا (Immutable)
فارغ	`NoneType`	`None`	❌ لا (Immutable)

> **ملاحظة تمهيدية مهمة:** لاحظ أن كل الأنواع التي تعلمناها اليوم هي **Immutable** (غير قابلة للتعديل المباشر). هذا المفهوم سيصبح شديد الأهمية في الفصل الثامن عندما نتعرّف على القوائم (Lists) التي هي **Mutable** (قابلة للتعديل) - الفرق بينهما سيُفسّر سلوكيات غريبة ستواجهها لاحقًا إن لم تفهم هذا الأساس جيدًا من الآن.

3.12 أخطاء شائعة يرتكبها المبتدئون (تجنّبها الآن)

⚠️ الخطأ 1: استخدام متغير قبل تعريفه

```python
print(age)  # ❌ NameError: name 'age' is not defined
age = 25
```

القاعدة: يجب تعريف (إسناد قيمة لـ) المتغير **قبل** استخدامه، فـPython تُنفّذ الكود سطرًا بسطر من الأعلى للأسفل.

⚠️ الخطأ 2: أخطاء إملائية في أسماء المتغيرات

```python
user_name = "أحمد"
print(usre_name)  # ❌ NameError - خطأ إملائي! (usre بدل user)
```

> **نصيحة عملية:** استخدم ميزة الإكمال التلقائي في VS Code (تظهر تلقائيًا أثناء الكتابة) بدل كتابة الأسماء يدويًا بالكامل في كل مرة - هذا يقضي على أغلب الأخطاء الإملائية.

⚠️ الخطأ 3: الخلط بين الأنواع بدون تحويل

```python
age = input("عمرك: ")  # str دائمًا
total = age + 10  # ❌ TypeError
```

⚠️ الخطأ 4: استخدام الكلمات المحجوزة كأسماء متغيرات

```python
type = "طالب"  # ⚠️ يعمل تقنيًا، لكنه يُخفي دالة type() المُدمجة!
```

⚠ **تحذير دقيق:** هذا لن يُسبب خطأ فوريًا، لكنه سيُسبب مشاكل خفية لاحقًا إن احتجت استخدام `type()` الحقيقية في نفس الجزء من الكود. تجنّب استخدام أسماء الدوال والأنواع المُدمجة (`type` , `list` , `str` , `input` ...) كأسماء لمتغيراتك.

3.13 أفضل الممارسات: ملخص شامل

```python
# ✅ ممارسات جيدة يجب اتباعها دائمًا

# 1. أسماء واضحة ومعبّرة بصيغة snake_case
user_age: int = 25

# 2. استخدام Type Hints للوضوح
total_price: float = 99.99

# 3. الثوابت بأحرف كبيرة
MAX_ATTEMPTS: int = 3

# 4. تحويل صريح عند الحاجة
user_input: str = input("أدخل رقمًا: ")
number: int = int(user_input)

# 5. تعليق توضيحي عند الحاجة لشرح "لماذا" وليس "ماذا"
discount_rate: float = 0.15  # خصم 15% للعملاء المميزين
```

3.14 تمارين عملية للفصل

🟢 تمرين 1 (مستوى مبتدئ)

أنشئ ملفًا جديدًا، وعرّف 5 متغيرات تمثّل معلومات عن كتاب تحبّه (العنوان، المؤلف، سنة النشر، عدد الصفحات، هل قرأته بالكامل)، مستخدمًا Type Hints مناسبة لكل متغير، ثم اطبعها جميعًا بصيغة منسّقة باستخدام f-strings.

🟡 تمرين 2 (مستوى متوسط)

اكتب برنامجًا يسأل المستخدم عن رقمين (باستخدام `input()`)، يحوّلهما لنوع `int`، ثم يطبع: مجموعهما، الفرق بينهما، حاصل ضربهما. تأكد من معالجة مشكلة أن `input()` تُعيد نصًا دائمًا.

🔴 تمرين 3 (تحدي)

بدون استخدام متغير مؤقت (`temp`)، اكتب كودًا يُبدّل قيم **ثلاثة** متغيرات بحيث: القيمة التي كانت في `a` تنتقل لـ `b` ، والتي كانت في `b` تنتقل لـ `c` ، والتي كانت في `c` تنتقل لـ `a` (تبديل دائري).

<details>
<summary>🔍 تلميح للتمرين 3</summary>

فكّر في الإسناد المتعدد الذي تعلمناه: `x, y = y, x` . هل يمكن توسيع نفس الفكرة لثلاثة متغيرات بترتيب معيّن؟

</details>

3.15 مشروع تطبيقي: «بطاقة التعريف الذكية - النسخة المُطوَّرة»

بما أننا تعمّقنا أكثر في المتغيرات والأنواع، لنُطوّر مشروع الفصل السابق ليكون أكثر احترافية.

📋 المهمة

أنشئ ملفًا باسم `advanced_id_card.py` يقوم بـ:

1. استقبال بيانات المستخدم: الاسم، سنة الميلاد، الطول (بالمتر)، هل هو موظف (نعم/لا)
2. **تحويل الأنواع بشكل صحيح** لكل قيمة (سنة الميلاد ← int، الطول ← float، حالة التوظيف ← bool)
3. استخدام **Type Hints** في كل متغير
4. حساب العمر الحالي تلقائيًا (بافتراض السنة الحالية 2025)
5. طباعة بطاقة تعريف منسّقة، مع التعامل الصحيح مع تحويل bool لنص عربي مفهوم («نعم»/«لا» بدل True/False)

💻 مثال على الكود المتوقع (حاول حله بنفسك أولاً قبل النظر!)

<details>
<summary>✅ حل نموذجي كامل (اضغط بعد المحاولة بنفسك)</summary>

Python advanced_id_card.py

```python
# استقبال البيانات
full_name: str = input("ما اسمك الكامل؟ ")
birth_year_text: str = input("ما سنة ميلادك؟ ")
height_text: str = input("ما طولك بالمتر؟ (مثال: 1.75) ")
is_employed_text: str = input("هل أنت موظف؟ (نعم/لا) ")

# تحويل الأنواع
birth_year: int = int(birth_year_text)
height: float = float(height_text)
is_employed: bool = is_employed_text.strip() == "نعم"

# الحسابات
CURRENT_YEAR: int = 2025
age: int = CURRENT_YEAR - birth_year

# تحويل bool لنص عربي مفهوم
employment_status: str = "نعم ✅" if is_employed else "لا ❌"

# طباعة البطاقة
print("=" * 35)
print("🪪 بطاقة التعريف الشخصية")
print("=" * 35)
print(f"👤 الاسم: {full_name}")
print(f"🎂 العمر: {age} سنة")
print(f"📏 الطول: {height} متر")
print(f"💼 موظف: {employment_status}")
print("=" * 35)
```

ملاحظة على الحل: استخدمنا `is_employed_text.strip() == "نعم"` لتحويل النص لـ bool بطريقة أذكى من `bool()` المباشرة (التي كانت ستُعيد `True` لأي نص غير فارغ، حتى لو كتب المستخدم «لا»!). هذا مثال عملي على التفكير النقدي حول تحويل الأنواع بدل الاعتماد الأعمى على الدوال الجاهزة.
</details>

3.16 اختبر فهمك (Quiz سريع)

1. ما الفرق الجوهري بين `=` و `==`؟
2. لماذا يُفضَّل وصف Python كـ«متغيرات تُشير لقيم» بدلاً من «صناديق تحتوي قيم»؟
3. ما الذي تُعيده دالة `input()` دائمًا، بغض النظر عمّا يكتبه المستخدم؟
4. اذكر الفرق بين لغة ديناميكية التنميط (Python) ولغة ثابتة التنميط (Java).
5. لماذا نستخدم Type Hints رغم أنها لا تمنع الأخطاء وقت التشغيل فعليًا؟

<details>
<summary>✅ الإجابات النموذجية</summary>

1. `=` تُستخدم لإسناد (تخزين) قيمة في متغير، بينما `==` تُستخدم للمقارنة بين قيمتين للتحقق من تساويهما.
2. لأن المتغير في Python لا «يحتوي» القيمة فعليًا، بل هو اسم (ملصق) يُشير إلى مكان في الذاكرة حيث تُخزَّن القيمة الفعلية - وهذا يُفسّر سلوكيات مهمة لاحقًا خصوصًا مع البيانات القابلة للتعديل.
3. تُعيد دائمًا نوع البيانات `str` (نص)، حتى لو كتب المستخدم أرقامًا.
4. في اللغة الديناميكية (Python)، يُحدَّد نوع المتغير تلقائيًا ويمكن تغييره لاحقًا بحرية. في اللغة الثابتة (Java)، يجب تحديد النوع صراحة عند الإنشاء ولا يمكن تغييره بعدها.
5. لأنها تجعل الكود أوضح للقراءة، تساعد المحررات على اكتشاف الأخطاء المحتملة قبل التشغيل، وتُحسّن الإكمال التلقائي - رغم أن Python نفسها تتجاهلها وقت التشغيل الفعلي.

</details>

3.17 📌 ملخص الفصل (Cheat Sheet)

المفهوم	مثال
إسناد قيمة	`age = 25`
Type Hint	`age: int = 25`
التحقق من النوع	`type(age)`
تحويل لـ int	`int("25")`
تحويل لـ float	`float("3.14")`
تحويل لـ str	`str(25)`

المفهوم	مثال
تحويل لـ bool	`bool(value)`
إسناد متعدد	`x, y = 5, 10`
تبديل قيم	`x, y = y, x`
ثابت (بالعُرف)	`MAX_SIZE = 100`
عنوان الذاكرة	`id(variable)`

⚠️ تذكّر دائمًا

- `input()` تُعيد **str** دائمًا - حوّلها قبل الاستخدام كرقم
- استخدم **snake_case** لكل متغيراتك
- استخدم **UPPER_CASE** للثوابت فقط
- `=` للتخزين، `==` للمقارنة

🎯 ماذا بعد؟

الآن تفهم كيف تُخزّن Python البيانات الأساسية. لكن ماذا لو أردنا التعامل مع نصوص بشكل أعمق - دمجها، تقسيمها، البحث فيها، تنسيقها باحترافية؟ في **الفصل الرابع**، سنغوص بعمق في عالم **النصوص (Strings)**، أحد أكثر أنواع البيانات استخدامًا في أي برنامج حقيقي، مع استكشاف شامل لـ f-strings وكل العمليات النصية الأساسية التي ستحتاجها يوميًا.

فصل 04 / 26 · الجزء الثاني

04

التعامل مع النصوص (Strings)

8 دقيقة قراءة | 16 قسمًا | 38 مثالًا | 1,664 كلمة

في هذا الفصل

4.1 مقدمة: لماذا النصوص بهذا الحجم من الأهمية؟
4.2 إنشاء النصوص: أكثر من مجرد اقتباسات
4.3 الحروف الخاصة (Escape Characters)
4.4 f-strings: الطريقة الحديثة والوحيدة التي يجب أن تتعلمها
4.5 الفهرسة (Indexing): الوصول لحرف معيّن
4.6 التقطيع (Slicing): استخراج جزء من النص
4.7 النصوص غير قابلة للتعديل (Immutable) - مفهوم حاسم
4.8 أهم Methods النصوص (الأكثر استخدامًا عمليًا)
4.9 دالة `len()`: طول النص
4.10 دمج النصوص: عمليات متعددة
4.11 جدول مرجعي شامل: أهم Methods النصوص
4.12 أخطاء شائعة ونصائح احترافية
4.13 تمارين عملية للفصل
4.14 مشروع تطبيقي: «محلل النصوص الذكي»
4.15 اختبر فهمك (Quiz سريع)
4.16 ملخص الفصل (Cheat Sheet)

> *«كل برنامج تقريبًا - من موقع ويب إلى نظام ذكاء اصطناعي - يتعامل مع النصوص بشكل أو بآخر. إتقان النصوص هو إتقان لغة التواصل بين الإنسان والآلة.»*

4.1 📖 مقدمة: لماذا النصوص بهذا الحجم من الأهمية؟

قد تظن أن النصوص (Strings) موضوع بسيط - بعد كل شيء، تعلمناها بالفعل في الفصل السابق كأحد الأنواع الأساسية. لكن الحقيقة أن النصوص هي **العمود الفقري** لمعظم البرمجة العملية: قراءة ملفات، معالجة مدخلات المستخدم، التواصل مع الإنترنت، تحليل البيانات، حتى الذكاء الاصطناعي الحديث (نماذج اللغة) مبني بالكامل على معالجة النصوص.

لهذا السبب، نُفرد فصلاً كاملاً لهذا النوع الواحد من البيانات - لأن العمق هنا سيُوفّر عليك أوقاتًا طويلة في كل مشروع مستقبلي.

4.2 إنشاء النصوص: أكثر من مجرد اقتباسات

✍️ الطرق الأساسية

```python
single_quoted: str = 'مرحبا'
double_quoted: str = "مرحبا"
```

كلاهما متطابق وظيفيًا. لكن متى تختار كل واحدة؟

🎯 متى تستخدم كل نوع اقتباس؟

```python
# استخدم مزدوجة عندما يحتوي النص على اقتباس مفرد
message: str = "اسمه كان 'محمد' في القصة"

# استخدم مفردة عندما يحتوي النص على اقتباس مزدوج
quote: str = '"النجاح يحتاج صبرًا" :قال المعلم'
```

✅ **أفضل الممارسات:** اختر نمطًا واحدًا (المزدوجة الأكثر شيوعًا عالميًا) والتزم به في كل مشروعك، واستخدم النوع الآخر فقط عند الحاجة الفعلية لتجنّب التعقيد. أداة `ruff` التي ثبّتناها في الفصل الثاني ستُنسّق هذا تلقائيًا لك بشكل متّسق.

النصوص متعددة الأسطر (Triple Quotes)

```python
paragraph: str = """
هذا نص يمتد
على عدة أسطر
دون الحاجة لأي رموز خاصة
"""

print(paragraph)
```

💡 **استخدام شائع جدًا:** النصوص ثلاثية الاقتباس تُستخدم أيضًا لكتابة **التوثيق (Docstrings)** للدوال والكلاسات، وهو ما سنتعلمه بالتفصيل في فصل الدوال (الفصل العاشر).

4.3 الحروف الخاصة (Escape Characters)

أحيانًا تحتاج لتضمين رموز خاصة داخل النص يصعب كتابتها مباشرة. Python توفر «رموز هروب» لهذا:

الرمز	المعنى	مثال
`\n`	سطر جديد (New Line)	`"سطر1\nسطر2"`
`\t`	مسافة تبويب (Tab)	`"اسم:\tمحمد"`
`\\`	شرطة مائلة فعلية	`"C:\\Users"`
`\"`	علامة اقتباس مزدوجة داخل نص مزدوج	`"قال: \"مرحبا\""`
`\'`	علامة اقتباس مفردة داخل نص مفرد	`'لا تقل: \'لا\''`

```python
message: str = "السطر الأول\nالسطر الثاني"
print(message)
# الناتج:
# السطر الأول
# السطر الثاني
```

النصوص الخام (Raw Strings)

أحيانًا تريد أن تُعامَل الشرطة المائلة `\` **حرفيًا** دون أي تفسير خاص - خصوصًا عند التعامل مع مسارات الملفات على Windows أو التعبيرات النمطية (Regex) لاحقًا:

```python
# بدون raw string - مشكلة!
path: str = "C:\new_folder"  # \n هنا تُفسَّر كسطر جديد! خطأ غير متوقع
print(path)
# الناتج: C:
# ew_folder  ← خطأ!

# مع raw string - الحل
path: str = r"C:\new_folder"
print(path)
# الناتج: C:\new_folder  ← صحيح تمامًا
```

⚠️ **تحذير عملي مهم:** هذا الخطأ شائع جدًا بين المبتدئين الذين يعملون على Windows ويتعاملون مع مسارات الملفات. دائمًا استخدم `r"..."` عند كتابة مسارات Windows، أو الأفضل - استخدم `pathlib` التي سنتعلمها في فصل لاحق وتتجنب هذه المشكلة بالكامل.

---

f-strings `4.4` : الطريقة الحديثة والوحيدة التي يجب أن تتعلمها

🕰️ خلفية تاريخية سريعة (للفهم لا للحفظ)

عبر تاريخ Python، وُجدت 3 طرق لتنسيق النصوص ودمج المتغيرات فيها:

```python
name = "سارة"
age = 28

# 1 الطريقة: % formatting (قديمة جدًا، نادرًا ما تُستخدم اليوم)
print("اسمي %s وعمري %d" % (name, age))

# 2 الطريقة: .format() (كانت شائعة، أصبحت أقل استخدامًا)
print("وعمري {} اسمي {}".format(name, age))

# 3 الطريقة: f-strings (من - الحديثة Python 3.6+)
print(f"اسمي {name} وعمري {age}")
```

🎯 **قرار تعليمي واضح:** في هذا الكتاب، سنستخدم **f-strings حصريًا تقريبًا**، لأنها الأسلوب الموصى به عالميًا اليوم - أوضح، أسرع تنفيذيًا، وأقل عرضة للأخطاء. ستجد الطريقتين القديمتين في كود آخرين أو مشاريع قديمة، لذا من المهم أن تتعرّف عليهما، لكن لا تستخدمهما في كودك الخاص.

✍️ الصيغة الأساسية لـ f-strings

ضع حرف `f` قبل علامة الاقتباس، وضع أي متغير داخل أقواس `{}` :

```python
name: str = "محمد"
age: int = 25

greeting: str = f"مرحبًا {name}, عمرك {age} سنة"
print(greeting)  # مرحبًا محمد, عمرك 25 سنة
```

🚀 قوة f-strings: التعبيرات الكاملة داخل الأقواس

الميزة الحقيقية لـ f-strings أنها لا تقبل فقط أسماء متغيرات، بل **أي تعبير Python صالح**:

```python
price: float = 49.99
quantity: int = 3

# حسابات مباشرة
print(f"الإجمالي: {price * quantity}")  # الإجمالي: 149.97

# استدعاء دوال
name: str = "محمد"
print(f"الاسم بحروف كبيرة: {name.upper()}")  # محمد :الاسم بحروف كبيرة (ملاحظة: upper لا تؤثر على العربية)

# شروط مباشرة (Ternary expressions)
age: int = 15
print(f"الحالة: {'قاصر' if age < 18 else 'بالغ'}")  # الحالة: قاصر
```

🎨 تنسيق متقدم: التحكم في عرض الأرقام

f-strings توفر صياغة قوية للتحكم في كيفية عرض الأرقام - مهارة ستحتاجها كثيرًا في أي تطبيق حقيقي:

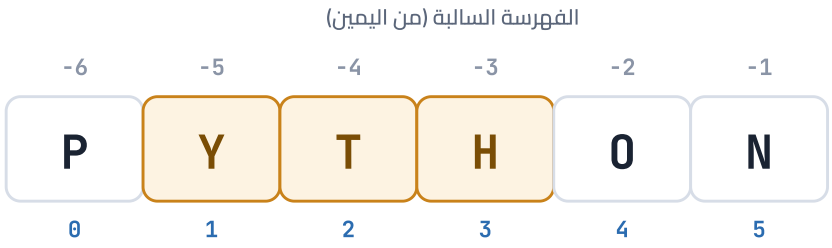
```python
price: float = 1234.5678

# تحديد عدد الخانات العشرية
print(f"{price:.2f}")  # 1234.57 (تقريب لخانتين عشريتين)

# إضافة فاصلة آلاف
print(f"{price:,.2f}")  # 1,234.57

# عرض كنسبة مئوية
ratio: float = 0.856
print(f"{ratio:.1%}")  # 85.6%

# محاذاة النص (مفيد جدًا للجداول في الطرفية)
name: str = "محمد"
print(f"[{name:>10}]")  # [      محمد] (محاذاة يمين، عرض 10)
print(f"[{name:<10}]")  # [محمد      ] (محاذاة يسار، عرض 10)
print(f"[{name:^10}]")  # [   محمد   ] (محاذاة وسط، عرض 10)
```

🐛 ميزة تشخيصية رائعة (Python 3.8+): `=` داخل f-string

```python
x: int = 10
y: int = 20

print(f"{x=}, {y=}, {x + y=}")
# الناتج: x=10, y=20, x + y=30
```

💡 **نصيحة عملية قوية:** هذه الميزة ممتازة أثناء تصحيح الأخطاء (Debugging) - بدلاً من كتابة `print(f"x is {x}")` يدويًا، فقط اكتب `print(f"{x=}")` وستحصل على اسم المتغير وقيمته معًا تلقائيًا. استخدمها بكثرة أثناء تعلّمك لفهم ما يحدث في كودك.

4.5 الفهرسة (Indexing): الوصول لحرف معيّن



YTH ← s[1:4] (البداية مشمولة، والنهاية غير مشمولة)

الفهرسة الموجبة تبدأ من 0، والسالبة من -1 عند آخر حرف.

شكل 4.1 — الفهرسة الموجبة والسالبة والتقطيع في النصوص

🔢 فهم النظام: يبدأ العد من صفر دائمًا!

في Python (وأغلب لغات البرمجة)، كل نص هو **سلسلة من الحروف**، وكل حرف له «فهرس» (رقم موضعه) يبدأ من **0** وليس من 1.

```python
word: str = "مرحبا"
#           م ر ح ب ا
# الفهرس:   0 1 2 3 4
```

```python
word: str = "مرحبا"

print(word[0])  # (الحرف الأول)  م
print(word[1])  # ر
print(word[4])  # (الحرف الأخير - الفهرس 4، لأن الطول 5)  ا
```

⚠️ **تحذير حرج - خطأ شائع جدًا:** البدء من 0 وليس 1 يُربك المبتدئين باستمرار. تذكّر دائمًا: **الفهرس الأول هو 0**، والفهرس الأخير هو **طول النص - 1**.

🔄 الفهرسة السالبة: ميزة ذكية في Python

Python تسمح بالعد من **النهاية** باستخدام أرقام سالبة:

```python
word: str = "مرحبا"

print(word[-1])  # (آخر حرف)  ا
print(word[-2])  # (قبل الأخير)  ب
```

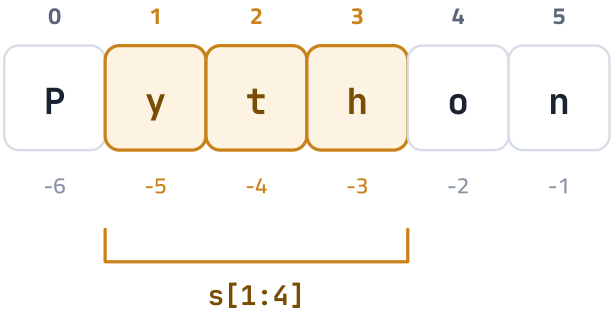
💡 **لماذا هذا مفيد؟** تخيّل أنك تريد آخر حرف في نص طويل لا تعرف طوله بالضبط. بدل حساب الطول يدويًا، فقط استخدم `[-1]` مباشرة - أنيق وبسيط.

⚠️ خطأ شائع: تجاوز حدود النص

```python
word: str = "مرحبا"
print(word[10])  # ❌ IndexError: string index out of range
```

الحل: دائمًا تأكد أن الفهرس الذي تطلبه موجود فعليًا ضمن طول النص (سنتعلم `len()` بعد قليل).

4.6 التقطيع (Slicing): استخراج جزء من النص



شكل 4.2 — التقطيع: البداية مشمولة والنهاية غير مشمولة

التقطيع يسمح لك باستخراج **جزء** من النص، وليس حرفًا واحدًا فقط:

✍️ الصيغة الأساسية

```python
text[start:end:step]
```

- `start` : الفهرس الذي يبدأ منه (مُضمَّن)
- `end` : الفهرس الذي ينتهي عنده (**غير** مُضمَّن - نقطة مهمة جدًا!)
- `step` : القفزة بين كل عنصر (اختياري، الافتراضي 1)

🎯 أمثلة متدرجة

```python
text: str = "البرمجة ممتعة"

print(text[0:8])    # البرمجة  (من الفهرس 0 حتى 7، الفهرس 8 غير مُضمَّن)
print(text[9:])     # ممتعة  (من الفهرس 9 حتى النهاية)
print(text[:8])     # البرمجة  (من البداية حتى الفهرس 7)
print(text[:])      # البرمجة ممتعة  (نسخة كاملة من النص)
```

⚠️ **تحذير مهم جدًا - أكثر الأخطاء شيوعًا في التقطيع:** الفهرس `end` **غير مُضمَّن** في النتيجة. هذا يُربك المبتدئين باستمرار:

```python
text: str = "Python"
print(text[0:3])  # Pyt  (ليس Pyth!) - لأن الفهرس 3 غير مُضمَّن
```

🔄 استخدام step للقفز بين الحروف

```python
text: str = "0123456789"

print(text[::2])    # 02468  (كل حرف ثانٍ)
print(text[1::2])   # 13579  (كل حرف ثانٍ، بدءًا من الفهرس 1)
print(text[::-1])   # 9876543210  (عكس النص بالكامل!)
```

💡 **نصيحة عملية مشهورة:** `text[::-1]` هي الطريقة الأكثر شيوعًا وأناقة في Python لعكس أي نص بالكامل - احفظ هذا النمط جيدًا، ستستخدمه كثيرًا.

4.7 النصوص غير قابلة للتعديل (Immutable) - مفهوم حاسم

🔒 الفهم الجوهري

تذكر من الفصل السابق: النصوص (`str`) هي نوع بيانات **Immutable** - أي **لا يمكن تعديلها مباشرة** بعد إنشائها.

```python
word: str = "مرحبا"
word[0] = "M"  # ❌ TypeError: 'str' object does not support item assignment
```

🤔 لكن ألسنا «نُعدّل» النصوص باستمرار؟

نعم، لكن ما يحدث فعليًا هو إنشاء نص **جديد بالكامل**، وليس تعديل النص الأصلي في مكانه:

```python
word: str = "مرحبا"
new_word: str = word.upper()  # لا يُعدّل word، بل يُنشئ نصًا جديدًا تمامًا

print(word)      # مرحبا  (لم يتغيّر!)
print(new_word)  # (نسخة معدّلة جديدة)
```

> **لماذا هذا مهم فعليًا؟** هذا يُفسّر لماذا دائمًا تحتاج لإعادة إسناد النتيجة إلى متغير (نفس المتغير أو جديد) عند استخدام methods تُغيّر النص - الـ method لا تُغيّر الأصل أبدًا:

```python
name: str = "محمد"
name.upper()        # ⚠️ النتيجة تُهدَر! لا يفعل شيئًا فعليًا
print(name)         # محمد  (لم يتغيّر!)

name = name.upper()  # ✅ الطريقة الصحيحة - إعادة الإسناد
print(name)          # محمد (بأحرف كبيرة إن كانت إنجليزية)
```

4.8 أهم Methods النصوص (الأكثر استخدامًا عمليًا)

Python توفر عشرات الـ methods للنصوص، لكن لن نُغرقك بكلها. إليك **الأكثر استخدامًا فعليًا** في المشاريع الحقيقية:

🔠 تغيير حالة الأحرف

```python
text: str = "Hello World"

print(text.upper())       # HELLO WORLD
print(text.lower())       # hello world
print(text.title())       # Hello World (كل كلمة تبدأ بحرف كبير)
print(text.capitalize())  # Hello world (أول حرف فقط كبير)
```

✂️ إزالة المسافات الزائدة

```python
text: str = "   مرحبا بالعالم   "

print(text.strip())   # "مرحبا بالعالم" (إزالة المسافات من الطرفين)
print(text.lstrip())  # "مرحبا بالعالم   " (من اليسار فقط)
print(text.rstrip())  # "   مرحبا بالعالم" (من اليمين فقط)
```

💡 **استخدام عملي شديد الأهمية:** `.strip()` أساسية جدًا عند معالجة مدخلات المستخدم من `input()`، لأن المستخدمين كثيرًا ما يكتبون مسافات زائدة عن غير قصد:

```python
name: str = input("اسمك: ").strip()
```

🔍 البحث داخل النص

```python
text: str = "مبرمجة بلغة Python ممتعة"

print("Python" in text)        # True
print("Java" in text)          # False
print(text.find("Python"))     # 11 (الفهرس حيث بدأت الكلمة)
print(text.find("Java"))       # -1 (غير موجودة)
print(text.count("ة"))         # عدد مرات تكرار الحرف
```

🔄 **مقارنة مهمة:** `in` مقابل `.find()`

الطريقة	متى تستخدمها
`in`	عندما تحتاج فقط معرفة: موجود أم لا؟ (يُعيد bool)
`.find()`	عندما تحتاج معرفة **أين** بالضبط (يُعيد رقم الفهرس، أو -1)

🔄 الاستبدال

```python
text: str = "أنا أحب Java"
new_text: str = text.replace("Java", "Python")
print(new_text)  # أنا أحب Python
```

✂️ التقسيم والدمج (من أهم الـ methods على الإطلاق)

```python
sentence: str = "التفاح,الموز,البرتقال"

# تقسيم النص إلى قائمة (سنتعلم القوائم بعمق في الفصل الثامن)
fruits: list[str] = sentence.split(",")
print(fruits)  # ['التفاح', 'الموز', 'البرتقال']

# العملية العكسية: دمج قائمة في نص واحد
joined: str = " - ".join(fruits)
print(joined)  # التفاح - الموز - البرتقال
```

💡 **تطبيق عملي حقيقي جدًا:** `.split()` و `.join()` هما من أكثر الـ methods استخدامًا في معالجة البيانات الحقيقية (مثل قراءة ملفات CSV التي سنتعلمها لاحقًا).

✅ التحقق من محتوى النص

```python
print("12345".isdigit())      # True - كلها أرقام؟
print("Hello".isalpha())      # True - كلها حروف؟
print("Hello123".isalnum())   # True - حروف أو أرقام فقط؟
print("   ".isspace())        # True - كلها مسافات؟
print("Hello".startswith("He"))  # True
print("Hello".endswith("lo"))    # True
```

🎯 **استخدام عملي شديد الشيوع:** التحقق من صحة مدخلات المستخدم:

```python
age_input: str = input("أدخل عمرك: ")

if age_input.isdigit():
    age: int = int(age_input)
    print(f"عمرك {age} سنة")
else:
    print("خطأ: يجب إدخال رقم فقط!")
```

4.9 دالة `len()`: طول النص

```python
text: str = "مرحبا"
print(len(text))  # 5
```

⚠️ **ملاحظة تقنية:** `len()` ليست method تابعة للنص (لا تُكتب كـ `text.len()`)، بل **دالة مستقلة** تأخذ النص كمُدخل. هذا تصميم مقصود في Python يُفرّق بين الدوال العامة (`len()`, `print()`) والـ methods الخاصة بنوع بيانات معين (`.upper()`, `.strip()`).

🎯 استخدام شائع: التحقق من طول كلمة مرور

```python
password: str = input("أدخل كلمة مرور: ")

if len(password) < 8:
    print("⚠️ كلمة المرور قصيرة جدًا! يجب أن تكون 8 أحرف على الأقل")
else:
    print("✅ كلمة المرور مقبولة")
```

4.10 دمج النصوص: عمليات متعددة

```python
first_name: str = "محمد"
last_name: str = "أحمد"

# الطريقة 1: عامل + (Concatenation)
full_name: str = first_name + " " + last_name
print(full_name)  # محمد أحمد

# الطريقة 2: f-string (الأفضل والأكثر مرونة)
full_name = f"{first_name} {last_name}"

# الطريقة 3: تكرار النص بعامل *
separator: str = "-" * 20
print(separator)  # --------------------
```

✅ **أفضل الممارسات:** استخدم f-strings دائمًا عند دمج أكثر من قيمتين أو عند الحاجة لتنسيق، واستخدم `+` فقط للدمج البسيط جدًا بين نصّين.

4.11 جدول مرجعي شامل: أهم Methods النصوص

Method	الوظيفة	مثال
`.upper()`	تحويل لأحرف كبيرة	`"ABC"` ← `"abc".upper()`
`.lower()`	تحويل لأحرف صغيرة	`"abc"` ← `"ABC".lower()`
`.strip()`	إزالة المسافات من الطرفين	`"hi"` ← `" hi ".strip()`
`.split(sep)`	تقسيم لقائمة	`["a","b"]` ← `"a,b".split(",")`
`.join(list)`	دمج قائمة لنص	`"a-b"` ← `"-".join(["a","b"])`
`.replace(old,new)`	استبدال جزء	`"Hi"` ← `"hi".replace("h","H")`
`.find(sub)`	إيجاد موضع	`2` ← `"hello".find("l")`
`.count(sub)`	عدّ التكرارات	`3` ← `"aaa".count("a")`
`.startswith(s)`	يبدأ بـ؟	`True` ← `"hi".startswith("h")`

Method	الوظيفة	مثال
`.endswith(s)`	ينتهي بـ؟	`True` ← `"hi".endswith("i")`
`.isdigit()`	كلها أرقام؟	`True` ← `"123".isdigit()`
`.isalpha()`	كلها حروف؟	`True` ← `"abc".isalpha()`
`len(text)`	طول النص	`5` ← `len("hello")`

4.12 أخطاء شائعة ونصائح احترافية

⚠️ الخطأ 1: نسيان أن methods لا تُعدّل الأصل

```python
name: str = "محمد"
name.upper()
print(name)  # محمد ← لم يتغيّر! يجب إعادة الإسناد
```

⚠️ الخطأ 2: الخلط بين فهرسة النصوص والتقطيع

```python
text: str = "Python"
print(text[0])    # P (حرف واحد)
print(text[0:1])  # P (لكن كنتيجة تقطيع - الفرق دقيق لكن مهم لاحقًا)
print(text[10])   # ❌ IndexError
print(text[10:20])  # "" نص فارغ (التقطيع لا يُسبب خطأ حتى لو تجاوز الحدود!)
```

> 💡 **ملاحظة دقيقة جدًا:** لاحظ الفرق: الفهرسة المباشرة `[10]` خارج الحدود تُسبب خطأ، بينما التقطيع `[10:20]` خارج الحدود يُعيد ببساطة نصًا فارغًا دون أي خطأ! هذا سلوك مقصود في Python يستحق الانتباه.

⚠️ الخطأ 3: استخدام `+` لدمج نص مع نوع آخر مباشرة

```python
age: int = 25
message: str = "عمري: " + age  # ❌ TypeError
message: str = "عمري: " + str(age)  # ✅ صحيح
message: str = f"عمري: {age}"       # ✅ الأفضل والأبسط
```

✅ نصيحة احترافية: أداء الدمج المتكرر

```python
# ❌ غير فعّال عند دمج نصوص كثيرة في حلقة (سنتعلم الحلقات في الفصل السابع)
result: str = ""
for i in range(1000):
    result += str(i)  # كل عملية += تُنشئ نصًا جديدًا بالكامل في الذاكرة!

# ✅ أكثر كفاءة (سنطبقها عمليًا لاحقًا)
parts: list[str] = [str(i) for i in range(1000)]
result: str = "".join(parts)
```

> 💡 لا تقلق إن لم تفهم مثال الحلقة الآن بالكامل - سنعود لهذا المفهوم بعمق في فصل القوائم والحلقات. فقط احتفظ بهذه النصيحة في ذهنك لاحقًا.

4.13 تمارين عملية للفصل

🟢 تمرين 1 (مستوى مبتدئ)

اطلب من المستخدم إدخال اسمه الكامل، ثم اطبع:
- الاسم بأحرف كبيرة بالكامل
- عدد الحروف في الاسم (بدون المسافات)
- الاسم معكوسًا

🟡 تمرين 2 (مستوى متوسط)

اكتب برنامجًا يطلب من المستخدم إدخال بريد إلكتروني، ويتحقق من:
- هل يحتوي على الرمز `@` ؟
- هل يحتوي على نقطة `.` بعد الـ `@` ؟

- إن تحققت الشروط، اطبع «بريد إلكتروني صالح الشكل»، وإلا اطبع رسالة خطأ مناسبة

🔴 تمرين 3 (تحدي)

اكتب برنامجًا يتحقق إن كانت جملة مُدخلة من المستخدم هي **Palindrome** (تُقرأ نفسها من الاتجاهين)، **متجاهلًا** المسافات وحالة الأحرف (كبيرة/صغيرة). مثال: "Was it a car or a cat I saw" يجب أن تُعتبر Palindrome صحيحة.

<details>
<summary>🔍 تلميح للتمرين 3</summary>

فكّر في الخطوات: (1) حوّل كل الأحرف لنفس الحالة باستخدام `.lower()`. (2) أزل المسافات باستخدام `.replace(" ", "")`. (3) قارن النص بنسخته المعكوسة `[::-1]`.

</details>

4.14 مشروع تطبيقي: «محلل النصوص الذكي»

📋 المهمة

أنشئ ملفًا باسم `text_analyzer.py` يقوم بتحليل أي نص يُدخله المستخدم، ويعرض تقريرًا كاملاً عنه:

المتطلبات:

1. طلب نص من المستخدم (جملة أو فقرة)
2. حساب: عدد الحروف (بدون مسافات)، عدد الكلمات، عدد المسافات
3. إيجاد أطول كلمة في النص
4. تحويل النص لصيغة «عنوان» (كل كلمة تبدأ بحرف كبير - إن كان إنجليزيًا)
5. عرض النص معكوسًا بالكامل
6. طباعة تقرير منسّق بشكل احترافي وجذاب

💻 حل نموذجي كامل

<details>
<summary>✅ اضغط لرؤية الحل الكامل (حاول بنفسك أولاً!)</summary>

```python
"""برنامج تحليل النصوص - يعرض إحصائيات شاملة عن أي نص مُدخَل."""

# استقبال النص من المستخدم
text: str = input("أدخل نصًا لتحليله: ").strip()

# تقسيم النص لكلمات لحساب الإحصائيات
words: list[str] = text.split()

# الحسابات الأساسية
total_chars: int = len(text.replace(" ", ""))
word_count: int = len(words)
space_count: int = text.count(" ")

# إيجاد أطول كلمة
longest_word: str = max(words, key=len) if words else ""

# تحويلات النص
title_case: str = text.title()
reversed_text: str = text[::-1]

# طباعة التقرير
print("\n" + "=" * 40)
print("📊 تقرير تحليل النص")
print("=" * 40)
print(f"📝 النص الأصلي: {text}")
print(f"🔤 عدد الحروف (بدون مسافات): {total_chars}")
print(f"📦 عدد الكلمات: {word_count}")
print(f"␣ عدد المسافات: {space_count}")
print(f"🏆 أطول كلمة: {longest_word}")
print(f"🔠 بصيغة العنوان: {title_case}")
print(f"🔄 النص معكوسًا: {reversed_text}")
print("=" * 40)
```

</details>

ملاحظة على الحل: استخدمنا `max(words, key=len)` لإيجاد أطول كلمة - وهذا مثال تشويقي بسيط على دالة `max()` مع معامل `key`، وهو مفهوم سنتعمّق فيه لاحقًا في فصل الدوال المتقدمة. لا تقلق إن بدا غامضًا الآن.

4.15 اختبر فهمك (Quiz سريع)

1. لماذا تُعتبر f-strings الطريقة المُفضَّلة لتنسيق النصوص في Python الحديثة؟
2. ما الفرق بين `text[3]` و `text[3:4]`؟
3. لماذا لا يُسبب `text[100:200]` خطأ حتى لو تجاوز طول النص، بينما `text[100]` يُسبب خطأ؟
4. ما الفرق بين `in` و `.find()` عند البحث داخل نص؟
5. لماذا يجب إعادة إسناد نتيجة method مثل `.upper()` لمتغير بدلاً من استدعائها فقط؟

<details>
<summary>✅ الإجابات النموذجية</summary>

1. لأنها أوضح في القراءة، تدعم تعبيرات Python كاملة داخل الأقواس، أسرع تنفيذيًا من الطرق القديمة، وتوفر تحكمًا قويًا بالتنسيق (كالخانات العشرية والمحاذاة).
2. `text[3]` يُعيد حرفًا واحدًا (نوعه str بطول 1)، بينما `text[3:4]` يُعيد نصًا ناتجًا عن تقطيع (أيضًا بطول 1 في هذه الحالة تحديدًا، لكن المفهوم مختلف تقنيًا - التقطيع يُعيد دائمًا str حتى لو كان حرفًا واحدًا أو فارغًا).
3. لأن التقطيع (Slicing) مصمَّم في Python ليكون «متسامحًا» - إن تجاوزت الحدود، يُعيد ببساطة أقصى ما هو متاح أو نصًا فارغًا، بعكس الفهرسة المباشرة التي تتطلب وجود الفهرس فعليًا.
4. `in` تُعيد فقط `True` / `False` (هل موجود أم لا)، بينما `.find()` تُعيد رقم الفهرس الدقيق لأول ظهور، أو `-1` إن لم تُوجد.
5. لأن النصوص في Python غير قابلة للتعديل (Immutable)، فكل method تُعدّل النص فعليًا تُنشئ نسخة جديدة معدّلة بدلاً من تغيير الأصل، لذا يجب حفظ هذه النسخة الجديدة في متغير لاستخدامها.

</details>

4.16 📌 ملخص الفصل (Cheat Sheet)

```python
# الإنشاء
text: str = "مرحبا"
multiline: str = """نص متعدد الأسطر"""
raw: str = r"C:\path"

# f-strings
f"{variable}"
f"{value:.2f}"       # خانتان عشريتان
f"{value:,}"         # فاصلة آلاف
f"{variable=}"       # تشخيص سريع

# الفهرسة والتقطيع
text[0]          # الحرف الأول
text[-1]         # الحرف الأخير
text[2:5]        # من الفهرس 2 حتى 4
text[::-1]       # عكس النص

# أهم Methods
text.upper() / text.lower()
text.strip()
text.split(",") / "-".join(list)
text.replace(old, new)
text.find(sub)
"sub" in text
len(text)
```

🎯 ماذا بعد؟

تعلمنا الآن كيف نتعامل مع النصوص باحتراف. لكن ماذا عن الأرقام؟ في **الفصل الخامس**، سنتعمّق في **العمليات الحسابية**، نكتشف فخ الدقة العشرية الشهير عمليًا، نتعلّم التقريب الصحيح، ونبني حاسبة علمية بسيطة من الطرفية تطبّق كل هذه المفاهيم.

فصل 05 / 26 · الجزء الثاني

05

الأرقام والعمليات الحسابية

8 دقيقة قراءة | 18 قسمًا | 37 مثالًا | 1,632 كلمة

في هذا الفصل

5.1 مقدمة: أكثر من مجرد جمع وطرح
5.2 العمليات الحسابية الأساسية
5.3 العمليات الحسابية المتقدمة
5.4 أولوية العمليات (Operator Precedence)
5.5 فخ الدقة العشرية: المشكلة الأشهر في كل لغات البرمجة
5.6 التقريب بعمق: `round()` وتفاصيلها الدقيقة
5.7 دوال رياضية مُدمجة مفيدة
5.8 تحويلات الأنواع الرقمية بعمق
5.9 العمليات المختصرة (Augmented Assignment Operators)
5.10 مقارنة الأرقام: عوامل المقارنة
5.11 الأعداد الكبيرة جدًا: ميزة فريدة في Python
5.12 الأعداد العشوائية: مقدمة سريعة لوحدة `random`
5.13 أخطاء شائعة ونصائح احترافية
5.14 جدول مرجعي شامل للفصل
5.15 تمارين عملية للفصل
5.16 مشروع تطبيقي: «الحاسبة العلمية التفاعلية»
5.17 اختبر فهمك (Quiz سريع)
5.18 ملخص الفصل (Cheat Sheet)

> *«الحاسوب آلة حسابية في جوهرها. فهم كيف تتعامل Python مع الأرقام هو فهم لب عمل الكمبيوتر نفسه.»*

5.1 📖 مقدمة: أكثر من مجرد جمع وطرح

قد تظن أن التعامل مع الأرقام هو أبسط جزء في البرمجة - بعد كل شيء، تعلمته في المدرسة الابتدائية. لكن في هذا الفصل، ستكتشف فخاخًا حقيقية تُربك حتى المبرمجين ذوي الخبرة إن لم يفهموها جيدًا، وستتعلم تقنيات احترافية للتعامل مع الأرقام بدقة ونظافة.

5.2 العمليات الحسابية الأساسية

```python
a: int = 10
b: int = 3

print(a + b)   # 13   ← الجمع
print(a - b)   # 7    ← الطرح
print(a * b)   # 30   ← الضرب
print(a / b)   # 3.3333333333333335  ← القسمة (تُعيد دائمًا float!)
```

⚠️ تحذير مهم: القسمة `/` تُعيد دائمًا `float`

```python
result = 10 / 2
print(result)        # 5.0  ← float! ليس int كـ 5
print(type(result))  # <class 'float'>
```

> 💡 **لماذا هذا مهم؟** في لغات كثيرة أخرى، قسمة عددين صحيحين تُعيد عددًا صحيحًا. لكن Python 3 غيّرت هذا السلوك عمدًا (مقارنة بـ Python 2 القديمة) لتجنّب فقدان الدقة العشرية بشكل غير متوقع.

5.3 العمليات الحسابية المتقدمة

➗ القسمة الصحيحة (Floor Division) `//`

```python
print(10 // 3)   # 3   ← ناتج القسمة بدون الباقي (مُقرَّب للأسفل)
print(10 / 3)    # 3.3333...  ← القسمة العادية للمقارنة
```

🔢 باقي القسمة (Modulo) `%`

```python
print(10 % 3)    # 1   ← الباقي بعد القسمة
print(20 % 5)    # 0   ← قابل للقسمة تمامًا
```

🎯 **استخدام عملي شديد الشيوع:** `%` هي الأداة الأساسية للتحقق من كون رقم زوجيًا أو فرديًا:

```python
number: int = 7
if number % 2 == 0:
    print("زوجي")
else:
    print("فردي")
```

🔺 الأُس (Exponentiation) `**`

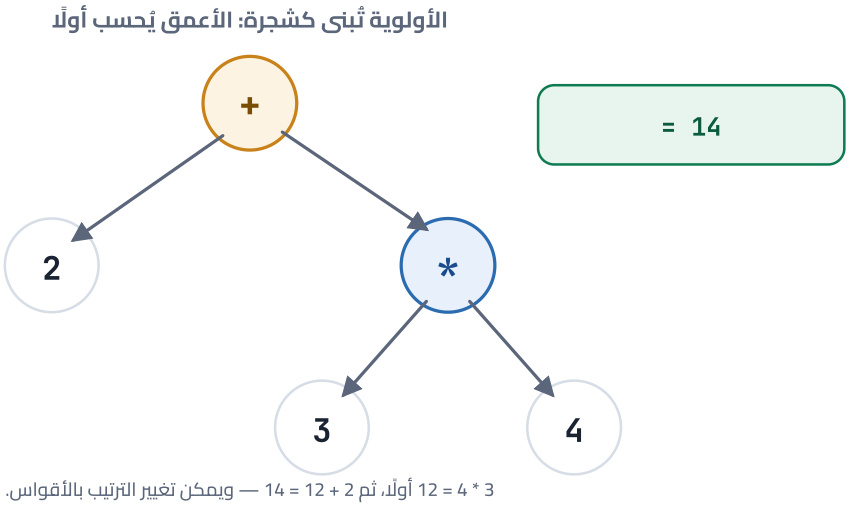
```python
print(2 ** 3)    # 8    ← 2 أُس 3
print(10 ** 2)   # 100  ← تربيع 10
print(4 ** 0.5)  # 2.0  ← الجذر التربيعي (أُس 0.5)
```

⚠️ **خطأ شائع عند القادمين من لغات أخرى:** في بعض اللغات (مثل JavaScript القديمة)، الأُس يُكتب كدالة `Math.pow()`. في Python، استخدم `**` مباشرة - لا تخلط مع `^` التي تعني شيئًا مختلفًا تمامًا في Python (XOR على مستوى البت - موضوع متقدم لن نحتاجه الآن).

📊 جدول شامل لكل العمليات الحسابية

العامل	الاسم	مثال	النتيجة
`+`	جمع	`7 + 3`	`10`
`-`	طرح	`7 - 3`	`4`
`*`	ضرب	`7 * 3`	`21`
`/`	قسمة عادية	`7 / 3`	`2.333...`
`//`	قسمة صحيحة	`7 // 3`	`2`
`%`	باقي القسمة	`7 % 3`	`1`
`**`	أُس	`7 ** 3`	`343`

5.4 أولوية العمليات (Operator Precedence)



شكل 5.1 — أولوية العمليات كشجرة

القاعدة المعروفة من الرياضيات (PEMDAS)

Python تتبع نفس القواعد الرياضية التقليدية التي تعلمتها في المدرسة:

```
1. الأقواس ()
2. الأُس **
3. الضرب * والقسمة / // (من اليسار لليمين)
4. الجمع + والطرح - (من اليسار لليمين)
```

```python
result = 2 + 3 * 4
print(result)  # 14  ← (14=12+2) ثم الجمع ،(12=4*3) الضرب أولاً

result = (2 + 3) * 4
print(result)  # 20  ← (20=4*5) ثم الضرب ،(5=3+2) الأقواس أولاً
```

✅ **نصيحة احترافية:** حتى عندما تكون متأكدًا من أولوية العمليات، استخدم الأقواس `()` لجعل الكود **أوضح** لأي قارئ آخر (أو لك مستقبلاً). الوضوح دائمًا أهم من توفير حرفين من الكتابة:

```python
# يعمل لكن يحتاج تفكيرًا لفهمه
total = price * quantity - discount * quantity

# أوضح بكثير بنفس النتيجة
total = (price * quantity) - (discount * quantity)
```

5.5 فخ الدقة العشرية: المشكلة الأشهر في كل لغات البرمجة

شكل 5.2 — لماذا 0.1 + 0.2 لا تساوي 0.3

🔬 التجربة الصادمة

```python
print(0.1 + 0.2)
# الناتج: 0.30000000000000004
# ليس 0.3 كما تتوقع!
```

هذا ليس خطأ في Python - **كل** لغات البرمجة تقريبًا (C++, Java, JavaScript) تعاني من نفس المشكلة بالضبط.

🤔 لماذا يحدث هذا؟

الكمبيوتر يُخزّن كل شيء في النظام **الثنائي** (0 و 1 فقط). بعض الكسور العشرية البسيطة في نظامنا العشري (مثل 0.1) لا يمكن تمثيلها **بدقة تامة** في النظام الثنائي - تمامًا كما أن الكسر 1/3 لا يمكن كتابته بدقة تامة كعدد عشري (0.3333... إلى ما لا نهاية).

> **تشبيه مبسّط:** تخيل أنك تحاول قياس طول بمسطرة مُقسّمة لأثلاث فقط (لا أرباع ولا أخماس). بعض الأطوال ستقيسها بدقة تامة، لكن أطوالاً أخرى ستحتاج تقريبًا. هذا بالضبط ما يحدث مع الكمبيوتر والنظام الثنائي مقابل بعض الكسور العشرية.

⚠️ تبعات عملية خطيرة لهذه المشكلة

```python
price: float = 19.99
quantity: int = 3
total: float = price * quantity

print(total)  # 59.96999999999999  ← ليس 59.97 بالضبط!
```

تخيل لو كان هذا نظام حسابات بنكية حقيقي - هذا الفرق الصغير جدًا قد يُسبب مشاكل محاسبية خطيرة متراكمة.

✅ الحل الأول: التقريب عند العرض باستخدام `round()`

```python
total: float = 0.1 + 0.2
print(round(total, 2))  # 0.3  ← تقريب لخانتين عشريتين
```

```python
price: float = 19.99
quantity: int = 3
total: float = round(price * quantity, 2)
print(total)  # 59.97
```

> ⚠️ **تحذير مهم:** `round()` يحل مشكلة **العرض**، لكن لا يحل مشكلة **الدقة الداخلية** أثناء الحسابات المتتالية. في عمليات حسابية طويلة ومتكررة (كأنظمة مالية حقيقية)، لا تعتمد فقط على `round()` .

✅ الحل الثاني (الاحترافي): استخدام وحدة `decimal` للحسابات المالية الدقيقة

عندما تتعامل مع أموال حقيقية أو حسابات تتطلب دقة تامة، استخدم وحدة `decimal` المُدمجة في Python:

```python
from decimal import Decimal

price = Decimal("19.99")  # مباشرة لاحظ: كنص! وليس float
quantity = Decimal("3")
total = price * quantity

print(total)  # 59.97 ← دقيق تمامًا!
```

🎯 **قاعدة ذهبية للمشاريع الحقيقية**: أي نظام يتعامل مع أموال فعلية (تطبيق متجر، نظام محاسبة، بنك) **يجب** أن يستخدم `Decimal` بدلاً من `float` العادي. هذا ليس اختياريًا - هو معيار صناعي معتمد عالميًا لتجنّب كوارث حسابية حقيقية حدثت في أنظمة مالية بسبب هذه المشكلة بالضبط.

🔄 مقارنة سريعة: متى تستخدم float ومتى decimal؟

الحالة	الاستخدام الموصى به
حسابات علمية، إحداثيات، قياسات تقريبية	✅ `float`
حسابات مالية، أسعار، فواتير، رواتب	✅ `Decimal`
عرض سريع لرقم تقريبي للمستخدم	`round()` كافية

5.6 التقريب بعمق: `round()` وتفاصيلها الدقيقة

✍️ الاستخدام الأساسي

```python
print(round(3.14159, 2))   # 3.14
print(round(3.14159, 0))   # 3.0
print(round(3.14159))      # 3  (بدون معامل ثانٍ، يُعيد int)
```

⚠️ سلوك غريب يجب معرفته: Banker's Rounding

```python
print(round(2.5))   # 2  ← ليس 3 كما قد تتوقع!
print(round(3.5))   # 4
print(round(0.5))   # 0  ← ليس 1!
```

هذا ليس خطأ - Python تستخدم أسلوبًا يُسمى **«التقريب المصرفي» (Banker's Rounding)**: عندما يكون الرقم بالضبط في المنتصف (مثل 0.5)، يُقرَّب لأقرب رقم **زوجي**. هذا الأسلوب مُستخدم إحصائيًا لتقليل التحيّز التراكمي في التقريب عبر آلاف العمليات.

> ⚠️ **تحذير عملي**: إذا كنت تتوقع التقريب «التقليدي» (0.5 دائمًا للأعلى)، ستُفاجأ بهذا السلوك. لحسابات حساسة، استخدم وحدة `decimal` مع إعدادات تقريب صريحة بدلاً من الاعتماد على `round()` الافتراضية.

5.7 دوال رياضية مُدمجة مفيدة

🔢 دوال أساسية بدون استيراد

```python
print(abs(-5))        # 5    ← القيمة المطلقة
print(max(3, 7, 2))   # 7    ← أكبر قيمة
print(min(3, 7, 2))   # 2    ← أصغر قيمة
print(pow(2, 3))      # 8    ← نفس 2**3
print(sum([1, 2, 3])) # 6    ← مجموع عناصر (سنتعمق في القوائم لاحقًا)
```

📐 وحدة `math`: دوال رياضية متقدمة

لدوال أكثر تخصصًا، نحتاج استيراد وحدة `math` المُدمجة:

```python
import math

print(math.sqrt(16))      # 4.0   ← الجذر التربيعي
print(math.ceil(4.1))     # 5     ← التقريب للأعلى دائمًا
print(math.floor(4.9))    # 4     ← التقريب للأسفل دائمًا
print(math.pi)            # 3.141592653589793
print(math.factorial(5))  # 120   ← المضروب (!5)
```

> **أول تعرّف على `import`** : هذا أول استخدام لنا لمفهوم **الاستيراد (import)** في الكتاب - طريقة Python لاستخدام كود إضافي مُنظَّم في «وحدات» (modules) منفصلة. سنتعمّق في هذا المفهوم بالتفصيل لاحقًا، لكن تذكّر الآن: أي وظيفة متقدمة غير متوفرة مباشرة، غالبًا موجودة في وحدة تحتاج استيرادها أولاً.

مقارنة: `round()` مقابل `math.floor()` مقابل `math.ceil()`

```python
number: float = 4.5

print(round(number))        # 4  (تقريب مصرفي للأقرب زوجي)
print(math.floor(number))   # 4  (دائمًا للأسفل)
print(math.ceil(number))    # 5  (دائمًا للأعلى)
```

5.8 تحويلات الأنواع الرقمية بعمق

int ↔ float

```python
# من int إلى float
x: int = 5
y: float = float(x)
print(y)  # 5.0

# من float إلى int (تحذير: يقطع الجزء العشري، لا يُقرِّب!)
a: float = 9.99
b: int = int(a)
print(b)  # 9  ← القطع وليس التقريب !10 ليس
```

> **تحذير حرج جدًا:** تحويل `float` إلى `int` باستخدام `int()` **يقطع** الجزء العشري ببساطة (Truncation)، ولا يُقرِّب أبدًا:

```python
print(int(9.99))   # 9  (10 ليس)
print(int(-9.99))  # -9 (10- ليس)
```

إذا أردت التقريب الفعلي قبل التحويل، استخدم `round()` أولاً:

```python
print(int(round(9.99)))  # 10  ← صحيح الآن
```

5.9 العمليات المختصرة (Augmented Assignment Operators)

✍️ الصيغة المختصرة لتحديث متغير

بدلاً من كتابة:

```python
counter: int = 10
counter = counter + 5
```

Python توفر صيغة مختصرة وأنيقة:

```python
counter: int = 10
counter += 5  # نفس تمامًا counter = counter + 5
print(counter)  # 15
```

📊 جدول العمليات المختصرة الكاملة

الصيغة المختصرة	تُكافئ
`x += 5`	`x = x + 5`
`x -= 5`	`x = x - 5`
`x *= 5`	`x = x * 5`
`x /= 5`	`x = x / 5`
`x //= 5`	`x = x // 5`
`x %= 5`	`x = x % 5`
`x **= 5`	`x = x ** 5`

```python
score: int = 0
score += 10  # score أصبحت 10
score += 5   # score أصبحت 15
score *= 2   # score أصبحت 30
print(score)  # 30
```

✅ **أفضل الممارسات**: استخدم الصيغ المختصرة دائمًا عند تحديث قيمة متغير بناءً على قيمته الحالية - أقصر، أوضح نية، وهي الأسلوب القياسي المتوقع من أي مبرمج Python محترف.

5.10 مقارنة الأرقام: عوامل المقارنة

سنتعمّق في استخدامها الكامل في الفصل السادس (التحكم بسير البرنامج)، لكن من الضروري التعرّف عليها الآن:

```python
print(5 > 3)    # True
print(5 < 3)    # False
print(5 == 5)   # True
print(5 ≠ 3)    # True
print(5 ⩾ 5)    # True
print(5 ⩽ 4)    # False
```

⚠️ تحذير: مقارنة الأعداد العشرية مباشرة

```python
result: float = 0.1 + 0.2
print(result == 0.3)  # False!  ← بسبب فخ الدقة العشرية الذي شرحناه
```

الحل الصحيح لمقارنة أعداد عشرية:

```python
result: float = 0.1 + 0.2
is_close_enough: bool = abs(result - 0.3) < 0.0001
print(is_close_enough)  # True
```

> **نصيحة متقدمة:** في المشاريع الاحترافية، يُستخدم غالبًا `math.isclose()` الجاهزة بدل كتابة هذا المنطق يدويًا:

```python
import math
print(math.isclose(0.1 + 0.2, 0.3))  # True
```

5.11 الأعداد الكبيرة جدًا: ميزة فريدة في Python

🔢 لا حدود عملية لحجم الأعداد الصحيحة

على عكس لغات مثل C++ أو Java حيث الأعداد الصحيحة لها حد أقصى ثابت (overflow)، Python تتعامل مع الأعداد الكبيرة جدًا تلقائيًا دون أي إعداد إضافي:

```python
huge: int = 2 ** 1000
print(huge)  # يعمل بشكل طبيعي تمامًا - رقم ضخم جدًا بمئات الخانات!
```

✍️ الفواصل في الأرقام الكبيرة (Underscore Separators)

لتحسين قراءة الأرقام الكبيرة في كودك، Python تسمح باستخدام شرطة سفلية `_` كفاصل بصري (يُتجاهَل فعليًا من Python):

```python
population: int = 1_000_000_000  # مليار - أوضح بكثير من 1000000000
print(population)  # 1000000000  (الشرطات تُحذَف تلقائيًا)
```

> **نصيحة عملية:** استخدم هذه الميزة دائمًا مع الأرقام الكبيرة (أكثر من 4 خانات) لتحسين قابلية القراءة - فرق كبير بين `1000000` و `1_000_000` من حيث سهولة الفهم الفورية.

5.12 الأعداد العشوائية: مقدمة سريعة لوحدة `random`

لن نتعمق هنا، لكن من المفيد التعرّف على هذه الأداة المُستخدَمة بكثرة لاحقًا في المشاريع (الألعاب، المحاكاة):

```python
import random

random_number: int = random.randint(1, 10)  # رقم عشوائي بين 1 و10 (شاملاً الطرفين)
print(random_number)

random_float: float = random.random()  # رقم عشوائي بين 0 و1
print(random_float)
```

🎯 سنستخدم هذه الوحدة بكثرة في مشاريع لاحقة، خصوصًا في مشروع «لعبة تخمين الأرقام» في الفصل السابع.

5.13 أخطاء شائعة ونصائح احترافية

⚠️ الخطأ 1: القسمة على صفر

```python
result = 10 / 0  # ❌ ZeroDivisionError: division by zero
```

سنتعلّم كيف «نمسك» هذا الخطأ بأناقة في فصل معالجة الأخطاء (الفصل الحادي عشر). الآن، فقط تذكّر أن القسمة على صفر غير معرَّفة رياضيًا، وPython ترفضها بخطأ واضح بدل نتيجة غير منطقية.

⚠️ الخطأ 2: نسيان أن `/` تُعيد float دائمًا

```python
total_items: int = 10
groups: int = 3
items_per_group = total_items / groups  # 3.333... وليس 3

# إذا أردت عددًا صحيحًا من المجموعات:
items_per_group = total_items // groups  # 3  ← استخدم القسمة الصحيحة
```

⚠️ الخطأ 3: الثقة العمياء بنتائج float في المقارنات

```python
# كود خاطئ محتمل
if (0.1 + 0.2) == 0.3:
    print("متساويان")
else:
    print("غير متساويين")  # هذا ما سيُطبع فعليًا!
```

✅ نصيحة احترافية: استخدام Type Hints مع الأرقام

```python
def calculate_total(price: float, quantity: int) -> float:
    return round(price * quantity, 2)
```

> 💡 لا تقلق إن لم تفهم صيغة الدالة بالكامل الآن - سنتعمق فيها في الفصل العاشر. لاحظ فقط كيف توضح Type Hints بدقة متوقع كل نوع بيانات.

5.14 جدول مرجعي شامل للفصل

العملية/الدالة	المثال	النتيجة
جمع/طرح/ضرب	`5 + 3`, `5 - 3`, `5 * 3`	`8`, `2`, `15`
قسمة عادية	`10 / 3`	`3.333...`
قسمة صحيحة	`10 // 3`	`3`
باقي القسمة	`10 % 3`	`1`
أُس	`2 ** 3`	`8`
تقريب	`round(3.14159, 2)`	`3.14`
قيمة مطلقة	`abs(-5)`	`5`
أكبر/أصغر قيمة	`max(1,5,3)`, `min(1,5,3)`	`5`, `1`
جذر تربيعي	`math.sqrt(16)`	`4.0`

العملية/الدالة	المثال	النتيجة
تقريب لأعلى/أسفل	`math.ceil(4.1)` , `math.floor(4.9)`	`5` , `4`
رقم عشوائي	`random.randint(1,10)`	رقم بين 1-10
حسابات مالية دقيقة	`Decimal("19.99")`	دقة تامة

5.15 تمارين عملية للفصل

🟢 تمرين 1 (مستوى مبتدئ)

اكتب برنامجًا يطلب من المستخدم رقمين، ثم يطبع نتائج كل العمليات الحسابية الأساسية بينهما (جمع، طرح، ضرب، قسمة، قسمة صحيحة، باقي القسمة، أُس) بتنسيق واضح ومرتّب.

🟡 تمرين 2 (مستوى متوسط)

اكتب برنامج «حاسبة فاتورة مطعم»: يطلب سعر الوجبة، عدد الأشخاص، ونسبة الإكرامية (Tip) كنسبة مئوية، ثم يحسب: نصيب كل شخص من السعر الأساسي، قيمة الإكرامية الإجمالية، والمبلغ الإجمالي شاملاً الإكرامية. استخدم `round()` بشكل صحيح لعرض نتائج مالية منطقية.

🔴 تمرين 3 (تحدي)

اكتب برنامجًا يحوّل درجة حرارة من فهرنهايت إلى مئوية والعكس (حسب اختيار المستخدم)، باستخدام الصيغ الرياضية الصحيحة:

- `celsius = (fahrenheit - 32) * 5/9`
- `fahrenheit = celsius * 9/5 + 32`

تأكد من تقريب النتيجة النهائية لخانة عشرية واحدة فقط.

5.16 مشروع تطبيقي: «الحاسبة العلمية التفاعلية»

📋 المهمة

أنشئ ملفًا باسم `scientific_calculator.py` يبني حاسبة متكاملة من الطرفية:

المتطلبات:

1. طلب رقمين من المستخدم
2. طلب العملية المطلوبة (كنص: "+"، "-"، "*"، "/"، "//"، "%"، "**")
3. تنفيذ العملية الصحيحة بناءً على اختيار المستخدم
4. معالجة حالة القسمة على صفر بشكل أنيق (رسالة واضحة بدل انهيار البرنامج - استخدام مبدئي بسيط، سنتعمّق في `try/except` لاحقًا)
5. عرض النتيجة بتنسيق احترافي، مع تقريب مناسب إن كانت النتيجة عشرية

💻 حل نموذجي كامل

✅ اضغط لرؤية الحل الكامل (حاول بنفسك أولاً!)

Python scientific_calculator.py

```python
"""حاسبة علمية تفاعلية بسيطة من سطر الأوامر."""

print("=" * 40)
print("🧮 الحاسبة العلمية التفاعلية")
print("=" * 40)

# استقبال المدخلات
num1: float = float(input("أدخل الرقم الأول: "))
operator: str = input("أدخل العملية (+ - * /// % **): ").strip()
num2: float = float(input("أدخل الرقم الثاني: "))

# معالجة حالة خاصة: القسمة على صفر
if operator in ("/", "//", "%") and num2 == 0:
    print("\n❌ خطأ: لا يمكن القسمة على صفر!")
else:
    result: float = 0.0
    valid_operation: bool = True

    if operator == "+":
        result = num1 + num2
    elif operator == "-":
        result = num1 - num2
    elif operator == "*":
        result = num1 * num2
    elif operator == "/":
        result = num1 / num2
    elif operator == "//":
        result = num1 // num2
    elif operator == "%":
        result = num1 % num2
    elif operator == "**":
        result = num1 ** num2
    else:
        valid_operation = False
        print(f"\n❌ خطأ: العملية '{operator}' غير معروفة!")

    if valid_operation:
        print("\n" + "=" * 40)
        print(f"📊 النتيجة: {num1} {operator} {num2} = {round(result, 4)}")
        print("=" * 40)
```

ملاحظة على الحل: استخدمنا هنا `if/elif` بشكل مبدئي (سنتعمّق فيها بالكامل في الفصل القادم مباشرة)، وهذا مثال تطبيقي ممتاز يُظهر كيف أن المفاهيم في البرمجة مترابطة ومتراكمة - الأرقام التي تعلمناها هنا ستُستخدم فعليًا بمجرد أن نتعلم أدوات اتخاذ القرار في الفصل التالي.

5.17 اختبر فهمك (Quiz سريع)

1. لماذا تُعيد `/` دائمًا نوع `float` حتى لو كانت القسمة «نظيفة» بدون باقي؟
2. ما الفرق بين `//` و `%`؟ وما الذي يُعيده كل منهما؟
3. لماذا تظهر `0.1 + 0.2` كـ `0.30000000000000004` بدلاً من `0.3`؟
4. متى يجب استخدام `Decimal` بدلاً من `float` العادي؟
5. ما الفرق بين `int(9.9)` و `round(9.9)`؟

<details>
<summary>✅ الإجابات النموذجية</summary>

1. هذا قرار تصميمي مقصود في Python 3 لتجنّب فقدان الدقة العشرية بشكل غير متوقع، على عكس بعض اللغات الأخرى التي تُعيد عددًا صحيحًا عند قسمة عددين صحيحين.
2. `//` تُعيد ناتج القسمة الصحيح (مُقرَّب للأسفل، بدون كسور)، بينما `%` تُعيد الباقي المتبقي بعد تلك القسمة.
3. بسبب طريقة تخزين الكمبيوتر للأعداد العشرية في النظام الثنائي، حيث بعض الكسور العشرية البسيطة في نظامنا العشري لا يمكن تمثيلها بدقة تامة في الثنائي.
4. عند التعامل مع حسابات مالية حقيقية (أسعار، فواتير، رواتب) تتطلب دقة تامة بدون أي أخطاء تقريب تراكمية.
5. `int(9.9)` يقطع الجزء العشري فورًا دون تقريب فينتج `9`، بينما `round(9.9)` يُقرِّب فعليًا للأقرب فينتج `10`.

</details>

5.18 📌 ملخص الفصل (Cheat Sheet)

```python
# العمليات الأساسية
a + b, a - b, a * b, a / b
a // b   # قسمة صحيحة
a % b    # باقي القسمة
a ** b   # أُس

# التقريب والدقة
round(3.14159, 2)     # 3.14
from decimal import Decimal
Decimal("19.99")      # دقة مالية تامة

# دوال مُدمجة
abs(-5), max(1,5,3), min(1,5,3)

# وحدة math
import math
math.sqrt(16), math.ceil(4.1), math.floor(4.9), math.pi

# عمليات مختصرة
x += 5   # x = x + 5
x *= 2   # x = x * 2

# فواصل الأرقام الكبيرة
population = 1_000_000

# مقارنة آمنة للأعداد العشرية
import math
math.isclose(0.1 + 0.2, 0.3)  # True
```

⚠️ تذكّر دائمًا

- `/` تُعيد **float** دائمًا، `//` تُعيد القسمة الصحيحة
- لا تقارن أعداد float بـ `==` مباشرة أبدًا
- استخدم `Decimal` للحسابات المالية الحقيقية
- `int()` يقطع الكسور، `round()` يُقرِّبها فعليًا

🎯 ماذا بعد؟

تعلمنا حتى الآن كيف نُخزّن البيانات (متغيرات)، ونتعامل مع النصوص والأرقام. لكن كل برامجنا حتى الآن «تسير في خط مستقيم» - تُنفَّذ من الأعلى للأسفل دون أي قرارات. في **الفصل السادس**، سندخل عالمًا جديدًا تمامًا: **اتخاذ القرار (Control Flow)** - كيف نجعل البرنامج «يفكر» ويتخذ مسارات مختلفة بناءً على الشروط، شاملاً `if/elif/else`

التقليدية، وأسلوب `match-case` الحديث الثوري في Python.

فصل 06 / 26 · الجزء الثاني

06

اتخاذ القرار (Control Flow)

9 دقيقة قراءة | 17 قسمًا | 35 مثالًا | 1,760 كلمة

في هذا الفصل

6.1 مقدمة: من التنفيذ الخطي إلى اتخاذ القرار
6.2 القيم المنطقية في صميم اتخاذ القرار
6.3 العبارة الشرطية الأساسية: `if`
6.4 التفريع: `if...else`
6.5 شروط متعددة: `if...elif...else`
6.6 العوامل المنطقية: `and`, `or`, `not`
6.7 القيم «الصادقة» و«الكاذبة» ضمنيًا (Truthy & Falsy)
6.8 الجُمل الشرطية المُدمجة: Ternary Expression
6.9 match-case: الأسلوب الحديث الثوري (Python 3.10+)
6.10 Walrus Operator `:=` - الأداة الذكية (Python 3.8+)
6.11 الشروط المتداخلة (Nested Conditions)
6.12 أخطاء شائعة ونصائح احترافية
6.13 جدول مرجعي شامل للفصل
6.14 تمارين عملية للفصل
6.15 مشروع تطبيقي: «نظام تقييم كلمة المرور الذكي»
6.16 اختبر فهمك (Quiz سريع)
6.17 ملخص الفصل (Cheat Sheet)

«حتى الآن، برامجك كانت 'مطيعة' تمامًا - تُنفِّذ كل سطر بالترتيب دون تفكير. في هذا الفصل، ستتعلّم كيف تجعل برنامجك 'يُفكّر' ويختار مسارات مختلفة بناءً على الظروف - هذا هو جوهر الذكاء في البرمجة.»

6.1 📖 مقدمة: من التنفيذ الخطي إلى اتخاذ القرار

كل البرامج التي كتبتها حتى الآن تشترك في صفة واحدة: **تنفيذ خطي (Linear Execution)** - تبدأ من السطر الأول، وتنتهي بالسطر الأخير، بدون أي انحراف. لكن البرامج الحقيقية ليست كذلك أبدًا. فكّر في أي تطبيق تستخدمه يوميًا: تطبيق البنك «يقرر» إظهار رسالة خطأ إن أدخلت كلمة مرور خاطئة، نظام الملاحة «يقرر» مسارًا بديلاً إن وُجد ازدحام. هذا «القرار» هو ما سنتعلمه الآن.

6.2 القيم المنطقية في صميم اتخاذ القرار

🔄 مراجعة سريعة: كل قرار يعتمد على `True` أو `False`

تذكر من الفصل الثالث: `bool` لها قيمتان فقط. كل جملة شرطية في Python تعتمد جوهريًا على تقييم تعبير ما إلى `True` أو `False`:

```python
age: int = 20
print(age ⩾ 18)  # True  ← هذا هو أساس كل قرار برمجي
```

📊 عوامل المقارنة الكاملة (مراجعة وتوسيع)

العامل	المعنى	مثال	النتيجة
`==`	يساوي	`5 == 5`	`True`
`≠`	لا يساوي	`5 ≠ 3`	`True`
`>`	أكبر من	`5 > 3`	`True`
`<`	أصغر من	`5 < 3`	`False`
`⩾`	أكبر أو يساوي	`5 ⩾ 5`	`True`
`⩽`	أصغر أو يساوي	`5 ⩽ 4`	`False`

6.3 العبارة الشرطية الأساسية: `if`

✍️ الصيغة الأساسية

```python
if condition:
    # الكود الذي يُنفَّذ فقط إذا كان الشرط True
```

```python
age: int = 20

if age ⩾ 18:
    print("مسموح لك بالدخول")
```

⚠️ تحذير حرج: المسافة البادئة (Indentation) ليست اختيارية في Python!

هذا من أهم الفروق الجوهرية بين Python ولغات أخرى كثيرة. في لغات مثل Java أو C++، المسافات البادئة للتنسيق فقط (تجميلية). في Python، هي **جزء من قواعد اللغة نفسها**:

```python
# ✅ صحيح - المسافة البادئة متّسقة (4 مسافات هي المعيار العالمي)
if age ⩾ 18:
    print("مسموح")
    print("أهلاً بك")

# ❌ خطأ - IndentationError
if age ⩾ 18:
print("مسموح")  # لا توجد مسافة بادئة!
```

```
IndentationError: expected an indented block after 'if' statement on line 1
```

🎯 **قاعدة ذهبية**: استخدم دائمًا **4 مسافات** (وليس Tab) لكل مستوى من المسافة البادئة - هذا هو المعيار الرسمي المُوصى به في PEP 8. VS Code يفعل هذا تلقائيًا عند ضغط مفتاح Tab (يُترجمه لـ4 مسافات تلقائيًا بالإعدادات الافتراضية لإضافة Python).

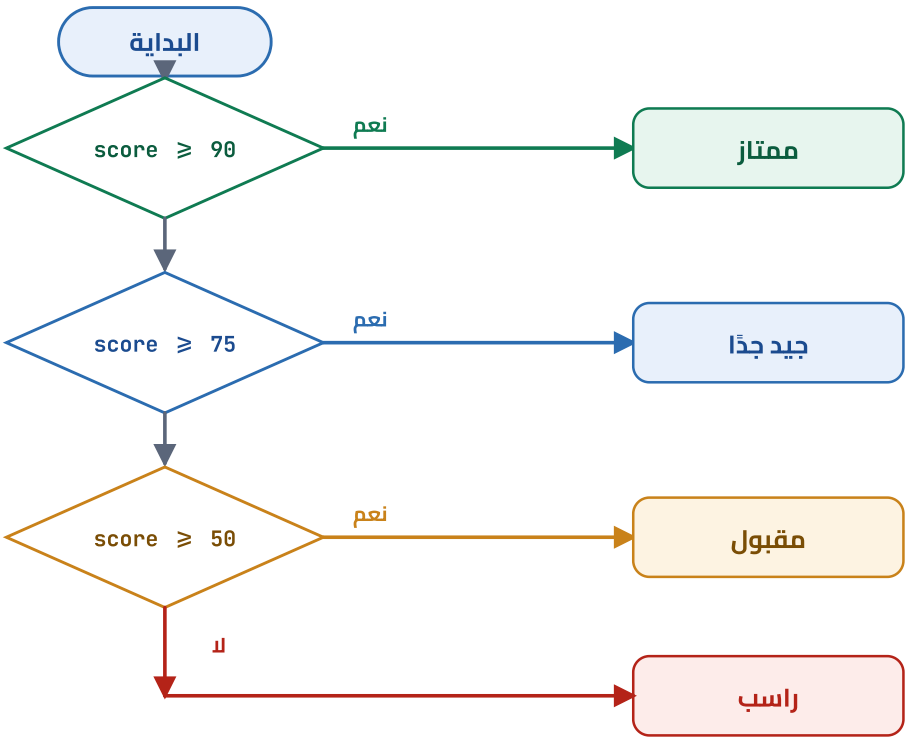
⚠️ **تحذير من خطأ خفي خطير**: لا تخلط بين المسافات (Spaces) ومفتاح Tab في نفس الملف! هذا يُسبب أخطاء محيّرة جدًا تبدو «صحيحة بصريًا» لكن Python ترفضها. VS Code يُظهر هذه المشاكل تلقائيًا، لكن انتبه جيدًا إن نسخت كودًا من مصادر مختلفة.

6.4 التفريع: `if...else`

```python
age: int = 15

if age ⩾ 18:
    print("مسموح لك بالدخول")
else:
    print("عذرًا، يجب أن تكون 18 سنة أو أكثر")
```

الفهم الجوهري: `else` تعني «في أي حالة أخرى غير الشرط أعلاه» - لا تحتاج شرطًا خاصًا بها، فهي «كل ما تبقى».

6.5 شروط متعددة: `if...elif...else`



يُفحص الشرط من الأعلى للأسفل، ويُنفَّذ أول فرع يتحقّق فقط.

شكل 6.1 — تدفّق if / elif / else

```python
score: int = 75

if score ⩾ 90:
    grade: str = "ممتاز"
elif score ⩾ 80:
    grade = "جيد جدًا"
elif score ⩾ 70:
    grade = "جيد"
elif score ⩾ 60:
    grade = "مقبول"
else:
    grade = "راسب"

print(f"تقديرك: {grade}")  # تقديرك: جيد
```

🔬 كيف تُقيَّم `elif` فعليًا؟ (فهم دقيق مهم)

Python تفحص الشروط **بالترتيب من الأعلى للأسفل**، وبمجرد أن يكون أحدها `True` ، **تتوقف فورًا** عن فحص الباقي (حتى لو كانت شروط لاحقة صحيحة أيضًا):

```python
score: int = 95

if score ⩾ 90:
    print("ممتاز")      # هذا يُطبع ويتوقف هنا
elif score ⩾ 80:
    print("جيد جدًا")   # أيضًا صحيح ⩽ 80 لن يُفحص حتى، رغم أن 95!
```

⚠️ **خطأ شائع جدًا بسبب هذا المفهوم:** ترتيب الشروط **مهم جدًا**. إذا كتبت الشروط بترتيب خاطئ، ستحصل على نتائج غير منطقية:

```python
# ❌ ترتيب خاطئ - منطق معكوس
score: int = 95

if score ⩾ 60:
    print("مقبول")   # هذا سيُطبع لـ 95! لأنه أول شرط صحيح يُقابَل
elif score ⩾ 90:
    print("ممتاز")   # لن يصل هنا أبدًا لأي درجة أعلى من 60!
```

✅ **القاعدة الذهبية:** عند استخدام نطاقات متدرجة، رتّب الشروط دائمًا من **الأكثر تحديدًا (تشددًا) إلى الأقل**.

6.6 العوامل المنطقية: `and` ، `or` ، `not`

أحيانًا تحتاج لدمج أكثر من شرط واحد معًا.

`and` : كلا الشرطين يجب أن يكونا صحيحين

```python
age: int = 25
has_license: bool = True

if age >= 18 and has_license:
    print("يمكنك القيادة")
```

`or` : شرط واحد على الأقل يكفي

```python
is_weekend: bool = False
is_holiday: bool = True

if is_weekend or is_holiday:
    print("يوم إجازة!")
```

`not` : عكس القيمة المنطقية

```python
is_raining: bool = False

if not is_raining:
    print("يمكننا الخروج للنزهة")
```

📊 جدول الحقيقة المبسّط (Truth Table)

A	B	A and B	A or B	not A
True	True	True	True	False
True	False	False	True	False
False	True	False	True	True
False	False	False	False	True

🎯 دمج عوامل متعددة مع الأقواس للوضوح

```python
age: int = 25
has_ticket: bool = True
is_vip: bool = False

if (age >= 18 and has_ticket) or is_vip:
    print("مسموح بالدخول")
```

✅ **أفضل الممارسات:** عند دمج `and` و `or` معًا في نفس الشرط، استخدم الأقواس دائمًا لتوضيح الأولوية - حتى لو كانت النتيجة نفسها بدون أقواس، فالوضوح للقارئ أهم بكثير.

⚠️ تحذير: Short-Circuit Evaluation (التقييم القصير)

Python «تتوقف مبكرًا» عن تقييم الشرط إن أمكنها معرفة النتيجة النهائية مسبقًا:

```python
def expensive_check() -> bool:
    print("تم استدعاء الفحص المكلف!")
    return True

# مع and: إذا كان الشرط الأول False، لن يُستدعى الثاني إطلاقًا
if False and expensive_check():
    pass
# لن تُطبع "تم استدعاء الفحص المكلف!" أبدًا!

# مع or: إذا كان الشرط الأول True، لن يُستدعى الثاني إطلاقًا
if True or expensive_check():
    pass
# لن تُطبع "تم استدعاء الفحص المكلف!" أبدًا!
```

> 💡 **فائدة عملية مهمة**: هذا السلوك يُستخدم بذكاء لتجنّب أخطاء محتملة:

```python
user_data: dict = {}  # قاموس فارغ (سنتعلمه بالفصل التاسع)

if "name" in user_data and user_data["name"] != "":
    print("الاسم موجود وغير فارغ")
# تتوقف عند الشرط الأول، and إذا لم يكن "name" موجودًا أصلاً
# وتتجنب خطأ محاولة الوصول لمفتاح غير موجود
```

6.7 القيم «الصادقة» و«الكاذبة» ضمنيًا (Truthy & Falsy)

🎭 مفهوم فريد ومهم جدًا في Python

في Python، ليس فقط `True` و `False` يُستخدمان في الشروط - أي قيمة يمكن تقييمها ضمنيًا كـ«صادقة» (Truthy) أو «كاذبة» (Falsy):

```python
name: str = ""

if name:
    print("الاسم موجود")
else:
    print("الاسم فارغ")  # هذا سيُطبع! لأن النص الفارغ يُعتبر Falsy
```

📊 القيم التي تُعتبر Falsy (كل ما عداها يُعتبر Truthy)

القيمة	النوع
`False`	bool
`0` , `0.0`	أرقام
`""` (نص فارغ)	str
`None`	NoneType
`[]` (قائمة فارغة)	list - سنتعلمها لاحقًا
`{}` (قاموس فارغ)	dict - سنتعلمه لاحقًا

Python

```python
value: int = 0

if value:
    print("له قيمة")
else:
    print("فارغ أو صفر")  # هذا سيُطبع
```

✅ **استخدام احترافي شائع جدًا:** التحقق من وجود قيمة مُدخلة من المستخدم:

Python

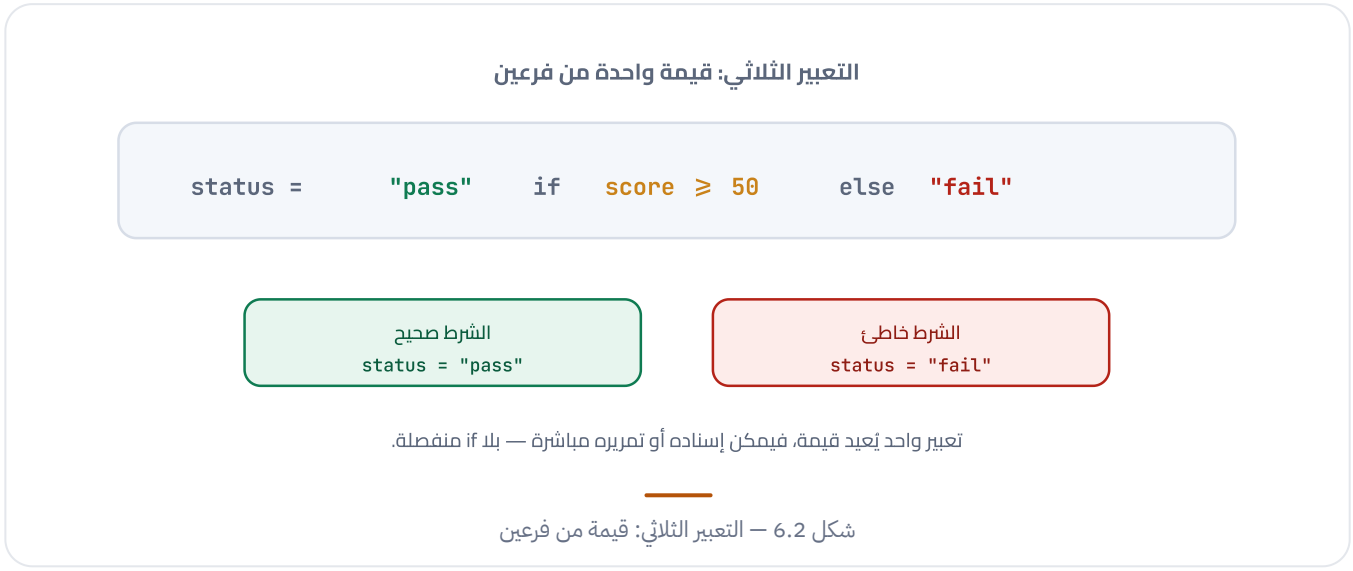
```python
name: str = input("(اختياري) أدخل اسمك: ").strip()

if name:
    print(f"مرحبًا {name}")
else:
    print("مرحبًا أيها الزائر المجهول")
```

⚠️ **تحذير دقيق:** لا تخلط بين `if value:` (فحص Truthy/Falsy) و `if value == True:` (مقارنة صريحة).
الأولى أكثر Pythonic (أسلوب Python الأصيل) والثانية أحيانًا تُسبب سلوكًا غير متوقع:

Python

```python
value = 5
print(value == True)  # False!  لأن 5 لا يساوي True رغم أنه Truthy حرفيًا
print(bool(value))    # True   لأن 5 يُقيَّم كـ Truthy
```

6.8 الجُمل الشرطية المُدمجة: Ternary Expression



شكل 6.2 — التعبير الثلاثي: قيمة من فرعين

✍️ صيغة مختصرة لـ if/else في سطر واحد

```python
age: int = 20

# الطريقة التقليدية
if age ⩾ 18:
    status = "بالغ"
else:
    status = "قاصر"

# الطريقة المختصرة (Ternary / Conditional Expression)
status = "بالغ" if age ⩾ 18 else "قاصر"
```

كلاهما يُعطي نفس النتيجة بالضبط، لكن الصيغة الثانية أقصر وأحيانًا أوضح للحالات البسيطة.

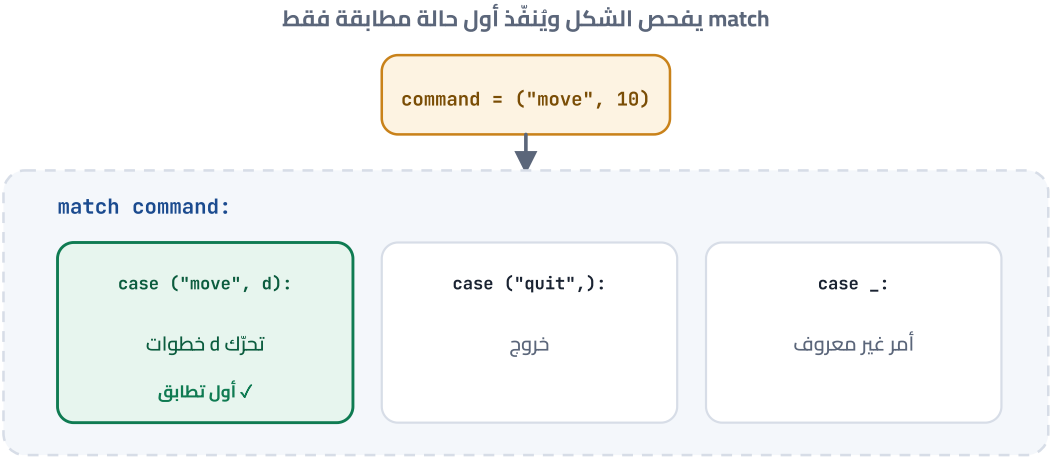
✅ متى تستخدم كل أسلوب؟

الحالة	الأسلوب المُفضَّل
إسناد قيمة بسيطة بناءً على شرط واحد	Ternary (مختصرة)
تنفيذ عدة أسطر أو منطق معقد	if/else التقليدية
أكثر من شرط (elif متعددة)	if/elif/else التقليدية فقط

```python
# ✅ استخدام ternary جيد للـ - بسيط وواضح
discount = 0.2 if is_vip else 0.0

# ❌ استخدام ternary سيء - معقد جدًا وغير مقروء كـ
result = "A" if x > 90 else "B" if x > 80 else "C" if x > 70 else "D"
# استخدم if/elif/else في هذه الحالة، هذا بدلاً من هذا التقليدية!
```

---

match-case 6.9 : الأسلوب الحديث الثوري (Python 3.10+)



شكل 6.3 — match-case: أول حالة مطابقة

🆕 لماذا هذه الميزة مهمة جدًا؟

حتى إصدار Python 3.9، كانت اللغة تفتقر لأداة «تبديل» (Switch) الموجودة في لغات أخرى كثيرة (Java, C++, JavaScript). بدءًا من Python 3.10، ظهرت أداة `match-case` - لكنها أقوى بكثير من أي `switch` تقليدي، لأنها تدعم ما يُسمى **Pattern Matching** (مطابقة الأنماط).

✍️ الاستخدام الأساسي

```python
day: str = "Monday"

match day:
    case "Monday":
        print("بداية الأسبوع")
    case "Friday":
        print("نهاية أسبوع العمل")
    case "Saturday" | "Sunday":  # هنا "أو" يعني | عامل
        print("عطلة نهاية الأسبوع")
    case _:
        print("يوم عادي")
```

💡 **ملاحظة مهمة**: الرمز `_` في `case _:` يعني «أي قيمة أخرى» (مشابه لـ `else` في if/elif/else) - يجب أن يكون دائمًا **آخر** حالة في match.

🔄 متى تستخدم `match-case` بدلاً من `if/elif`؟

```python
# سيناريو مثالي لـ match-case: فحص قيمة واحدة ضد عدة احتمالات محددة
http_status: int = 404

match http_status:
    case 200:
        print("نجاح")
    case 404:
        print("غير موجود")
    case 500:
        print("خطأ في الخادم")
    case _:
        print("حالة غير معروفة")
```

🚀 قوة إضافية: مطابقة الأنماط المتقدمة (Pattern Matching الحقيقي)

هنا تكمن القوة الحقيقية لـ `match-case` - أبعد بكثير من مجرد مقارنة قيم بسيطة:

Python

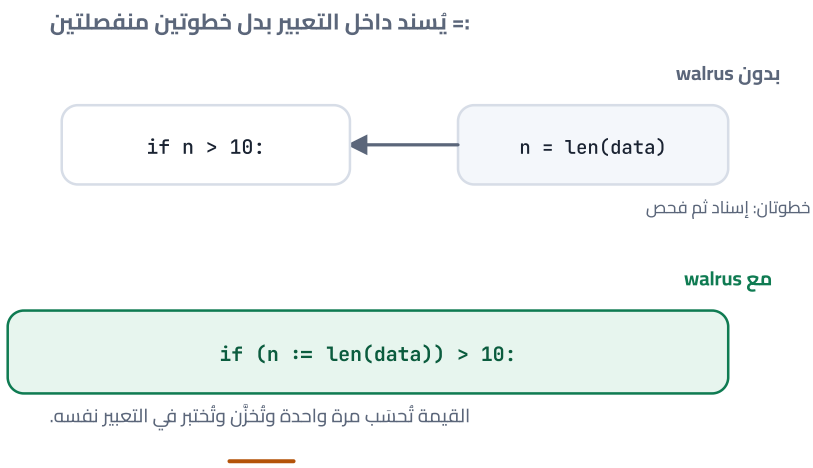
```python
# مطابقة أنواع البيانات والبنى (سنفهم هذا بعمق أكبر لاحقًا مع القوائم)
def describe_value(value):
    match value:
        case 0:
            return "صفر"
        case int() if value > 0:
            return "رقم صحيح موجب"
        case int() if value < 0:
            return "رقم صحيح سالب"
        case str():
            return "نص"
        case _:
            return "نوع غير معروف"

print(describe_value(5))        # رقم صحيح موجب
print(describe_value(-3))       # رقم صحيح سالب
print(describe_value("hi"))     # نص
```

🎯 لا تقلق إن بدت الأمثلة المتقدمة معقدة الآن - هذا طبيعي تمامًا. سنعود لـ `match-case` بعمق أكبر لاحقًا بعد تعلم القوائم والكائنات، حيث ستظهر قوتها الحقيقية بوضوح أكبر.

🔄 مقارنة شاملة: متى if/elif ومتى match-case؟

الحالة	الأسلوب المُفضَّل
فحص نطاقات (مثل `score ⩾ 90`)	✅ `if/elif/else`
فحص قيمة واحدة ضد عدة قيم محددة	✅ `match-case`
شروط منطقية معقدة (`and` / `or` متعددة)	✅ `if/elif/else`
مطابقة أنواع بيانات أو بنى معقدة	✅ `match-case`

6.10 Walrus Operator - الأداة الذكية (Python 3.8+) :=



شكل 6.4 — walrus: إسناد داخل التعبير

🤔 المشكلة التي يحلها

أحيانًا تحتاج لحساب قيمة واستخدامها في شرط **ونفس الوقت** تخزينها لاستخدام لاحق، مما يُسبب تكرارًا غير ضروري:

```python
# بدون walrus operator - تكرار الحساب مرتين
import random
number = random.randint(1, 10)

if number > 5:
    print(f"الرقم {number} أكبر من 5")
```

✍️ الحل بـ Walrus Operator

```python
import random

if (number := random.randint(1, 10)) > 5:
    print(f"الرقم {number} أكبر من 5")
```

العامل `:=` يُسند القيمة للمتغير **وفي نفس الوقت** يُعيد تلك القيمة لاستخدامها مباشرة في الشرط - كل هذا في سطر واحد.

متى تستخدمه فعليًا؟ هذا العامل اختياري بالكامل، ويُستخدم بكثرة أكثر في حلقات `while` (سنراها في الفصل القادم) وفي list comprehensions (الفصل الثامن). لا تُفرط باستخدامه في الشروط البسيطة إن كان يُقلّل الوضوح.

⚠ **تحذير من الإفراط**: لا تستخدم `:=` فقط «لأنها أداة جديدة ومثيرة». استخدمها فقط عندما تُحسّن **الوضوح فعليًا** وتتجنب تكرار كود حقيقي.

6.11 الشروط المتداخلة (Nested Conditions)

```python
age: int = 25
has_id: bool = True

if age ⩾ 18:
    if has_id:
        print("يمكنك الدخول")
    else:
        print("تحتاج إثبات هوية")
else:
    print("عمرك غير كافٍ")
```

⚠ **تحذير احترافي**: الشروط المتداخلة بعمق (أكثر من 2-3 مستويات) تُصعّب قراءة الكود بشكل كبير. غالبًا يمكن تبسيطها بدمج الشروط بـ `and` :

```python
# بدلاً من التداخل العميق
if age ⩾ 18:
    if has_id:
        if not is_banned:
            print("مسموح")

# الأفضل: دمج الشروط
if age ⩾ 18 and has_id and not is_banned:
    print("مسموح")
```

✅ تقنية احترافية: Early Return / Guard Clauses

تقنية متقدمة (سنتعمق فيها أكثر مع الدوال في الفصل العاشر) تُجنّبك التداخل العميق:

```python
# بدلاً من هذا (متداخل)
def check_access(age, has_id):
    if age ⩾ 18:
        if has_id:
            return "مسموح"
        else:
            return "تحتاج هوية"
    else:
        return "عمر غير كافٍ"

# استخدم هذا (أوضح - Guard Clauses)
def check_access(age, has_id):
    if age < 18:
        return "عمر غير كافٍ"
    if not has_id:
        return "تحتاج هوية"
    return "مسموح"
```

لا تقلق إن لم تفهم صيغة الدالة بالكامل بعد - النقطة المهمة هنا هي **مبدأ** تقليل التداخل، وسنعود لهذا بعمق في فصل الدوال.

6.12 أخطاء شائعة ونصائح احترافية

⚠️ الخطأ 1: استخدام `=` بدلاً من `==`

```python
age = 18
if age = 18:  # ❌ SyntaxError!
    print("البالغ")

if age == 18:  # ✅ صحيح
    print("البالغ")
```

⚠️ الخطأ 2: مقارنة Boolean بشكل زائد

```python
is_active: bool = True

# ❌ غير احترافي (زائد عن الحاجة)
if is_active == True:
    print("نشط")

# ✅ احترافي (Pythonic)
if is_active:
    print("نشط")
```

⚠️ الخطأ 3: نسيان `:` بعد الشرط

```python
if age >= 18  # ❌ SyntaxError: expected ':'
    print("بالغ")
```

⚠️ الخطأ 4: خلط المسافات البادئة (مسافات مع Tab)

```python
if age >= 18:
    print("بالغ")  # 4 مسافات
    print("مرحبا")  # Tab! ← سيُسبب IndentationError أو سلوكًا غير متوقع
```

✅ نصيحة احترافية: تجنّب شروط `== True` و`== False`

```python
# ❌ غير احترافي
if is_valid == False:
    print("غير صالح")

# ✅ احترافي
if not is_valid:
    print("غير صالح")
```

6.13 جدول مرجعي شامل للفصل

الأداة	الاستخدام	مثال
`if`	شرط بسيط	`if x > 5:`
`if/else`	شرط مع بديل	`if x > 5: ... else: ...`
`if/elif/else`	شروط متعددة	`if...elif...elif...else`
`and`	كلا الشرطين صحيح	`x > 5 and x < 10`
`or`	أحد الشرطين على الأقل	`x < 5 or x > 10`
`not`	عكس القيمة	`not is_valid`
Ternary	شرط مختصر بسطر واحد	`"A" if x else "B"`
`match-case`	مطابقة قيمة لعدة احتمالات	`match x: case 1: ...`
`:=`	إسناد واستخدام في آن واحد	`if (n := get()) > 5:`

6.14 تمارين عملية للفصل

🟢 تمرين 1 (مستوى مبتدئ)

اكتب برنامجًا يطلب من المستخدم إدخال عمره، ثم يطبع فئته العمرية باستخدام `if/elif/else`:
- أقل من 13: «طفل»
- 13 إلى 19: «مراهق»
- 20 إلى 59: «بالغ»
- 60 فأكثر: «كبير السن»

🟡 تمرين 2 (مستوى متوسط)

اكتب برنامج تسجيل دخول بسيط: يطلب اسم مستخدم وكلمة مرور، ويتحقق من تطابقهما مع قيم محددة مسبقًا في الكود (مثل `"admin"` و `"12345"`). استخدم العوامل المنطقية `and` بشكل صحيح، واطبع رسائل مختلفة حسب الحالة (كلاهما صحيح، اسم المستخدم خطأ فقط، كلمة المرور خطأ فقط، كلاهما خطأ).

🔴 تمرين 3 (تحدي)

أعد كتابة الفصل السادس بالكامل - تمرين «تصنيف المثلث»: اطلب من المستخدم أطوال 3 أضلاع، وحدد نوع المثلث:
- إذا كانت الأضلاع الثلاثة متساوية: «متساوي الأضلاع»
- إذا كان ضلعان متساويين فقط: «متساوي الساقين»
- إذا كانت كلها مختلفة: «مختلف الأضلاع»
- تحقّق أولاً إن كانت الأضلاع تُشكّل مثلثًا صالحًا أصلاً (مجموع أي ضلعين يجب أن يكون أكبر من الضلع الثالث)

6.15 مشروع تطبيقي: «نظام تقييم كلمة المرور الذكي»

📋 المهمة

أنشئ ملفًا باسم `password_checker.py` يُقيّم قوة كلمة مرور أدخلها المستخدم بناءً على معايير متعددة:

المتطلبات:

1. طلب كلمة مرور من المستخدم
2. التحقق من الشروط التالية باستخدام methods النصوص من الفصل الرابع:
   - الطول 8 أحرف على الأقل
   - تحتوي رقمًا واحدًا على الأقل
   - تحتوي حرفًا كبيرًا واحدًا على الأقل (إن كانت إنجليزية)
   - تحتوي رمزًا خاصًا واحدًا على الأقل (مثل `!@#$%`)
3. حساب «نقاط القوة» بناءً على عدد الشروط المُستوفاة
4. تصنيف القوة: ضعيفة (0-1 شروط) / متوسطة (2-3 شروط) / قوية (4 شروط)
5. طباعة تقرير مفصّل يوضح أي الشروط مُستوفاة وأيها ناقصة

💻 حل نموذجي كامل

✅ اضغط لرؤية الحل الكامل (حاول بنفسك أولاً!)

Python password_checker.py

```python
"""نظام تقييم قوة كلمة المرور."""

SPECIAL_CHARS: str = "!@#$%^&*()_+-="

password: str = input("أدخل كلمة مرور لتقييمها: ")

# فحص كل شرط على حدة
has_min_length: bool = len(password) >= 8
has_digit: bool = any(char.isdigit() for char in password)
has_upper: bool = any(char.isupper() for char in password)
has_special: bool = any(char in SPECIAL_CHARS for char in password)

# حساب عدد الشروط المُستوفاة
strength_score: int = sum([has_min_length, has_digit, has_upper, has_special])

# تحديد مستوى القوة
if strength_score <= 1:
    strength_level: str = "ضعيفة ❌"
elif strength_score <= 3:
    strength_level = "متوسطة ⚠️"
else:
    strength_level = "قوية ✅"

# طباعة التقرير
print("\n" + "=" * 40)
print("🔐 تقرير تقييم كلمة المرور")
print("=" * 40)
print(f"الطول (8+ أحرف): {'✅' if has_min_length else '❌'}")
print(f"يحتوي رقمًا: {'✅' if has_digit else '❌'}")
print(f"يحتوي حرفًا كبيرًا: {'✅' if has_upper else '❌'}")
print(f"يحتوي رمزًا خاصًا: {'✅' if has_special else '❌'}")
print("-" * 40)
print(f"النتيجة: {strength_score}/4")
print(f"مستوى القوة: {strength_level}")
print("=" * 40)
```

ملاحظة على الحل: استخدمنا هنا `any()` مع تعبير يُشبه list comprehension (`for char in password`) - هذا تشويق بسيط لمفاهيم ستتعلمها بعمق أكبر في الفصل الثامن (القوائم). لا تقلق إن بدت غامضة الآن؛ لاحظ فقط كيف تتكامل المفاهيم مع تقدمك في الكتاب.

6.16 اختبر فهمك (Quiz سريع)

1. لماذا تُعتبر المسافة البادئة (Indentation) جزءًا من قواعد Python النحوية، وليست مجرد تنسيق جمالي؟
2. ما الفرق بين `and` و `or` من حيث الشروط اللازمة ليكون الناتج `True` ؟
3. ما معنى "Short-Circuit Evaluation"؟ أعطِ مثالاً عمليًا على فائدته.
4. متى يكون استخدام `match-case` أنسب من `if/elif/else` ؟
5. ما القيم التي تُعتبر "Falsy" ضمنيًا في Python حتى لو لم تكن `False` حرفيًا؟

<details>
<summary>✅ الإجابات النموذجية</summary>

1. لأن Python تعتمد على المسافة البادئة لتحديد أي الأسطر تنتمي لأي كتلة شرطية أو حلقة، بدلاً من استخدام أقواس `{}` كما في لغات أخرى - أي خطأ في المسافة البادئة يُسبب خطأ نحويًا (IndentationError) مباشرة.
2. `and` تتطلب أن يكون **كلا** الشرطين صحيحين ليكون الناتج True، بينما `or` تكتفي بأن يكون **واحد فقط** من الشرطين صحيحًا.
3. يعني أن Python تتوقف عن تقييم بقية الشرط بمجرد معرفة النتيجة النهائية (مثلاً مع `and` ، إن كان أول شرط False، لا داعي لفحص الباقي). فائدته العملية: تجنّب أخطاء محتملة، مثل التحقق من وجود مفتاح في قاموس قبل محاولة الوصول لقيمته في نفس الشرط.
4. عندما تحتاج مقارنة قيمة واحدة ضد عدة احتمالات محددة بوضوح (كمقارنة قيمة HTTP status code، أو يوم من أيام الأسبوع)، أو عند الحاجة لمطابقة أنماط/أنواع بيانات معقدة.
5. `False` , `0` , `0.0` , `""` (نص فارغ), `None` , `[]` (قائمة فارغة), `{}` (قاموس فارغ) - كل هذه القيم تُقيَّم كـ Falsy رغم أنها ليست `False` حرفيًا.

</details>

6.17 📌 ملخص الفصل (Cheat Sheet)

Python

```python
# الشرط الأساسي
if condition:
    pass
elif other_condition:
    pass
else:
    pass

# العوامل المنطقية
x and y    # كلاهما True
x or y     # أحدهما True
not x      # عكس x

# Ternary Expression
result = "A" if condition else "B"

# match-case (Python 3.10+)
match value:
    case 1:
        pass
    case 2 | 3:  # أو
        pass
    case _:  # الحالة الافتراضية
        pass

# Walrus Operator
if (n := get_value()) > 5:
    print(n)

# Truthy / Falsy
if my_string:  # True إن لم يكن فارغًا
    pass
```

⚠️ تذكّر دائمًا

- استخدم `==` للمقارنة، `=` للإسناد فقط
- 4 مسافات دائمًا للمسافة البادئة، لا تخلط مع Tab
- `if is_valid:` أفضل من `if is_valid == True:`
- رتّب شروط `elif` من الأكثر تحديدًا للأقل

🎯 ماذا بعد؟

تعلمنا الآن كيف يتخذ برنامجنا قرارات. لكن ماذا لو أردنا تنفيذ نفس الكود عدة مرات؟ تخيل طباعة «مرحبا» 100 مرة - هل ستكتب `print()` مئة مرة؟ بالطبع لا! في **الفصل السابع**، سنتعلم **التكرار (Loops)** - إحدى أقوى الأدوات في البرمجة، شاملة `for` و `while`، مع بناء مشاريع تفاعلية ممتعة مثل لعبة تخمين الأرقام.

فصل 07 / 26 · الجزء الثاني

07

التكرار (Loops)

7 دقيقة قراءة · 19 قسمًا · 29 مثالًا · 1,496 كلمة

في هذا الفصل

7.1 مقدمة: لماذا نحتاج التكرار؟
7.2 حلقة `while`: التكرار الشرطي
7.3 ⚠ الفخ الأخطر: الحلقات اللا نهائية (Infinite Loops)
7.4 حلقة `for`: التكرار على عناصر معروفة
7.5 متى تستخدم `for` ومتى تستخدم `while`؟ (قاعدة ذهبية لا تُنسى)
7.6 `break`: الخروج الفوري من الحلقة
7.7 `continue`: تخطّي التكرار الحالي فقط
7.8 `else` في الحلقات: ميزة نادرًا ما تُشرح بوضوح
7.9 الحلقات المتداخلة (Nested Loops)
7.10 `break` و `continue` في الحلقات المتداخلة
7.11 دالة `enumerate()`: الحصول على الفهرس والقيمة معًا
7.12 أداء الحلقات: نصائح احترافية مبكرة
7.13 أخطاء شائعة ونصائح احترافية
7.14 جدول مرجعي شامل للفصل
7.15 تمارين عملية للفصل
7.16 مشروع تطبيقي 1: «لعبة تخمين الأرقام»
7.17 مشروع تطبيقي 2: «جدول الضرب التفاعلي»
7.18 اختبر فهمك (Quiz سريع)
7.19 ملخص الفصل (Cheat Sheet)

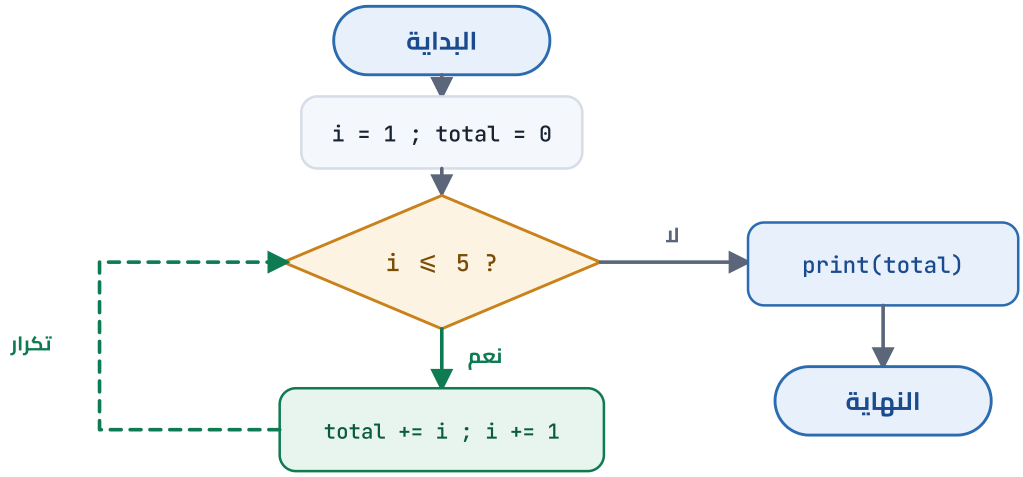
> *«التكرار هو أحد أعظم اختراعات البرمجة - تخيل كتابة سطر واحد من الكود يُنفَّذ آلاف المرات دون أن تكتبه آلاف المرات يدويًا. هذا هو السحر الحقيقي للحوسبة.»*

7.1 📖 مقدمة: لماذا نحتاج التكرار؟

تخيل أنك تريد طباعة الأرقام من 1 إلى 1000. هل ستكتب 1000 سطر `print()`؟ بالطبع لا. تخيل أيضًا أنك تريد معالجة بيانات 10,000 عميل في قاعدة بيانات - هل ستكتب كودًا منفصلاً لكل عميل؟ هذا مستحيل عمليًا.

الحلقات (Loops) هي الحل: أداة تسمح لك بتكرار تنفيذ كتلة من الكود عدة مرات، إما لعدد محدد من المرات، أو حتى تحقق شرط معين.

7.2 حلقة `while` : التكرار الشرطي



حلقة while تُعاد طالما بقي الشرط صحيحًا؛ ويجب أن يقرّب الجسم الحالة من الانتهاء.

شكل 7.1 — تدفّق حلقة while

✍️ الصيغة الأساسية

```python
while condition:
    # الكود الذي يتكرر طالما الشرط True
```

```python
count: int = 1

while count ⩽ 5:
    print(f"العدّ: {count}")
    count += 1

print("انتهى العد!")
```

الناتج:

```
العدّ: 1
العدّ: 2
العدّ: 3
العدّ: 4
العدّ: 5
انتهى العد!
```

🧹 كيف تعمل `while` خطوة بخطوة؟

```
1. فحص الشرط (count ⩽ 5)
2. إذا True → نفّذ الكود بالداخل → عُد للخطوة 1
3. إذا False → اخرج من الحلقة بالكامل
```

> 💡 **ملاحظة جوهرية:** لاحظ أننا احتجنا `count += 1` **بأنفسنا** داخل الحلقة. `while` لا «تعرف» متى تتوقف تلقائيًا - **أنت** المسؤول عن تحديث الشرط بحيث يُصبح `False` في وقت ما، وإلا...

7.3 الفخ الأخطر: الحلقات اللا نهائية (Infinite Loops)

🔥 المشكلة

```python
count: int = 1

while count ⩽ 5:
    print(f"العدّ: {count}")
    # نسيان count += 1 !
```

هذا الكود سيطبع `"العدّ: 1"` **إلى الأبد** دون توقف، لأن `count` يبقى `1` دائمًا، والشرط `1 ⩽ 5` يبقى `True` للأبد.

🚨 كيف تتعامل مع حلقة لا نهائية حدثت فعليًا؟

إذا وقعت في هذا الموقف (سيحدث للجميع في مرحلة ما):

- **في الطرفية:** اضغط `Ctrl + C` لإيقاف البرنامج فورًا
- **في VS Code:** اضغط زر "Stop" (مربع أحمر) في شريط التشغيل

⚠️ **تحذير حرج جدًا:** قبل تشغيل أي حلقة `while`، **تأكد دائمًا** أن هناك مسارًا منطقيًا واضحًا يجعل الشرط `False` في وقت ما. هذا أحد أكثر الأخطاء إحباطًا للمبتدئين - خصوصًا عند نسيان تحديث متغير العدّاد.

✅ قائمة تحقق قبل كتابة أي حلقة `while`

```
✓ هل قمت بتهيئة المتغير المُستخدَم في الشرط قبل الحلقة؟
✓ هل يوجد سطر داخل الحلقة يُغيّر قيمة ذلك المتغير؟
✓ هل التغيير يقترب فعليًا من جعل الشرط False؟
```

🎯 استخدام متعمّد للحلقة اللانهائية (مع `break`)

أحيانًا، نُريد فعليًا حلقة «لا نهائية» لكن مع شرط خروج داخلي واضح - سنتعلم `break` بعد قليل:

```python
while True:  # لا نهائية عمدًا
    user_input: str = input("اكتب 'خروج' لإنهاء البرنامج: ")
    if user_input == "خروج":
        break  # نخرج من الحلقة هنا
    print(f"كتبت: {user_input}")
```

💡 هذا النمط (`while True` مع `break` داخلي) **شائع جدًا واحترافي تمامًا** في البرمجة الحقيقية - خصوصًا في البرامج التفاعلية التي تنتظر مدخلات المستخدم باستمرار حتى يختار الخروج.

7.4 حلقة `for`: التكرار على عناصر معروفة

✍️ الصيغة الأساسية مع `range()`

```python
for i in range(5):
    print(i)
```

الناتج:

```
0
1
2
3
4
```

⚠️ **تحذير مهم:** `range(5)` يُنتج الأرقام من **0 إلى 4** (خمسة أرقام)، **وليس** من 1 إلى 5! هذا يتبع نفس منطق الفهرسة التي تعلمناها (تبدأ من 0، والنهاية غير مُضمَّنة).

📊 صيغ `range()` الثلاث

Python

```python
# range(end) - من 0 حتى end-1
for i in range(5):
    print(i)  # 0, 1, 2, 3, 4

# range(start, end) - من start حتى end-1
for i in range(2, 6):
    print(i)  # 2, 3, 4, 5

# range(start, end, step) - بقفزات محددة
for i in range(0, 10, 2):
    print(i)  # 0, 2, 4, 6, 8

# step سالب للعد التنازلي
for i in range(10, 0, -1):
    print(i)  # 10, 9, 8, 7, 6, 5, 4, 3, 2, 1
```

🔄 `for` على النصوص مباشرة

Python

```python
word: str = "Python"

for letter in word:
    print(letter)
```

الناتج:

```
P
y
t
h
o
n
```

> **فهم جوهري مهم:** `for` في Python تختلف عن لغات أخرى كثيرة (مثل C++ أو Java التقليدية) حيث تعتمد على عدّاد رقمي صريح. في Python، `for` تعني حرفيًا: **«لكل عنصر في هذه المجموعة القابلة للتكرار (Iterable)»**. هذا سيصبح أوضح بكثير عندما نتعلم القوائم في الفصل القادم.

7.5 متى تستخدم `for` ومتى تستخدم `while`؟ (قاعدة ذهبية لا تُنسى)

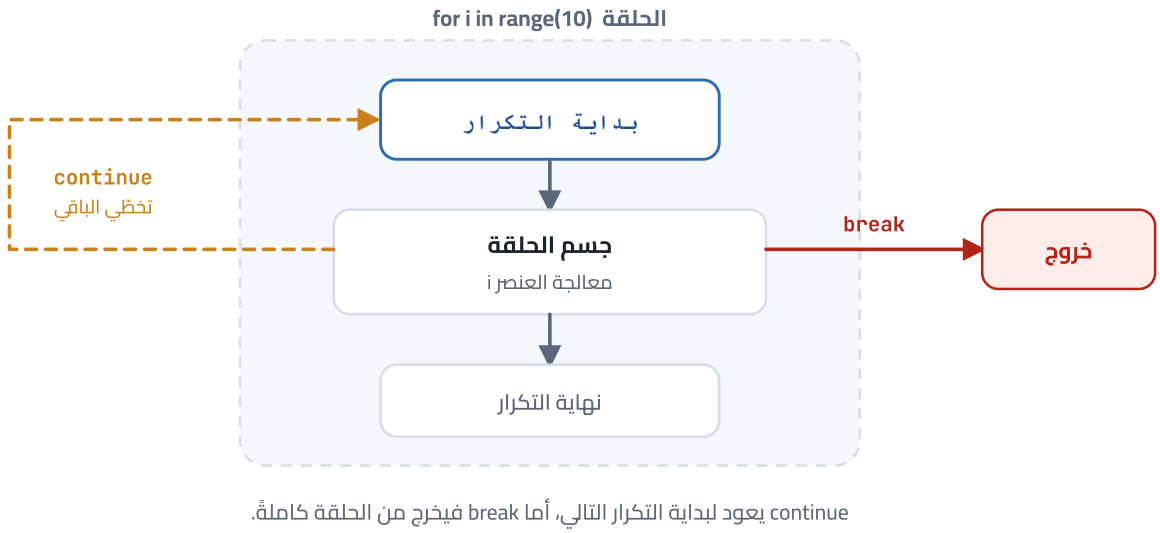
هذا سؤال يُربك كثيرًا من المبتدئين. إليك القاعدة البسيطة والحاسمة:

الحالة	استخدم
تعرف **بالضبط** عدد مرات التكرار مسبقًا	✅ `for`
التكرار يعتمد على **شرط** قد يتغيّر بشكل غير متوقع	✅ `while`
التكرار على عناصر مجموعة (نص، قائمة...)	✅ `for`
انتظار إدخال معيّن من المستخدم (لا تعرف متى سيأتي)	✅ `while`

Python

```python
# مثال for: نعرف بالضبط أننا نريد 5 تكرارات
for i in range(5):
    print("مرحبا")

# مثال while: لا نعرف متى سيُدخل المستخدم كلمة المرور الصحيحة
password: str = ""
while password != "1234":
    password = input("أدخل كلمة المرور: ")
```

7.6 break : الخروج الفوري من الحلقة



continue يعود لبداية التكرار التالي، أما break فيخرج من الحلقة كاملةً.

شكل 7.2 — الفرق بين break وcontinue

```python
for number in range(1, 11):
    if number == 5:
        break  # إيقاف الحلقة بالكامل فور الوصول لـ 5
    print(number)
```

الناتج:

```
1
2
3
4
```

`break` يُنهي الحلقة **بالكامل** فورًا، بغض النظر عن باقي عدد مرات التكرار المحتملة.

🎯 استخدام عملي: البحث عن عنصر والتوقف فور إيجاده

```python
numbers: list[int] = [4, 7, 2, 9, 5, 1]  # سنتعلم القوائم بعمق قريبًا
target: int = 9
found: bool = False

for num in numbers:
    if num == target:
        found = True
        break  # لا داعي لمتابعة البحث بعد إيجاده!

if found:
    print("العنصر موجود!")
```

7.7 `continue`: تخطّي التكرار الحالي فقط

```python
for number in range(1, 11):
    if number % 2 == 0:  # إذا كان زوجيًا
        continue  # تخطّى باقي كود هذا التكرار، انتقل للتالي مباشرة
    print(number)
```

الناتج:

```
1
3
5
7
9
```

الفرق الجوهري بين `break` و `continue`:

الأمر	التأثير
`break`	يُنهي الحلقة **بالكامل** فورًا
`continue`	يتخطّى **فقط التكرار الحالي**، ويستمر للتكرار التالي

7.8 `else` في الحلقات: ميزة نادرًا ما تُشرح بوضوح

🤔 مفهوم فريد في Python

Python تسمح بإضافة `else` بعد حلقة `for` أو `while` - لكن سلوكها مختلف تمامًا عمّا قد تتوقعه:

> ***قاعدة `else` في الحلقات:*** *تُنفَّذ كتلة `else` فقط إذا* ***اكتملت الحلقة بالكامل دون استخدام `break`.***

```python
for number in range(1, 6):
    print(number)
else:
    print("اكتملت الحلقة بنجاح!")
```

الناتج:

```
1
2
3
4
5
اكتملت الحلقة بنجاح!
```

لكن إذا استخدمنا `break`:

```python
for number in range(1, 6):
    if number == 3:
        break
    print(number)
else:
    print("اكتملت الحلقة بنجاح!")  # لن تُطبع!
```

الناتج:

```
1
2
```

💡 **استخدام عملي حقيقي:** هذا مفيد جدًا في سيناريوهات البحث - «هل وجدت العنصر؟ إن لم أجده بعد فحص الكل، نفّذ شيئًا»:

```python
numbers: list[int] = [1, 3, 5, 7, 9]
target: int = 4

for num in numbers:
    if num == target:
        print("وُجد!")
        break
else:
    print("لم يُوجد العنصر في القائمة")  # سنُطبع هذه، لأن 4 غير موجود
```

⚠️ **تحذير:** هذه الميزة تُربك حتى المبرمجين ذوي الخبرة لأنها غير بديهية (`else` هنا لا تعني «عكس الحلقة» بل «لم يحدث break»). استخدمها بحذر، وتأكد أن فريقك (أو القارئ المستقبلي لكودك) يفهم هذا السلوك إن استخدمتها.

7.9 الحلقات المتداخلة (Nested Loops)

```
for i in range(3):
    for j in range(3):
        print(i, j)
```

الدورة الخارجية تختار الصف
والداخلية تمرّ على كل عمود فيه

i \ j	0	1	2
0	0,0	0,1	0,2
1	1,0	1,1	1,2
2	2,0	2,1	2,2

تسع طباعات = 3 × 3

عدد التكرارات الكلي = حاصل ضرب عددَي دورات الحلقتين.

شكل 7.3 — الحلقات المتداخلة على شبكة 3×3

```python
for i in range(1, 4):
    for j in range(1, 4):
        print(f"i={i}, j={j}")
```

الناتج:

```
i=1, j=1
i=1, j=2
i=1, j=3
i=2, j=1
i=2, j=2
i=2, j=3
i=3, j=1
i=3, j=2
i=3, j=3
```

كيف تفهم الحلقات المتداخلة؟ لكل تكرار واحد في الحلقة **الخارجية**، تُنفَّذ الحلقة **الداخلية** بالكامل من البداية للنهاية. هنا: الحلقة الخارجية 3 مرات × الحلقة الداخلية 3 مرات = 9 تكرارات إجمالية.

🎯 مثال عملي: جدول الضرب الكامل

```python
for i in range(1, 4):
    for j in range(1, 4):
        result: int = i * j
        print(f"{i} × {j} = {result}")
    print("-" * 15)  # فاصل بعد كل جدول فرعي
```

تحذير أداء: الحلقات المتداخلة بعمق (3+ مستويات) يمكن أن تُصبح بطيئة جدًا مع البيانات الكبيرة (تعقيد $O(n^2)$ أو أسوأ - مفهوم سنلمسه أكثر لاحقًا في فصل الأداء). كن حذرًا عند التعامل مع مجموعات بيانات كبيرة.

7.10 `break` و `continue` في الحلقات المتداخلة

⚠️ نقطة دقيقة يجب فهمها جيدًا

```python
for i in range(3):
    for j in range(3):
        if j == 1:
            break  # يُخرج فقط من الحلقة الداخلية!
        print(f"i={i}, j={j}")
```

الناتج:

```
i=0, j=0
i=1, j=0
i=2, j=0
```

🎯 **قاعدة مهمة:** `break` و `continue` يؤثران فقط على **الحلقة التي يُكتبان مباشرة بداخلها** (الأقرب لهما)، وليس على كل الحلقات المتداخلة. إذا أردت الخروج من كل الحلقات المتداخلة دفعة واحدة، تحتاج تقنيات أكثر تقدمًا (مثل استخدام متغير علم "flag"، أو تحويل الكود لدالة واستخدام `return` - سنتعلم هذا في الفصل العاشر).

7.11 دالة `enumerate()`: الحصول على الفهرس والقيمة معًا

```python
fruits: list[str] = ["تفاح", "موز", "برتقال"]  # تشويق للفصل القادم

for index, fruit in enumerate(fruits):
    print(f"{index}: {fruit}")
```

الناتج:

```
0: تفاح
1: موز
2: برتقال
```

> **استخدام عملي شائع جدًا:** تُستخدم `enumerate()` كثيرًا عندما تحتاج **موضع** العنصر بالإضافة لقيمته - أنيق جدًا مقارنة بإدارة عدّاد يدوي منفصل.

```python
# بدء العدّ من 1 بدلاً من 0
for index, fruit in enumerate(fruits, start=1):
    print(f"{index}: {fruit}")
```

7.12 أداء الحلقات: نصائح احترافية مبكرة

✅ نصيحة 1: تجنّب الحسابات المتكررة غير الضرورية

```python
import math

numbers: list[int] = [1, 2, 3, 4, 5]

# ❌ يُعيد حساب len() في كل تكرار (رغم أنها سريعة، المبدأ عام) - غير فعّال
for i in range(len(numbers)):
    print(numbers[i])

# ✅ أفضل - أسلوب Pythonic مباشر
for number in numbers:
    print(number)
```

✅ نصيحة 2: استخدم `range()` بدلاً من بناء قوائم يدوية للعد

```python
# ❌ غير ضروري وغير فعّال من حيث الذاكرة
numbers_list = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
for n in numbers_list:
    print(n)

# ✅ أفضل - range لا يُنشئ قائمة فعلية في الذاكرة، بل يُولّد القيم عند الحاجة فقط
for n in range(10):
    print(n)
```

> **ملاحظة متقدمة للفضوليين:** `range()` في Python 3 هي كائن «كسول» (Lazy) - لا تُخزّن كل الأرقام في الذاكرة دفعة واحدة، بل تُولّدها واحدًا تلو الآخر عند الحاجة فعليًا. هذا يجعلها فعّالة جدًا حتى مع نطاقات ضخمة مثل `range(1_000_000_000)` .

7.13 أخطاء شائعة ونصائح احترافية

⚠️ الخطأ 1: الحلقة اللا نهائية بسبب نسيان تحديث المتغير

```python
# ❌ خطأ فادح - حلقة لا نهائية
i = 0
while i < 5:
    print(i)
    # نسيان i += 1 !
```

⚠️ الخطأ 2: استخدام `range()` بشكل خاطئ (توقّع تضمين النهاية)

```python
for i in range(1, 5):
    print(i)  # 1, 2, 3, 4 5 ليس - فقط!
```

إذا أردت تضمين 5:

```python
for i in range(1, 6):  # ← 1+ يجب أن تكون النهاية
    print(i)  # 1, 2, 3, 4, 5
```

⚠️ الخطأ 3: تعديل متغير العدّاد يدويًا داخل حلقة `for`

```python
for i in range(5):
    print(i)
    i += 10  # ⚠️ هذا لا يُؤثر على تكرارات range() القادمة!
```

> **توضيح مهم:** على عكس `while`، حلقة `for` مع `range()` تتحكم في قيمة `i` تلقائيًا من `range()` نفسها في كل تكرار - تعديلك اليدوي لـ `i` بالداخل لن يُغيّر مسار الحلقة إطلاقًا.

✅ نصيحة احترافية: تسمية متغيرات الحلقة بوضوح

```python
# ❌ غامض عند التداخل
for i in range(10):
    for j in range(10):
        pass

# ✅ أوضح عند وجود معنى حقيقي
for row in range(10):
    for column in range(10):
        pass
```

7.14 جدول مرجعي شامل للفصل

الأداة	الوظيفة	مثال
`while`	تكرار حسب شرط	`while x < 5:`
`for ... in`	تكرار على عناصر	`for item in items:`
`range(n)`	أرقام من 0 إلى n-1	`range(5)`
`range(a,b)`	أرقام من a إلى b-1	`range(2,8)`
`range(a,b,s)`	بقفزة s	`range(0,10,2)`
`break`	خروج فوري كامل	-
`continue`	تخطّي التكرار الحالي	-
`else` (مع حلقة)	يُنفَّذ إن لم يحدث break	-
`enumerate()`	فهرس + قيمة معًا	`enumerate(items)`

7.15 تمارين عملية للفصل

🟢 تمرين 1 (مستوى مبتدئ)

اكتب برنامجًا يطبع جدول الضرب الكامل لرقم يُدخله المستخدم (من 1 إلى 10)، باستخدام حلقة `for`.

🟡 تمرين 2 (مستوى متوسط)

اكتب برنامجًا يطلب من المستخدم أرقامًا متتالية (باستخدام `while`)، ويتوقف فقط عندما يُدخل المستخدم الرقم `0`، ثم يطبع: مجموع كل الأرقام المُدخلة (غير شاملة الصفر)، وعددها، ومتوسطها.

🔴 تمرين 3 (تحدي)

اكتب برنامجًا يطبع «مثلث النجوم» التالي باستخدام حلقات متداخلة، حيث يُحدد المستخدم عدد الصفوف:

```
*
**
***
****
*****
```

7.16 مشروع تطبيقي 1: «لعبة تخمين الأرقام»

📋 المهمة

أنشئ ملفًا باسم `number_guessing_game.py`:

المتطلبات:

1. البرنامج يختار رقمًا عشوائيًا بين 1 و100 (باستخدام `random.randint`)
2. يطلب من المستخدم تخمين الرقم باستمرار (`while True` مع `break`)
3. بعد كل محاولة، يُخبر المستخدم إن كان تخمينه «أكبر من اللازم» أو «أصغر من اللازم»
4. عند التخمين الصحيح، يطبع تهنئة مع عدد المحاولات التي استغرقها
5. (تحدٍ إضافي) حدّد حدًا أقصى لعدد المحاولات (مثلاً 7)، وإن لم ينجح، اطبع «انتهت محاولاتك» مع كشف الرقم الصحيح

💻 حل نموذجي كامل

<details>
<summary>✅ اضغط لرؤية الحل الكامل (حاول بنفسك أولاً!)</summary>

Python number_guessing_game.py

```python
"""لعبة تخمين الأرقام - احزر الرقم السري!"""

import random

MAX_ATTEMPTS: int = 7
secret_number: int = random.randint(1, 100)
attempts: int = 0

print("=" * 40)
print("🎯  لعبة تخمين الأرقام")
print("=" * 40)
print(f"لديك {MAX_ATTEMPTS} محاولات لتخمين رقمًا بين 1 و100!")

while attempts < MAX_ATTEMPTS:
    guess_text: str = input("\nخمّن الرقم: ").strip()

    if not guess_text.isdigit():
        print("⚠️ من فضلك أدخل رقمًا صحيحًا فقط!")
        continue

    guess: int = int(guess_text)
    attempts += 1

    if guess == secret_number:
        print(f"\n🎉 مبروك! خمّنت الرقم الصحيح في {attempts} محاولة!")
        break
    elif guess < secret_number:
        print("📈 الرقم السري أكبر من تخمينك")
    else:
        print("📉 الرقم السري أصغر من تخمينك")

    remaining: int = MAX_ATTEMPTS - attempts
    if remaining > 0:
        print(f"محاولاتك المتبقية: {remaining}")
else:
    print(f"\n😢 انتهت محاولاتك! الرقم السري كان: {secret_number}")

print("=" * 40)
```

</details>

ملاحظة على الحل: لاحظ استخدامنا لـ `else` الخاصة بالحلقة `while` - تُنفَّذ فقط إذا استُنفدت كل المحاولات دون `break` (أي دون تخمين صحيح). هذا تطبيق عملي ممتاز لما تعلمناه في القسم 7.7.

7.17 مشروع تطبيقي 2: «جدول الضرب التفاعلي»

📋 المهمة

أنشئ ملفًا باسم `multiplication_table.py` يطبع جدول ضرب كامل ومنسّق جميل بصريًا (من 1×1 حتى 10×10) بشكل جدول حقيقي باستخدام حلقات متداخلة و f-strings للمحاذاة.

💻 حل نموذجي

<details>
<summary>✅ اضغط لرؤية الحل الكامل</summary>

Python multiplication_table.py

```python
"""طباعة جدول ضرب منسّق من 1 إلى 10."""

SIZE: int = 10

# طباعة رأس الجدول
print("    ", end="")
for col in range(1, SIZE + 1):
    print(f"{col:4}", end="")
print()
print("-" * (SIZE * 4 + 4))

# طباعة الجدول الكامل
for row in range(1, SIZE + 1):
    print(f"{row:3} |", end="")
    for col in range(1, SIZE + 1):
        result: int = row * col
        print(f"{result:4}", end="")
    print()  # سطر جديد بعد كل صف
```

</details>

ملاحظة على الحل: استخدمنا `end=""` في `print()` لمنع الانتقال التلقائي لسطر جديد (الافتراضي)، مما يسمح لنا بطباعة عدة قيم في نفس السطر - تقنية مفيدة جدًا لبناء جداول نصية.

7.18 اختبر فهمك (Quiz سريع)

1. ما الفرق الجوهري بين متى تستخدم `for` ومتى تستخدم `while`؟
2. لماذا `range(1, 5)` يُنتج الأرقام 1 إلى 4 فقط، وليس حتى 5؟
3. ما الفرق بين `break` و `continue`؟
4. متى تُنفَّذ كتلة `else` المرتبطة بحلقة `for` أو `while`؟
5. ما السبب الأكثر شيوعًا لحدوث حلقة لا نهائية؟

✅ الإجابات النموذجية

1. `for` تُستخدم عندما تعرف مسبقًا عدد مرات التكرار أو تُكرر على عناصر مجموعة معروفة، بينما `while` تُستخدم عندما يعتمد التكرار على شرط قد يتغيّر بشكل غير متوقع (كانتظار إدخال معيّن من المستخدم).
2. لأن `range()` تتبع نفس منطق الفهرسة في Python حيث نقطة البداية مُضمَّنة والنهاية غير مُضمَّنة، تمامًا كالتقطيع في النصوص.
3. `break` يُنهي الحلقة بالكامل فورًا، بينما `continue` يتخطّى فقط باقي كود التكرار الحالي وينتقل مباشرة للتكرار التالي.
4. تُنفَّذ فقط إذا اكتملت الحلقة بالكامل بشكل طبيعي دون استخدام `break` في أي تكرار.
5. نسيان تحديث المتغير المُستخدَم في شرط حلقة `while` بحيث يبقى الشرط `True` دائمًا دون أي تغيير فعلي يقربه من `False` .

7.19 📌 ملخص الفصل (Cheat Sheet)

```python
# while - تكرار شرطي
while condition:
    # الكود
    update_condition()  # لا تنسَ هذا!

# for - تكرار على عناصر معروفة
for item in collection:
    print(item)

for i in range(5):         # 0 إلى 4
for i in range(2, 8):      # 2 إلى 7
for i in range(0, 10, 2): # 2 بقفزة

# التحكم بالحلقة
break      # خروج فوري كامل
continue   # تخطّي التكرار الحالي فقط

# else مع الحلقة (نادر لكن مفيد)
for item in items:
    if condition:
        break
else:
    print("لم يحدث break")

# enumerate - فهرس + قيمة
for index, value in enumerate(items):
    print(index, value)
```

⚠️ تذكّر دائمًا

- في `while`: تأكد دائمًا من وجود تحديث يُقرِّب الشرط من `False`
- `range(a, b)` لا يشمل `b` - فقط حتى `b-1`
- `break` يخرج من الحلقة **الأقرب** فقط في التداخل
- استخدم `Ctrl+C` لإيقاف حلقة لا نهائية عالقة

🎯 ماذا بعد؟

حتى الآن، تعاملنا مع متغيرات تحمل **قيمة واحدة** فقط (رقم، نص، منطقية). لكن ماذا لو أردنا تخزين **مجموعة كاملة** من القيم معًا - كقائمة بأسماء الطلاب، أو درجاتهم؟ في **الفصل الثامن**، سندخل عالم **هياكل البيانات** بدءًا من أهمها وأكثرها استخدامًا: **القوائم (Lists)** - حيث ستتكامل كل مهارات الحلقات والشروط التي تعلمناها لبناء برامج حقيقية وقوية.

فصل 08 / 26 · الجزء الثاني

08

هياكل البيانات - القوائم (Lists)

8 دقيقة قراءة · 18 قسمًا · 45 مثالًا · 1,527 كلمة

في هذا الفصل

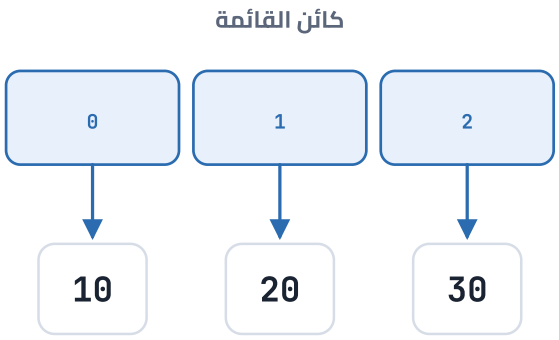
8.1 مقدمة: نقطة تحوّل حقيقية في رحلتك
8.2 ما هي القائمة (List)؟
8.3 الوصول للعناصر: الفهرسة والتقطيع (مألوفة من النصوص!)
8.4 الفرق الجوهري: القوائم Mutable (قابلة للتعديل)!
8.5 أهم Methods القوائم
8.6 القوائم المتداخلة (Nested Lists)
8.7 ⚠️ الفخ الأشهر على الإطلاق: نسخ القوائم (Shallow Copy Trap)
8.8 List Comprehension: الأسلوب الأنيق والـ"Pythonic"
8.9 دوال مُدمجة قوية تعمل مع القوائم
8.10 القوائم والحلقات: دمج كل ما تعلمناه
8.11 🔄 متى تستخدم القوائم؟ (تمهيد لمقارنة أشمل قادمة)
8.12 Tuples: الأخ «غير القابل للتعديل» للقائمة
8.13 أخطاء شائعة ونصائح احترافية
8.14 جدول مرجعي شامل للفصل
8.15 تمارين عملية للفصل
8.16 مشروع تطبيقي: «نظام إدارة قائمة مهام (To-Do List)»
8.17 اختبر فهمك (Quiz سريع)
8.18 ملخص الفصل (Cheat Sheet)

> *«إذا كانت المتغيرات التي تعلمناها حتى الآن أشبه بصناديق فردية، فإن القوائم هي أرفف كاملة يمكنها احتواء عدد غير محدود من هذه الصناديق، منظّمة بترتيب محدد.»*

8.1 📖 مقدمة: نقطة تحوّل حقيقية في رحلتك

كل ما تعلمته حتى الآن كان يتعامل مع **قيمة واحدة** في كل متغير. لكن العالم الحقيقي مليء بالمجموعات: قائمة أصدقائك، درجات طلاب فصل كامل، منتجات متجر إلكتروني. في هذا الفصل، ستتعلم أول **هيكل بيانات** حقيقي في Python - والذي سيغيّر جذريًا نوعية البرامج التي يمكنك بناءها.

8.2 ما هي القائمة (List)؟



كل خليّة تخزّن مرجعًا (عنوانًا) إلى كائن مستقل، لا القيمة نفسها.

لهذا يمكن للقائمة أن تحتوي أنواعًا مختلفة، وأن تتغيّر عناصرها بحرية.

شكل 8.1 — القائمة مصفوفة مراجع إلى كائنات

🎁 التعريف والإنشاء

القائمة هي مجموعة **مُرتَّبة** و**قابلة للتعديل** من العناصر، يمكن أن تحتوي أي نوع بيانات - وحتى أنواعًا مختلفة معًا في نفس القائمة:

```python
fruits: list[str] = ["تفاح", "موز", "برتقال"]
numbers: list[int] = [10, 20, 30, 40]
mixed: list = ["25", "محمد", True, 3.14]  # مسموح، لكن نادرًا ما يُستخدم عمليًا

empty_list: list = []  # قائمة فارغة
```

💡 **Type Hint للقوائم**: لاحظ الصيغة `list[str]` - تخبر القارئ والأدوات أن هذه قائمة تحتوي عناصر من نوع `str` تحديدًا. هذا النمط (`list[النوع]`) أصبح قياسيًا في Python الحديثة (3.9+).

8.3 الوصول للعناصر: الفهرسة والتقطيع (مألوفة من النصوص!)

🔢 نفس منطق فهرسة النصوص بالضبط

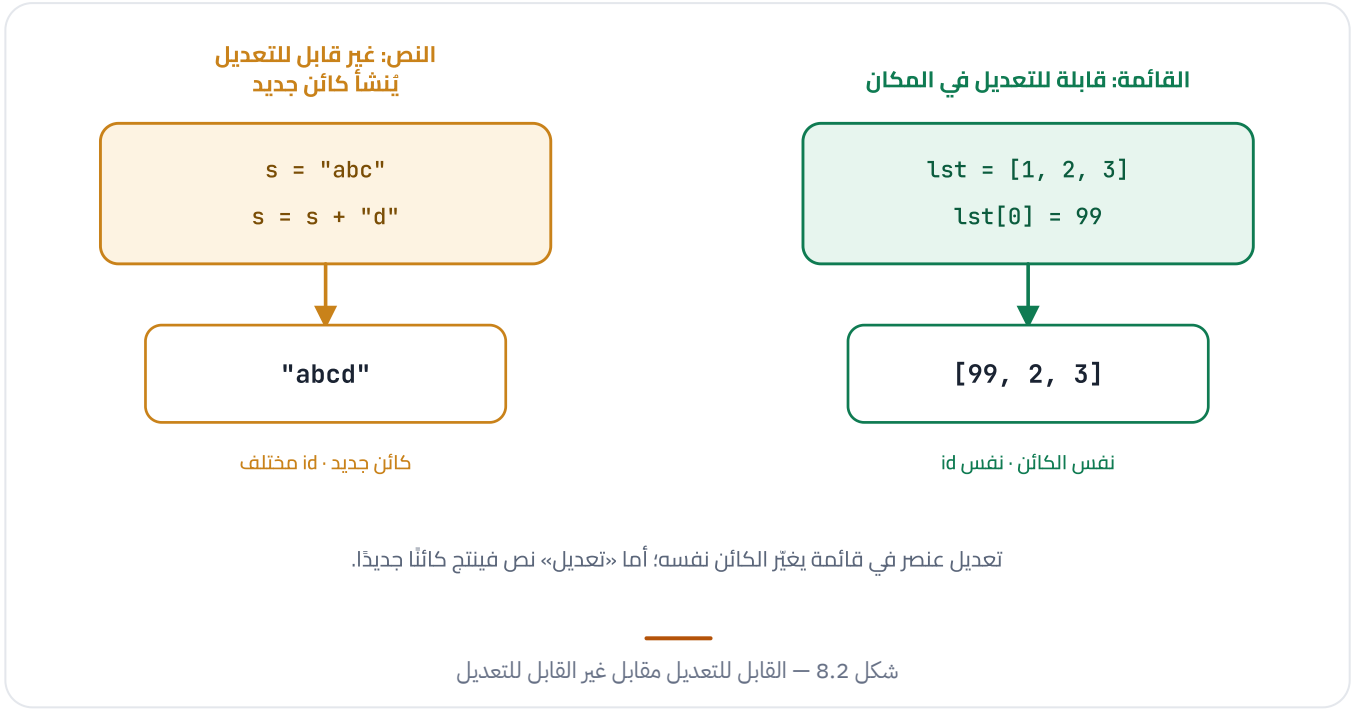
```python
fruits: list[str] = ["تفاح", "موز", "برتقال", "عنب"]

print(fruits[0])    # تفاح  (أول عنصر)
print(fruits[-1])   # عنب  (آخر عنصر)
print(fruits[1:3])  # ['موز', 'برتقال']  (تقطيع - نفس منطق النصوص)
```

🎯 **لاحظ التكامل:** كل ما تعلمته عن الفهرسة والتقطيع في الفصل الرابع (النصوص) ينطبق **حرفيًا** على القوائم. هذا التصميم المتّسق هو أحد أسباب جمال Python.

⚠️ تحذير: IndexError لا يزال قائمًا

```python
fruits: list[str] = ["تفاح", "موز"]
print(fruits[5])  # ❌ IndexError: list index out of range
```

8.4 الفرق الجوهري: القوائم Mutable (قابلة للتعديل)!



تعديل عنصر في قائمة يغيّر الكائن نفسه؛ أما «تعديل» نص فينتج كائنًا جديدًا.

شكل 8.2 — القابل للتعديل مقابل غير القابل للتعديل

🔒 أهم فرق بين القوائم والنصوص

تذكر من الفصل الرابع: النصوص **Immutable** (غير قابلة للتعديل المباشر). القوائم **مختلفة تمامًا** - هي **Mutable**:

```python
fruits: list[str] = ["تفاح", "موز", "برتقال"]
fruits[0] = "مانجو"  # ✅ يعمل تمامًا! تعديل مباشر
print(fruits)  # ['مانجو', 'موز', 'برتقال']
```

مقارنة بالنصوص:

```python
word: str = "Python"
word[0] = "J"  # ❌ TypeError - النصوص لا تسمح بهذا
```

🎯 **هذا الفرق سيُفسّر سلوكيات مهمة جدًا لاحقًا** - خصوصًا فيما يتعلق بنسخ القوائم (سنتعمق فيه قريبًا في هذا الفصل).

8.5 أهم Methods القوائم

➕ إضافة عناصر

```python
fruits: list[str] = ["تفاح", "موز"]

# إضافة عنصر واحد في النهاية
fruits.append("برتقال")
print(fruits)  # ['تفاح', 'موز', 'برتقال']

# إدراج عنصر في موضع محدد
fruits.insert(1, "عنب")
print(fruits)  # ['تفاح', 'عنب', 'موز', 'برتقال']

# إضافة عدة عناصر دفعة واحدة
fruits.extend(["مانجو", "أناناس"])
print(fruits)  # ['تفاح', 'عنب', 'موز', 'برتقال', 'مانجو', 'أناناس']
```

⚠️ **تحذير شائع:** الخلط بين `.append()` و `.extend()` :

```python
numbers: list[int] = [1, 2, 3]

numbers.append([4, 5])
print(numbers)  # [1, 2, 3, [4, 5]]  ← قائمة متداخلة! ليس هذا المقصود غالبًا

numbers2: list[int] = [1, 2, 3]
numbers2.extend([4, 5])
print(numbers2)  # [1, 2, 3, 4, 5]  ← هذا هو المقصود عادةً
```

حذف عناصر

```python
fruits: list[str] = ["تفاح", "موز", "برتقال", "موز"]

# حذف بالقيمة (أول ظهور فقط)
fruits.remove("موز")
print(fruits)  # ['تفاح', 'برتقال', 'موز']

# حذف بالفهرس، وإعادة القيمة المحذوفة
removed_item: str = fruits.pop(0)
print(removed_item)  # تفاح
print(fruits)        # ['برتقال', 'موز']

# pop() بدون فهرس يحذف آخر عنصر
last_item: str = fruits.pop()
print(last_item)  # موز

# حذف بالفهرس دون الحاجة للقيمة المحذوفة
del fruits[0]

# إفراغ القائمة بالكامل
fruits.clear()
print(fruits)  # []
```

⚠ **تحذير:** `.remove()` تحذف **أول ظهور فقط** للقيمة، وتُسبب `ValueError` إن لم تكن القيمة موجودة إطلاقًا:

```python
fruits: list[str] = ["تفاح", "موز"]
fruits.remove("مانجو")  # ❌ ValueError: list.remove(x): x not in list
```

🔄 الترتيب والعكس

```python
numbers: list[int] = [5, 2, 8, 1, 9]

numbers.sort()
print(numbers)  # [1, 2, 5, 8, 9]  ← تُعدّل القائمة الأصلية مباشرة!

numbers.sort(reverse=True)
print(numbers)  # [9, 8, 5, 2, 1]

numbers.reverse()
print(numbers)  # [1, 2, 5, 8, 9]  ← عكس الترتيب الحالي
```

⚠️ **تحذير حرج:** `.sort()` تُعدّل القائمة **في مكانها مباشرة** وتُعيد `None` (وليس القائمة المُرتَّبة!):

```python
numbers: list[int] = [3, 1, 2]
result = numbers.sort()
print(result)   # None  ← فخ شائع جدًا!
print(numbers)  # [1, 2, 3]  ← هنا التعديل الفعلي
```

✅ الحل الاحترافي: دالة `sorted()` (لا تُعدّل الأصل)

```python
numbers: list[int] = [3, 1, 2]
sorted_numbers: list[int] = sorted(numbers)

print(numbers)         # [3, 1, 2]  ← لم تتغيّر!
print(sorted_numbers)  # [1, 2, 3]  ← قائمة جديدة مُرتَّبة
```

🔄 **مقارنة مهمة:** `.sort()` مقابل `sorted()`

الطريقة	تُعدّل الأصل؟	تُعيد ماذا؟
`list.sort()`	✅ نعم (in-place)	`None`
`sorted(list)`	❌ لا	قائمة جديدة مُرتَّبة

✅ **أفضل الممارسات:** استخدم `sorted()` دائمًا إلا إن كنت متأكدًا تمامًا أنك تريد تعديل القائمة الأصلية وتوفير الذاكرة.

🔍 البحث والعد

```python
fruits: list[str] = ["عنب" ,"تفاح" ,"موز" ,"تفاح"]

print(fruits.count("تفاح"))   # عدد مرات التكرار ← 2
print(fruits.index("موز"))    # فهرس أول ظهور ← 1
print("عنب" in fruits)        # True
print("مانجو" in fruits)      # False
```

8.6 القوائم المتداخلة (Nested Lists)

يمكن أن تحتوي القائمة قوائم أخرى بداخلها - مفهوم أساسي للتعامل مع بيانات جدولية (مثل مصفوفة أو شبكة):

```python
matrix: list[list[int]] = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]

print(matrix[0])      # [1, 2, 3]  ← الصف الأول كاملاً
print(matrix[0][1])   # 2          ← العنصر الثاني في الصف الأول
print(matrix[2][2])   # 9          ← العنصر الأخير
```

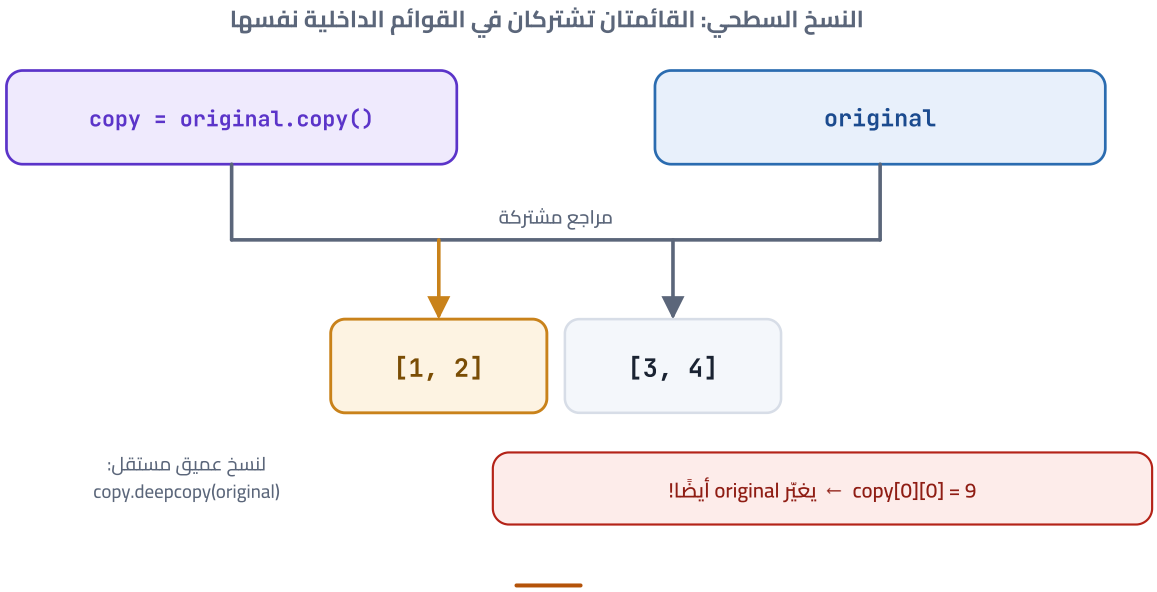
🎯 مثال عملي: طباعة مصفوفة كاملة

```python
matrix: list[list[int]] = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]

for row in matrix:
    for element in row:
        print(element, end=" ")
    print()  # سطر جديد بعد كل صف
```

الناتج:

```
1 2 3
4 5 6
7 8 9
```

8.7 الفخ الأشهر على الإطلاق: نسخ القوائم (Shallow Copy Trap)



شكل 8.3 — فخ النسخ السطحي: مراجع مشتركة

🔥 المشكلة الأكثر إرباكًا لجميع المبتدئين

تذكر من الفصل الثالث: المتغيرات في Python هي «ملصقات» تُشير لكائنات في الذاكرة، وليست «صناديق». مع القوائم (كونها Mutable)، هذا المفهوم يُسبب سلوكًا قد يصدمك تمامًا:

Python

```python
original: list[int] = [1, 2, 3]
copy: list[int] = original  # ⚠️ هذا لا ينسخ القائمة!

copy.append(4)

print(original)  # [1, 2, 3, 4]  ← رغم أننا عدّلنا !! copy فقط تغيّرت أيضًا!
print(copy)      # [1, 2, 3, 4]
```

😱 لماذا حدث هذا؟

كلا المتغيرين `original` و `copy` يُشيران لنفس القائمة **بالضبط** في الذاكرة - لم يتم إنشاء نسخة جديدة فعليًا، فقط ملصق إضافي يُشير لنفس المكان:

```python
original: list[int] = [1, 2, 3]
copy: list[int] = original

print(id(original))  # 140234567890 :مثلاً
print(id(copy))      # 140234567890 نفس الرقم بالضبط!
```

🎯 **هذا بالضبط الفرق الذي حذّرنا منه في الفصل الثالث** بين «الصناديق» و«الملصقات» - والآن ترى تبعاته العملية الحقيقية والمهمة جدًا.

✅ الحل الصحيح: إنشاء نسخة فعلية (Shallow Copy)

```python
original: list[int] = [1, 2, 3]

# 3 طرق صحيحة لإنشاء نسخة فعلية مستقلة
copy1: list[int] = original.copy()
copy2: list[int] = original[:]  # تقطيع كامل
copy3: list[int] = list(original)

copy1.append(4)
print(original)  # [1, 2, 3]  ← لم تتأثر! ✅
print(copy1)     # [1, 2, 3, 4]
```

⚠️ تحذير متقدم: Shallow Copy لا تكفي دائمًا مع القوائم المتداخلة

```python
original: list[list[int]] = [[1, 2], [3, 4]]
copy: list[list[int]] = original.copy()  # نسخة سطحية فقط!

copy[0].append(99)

print(original)  # [[1, 2, 99], [3, 4]]  ← تأثرت رغم النسخ!
print(copy)      # [[1, 2, 99], [3, 4]]
```

🤔 **لماذا؟** `.copy()` تُنشئ قائمة جديدة، لكنها **لا تنسخ العناصر الداخلية بعمق** - القوائم الداخلية لا تزال نفس الكائنات المُشتركة بين original و copy.

✅ الحل الكامل: Deep Copy

```python
import copy

original: list[list[int]] = [[1, 2], [3, 4]]
deep_copy: list[list[int]] = copy.deepcopy(original)

deep_copy[0].append(99)

print(original)   # [[1, 2], [3, 4]]  ← لم تتأثر الآن! ✅
print(deep_copy)  # [[1, 2, 99], [3, 4]]
```

🔄 جدول ملخّص: أنواع النسخ

النوع	الطريقة	يحمي من تأثر الأصل؟
لا نسخ (مرجع فقط)	`copy = original`	❌ أبدًا
نسخ سطحي (Shallow)	`.copy()` , `[:]` , `list()`	✅ للمستوى الأول فقط
نسخ عميق (Deep)	`copy.deepcopy()`	✅ لكل المستويات

🎯 **قاعدة ذهبية**: إن كانت قائمتك تحتوي فقط أنواعًا بسيطة (أرقام، نصوص)، Shallow Copy كافية تمامًا. إن كانت تحتوي قوائم أو كائنات أخرى متداخلة، استخدم `deepcopy()` .

8.8 List Comprehension : الأسلوب الأنيق والـ"Pythonic"

الترتيب: النتيجة، ثم المصدر، ثم الشرط — ويُقرأ كحلقة مختصرة.

شكل 8.4 — من الحلقة إلى List Comprehension

🤔 المشكلة التقليدية

```python
# الطريقة التقليدية لإنشاء قائمة بمربعات الأرقام
squares: list[int] = []
for number in range(1, 6):
    squares.append(number ** 2)

print(squares)  # [1, 4, 9, 16, 25]
```

✨ الحل الأنيق: List Comprehension

```python
squares: list[int] = [number ** 2 for number in range(1, 6)]
print(squares)  # [1, 4, 9, 16, 25]
```

📐 الصيغة العامة

```python
[expression for item in iterable]
```

🎯 أمثلة متدرجة الصعوبة

مثال بسيط:

```python
names: list[str] = ["أحمد" ,"سارة" ,"محمد"]
uppercase_names: list[str] = [name.upper() for name in names]
print(uppercase_names)
```

مع شرط (Filter):

```python
numbers: list[int] = range(1, 11)
even_numbers: list[int] = [n for n in numbers if n % 2 == 0]
print(even_numbers)  # [2, 4, 6, 8, 10]
```

الصيغة الكاملة مع شرط:

```python
[expression for item in iterable if condition]
```

مع شرط if/else (مختلف عن الـ filter أعلاه):

```python
numbers: list[int] = range(1, 6)
labels: list[str] = ["زوجي" if n % 2 == 0 else "فردي" for n in numbers]
print(labels)  # ['فردي' ,'زوجي' ,'فردي' ,'زوجي' ,'فردي']
```

⚠️ **تحذير على الصيغة:** لاحظ الفرق الدقيق في الموضع:

```python
# shbr Filter (يُكتب في النهاية)
[x for x in range(10) if x % 2 == 0]

# شرط Ternary (يُكتب قبل for مباشرة)
[x if x % 2 == 0 else -x for x in range(10)]
```

مع حلقات متداخلة:

```python
matrix: list[list[int]] = [[1, 2], [3, 4], [5, 6]]
flattened: list[int] = [num for row in matrix for num in row]
print(flattened)  # [1, 2, 3, 4, 5, 6]
```

🔄 مقارنة الأداء: List Comprehension مقابل الحلقة التقليدية

List comprehension ليست فقط أجمل، بل أيضًا **أسرع تنفيذيًا** غالبًا من الحلقة التقليدية المكافئة، لأن Python تُحسّنها داخليًا بشكل أفضل.

✅ **أفضل الممارسات**: استخدم list comprehension للحالات **البسيطة والواضحة**. إن أصبح التعبير معقدًا جدًا (أكثر من شرط واحد، أو منطق متداخل كثيرًا)، استخدم الحلقة التقليدية لصالح **وضوح القراءة** - القاعدة الذهبية في Python: "Readability counts".

```python
# ❌ list comprehension صعبة القراءة - معقدة جدًا
result = [x**2 if x % 2 == 0 else x**3 for x in range(20) if x > 5 if x < 15]

# ✅ أوضح كحلقة تقليدية في هذه الحالة المعقدة
result = []
for x in range(20):
    if 5 < x < 15:
        if x % 2 == 0:
            result.append(x ** 2)
        else:
            result.append(x ** 3)
```

8.9 دوال مُدمجة قوية تعمل مع القوائم

```python
numbers: list[int] = [4, 2, 8, 1, 9, 5]

print(len(numbers))    # 6   ← عدد العناصر
print(sum(numbers))    # 29  ← المجموع
print(max(numbers))    # 9   ← أكبر قيمة
print(min(numbers))    # 1   ← أصغر قيمة
```

🎯 دالة `zip()`: دمج عدة قوائم معًا

```python
names: list[str] = ["أحمد", "سارة", "محمد"]
ages: list[int] = [25, 30, 28]

for name, age in zip(names, ages):
    print(f"{name}: {age} سنة")
```

الناتج:

```
محمد: 25 سنة
سارة: 30 سنة
أحمد: 28 سنة
```

> 💡 **استخدام عملي شديد الشيوع:** `zip()` مثالية عند التعامل مع بيانات موزّعة على عدة قوائم متوازية (نفس الترتيب والطول).

8.10 القوائم والحلقات: دمج كل ما تعلمناه

```python
students: list[str] = ["فاطمة", "أحمد", "سارة", "محمد"]
scores: list[int] = [85, 92, 78, 95]

for index, student in enumerate(students):
    score: int = scores[index]
    status: str = "ناجح ✅" if score ⩾ 60 else "راسب ❌"
    print(f"{student}: {score} - {status}")
```

🎯 مثال أقوى بـ `zip()` بدلاً من الفهرسة اليدوية

```python
for student, score in zip(students, scores):
    status: str = "ناجح ✅" if score ⩾ 60 else "راسب ❌"
    print(f"{student}: {score} - {status}")
```

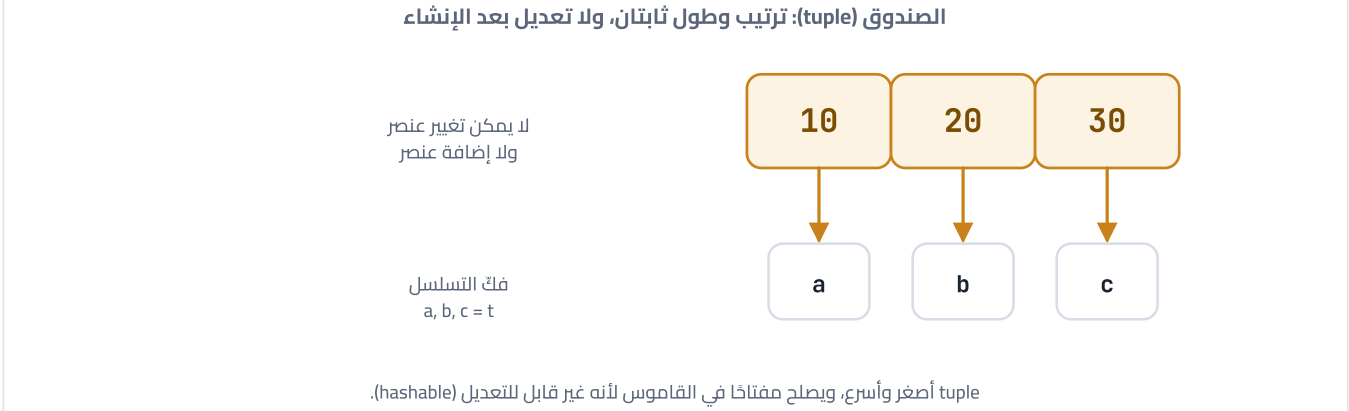
✅ **أفضل الممارسات:** استخدم `zip()` بدلاً من الوصول بالفهرس (`scores[index]`) عند التكرار المتوازي على عدة قوائم - أوضح وأقل عرضة للأخطاء.

8.11 متى تستخدم القوائم؟ (تمهيد لمقارنة أشمل قادمة)

القوائم مثالية عندما:
- تحتاج **ترتيبًا** محددًا للعناصر (الأول، الثاني، إلخ)
- تحتاج **السماح بالتكرار** (نفس القيمة أكثر من مرة)
- تحتاج **تعديل** المحتوى لاحقًا (إضافة/حذف/تغيير)

💡 في الفصل القادم، سنتعرّف على **Sets** و **Dictionaries** - هياكل بيانات أخرى لها استخدامات مختلفة تمامًا، وسنقدّم جدول مقارنة شامل يساعدك على اختيار الهيكل الصحيح لكل مشكلة.

8.12 Tuples : الأخ «غير القابل للتعديل» للقائمة



شكل 8.5 — tuple: ثابت الطول والترتيب، وفكّ التسلسل

🤔 لماذا نتعرّف على Tuples هنا؟

قبل الانتقال للفصل القادم، من المهم التعرّف سريعًا على **Tuple** - هيكل بيانات شبيه جدًا بالقائمة، لكن بفرق جوهري واحد:

```python
coordinates: tuple[int, int] = (10, 20)
colors: tuple[str, ...] = ("أحمر", "أخضر", "أزرق")
```

🔒 الفرق الجوهري: Tuples غير قابلة للتعديل (Immutable)

```python
coordinates: tuple[int, int] = (10, 20)
coordinates[0] = 99  # ❌ TypeError: 'tuple' object does not support item assignment
```

🤔 لماذا نحتاج Tuple إن كانت القوائم تفعل كل شيء؟

الخاصية	List	Tuple
قابلة للتعديل	✅ نعم	❌ لا
السرعة	أبطأ قليلاً	⚡ أسرع قليلاً
الاستخدام المناسب	بيانات تتغيّر	بيانات ثابتة (كإحداثيات، أو تاريخ ميلاد)
يمكن استخدامها كمفتاح Dictionary	❌ لا	✅ نعم (سنتعلم هذا قريبًا)

```python
# استخدام منطقي لـ Tuple: إحداثيات لن تتغيّر أبدًا بهذا الشكل
point: tuple[int, int] = (10, 20)

# استخدام منطقي لـ List: قائمة تسوق قابلة للتعديل باستمرار
shopping_list: list[str] = ["خبز", "حليب"]
shopping_list.append("بيض")  # منطقي - القوائم تتغيّر
```

> 💡 **نصيحة عملية:** استخدم Tuple كلما كانت لديك بيانات تعرف أنها **لن تتغيّر منطقيًا** طوال حياة البرنامج - هذا يُرسل إشارة واضحة لأي قارئ للكود (وأحيانًا يُحسّن الأداء قليلاً).

✍️ العمليات الأساسية على Tuples (مشابهة جدًا للقوائم، عدا التعديل)

```python
point: tuple[int, int] = (10, 20)

print(point[0])     # 10  ← الفهرسة تعمل
print(len(point))   # 2   ← len تعمل

x, y = point  # Unpacking - تفكيك القيم مباشرة لمتغيرات!
print(x, y)   # 10 20
```

🎯 **ميزة أنيقة جدًا - Tuple Unpacking**: هذا النمط مفيد جدًا وشائع الاستخدام في Python الاحترافية، وسنراه بكثرة لاحقًا مع الدوال التي تُعيد قيمًا متعددة.

8.13 أخطاء شائعة ونصائح احترافية

⚠️ الخطأ 1: الاعتقاد أن `=` تنسخ القائمة

```python
list_a: list[int] = [1, 2, 3]
list_b: list[int] = list_a  # ⚠️ مرجع فقط، وليس نسخة!
```

⚠️ الخطأ 2: نسيان أن `.sort()` تُعيد None

```python
numbers: list[int] = [3, 1, 2]
sorted_list = numbers.sort()
print(sorted_list)  # None ← خطأ شائع جدًا!
```

⚠️ الخطأ 3: تعديل قائمة أثناء التكرار عليها مباشرة

```python
numbers: list[int] = [1, 2, 3, 4, 5]

# ❌ خطر! نتائج غير متوقعة
for number in numbers:
    if number % 2 == 0:
        numbers.remove(number)  # تعديل القائمة أثناء المرور عليها!

print(numbers)  # [1, 3, 5] لكن بطريقة غير موثوقة ومحتملة الخطأ
```

⚠️ **تحذير حرج:** تعديل قائمة (بالإضافة أو الحذف) أثناء المرور عليها بـ `for` يُسبب سلوكًا غير متوقع، لأن Python تفقد «مكانها» في التتبّع. **الحل الآمن:**

```python
# ✅ الحل الصحيح: التكرار على نسخة، والتعديل على الأصل
numbers: list[int] = [1, 2, 3, 4, 5]

for number in numbers.copy():  # أو numbers[:]
    if number % 2 == 0:
        numbers.remove(number)

print(numbers)  # [1, 3, 5] ← آمن وصحيح
```

أو الأفضل - استخدام list comprehension لإنشاء قائمة جديدة بدلاً من التعديل المباشر:

```python
numbers: list[int] = [1, 2, 3, 4, 5]
odd_numbers: list[int] = [n for n in numbers if n % 2 != 0]
print(odd_numbers)  # [1, 3, 5]
```

8.14 جدول مرجعي شامل للفصل

العملية	الكود	ملاحظة
إنشاء	`[1, 2, 3]`	-

العملية	الكود	ملاحظة
إضافة في النهاية	`.append(x)`	-
إضافة عدة عناصر	`.extend([x, y])`	-
إدراج في موضع	`.insert(i, x)`	-
حذف بالقيمة	`.remove(x)`	أول ظهور فقط
حذف بالفهرس	`.pop(i)`	تُعيد القيمة المحذوفة
ترتيب (يُعدّل الأصل)	`.sort()`	تُعيد None
ترتيب (نسخة جديدة)	`sorted(list)`	لا تُعدّل الأصل
نسخ سطحي	`.copy()` أو `[:]`	مستوى واحد فقط
نسخ عميق	`copy.deepcopy()`	كل المستويات
List Comprehension	`[x for x in items]`	-
دمج قوائم متوازية	`zip(list1, list2)`	-

8.15 تمارين عملية للفصل

🟢 تمرين 1 (مستوى مبتدئ)

أنشئ قائمة بأسماء 5 مدن تحب زيارتها، ثم: أضف مدينة جديدة في النهاية، أدرج مدينة أخرى في البداية، احذف إحدى المدن بالاسم، واطبع القائمة مُرتَّبة أبجديًا.

🟡 تمرين 2 (مستوى متوسط)

لديك قائمة بدرجات طلاب: `scores = [45, 78, 92, 60, 55, 88, 70]`. استخدم list comprehension لإنشاء قائمة جديدة تحتوي فقط الدرجات الناجحة (60 فأكثر)، ثم احسب متوسط الدرجات الناجحة فقط.

🔴 تمرين 3 (تحدي)

اكتب برنامجًا يأخذ قائمتين من الأرقام، ويُعيد قائمة ثالثة تحتوي **فقط** العناصر المشتركة بينهما (الموجودة في كلتا القائمتين)، دون تكرار، باستخدام list comprehension فقط (بدون استخدام `set` التي سنتعلمها لاحقًا).

8.16 مشروع تطبيقي: «نظام إدارة قائمة مهام (To-Do List)»

📋 المهمة

أنشئ ملفًا باسم `todo_list.py` يبني نظام قائمة مهام تفاعلي كامل من الطرفية:

المتطلبات:

1. قائمة فارغة لحفظ المهام
2. قائمة بخيارات (عرض باستمرار): إضافة مهمة، عرض كل المهام، حذف مهمة، إنهاء البرنامج
3. استخدام `while True` مع `break` للتحكم بدورة حياة البرنامج
4. استخدام `enumerate()` لعرض المهام مع أرقامها
5. معالجة الحالات الاستثنائية (حذف مهمة برقم غير موجود)

💻 حل نموذجي كامل

✅ اضغط لرؤية الحل الكامل (حاول بنفسك أولاً!)

Python todo_list.py

```
"""نظام إدارة قائمة مهام تفاعلي بسيط."""

tasks: list[str] = []

def display_menu() -> None:
    print("\n" + "=" * 35)
    print("📋  قائمة المهام")
    print("=" * 35)
    print("1. إضافة مهمة جديدة")
    print("2. عرض كل المهام")
    print("3. حذف مهمة")
    print("4. إنهاء البرنامج")
    print("=" * 35)

while True:
    display_menu()
    choice: str = input("اختر رقمًا: ").strip()

    if choice == "1":
        new_task: str = input("اكتب المهمة الجديدة: ").strip()
        if new_task:
            tasks.append(new_task)
            print(f"✅ تمت إضافة: {new_task}")
        else:
            print("⚠️ لا يمكن إضافة مهمة فارغة!")

    elif choice == "2":
        if not tasks:
            print("📭 لا توجد مهام حاليًا!")
        else:
            print("\n📌 مهامك الحالية:")
            for index, task in enumerate(tasks, start=1):
                print(f"  {index}. {task}")

    elif choice == "3":
        if not tasks:
            print("📭 لا توجد مهام لحذفها!")
        else:
            for index, task in enumerate(tasks, start=1):
                print(f"  {index}. {task}")

            task_number: str = input("أدخل رقم المهمة المراد حذفها: ").strip()
```

```
            if task_number.isdigit() and 1 <= int(task_number) <= len(tasks):
                removed: str = tasks.pop(int(task_number) - 1)
                print(f"🗑️ تم حذف: {removed}")
            else:
                print("⚠️ رقم غير صالح!")

        elif choice == "4":
            print("👋 إلى اللقاء!")
            break

        else:
            print("⚠️ اختيار غير صحيح، حاول مرة أخرى")
```

ملاحظة على الحل: لاحظ استخدامنا لأول `def` (تعريف دالة) في هذا الكتاب - `display_menu()`. لا تقلق إن بدت غامضة؛ سنتعمّق في الدوال بالتفصيل الكامل في الفصل العاشر. هذا تشويق بسيط يُظهر كيف ستُنظّم كودك بشكل أفضل قريبًا.

8.17 اختبر فهمك (Quiz سريع)

1. ما الفرق الجوهري بين القوائم (List) والنصوص (String) من حيث قابلية التعديل؟
2. لماذا يُسبب `copy_list = original_list` مشاكل غير متوقعة عند تعديل إحداهما؟
3. ما الفرق بين `.sort()` و `sorted()`؟
4. متى تحتاج `copy.deepcopy()` بدلاً من `.copy()` العادية؟
5. ما الفرق الجوهري بين List و Tuple؟

✅ **الإجابات النموذجية**

1. القوائم Mutable (قابلة للتعديل المباشر بعد الإنشاء)، بينما النصوص Immutable (أي عملية «تعديل» تُنشئ نصًا جديدًا بالكامل بدلاً من تغيير الأصل).
2. لأن عملية الإسناد `=` لا تنسخ القائمة فعليًا، بل تجعل المتغير الجديد «ملصقًا» إضافيًا يُشير لنفس القائمة في الذاكرة - أي تعديل عبر أي من الاسمين يؤثر على نفس الكائن المشترك.
3. `.sort()` method تُعدّل القائمة الأصلية مباشرة (in-place) وتُعيد `None`، بينما `sorted()` دالة مستقلة تُعيد قائمة **جديدة** مُرتَّبة دون التأثير على الأصل.
4. عندما تحتوي القائمة على عناصر متداخلة قابلة للتعديل (كقوائم داخل قوائم)، لأن `.copy()` العادية تنسخ فقط المستوى الأول (Shallow)، بينما العناصر الداخلية تبقى مُشتركة مع الأصل.
5. List قابلة للتعديل (Mutable) بعد الإنشاء، بينما Tuple غير قابلة للتعديل (Immutable) - تُستخدم Tuple عادة للبيانات التي من المنطقي أن تبقى ثابتة طوال حياة البرنامج.

8.18 📌 ملخص الفصل (Cheat Sheet)

```python
# الإنشاء
my_list: list[int] = [1, 2, 3]
my_tuple: tuple[int, ...] = (1, 2, 3)

# الإضافة/الحذف
my_list.append(4)
my_list.insert(0, 99)
my_list.remove(2)        # بالقيمة
my_list.pop(0)           # بالفهرس، تُعيد القيمة

# الترتيب
my_list.sort()           # يُعدّل الأصل، يُعيد None
sorted(my_list)          # نسخة جديدة مُرتَّبة

# النسخ (مهم جدًا!)
copy = original          # ⚠️ مرجع فقط، ليس نسخة!
copy = original.copy()   # نسخة سطحية
import copy as cp
deep = cp.deepcopy(original)  # نسخة عميقة كاملة

# List Comprehension
[x**2 for x in range(5)]
[x for x in range(10) if x % 2 == 0]

# دوال مفيدة
len(my_list), sum(my_list), max(my_list), min(my_list)
zip(list1, list2)
enumerate(my_list, start=1)
```

⚠️ تذكّر دائمًا

- القوائم Mutable، النصوص والـ Tuples ليست كذلك
- `=` لا تنسخ القوائم - استخدم `.copy()` أو `deepcopy()`
- `.sort()` تُعدّل وتُعيد `None` - `sorted()` تُعيد نسخة جديدة
- لا تُعدّل قائمة أثناء التكرار المباشر عليها

🎯 ماذا بعد؟

القوائم رائعة، لكنها ليست الحل الأمثل لكل مشكلة. ماذا لو أردت التأكد من عدم تكرار أي عنصر تلقائيًا؟ أو تخزين بيانات بصيغة «مفتاح → قيمة» (مثل اسم ورقم هاتف)؟ في **الفصل التاسع**، سنتعرّف على **Dictionaries** و **Sets** - هيكلان قويان جدًا، مع جدول مقارنة شامل يوضح متى تستخدم كل هيكل بيانات تعلمناه حتى الآن.

فصل 09 / 26 · الجزء الثاني

09 القواميس والمجموعات (Dict & Set)

8 دقيقة قراءة | 9 قسمًا | 32 مثالًا | 1,554 كلمة

في هذا الفصل

9.1 لماذا نحتاج هيكل بيانات جديد؟
9.2 القواميس (Dictionaries)
9.3 المجموعات (Sets)
9.4 جدول المقارنة الشامل: متى تستخدم ماذا؟
9.5 مشروع تطبيقي: نظام إدارة مخزون بسيط
9.6 تمارين
9.7 اختبر فهمك (Quiz)
9.8 ملخص الفصل (Cheat Sheet)
9.9 ماذا بعد؟

9.1 لماذا نحتاج هيكل بيانات جديد؟

تخيّل أنك تريد تخزين بيانات طالب: اسمه، عمره، درجاته، تخصصه. باستخدام ما تعلمناه في الفصل الثامن، قد تكتب:

```python
student = ["Ahmed", 22, "Computer Science", 85.5]
```

لكن هذا الحل يحمل مشكلة خطيرة: **كيف تعرف أن** `student[0]` **هو الاسم و** `student[1]` **هو العمر؟** عليك أن تتذكر الترتيب! إذا أراد زميلك في العمل قراءة هذا الكود بعد ستة أشهر، أو إذا أضفت حقلاً جديداً في المنتصف، سينهار كل شيء.

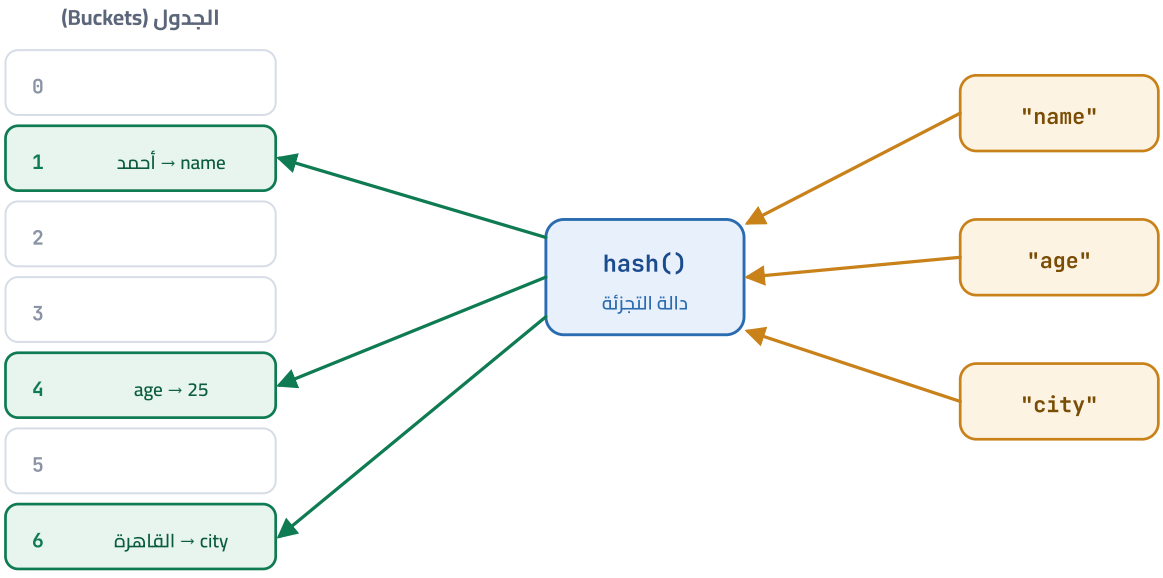
ما نحتاجه هو طريقة لتخزين البيانات **بالاسم** وليس بالموقع. هنا يأتي دور **القاموس (Dictionary)**:

```python
student: dict[str, object] = {
    "name": "Ahmed",
    "age": 22,
    "major": "Computer Science",
    "gpa": 85.5
}

print(student["name"])  # Ahmed - واضح ومباشر!
```

أما المشكلة الثانية التي سنحلها في هذا الفصل: كيف تتأكد أن قائمة أسماء المستخدمين لا تحتوي على تكرار؟ كيف تجد العناصر المشتركة بين قائمتين بسرعة؟ هنا يأتي دور **المجموعة (Set)**.

بنهاية هذا الفصل، ستمتلك الصورة الكاملة لكل هياكل البيانات الأساسية في Python، وستعرف متى تستخدم كل واحد منها بثقة تامة.

9.2 القواميس (Dictionaries)



شكل 9.1 — كيف يصل القاموس للقيمة عبر التجزئة

ما هو القاموس؟

القاموس هو مجموعة من أزواج **«مفتاح: قيمة»** (Key: Value). تماماً كقاموس اللغة الحقيقي: تبحث عن **كلمة** (المفتاح) لتجد **معناها** (القيمة).

```python
# قاموس حقيقي
# "Serendipity" → "اكتشاف شيء جميل بالصدفة"

ages: dict[str, int] = {
    "Sara": 25,
    "Omar": 30,
    "Lina": 28
}

print(ages["Sara"])  # 25
```

خصائص القاموس الجوهرية

الخاصية	الوصف
غير مرتب منطقياً لكنه **يحفظ ترتيب الإدخال**	منذ Python 3.7، القواميس تحافظ على ترتيب إضافة العناصر (لكن لا تعتمد عليه منطقياً كترتيب «أول/آخر» بمعنى الفرز)
قابل للتعديل (Mutable)	يمكنك إضافة/حذف/تعديل العناصر بعد الإنشاء
المفاتيح فريدة	لا يمكن تكرار نفس المفتاح؛ أي قيمة جديدة لمفتاح موجود تستبدل القديمة
المفاتيح يجب أن تكون Immutable	str, int, float, tuple مسموحة كمفاتيح؛ list أو dict ممنوعة
القيم يمكن أن تكون أي شيء	نصوص، أرقام، قوائم، حتى قواميس أخرى

إنشاء القواميس

```python
# الطريقة الأولى: الأقواس المعقوفة
empty_dict: dict = {}
person: dict[str, str] = {"name": "Khaled", "city": "Riyadh"}

# الطريقة الثانية: دالة dict()
person2 = dict(name="Noor", city="Cairo")  # بدون علامات تنصيص للمفاتيح هنا!

# الطريقة الثالثة: من قائمة أزواج (tuples)
pairs = [("a", 1), ("b", 2)]
person3 = dict(pairs)
print(person3)  # {'a': 1, 'b': 2}

# قاموس متداخل (Nested Dictionary) - شائع جداً في البيانات الحقيقية
company: dict[str, dict] = {
    "employee_1": {"name": "Sami", "salary": 8000},
    "employee_2": {"name": "Huda", "salary": 9500}
}
print(company["employee_1"]["name"])  # Sami
```

نصيحة الخبراء: استخدم `dict[str, int]` كـ Type Hint عندما تعرف أنواع المفاتيح والقيم. إذا كانت القيم متنوعة الأنواع (كما في مثال `student` أعلاه)، استخدم `dict[str, object]` أو الأفضل: استخدم **Dataclass** (سنتعلمها لاحقاً في الفصل 13) بدلاً من قاموس بقيم مختلطة.

الوصول للقيم والتعامل مع المفاتيح المفقودة

```python
student = {"name": "Ahmed", "age": 22}

# الطريقة الأولى: الأقواس المربعة - تُطلق خطأ إذا المفتاح غير موجود
print(student["name"])  # Ahmed
# print(student["grade"])  # KeyError: 'grade' 💥

# الطريقة الثانية: get() - آمنة، تُعيد None إذا لم يوجد المفتاح
print(student.get("grade"))        # None
print(student.get("grade", "N/A"))  # N/A - قيمة افتراضية مخصصة
```

تحذير شائع: المبتدئون يستخدمون `student["key"]` دائماً، فتنهار برامجهم فجأة بـ `KeyError` عند أول بيانات غير متوقعة. **القاعدة الذهبية:** إذا لم تكن متأكداً 100% أن المفتاح موجود، استخدم `.get()` دائماً.

Python

```
# التحقق من وجود مفتاح قبل استخدامه
if "grade" in student:
    print(student["grade"])
else:
    print("لا توجد درجة مسجلة")

# in تبحث في المفاتيح فقط، وليس القيم!
print("name" in student)   # True
print("Ahmed" in student)  # False (هذا بحث في القيم، وهو غير مدعوم مباشرة بـ in)
```

إضافة وتعديل وحذف العناصر

Python

```
student = {"name": "Ahmed", "age": 22}

# الإضافة والتعديل بنفس الصيغة
student["age"] = 23           # تعديل (المفتاح موجود)
student["major"] = "AI"       # إضافة (المفتاح جديد)
print(student)
# {'name': 'Ahmed', 'age': 23, 'major': 'AI'}

# الحذف بطريقة del - تطلق خطأ إذا المفتاح غير موجود
del student["age"]

# الحذف بطريقة pop() - آمنة، وتُعيد القيمة المحذوفة
major = student.pop("major")
print(major)  # AI

# pop مع قيمة افتراضية لتجنب الخطأ
removed = student.pop("not_exist", "لا يوجد")
print(removed)  # لا يوجد

# حذف كل العناصر
student.clear()
print(student)  # {}
```

التكرار على القواميس (Looping)

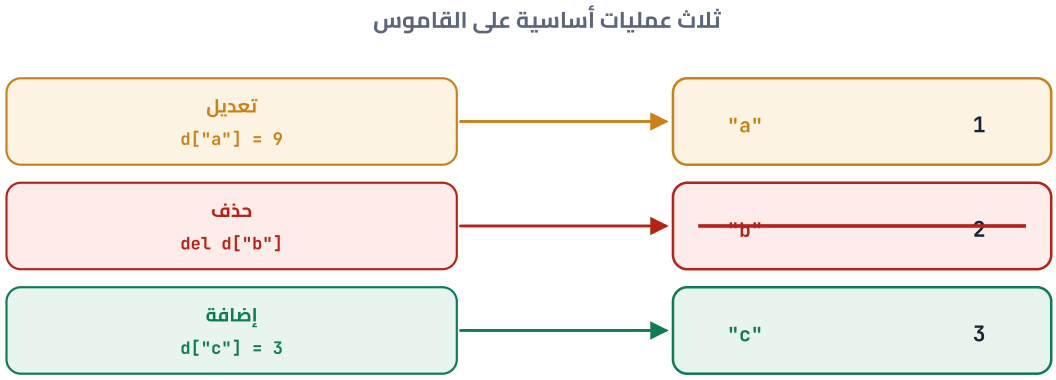
هذه من أهم المهارات العملية:

Python

```python
scores = {"Sara": 90, "Omar": 75, "Lina": 88}

# التكرار على المفاتيح فقط (الافتراضي)
for name in scores:
    print(name)

# التكرار على القيم فقط
for score in scores.values():
    print(score)

# التكرار على المفاتيح والقيم معاً - الأكثر استخداماً عملياً
for name, score in scores.items():
    print(f"{name}: {score}")
```

مقارنة: في JavaScript تكتب `for (const [key, value] of Object.entries(obj))`، وفي Python الطريقة أبسط وأوضح: `for key, value in dict.items()`. هذا أحد الأسباب التي تجعل Python محبوبة في تعليم البرمجة.

أهم Methods القاموس - جدول مرجعي شامل



شكل 9.2 — إضافة وتعديل وحذف في القاموس

Method	الوصف	مثال
`.get(key, default)`	إرجاع القيمة أو قيمة افتراضية	`d.get("x", 0)`
`.keys()`	إرجاع كل المفاتيح	`d.keys()`
`.values()`	إرجاع كل القيم	`d.values()`
`.items()`	إرجاع أزواج (مفتاح, قيمة)	`d.items()`
`.pop(key, default)`	حذف وإرجاع القيمة	`d.pop("x")`
`.update(other_dict)`	دمج قاموس آخر (الكتابة فوق المشترك)	`d.update({"y": 2})`
`.setdefault(key, default)`	إرجاع القيمة، وإنشاؤها إن لم توجد	`d.setdefault("x", [])`
`.copy()`	نسخة سطحية	`d.copy()`
`.clear()`	حذف كل العناصر	`d.clear()`

Python

```python
# .update() - دمج قاموسين
defaults = {"theme": "dark", "lang": "ar", "font_size": 14}
user_settings = {"lang": "en"}

defaults.update(user_settings)
print(defaults)
# {'theme': 'dark', 'lang': 'en', 'font_size': 14}

# .setdefault() - مفيدة جداً لتجميع البيانات
word_groups: dict[str, list[str]] = {}
words = ["apple", "ant", "banana", "bee", "avocado"]

for word in words:
    first_letter = word[0]
    word_groups.setdefault(first_letter, []).append(word)

print(word_groups)
# {'a': ['apple', 'ant', 'avocado'], 'b': ['banana', 'bee']}
```

نصيحة الخبراء: النمط أعلاه (`setdefault`) لتجميع عناصر في قوائم فرعية) نمط شائع جداً في معالجة البيانات. هناك بديل أنظف وأسرع له اسم `defaultdict` من مكتبة `collections`:

```python
from collections import defaultdict

word_groups = defaultdict(list)
for word in words:
    word_groups[word[0]].append(word)  # لا حاجة لـ setdefault!

print(dict(word_groups))
```

Dictionary Comprehension

تماماً مثل List Comprehension، لكن لإنشاء قواميس:

```python
# الصيغة: {key_expr: value_expr for item in iterable}

# مثال 1: تربيع الأرقام
numbers = [1, 2, 3, 4, 5]
squares = {n: n**2 for n in numbers}
print(squares)  # {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}

# مثال 2: مع شرط
even_squares = {n: n**2 for n in numbers if n % 2 == 0}
print(even_squares)  # {2: 4, 4: 16}

# مثال 3: عكس قاموس (قلب المفاتيح والقيم)
original = {"a": 1, "b": 2, "c": 3}
reversed_dict = {value: key for key, value in original.items()}
print(reversed_dict)  # {1: 'a', 2: 'b', 3: 'c'}

# مثال 4: تحويل قائمتين متوازيتين إلى قاموس باستخدام zip
names = ["Sara", "Omar", "Lina"]
scores = [90, 75, 88]
result = {name: score for name, score in zip(names, scores)}
print(result)  # {'Sara': 90, 'Omar': 75, 'Lina': 88}
```

⚠ **خطأ شائع**: استخدام قيمة قابلة للتعديل (mutable) كمفتاح:

```python
# d = {[1, 2]: "value"}  # TypeError: unhashable type: 'list'
d = {(1, 2): "value"}  # صحيح! tuple غير قابلة للتعديل، لذا مسموحة كمفتاح
```

السبب التقني: المفاتيح يجب أن تكون **Hashable** (قابلة للتحويل إلى قيمة hash ثابتة)، والقيم القابلة للتعديل (list, dict, set) لا تملك hash ثابتاً لأن محتواها قد يتغير.

9.3 المجموعات (Sets)

شكل 9.3 — اتحاد وتقاطع وفرق المجموعات

ما هي المجموعة؟

المجموعة هي تجميعة من عناصر **فريدة** (بدون تكرار) **وغير مرتبة**. فكّر بها كصندوق تضع فيه أشياء، لكن لا يمكن أن يحتوي على نفس الشيء مرتين، ولا يهمك ترتيبه.

```python
fruits: set[str] = {"apple", "banana", "apple", "orange"}
print(fruits)
# {'apple', 'banana', 'orange'}  -- لاحظ التكرار اختفى تلقائياً!
```

هذه هي الاستخدام الأول والأشهر للمجموعات: **إزالة التكرار بسهولة**.

```python
numbers = [1, 2, 2, 3, 4, 4, 4, 5]
unique_numbers = list(set(numbers))
print(unique_numbers)  # [1, 2, 3, 4, 5] (الترتيب قد يختلف)
```

⚠ **تحذير:** المجموعات **غير مرتبة**، فلا تعتمد على ترتيب معين عند تحويل قائمة إلى set ثم إعادتها إلى list. إذا احتجت الحفاظ على الترتيب مع إزالة التكرار، استخدم:

```python
numbers = [3, 1, 2, 3, 1, 4]
unique_ordered = list(dict.fromkeys(numbers))
print(unique_ordered)  # [3, 1, 2, 4] - الترتيب محفوظ!
```

إنشاء المجموعات

```python
# الأقواس المعقوفة (بدون نقطتين - هذا ما يميزها عن dict)
s1: set[int] = {1, 2, 3}

# دالة set() - ضرورية للمجموعة الفارغة!
s2 = set()

# ⚠ تحذير حرج
empty = {}         # هذا قاموس فارغ وليس مجموعة فارغة!
print(type(empty))  # <class 'dict'>

# set() من قائمة
s3 = set([1, 2, 2, 3])
print(s3)  # {1, 2, 3}
```

⚠ **خطأ شائع جداً عند المبتدئين:** كتابة `{}` معتقدين أنها مجموعة فارغة، بينما Python تفسرها كقاموس فارغ دائماً. **لإنشاء مجموعة فارغة استخدم `set()` فقط.**

العمليات الأساسية على المجموعات

```python
colors: set[str] = {"red", "green", "blue"}

# إضافة عنصر واحد
colors.add("yellow")

# إضافة عدة عناصر
colors.update(["purple", "pink"])

# حذف عنصر - يطلق خطأ إذا غير موجود
colors.remove("red")

# حذف آمن - لا يطلق خطأ
colors.discard("not_exist")  # لا شيء يحدث

# حذف عنصر عشوائي وإرجاعه
item = colors.pop()

# التحقق من وجود عنصر - سريع جداً (انظر الأداء أدناه)
print("blue" in colors)
```

العمليات الرياضية على المجموعات (القوة الحقيقية)

هنا يكمن السر الحقيقي لأهمية المجموعات: **عمليات نظرية المجموعات الرياضية**.

Python

```
python_devs = {"Ahmed", "Sara", "Omar", "Lina"}
js_devs = {"Omar", "Lina", "Khaled", "Noor"}

# الاتحاد (Union): كل العناصر من الاثنين بدون تكرار
all_devs = python_devs | js_devs
# أو: python_devs.union(js_devs)
print(all_devs)
# {'Ahmed', 'Sara', 'Omar', 'Lina', 'Khaled', 'Noor'}

# التقاطع (Intersection): العناصر المشتركة فقط
both_devs = python_devs & js_devs
# أو: python_devs.intersection(js_devs)
print(both_devs)
# {'Omar', 'Lina'}

# الفرق (Difference): في الأولى وليس في الثانية
only_python = python_devs - js_devs
# أو: python_devs.difference(js_devs)
print(only_python)
# {'Ahmed', 'Sara'}

# الفرق التماثلي (Symmetric Difference): في أحدهما فقط وليس الاثنين
exclusive = python_devs ^ js_devs
# أو: python_devs.symmetric_difference(js_devs)
print(exclusive)
# {'Ahmed', 'Sara', 'Khaled', 'Noor'}
```

تطبيق عملي واقعي: تخيل نظام توصيات في موقع تسوق إلكتروني:

Python

```
user_a_purchases = {"laptop", "mouse", "keyboard"}
user_b_purchases = {"mouse", "keyboard", "monitor", "webcam"}

# ماذا اشترى B ولم يشتره A؟ (توصية محتملة لـ A)
recommendations_for_a = user_b_purchases - user_a_purchases
print(recommendations_for_a)  # {'monitor', 'webcam'}
```

```python
# فحص العلاقات بين المجموعات
a = {1, 2, 3}
b = {1, 2, 3, 4, 5}

print(a.issubset(b))      # True -- a مجموعة جزئية من b
print(b.issuperset(a))    # True -- b تحتوي كل عناصر a
print(a.isdisjoint({9, 10}))  # True -- لا يوجد أي عنصر مشترك
```

لماذا المجموعات سريعة جداً؟

مقارنة الأداء - نظرة تحت غطاء المحرك

العملية	List	Set
البحث عن عنصر (`x in collection`)	O(n) - يفحص كل عنصر تباعاً	O(1) - وصول مباشر تقريباً
إضافة عنصر	O(1) في الآخر	O(1)
إزالة التكرار	تحتاج كود يدوي	تلقائي بالتعريف

```python
import time

big_list = list(range(1_000_000))
big_set = set(range(1_000_000))

start = time.perf_counter()
999_999 in big_list
print(f"List: {time.perf_counter() - start:.6f} ثانية")

start = time.perf_counter()
999_999 in big_set
print(f"Set: {time.perf_counter() - start:.6f} ثانية")

# النتيجة التقريبية: List أبطأ بآلاف المرات!
```

السبب التقني: القوائم تبحث عنصراً تلو الآخر (Linear Search)، بينما المجموعات والقواميس تستخدم بنية بيانات تسمى **Hash Table** تحسب «عنوان» كل عنصر مباشرة، فتصل إليه فوراً تقريباً بغض النظر عن حجم المجموعة.

نصيحة الخبراء: إذا كان كودك يحتوي على سطر مثل `if x in my_list:` داخل حلقة تتكرر آلاف المرات، وكانت `my_list` كبيرة، **حوّلها إلى set أولاً**:

```python
# بطيء مع قوائم كبيرة
allowed_ids = [1, 5, 10, 15, ..., 50000]  # قائمة ضخمة
for user_id in incoming_ids:
    if user_id in allowed_ids:  # بحث خطي بطيء كل مرة!
        process(user_id)

# سريع جداً
allowed_ids_set = set(allowed_ids)  # تحويل مرة واحدة
for user_id in incoming_ids:
    if user_id in allowed_ids_set:  # بحث فوري!
        process(user_id)
```

Set Comprehension

```python
numbers = [1, 2, 2, 3, 4, 4, 5]
squared_unique = {n**2 for n in numbers}
print(squared_unique)  # {1, 4, 9, 16, 25}
```

Frozenset: المجموعة غير القابلة للتعديل

```python
# frozenset مثل set لكنها immutable - يمكن استخدامها كمفتاح قاموس!
frozen = frozenset([1, 2, 3])
# frozen.add(4)  # AttributeError - لا توجد طريقة للتعديل

cache: dict[frozenset, str] = {
    frozenset([1, 2]): "combo A",
    frozenset([3, 4]): "combo B"
}
```

9.4 جدول المقارنة الشامل: متى تستخدم ماذا؟

هذا هو الجدول الذي وعدناك به في نهاية الفصل السابق. ادرسه جيداً، وارجع إليه كلما ترددت في اختيار هيكل البيانات المناسب.

الخاصية	List `[]`	Tuple `()`	Dict `{k: v}`	Set
قابل للتعديل؟	✅ نعم	❌ لا	✅ نعم	✅
مرتب (يحفظ الترتيب)؟	✅ نعم	✅ نعم	✅ نعم (منذ 3.7)	❌
يسمح بالتكرار؟	✅ نعم	✅ نعم	المفاتيح: ❌ / القيم: ✅	❌
الوصول للعنصر	بالموقع `[0]`	بالموقع `[0]`	بالمفتاح `["key"]`	لا يـ
سرعة البحث `in`	بطيء O(n)	بطيء O(n)	سريع جداً O(1)	سرـ
الاستخدام المثالي	تسلسل عناصر قد يتغير (قائمة مهام، نتائج)	بيانات ثابتة لا تتغير (إحداثيات، تاريخ ميلاد)	بيانات مرتبطة بأسماء/معرّفات (سجل مستخدم، إعدادات)	ضـ مجـ
مثال واقعي	`cart_items = ["pen", "book"]`	`point = (3, 4)`	`user = {"name": "Ali"}`	`'}`
Comprehension	`[x for x in ...]`	غير مباشر (generator)	`{k: v for ...}`	`}`

🧭 خريطة قرار سريعة

```
هل البيانات ثابتة لن تتغير أبداً؟
├── نعم → Tuple
└── لا
    ├── هل أحتاج الوصول بالاسم/المفتاح وليس بالرقم؟
    │   └── نعم → Dict
    └── لا
        ├── هل يهمني منع التكرار أو عمل عمليات رياضية (تقاطع/اتحاد)؟
        │   └── نعم → Set
        └── لا → List (الخيار الافتراضي الأكثر مرونة)
```

> **مقارنة مع لغات أخرى**: في JavaScript، `Object` يقابل `dict`، و `Array` يقابل `list`، و `Set` موجود بنفس الاسم والمفهوم تقريباً. في Java، `HashMap` يقابل `dict`، و `HashSet` يقابل `set`. المفاهيم عالمية؛ فقط الأسماء والصياغة تختلف.

9.5 مشروع تطبيقي: نظام إدارة مخزون بسيط

سنبني نظاماً لإدارة مخزون متجر صغير، يستخدم القواميس للمنتجات، والمجموعات لتتبع الفئات الفريدة.

المتطلبات:

1. تخزين المنتجات: الاسم، السعر، الكمية، الفئة
2. إضافة منتج جديد
3. تحديث كمية منتج (بيع/شراء)
4. عرض كل المنتجات التي كميتها أقل من حد معين (تنبيه نفاد المخزون)
5. عرض كل الفئات الفريدة الموجودة في المخزون
6. حساب القيمة الإجمالية للمخزون

Python

```
inventory: dict[str, dict[str, object]] = {}

def add_product(name: str, price: float, quantity: int, category: str) → None:
    """إضافة منتج جديد إلى المخزون."""
    if name in inventory:
        print(f"⚠️ المنتج '{name}' موجود مسبقاً. استخدم update_quantity للتعديل.")
        return
    inventory[name] = {
        "price": price,
        "quantity": quantity,
        "category": category
    }
    print(f"✅ تمت إضافة '{name}' بنجاح.")

def update_quantity(name: str, change: int) → None:
    """تحديث كمية منتج (موجب للشراء، سالب للبيع)."""
    if name not in inventory:
        print(f"❌ المنتج '{name}' غير موجود.")
        return

    new_quantity = inventory[name]["quantity"] + change
    if new_quantity < 0:
        print(f"❌ لا توجد كمية كافية من '{name}'.")
        return

    inventory[name]["quantity"] = new_quantity
    print(f"✅ كمية '{name}' الآن: {new_quantity}")

def low_stock_alert(threshold: int = 5) → list[str]:
    """إرجاع قائمة بأسماء المنتجات التي كميتها أقل من الحد المسموح."""
    low_items = [
        name for name, info in inventory.items()
        if info["quantity"] < threshold
    ]
    return low_items

def unique_categories() → set[str]:
    """إرجاع كل الفئات الفريدة الموجودة في المخزون."""
```

```
    return {info["category"] for info in inventory.values()}

def total_inventory_value() -> float:
    """حساب القيمة الإجمالية لكل المخزون (السعر × الكمية)."""
    total = sum(
        info["price"] * info["quantity"]
        for info in inventory.values()
    )
    return total

def print_report() -> None:
    """طباعة تقرير شامل عن حالة المخزون."""
    print("\n" + "=" * 40)
    print("📊 تقرير المخزون")
    print("=" * 40)

    for name, info in inventory.items():
        print(f"- {name}: {info['quantity']} قطعة | "
              f"{info['price']:.2f} ر.س | الفئة: {info['category']}")

    print("-" * 40)
    print(f"💰 القيمة الإجمالية: {total_inventory_value():,.2f} ر.س")
    print(f"🏷️  الفئات المتاحة: {', '.join(unique_categories())}")

    low_items = low_stock_alert(threshold=5)
    if low_items:
        print(f"⚠️  تنبيه نفاد المخزون: {', '.join(low_items)}")
    else:
        print("✅ لا توجد منتجات على وشك النفاد.")
    print("=" * 40)

# --- تشغيل تجريبي للنظام ---
if __name__ == "__main__":
    add_product("لابتوب", 3500.0, 10, "إلكترونيات")
    add_product("ماوس", 50.0, 3, "إلكترونيات")
    add_product("كرسي مكتب", 450.0, 8, "أثاث")
    add_product("طاولة", 700.0, 2, "أثاث")

    update_quantity("لابتوب", -2)   # بيع لابتوبين
    update_quantity("ماوس", 10)     # شراء 10 ماوسات جديدة

    print_report()
```

الناتج المتوقع:

```
✅ تمت إضافة 'لابتوب' بنجاح.
✅ تمت إضافة 'ماوس' بنجاح.
✅ تمت إضافة 'كرسي مكتب' بنجاح.
✅ تمت إضافة 'طاولة' بنجاح.
✅ كمية 'لابتوب' الآن: 8
✅ كمية 'ماوس' الآن: 13

========================================
📊 تقرير المخزون
========================================
- لابتوب: 8 قطعة | 3500.00 ر.س | الفئة: إلكترونيات
- ماوس: 13 قطعة | 50.00 ر.س | الفئة: إلكترونيات
- كرسي مكتب: 8 قطعة | 450.00 ر.س | الفئة: أثاث
- طاولة: 2 قطعة | 700.00 ر.س | الفئة: أثاث
----------------------------------------
💰 القيمة الإجمالية: 32,250.00 ر.س
🏷️ الفئات المتاحة: إلكترونيات, أثاث
⚠️ تنبيه نفاد المخزون: طاولة
========================================
```

✅ تحدي إضافي: دالة مقارنة مخزونين (الحل)

Python

```python
def compare_inventories(inv_a: dict, inv_b: dict) -> dict[str, set]:
    """مقارنة منتجات مخزونين مختلفين (مثلاً فرعين لنفس المتجر)."""
    products_a = set(inv_a.keys())
    products_b = set(inv_b.keys())

    return {
        "only_in_a": products_a - products_b,
        "only_in_b": products_b - products_a,
        "in_both": products_a & products_b
    }

branch_1 = {"لابتوب": {}, "ماوس": {}, "كرسي": {}}
branch_2 = {"ماوس": {}, "كرسي": {}, "طابعة": {}}

result = compare_inventories(branch_1, branch_2)
print(result)
# {'only_in_a': {'لابتوب'}, 'only_in_b': {'طابعة'}, 'in_both': {'ماوس', 'كرسي'}}
```

9.6 تمارين

🟢 مستوى مبتدئ

تمرين 1: أنشئ قاموساً باسم `car` يحتوي على: `brand` , `model` , `year` . ثم اطبع كل قيمة باستخدام f-string.

تمرين 2: لديك القائمة `visitors = ["Ali", "Sara", "Ali", "Omar", "Sara", "Ali"]` . استخدم set لمعرفة عدد الزوار الفريدين (بدون تكرار) واطبع العدد.

تمرين 3: أنشئ قاموساً فارغاً، ثم أضف إليه 3 عناصر باستخدام `[]` ، ثم احذف عنصراً واحداً باستخدام `.pop()` .

✅ الحل

Python

```python
# تمرين 1
car = {"brand": "Toyota", "model": "Corolla", "year": 2023}
print(f"الماركة: {car['brand']}")
print(f"الموديل: {car['model']}")
print(f"السنة: {car['year']}")

# تمرين 2
visitors = ["Ali", "Sara", "Ali", "Omar", "Sara", "Ali"]
unique_visitors = set(visitors)
print(f"عدد الزوار الفريدين: {len(unique_visitors)}")  # 3

# تمرين 3
my_dict = {}
my_dict["a"] = 1
my_dict["b"] = 2
my_dict["c"] = 3
removed = my_dict.pop("b")
print(my_dict)  # {'a': 1, 'c': 3}
```

🟡 مستوى متوسط

تمرين 4: اكتب دالة `count_words(text: str) → dict[str, int]` تأخذ نصاً وتُرجع قاموساً بعدد تكرار كل كلمة فيه (تجاهل حالة الأحرف).

تمرين 5: لديك قائمتان:

```python
team_a = {"Ali", "Sara", "Omar"}
team_b = {"Omar", "Lina", "Khaled"}
```

اطبع: من في الفريقين معاً، من في A فقط، من في B فقط، ومن في فريق واحد فقط (وليس الاثنين).

تمرين 6: اكتب دالة `merge_dicts(d1: dict, d2: dict) → dict` تدمج قاموسين، وفي حال وجود مفتاح مشترك، تجمع القيم (بافتراض أنها أرقام).

✅ الحل

Python

```python
# تمرين 4
def count_words(text: str) -> dict[str, int]:
    words = text.lower().split()
    counts: dict[str, int] = {}
    for word in words:
        counts[word] = counts.get(word, 0) + 1
    return counts

result = count_words("Python is great and Python is fun")
print(result)
# {'python': 2, 'is': 2, 'great': 1, 'and': 1, 'fun': 1}

# تمرين 5
team_a = {"Ali", "Sara", "Omar"}
team_b = {"Omar", "Lina", "Khaled"}

print("في الفريقين:", team_a & team_b)
print("في A فقط:", team_a - team_b)
print("في B فقط:", team_b - team_a)
print("في فريق واحد فقط:", team_a ^ team_b)

# تمرين 6
def merge_dicts(d1: dict, d2: dict) -> dict:
    result = d1.copy()
    for key, value in d2.items():
        result[key] = result.get(key, 0) + value
    return result

a = {"apples": 5, "bananas": 3}
b = {"bananas": 2, "oranges": 7}
print(merge_dicts(a, b))
# {'apples': 5, 'bananas': 5, 'oranges': 7}
```

🔴 تحدي

تمرين 7: اكتب برنامجاً يقرأ قائمة طلبات (كل طلب عبارة عن قاموس فيه `product` و `quantity` و `price`)، ويحسب:
- إجمالي المبيعات لكل منتج
- المنتج الأكثر مبيعاً (بالكمية)
- استخدم `defaultdict` من `collections` في الحل.

✅ الحل

Python

```python
from collections import defaultdict

orders = [
    {"product": "قلم", "quantity": 10, "price": 2.0},
    {"product": "دفتر", "quantity": 5, "price": 8.0},
    {"product": "قلم", "quantity": 7, "price": 2.0},
    {"product": "ممحاة", "quantity": 20, "price": 1.0},
    {"product": "دفتر", "quantity": 3, "price": 8.0},
]

sales_total: dict[str, float] = defaultdict(float)
quantity_total: dict[str, int] = defaultdict(int)

for order in orders:
    product = order["product"]
    sales_total[product] += order["quantity"] * order["price"]
    quantity_total[product] += order["quantity"]

print("💰 إجمالي المبيعات لكل منتج:")
for product, total in sales_total.items():
    print(f"  {product}: {total:.2f} ر.س")

best_seller = max(quantity_total, key=quantity_total.get)
print(f"\n🏆 المنتج الأكثر مبيعاً: {best_seller} ({quantity_total[best_seller]} قطعة)")
```

الناتج:

```
💰 إجمالي المبيعات لكل منتج:
  قلم: 34.00 ر.س
  دفتر: 64.00 ر.س
  ممحاة: 20.00 ر.س

🏆 المنتج الأكثر مبيعاً: ممحاة (20 قطعة)
```

💡 ملاحظة: `max(dict, key=dict.get)` نمط احترافي شائع جداً للعثور على المفتاح صاحب أكبر قيمة في القاموس.

9.7 اختبر فهمك (Quiz)

السؤال 1: ما الفرق بين `{}` و `set()`؟

الإجابة: `{}` تُنشئ **قاموساً فارغاً** دائماً في Python، بينما `set()` هي الطريقة الوحيدة لإنشاء **مجموعة فارغة**. هذا من أكثر الأخطاء شيوعاً عند المبتدئين.

السؤال 2: لماذا لا يمكن استخدام list كمفتاح في dict؟

الإجابة: لأن المفاتيح يجب أن تكون **Hashable** (قابلة لحساب قيمة hash ثابتة لا تتغير). القوائم قابلة للتعديل (mutable)، فلو سمحت Python باستخدامها كمفتاح، ثم عدّلتها، سيختل موقعها في بنية الـ Hash Table الداخلية. لهذا فقط الأنواع غير القابلة للتعديل (str, int, float, tuple, frozenset) مسموحة كمفاتيح.

السؤال 3: ما ناتج الكود التالي؟

```python
a = {1, 2, 3}
b = {3, 4, 5}
print(a ^ b)
```

الإجابة: `{1, 2, 4, 5}` — هذا هو الفرق التماثلي (Symmetric Difference): كل العناصر الموجودة في أحد المجموعتين فقط، وليس في كليهما.

السؤال 4: متى تفضل استخدام .get() بدلاً من []؟

الإجابة: عندما لا تكون متأكداً 100% من وجود المفتاح في القاموس. `[]` تطلق `KeyError` إذا لم يوجد المفتاح وتُوقف البرنامج، بينما `.get()` تُرجع `None` (أو قيمة افتراضية تحددها أنت) بأمان دون توقف البرنامج.

❓ السؤال 5: ما الفرق بين البحث عن عنصر في list مقابل set من ناحية الأداء؟

الإجابة: البحث في list هو O(n) — أي أن Python تفحص العناصر واحداً تلو الآخر حتى تجد المطلوب أو تصل للنهاية، فكلما كبرت القائمة طال وقت البحث. البحث في set (وكذلك dict) هو O(1) تقريباً — أي وقت شبه ثابت بغض النظر عن حجم المجموعة، بفضل بنية Hash Table الداخلية.

9.8 ملخص الفصل (Cheat Sheet)

```python
# ============ DICT ============
d = {"key": "value"}                # إنشاء
d["key"]                            # وصول (يطلق KeyError إن لم يوجد)
d.get("key", "default")             # وصول آمن
d["new_key"] = "value"              # إضافة/تعديل
del d["key"]  /  d.pop("key")       # حذف
"key" in d                          # فحص وجود مفتاح
for k, v in d.items(): ...          # تكرار على المفاتيح والقيم
{k: v for k, v in ...}              # Dict Comprehension
d.update(other_dict)                # دمج قاموسين
d.setdefault(k, default)            # إرجاع أو إنشاء افتراضي

from collections import defaultdict
dd = defaultdict(list)              # قاموس بقيمة افتراضية تلقائية

# ============ SET ============
s = {1, 2, 3}                       # إنشاء (وحدها {} :لاحظ = dict فارغ!)
empty_set = set()                   # مجموعة فارغة - الطريقة الوحيدة
s.add(4)                            # إضافة عنصر
s.remove(1)  /  s.discard(1)        # حذف (remove يطلق خطأ، discard آمن)
a | b   # union() - اتحاد
a & b   # intersection() - تقاطع
a - b   # difference() - فرق
a ^ b   # symmetric_difference() - فرق تماثلي
{x for x in ...}                    # Set Comprehension
```

المهمة	الحل الأمثل
بيانات مرتبطة بأسماء (user, config)	`dict`
إزالة التكرار من قائمة	`set(my_list)`
بحث متكرر وسريع عن عناصر	`set` بدلاً من `list`
تجميع عناصر حسب خاصية	`dict` + `setdefault` أو `defaultdict`
مقارنة مجموعتين (مشترك/فريد)	عمليات `set`: `& - | ^`

9.9 ماذا بعد؟

الآن تمتلك ترسانة كاملة من هياكل البيانات: **List** للتسلسلات المرنة، **Tuple** للبيانات الثابتة، **Dict** للبيانات المرتبطة بالأسماء، و**Set** لضمان التفرد والعمليات الرياضية.

لكن حتى الآن، كل برامجنا كانت عبارة عن كود «مسطّح» يُنفَّذ من أعلى لأسفل. ماذا لو أردنا إعادة استخدام نفس الكود عدة مرات دون نسخه ولصقه في كل مرة؟ ماذا لو أردنا تقسيم برنامج كبير إلى قطع صغيرة منظمة وسهلة الفهم والاختبار؟

هذا بالضبط ما ستتعلمه في **الفصل العاشر: الدوال - تنظيم الكود**. ستتعلم كيف تكتب كودك مرة واحدة وتستخدمه ألف مرة، وكيف يفكر المبرمجون المحترفون في «تقسيم المشكلة» إلى دوال صغيرة قابلة لإعادة الاستخدام — وهي من أهم المهارات التي تفصل المبرمج المبتدئ عن المحترف.

فصل 10 / 26 · الجزء الثاني

10

الدوال - تنظيم الكود

9 دقيقة قراءة | 13 قسمًا | 41 مثالًا | 1,750 كلمة

في هذا الفصل

10.1 لماذا تُعتبر الدوال نقطة تحوّل في رحلتك البرمجية؟
10.2 أساسيات تعريف واستدعاء الدالة
10.3 المعاملات (Parameters) بعمق
10.4 القيمة المُرجعة (Return Values) بعمق
10.5 النطاق (Scope): أين «تعيش» المتغيرات؟
10.6 تمرير المعاملات: القيمة مقابل المرجع
10.7 دوال من الدرجة الأولى (First-Class Functions) - مقدمة
10.8 Type Hints المتقدمة في الدوال
10.9 مشروع تطبيقي: حاسبة علمية بالدوال
10.10 تمارين
10.11 اختبر فهمك (Quiz)
10.12 ملخص الفصل (Cheat Sheet)
10.13 ماذا بعد؟

10.1 لماذا تُعتبر الدوال نقطة تحوّل في رحلتك البرمجية؟

تخيّل أنك تكتب برنامجاً لحساب مساحة دائرة في خمسة أماكن مختلفة من الكود. إذا احتجت لاحقاً تعديل طريقة الحساب (مثلاً لتحسين الدقة)، ستحتاج للبحث والتعديل في خمسة أماكن! وإذا نسيت مكاناً واحداً، سيظهر خطأ غامض يصعب تتبعه.

هذا بالضبط ما تحله **الدوال (Functions)**: بدلاً من تكرار نفس الكود، تكتبه **مرة واحدة** بداخل دالة، ثم «تستدعيها» (تطلب تنفيذها) من أي مكان تحتاجه، بقدر ما تشاء من المرات.

Python

```python
# بدون دالة - تكرار مزعج وخطير
radius1 = 5
area1 = 3.14159 * radius1 ** 2
print(f"المساحة: {area1}")

radius2 = 8
area2 = 3.14159 * radius2 ** 2
print(f"المساحة: {area2}")

# مع دالة - اكتب مرة، استخدم دائماً
def calculate_circle_area(radius: float) -> float:
    return 3.14159 * radius ** 2

print(f"المساحة: {calculate_circle_area(5)}")
print(f"المساحة: {calculate_circle_area(8)}")
```

الدوال ليست مجرد «اختصار لتقليل الكتابة» — إنها أداة تفكير. المبرمج المحترف ينظر لأي مشكلة كبيرة ويسأل: «كيف أقسّمها إلى قطع أصغر، كل قطعة تقوم بمهمة واحدة واضحة؟» هذا المبدأ يُسمى **Separation of Concerns** (فصل الاهتمامات)، وهو حجر الأساس لكل كود احترافي، صغيراً كان أو ضخماً بحجم أنظمة شركات عملاقة.

بنهاية هذا الفصل، ستكتب دوالاً احترافية، تفهم كيف تتدفق البيانات داخل وخارج الدالة، وتتقن أحد أهم المفاهيم في Python: **النطاق (Scope)**.

10.2 أساسيات تعريف واستدعاء الدالة

كل استدعاء يضيف إطارًا أعلى المكدّس، وكل return يحذفه.

شكل 10.1 — المكدّس أثناء الاستدعاءات المتداخلة

الصيغة الأساسية

```python
def function_name(parameters) -> return_type:
    """Docstring: وصف مختصر لما تفعله الدالة."""
    # جسم الدالة
    return value
```

```python
def greet(name: str) -> str:
    """تُرجع رسالة ترحيب للمستخدم."""
    return f"أهلاً بك، {name}!"

message = greet("سارة")
print(message)  # أهلاً بك، سارة!
```

تشريح الدالة:

الجزء	الوصف
`def`	كلمة مفتاحية تُعلن بداية تعريف دالة
`greet`	اسم الدالة (نفس قواعد تسمية المتغيرات: snake_case)
`(name: str)`	المعاملات (Parameters) مع Type Hints
`→ str`	Type Hint لنوع القيمة المُرجعة
`"""..."""`	Docstring - توثيق وظيفة الدالة
`return`	القيمة التي تُعاد للمستدعي

نصيحة الخبراء: اختر أسماء دوال تبدأ بفعل يصف ما تفعله: `calculate_total()` , `is_valid()` , `get_user_data()` , `send_email()` . اسم الدالة الجيد يجعل الكود «يُقرأ كجملة إنجليزية» دون الحاجة لتعليقات إضافية.

Docstrings: التوثيق الاحترافي

Python

```python
def calculate_bmi(weight_kg: float, height_m: float) → float:
    """
    حساب مؤشر كتلة الجسم (BMI).

    المعاملات:
        weight_kg: الوزن بالكيلوغرام
        height_m: الطول بالمتر

    تُرجع:
        قيمة BMI كرقم عشري
    """
    return weight_kg / (height_m ** 2)

# يمكنك قراءة التوثيق في أي وقت باستخدام:
print(calculate_bmi.__doc__)
help(calculate_bmi)
```

نصيحة الخبراء: في VS Code، عندما تكتب docstring بصيغة منظمة (كما أعلاه)، ستظهر هذه المعلومات تلقائياً كـ Tooltip عند استدعاء الدالة في أي مكان بالمشروع — هذا يوفر وقتاً هائلاً لك ولزملائك.

دالة بدون `return`

```python
def print_separator() -> None:
    """تطبع خطاً فاصلاً. لا ترجع أي قيمة."""
    print("-" * 40)

result = print_separator()
print(result)  # None
```

⚠ **تحذير شائع:** إذا لم تكتب `return` صراحة، الدالة تُرجع `None` تلقائياً. هذا سبب شائع لخطأ "لماذا متغيري أصبح `None`؟" — المبرمج يستدعي دالة تطبع نتيجة لكن ينسى أنها لا «تُرجع" شيئاً فعلياً.

```python
# خطأ شائع
def add(a: int, b: int) -> None:
    print(a + b)  # يطبع فقط، لا يُرجع!

result = add(3, 5)  # result = None وليس 8!
print(result + 1)  # 💥 TypeError: unsupported operand type(s)
```

```python
# الصح
def add(a: int, b: int) -> int:
    return a + b

result = add(3, 5)  # result = 8
print(result + 1)  # 9
```

10.3 المعاملات (Parameters) بعمق

Parameters مقابل Arguments

مصطلحان كثيراً ما يُخلط بينهما:

- **Parameter (معامل)**: الاسم المُعرَّف في تعريف الدالة
- **Argument (وسيط)**: القيمة الفعلية المُمرَّرة عند الاستدعاء

```python
def greet(name: str) -> str:  # name = parameter
    return f"Hello, {name}"

greet("Ahmed")  # "Ahmed" = argument
```

المعاملات الموضعية والمسماة (Positional & Keyword Arguments)

```python
def create_profile(name: str, age: int, city: str) -> str:
    return f"{name}, {age} سنة، من {city}"

# موضعية (Positional) - الترتيب مهم جداً
print(create_profile("Ahmed", 25, "Riyadh"))

# مسماة (Keyword) - الترتيب لا يهم
print(create_profile(city="Riyadh", name="Ahmed", age=25))

# مزيج - الموضعية أولاً، ثم المسماة
print(create_profile("Ahmed", city="Riyadh", age=25))
```

⚠ **تحذير:** لا يمكن وضع معامل موضعي بعد معامل مسمى:

```python
# print(create_profile(name="Ahmed", 25, "Riyadh"))
# 💥 SyntaxError: positional argument follows keyword argument
```

💡 **نصيحة الخبراء:** عندما تستدعي دالة لها أكثر من 3 معاملات، استخدم الصيغة المسماة دائماً لتحسين القراءة:

```python
# هنا؟ True و False غامض - ما معنى
send_email("user@test.com", "Subject", "Body", True, False)

# واضح تماماً
send_email(
    to="user@test.com",
    subject="Subject",
    body="Body",
    is_urgent=True,
    send_copy=False
)
```

القيم الافتراضية (Default Parameters)

```python
def greet(name: str, greeting: str = "أهلاً") -> str:
    return f"{greeting}, {name}!"

print(greet("سارة"))              # أهلاً, سارة!
print(greet("عمر", "مرحباً"))     # مرحباً, عمر!
print(greet("لينا", greeting="صباح الخير"))  # صباح الخير, لينا!
```

⚠ **تحذير حرج - فخ القيم الافتراضية القابلة للتعديل (Mutable Default Argument)**

هذا من أخطر وأشهر الأخطاء في Python، حتى بين المبرمجين ذوي الخبرة:

```python
# ❌ خطأ خطير جداً
def add_item(item: str, cart: list = []) -> list:
    cart.append(item)
    return cart

cart1 = add_item("apple")
print(cart1)  # ['apple']

cart2 = add_item("banana")
print(cart2)  # ['apple', 'banana']  💥 لماذا تذكرت العنصر السابق؟!
```

السبب: القيمة الافتراضية `[]` تُنشأ **مرة واحدة فقط** عند تعريف الدالة، وليس في كل استدعاء! لذا كل الاستدعاءات التي لا تمرر `cart` تشترك في **نفس القائمة بالذاكرة**.

```python
# ✅ الحل الصحيح: استخدم None كقيمة افتراضية
def add_item(item: str, cart: list | None = None) -> list:
    if cart is None:
        cart = []  # قائمة جديدة في كل استدعاء
    cart.append(item)
    return cart

cart1 = add_item("apple")
print(cart1)  # ['apple']

cart2 = add_item("banana")
print(cart2)  # ['banana']  ✅ صحيح الآن!
```

مقارنة: هذا السلوك مرتبط بكون Python تُنشئ كائنات القيم الافتراضية مرة واحدة عند **تعريف** الدالة (وقت التحميل)، وليس عند **استدعائها**. لغات أخرى مثل JavaScript لا تملك هذا الفخ لأن طريقة التعامل مع المعاملات الافتراضية تختلف جوهرياً. **القاعدة الذهبية في Python**: لا تستخدم أبداً `list` , `dict` , أو `set` كقيمة افتراضية مباشرة — استخدم `None` دائماً كبديل آمن.

`*args` : عدد غير محدد من المعاملات الموضعية

```python
def calculate_sum(*numbers: float) -> float:
    """تجمع أي عدد من الأرقام الممررة."""
    total = 0
    for num in numbers:
        total += num
    return total

print(calculate_sum(1, 2, 3))          # 6
print(calculate_sum(1, 2, 3, 4, 5))    # 15
print(calculate_sum())                  # 0

# داخل الدالة، numbers تصبح tuple
def show_type(*args):
    print(type(args))  # <class 'tuple'>

show_type(1, 2, 3)
```

`**kwargs` : عدد غير محدد من المعاملات المسماة

```python
def create_user(**details: str) -> dict:
    """تنشئ قاموس مستخدم من أي عدد من الحقول المسماة."""
    return details

user = create_user(name="Ahmed", age=25, city="Jeddah")
print(user)
# {'name': 'Ahmed', 'age': 25, 'city': 'Jeddah'}

# داخل الدالة، kwargs تصبح dict
def show_type(**kwargs):
    print(type(kwargs))  # <class 'dict'>

show_type(a=1, b=2)
```

ترتيب المعاملات الكامل

```python
def full_example(
    positional: str,
    default: int = 10,
    *args: int,
    keyword_only: str = "default_value",
    **kwargs: str
) -> None:
    print(f"positional: {positional}")
    print(f"default: {default}")
    print(f"args: {args}")
    print(f"keyword_only: {keyword_only}")
    print(f"kwargs: {kwargs}")

full_example("A", 20, 1, 2, 3, keyword_only="X", extra="Y")
```

الناتج:

```
positional: A
default: 20
args: (1, 2, 3)
keyword_only: X
kwargs: {'extra': 'Y'}
```

الترتيب الإلزامي	الوصف
1. Positional parameters	معاملات عادية بدون/مع قيمة افتراضية
2. `*args`	معاملات موضعية إضافية غير محددة العدد
3. Keyword-only parameters	يجب تمريرها بالاسم (بعد `*args` أو `*`)
4. `**kwargs`	معاملات مسماة إضافية غير محددة العدد

Positional-Only Arguments (Python 3.8+) g Keyword-Only

Python

```
# استخدام * لإجبار ما بعدها أن تكون keyword-only
def create_order(product: str, *, quantity: int = 1, urgent: bool = False) → str:
    return f"{quantity}x {product} {'(عاجل)' if urgent else ''}"

# print(create_order("Laptop", 2))
# 💥 TypeError: يجب تمرير quantity بالاسم

print(create_order("Laptop", quantity=2, urgent=True))  # ✅ صحيح

# استخدام / لإجبار ما قبلها أن تكون positional-only
def divide(a: float, b: float, /) → float:
    return a / b

print(divide(10, 2))      # ✅ صحيح
# print(divide(a=10, b=2))
# 💥 TypeError: يجب تمريرهما بالموضع فقط a و b
```

نصيحة الخبراء: استخدم `*` لإجبار الوضوح في المعاملات التي قد يُساء فهمها (مثل `urgent=True` بدلاً من `True` غامضة)، واستخدم `/` عندما يكون اسم المعامل تفصيلاً داخلياً لا يهم المستخدم (شائع في مكتبات built-in مثل `len(obj, /)`).

10.4 القيمة المُرجعة (Return Values) بعمق

إرجاع قيم متعددة

```python
def get_min_max(numbers: list[float]) -> tuple[float, float]:
    """تُرجع أصغر وأكبر قيمة في القائمة."""
    return min(numbers), max(numbers)

low, high = get_min_max([5, 2, 9, 1, 7])
print(f"الأصغر: {low}, الأكبر: {high}")  # الأصغر: 1, الأكبر: 9
```

في الحقيقة، Python تُرجع **tuple واحدة** هنا (`(1, 9)`) ثم يحدث **Unpacking** تلقائي إلى المتغيرين.

الخروج المبكر (Early Return / Guard Clauses)

تعلمنا هذا المفهوم في الفصل السادس؛ الدوال هي المكان المثالي لتطبيقه:

```python
def calculate_discount(price: float, is_member: bool) -> float:
    """حساب السعر بعد الخصم."""
    if price <= 0:
        return 0.0  # خروج مبكر - لا داعي لمتابعة الحساب

    if not is_member:
        return price  # خروج مبكر - لا خصم

    return price * 0.85  # الكود الرئيسي، بدون تداخل if متعدد
```

مقارنة: بدون Guard Clauses، سيبدو الكود هكذا مع تداخل مزعج:

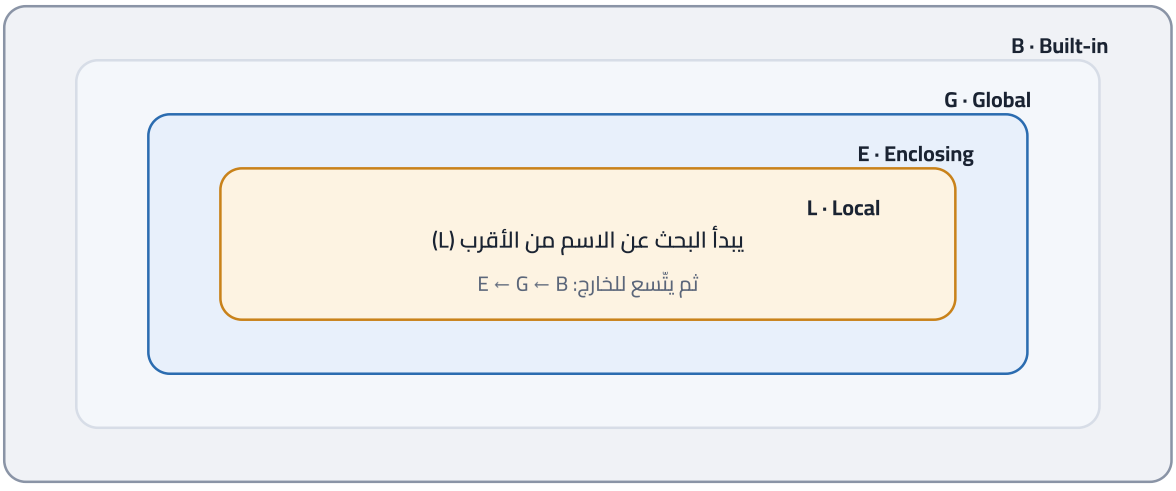
```python
def calculate_discount_messy(price: float, is_member: bool) -> float:
    if price > 0:
        if is_member:
            return price * 0.85
        else:
            return price
    else:
        return 0.0
```

النمط الأول أسهل بكثير للقراءة والصيانة، خاصة مع تعقّد الشروط لاحقاً.

دوال بدون return صريح مقابل `return None`

```python
def log_message(message: str) -> None:
    """تسجل رسالة. لا ترجع شيئاً (None صراحة)."""
    print(f"[LOG]: {message}")
    return None  # اختياري لكنه أوضح من حيث القصد

def validate_age(age: int) -> bool | None:
    """ترجع True/False؛ تتحقق من صحة العمر. إذا كانت القيمة غير منطقية None."""
    if age < 0 or age > 150:
        return None
    return age >= 18
```

10.5 النطاق (Scope): أين «تعيش» المتغيرات؟



شكل 10.2 — ترتيب البحث عن الأسماء: LEGB

هذا من أهم المفاهيم التي يسيء المبتدئون فهمها. **النطاق** يحدد أين يمكن الوصول لمتغير معين في الكود.

Global Scope g Local Scope

```python
x = 10  # متغير عام (Global)

def my_function():
    y = 20  # متغير محلي (Local) - يعيش فقط داخل الدالة
    print(x)  # يمكن قراءة المتغير العام من داخل الدالة
    print(y)

my_function()
print(x)  # 10 - متاح
# print(y)
# 💥 NameError: name 'y' is not defined -- y لا توجد خارج الدالة!
```

⚠ **تحذير شائع**: المبتدئون يستغربون لماذا متغير عرّفوه داخل دالة «يختفي» خارجها. هذا ليس خطأ — إنه التصميم المقصود للنطاق، ويمنع الفوضى في البرامج الكبيرة.

لماذا لا يمكن تعديل متغير عام من داخل دالة مباشرة؟

```python
counter = 0

def increment():
    counter += 1  # 💥 UnboundLocalError!

increment()
```

لماذا يحدث هذا الخطأ؟ لأن Python عندما ترى `counter += 1` داخل الدالة، تفترض أن متغير **محلي** جديد (لأنك تحاول الكتابة إليه)، لكنك تحاول قراءته قبل تعريفه محلياً، فتحدث مشكلة.

الكلمة المفتاحية `global`

```python
counter = 0

def increment():
    global counter  # أخبر Python: استخدم المتغير العام، لا تنشئ واحداً محلياً
    counter += 1

increment()
increment()
print(counter)  # 2
```

⚠ **تحذير احترافي:** استخدام `global` يُعتبر من أفضل الممارسات **السيئة** (Anti-pattern) في البرمجة الحديثة! لأنه يجعل الدوال «تعتمد» على حالة خارجية خفية، مما يصعّب تتبع الأخطاء وفهم الكود. **البديل الأفضل دائماً:**

```python
# ✅ الطريقة الاحترافية: مرر القيمة وأعد النتيجة
def increment(counter: int) -> int:
    return counter + 1

counter = 0
counter = increment(counter)
counter = increment(counter)
print(counter)  # 2
```

🔄 **مقارنة:** في البرمجة الكائنية (الفصل 13)، سنرى حلاً أنظف بكثير لهذه المشكلة باستخدام `self.counter` داخل الكائنات (Objects)، مما يلغي الحاجة لـ `global` نهائياً في الكود الاحترافي.

قاعدة LEGB

Python تبحث عن أي متغير بالترتيب التالي:

المستوى	الاسم	الوصف
1	**L**ocal	داخل الدالة الحالية
2	**E**nclosing	داخل أي دالة «محيطة» (دوال متداخلة - الفصل 12)
3	**G**lobal	على مستوى الملف كله
4	**B**uilt-in	الأسماء المدمجة في Python مثل `len` , `print`

Python

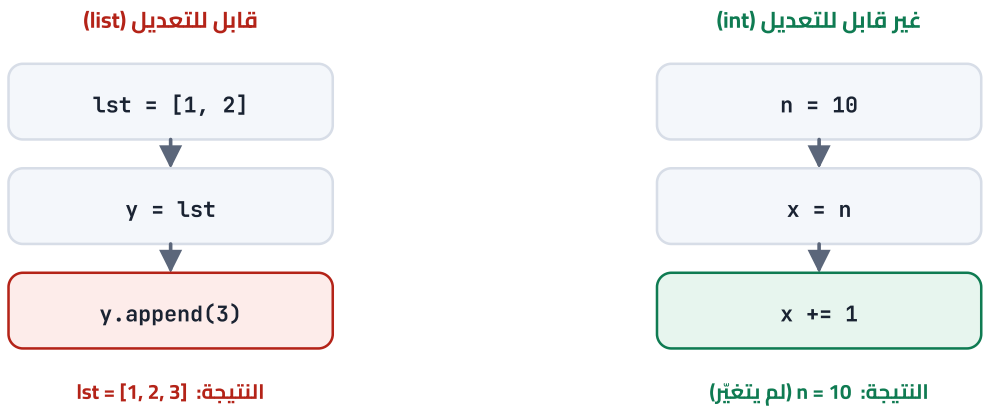
```
x = "global x"

def outer():
    x = "enclosing x"

    def inner():
        x = "local x"
        print(x)  # local x - توجد في المستوى المحلي، تتوقف هنا

    inner()
    print(x)  # enclosing x

outer()
print(x)  # global x
```

نصيحة الخبراء: كلما قلّت اعتمادية دالتك على متغيرات خارجية (Global)، وكلما اعتمدت فقط على معاملاتها (Parameters)، أصبحت أسهل للاختبار، وأسهل لإعادة الاستخدام، وأقل عرضة للأخطاء الغامضة. هذا المبدأ يُسمى **Pure Functions** (الدوال النقية): الدالة التي تُعطي نفس النتيجة دائماً لنفس المدخلات، ولا تُغيّر أي شيء خارج نطاقها.

Python

```
# دالة نقية (Pure Function) - يُفضّل هذا النمط
def add_tax(price: float, tax_rate: float) -> float:
    return price * (1 + tax_rate)

# دالة غير نقية - تعتمد على وتُعدّل حالة خارجية
total = 0

def add_to_total(amount: float) -> None:
    global total
    total += amount
```

10.6 تمرير المعاملات: القيمة مقابل المرجع



تمرير المعامل يمرّر المرجع؛ لكن إعادة الإسناد داخل الدالة لا تمسّ المتغيّر الأصلي.

شكل 10.3 — تمرير المعاملات: القابل للتعديل وغير القابل

هذا المفهوم يُربك الكثير من المبرمجين القادمين من لغات أخرى.

```python
def modify_number(n: int) -> None:
    n += 10
    print(f"داخل الدالة: {n}")

x = 5
modify_number(x)
print(f"خارج الدالة: {x}")

# الناتج:
# داخل الدالة: 15
# خارج الدالة: 5    !لم يتغير--
```

لكن مع القوائم (كائنات قابلة للتعديل):

```python
def modify_list(lst: list) -> None:
    lst.append(100)
    print(f"داخل الدالة: {lst}")

my_list = [1, 2, 3]
modify_list(my_list)
print(f"خارج الدالة: {my_list}")

# الناتج:
# داخل الدالة: [1, 2, 3, 100]
# خارج الدالة: [1, 2, 3, 100]  -- تغيّرت فعلاً!
```

السبب: Python تُمرر دائماً «مرجع» (Reference) للكائن، لكن التأثير يختلف حسب نوع الكائن:
- الأنواع **غير القابلة للتعديل** (int, float, str, tuple): أي عملية كـ `n += 10` تُنشئ كائناً **جديداً** بالكامل، فالمتغير الأصلي خارج الدالة لا يتأثر.
- الأنواع **القابلة للتعديل** (list, dict, set): العمليات مثل `.append()` تُعدّل **نفس الكائن** في الذاكرة، فالتغيير يظهر خارج الدالة أيضاً.

⚠ **تحذير حرج**: هذا السلوك مصدر شائع جداً للأخطاء الغامضة. إذا مررت قائمة لدالة ولم تقصد تعديلها، انسخها أولاً:

```python
def process_safely(lst: list) -> list:
    lst_copy = lst.copy()  # اعمل على نسخة
    lst_copy.append(999)
    return lst_copy
```

🔄 **مقارنة مع لغات أخرى**: هذا المفهوم يُسمى أحياناً "Pass by Object Reference"، وهو مختلف عن "Pass by Value" الصِرف (كما في C عند تمرير أنواع بسيطة) و"Pass by Reference" الصِرف (كما في بعض حالات C++). Python تقف في منتصف الطريق: المرجع نفسه يُمرر بالقيمة، لكن الكائن المُشار إليه يبقى نفسه.

10.7 دوال من الدرجة الأولى (First-Class Functions) - مقدمة

في Python، الدوال هي «مواطنون من الدرجة الأولى» (First-Class Citizens) — يمكن التعامل معها كأي قيمة أخرى: تخزينها في متغير، تمريرها كمعامل لدالة أخرى، أو إرجاعها من دالة.

```python
def square(x: int) -> int:
    return x ** 2

# تخزين الدالة في متغير (بدون أقواس استدعاء!)
my_func = square
print(my_func(5))  # 25

# تمرير دالة كمعامل لدالة أخرى
def apply_function(func, value: int) -> int:
    return func(value)

print(apply_function(square, 6))  # 36
```

مثال عملي باستخدام `sorted()` مع معامل `key` (دالة):

```python
students = [
    {"name": "Ahmed", "score": 85},
    {"name": "Sara", "score": 92},
    {"name": "Omar", "score": 78}
]

def get_score(student: dict) -> int:
    return student["score"]

sorted_students = sorted(students, key=get_score, reverse=True)
for s in sorted_students:
    print(f"{s['name']}: {s['score']}")

# الناتج:
# Sara: 92
# Ahmed: 85
# Omar: 78
```

هذا تمهيد لمفهوم **Lambda Functions** و**Higher-Order Functions** الذي سنتعمق فيه بالكامل في **الفصل الثاني عشر**.

10.8 Type Hints المتقدمة في الدوال

Python

```python
# استخدام | للسماح بأكثر من نوع (Python 3.10+)
def format_value(value: int | float | str) -> str:
    return f"القيمة: {value}"

# استخدام Optional (أو | None) لمعامل قد يكون غائباً
def find_user(user_id: int) -> dict | None:
    users = {1: {"name": "Ahmed"}, 2: {"name": "Sara"}}
    return users.get(user_id)

# Callable: Type Hint لمعامل يكون دالة
from collections.abc import Callable

def apply_twice(func: Callable[[int], int], value: int) -> int:
    return func(func(value))

def add_five(x: int) -> int:
    return x + 5

print(apply_twice(add_five, 10))  # 20
```

نصيحة الخبراء: استخدم أداة `mypy` (سنتعمق فيها في الفصل 15) للتحقق التلقائي من أن كل استدعاءات دوالك تتوافق مع الـ Type Hints المُعلنة. هذا يكشف أخطاءً كثيرة **قبل** تشغيل الكود فعلياً.

10.9 مشروع تطبيقي: حاسبة علمية بالدوال

سنبني حاسبة علمية مُنظّمة بالكامل بدوال، تدعم العمليات الأساسية والمتقدمة، مع معالجة آمنة للأخطاء الشائعة.

Python

```python
import math

def add(a: float, b: float) -> float:
    """جمع رقمين."""
    return a + b

def subtract(a: float, b: float) -> float:
    """طرح رقمين."""
    return a - b
```

```python
def multiply(a: float, b: float) -> float:
    """ضرب رقمين."""
    return a * b

def divide(a: float, b: float) -> float | None:
    """قسمة رقمين، مع حماية من القسمة على صفر."""
    if b == 0:
        print("❌ خطأ: لا يمكن القسمة على صفر.")
        return None
    return a / b

def power(base: float, exponent: float) -> float:
    """رفع رقم لأس معين."""
    return base ** exponent

def square_root(number: float) -> float | None:
    """الجذر التربيعي، مع حماية من الأرقام السالبة."""
    if number < 0:
        print("❌ خطأ: لا يمكن حساب الجذر التربيعي لرقم سالب.")
        return None
    return math.sqrt(number)

def percentage(part: float, whole: float) -> float | None:
    """حساب النسبة المئوية لـ part من whole."""
    if whole == 0:
        print("❌ خطأ: القيمة الكلية لا يمكن أن تكون صفراً.")
        return None
    return (part / whole) * 100

# --- قاموس يربط الرمز بالدالة المقابلة (نمط احترافي) ---
OPERATIONS: dict[str, tuple] = {
    "+": (add, 2),
    "-": (subtract, 2),
    "*": (multiply, 2),
    "/": (divide, 2),
    "^": (power, 2),
    "sqrt": (square_root, 1),
    "%": (percentage, 2),
}

def display_menu() -> None:
    """طباعة قائمة العمليات المتاحة."""
    print("\n" + "=" * 40)
    print("🧮 الحاسبة العلمية")
    print("=" * 40)
    print("العمليات المتاحة:")
    print("  +     : جمع")
    print("  -     : طرح")
    print("  *     : ضرب")
    print("  /     : قسمة")
    print("  ^     : أس (قوة)")
```

```
    print("  sqrt  : جذر تربيعي")
    print("  %     : نسبة مئوية")
    print("  q     : خروج")
    print("=" * 40)

def run_calculator() → None:
    """تشغيل الحاسبة في حلقة تفاعلية."""
    display_menu()

    while True:
        choice = input("\nاختر العملية: ").strip()

        if choice == "q":
            print("👋 إلى اللقاء!")
            break

        if choice not in OPERATIONS:
            print("❌ عملية غير معروفة. حاول مجدداً.")
            continue

        func, num_args = OPERATIONS[choice]

        try:
            if num_args == 1:
                x = float(input("أدخل الرقم: "))
                result = func(x)
            else:
                x = float(input("أدخل الرقم الأول: "))
                y = float(input("أدخل الرقم الثاني: "))
                result = func(x, y)

            if result is not None:
                print(f"✅ النتيجة: {result}")

        except ValueError:
            print("❌ خطأ: الرجاء إدخال أرقام صحيحة فقط.")

if __name__ == "__main__":
    run_calculator()
```

مثال تشغيل:

```
========================================
🧮 الحاسبة العلمية
========================================
العمليات المتاحة:
  +     : جمع
  -     : طرح
  *     : ضرب
  /     : قسمة
  ^     : أس (قوة)
  sqrt  : جذر تربيعي
  %     : نسبة مئوية
  q     : خروج
========================================

اختر العملية: +
أدخل الرقم الأول: 5
أدخل الرقم الثاني: 3
✅ النتيجة: 8.0

اختر العملية: sqrt
أدخل الرقم: 16
✅ النتيجة: 4.0

اختر العملية: /
أدخل الرقم الأول: 10
أدخل الرقم الثاني: 0
❌ خطأ: لا يمكن القسمة على صفر.

اختر العملية: q
👋 إلى اللقاء!
```

💡 **ملاحظة تصميمية مهمة:** لاحظ استخدام قاموس `OPERATIONS` لربط الرموز بالدوال المقابلة بدلاً من سلسلة طويلة من `if/elif`. هذا النمط يُسمى **Dispatch Table**، وهو أسلوب احترافي شائع لتنظيم الكود عندما يكون لديك عدة خيارات تؤدي لاستدعاء دوال مختلفة.

✅ تحدي إضافي: إضافة سجل العمليات (الحل)

Python

```python
calculation_history: list[str] = []

def log_calculation(operation: str, result: float) -> None:
    """تسجل كل عملية حساب في سجل."""
    calculation_history.append(f"{operation} = {result}")

def show_history() -> None:
    """تعرض كل العمليات المحسوبة سابقاً."""
    if not calculation_history:
        print("لا توجد عمليات سابقة.")
        return

    print("\n📜 سجل العمليات:")
    for entry in calculation_history:
        print(f"  {entry}")

# يمكن استدعاء log_calculation بعد كل نتيجة ناجحة في run_calculator
```

10.10 تمارين

🟢 مستوى مبتدئ

تمرين 1: اكتب دالة `is_even(number: int) → bool` تُرجع `True` إذا كان الرقم زوجياً.

تمرين 2: اكتب دالة `greet_user(name: str, title: str = "الأستاذ/ة") → str` تُرجع تحية مع لقب افتراضي.

تمرين 3: اكتب دالة `average(*numbers: float) → float` تحسب متوسط أي عدد من الأرقام الممررة.

✅ الحل

Python

```python
# تمرين 1
def is_even(number: int) -> bool:
    return number % 2 == 0

print(is_even(4))  # True
print(is_even(7))  # False

# تمرين 2
def greet_user(name: str, title: str = "الأستاذ/ة") -> str:
    return f"مرحباً {title} {name}"

print(greet_user("سمير"))                    # مرحباً الأستاذ/ة سمير
print(greet_user("نورة", title="الدكتورة"))  # مرحباً الدكتورة نورة

# تمرين 3
def average(*numbers: float) -> float:
    if not numbers:
        return 0.0
    return sum(numbers) / len(numbers)

print(average(10, 20, 30))  # 20.0
```

🟡 مستوى متوسط

تمرين 4: اكتب دالة `is_valid_password(password: str) -> tuple[bool, str]` تتحقق أن كلمة المرور: طولها 8 أحرف على الأقل، تحتوي رقماً واحداً على الأقل، وحرفاً كبيراً واحداً على الأقل. تُرجع `(True, "صالحة")` أو `(False, "سبب الرفض")`.

تمرين 5: اكتب دالة `filter_products(products: list[dict], **criteria) -> list[dict]` تُرجع المنتجات المطابقة لأي معايير مُمرَّرة (مثلاً `category="إلكترونيات"`).

✅ الحل

Python

```python
# تمرين 4
def is_valid_password(password: str) -> tuple[bool, str]:
    if len(password) < 8:
        return False, "يجب أن تكون 8 أحرف على الأقل"
    if not any(char.isdigit() for char in password):
        return False, "يجب أن تحتوي على رقم واحد على الأقل"
    if not any(char.isupper() for char in password):
        return False, "يجب أن تحتوي على حرف كبير واحد على الأقل"
    return True, "صالحة"

valid, message = is_valid_password("abc123")
print(f"{valid}: {message}")  # False: يجب أن تكون 8 أحرف على الأقل

valid, message = is_valid_password("Abcd1234")
print(f"{valid}: {message}")  # True: صالحة

# تمرين 5
def filter_products(products: list[dict], **criteria) -> list[dict]:
    result = []
    for product in products:
        if all(product.get(key) == value for key, value in criteria.items()):
            result.append(product)
    return result

products = [
    {"name": "لابتوب", "category": "إلكترونيات", "price": 3500},
    {"name": "كرسي", "category": "أثاث", "price": 450},
    {"name": "ماوس", "category": "إلكترونيات", "price": 50},
]

electronics = filter_products(products, category="إلكترونيات")
print(electronics)
# [{'name': 'لابتوب',...  }, {'name': 'ماوس',...  }]
```

🔴 تحدي

تمرين 6: اكتب دالة `memoize(func)` تأخذ دالة أخرى كمعامل، وتُرجع نسخة «محسّنة» منها تُخزّن (تتذكر) نتائج الاستدعاءات السابقة بنفس المعاملات، بدلاً من إعادة حسابها (هذا تمهيد لمفهوم Decorators في الفصل 12).

✅ الحل

Python

```python
def memoize(func):
    """تُرجع نسخة من func السابقة تخزّن النتائج."""
    cache: dict = {}

    def wrapper(*args):
        if args in cache:
            print(f"(من الذاكرة المؤقتة لـ {args})")
            return cache[args]
        result = func(*args)
        cache[args] = result
        return result

    return wrapper

def slow_square(n: int) -> int:
    import time
    time.sleep(1)  # محاكاة عملية بطيئة
    return n ** 2

fast_square = memoize(slow_square)

print(fast_square(5))  # يأخذ ثانية واحدة، النتيجة: 25
print(fast_square(5))  # فوري! (من الذاكرة المؤقتة لـ (5,))
print(fast_square(7))  # يأخذ ثانية واحدة، النتيجة: 49
```

ملاحظة: مكتبة Python القياسية توفر هذا جاهزاً عبر `functools.lru_cache` — سنتعرف عليها في الفصل 12 عند دراسة Decorators بعمق.

10.11 اختبر فهمك (Quiz)

❓ السؤال 1: ما الفرق بين Parameter و Argument؟

الإجابة: Parameter هو الاسم المُعرَّف داخل أقواس تعريف الدالة (`def func(name):`)، أما Argument هو القيمة الفعلية التي تُمرر عند استدعاء الدالة (`func("Ahmed")`).

❓ السؤال 2: لماذا يُعتبر استخدام list كقيمة افتراضية لمعامل خطأً خطيراً؟

الإجابة: لأن القيمة الافتراضية تُنشأ **مرة واحدة فقط** عند تعريف الدالة، وليس في كل استدعاء. فإذا عدّلتها (مثل `.append()`) في أحد الاستدعاءات، ستبقى معدّلة وتؤثر على كل الاستدعاءات اللاحقة التي لا تمرر قيمة صريحة لهذا المعامل. الحل: استخدم `None` كقيمة افتراضية، وأنشئ القائمة الفعلية داخل جسم الدالة.

❓ السؤال 3: ما ناتج الكود التالي ولماذا؟

```python
def modify(x, lst):
    x += 1
    lst.append(x)

a = 10
b = [1, 2, 3]
modify(a, b)
print(a, b)
```

الإجابة: `10 [1, 2, 3, 11]`. المتغير `a` (int، غير قابل للتعديل) لم يتأثر لأن `x += 1` أنشأ كائناً جديداً داخل الدالة فقط. أما `b` (list، قابلة للتعديل) تأثرت فعلياً لأن `.append()` عدّلت نفس الكائن في الذاكرة الذي يُشير إليه `b` خارج الدالة أيضاً.

❓ السؤال 4: لماذا يُعتبر استخدام global ممارسة سيئة غالباً؟

الإجابة: لأنه يجعل الدالة تعتمد على حالة خارجية خفية غير واضحة من توقيع الدالة نفسها، مما يصعّب فهم الدالة بمعزل عن باقي الكود، ويصعّب اختبارها، ويزيد احتمال حدوث أخطاء غير متوقعة عند تعديل الكود لاحقاً. البديل الأفضل: تمرير القيم كمعاملات وإرجاع النتائج بدلاً من تعديل متغيرات خارجية مباشرة.

❓ السؤال 5: ما الفرق بين *args و **kwargs؟

الإجابة: `*args` تجمع أي عدد من المعاملات **الموضعية** الإضافية في tuple، بينما `**kwargs` تجمع أي عدد من المعاملات **المسماة** الإضافية في dict.

10.12 ملخص الفصل (Cheat Sheet)

Python

```python
# تعريف دالة أساسية
def func_name(param: type) -> return_type:
    """Docstring."""
    return value

# قيم افتراضية
def func(a: int, b: int = 10) -> int: ...

# ⚠️ لا تستخدم mutable كقيمة افتراضية
def bad(lst: list = []): ...            # ❌
def good(lst: list | None = None):      # ✅
    if lst is None:
        lst = []

# *args و **kwargs
def func(*args, **kwargs): ...
func(1, 2, 3, key="value")

# Keyword-only و Positional-only
def func(a, b, /, c, d, *, e, f): ...
#         ^positional-only  ^keyword-only

# إرجاع قيم متعددة (tuple unpacking)
def minmax(nums: list) -> tuple:
    return min(nums), max(nums)
low, high = minmax([1, 5, 3])

# Scope
x = 10                  # Global
def f():
    y = 20              # Local
    global x; x = 99    # تعديل Global (تجنّبه قدر الإمكان)

# الدوال كقيم (First-Class Functions)
my_func = some_function   # بدون ()
sorted(data, key=my_func) # تمرير كمعامل
```

المفهوم	القاعدة الذهبية
القيم الافتراضية	لا تستخدم list/dict/set مباشرة، استخدم None
التعديل على الحالة الخارجية	تجنّب global، مرر وأعد القيم بدلاً منه
وضوح الاستدعاء	استخدم Keyword Arguments مع أكثر من 3 معاملات
عدد غير محدد من المدخلات	`*args` للموضعية، `**kwargs` للمسماة
الدوال النقية (Pure Functions)	يُفضّل دائماً: نفس المدخل = نفس المخرج، بدون آثار جانبية

10.13 ماذا بعد؟

تهانينا! لقد أتممت الآن **الجزء الأول من الكتاب بالكامل**: الأساسيات المتينة في Python. تمتلك الآن كل الأدوات الأساسية: المتغيرات، النصوص، الأرقام، التحكم بالتدفق، الحلقات، هياكل البيانات الأربعة، والدوال.

لكن حتى الآن، تعاملنا مع الأخطاء بطريقة بدائية: برنامجك ينهار فوراً عند حدوث أي خطأ غير متوقع (`KeyError`, `ValueError`, `ZeroDivisionError`...). في العالم الحقيقي، البرامج الاحترافية **تتوقع** الأخطاء وتتعامل معها بأناقة دون أن تنهار بالكامل.

في **الفصل الحادي عشر: معالجة الأخطاء**، ستتعلم أسلوب `try/except/else/finally`، كيف «ترمي» أخطاءك المخصصة، وكيف تكتب برامج قوية تصمد أمام المدخلات غير المتوقعة والظروف الاستثنائية — مهارة تفصل البرامج الهشة عن البرامج الاحترافية الجاهزة للإنتاج.

الجزء الثالث · 4 فصول

03

البرمجة الوظيفية والكائنية

الانتقال من «كود يعمل» إلى «كود مُهندَس»: معالجة الأخطاء، الدوال المتقدمة، والبرمجة الكائنية بأساسها وتعمّقها.

11 · معالجة الأخطاء
12 · الدوال المتقدمة
13 · البرمجة الكائنية
14 · البرمجة الكائنية

فصل 11 / 26 · الجزء الثالث

11 معالجة الأخطاء (Error Handling)

9 دقيقة قراءة | 14 قسمًا | 31 مثالًا | 1,816 كلمة

في هذا الفصل

11.1 لماذا يحتاج كل برنامج احترافي لمعالجة الأخطاء؟
11.2 فهم الاستثناءات (Exceptions)
11.3 الصيغة الأساسية: try / except
11.4 else و finally: إكمال الصورة
11.5 إطلاق الأخطاء بنفسك: raise
11.6 إنشاء استثناءات مخصصة (Custom Exceptions)
11.7 فلسفتان في البرمجة: EAFP مقابل LBYL
11.8 دالة assert: التحقق أثناء التطوير
11.9 مقدمة مختصرة لـ logging بدلاً من print
11.10 مشروع تطبيقي: نظام تسجيل دخول آمن
11.11 تمارين
11.12 اختبر فهمك (Quiz)
11.13 ملخص الفصل (Cheat Sheet)
11.14 ماذا بعد؟

11.1 لماذا يحتاج كل برنامج احترافي لمعالجة الأخطاء؟

تخيّل تطبيق بنك يطلب من المستخدم إدخال مبلغ للسحب. ماذا لو كتب المستخدم حروفاً بدلاً من أرقام؟ ماذا لو حاول السحب أكثر من رصيده؟ ماذا لو انقطع الاتصال بالإنترنت أثناء معالجة العملية؟

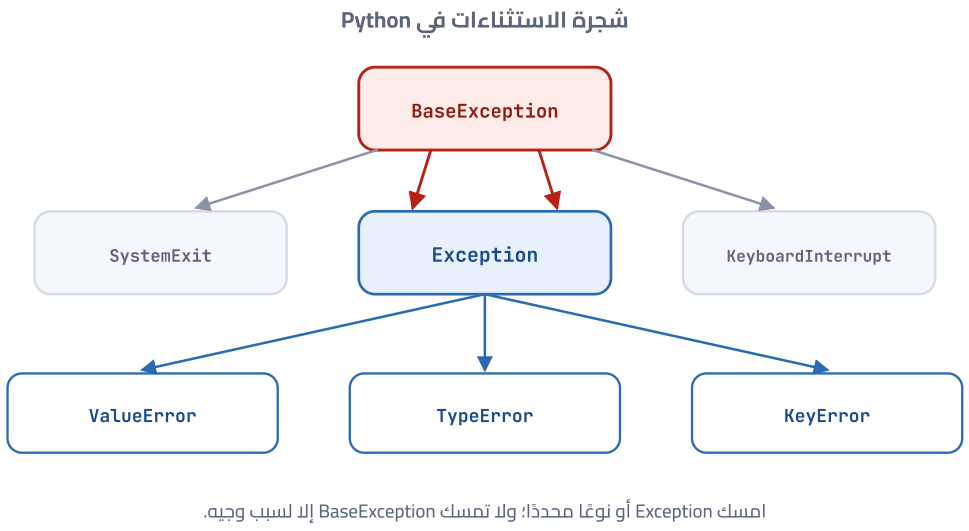
بالطريقة التي تعلمناها حتى الآن، أي من هذه الحالات ستجعل البرنامج **ينهار فوراً** برسالة خطأ مخيفة (Traceback) يراها المستخدم النهائي — وهذا غير مقبول إطلاقاً في أي تطبيق حقيقي.

```python
balance = 1000
withdrawal = input("أدخل مبلغ السحب: ")  # المستخدم يكتب "مئة"
balance -= int(withdrawal)  # 💥 ValueError: invalid literal for int()
# البرنامج ينهار، والمستخدم يرى رسالة تقنية غير مفهومة
```

المبرمج المحترف لا يفترض أن كل شيء سيسير كما خطط له. إنه **يتوقع** الأخطاء المحتملة، ويتعامل معها بأناقة: رسالة واضحة للمستخدم، محاولة بديلة، أو تسجيل الخطأ للمراجعة لاحقاً — كل ذلك **دون** أن ينهار البرنامج بالكامل.

في هذا الفصل، ستتعلم فلسفة Python الكاملة في التعامل مع الأخطاء، من الأساسيات إلى إنشاء أخطائك المخصصة، وستكتسب عقلية «البرمجة الدفاعية» التي تميز الكود الجاهز للإنتاج (Production-Ready) عن كود التجارب.

11.2 فهم الاستثناءات (Exceptions)



شكل 11.1 — شجرة الاستثناءات في Python

ما هو الاستثناء؟

الاستثناء (Exception) هو حدث غير طبيعي يحدث أثناء تنفيذ البرنامج، يُوقف التنفيذ العادي إذا لم تتم معالجته. تذكّر من الفصل الثاني أنواع الأخطاء التي واجهناها:

```python
print(10 / 0)        # ZeroDivisionError
print(int("abc"))    # ValueError
print(my_list[99])   # IndexError
print(my_dict["x"])  # KeyError
print(undefined_var) # NameError
```

كل خطأ من هذه له **نوع (Exception Type)** محدد، وهذه هي قوة نظام الاستثناءات في Python: يمكنك التعامل مع كل نوع خطأ بطريقة مختلفة.

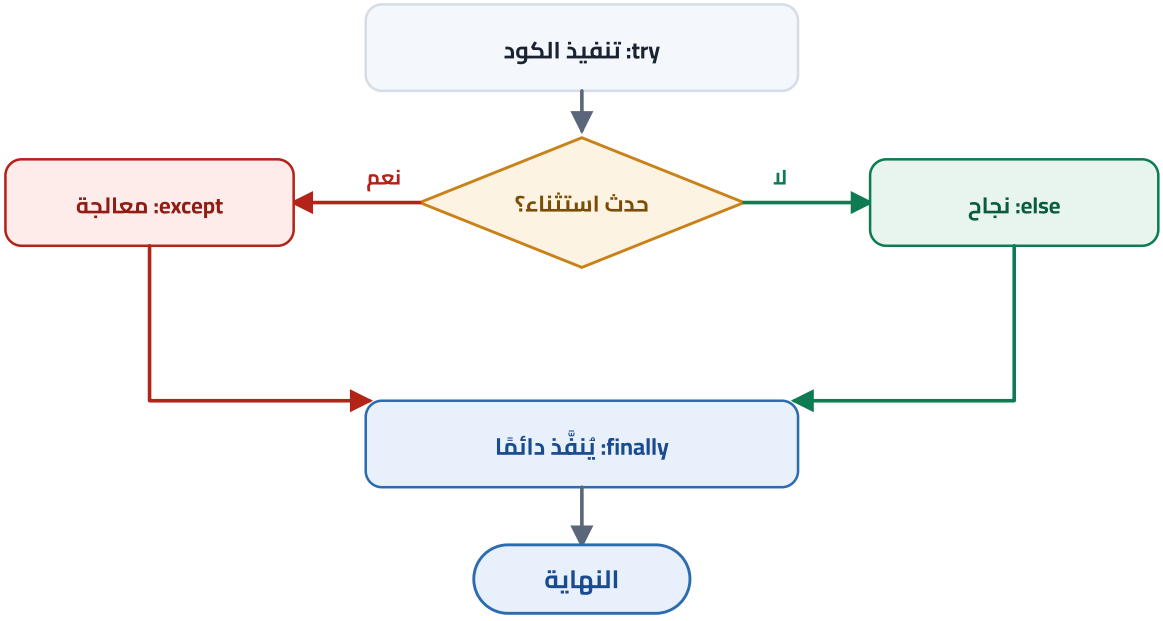
تشريح Traceback

```python
def divide(a, b):
    return a / b

result = divide(10, 0)
```

```
Traceback (most recent call last):
  File "script.py", line 4, in <module>
    result = divide(10, 0)
             ^^^^^^^^^^^^^
  File "script.py", line 2, in divide
    return a / b
           ~~^~~
ZeroDivisionError: division by zero
```

كيفية القراءة: اقرأ من **الأسفل للأعلى**. السطر الأخير يخبرك بنوع الخطأ ورسالته. الأسطر الأعلى تريك «سلسلة الاستدعاءات» (Call Stack) التي أدت للخطأ — أي الدالة التي استدعت الدالة التي استدعت... إلخ.

11.3 الصيغة الأساسية: try / except



else يعمل فقط عند النجاح، وfinally يعمل في كل الحالات.

شكل 11.2 — مسار try / except / else / finally

```python
try:
    number = int(input("أدخل رقماً: "))
    print(f"ضعف الرقم: {number * 2}")
except ValueError:
    print("❌ خطأ: الرجاء إدخال رقم صحيح فقط.")

print("البرنامج استمر في العمل!")
```

كيف يعمل:

1. Python تُنفّذ الكود داخل `try`
2. إذا حدث خطأ من النوع المحدد في `except`، يتوقف تنفيذ `try` فوراً، وينتقل التنفيذ إلى كتلة `except`
3. إذا لم يحدث أي خطأ، تُتجاهل كتلة `except` بالكامل
4. البرنامج يستمر بالعمل بعد كتلة `try/except` بدلاً من الانهيار

معالجة أنواع أخطاء متعددة

```python
def safe_divide(a: float, b: float) -> float | None:
    try:
        return a / b
    except ZeroDivisionError:
        print("❌ لا يمكن القسمة على صفر.")
        return None
    except TypeError:
        print("❌ يجب أن تكون القيم أرقاماً.")
        return None

print(safe_divide(10, 0))      # ❌ لا يمكن القسمة على صفر. → None
print(safe_divide(10, "abc"))  # ❌ يجب أن تكون القيم أرقاماً. → None
print(safe_divide(10, 2))      # 5.0
```

معالجة عدة أخطاء بنفس الطريقة

```python
try:
    value = my_dict[key]
    result = 100 / value
except (KeyError, ZeroDivisionError) as e:
    print(f"حدث خطأ: {e}")
```

التقاط كل الأخطاء (استخدمه بحذر شديد!)

```python
try:
    risky_operation()
except Exception as e:
    print(f"حدث خطأ غير متوقع: {e}")
```

> ⚠️ **تحذير خطير جداً - أخطر عادة سيئة في معالجة الأخطاء:**

```python
# ❌ الأسوأ على الإطلاق - لا تفعل هذا أبداً!
try:
    risky_operation()
except:  # bare except - يلتقط كل شيء حتى KeyboardInterrupt!
    pass  # يُخفي الخطأ تماماً دون أي أثر!
```

هذا النمط يُسمى "Silent Failure" (الفشل الصامت) — البرنامج يستمر وكأن لا شيء حدث، بينما خطأ حقيقي حدث واختفى بدون أي أثر، مما يجعل تصحيح الأخطاء لاحقاً شبه مستحيل. **القاعدة الذهبية**: التقط دائماً نوع الخطأ المحدد الذي تتوقعه، ولا تستخدم `except:` فارغة أبداً. إذا احتجت فعلاً التقاط كل شيء، استخدم `except Exception as e:` على الأقل (لتجنب التقاط إشارات النظام الحرجة)، واطبع/سجّل تفاصيل الخطأ دائماً.

هرمية الاستثناءات (Exception Hierarchy)

جميع الأخطاء في Python ترث من فئة أساسية واحدة: `BaseException`. فهم الهرمية يساعدك في معرفة أي استثناء يُغطي أي حالات:

```
BaseException
 ├── SystemExit
 ├── KeyboardInterrupt
 └── Exception                    ← كل أخطائك المخصصة يجب أن ترث من هنا
      ├── ArithmeticError
      │    └── ZeroDivisionError
      ├── LookupError
      │    ├── IndexError
      │    └── KeyError
      ├── ValueError
      ├── TypeError
      ├── NameError
      ├── AttributeError
      ├── FileNotFoundError
      ├── OSError
      └── ... والمزيد
```

نصيحة الخبراء: إذا كتبت `except Exception` فستلتقط كل الأخطاء الموجودة تحتها في الهرمية، لكنها لن تلتقط `SystemExit` و `KeyboardInterrupt` (أي ضغطة Ctrl+C)، وهذا هو السلوك الصحيح والمرغوب — لا تريد منع المستخدم من إيقاف برنامجه بـ Ctrl+C!

Python

```python
# ترتيب except مهم جداً! الأكثر تحديداً أولاً
try:
    value = my_list[10]
except IndexError:
    print("فهرس خارج النطاق")
except LookupError:  # تأتي بعد IndexError لأنها أعم منها
    print("خطأ بحث عام")
```

خطأ شائع: وضع `except Exception` **قبل** الأخطاء المحددة يجعل الأخطاء المحددة لا تُلتقط أبداً (لأن `Exception` تلتقطها جميعاً قبل الوصول لها):

Python

```python
try:
    x = 1 / 0
except Exception:
    print("خطأ عام")  # هذا ما سيُطبع
except ZeroDivisionError:  # ⚠️ لن يُنفَّذ أبداً ميت!
    print("قسمة على صفر")
```

القاعدة: رتّب كتل `except` من الأكثر تحديداً إلى الأعم دائماً.

11.4 else و finally: إكمال الصورة

الصيغة الكاملة

Python

```
try:
    # الكود الذي قد يُطلق خطأ
    pass
except SomeError:
    # يُنفَّذ فقط إذا حدث هذا الخطأ المحدد
    pass
else:
    # يُنفَّذ فقط إذا لم يحدث أي خطأ في try
    pass
finally:
    # يُنفَّذ دائماً، سواء حدث خطأ أم لا
    pass
```

مثال شامل عملي

Python

```
def read_and_process_number(text: str) -> None:
    try:
        number = int(text)
    except ValueError:
        print(f"❌ '{text}' ليس رقماً صحيحاً.")
    else:
        # يُنفَّذ فقط إذا نجح التحويل بدون خطأ
        print(f"✅ تم تحويل الرقم بنجاح: {number}")
        print(f"   ضعفه: {number * 2}")
    finally:
        # يُنفَّذ دائماً - مثالي للتنظيف
        print("--- انتهت المحاولة ---  ")

read_and_process_number("42")
print()
read_and_process_number("abc")
```

الناتج:

```
✅ تم تحويل الرقم بنجاح: 42
   ضعفه: 84
---  انتهت المحاولة  ---

❌ 'abc' ليس رقماً صحيحاً.
---  انتهت المحاولة  ---
```

نصيحة الخبراء - لماذا نستخدم `else` بدلاً من وضع الكود داخل `try` مباشرة؟

```python
# أقل وضوحاً - أي سطر بالضبط قد يُطلق الخطأ؟
try:
    number = int(text)
    print(f"ضعفه: {number * 2}")  # ماذا لو هذا السطر نفسه فشل لسبب آخر؟
except ValueError:
    print("خطأ")

# أوضح بكثير - try تحتوي فقط على الكود المحتمل فشله
try:
    number = int(text)
except ValueError:
    print("خطأ")
else:
    print(f"ضعفه: {number * 2}")  # تم تحويله بنجاح number نعلم بالتأكيد أن
```

استخدام `else` يفصل بوضوح «الكود الخطير» (داخل try) عن «الكود الذي يعتمد على نجاحه» (داخل else)، مما يحسّن وضوح الكود ويمنع التقاط أخطاء غير مقصودة.

متى تستخدم `finally` ؟

`finally` مثالية لعمليات «التنظيف» التي يجب أن تحدث دائماً، بغض النظر عن نجاح أو فشل العملية: إغلاق ملف، قطع اتصال بقاعدة بيانات، تحرير موارد.

```python
def process_file(filename: str) -> None:
    file = None
    try:
        file = open(filename, "r", encoding="utf-8")
        content = file.read()
        print(f"تم قراءة {len(content)} حرف.")
    except FileNotFoundError:
        print(f"❌ الملف '{filename}' غير موجود.")
    finally:
        if file is not None:
            file.close()
            print("✅ تم إغلاق الملف.")  # يحدث دائماً!

process_file("not_exist.txt")
```

> **نصيحة الخبراء:** في الفصل السادس عشر سنتعلم `with open(...) as file:` التي تُغني تماماً عن الحاجة لـ `finally` اليدوية لإغلاق الملفات — إنها تفعل ذلك تلقائياً باستخدام **Context Managers**. لكن فهم `finally` ضروري لفهم أي موارد أخرى تحتاج تنظيفاً يدوياً (اتصالات شبكة، قواعد بيانات، أقفال Threading).

11.5 إطلاق الأخطاء بنفسك: raise

أحياناً تريد أنت أن «تُطلق» خطأً عمداً عندما تكتشف حالة غير منطقية، حتى لو لم تُطلقها Python تلقائياً.

```python
def withdraw(balance: float, amount: float) -> float:
    if amount <= 0:
        raise ValueError("مبلغ السحب يجب أن يكون موجباً.")
    if amount > balance:
        raise ValueError("الرصيد غير كافٍ لإتمام العملية.")
    return balance - amount

try:
    new_balance = withdraw(1000, 1500)
except ValueError as e:
    print(f"❌ فشلت العملية: {e}")
```

الناتج: `❌ فشلت العملية: الرصيد غير كافٍ لإتمام العملية.`

إعادة إطلاق الخطأ (Re-raising)

أحياناً تريد التقاط خطأ لتسجيله، ثم إطلاقه من جديد ليتعامل معه مستوى أعلى من الكود:

```python
def process_data(data: list) -> float:
    try:
        return sum(data) / len(data)
    except ZeroDivisionError:
        print("⚠️ تحذير: القائمة فارغة، سيتم إعادة إطلاق الخطأ.")
        raise  # إعادة إطلاق نفس الخطأ الملتقط — بدون معامل

try:
    process_data([])
except ZeroDivisionError:
    print("❌ تم التعامل مع القائمة الفارغة على المستوى الأعلى.")
```

تسلسل الاستثناءات (Exception Chaining)

```python
def parse_age(text: str) -> int:
    try:
        return int(text)
    except ValueError as e:
        raise ValueError(f"فشل تحليل العمر من النص: '{text}'") from e

try:
    parse_age("twenty")
except ValueError as e:
    print(e)
    print(f"السبب الأصلي: {e.__cause__}")
```

الناتج:

```
فشل تحليل العمر من النص: 'twenty'
السبب الأصلي: invalid literal for int() with base 10: 'twenty'
```

> **نصيحة الخبراء:** استخدام `raise NewError(...) from original_error` يحافظ على «السلسلة الكاملة» للخطأ الأصلي في الـ Traceback، مما يسهّل تصحيح الأخطاء كثيراً بدلاً من إخفاء السبب الجذري.

11.6 إنشاء استثناءات مخصصة (Custom Exceptions)

في المشاريع الحقيقية، الأخطاء المدمجة في Python (`ValueError` , `TypeError` ...) لا تكفي دائماً للتعبير عن أخطاء خاصة بمجال عملك (Business Logic).

```python
class InsufficientFundsError(Exception):
    """تُطلق عندما يحاول المستخدم سحب مبلغ أكبر من رصيده."""
    pass

class InvalidAmountError(Exception):
    """تُطلق عندما يكون المبلغ المُدخل غير منطقي (سالب أو صفر)."""
    pass

def withdraw(balance: float, amount: float) -> float:
    if amount <= 0:
        raise InvalidAmountError(f"المبلغ {amount} غير صالح؛ يجب أن يكون موجباً.")
    if amount > balance:
        raise InsufficientFundsError(
            f"طلبت سحب {amount}، لكن رصيدك الحالي {balance} فقط."
        )
    return balance - amount

try:
    withdraw(500, 1000)
except InsufficientFundsError as e:
    print(f"💰 خطأ في الرصيد: {e}")
except InvalidAmountError as e:
    print(f"🔢 خطأ في المبلغ: {e}")
```

الناتج: `خطأ في الرصيد: طلبت سحب 1000، لكن رصيدك الحالي 500 فقط 💰`

> **لماذا نُنشئ استثناءات مخصصة؟**
> 1. **وضوح أكبر:** `InsufficientFundsError` أوضح بكثير من `ValueError` عامة
> 2. **معالجة دقيقة:** يمكنك التعامل مع كل حالة عمل (Business Case) بطريقة مختلفة تماماً
> 3. **توثيق ذاتي:** الكود الذي يقرأ `except InsufficientFundsError` يفهم فوراً السيناريو دون شرح إضافي

هرمية استثناءات مخصصة

يمكنك بناء هرمية من الأخطاء المخصصة الخاصة بتطبيقك، تماماً كما تفعل Python داخلياً:

Python

```python
class BankError(Exception):
    """الفئة الأساسية لكل أخطاء النظام البنكي."""
    pass

class InsufficientFundsError(BankError):
    """رصيد غير كافٍ."""
    pass

class InvalidAmountError(BankError):
    """مبلغ غير صالح."""
    pass

class AccountFrozenError(BankError):
    """الحساب مجمّد."""
    pass

def process_transaction(account_status: str, balance: float, amount: float) -> float:
    if account_status == "frozen":
        raise AccountFrozenError("هذا الحساب مجمّد حالياً.")
    if amount <= 0:
        raise InvalidAmountError("المبلغ يجب أن يكون موجباً.")
    if amount > balance:
        raise InsufficientFundsError("الرصيد غير كافٍ.")
    return balance - amount

try:
    process_transaction("frozen", 1000, 100)
except BankError as e:  # التقاط أي خطأ بنكي بغض النظر عن نوعه الدقيق
    print(f"🏦 خطأ في النظام البنكي: {type(e).__name__}: {e}")
```

الناتج: `🏦 خطأ في النظام البنكي: AccountFrozenError: هذا الحساب مجمّد حالياً.`

إضافة بيانات إضافية للاستثناء المخصص

```python
class ValidationError(Exception):
    """خطأ تحقق من صحة البيانات، مع تفاصيل الحقل المُخالف."""

    def __init__(self, field: str, message: str):
        self.field = field
        self.message = message
        super().__init__(f"خطأ في حقل '{field}': {message}")

def validate_age(age: int) -> None:
    if age < 0 or age > 150:
        raise ValidationError(field="age", message="يجب أن يكون بين 0 و150")

try:
    validate_age(-5)
except ValidationError as e:
    print(f"الحقل المُخالف: {e.field}")
    print(f"الرسالة: {e.message}")
```

مقارنة مع لغات أخرى: نظام الاستثناءات في Python مشابه جداً لـ Java وC#، حيث توجد هرمية استثناءات، وإمكانية إنشاء استثناءات مخصصة ترث من فئة أساسية. أما في لغات مثل C أو Go، لا يوجد نظام استثناءات مدمج بنفس القوة؛ يعتمد المبرمجون على إرجاع قيم خطأ (Error Codes) أو أزواج `(result, error)` بشكل يدوي.

11.7 فلسفتان في البرمجة: EAFP مقابل LBYL

هذا مفهوم فلسفي عميق يميز أسلوب Python عن لغات أخرى.

LBYL - Look Before You Leap (انظر قبل أن تقفز)

```python
# تحقق أولاً، ثم نفّذ
import os

if os.path.exists("data.txt"):
    with open("data.txt") as f:
        content = f.read()
else:
    print("الملف غير موجود")
```

EAFP - Easier to Ask Forgiveness than Permission (أسهل أن تطلب الصفح من أن تطلب الإذن)

```python
# نفّذ مباشرة، وتعامل مع الخطأ إن حدث
try:
    with open("data.txt") as f:
        content = f.read()
except FileNotFoundError:
    print("الملف غير موجود")
```

أيهما يُفضّل في Python؟ الأسلوب **الثاني (EAFP)** هو الأسلوب «Pythonic» الموصى به رسمياً. لماذا؟
1. **تجنب Race Conditions**: بين لحظة `os.path.exists()` ولحظة `open()`، قد يُحذف الملف فعلياً من قبل عملية أخرى! بينما `try/except` يتعامل مع اللحظة الفعلية للتنفيذ.
2. **أداء أفضل عادةً**: التحقق المسبق يعني فحصاً إضافياً دائماً، بينما `try/except` لا يكلّف شيئاً إضافياً إذا لم يحدث خطأ فعلياً.
3. **كود أنظف**: تقل الحاجة لسلاسل `if` متداخلة للتحقق من كل شرط ممكن مسبقاً.

⚠ **تنبيه**: هذا لا يعني أن `if` للتحقق المسبق سيء دائماً! في حالات بسيطة وواضحة (مثل `if value is not None:`)، التحقق المباشر أوضح من `try/except`. القاعدة العملية: استخدم EAFP عندما يكون الفشل **محتملاً لكن نادراً** وستتعامل معه برسالة/بديل، واستخدم LBYL عندما يكون التحقق **بسيطاً جداً وسريعاً** (مثل فحص قيمة None).

11.8 دالة assert: التحقق أثناء التطوير

```python
def calculate_average(numbers: list[float]) -> float:
    assert len(numbers) > 0, "القائمة يجب ألا تكون فارغة"
    return sum(numbers) / len(numbers)

print(calculate_average([10, 20, 30]))  # 20.0
# print(calculate_average([]))
# 💥 AssertionError: القائمة يجب ألا تكون فارغة
```

⚠ **تحذير حرج:** `assert` **ليست** بديلاً عن `try/except` لمعالجة أخطاء المستخدم أو البيانات الخارجية! السبب: عند تشغيل Python بوضع «التحسين» (`python -O script.py`)، تُحذف كل عبارات `assert` بالكامل من الكود المُنفَّذ! لذلك:
- استخدم `assert` فقط للتحقق من افتراضات داخلية أثناء **التطوير والاختبار** (مثل: «هذه الدالة يجب ألا تُستدعى أبداً بقائمة فارغة، وإذا حدث فهذا خطأ برمجي مني أنا المطور»)
- استخدم `raise` و `try/except` دائماً للتعامل مع أي بيانات قد تأتي من **مستخدم حقيقي أو مصدر خارجي** (إدخال، ملف، API، قاعدة بيانات)

```python
# ❌ خطأ: استخدام assert للتحقق من مدخلات مستخدم
def set_age(age: int) -> None:
    assert age >= 0, "العمر يجب أن يكون موجباً!"  # قد يُحذف بالكامل

# ✅ صحيح: استخدام raise للتحقق من مدخلات مستخدم
def set_age(age: int) -> None:
    if age < 0:
        raise ValueError("العمر يجب أن يكون موجباً")
```

11.9 مقدمة مختصرة لـ logging بدلاً من print

في المشاريع الحقيقية، لا نستخدم `print()` لتسجيل الأخطاء؛ نستخدم وحدة `logging` المدمجة التي تمنحنا مستويات خطورة، وتوقيت تلقائي، وإمكانية الحفظ بملفات.

```python
import logging

logging.basicConfig(level=logging.INFO, format="%(asctime)s - %(levelname)s - %(message)s")

def divide(a: float, b: float) -> float | None:
    try:
        return a / b
    except ZeroDivisionError:
        logging.error(f"محاولة قسمة {a} على صفر!")
        return None

divide(10, 0)
```

الناتج:

```
2025-01-15 10:30:45,123 - ERROR - محاولة قسمة 10 على صفر!
```

> **نصيحة الخبراء:** `logging` خارج نطاق هذا الفصل بالتفصيل (سنتعمق فيه لاحقاً عند الحديث عن بناء تطبيقات حقيقية)، لكن تذكّر هذه القاعدة من الآن: **print للتجربة السريعة، logging للمشاريع الحقيقية**. الفرق الجوهري أن `logging` يمنحك تحكماً كاملاً (تشغيل/إيقاف أنواع معينة من الرسائل، حفظها بملف، إرسالها لنظام مراقبة) دون تعديل الكود نفسه.

11.10 مشروع تطبيقي: نظام تسجيل دخول آمن

نظام متكامل يستخدم كل ما تعلمناه: استثناءات مخصصة، هرمية أخطاء، try/except/else/finally، وEAFP.

Python

```python
class AuthError(Exception):
    """الفئة الأساسية لكل أخطاء المصادقة."""
    pass

class UserNotFoundError(AuthError):
    """المستخدم غير موجود في النظام."""
    pass

class IncorrectPasswordError(AuthError):
    """كلمة المرور غير صحيحة."""
    pass

class AccountLockedError(AuthError):
    """الحساب مقفل بسبب محاولات فاشلة متكررة."""
    pass

class WeakPasswordError(AuthError):
    """كلمة مرور ضعيفة عند التسجيل."""
    pass

# قاعدة بيانات وهمية (في الواقع ستكون قاعدة بيانات حقيقية)
users_db: dict[str, dict] = {
    "ahmed@test.com": {"password": "Secure123", "failed_attempts": 0, "locked": False},
}

MAX_FAILED_ATTEMPTS = 3

def register(email: str, password: str) -> None:
    """تسجيل مستخدم جديد مع التحقق من قوة كلمة المرور."""
    if len(password) < 8:
```

```
        raise WeakPasswordError("كلمة المرور يجب أن تكون 8 أحرف على الأقل.")
    if not any(char.isdigit() for char in password):
        raise WeakPasswordError("كلمة المرور يجب أن تحتوي رقماً واحداً على الأقل.")

    if email in users_db:
        raise ValueError("هذا البريد الإلكتروني مسجّل مسبقاً.")

    users_db[email] = {"password": password, "failed_attempts": 0, "locked": False}
    print(f"✅ تم تسجيل المستخدم {email} بنجاح.")

def login(email: str, password: str) -> bool:
    """محاولة تسجيل دخول، مع حماية من المحاولات المتكررة."""
    try:
        user = users_db[email]
    except KeyError:
        raise UserNotFoundError(f"لا يوجد مستخدم بالبريد: {email}")

    if user["locked"]:
        raise AccountLockedError("هذا الحساب مقفل. تواصل مع الدعم الفني.")

    if user["password"] ≠ password:
        user["failed_attempts"] += 1
        if user["failed_attempts"] ⩾ MAX_FAILED_ATTEMPTS:
            user["locked"] = True
            raise AccountLockedError(
                "تم قفل الحساب بسبب تجاوز عدد المحاولات الفاشلة المسموحة."
            )
        remaining = MAX_FAILED_ATTEMPTS - user["failed_attempts"]
        raise IncorrectPasswordError(
            f"كلمة المرور خاطئة. المحاولات المتبقية: {remaining}"
        )

    user["failed_attempts"] = 0  # إعادة تصفير عند النجاح
    return True

def run_login_attempt(email: str, password: str) -> None:
    """واجهة تفاعلية تلتقط كل أنواع أخطاء المصادقة بوضوح."""
    try:
        login(email, password)
    except UserNotFoundError as e:
        print(f"🕵️ {e}")
    except AccountLockedError as e:
        print(f"🔒 {e}")
    except IncorrectPasswordError as e:
        print(f"🔑 {e}")
    except AuthError as e:
        print(f"⚠️ خطأ مصادقة غير متوقع: {e}")
    else:
        print(f"✅ تم تسجيل الدخول بنجاح! أهلاً بك {email}")
    finally:
        print("-" * 40)
```

```
# --- سيناريوهات تجريبية ---
if __name__ == "__main__":
    try:
        register("sara@test.com", "weak")
    except WeakPasswordError as e:
        print(f"❌ فشل التسجيل: {e}")
    print("-" * 40)

    register("sara@test.com", "Strong123")
    print("-" * 40)

    run_login_attempt("unknown@test.com", "anything")   # UserNotFoundError
    run_login_attempt("ahmed@test.com", "wrong_pass")   # IncorrectPasswordError (محاولة 1)
    run_login_attempt("ahmed@test.com", "wrong_pass")   # IncorrectPasswordError (محاولة 2)
    run_login_attempt("ahmed@test.com", "wrong_pass")   # AccountLockedError (قفل - محاولة 3!)
    run_login_attempt("ahmed@test.com", "Secure123")    # AccountLockedError (مقفل مسبقاً)
    run_login_attempt("sara@test.com", "Strong123")     # ✅ نجاح
```

الناتج المتوقع:

```
❌ فشل التسجيل: كلمة المرور يجب أن تكون 8 أحرف على الأقل.
----------------------------------------
✅ تم تسجيل المستخدم sara@test.com بنجاح.
----------------------------------------
🕵 لا يوجد مستخدم بالبريد: unknown@test.com
----------------------------------------
🔑 كلمة المرور خاطئة. المحاولات المتبقية: 2
----------------------------------------
🔑 كلمة المرور خاطئة. المحاولات المتبقية: 1
----------------------------------------
🔒 تم قفل الحساب بسبب تجاوز عدد المحاولات الفاشلة المسموحة.
----------------------------------------
🔒 هذا الحساب مقفل. تواصل مع الدعم الفني.
----------------------------------------
✅ تم تسجيل الدخول بنجاح! أهلاً بك sara@test.com
----------------------------------------
```

<details>
<summary>✅ تحدي إضافي: دالة إعادة تعيين كلمة المرور (الحل)</summary>

```python
def reset_password(email: str, new_password: str) -> None:
    """إعادة تعيين كلمة مرور، مع إلغاء القفل إن وُجد."""
    try:
        user = users_db[email]
    except KeyError:
        raise UserNotFoundError(f"لا يوجد مستخدم بالبريد: {email}")

    if len(new_password) < 8 or not any(c.isdigit() for c in new_password):
        raise WeakPasswordError("كلمة المرور الجديدة ضعيفة جداً.")

    user["password"] = new_password
    user["failed_attempts"] = 0
    user["locked"] = False
    print(f"🔓 تم إعادة تعيين كلمة المرور وفك قفل الحساب لـ {email}")

try:
    reset_password("ahmed@test.com", "NewPass123")
except AuthError as e:
    print(f"❌ {e}")
```

</details>

11.11 تمارين

🟢 مستوى مبتدئ

تمرين 1: اكتب كود `try/except` يطلب من المستخدم رقماً، ويطبع «رقم صحيح» أو «خطأ: ليس رقماً» حسب الحالة.

تمرين 2: اكتب دالة `safe_list_access(lst: list, index: int)` تُرجع العنصر في الفهرس المطلوب، أو `None` إذا كان الفهرس خارج النطاق (باستخدام `IndexError`).

✅ الحل

Python

```python
# تمرين 1
try:
    num = int(input("أدخل رقماً: "))
    print("رقم صحيح")
except ValueError:
    print("خطأ: ليس رقماً")

# تمرين 2
def safe_list_access(lst: list, index: int):
    try:
        return lst[index]
    except IndexError:
        return None

print(safe_list_access([1, 2, 3], 1))   # 2
print(safe_list_access([1, 2, 3], 10))  # None
```

🟡 مستوى متوسط

تمرين 3: اكتب استثناءً مخصصاً `NegativeValueError`، ودالة `calculate_square_root(n)` تُطلقه إذا كان الرقم سالباً، وتُرجع الجذر التربيعي خلاف ذلك.

تمرين 4: اكتب دالة `parse_config(data: dict)` تقرأ المفاتيح `"host"` و `"port"` من قاموس إعدادات، وتُطلق استثناءً مخصصاً `MissingConfigError` مع اسم المفتاح المفقود إن لم يكن موجوداً.

✅ الحل

Python

```python
import math

# تمرين 3
class NegativeValueError(Exception):
    pass

def calculate_square_root(n: float) -> float:
    if n < 0:
        raise NegativeValueError(f"لا يمكن حساب جذر رقم سالب: {n}")
    return math.sqrt(n)

try:
    print(calculate_square_root(-9))
except NegativeValueError as e:
    print(f"❌ {e}")

# تمرين 4
class MissingConfigError(Exception):
    def __init__(self, key: str):
        self.key = key
        super().__init__(f"الإعداد المطلوب '{key}' غير موجود.")

def parse_config(data: dict) -> tuple[str, int]:
    try:
        host = data["host"]
    except KeyError:
        raise MissingConfigError("host")

    try:
        port = data["port"]
    except KeyError:
        raise MissingConfigError("port")

    return host, port

try:
    parse_config({"host": "localhost"})
except MissingConfigError as e:
    print(f"❌ {e}")  # ❌ الإعداد المطلوب 'port' غير موجود.
```

🔴 تحدي

تمرين 5: اكتب نظام معالجة طلبات API وهمي: دالة `process_api_request(endpoint: str, data: dict)` تتحقق من:
- أن `endpoint` موجود في قائمة مسارات مسموحة، وإلا تُطلق `EndpointNotFoundError`
- أن `data` تحتوي حقل `"user_id"`، وإلا تُطلق `MissingFieldError`
- أن `user_id` رقم صحيح موجب، وإلا تُطلق `InvalidFieldError`

استخدم هرمية استثناءات (`APIError` كفئة أساسية)، واكتب دالة تستدعي `process_api_request` وتلتقط كل الحالات بوضوح.

✅ الحل

Python

```python
class APIError(Exception):
    """الفئة الأساسية لكل أخطاء API."""
    pass

class EndpointNotFoundError(APIError):
    def __init__(self, endpoint: str):
        self.endpoint = endpoint
        super().__init__(f"المسار '{endpoint}' غير موجود.")

class MissingFieldError(APIError):
    def __init__(self, field: str):
        self.field = field
        super().__init__(f"الحقل المطلوب '{field}' مفقود.")

class InvalidFieldError(APIError):
    def __init__(self, field: str, reason: str):
        self.field = field
        self.reason = reason
        super().__init__(f"الحقل '{field}' غير صالح: {reason}")

VALID_ENDPOINTS = {"/users", "/orders", "/products"}

def process_api_request(endpoint: str, data: dict) -> dict:
    if endpoint not in VALID_ENDPOINTS:
        raise EndpointNotFoundError(endpoint)

    if "user_id" not in data:
        raise MissingFieldError("user_id")

    user_id = data["user_id"]
    if not isinstance(user_id, int) or user_id <= 0:
        raise InvalidFieldError("user_id", "يجب أن يكون رقماً صحيحاً موجباً")

    return {"status": "success", "endpoint": endpoint, "user_id": user_id}

def handle_request(endpoint: str, data: dict) -> None:
    try:
        result = process_api_request(endpoint, data)
    except EndpointNotFoundError as e:
        print(f"404 - {e}")
    except MissingFieldError as e:
        print(f"400 - حقل مفقود: {e}")
    except InvalidFieldError as e:
        print(f"400 - حقل غير صالح: {e}")
```

```
        except APIError as e:
            print(f"500 - خطأ API غير متوقع: {e}")
        else:
            print(f"200 - {result}")

handle_request("/comments", {"user_id": 1})         # 404
handle_request("/users", {})                         # 400 مفقود
handle_request("/users", {"user_id": -5})            # 400 غير صالح
handle_request("/users", {"user_id": 42})            # 200 نجاح
```

11.12 اختبر فهمك (Quiz)

❓ السؤال 1: ما الفرق بين else و finally في try/except؟

الإجابة: كتلة `else` تُنفَّذ فقط إذا **لم يحدث أي خطأ** في `try`. كتلة `finally` تُنفَّذ **دائماً**، سواء حدث خطأ أم لا، وحتى لو كان هناك `return` داخل `try` أو `except`. `finally` مخصصة لعمليات التنظيف الحتمية (إغلاق ملفات، اتصالات).

❓ السؤال 2: لماذا تُعتبر except: الفارغة (bare except) ممارسة خطيرة؟

الإجابة: لأنها تلتقط **كل** أنواع الاستثناءات دون تمييز، بما فيها `KeyboardInterrupt` و`SystemExit` التي يجب أن تمر دون إعاقة. كما أنها تُخفي الأخطاء الحقيقية دون رسالة واضحة، مما يجعل تصحيح الأخطاء لاحقاً صعباً جداً (Silent Failure). الأفضل دائماً تحديد نوع الخطأ المتوقع بدقة.

❓ السؤال 3: متى يجب أن تستخدم استثناءً مخصصاً بدلاً من ValueError عامة؟

الإجابة: عندما يكون الخطأ مرتبطاً بمنطق عمل خاص بتطبيقك (Business Logic)، مثل `InsufficientFundsError` في نظام بنكي. هذا يجعل الكود أوضح للقراءة، ويسمح بمعالجة دقيقة لكل سيناريو على حدة، ويوثق نية الكود ذاتياً دون الحاجة لتعليقات إضافية.

❓ السؤال 4: ما الفرق بين EAFP و LBYL، ولماذا تُفضّل Python عادة EAFP؟

الإجابة: LBYL (Look Before You Leap) يعني التحقق من الشرط قبل التنفيذ (مثل `if os.path.exists(f):`)، بينما EAFP (Easier to Ask Forgiveness than Permission) يعني التنفيذ المباشر مع معالجة الخطأ إن حدث (`try/except`). Python تُفضّل EAFP لأنه يتجنب حالات Race Condition، وغالباً أداؤه أفضل عندما يكون الفشل نادراً، وينتج كوداً أنظف بدون تحققات متداخلة.

❓ السؤال 5: لماذا لا يجب استخدام assert للتحقق من مدخلات المستخدم؟

الإجابة: لأن عبارات `assert` تُحذف بالكامل من الكود عند تشغيل Python بوضع التحسين (`python -O`)، مما يعني أن أي تحقق مبني عليها قد يختفي تماماً في بيئة الإنتاج، تاركاً الباب مفتوحاً لبيانات غير صالحة. `assert` مخصصة فقط للتحقق من افتراضات داخلية أثناء التطوير، وليس لمعالجة أخطاء قادمة من مصادر خارجية.

11.13 ملخص الفصل (Cheat Sheet)

Python

```python
# الصيغة الكاملة
try:
    risky_code()
except SpecificError as e:
    handle_specific_case(e)
except (ErrorA, ErrorB) as e:
    handle_multiple_cases(e)
except Exception as e:       # الأعم - دائماً في الأخير
    handle_general_case(e)
else:
    runs_if_no_error()
finally:
    always_runs()            # تنظيف الموارد

# إطلاق خطأ
raise ValueError("رسالة واضحة")

# إعادة إطلاق
except SomeError:
    raise  # بدون معامل

# تسلسل الاستثناءات
raise NewError("...") from original_error

# استثناء مخصص
class MyAppError(Exception):
    pass

class SpecificError(MyAppError):
    def __init__(self, detail: str):
        self.detail = detail
        super().__init__(f"فشل بسبب: {detail}")

# assert - للتطوير فقط، ليس لمدخلات المستخدم
assert condition, "رسالة إذا فشل الشرط"
```

الموقف	الحل الموصى به
خطأ متوقع من مستخدم/ملف/شبكة	`try/except` + استثناء محدد
التحقق من افتراض داخلي أثناء التطوير	`assert`
منطق عمل خاص بتطبيقك	استثناء مخصص يرث من `Exception`
تنظيف موارد (ملفات، اتصالات)	`finally` أو أفضل: `with` (الفصل 16)
تسجيل الأخطاء في مشروع حقيقي	`logging` بدلاً من `print`
خطأ غير متوقع تماماً	`except Exception as e:` (نادراً، وبحذر)
لا تفعل أبداً	`except:` فارغة، أو `except Exception: pass`

11.14 ماذا بعد؟

الآن برامجك لا تنهار فجأة أمام مستخدميك — بل تتعامل مع المواقف الاستثنائية بأناقة واحترافية. هذه مهارة جوهرية تفصل الكود «الذي يعمل في حالات مثالية» عن الكود «الجاهز للعالم الحقيقي».

لكن حتى الآن، كل دوالنا كانت «عادية» نسبياً. في Python توجد أدوات أكثر قوة وأناقة: دوال تُكتب في سطر واحد (Lambda)، دوال تُعدّل سلوك دوال أخرى (Decorators)، دوال «تتذكر» حالتها بين الاستدعاءات (Generators و Closures). هذه الأدوات هي ما يميز الكود البايثوني (Pythonic) الاحترافي، وتُستخدم بكثافة في كل مكتبات Python الكبرى مثل Flask وFastAPI وPytest.

في **الفصل الثاني عشر: الدوال المتقدمة**، سنفتح هذا الباب بالكامل، بعمق وبأمثلة عملية تطبيقية.

فصل 12 / 26 · الجزء الثالث

12

الدوال المتقدمة (Lambda, Decorators, Generators, Closures)

9 دقيقة قراءة | 11 قسمًا | 41 مثالًا | 1,875 كلمة

في هذا الفصل

12.1 لماذا هذا الفصل هو نقطة التحوّل نحو الاحتراف؟
12.2 دوال Lambda: الدوال المجهولة
12.3 الإغلاقات (Closures): الدوال التي تتذكر
12.4 المزخرفات (Decorators): تعديل سلوك الدوال
12.5 المولّدات (Generators): الكفاءة في الذاكرة
12.6 جدول مقارنة شامل: المفاهيم الأربعة
12.7 مشروع تطبيقي: نظام مراقبة أداء مع Decorators وGenerators
12.8 تمارين
12.9 اختبر فهمك (Quiz)
12.10 ملخص الفصل (Cheat Sheet)
12.11 ماذا بعد؟

12.1 لماذا هذا الفصل هو نقطة التحوّل نحو الاحتراف؟

إذا فتحت الكود المصدري لأي مكتبة Python شهيرة — Flask، FastAPI، Django، Pytest — ستجد هذه الأربعة مفاهيم في كل مكان: `@app.route(...)`، `@pytest.fixture`، `yield`، دوال `lambda` صغيرة كمعاملات. هذه ليست «حيل متقدمة» للمحترفين فقط؛ إنها **اللغة الأساسية** التي يتحدث بها كود Python الحديث.

حتى الآن، تعاملت مع الدوال كـ«آلات»: تُدخل مُدخلات، تحصل على مُخرجات. في هذا الفصل، ستكتشف أن الدوال في Python أقوى بكثير من ذلك:

- يمكنك كتابة دالة **صغيرة جداً وسريعة** دون حتى تسميتها (Lambda)
- يمكنك **تعديل سلوك دالة موجودة** دون تعديل كودها الأصلي (Decorators)
- يمكنك كتابة دالة **تتذكر مكانها** بين الاستدعاءات، وتنتج قيماً واحدة تلو الأخرى بكفاءة هائلة في الذاكرة (Generators)
- يمكنك جعل دالة «تحمل معها» بيانات من بيئتها حتى بعد انتهاء تلك البيئة (Closures)

هذه المفاهيم الأربعة مترابطة، وستبني كل واحدة على سابقتها. لنبدأ.

12.2 دوال Lambda: الدوال المجهولة

الفكرة الأساسية

Lambda هي دالة صغيرة «مجهولة» (بدون اسم)، تُكتب في سطر واحد، ومُصممة للاستخدام السريع مرة واحدة أو كمعامل لدالة أخرى.

```python
# دالة عادية
def square(x: int) -> int:
    return x ** 2

# نفس الدالة بصيغة lambda
square_lambda = lambda x: x ** 2

print(square(5))         # 25
print(square_lambda(5))  # 25
```

الصيغة العامة: `lambda parameters: expression`

> ⚠ **قيود مهمة على lambda:**
> - جسمها **تعبير واحد فقط** (Expression)، وليس سلسلة من الجُمل (Statements)
> - لا يمكن أن تحتوي على `if/else` كجُمل كاملة (لكن يمكن استخدام Ternary Expression)
> - لا تحتوي `return` صراحة — القيمة الناتجة من التعبير تُعاد تلقائياً

```python
# lambda مع عدة معاملات
add = lambda a, b: a + b
print(add(3, 4))  # 7

# lambda مع Ternary Expression
is_even = lambda n: "زوجي" if n % 2 == 0 else "فردي"
print(is_even(7))  # فردي

# lambda بدون معاملات
greet = lambda: "مرحباً!"
print(greet())  # مرحباً!
```

متى تُستخدم lambda فعلياً؟

⚠ **تحذير احترافي مهم:** لا تُسند دالة lambda لمتغير كما في `square_lambda = lambda x: x**2` في الكود الحقيقي! هذا يُعتبر ممارسة سيئة (حتى أن PEP 8 ينص على تجنبها صراحة). إذا احتجت لدالة لها اسم، استخدم `def` العادية:

```python
# ❌ استخدم - ممارسة سيئة def بدلاً منها
square = lambda x: x ** 2

# ✅ أفضل
def square(x: int) -> int:
    return x ** 2
```

الاستخدام الصحيح والشائع لـ lambda: كمعامل مؤقت «يُستهلك فوراً» داخل دالة أخرى، دون الحاجة لتسميته:

```python
students = [
    {"name": "Ahmed", "score": 85},
    {"name": "Sara", "score": 92},
    {"name": "Omar", "score": 78}
]

# الاستخدام الأشهر لـ lambda - مع sorted()
sorted_by_score = sorted(students, key=lambda s: s["score"], reverse=True)
for s in sorted_by_score:
    print(s["name"], s["score"])

# مع min() و max()
youngest_scorer = min(students, key=lambda s: s["score"])
print(youngest_scorer)
```

🔄 **مقارنة:** لماذا لا نكتب دالة `def get_score(s): return s["score"]` بدلاً من lambda؟ كلاهما صحيح! لكن عندما تكون المنطق بسيطاً جداً (سطر واحد) ولن يُستخدم إلا هنا فقط، lambda توفر إيجازاً دون تلويث مساحة الاسم (Namespace) بدالة صغيرة لن تُستخدم مجدداً. القاعدة العملية: **إذا كانت الدالة ستُستخدم أكثر من مرة، أو منطقها معقد قليلاً، استخدم `def`. إذا كانت بسيطة واستخدامها لمرة واحدة فقط كمعامل، استخدم lambda.**

Lambda مع map() و filter()

```python
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

# map(): تطبيق دالة على كل عنصر
squared = list(map(lambda x: x ** 2, numbers))
print(squared)  # [1, 4, 9, 16, 25, 36, 49, 64, 81, 100]

# filter(): تصفية العناصر حسب شرط
evens = list(filter(lambda x: x % 2 == 0, numbers))
print(evens)  # [2, 4, 6, 8, 10]
```

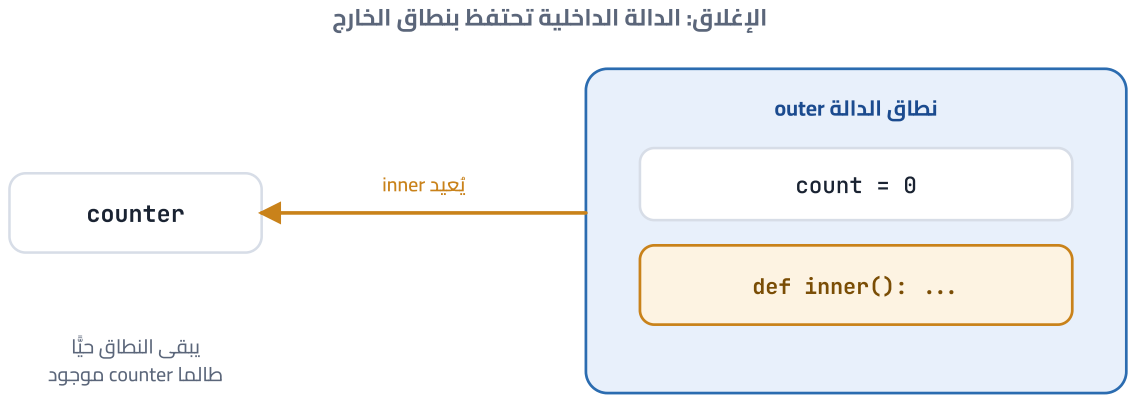
مقارنة - أيهما أفضل: map/filter أم List Comprehension؟

```python
# بـ map/filter
result = list(map(lambda x: x ** 2, filter(lambda x: x % 2 == 0, numbers)))

# بـ List Comprehension - الأسلوب الـPythonic الموصى به
result = [x ** 2 for x in numbers if x % 2 == 0]
```

معظم خبراء Python (بمن فيهم مُنشئ اللغة نفسه Guido van Rossum) يُفضّلون **List Comprehension** على `map/filter` لأنها أوضح وأقرب للغة الطبيعية. لكن من المهم معرفة `map` و`filter` لأنك ستجدهما كثيراً في كود الآخرين، وفي بعض الحالات (خصوصاً مع دوال موجودة مسبقاً وليس lambda) تكون `map` مفيدة وواضحة:

```python
# استخدام map مقبول تماماً وشائع - مع دالة موجودة
str_numbers = ["1", "2", "3"]
int_numbers = list(map(int, str_numbers))
print(int_numbers)  # [1, 2, 3]
```

12.3 الإغلاقات (Closures): الدوال التي تتذكر



شكل 12.1 — الإغلاق يحتفظ بنطاق الدالة الخارجية

الدوال المتداخلة (Nested Functions)

قبل فهم Closures، لنفهم أن Python تسمح بتعريف دالة **داخل** دالة أخرى:

```python
def outer_function():
    message = "مرحباً من outer!"

    def inner_function():
        print(message)  # inner تصل لمتغير outer بفضل قاعدة LEGB (الفصل 10)

    inner_function()

outer_function()  # مرحباً من outer!
```

ما هو الـ Closure فعلياً؟

Closure يحدث عندما تُرجع دالة داخلية من دالة خارجية، وتبقى الدالة الداخلية «تتذكر» المتغيرات من بيئة الدالة الخارجية، **حتى بعد انتهاء تنفيذ الدالة الخارجية!**

```python
def make_multiplier(factor: int):
    """ترجع دالة تضرب أي رقم بـ factor."""
    def multiplier(number: int) -> int:
        return number * factor  # factor "مُغلَقة" داخل هذه الدالة
    return multiplier

double = make_multiplier(2)
triple = make_multiplier(3)

print(double(5))   # 10
print(triple(5))   # 15
```

ماذا حدث هنا بالضبط؟

1. `make_multiplier(2)` تُنفَّذ، فيُنشأ متغير محلي `factor = 2`
2. تُعرَّف دالة `multiplier` داخلياً، وتُستخدم `factor` بداخلها
3. `make_multiplier` تُعيد `multiplier` **كقيمة** (بدون استدعائها)، وتنتهي
4. منطقياً، يجب أن يختفي `factor` الآن لأن الدالة انتهت... **لكنه لا يختفي!**
5. Python تحتفظ بـ `factor` «محبوسة» داخل الدالة المُعادة، لأن `multiplier` ما زالت تحتاجها

هذا يُسمى **Closure**: الدالة الداخلية «تُغلِق» على المتغيرات من بيئتها الخارجية.

```python
# تأكيد عملي أن كل closure يحتفظ بنسخته الخاصة
print(double(10))  # 20 - لا تزال تتذكر factor=2
print(triple(10))  # 30 - لا تزال تتذكر factor=3
```

استخدام `nonlocal` لتعديل متغير من البيئة الخارجية

```python
def make_counter():
    """تُرجع دالة تعد تصاعدياً في كل استدعاء."""
    count = 0

    def increment():
        nonlocal count  # بدون هذه الكلمة، سيحدث UnboundLocalError
        count += 1
        return count

    return increment

counter1 = make_counter()
print(counter1())  # 1
print(counter1())  # 2
print(counter1())  # 3

counter2 = make_counter()  # عداد جديد مستقل تماماً
print(counter2())  # 1  -- لا علاقة له بـ counter1!
```

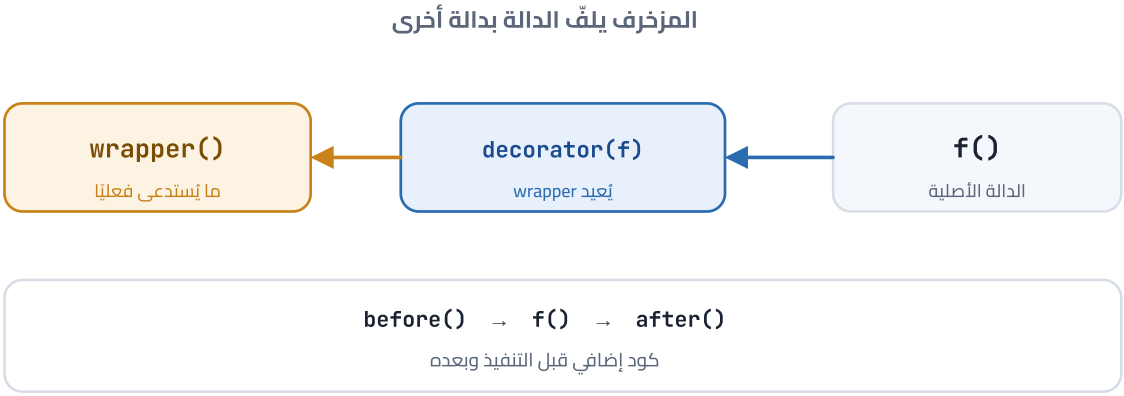
نصيحة الخبراء: `nonlocal` تشبه `global` (الفصل 10)، لكنها تشير لمتغير في **الدالة المحيطة (Enclosing)** وليس المستوى العام (Global) للملف. استخدمها فقط عندما تحتاج لتعديل متغير من البيئة الخارجية داخل closure.

تطبيق عملي: Closures لإنشاء دوال مخصصة

```python
def make_validator(min_length: int):
    """تُنشئ دالة تحقق مخصصة بحد أدنى معين."""
    def validate(text: str) -> bool:
        return len(text) >= min_length
    return validate

validate_username = make_validator(5)
validate_password = make_validator(8)

print(validate_username("ali"))      # False (أحرف فقط 3)
print(validate_username("ahmed99"))  # True
print(validate_password("short"))    # False
print(validate_password("secure123"))  # True
```

> **مقارنة مع البرمجة الكائنية:** Closures هي في الحقيقة طريقة «خفيفة الوزن» لتحقيق ما تحققه الكائنات (Objects) عادة: بيانات مرتبطة بسلوك. سنرى في الفصل 13 أن الـ Classes توفر نفس الفكرة لكن بقدرات أكبر عندما تحتاج حالة معقدة متعددة الخصائص. القاعدة العملية: استخدم Closures للحالات البسيطة (متغير أو اثنين)، واستخدم Classes عندما تكبر الحاجة.

12.4 المزخرفات (Decorators): تعديل سلوك الدوال



شكل 12.2 — المزخرف يلفّ الدالة بدالة أخرى

الفكرة الأساسية

Decorator هو دالة تأخذ دالة أخرى كمُدخل، وتُرجع دالة جديدة «تُحيط» بسلوك الدالة الأصلية — تُضيف سلوكاً قبل و/أو بعد تنفيذها، **دون تعديل كود الدالة الأصلية نفسه**.

لنبني Decorator خطوة بخطوة لفهم آلية عمله الداخلية:

```python
def my_decorator(func):
    """decorator بسيط يطبع رسالة قبل وبعد تنفيذ الدالة."""
    def wrapper():
        print("🔵 قبل تنفيذ الدالة.  ")
        func()  # استدعاء الدالة الأصلية
        print("🔴 بعد تنفيذ الدالة.  ")
    return wrapper

def say_hello():
    print("مرحباً!")

# التطبيق اليدوي (بدون @)
decorated_hello = my_decorator(say_hello)
decorated_hello()
```

الناتج:

```
🔵 قبل تنفيذ الدالة.
مرحباً!
🔴 بعد تنفيذ الدالة.
```

الصيغة الأنيقة: `@`

Python توفر صياغة خاصة (Syntactic Sugar) لتطبيق الـ decorator مباشرة:

```python
def my_decorator(func):
    def wrapper():
        print("🔵 قبل تنفيذ الدالة.  ")
        func()
        print("🔴 بعد تنفيذ الدالة.  ")
    return wrapper

@my_decorator  # هذا يُعادل: say_hello = my_decorator(say_hello)
def say_hello():
    print("مرحباً!")

say_hello()  # نفس النتيجة السابقة تماماً!
```

`@my_decorator` فوق `def say_hello():` هي ببساطة اختصار لـ `say_hello = my_decorator(say_hello)`.

Decorators مع دوال لها معاملات

المشكلة: دالة `wrapper` أعلاه لا تقبل أي معاملات. لجعل الـ decorator يعمل مع **أي** دالة بغض النظر عن معاملاتها، نستخدم `*args` و `**kwargs`:

```python
def my_decorator(func):
    def wrapper(*args, **kwargs):
        print(f"🔵 استدعاء {func.__name__} بالمعاملات: {args}, {kwargs}")
        result = func(*args, **kwargs)
        print(f"🔴 النتيجة: {result}")
        return result
    return wrapper

@my_decorator
def add(a: int, b: int) -> int:
    return a + b

result = add(3, 5)
```

الناتج:

```
🔵 استدعاء add بالمعاملات: (3, 5), {}
🔴 النتيجة: 8
```

⚠️ **خطأ شائع:** نسيان `return result` في `wrapper`، مما يجعل أي دالة مُزخرفة تُرجع `None` دائماً بغض النظر عن قيمتها الحقيقية:

```python
def bad_decorator(func):
    def wrapper(*args, **kwargs):
        func(*args, **kwargs)  # ⚠️ نُفِّذت لكن النتيجة ضاعت!
        # هنا لا يوجد return!
    return wrapper

@bad_decorator
def add(a, b):
    return a + b

print(add(3, 5))  # None 💥 بدلاً من 8!
```

الحفاظ على معلومات الدالة الأصلية: `functools.wraps`

```python
def my_decorator(func):
    def wrapper(*args, **kwargs):
        return func(*args, **kwargs)
    return wrapper

@my_decorator
def greet(name: str) -> str:
    """ترجع تحية للمستخدم."""
    return f"مرحباً {name}"

print(greet.__name__)  # wrapper  ⚠️ فقدنا الاسم الأصلي!
print(greet.__doc__)   # None     ⚠️ فقدنا التوثيق الأصلي!
```

الحل:

```python
from functools import wraps

def my_decorator(func):
    @wraps(func)  # يحافظ على __name__ و __doc__ الأصليين
    def wrapper(*args, **kwargs):
        return func(*args, **kwargs)
    return wrapper

@my_decorator
def greet(name: str) -> str:
    """ترجع تحية للمستخدم."""
    return f"مرحباً {name}"

print(greet.__name__)  # greet ✅
print(greet.__doc__)   # ترجع تحية للمستخدم. ✅
```

> 💡 **نصيحة الخبراء - قاعدة ذهبية: كل decorator تكتبه يجب أن يحتوي `@wraps(func)`** فوق تعريف `wrapper`. هذا جزء أساسي من «النمط الصحيح» لكتابة decorators في Python، ونسيانه يُسبب مشاكل غامضة لاحقاً في الأدوات التي تعتمد على `__name__` مثل أنظمة التوثيق التلقائي وبعض أطر العمل.

Decorators عملية حقيقية

1. قياس زمن التنفيذ:

Python

```python
import time
from functools import wraps

def measure_time(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        start = time.perf_counter()
        result = func(*args, **kwargs)
        end = time.perf_counter()
        print(f"⏱️ {func.__name__} استغرقت {end - start:.4f} ثانية")
        return result
    return wrapper

@measure_time
def slow_calculation(n: int) -> int:
    total = sum(i ** 2 for i in range(n))
    return total

result = slow_calculation(1_000_000)
print(result)
```

2. تسجيل الاستدعاءات (Logging):

Python

```python
from functools import wraps

def log_calls(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        print(f"📝 استُدعيت: {func.__name__}({args}, {kwargs})")
        result = func(*args, **kwargs)
        print(f"📝 أُرجعت: {result}")
        return result
    return wrapper

@log_calls
def divide(a: float, b: float) -> float:
    return a / b

divide(10, 2)
```

3. إعادة المحاولة عند الفشل (Retry):

Python

```python
from functools import wraps
import random

def retry(max_attempts: int = 3):
    """Decorator factory: يُرجع decorator حقيقياً حسب max_attempts."""
    def decorator(func):
        @wraps(func)
        def wrapper(*args, **kwargs):
            for attempt in range(1, max_attempts + 1):
                try:
                    return func(*args, **kwargs)
                except Exception as e:
                    print(f"⚠️ محاولة {attempt} فشلت: {e}")
                    if attempt == max_attempts:
                        raise
            return None
        return wrapper
    return decorator

@retry(max_attempts=3)
def unreliable_operation():
    if random.random() < 0.7:
        raise ConnectionError("فشل الاتصال!")
    return "✅ نجحت العملية!"

print(unreliable_operation())
```

ملاحظة مهمة: `retry(max_attempts=3)` هذا نمط يُسمى **Decorator Factory** — دالة تُرجع decorator، مما يسمح بتمرير معاملات مخصصة للـ decorator نفسه. لاحظ وجود 3 مستويات من الدوال المتداخلة: `retry` → `decorator` → `wrapper`.

4. التحقق من الصلاحيات (نموذج شائع في تطوير الويب):

```python
from functools import wraps

def require_login(func):
    @wraps(func)
    def wrapper(user: dict, *args, **kwargs):
        if not user.get("is_logged_in"):
            print("❌ يجب تسجيل الدخول أولاً.")
            return None
        return func(user, *args, **kwargs)
    return wrapper

@require_login
def view_profile(user: dict) -> None:
    print(f"👤 ملف {user['name']} الشخصي")

guest = {"name": "زائر", "is_logged_in": False}
logged_in_user = {"name": "أحمد", "is_logged_in": True}

view_profile(guest)          # ❌ يجب تسجيل الدخول أولاً.
view_profile(logged_in_user) # 👤 ملف أحمد الشخصي
```

تطبيق عدة decorators معاً

```python
@measure_time
@log_calls
def process(n: int) -> int:
    return n ** 2

process(10)
```

⚠ **تحذير مهم:** ترتيب الـ decorators مهم! تُطبَّق **من الأسفل للأعلى**. الكود أعلاه يُعادل: `process = measure_time(log_calls(process))`. أي أن `log_calls` تُطبَّق أولاً (الأقرب للدالة)، ثم `measure_time` تُحيط بالنتيجة بالكامل.

Decorators مدمجة شهيرة في Python

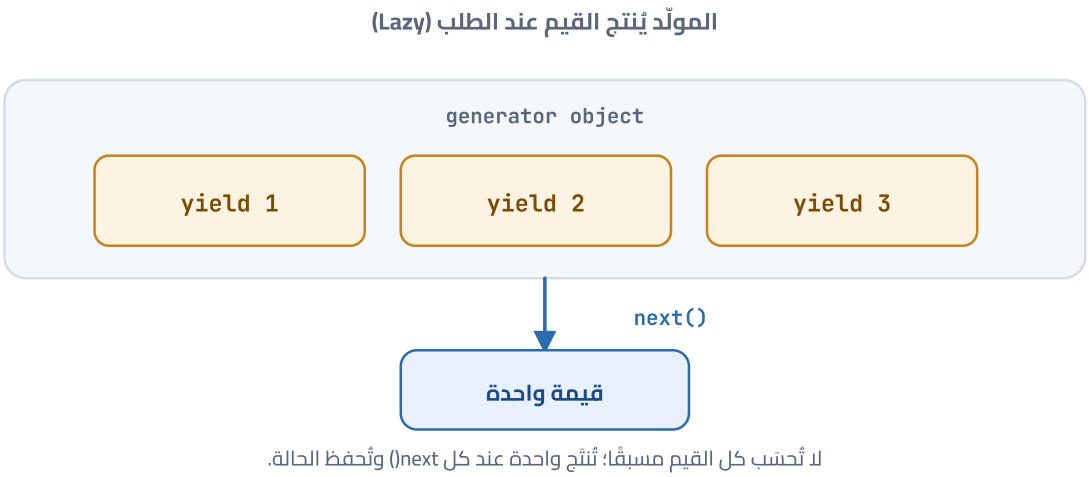
سنتعمق في هذه في الفصل 13 و14، لكن إليك لمحة سريعة لأنها decorators حقيقية تستخدم نفس المبدأ:

Python

```python
class Circle:
    def __init__(self, radius: float):
        self._radius = radius

    @property  # decorator يجعل - مدمج method تُستخدم كخاصية
    def area(self) -> float:
        return 3.14159 * self._radius ** 2

c = Circle(5)
print(c.area)  # 78.53975 -- بدون أقواس استدعاء!

from functools import lru_cache

@lru_cache(maxsize=None)  # تذكر - caching (10 الفصل في يدوياً كتبناه) تلقائي!
def fibonacci(n: int) -> int:
    if n <= 1:
        return n
    return fibonacci(n - 1) + fibonacci(n - 2)

print(fibonacci(30))  # بفضل جداً سريع lru_cache، رغم أن fibonacci بطيئة أصلاً
```

نصيحة الخبراء: `@lru_cache` هي نفس فكرة دالة `memoize` التي بنيتها يدوياً في تمرين الفصل العاشر، لكنها جاهزة ومُحسّنة من المكتبة القياسية. استخدمها دائماً بدلاً من كتابة caching يدوي، إلا إذا احتجت سلوكاً مخصصاً جداً.

12.5 المولّدات (Generators): الكفاءة في الذاكرة



شكل 12.3 — المولّد يُنتج القيم عند الطلب

المشكلة: القوائم الكبيرة تستهلك ذاكرة هائلة

```python
def get_squares(n: int) -> list[int]:
    """تُنشئ قائمة كاملة بكل المربعات - تستهلك ذاكرة كبيرة!"""
    return [i ** 2 for i in range(n)]

# إذا n = 10,000,000، هذا ينشئ قائمة ضخمة بالكامل في الذاكرة فوراً!
squares = get_squares(10_000_000)
```

ماذا لو كنت تحتاج فقط لمعالجة هذه الأرقام واحداً تلو الآخر، دون الحاجة لتخزينها كلها في نفس اللحظة؟ هنا يأتي دور **Generators**.

إنشاء Generator بـ `yield`

```python
def get_squares_generator(n: int):
    """تُنتج المربعات واحداً تلو الآخر، دون تخزين كل شيء في الذاكرة."""
    for i in range(n):
        yield i ** 2  # yield بدلاً من return

gen = get_squares_generator(5)
print(gen)  # <generator object get_squares_generator at 0x...>

print(next(gen))  # 0
print(next(gen))  # 1
print(next(gen))  # 4
print(next(gen))  # 9
print(next(gen))  # 16
# print(next(gen))  # 💥 StopIteration - انتهت القيم
```

الفرق الجوهري بين `return` و `yield`:

- `return` تُنهي الدالة فوراً وتُعيد قيمة واحدة
- `yield` «تُجمّد» الدالة في مكانها، تُعيد قيمة واحدة، **وتتذكر مكانها بالضبط** لتكمل من هناك عند الاستدعاء التالي

```python
# اليدوية next() مباشرة بدلاً من for غالباً نستخدم
for square in get_squares_generator(5):
    print(square)
# 0
# 1
# 4
# 9
# 16
```

توضيح آلية العمل خطوة بخطوة

Python

```
def demo_generator():
    print("البداية")
    yield 1
    print("بعد العنصر الأول")
    yield 2
    print("بعد العنصر الثاني")
    yield 3
    print("النهاية")

gen = demo_generator()  # لا شيء يُطبع بعد! الدالة لم تبدأ التنفيذ بعد

print("--- أول next ---")
print(next(gen))  # البداية ثم 1

print("--- ثاني next ---")
print(next(gen))  # بعد العنصر الأول ثم 2

print("--- ثالث next ---")
print(next(gen))  # بعد العنصر الثاني ثم 3
```

الناتج:

```
---  أول next ---
البداية
1
---  ثاني next ---
بعد العنصر الأول
2
---  ثالث next ---
بعد العنصر الثاني
3
```

هذا يكشف حقيقة مهمة: مجرد استدعاء `demo_generator()` **لا يُنفّذ أي كود فعلياً**! الدالة تبدأ التنفيذ فقط عند أول `next()`، وتتوقف عند كل `yield`، منتظرة الطلب التالي.

Generator Expressions: نسخة مختصرة من Comprehension

```python
# List Comprehension - ينشئ القائمة بالكامل فوراً في الذاكرة
squares_list = [x ** 2 for x in range(1_000_000)]

# Generator Expression - ينتج القيم عند الطلب فقط (lazy evaluation)
squares_gen = (x ** 2 for x in range(1_000_000))

print(type(squares_list))  # <class 'list'>
print(type(squares_gen))   # <class 'generator'>
```

مقارنة الذاكرة - دليل عملي:

```python
import sys

squares_list = [x ** 2 for x in range(1_000_000)]
squares_gen = (x ** 2 for x in range(1_000_000))

print(f"حجم القائمة: {sys.getsizeof(squares_list):,} بايت")
print(f"حجم المولّد: {sys.getsizeof(squares_gen):,} بايت")
```

الناتج التقريبي:

```
حجم القائمة: 8,448,728 بايت  (حوالي 8 ميجابايت!)
حجم المولّد: 200 بايت          (ثابت بغض النظر عن n!)
```

هذا هو السبب الجوهري لاستخدام Generators: **كفاءة هائلة في الذاكرة** عند التعامل مع بيانات كبيرة أو تدفقات لا نهائية.

تحذير مهم: Generator يُستهلك **مرة واحدة فقط**! بعد المرور على كل عناصره، لا يمكن إعادة استخدامه:

```python
gen = (x for x in range(3))

print(list(gen))  # [0, 1, 2]
print(list(gen))  # []  ⚠ فارغة! تم استهلاك المولّد بالكامل
```

إذا احتجت المرور على البيانات أكثر من مرة، استخدم `list()` أو أعد إنشاء الـ generator من جديد.

متى تستخدم Generator بدلاً من List؟

الحالة	الخيار الأفضل
تحتاج كل العناصر في الذاكرة فعلياً للوصول العشوائي (`lst[5]`)	List
تحتاج المرور على البيانات أكثر من مرة	List
البيانات ضخمة جداً أو لا نهائية	Generator
تحتاج فقط المرور مرة واحدة بالترتيب	Generator
معالجة ملفات ضخمة سطراً سطراً	Generator
سلسلة عمليات متتالية (Pipeline) على بيانات كبيرة	Generator

تطبيق عملي: معالجة ملف ضخم

Python

```python
def read_large_file(file_path: str):
    """تقرأ ملفاً ضخماً سطراً سطراً دون تحميله بالكامل في الذاكرة."""
    with open(file_path, "r", encoding="utf-8") as file:
        for line in file:
            yield line.strip()

def count_lines_with_keyword(file_path: str, keyword: str) -> int:
    """تحسب عدد الأسطر المحتوية على كلمة معينة، بكفاءة ذاكرة عالية."""
    count = 0
    for line in read_large_file(file_path):
        if keyword in line:
            count += 1
    return count

# حتى لو كان الملف بحجم عدة جيجابايت، هذا الكود يستهلك ذاكرة ثابتة تقريباً!
# count = count_lines_with_keyword("huge_log.txt", "ERROR")
```

تمرين توضيحي: سلسلة Fibonacci لا نهائية

```python
def infinite_fibonacci():
    """مولّد لا نهائي لمتتالية فيبوناتشي."""
    a, b = 0, 1
    while True:  # آمنة هنا لأن المولّد lazy! - حلقة لا نهائية
        yield a
        a, b = b, a + b

fib = infinite_fibonacci()
first_10 = [next(fib) for _ in range(10)]
print(first_10)
# [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
```

نصيحة الخبراء: لاحظ أن `while True` هنا **آمنة تماماً**! في الفصل السابع حذّرنا من الحلقات اللا نهائية، لكن داخل generator، الحلقة تتوقف عند كل `yield` ولا تُكمل أبداً إلا عند طلب القيمة التالية صراحة. هذا نمط شائع جداً ومفيد لإنتاج تدفقات بيانات لا نهائية نظرياً (مثل قراءة بيانات حساسة بالوقت، أو تسلسل أرقام).

دالة `itertools` : صندوق أدوات Generators المتقدم

```python
from itertools import islice, count, cycle

# islice: أخذ جزء محدد من generator لا نهائي
fib = infinite_fibonacci()
first_5 = list(islice(fib, 5))
print(first_5)  # [0, 1, 1, 2, 3]

# count: عداد لا نهائي يبدأ من رقم معين
for i in islice(count(10, 2), 5):  # 2 يبدأ من 10، بخطوة
    print(i, end=" ")
print()  # 10 12 14 16 18

# cycle: تكرار عناصر قائمة بشكل لا نهائي
colors = cycle(["red", "green", "blue"])
for color in islice(colors, 7):
    print(color, end=" ")
print()  # red green blue red green blue red
```

12.6 جدول مقارنة شامل: المفاهيم الأربعة

المفهوم	الهدف الأساسي	متى تستخدمه
Lambda	دالة مجهولة سريعة الكتابة	كمعامل مؤقت لـ `sorted` , `map` , `filter` , وما شابه
Closure	دالة تتذكر بيانات من بيئتها	إنشاء دوال «مُخصّصة» بإعدادات ثابتة (factories)
Decorator	تعديل سلوك دالة دون تعديل كودها	Logging, قياس الوقت, التحقق من الصلاحيات, Caching
Generator	إنتاج قيم بكفاءة ذاكرة عالية	بيانات ضخمة, تدفقات لا نهائية, معالجة ملفات كبيرة

12.7 مشروع تطبيقي: نظام مراقبة أداء مع Decorators وGeneratorsg

سنبني نظاماً يدمج كل مفاهيم الفصل: decorator لقياس الأداء وتسجيل الاستدعاءات، وgenerator لمعالجة بيانات ضخمة بكفاءة.

```python
import time
import random
from functools import wraps

# --- الجزء 1: Decorators للمراقبة ---

call_log: list[dict] = []

def monitor(func):
    """decorator يسجل كل استدعاء: الاسم، الوقت المستغرق، النتيجة."""
    @wraps(func)
    def wrapper(*args, **kwargs):
        start = time.perf_counter()
        try:
            result = func(*args, **kwargs)
            status = "نجاح"
        except Exception as e:
            result = str(e)
            status = "فشل"
            raise
        finally:
            duration = time.perf_counter() - start
            call_log.append({
```

```
                "function": func.__name__,
                "duration": round(duration, 6),
                "status": status,
            })
        return result
    return wrapper

def retry(max_attempts: int = 3, delay: float = 0.1):
    """decorator factory لإعادة المحاولة عند الفشل."""
    def decorator(func):
        @wraps(func)
        def wrapper(*args, **kwargs):
            last_error = None
            for attempt in range(1, max_attempts + 1):
                try:
                    return func(*args, **kwargs)
                except Exception as e:
                    last_error = e
                    time.sleep(delay)
            raise last_error
        return wrapper
    return decorator

# --- جزء 2: دوال العمل الفعلية ---

@monitor
@retry(max_attempts=3)
def fetch_data_from_source(source_id: int) -> dict:
    """تحاكي جلب بيانات من مصدر خارجي، قد يفشل أحياناً."""
    if random.random() < 0.3:
        raise ConnectionError(f"فشل الاتصال بالمصدر {source_id}")
    return {"source_id": source_id, "value": random.randint(1, 100)}

# --- جزء 3: Generator لمعالجة البيانات بكفاءة ---

def data_source_generator(count: int):
    """يولّد معرفات مصادر بيانات واحداً تلو الآخر."""
    for i in range(1, count + 1):
        yield i

def process_all_sources(count: int) -> list[dict]:
    """يعالج كل المصادر، مستخدماً generator لتوفير الذاكرة."""
    results = []
    for source_id in data_source_generator(count):
        try:
            data = fetch_data_from_source(source_id)
            results.append(data)
        except ConnectionError as e:
            print(f"❌ فشل نهائي: {e}")
    return results
```

```python
# --- جزء 4: تقرير الأداء ---

def print_performance_report() -> None:
    print("\n" + "=" * 50)
    print("📊 تقرير مراقبة الأداء")
    print("=" * 50)

    total_calls = len(call_log)
    successful = sum(1 for log in call_log if log["status"] == "نجاح")
    total_time = sum(log["duration"] for log in call_log)

    print(f"إجمالي الاستدعاءات: {total_calls}")
    print(f"الناجحة: {successful} | الفاشلة: {total_calls - successful}")
    print(f"إجمالي الوقت: {total_time:.4f} ثانية")

    if call_log:
        avg_time = total_time / total_calls
        print(f"متوسط الوقت لكل استدعاء: {avg_time:.6f} ثانية")
    print("=" * 50)

# --- التشغيل ---
if __name__ == "__main__":
    results = process_all_sources(10)

    print(f"\n✅ تم جلب {len(results)} نتيجة بنجاح:")
    for r in results:
        print(f"  مصدر {r['source_id']}: قيمة = {r['value']}")

    print_performance_report()
```

مثال ناتج (قد يختلف بسبب العشوائية):

```
✅ تم جلب 10 نتيجة بنجاح:
  مصدر 1: قيمة = 42
  مصدر 2: قيمة = 17
...

==================================================
📊 تقرير مراقبة الأداء
==================================================
إجمالي الاستدعاءات: 13
الناجحة: 10 | الفاشلة: 3
إجمالي الوقت: 0.3201 ثانية
متوسط الوقت لكل استدعاء: 0.024623 ثانية
==================================================
```

<details>
<summary>✅ تحدي إضافي: إضافة decorator لتحديد السرعة (Rate Limiting) - الحل</summary>

Python

```python
import time
from functools import wraps

def rate_limit(calls_per_second: float):
    """decorator factory يمنع استدعاء الدالة أسرع من معدل معين."""
    min_interval = 1.0 / calls_per_second
    last_called = [0.0]  # قائمة بعنصر واحد لتجنب مشاكل nonlocal في closures متداخلة

    def decorator(func):
        @wraps(func)
        def wrapper(*args, **kwargs):
            elapsed = time.perf_counter() - last_called[0]
            wait_time = min_interval - elapsed
            if wait_time > 0:
                time.sleep(wait_time)
            last_called[0] = time.perf_counter()
            return func(*args, **kwargs)
        return wrapper
    return decorator

@rate_limit(calls_per_second=2)  # بحد أقصى استدعائين في الثانية
def api_call(n: int) -> str:
    return f"نتيجة الاستدعاء {n}"

start = time.perf_counter()
for i in range(5):
    print(api_call(i))
print(f"الوقت الإجمالي: {time.perf_counter() - start:.2f} ثانية")  # ≈ 2 ثانية
```

</details>

12.8 تمارين

🟢 مستوى مبتدئ

تمرين 1: استخدم `lambda` مع `sorted()` لترتيب قائمة `["banana", "kiwi", "apple", "fig"]` حسب طول الكلمة.

تمرين 2: اكتب generator باسم `countdown(n)` يُنتج الأرقام من n إلى 1 تنازلياً.

✅ الحل

Python

```python
# تمرين 1
fruits = ["banana", "kiwi", "apple", "fig"]
sorted_fruits = sorted(fruits, key=lambda word: len(word))
print(sorted_fruits)  # ['fig', 'kiwi', 'apple', 'banana']

# تمرين 2
def countdown(n: int):
    while n > 0:
        yield n
        n -= 1

for num in countdown(5):
    print(num, end=" ")  # 5 4 3 2 1
```

🟡 مستوى متوسط

تمرين 3: اكتب decorator باسم `uppercase_result` يحوّل أي نتيجة نصية من دالة إلى أحرف كبيرة تلقائياً.

تمرين 4: استخدم Closure لكتابة دالة `make_bank_account(initial_balance)` تُرجع دالتين: `deposit(amount)` و`check_balance()`، بحيث يبقى الرصيد محفوظاً بين الاستدعاءات.

✅ الحل

Python

```python
# تمرين 3
from functools import wraps

def uppercase_result(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        result = func(*args, **kwargs)
        return result.upper()
    return wrapper

@uppercase_result
def get_greeting(name: str) -> str:
    return f"hello, {name}"

print(get_greeting("ahmed"))  # HELLO, AHMED

# تمرين 4
def make_bank_account(initial_balance: float):
    balance = initial_balance

    def deposit(amount: float) -> None:
        nonlocal balance
        balance += amount
        print(f"✅ تم إيداع {amount}. الرصيد الآن: {balance}")

    def check_balance() -> float:
        return balance

    return deposit, check_balance

deposit, check_balance = make_bank_account(1000)
deposit(500)                    # ✅ تم إيداع 500. الرصيد الآن: 1500
print(check_balance())          # 1500
deposit(200)                    # ✅ تم إيداع 200. الرصيد الآن: 1700
print(check_balance())          # 1700
```

🔴 تحدي

تمرين 5: اكتب generator باسم `batch_generator(items, batch_size)` يأخذ قائمة ويُنتج «دفعات» (قوائم فرعية) بحجم `batch_size` في كل مرة. مثال: `batch_generator([1,2,3,4,5,6,7], 3)` يُنتج `[1,2,3]` ثم `[4,5,6]` ثم `[7]`.

✅ الحل

Python

```python
def batch_generator(items: list, batch_size: int):
    """يقسّم قائمة إلى دفعات، مفيد عند معالجة بيانات ضخمة على أجزاء."""
    batch = []
    for item in items:
        batch.append(item)
        if len(batch) == batch_size:
            yield batch
            batch = []
    if batch:  # إرسال أي عناصر متبقية أقل من batch_size
        yield batch

items = [1, 2, 3, 4, 5, 6, 7]
for batch in batch_generator(items, 3):
    print(batch)

# [1, 2, 3]
# [4, 5, 6]
# [7]
```

هذا النمط شائع جداً في معالجة البيانات الضخمة (مثل إرسال طلبات API بدفعات لتجنب تجاوز حدود المعدل) وفي تدريب نماذج الذكاء الاصطناعي (تقسيم البيانات إلى batches).

12.9 اختبر فهمك (Quiz)

❓ السؤال 1: ما الفرق بين return وyield؟

الإجابة: `return` تُنهي تنفيذ الدالة فوراً وتُعيد قيمة واحدة نهائية. `yield` «تُجمّد» حالة الدالة عند تلك النقطة، تُعيد قيمة واحدة، لكنها تتذكر مكانها بالضبط (كل المتغيرات المحلية) لتُكمل التنفيذ من حيث توقفت عند الاستدعاء التالي لـ `next()`. دالة تحتوي `yield` تُصبح تلقائياً "Generator Function".

❓ السؤال 2: لماذا يجب استخدام @wraps(func) داخل كل decorator؟

الإجابة: بدون `@wraps(func)`، تفقد الدالة المُزخرفة هويتها الأصلية — `__name__` يصبح `"wrapper"` بدلاً من اسم الدالة الحقيقي، و`__doc__` يصبح `None` بدلاً من التوثيق الأصلي. هذا يُسبب مشاكل في التصحيح (Debugging) والتوثيق التلقائي، وقد يكسر أدوات تعتمد على معرفة اسم الدالة الحقيقي.

❓ السؤال 3: ما ناتج الكود التالي؟

```python
def make_adder(x):
    def add(y):
        return x + y
    return add

add5 = make_adder(5)
add10 = make_adder(10)
print(add5(3))
print(add10(3))
```

الإجابة: `8` ثم `13`. هذا مثال على Closure: كل استدعاء لـ `make_adder` يُنشئ دالة `add` جديدة «تتذكر» قيمة `x` الخاصة بها بشكل مستقل تماماً عن الاستدعاءات الأخرى.

❓ السؤال 4: لماذا تُعتبر Generators أكثر كفاءة في الذاكرة من القوائم؟

الإجابة: القائمة تُنشئ وتُخزّن **كل** عناصرها في الذاكرة فور إنشائها. الـ Generator يُنتج عنصراً واحداً فقط عند الطلب (`next()`)، ولا يحتفظ بالعناصر السابقة أو اللاحقة في الذاكرة، مما يجعل حجمه في الذاكرة ثابتاً تقريباً بغض النظر عن عدد العناصر المحتملة، حتى لو كانت لا نهائية.

❓ السؤال 5: متى يجب استخدام lambda بدلاً من def؟

الإجابة: عندما تحتاج دالة بسيطة جداً (تعبير واحد) تُستخدم لمرة واحدة فقط، عادة كمعامل لدالة أخرى مثل `sorted()` , `map()` , أو `filter()` . لا يُفضّل إسناد lambda لمتغير باسم دائم؛ في تلك الحالة `def` أوضح وأفضل دائماً.

12.10 ملخص الفصل (Cheat Sheet)

```python
# ============ LAMBDA ============
square = lambda x: x ** 2              # ⚠️ تجنّب تسميتها؛ استخدم def
sorted(data, key=lambda x: x["score"]) # ✅ الاستخدام الصحيح

# ============ CLOSURE ============
def outer(x):
    def inner(y):
        return x + y    # inner "على" x تُغلِق
    return inner

add5 = outer(5)
add5(3)  # 8

# nonlocal لتعديل متغير البيئة الخارجية
def counter():
    count = 0
    def increment():
        nonlocal count
        count += 1
        return count
    return increment

# ============ DECORATOR ============
from functools import wraps

def my_decorator(func):
    @wraps(func)                # ✅ دائماً!
    def wrapper(*args, **kwargs):
        # كود قبل
        result = func(*args, **kwargs)
        # كود بعد
        return result           # ✅ لا تنسَ return!
    return wrapper
```

```python
@my_decorator
def my_function(): ...

# Decorator Factory (مع معاملات)
def repeat(times):
    def decorator(func):
        @wraps(func)
        def wrapper(*args, **kwargs):
            for _ in range(times):
                result = func(*args, **kwargs)
            return result
        return wrapper
    return decorator

@repeat(times=3)
def greet(): ...

# ============ GENERATOR ============
def my_generator():
    yield 1
    yield 2
    yield 3

for value in my_generator():
    print(value)

# Generator Expression
gen = (x ** 2 for x in range(1000000))  # lazy - كفاءة ذاكرة عالية

# itertools
from itertools import islice, count, cycle
```

المفهوم	الكلمة المفتاحية	الاستخدام الأمثل
Lambda	`lambda`	معامل سريع لـ sorted/map/filter
Closure	`nonlocal`	دوال «مُخصّصة» تحمل حالة بسيطة
Decorator	`@` , `functools.wraps`	Logging, Timing, Caching, Auth
Generator	`yield`	بيانات ضخمة، تدفقات، كفاءة ذاكرة

12.11 ماذا بعد؟

لقد انتهيت الآن من **الجزء الثاني بالكامل تقريباً**: البرمجة الوظيفية المتقدمة. تمتلك الآن أدوات يستخدمها كبار مبرمجي Python يومياً، وستراها منتشرة في كل مكتبة احترافية تتعامل معها لاحقاً.

لكن حتى الآن، كل برامجنا نظّمت البيانات والسلوك **بشكل منفصل**: بيانات في قواميس وقوائم، وسلوك في دوال مستقلة. ماذا لو أردنا تجميع البيانات **والسلوك المرتبط بها** في «شيء واحد» متكامل؟ ماذا لو أردنا إنشاء «قالب» نصنع منه كائنات متعددة، كل واحد له بياناته الخاصة لكن بنفس السلوك؟

هذا هو جوهر **البرمجة الكائنية (Object-Oriented Programming - OOP)** — النموذج الذي بُنيت عليه أغلب أنظمة البرمجيات الكبرى في العالم، من تطبيقات الويب إلى الألعاب إلى أنظمة التشغيل.

في **الفصل الثالث عشر: البرمجة الكائنية - الأساس**، ستتعلم `class`، `object`، `__init__`، و`self` — بداية رحلة ستغيّر جذرياً طريقة تفكيرك في بناء البرامج.

فصل 13 / 26 · الجزء الثالث

البرمجة الكائنية - الأساس (Class, Object, self, __init__)

9 دقيقة قراءة | 12 قسمًا | 34 مثالًا | 1,899 كلمة

في هذا الفصل

13.1 لماذا تُعتبر البرمجة الكائنية نقلة نوعية في التفكير؟
13.2 المفاهيم الأساسية: Class و Object
13.3 بناء أول Class: التشريح الكامل
13.4 Instance Attributes مقابل Class Attributes
13.5 Methods بأنواعها الثلاثة
13.6 Magic Methods (Dunder Methods): تخصيص سلوك الكائن
13.7 خصوصية البيانات: Public, Protected, Private (بأسلوب Python)
13.8 مشروع تطبيقي: نظام إدارة مكتبة شخصية
13.9 تمارين
13.10 اختبر فهمك (Quiz)
13.11 ملخص الفصل (Cheat Sheet)
13.12 ماذا بعد؟

13.1 لماذا تُعتبر البرمجة الكائنية نقلة نوعية في التفكير؟

تخيّل أنك تبني نظاماً لإدارة طلاب مدرسة. بالطريقة التي تعلمناها حتى الآن، قد تكتب:

```python
student1_name = "أحمد"
student1_age = 20
student1_grades = [85, 90, 78]

student2_name = "سارة"
student2_age = 22
student2_grades = [92, 88, 95]
```

بمجرد وصولك لثلاثة طلاب، ستحتاج تسعة متغيرات منفصلة! وإذا أردت دالة تحسب متوسط درجات طالب، ستحتاج تمرير ثلاثة معاملات منفصلة لكل طالب في كل مرة. الكود يصبح فوضوياً وصعب الصيانة بسرعة.

الحل الأنيق: اجعل Python «تفهم» مفهوم «طالب» ككيان واحد متكامل، يحمل بياناته الخاصة (الاسم، العمر، الدرجات) **وسلوكه** الخاص (حساب المتوسط، إضافة درجة) في **حزمة واحدة**.

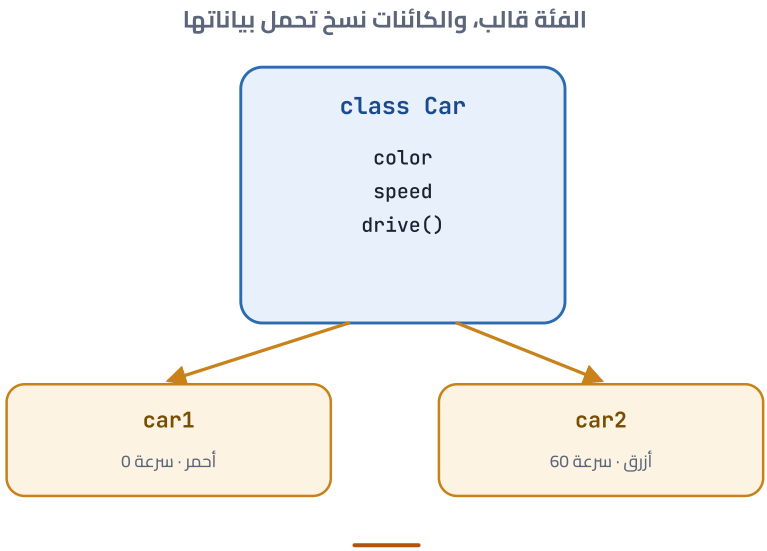
```python
class Student:
    def __init__(self, name: str, age: int):
        self.name = name
        self.age = age
        self.grades: list[float] = []

    def add_grade(self, grade: float) -> None:
        self.grades.append(grade)

    def average_grade(self) -> float:
        if not self.grades:
            return 0.0
        return sum(self.grades) / len(self.grades)

student1 = Student("20 ,"أحمد)
student1.add_grade(85)
student1.add_grade(90)
print(student1.average_grade())  # 87.5
```

الآن `student1` هو كائن واحد متكامل يحمل بياناته وسلوكه معاً. يمكنك إنشاء مئات الطلاب بنفس «القالب» (Class)، وكل واحد منهم مستقل تماماً عن الآخر.

هذا النموذج — **Object-Oriented Programming (OOP)** — هو الأساس الذي بُنيت عليه تقريباً كل أنظمة البرمجيات الكبرى: من تطبيقات الويب، إلى الألعاب، إلى أنظمة إدارة قواعد البيانات. حتى في Python نفسها: كل شيء تقريباً — النصوص، القوائم، القواميس — هو كائن (Object) من فئة (Class) معينة!

بنهاية هذا الفصل، ستفهم بعمق كيف تُصمم «قوالب» (Classes) لإنشاء كائنات (Objects) حقيقية في برامجك.

13.2 المفاهيم الأساسية: Class و Object



شكل 13.1 — الفئة قالب، والكائنات نسخ تحمل بياناتها

الفرق بين Class و Object

- **Class (الفئة/الصنف)**: قالب أو «مخطط هندسي» يصف شكل وسلوك شيء معين، لكنه ليس الشيء نفسه.
- **Object (الكائن)**: نسخة فعلية (Instance) تم إنشاؤها من القالب، ولها بيانات حقيقية خاصة بها.

تشبيه عملي: الـ Class أشبه بـ«مخطط بناء منزل» يحدد: عدد الغرف، شكل النوافذ، التصميم العام. الـ Object هو «المنزل الفعلي» المبني من هذا المخطط — يمكنك بناء عشرات المنازل من نفس المخطط، وكل منزل (Object) مستقل، يمكن طلاؤه بلون مختلف وتأثيثه بأثاث مختلف، رغم أنه بُني من نفس التصميم.

Python

```
class Dog:
    """قالب (Class) يصف أي كلب."""
    def __init__(self, name: str, breed: str):
        self.name = name
        self.breed = breed

    def bark(self) -> str:
        return f"{self.name} يقول: واق واق!"

# إنشاء كائنات (Objects / Instances) من نفس القالب
dog1 = Dog("ريكس", "جيرمن شيبرد")
dog2 = Dog("لونا", "بودل")

print(dog1.bark())  # ريكس يقول: واق واق!
print(dog2.bark())  # لونا يقول: واق واق!
print(dog1.name)    # ريكس
print(dog2.breed)   # بودل
```

نصيحة الخبراء: اصطلاح تسمية الفئات في Python هو **PascalCase** (كل كلمة تبدأ بحرف كبير، بدون شرطات سفلية)، بخلاف المتغيرات والدوال التي تستخدم `snake_case`. مثال: `class StudentRecord:` وليس `class student_record:`.

كل شيء في Python هو كائن!

Python

```
x = 5
print(type(x))        # <class 'int'>

name = "Ahmed"
print(type(name))     # <class 'str'>

my_list = [1, 2, 3]
print(type(my_list))  # <class 'list'>
```

هذا يكشف حقيقة جوهرية: `int`، `str`، `list` نفسها هي **Classes** مبنية داخل Python! عندما تكتب `x = 5`، أنت فعلياً تُنشئ **Object** من فئة `int`. وعندما تكتب `my_list.append(4)`، أنت تستدعي **method** (دالة مرتبطة بالكائن) مُعرّفة داخل فئة `list`.

13.3 بناء أول Class: التشريح الكامل

الصيغة الأساسية

```python
class ClassName:
    def __init__(self, parameter1, parameter2):
        self.attribute1 = parameter1
        self.attribute2 = parameter2

    def method_name(self):
        # كود يستخدم self.attribute1 وغيرها
        pass
```

لنبني مثالاً كاملاً ونُشرّح كل جزء:

```python
class Car:
    """تمثل سيارة بخصائصها الأساسية."""

    def __init__(self, brand: str, model: str, year: int):
        self.brand = brand
        self.model = model
        self.year = year
        self.mileage: int = 0  # قيمة ابتدائية افتراضية

    def drive(self, distance: int) -> None:
        """تُحرّك السيارة مسافة معينة، وتُحدّث العداد."""
        self.mileage += distance
        print(f"🚗 قطعت {self.brand} {distance} كم. المسافة الكلية: {self.mileage} كم")

    def display_info(self) -> str:
        return f"{self.year} {self.brand} {self.model} - {self.mileage} كم"

my_car = Car("2023", "كورولا", "تويوتا")
my_car.drive(150)
my_car.drive(80)
print(my_car.display_info())
```

الناتج:

```
🚗 قطعت تويوتا 150 كم. المسافة الكلية: 150 كم
🚗 قطعت تويوتا 80 كم. المسافة الكلية: 230 كم
2023 تويوتا كورولا - 230 كم
```

تشريح `__init__`: دالة البناء (Constructor)

`__init__` هي دالة خاصة (تُسمى **Magic Method** أو **Dunder Method** — اختصار لـ "Double Underscore") تُستدعى **تلقائياً** عند إنشاء كائن جديد من الفئة.

```python
my_car = Car("2023 ,"كورولا" ,"تويوتا")
# هذا يُستدعى داخلياً:
# Car.__init__(my_car, "2023 ,"كورولا" ,"تويوتا")
```

> **نصيحة الخبراء:** `__init__` ليست «دالة بناء» بالمعنى الحرفي الصارم (في لغات أخرى، الـ Constructor الحقيقي هو `__new__` الذي ينشئ الكائن فعلياً في الذاكرة)؛ `__init__` تُهيّئ الكائن **بعد** إنشائه. لكن عملياً وتعليمياً، التعامل معها كـ«دالة البناء» كافٍ تماماً في 99% من الحالات، ولن نحتاج `__new__` إلا في حالات متقدمة جداً خارج نطاق هذا الكتاب.

فهم `self` بعمق

`self` هو المعامل الأول في كل method داخل الفئة، ويُمثّل **الكائن نفسه** الذي استُدعيت منه الدالة.

```python
class Counter:
    def __init__(self):
        self.count = 0

    def increment(self):
        self.count += 1

c1 = Counter()
c2 = Counter()

c1.increment()
c1.increment()
c2.increment()

print(c1.count)  # 2
print(c2.count)  # 1  -- مستقل تماماً عن c1!
```

ماذا يحدث فعلياً عند `c1.increment()`؟ تُترجمها Python داخلياً إلى:

```python
Counter.increment(c1)
```

أي أن `self` داخل `increment` تُصبح `c1` بالضبط. هذا هو السبب في أن `self.count += 1` تُعدّل بيانات `c1` فقط، وليس `c2` .

⚠ **تحذير شائع جداً عند المبتدئين:** نسيان `self` كمعامل أول في تعريف method:

```python
class Counter:
    def increment():  # ❌ نسيان self!
        pass

c = Counter()
c.increment()
# 💥 TypeError: increment() takes 0 positional arguments but 1 was given
```

السبب: Python تُمرر الكائن تلقائياً كأول معامل في أي استدعاء method، بغض النظر عن كتابتك له أم لا. إذا لم تستقبله بمعامل `self` ، سيحدث تعارض في عدد المعاملات.

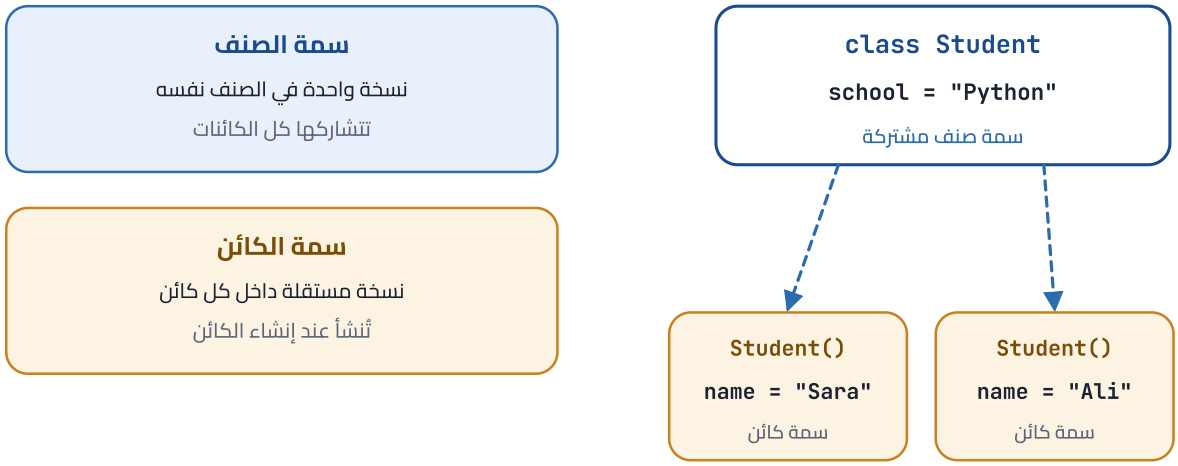
💡 **نصيحة الخبراء:** اسم `self` هو **اصطلاح (Convention)** وليس كلمة مفتاحية حقيقية في Python — يمكنك تسميته أي اسم آخر تقنياً، لكن **لا تفعل ذلك أبداً**. كل مبرمج Python في العالم يتوقع رؤية `self` ، وتغييره يجعل كودك غريباً ومربكاً لأي شخص آخر يقرأه.

Attributes مقابل Methods

المصطلح	الوصف	مثال
Attribute (خاصية)	بيانات مُخزّنة داخل الكائن	`self.age` , `self.name`
Method (طريقة/تابع)	دالة مُعرّفة داخل الفئة، تعمل على بيانات الكائن	`def drive(self):`

```python
class Rectangle:
    def __init__(self, width: float, height: float):
        self.width = width    # attribute
        self.height = height  # attribute

    def area(self) -> float:  # method
        return self.width * self.height

    def perimeter(self) -> float:  # method
        return 2 * (self.width + self.height)

rect = Rectangle(5, 3)
print(rect.width)        # 5 - الوصول لـ attribute (بدون أقواس)
print(rect.area())       # 15.0 - استدعاء method (مع أقواس)
```

13.4 Instance Attributes مقابل Class Attributes



شكل 13.2 — سمة الصنف مشتركة وسمة الكائن مستقلة

Instance Attributes (خصائص الكائن)

هذه ما رأيناه حتى الآن: خصائص خاصة بكل كائن (Instance) على حدة، تُعرَّف عادة داخل `__init__` باستخدام `self.`.

Class Attributes (خصائص الفئة)

خصائص **مشتركة بين كل الكائنات** المُنشأة من نفس الفئة، تُعرَّف مباشرة داخل الفئة (خارج أي method).

```python
class Dog:
    species = "Canis familiaris"  # Class Attribute - مشتركة بين كل الكلاب

    def __init__(self, name: str):
        self.name = name  # Instance Attribute - خاصة بكل كلب

    def __str__(self) -> str:
        return f"{self.name} ({self.species})"

dog1 = Dog("ريكس")
dog2 = Dog("لونا")

print(dog1.species)  # Canis familiaris
print(dog2.species)  # Canis familiaris -- نفس القيمة للجميع

# الوصول من الفئة مباشرة أيضاً ممكن
print(Dog.species)   # Canis familiaris
```

⚠ **تحذير حرج - فخ تعديل Class Attribute من خلال Instance:**

```python
dog1.species = "Wolf"  # ⚠ هذا لا يُعدّل Class Attribute!
                       # بل يُنشئ Instance Attribute جديد خاص بـ dog1 فقط!

print(dog1.species)  # Wolf (الآن Instance Attribute خاص به)
print(dog2.species)  # Canis familiaris (لم يتأثر إطلاقاً!)
print(Dog.species)   # Canis familiaris (الفئة نفسها لم تتغير)
```

لتعديل Class Attribute فعلياً بحيث يؤثر على كل الكائنات، يجب تعديلها عبر اسم الفئة مباشرة: `Dog.species = "Wolf"`.

💡 **نصيحة الخبراء - متى تستخدم Class Attributes؟** عندما تكون القيمة **ثابتة ومشتركة منطقياً** بين كل الكائنات، مثل: عدد الكائنات المُنشأة، قيمة ثابتة فيزيائية، إعدادات افتراضية عامة.

```python
class Employee:
    company_name = "شركة التقنية المتقدمة"  # مشتركة للجميع
    total_employees = 0  # عداد مشترك

    def __init__(self, name: str, salary: float):
        self.name = name
        self.salary = salary
        Employee.total_employees += 1  # تعديل صحيح عبر اسم الفئة

emp1 = Employee("أحمد", 8000)
emp2 = Employee("سارة", 9500)
emp3 = Employee("عمر", 7500)

print(f"عدد الموظفين: {Employee.total_employees}")  # 3
print(f"الشركة: {emp1.company_name}")  # شركة التقنية المتقدمة
```

مقارنة: Class Attributes تشبه «المتغيرات الساكنة» (Static Variables) في لغات مثل Java وC++ — بيانات تنتمي للفئة نفسها وليس لنسخة معينة منها.

Methods بأنواعها الثلاثة 13.5

شكل 13.3 — الأنواع الثلاثة للدوال

Instance Methods (الأكثر شيوعاً)

كل ما رأيناه حتى الآن — دوال تأخذ `self` وتعمل على بيانات كائن معين.

```python
class BankAccount:
    def __init__(self, owner: str, balance: float = 0):
        self.owner = owner
        self.balance = balance

    def deposit(self, amount: float) -> None:  # instance method
        self.balance += amount
```

Class Methods: تعمل على الفئة نفسها

```python
class Employee:
    company_name = "شركة التقنية"
    total_employees = 0

    def __init__(self, name: str, salary: float):
        self.name = name
        self.salary = salary
        Employee.total_employees += 1

    @classmethod
    def get_total_employees(cls) -> int:
        """تستقبل cls (الفئة نفسها) بدلاً من self (كائن محدد)."""
        return cls.total_employees

    @classmethod
    def from_string(cls, data_string: str) -> "Employee":
        """Alternative Constructor - نمط شائع جداً لإنشاء كائنات من صيغ بيانات مختلفة."""
        name, salary = data_string.split(",")
        return cls(name.strip(), float(salary.strip()))

emp1 = Employee("أحمد ,8000")
emp2 = Employee.from_string("سارة ,9500")  # إنشاء من نص!

print(emp2.name, emp2.salary)              # سارة 9500.0
print(Employee.get_total_employees())      # 2
```

نصيحة الخبراء - نمط Alternative Constructor: استخدام `@classmethod` لإنشاء «طرق بديلة» لإنشاء كائنات (مثل `from_string` , `from_dict` , `from_json`) هو نمط احترافي شائع جداً في مكتبات Python الكبرى. بدلاً من كتابة منطق تحليل البيانات خارج الفئة، تجعله جزءاً من الفئة نفسها بوضوح.

Static Methods: لا تحتاج self ولا cls

Python

```python
class MathHelper:
    @staticmethod
    def is_prime(n: int) -> bool:
        """دالة مساعدة لا تحتاج الوصول لأي بيانات خاصة بالفئة أو الكائن."""
        if n < 2:
            return False
        for i in range(2, int(n ** 0.5) + 1):
            if n % i == 0:
                return False
        return True

# تُستدعى عبر الفئة مباشرة، بدون إنشاء كائن!
print(MathHelper.is_prime(17))  # True
print(MathHelper.is_prime(18))  # False
```

نصيحة الخبراء - متى نستخدم static method؟ عندما تكون الدالة منطقياً **مرتبطة** بموضوع الفئة، لكنها **لا تحتاج** الوصول لـ `self` (بيانات كائن محدد) ولا `cls` (بيانات الفئة نفسها). إنها ببساطة دالة عادية «مُنظَّمة» داخل الفئة لأسباب تنظيمية ودلالية.

جدول مقارنة الأنواع الثلاثة

النوع	Decorator	المعامل الأول	متى تستخدمه
Instance Method	(لا يوجد)	`self`	السلوك الطبيعي الذي يعمل على بيانات كائن محدد
Class Method	`@classmethod`	`cls`	التعامل مع بيانات الفئة، أو Alternative Constructors
Static Method	`@staticmethod`	(لا شيء)	دالة مساعدة منطقياً مرتبطة، لا تحتاج self/cls

Python

```python
class Pizza:
    def __init__(self, toppings: list[str]):
        self.toppings = toppings

    def instance_method(self):
        return f"هذه بيتزا بإضافات: {self.toppings}"

    @classmethod
    def margherita(cls) -> "Pizza":
        return cls(["جبنة" ,"طماطم"])  # Alternative constructor

    @staticmethod
    def calculate_cooking_time(size_cm: int) -> int:
        return size_cm // 2  # منطق عام لا يحتاج بيانات بيتزا معينة

p1 = Pizza.margherita()
print(p1.toppings)  # ['جبنة' ,'طماطم']
print(Pizza.calculate_cooking_time(30))  # 15
```

13.6 Magic Methods (Dunder Methods) : تخصيص سلوك الكائن

ما تكتبه		ما يُستدعى تلقائيًا
`len(obj)`	←	`__len__`
`obj + other`	←	`__add__`
`print(obj)`	←	`__str__`
`obj == other`	←	`__eq__`
`obj[0]`	←	`__getitem__`

لا تُنادى هذه الدوال بالاسم؛ Python تناديها نيابةً عنك عند استخدام الصيغة.

شكل 13.4 — الصيغة التي تكتبها والدالة التي تُستدعى

`__str__` و `__repr__`: كيف يُطبع الكائن؟

```python
class Point:
    def __init__(self, x: float, y: float):
        self.x = x
        self.y = y

p = Point(3, 4)
print(p)  # <__main__.Point object at 0x7f8a1c0b3d90>  -- غير مفيد!
```

بدون تخصيص، طباعة الكائن تُظهر عنوان الذاكرة فقط — غير مفيد إطلاقاً. نُصلح هذا بـ `__str__`:

```python
class Point:
    def __init__(self, x: float, y: float):
        self.x = x
        self.y = y

    def __str__(self) -> str:
        """يُستدعى عند print() أو str() - للعرض البشري المقروء."""
        return f"نقطة({self.x}, {self.y})"

    def __repr__(self) -> str:
        """يُستدعى في REPL - للعرض التقني الدقيق أو داخل قوائم."""
        return f"Point(x={self.x}, y={self.y})"

p = Point(3, 4)
print(p)       # يستخدم __str__ -- نقطة(3, 4)
print([p])     # [Point(x=3, y=4)]  -- القوائم تستخدم __repr__ للعناصر
print(repr(p)) # Point(x=3, y=4)
```

نصيحة الخبراء - القاعدة الذهبية: `__str__` للمستخدم النهائي (واضح ومقروء)، `__repr__` للمطور (دقيق وغالباً قابل لإعادة إنشاء الكائن منه). **الأفضل دائماً:** عرّف `__repr__` على الأقل؛ إذا لم تُعرّف `__str__`، Python تستخدم `__repr__` كبديل تلقائي.

`__eq__` : المقارنة بين الكائنات

```python
class Point:
    def __init__(self, x: float, y: float):
        self.x = x
        self.y = y

    def __eq__(self, other) -> bool:
        if not isinstance(other, Point):
            return False
        return self.x == other.x and self.y == other.y

p1 = Point(3, 4)
p2 = Point(3, 4)
p3 = Point(5, 6)

print(p1 == p2)  # True  -- بدون __eq__ كانت ستكون False (مقارنة مراجع الذاكرة)!
print(p1 == p3)  # False
```

⚠ **تحذير شائع:** بدون تخصيص `__eq__`، عامل `==` يقارن **عناوين الذاكرة** (هل هما نفس الكائن فعلياً؟) وليس محتوى البيانات. هذا سبب شائع لحيرة المبتدئين: "لماذا كائنان متطابقان في البيانات لكن `==` تُعطي False؟"

`__len__` : دعم دالة len()

```python
class Playlist:
    def __init__(self, songs: list[str]):
        self.songs = songs

    def __len__(self) -> int:
        return len(self.songs)

playlist = Playlist(["أغنية 1", "أغنية 2", "أغنية 3"])
print(len(playlist))  # 3  -- بدون __len__ ستحصل على TypeError
```

`__add__` وتحميل العوامل (Operator Overloading)

```python
class Vector:
    def __init__(self, x: float, y: float):
        self.x = x
        self.y = y

    def __add__(self, other: "Vector") -> "Vector":
        return Vector(self.x + other.x, self.y + other.y)

    def __repr__(self) -> str:
        return f"Vector({self.x}, {self.y})"

v1 = Vector(1, 2)
v2 = Vector(3, 4)
v3 = v1 + v2  # يستدعي v1.__add__(v2) تلقائياً!
print(v3)  # Vector(4, 6)
```

جدول مرجعي لأهم Magic Methods

Magic Method	يُستدعى عند	مثال
`__init__`	إنشاء كائن جديد	`Point(3, 4)`
`__str__`	`print()` ، `str()`	`print(p)`
`__repr__`	عرض تقني، REPL	`repr(p)` ، `[p]`
`__eq__`	`==`	`p1 == p2`
`__lt__`	`<`	`p1 < p2`
`__len__`	`len()`	`len(playlist)`
`__add__`	`+`	`v1 + v2`
`__getitem__`	`obj[index]`	`playlist[0]`
`__contains__`	`in`	`"x" in obj`
`__call__`	استدعاء الكائن كدالة	`obj()`

مقارنة مع لغات أخرى: هذا المفهوم يُسمى **Operator Overloading**، وهو موجود في C++ وJava (بصيغ مختلفة) وغيرها. في Python، آلية التنفيذ أنظف وأوضح بكثير بفضل الـ Dunder Methods الموحدة.

13.7 خصوصية البيانات: Public, Protected, Private (بأسلوب Python)

فلسفة "Python: "We're all consenting adults here

على عكس لغات مثل Java التي تفرض `private` / `protected` / `public` بشكل صارم على مستوى اللغة، Python تعتمد على **اصطلاحات التسمية** (Naming Conventions) بدلاً من قيود حقيقية.

```python
class BankAccount:
    def __init__(self, owner: str, balance: float):
        self.owner = owner           # Public - وصول حر من أي مكان
        self._balance = balance      # Protected (اصطلاحاً) - "استخدم بحذر"
        self.__pin = "1234"          # Private (اصطلاحاً) - "لا تلمس من خارج الفئة"
```

النمط	الاسم	المعنى
`self.name`	Public	وصول حر تماماً، جزء من الواجهة العامة للفئة
`self._name`	Protected (اصطلاحي)	«علامة تحذير» للمطورين: هذا تفصيل داخلي، لا تعتمد عليه من خارج الفئة
`self.__name`	Private (شبه حقيقي)	يخضع لـ Name Mangling (انظر أدناه)

Single Underscore: `_balance`

```python
class BankAccount:
    def __init__(self, balance: float):
        self._balance = balance  # "تلميح" أنه داخلي، لكن لا شيء يمنع الوصول فعلياً

account = BankAccount(1000)
print(account._balance)  # 1000 -- يعمل! Python لا تمنعك، لكنها تُعتبر ممارسة سيئة
```

نصيحة الخبراء: `_balance` بشرطة سفلية واحدة هي **اتفاق بين المبرمجين**، وليست قيداً تقنياً حقيقياً. معناها الضمني: «هذا تفصيل تنفيذي داخلي، قد يتغير في أي وقت، فلا تعتمد عليه مباشرة من خارج الفئة — استخدم methods عامة بدلاً من ذلك.»

Name Mangling و `__pin` :Double Underscore

```python
class BankAccount:
    def __init__(self, pin: str):
        self.__pin = pin

    def verify_pin(self, attempt: str) -> bool:
        return self.__pin == attempt

account = BankAccount("1234")
print(account.verify_pin("1234"))  # True

# print(account.__pin)
# 💥 AttributeError: 'BankAccount' object has no attribute '__pin'
```

ماذا حدث؟ Python تُطبّق **Name Mangling**: تُحوّل `__pin` داخلياً إلى `_BankAccount__pin`. يمكنك تقنياً الوصول إليه بهذا الاسم المُعدَّل، لكن هذا يُعتبر اختراقاً متعمداً لا يجب فعله أبداً:

```python
print(account._BankAccount__pin)  # 1234  -- يعمل تقنياً، لكن لا تفعل هذا أبداً!
```

> 💡 **الهدف الحقيقي من Name Mangling**: ليس «الحماية الأمنية» (فهو قابل للتجاوز كما رأينا)، بل **منع تعارض الأسماء** عند الوراثة المتعددة (سنتعمق في الوراثة في الفصل 14).

Getters و Setters بأسلوب Python: `@property`

في لغات مثل Java، من الشائع كتابة `getBalance()` و`setBalance()` صراحة. في Python، نستخدم `@property` لإعطاء نفس الحماية لكن بواجهة أنظف تبدو كخاصية عادية:

```python
class BankAccount:
    def __init__(self, balance: float):
        self._balance = balance

    @property
    def balance(self) -> float:
        """Getter - تُستدعى عند قراءة account.balance"""
        return self._balance

    @balance.setter
    def balance(self, value: float) -> None:
        """Setter - تُستدعى عند account.balance = value"""
        if value < 0:
            raise ValueError("الرصيد لا يمكن أن يكون سالباً!")
        self._balance = value

account = BankAccount(1000)
print(account.balance)   # 1000 -- يبدو كخاصية عادية، لكنه method بالخلف!

account.balance = 1500   # تلقائياً setter يستدعي الـ
print(account.balance)   # 1500

# account.balance = -500
# 💥 ValueError: الرصيد لا يمكن أن يكون سالباً!
```

نصيحة الخبراء - لماذا هذا أفضل من getter/setter التقليدية؟ يمكنك البدء بخاصية عادية `public` (`self.balance`)، ثم إذا احتجت لاحقاً إضافة منطق تحقق (Validation)، تُحوّلها لـ `@property` **دون تغيير** الكود الذي يستخدمها في أي مكان آخر (`account.balance` يبقى يعمل بنفس الصيغة). هذا مثال رائع على مبدأ **«الواجهة المستقرة، التنفيذ الداخلي مرن»**.

سنتعمق أكثر في `@property` والـ Dataclasses في **الفصل الرابع عشر**.

13.8 مشروع تطبيقي: نظام إدارة مكتبة شخصية

```python
class Book:
    """يمثل كتاباً واحداً في المكتبة."""

    total_books = 0  # Class Attribute - عداد لكل الكتب المُنشأة
```

```
    def __init__(self, title: str, author: str, pages: int, is_read: bool = False):
        self.title = title
        self.author = author
        self.pages = pages
        self._is_read = is_read
        Book.total_books += 1

    @property
    def is_read(self) -> bool:
        return self._is_read

    def mark_as_read(self) -> None:
        self._is_read = True
        print(f"✅ '{self.title}' عليه 'مقروء' علامة وضع تم.")

    def __str__(self) -> str:
        status = "📖 مقروء" if self._is_read else "📕 غير مقروء"
        return f"{self.title} - {self.author} ({self.pages} صفحة) [{status}]"

    def __repr__(self) -> str:
        return f"Book(title='{self.title}', author='{self.author}')"

    def __eq__(self, other) -> bool:
        if not isinstance(other, Book):
            return False
        return self.title == other.title and self.author == other.author

    @classmethod
    def from_string(cls, data: str) -> "Book":
        """ينشئ كتاباً من نص بصيغة: 'العنوان|المؤلف|الصفحات'"""
        title, author, pages = data.split("|")
        return cls(title.strip(), author.strip(), int(pages.strip()))

    @staticmethod
    def estimate_reading_hours(pages: int, pages_per_hour: int = 30) -> float:
        """تقدير عدد الساعات اللازمة لقراءة كتاب بعدد صفحات معين."""
        return round(pages / pages_per_hour, 1)

class Library:
    """تمثل مكتبة تحتوي على مجموعة من الكتب."""

    def __init__(self, name: str):
        self.name = name
        self.books: list[Book] = []

    def add_book(self, book: Book) -> None:
        if book in self.books:
            print(f"⚠️ '{book.title}' موجود مسبقاً في المكتبة.")
            return
        self.books.append(book)
        print(f"✅ تمت إضافة '{book.title}' إلى مكتبة '{self.name}'.")
```

```
    def remove_book(self, title: str) -> None:
        for book in self.books:
            if book.title == title:
                self.books.remove(book)
                print(f"🗑️ تم حذف '{title}'.")
                return
        print(f"❌ لم يُعثر على '{title}'.")

    def get_unread_books(self) -> list[Book]:
        return [book for book in self.books if not book.is_read]

    def total_pages(self) -> int:
        return sum(book.pages for book in self.books)

    def find_by_author(self, author: str) -> list[Book]:
        return [book for book in self.books if book.author == author]

    def __len__(self) -> int:
        return len(self.books)

    def __str__(self) -> str:
        return f"مكتبة '{self.name}' - {len(self.books)} كتاب"

    def print_all(self) -> None:
        print(f"\n📚 {self}")
        print("-" * 50)
        for i, book in enumerate(self.books, start=1):
            print(f"{i}. {book}")
        print("-" * 50)

# --- التشغيل التجريبي ---
if __name__ == "__main__":
    my_library = Library("مكتبة أحمد")

    book1 = Book("250 ,"خالد عبدالله" ,"نادي السيارات")
    book2 = Book.from_string("180 | باولو كويلو | الخيميائي")
    book3 = Book("Clean Code", "Robert Martin", 400, is_read=True)

    my_library.add_book(book1)
    my_library.add_book(book2)
    my_library.add_book(book3)
    my_library.add_book(book1)  # محاولة إضافة مكررة

    my_library.print_all()

    book1.mark_as_read()

    print(f"\n📖 عدد الكتب غير المقروءة: {len(my_library.get_unread_books())}")
    print(f"📄 إجمالي الصفحات: {my_library.total_pages()}")
    print(f"📊 إجمالي الكتب في كل المكتبات: {Book.total_books}")
```

```
    hours = Book.estimate_reading_hours(book2.pages)
    print(f"⏱️ وقت قراءة متوقع لـ '{book2.title}': {hours} ساعة")

    print(f"\n{my_library}")  # يستخدم __str__
    print(f"عدد الكتب (len): {len(my_library)}")  # يستخدم __len__
```

الناتج المتوقع:

```
✅ تمت إضافة 'نادي السيارات' إلى مكتبة أحمد.
✅ تمت إضافة 'الخيميائي' إلى مكتبة أحمد.
✅ تمت إضافة 'Clean Code' إلى مكتبة أحمد.
⚠️ 'نادي السيارات' موجود مسبقاً في المكتبة.

📚 مكتبة 'أحمد' - 3 كتاب
--------------------------------------------------
1. نادي السيارات - خالد عبدالله (250 صفحة) [📕 غير مقروء]
2. الخيميائي - باولو كويلو (180 صفحة) [📕 غير مقروء]
3. Clean Code - Robert Martin (400 صفحة) [📖 مقروء]
--------------------------------------------------
✅ تم وضع علامة 'مقروء' على 'نادي السيارات'.

📖 عدد الكتب غير المقروءة: 1
📄 إجمالي الصفحات: 830
📊 إجمالي الكتب في كل المكتبات: 3
⏱️ وقت قراءة متوقع لـ 'الخيميائي': 6.0 ساعة

مكتبة 'أحمد' - 3 كتاب
عدد الكتب (len): 3
```

✅ تحدي إضافي: إضافة تصنيف وترتيب حسب الصفحات (الحل)

Python

```
def sort_by_pages(self, reverse: bool = False) -> list[Book]:
    """ترتيب كتب المكتبة حسب عدد الصفحات."""
    return sorted(self.books, key=lambda book: book.pages, reverse=reverse)

# إضافة هذا كـ method داخل Library، ثم:
sorted_books = my_library.sort_by_pages()
for book in sorted_books:
    print(f"{book.title}: {book.pages} صفحة")
```

13.9 تمارين

🟢 مستوى مبتدئ

تمرين 1: أنشئ فئة `Person` مع `__init__` يستقبل `name` و`age`، وmethod باسم `introduce()` تطبع جملة تعارف.

تمرين 2: أضف Class Attribute باسم `species = "Homo sapiens"` لفئة `Person`، واطبعه من كائنين مختلفين.

✅ الحل

Python

```
class Person:
    species = "Homo sapiens"

    def __init__(self, name: str, age: int):
        self.name = name
        self.age = age

    def introduce(self) -> None:
        print(f"مرحباً، اسمي {self.name} وعمري {self.age} سنة.")

p1 = Person("خالد", 30)
p2 = Person("منى", 25)

p1.introduce()  # مرحباً، اسمي خالد وعمري 30 سنة.
p2.introduce()  # مرحباً، اسمي منى وعمري 25 سنة.

print(p1.species)  # Homo sapiens
print(p2.species)  # Homo sapiens
```

🟡 مستوى متوسط

تمرين 3: أنشئ فئة `Rectangle` تحتوي `width`، `height`، وmethods: `area()`، `perimeter()`، و`__str__` تعرض الأبعاد والمساحة. أضف `__eq__` لمقارنة مستطيلين بناءً على المساحة.

تمرين 4: أنشئ فئة `TemperatureConverter` باستخدام `@staticmethod` لدالتين: `celsius_to_fahrenheit(c)` و`fahrenheit_to_celsius(f)`.

✅ الحل

Python

```python
# تمرين 3
class Rectangle:
    def __init__(self, width: float, height: float):
        self.width = width
        self.height = height

    def area(self) -> float:
        return self.width * self.height

    def perimeter(self) -> float:
        return 2 * (self.width + self.height)

    def __str__(self) -> str:
        return f"مستطيل {self.width}x{self.height}, المساحة={self.area()}"

    def __eq__(self, other) -> bool:
        if not isinstance(other, Rectangle):
            return False
        return self.area() == other.area()

r1 = Rectangle(4, 5)
r2 = Rectangle(2, 10)
print(r1)              # مستطيل 4x5, المساحة=20
print(r1 == r2)        # True -- نفس المساحة رغم اختلاف الأبعاد!

# تمرين 4
class TemperatureConverter:
    @staticmethod
    def celsius_to_fahrenheit(c: float) -> float:
        return (c * 9/5) + 32

    @staticmethod
    def fahrenheit_to_celsius(f: float) -> float:
        return (f - 32) * 5/9

print(TemperatureConverter.celsius_to_fahrenheit(100))  # 212.0
print(TemperatureConverter.fahrenheit_to_celsius(32))   # 0.0
```

🔴 تحدي

تمرين 5: أنشئ فئة `ShoppingCart` تحتوي:
- قائمة داخلية من المنتجات (كل منتج قاموس فيه `name` و`price` و`quantity`)
- method لإضافة منتج
- method لحساب الإجمالي
- `@property` باسم `item_count` تُرجع عدد القطع الكلي
- `__str__` تعرض محتويات السلة بشكل منسّق
- `__len__` تُرجع عدد المنتجات المختلفة (وليس الكمية الكلية)

✅ الحل

Python

```
class ShoppingCart:
    def __init__(self):
        self._items: list[dict] = []

    def add_item(self, name: str, price: float, quantity: int = 1) -> None:
        self._items.append({"name": name, "price": price, "quantity": quantity})
        print(f"✅ أُضيف {quantity}x {name} إلى السلة.")

    def total_price(self) -> float:
        return sum(item["price"] * item["quantity"] for item in self._items)

    @property
    def item_count(self) -> int:
        """إجمالي عدد القطع (مجموع الكميات)."""
        return sum(item["quantity"] for item in self._items)

    def __len__(self) -> int:
        """عدد المنتجات المختلفة في السلة."""
        return len(self._items)

    def __str__(self) -> str:
        lines = [f"🛒 سلة التسوق ({len(self)} منتج مختلف):"]
        for item in self._items:
            subtotal = item["price"] * item["quantity"]
            lines.append(f"  - {item['name']}: {item['quantity']} × {item['price']} =
{subtotal}")
        lines.append(f"الإجمالي: {self.total_price()} ر.س")
        return "\n".join(lines)

cart = ShoppingCart()
cart.add_item("لابتوب", 3500, 1)
cart.add_item("ماوس", 50, 2)
cart.add_item("كيبورد", 150, 1)

print(cart)
print(f"\nعدد القطع الكلي: {cart.item_count}")  # 4
print(f"عدد المنتجات المختلفة: {len(cart)}")    # 3
```

13.10 اختبر فهمك (Quiz)

❓ السؤال 1: ما الفرق بين Class و Object؟

الإجابة: Class هو القالب/المخطط الذي يصف الخصائص والسلوك، بينما Object هو نسخة فعلية (Instance) مُنشأة من ذلك القالب، تحمل بيانات حقيقية خاصة بها. يمكن إنشاء عدة Objects مستقلة من نفس Class.

❓ السؤال 2: لماذا يجب أن يكون self أول معامل في كل instance method؟

الإجابة: لأن Python تُمرر الكائن نفسه تلقائياً كأول معامل عند استدعاء أي method عبر `object.method()`. إذا لم تستقبل الدالة هذا المعامل، سيحدث تعارض في عدد المعاملات (TypeError). `self` يُمكّن الـ method من الوصول لبيانات ذلك الكائن المحدد بالذات.

❓ السؤال 3: ما الفرق بين Instance Attribute و Class Attribute؟

الإجابة: Instance Attribute خاص بكل كائن على حدة (يُعرَّف عادة بـ `self.x` داخل `__init__`)، بينما Class Attribute مشترك بين كل الكائنات المُنشأة من نفس الفئة (يُعرَّف مباشرة داخل جسم الفئة، خارج أي method). تعديل Instance Attribute لا يؤثر على الكائنات الأخرى، بينما تعديل Class Attribute (عبر اسم الفئة) يؤثر على الجميع.

❓ السؤال 4: متى تستخدم @staticmethod بدلاً من @classmethod أو instance method عادية؟

الإجابة: تستخدم `@staticmethod` عندما تكون الدالة منطقياً مرتبطة بموضوع الفئة، لكنها لا تحتاج الوصول لا لبيانات كائن محدد (`self`) ولا لبيانات الفئة نفسها (`cls`). إنها دالة مستقلة «مُنظَّمة» داخل الفئة فقط لأسباب دلالية وتنظيمية.

❓ السؤال 5: لماذا نستخدم @property بدلاً من الوصول المباشر لـ attribute؟

الإجابة: `@property` تسمح بإضافة منطق تحقق (Validation) أو حساب عند قراءة/تعديل قيمة، مع الحفاظ على واجهة استخدام بسيطة تبدو كخاصية عادية (`obj.value` بدلاً من `obj.get_value()`). هذا يسمح بالبدء بخاصية بسيطة ثم تطويرها لاحقاً بإضافة تحققات دون تغيير الكود الذي يستخدمها في أي مكان آخر بالمشروع.

13.11 ملخص الفصل (Cheat Sheet)

Python

```python
# تعريف فئة أساسية
class ClassName:
    class_attribute = "مشترك بين الجميع"  # Class Attribute

    def __init__(self, param1, param2):
        self.attr1 = param1    # Instance Attribute
        self.attr2 = param2

    def instance_method(self):          # self = الكائن نفسه
        return self.attr1

    @classmethod
    def class_method(cls):              # cls = الفئة نفسها
        return cls.class_attribute

    @staticmethod
    def static_method():                # لا self ولا cls
        return "دالة مستقلة منطقياً"

# إنشاء كائن
obj = ClassName("value1", "value2")

# Magic Methods الأساسية
class MyClass:
    def __init__(self, x): self.x = x
    def __str__(self): return f"قابل للقراءة: {self.x}"
    def __repr__(self): return f"MyClass({self.x})"
    def __eq__(self, other): return self.x == other.x
    def __len__(self): return self.x
    def __add__(self, other): return MyClass(self.x + other.x)
```

```python
# (اصطلاحية) خصوصية البيانات
self.public_attr      # وصول حر
self._protected_attr  # "تحذير" - داخلي، لا تعتمد عليه من الخارج
self.__private_attr   # Name Mangling → _ClassName__private_attr

# Property - getter/setter أنيق
class Account:
    def __init__(self, balance):
        self._balance = balance

    @property
    def balance(self):
        return self._balance

    @balance.setter
    def balance(self, value):
        if value < 0:
            raise ValueError("لا يمكن أن يكون سالباً")
        self._balance = value
```

المفهوم	القاعدة الذهبية
تسمية الفئات	PascalCase: `class StudentRecord:`
self	يجب أن يكون أول معامل في كل instance method
Class Attribute	للبيانات المشتركة فعلاً بين كل الكائنات
`__str__`	دائماً أضفها للعرض المقروء البشري
`_` و `__`	اصطلاحات فقط، وليست حماية حقيقية صارمة
`@property`	الخيار الافتراضي الاحترافي بدلاً من get/set يدوية

13.12 ماذا بعد؟

لقد بنيت الآن أساساً متيناً في البرمجة الكائنية: تعرف كيف تُصمم فئة، تُنشئ كائنات، تُخصص سلوكها عبر Magic Methods، وتتحكم في خصوصية بياناتها.

لكن هناك سؤال جوهري لم نُجب عليه بعد: ماذا لو أردت إنشاء فئة `Dog` وفئة `Cat`، وكلاهما «حيوان» يشترك في خصائص كثيرة (اسم، عمر، صوت) لكن لكل منهما سلوك خاص؟ هل تُكرر نفس الكود في الفئتين؟ بالطبع لا!

هذا هو موضوع **الوراثة (Inheritance)** — أحد الأعمدة الأربعة للبرمجة الكائنية. في **الفصل الرابع عشر: البرمجة الكائنية - التعمق**، سنغوص في الوراثة، وتعدد الأشكال (Polymorphism)، والـ Dataclasses التي تُبسّط كتابة الفئات بشكل كبير، وسنختم بمشروع متكامل: **نظام حجز مكتبة** يجمع كل مفاهيم الفصلين 13 و14 معاً.

فصل 14 / 26 · الجزء الثالث

14

البرمجة الكائنية

التعمق (الوراثة، تعدد الأشكال، Dataclasses) + مشروع نظام حجز مكتبة

8 دقيقة قراءة | 11 قسمًا | 27 مثالًا | 1,618 كلمة

في هذا الفصل

14.1 لماذا الوراثة أحد أهم اختراعات البرمجة الكائنية؟
14.2 الوراثة (Inheritance) بعمق
14.3 تعدد الأشكال (Polymorphism)
14.4 الفئات المجردة (Abstract Base Classes)
14.5 Dataclasses: اختصار كتابة الفئات
14.6 مزيد من Magic Methods: `__call__`, `__getitem__`, Context Managers
14.7 مشروع تطبيقي متكامل: نظام حجز مكتبة
14.8 تمارين
14.9 اختبر فهمك (Quiz)
14.10 ملخص الفصل (Cheat Sheet)
14.11 ماذا بعد؟

14.1 لماذا الوراثة أحد أهم اختراعات البرمجة الكائنية؟

تخيّل أنك تبني نظاماً لمتجر يبيع منتجات متنوعة: كتب، إلكترونيات، ملابس. كل هذه المنتجات تشترك في خصائص أساسية (اسم، سعر، رقم تعريف)، لكن لكل نوع خصائص وسلوك إضافي خاص به (الكتاب له مؤلف، الإلكترونيات لها ضمان، الملابس لها مقاس).

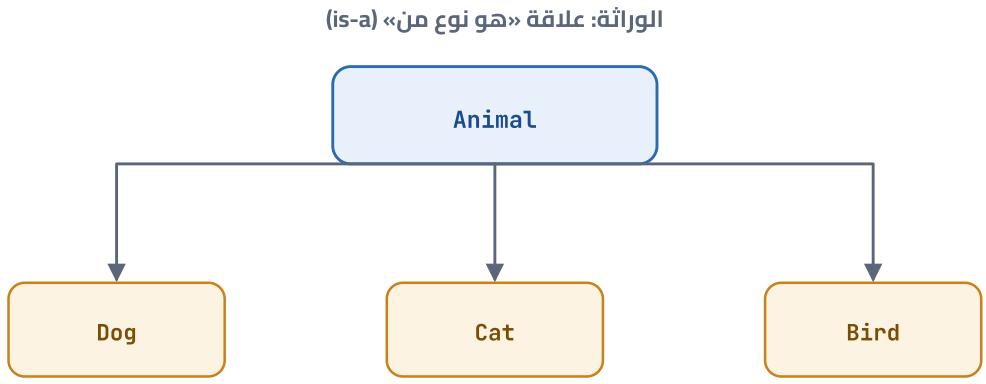
بدون الوراثة، ستكتب ثلاث فئات منفصلة تماماً، كل واحدة تُكرر نفس كود `name` , `price` , `id` . هذا يخالف أحد أهم مبادئ البرمجة: **DRY (Don't Repeat Yourself)**.

الوراثة تحل هذه المشكلة بأناقة: تُعرّف فئة «أساسية» (Parent/Base Class) تحتوي الخصائص المشتركة، ثم تُنشئ فئات «فرعية» (Child/Derived Classes) تَرِث كل شيء من الفئة الأساسية تلقائياً، وتُضيف فقط ما يميزها.

```python
class Product:
    def __init__(self, name: str, price: float):
        self.name = name
        self.price = price

    def display_info(self) -> str:
        return f"{self.name}: {self.price} ر.س"

class Book(Product):  # Book ترث من Product
    def __init__(self, name: str, price: float, author: str):
        super().__init__(name, price)  # استدعاء __init__ من الفئة الأساسية
        self.author = author

    def display_info(self) -> str:
        return f"{super().display_info()} - تأليف: {self.author}"

book = Book("بايثون للمحترفين", 120, "محمد أحمد")
print(book.display_info())
# بايثون للمحترفين: 120 ر.س - تأليف: محمد أحمد
```

في هذا الفصل، ستتعمق في الوراثة، وتعدد الأشكال (Polymorphism)، والفئات المجردة (Abstract Classes)، وأداة ثورية تُسمى **Dataclasses** تختصر عليك كتابة أكواد متكررة، ثم نختم بمشروع متكامل يجمع كل ما تعلمته في الفصلين الأخيرين.

14.2 الوراثة (Inheritance) بعمق



شكل 14.1 — شجرة الوراثة وترتيب MRO

الصيغة الأساسية

```python
class Animal:  # Parent / Base / Super Class
    def __init__(self, name: str):
        self.name = name

    def make_sound(self) -> str:
        return "صوت ما.  "

    def __str__(self) -> str:
        return f"{self.name} يصدر صوت: {self.make_sound()}"

class Dog(Animal):  # Child / Derived / Sub Class
    def make_sound(self) -> str:  # Override - استبدال السلوك
        return "واق واق!"

class Cat(Animal):
    def make_sound(self) -> str:
        return "ميـاو!"

dog = Dog("ريـكس")
cat = Cat("لـونـا")

print(dog)  # ريكس يصدر صوت: واق واق!
print(cat)  # لونا يصدر صوت: مياو!
```

لاحظ أن `Dog` و`Cat` لم تُعرّفا `__init__` و`__str__` من جديد — ورثتاها كاملة من `Animal`، وغيّرتا فقط `make_sound()`.

دالة `super()` : الوصول للفئة الأساسية

```python
class Employee:
    def __init__(self, name: str, salary: float):
        self.name = name
        self.salary = salary

    def display(self) -> str:
        return f"{self.name}: {self.salary} ر.س"

class Manager(Employee):
    def __init__(self, name: str, salary: float, team_size: int):
        super().__init__(name, salary)  # استدعاء __init__ الأساسي أولاً
        self.team_size = team_size

    def display(self) -> str:
        base_info = super().display()  # استدعاء display الأساسي، ثم إضافة عليه
        return f"{base_info} (يدير فريق من {self.team_size} أشخاص)"

manager = Manager("نورة", 15000, 8)
print(manager.display())
# نورة: 15000 ر.س (يدير فريق من 8 أشخاص)
```

⚠ **تحذير شائع جداً**: نسيان استدعاء `super().__init__()` عند تخصيص `__init__` في فئة فرعية:

```python
class Manager(Employee):
    def __init__(self, name, salary, team_size):
        # ⚠ نسيان super().__init__(name, salary)!
        self.team_size = team_size

manager = Manager("نورة", 15000, 8)
print(manager.name)
# 💥 AttributeError: 'Manager' object has no attribute 'name'
```

بدون استدعاء `super().__init__()`، لن تُهيَّأ الخصائص المُعرَّفة في الفئة الأساسية إطلاقاً.

💡 **نصيحة الخبراء**: استدعِ `super().__init__()` عادة **كأول سطر** في `__init__` الفئة الفرعية، لضمان تهيئة كل الخصائص الموروثة قبل إضافة أي خصائص جديدة.

التحقق من العلاقات: `isinstance()` و `issubclass()`

```python
print(isinstance(dog, Dog))     # True
print(isinstance(dog, Animal))  # True - Dog أيضاً هي Animal!
print(isinstance(dog, Cat))     # False

print(issubclass(Dog, Animal))  # True
print(issubclass(Animal, Dog))  # False - العلاقة أحادية الاتجاه
```

نصيحة الخبراء: استخدم `isinstance()` دائماً بدلاً من `type(obj) == SomeClass` للتحقق من نوع كائن، لأن `isinstance()` تأخذ الوراثة بعين الاعتبار:

```python
# ❌ لا يأخذ الوراثة في الحسبان
if type(dog) == Animal:  # False حتى لو dog هو كلب (نوع فرعي من Animal)!
    pass

# ✅ يأخذ الوراثة في الحسبان
if isinstance(dog, Animal):  # True
    pass
```

الوراثة متعددة المستويات (Multi-Level Inheritance)

Python

```python
class Vehicle:
    def __init__(self, brand: str):
        self.brand = brand

    def describe(self) -> str:
        return f"مركبة من {self.brand}"

class Car(Vehicle):
    def __init__(self, brand: str, doors: int):
        super().__init__(brand)
        self.doors = doors

    def describe(self) -> str:
        return f"{super().describe()}, عدد الأبواب: {self.doors}"

class SportsCar(Car):
    def __init__(self, brand: str, doors: int, top_speed: int):
        super().__init__(brand, doors)
        self.top_speed = top_speed

    def describe(self) -> str:
        return f"{super().describe()}, السرعة القصوى: {self.top_speed} كم/س"

ferrari = SportsCar("Ferrari", 2, 340)
print(ferrari.describe())
# مركبة من Ferrari, 340 كم/س :السرعة القصوى ,2 :عدد الأبواب

print(isinstance(ferrari, Vehicle))   # True
print(isinstance(ferrari, Car))       # True
print(isinstance(ferrari, SportsCar)) # True
```

الوراثة المتعددة (Multiple Inheritance) و MRO

Python تسمح بوراثة فئة من أكثر من فئة أساسية واحدة:

```python
class Swimmer:
    def swim(self) -> str:
        return "يسبح 🏊"

class Flyer:
    def fly(self) -> str:
        return "يطير 🦅"

class Duck(Swimmer, Flyer):  # وراثة من فئتين معاً
    def __init__(self, name: str):
        self.name = name

duck = Duck("دونالد")
print(duck.swim())  # يسبح 🏊
print(duck.fly())   # يطير 🦅
```

⚠ **تحذير احترافي:** الوراثة المتعددة قوية لكنها قد تُسبب تعقيداً كبيراً، خاصة عند تعارض الأسماء بين الفئات الأساسية. Python تحل هذا عبر خوارزمية تُسمى **MRO (Method Resolution Order)**:

```python
print(Duck.__mro__)
# (<class 'Duck'>, <class 'Swimmer'>, <class 'Flyer'>, <class 'object'>)
```

هذا يُحدد ترتيب البحث عن أي attribute: Python تبحث أولاً في `Duck`، ثم `Swimmer`، ثم `Flyer`، ثم `object` (الفئة الأساسية لكل شيء في Python).

💡 **نصيحة الخبراء:** تجنّب الوراثة المتعددة المعقدة قدر الإمكان في مشاريعك الخاصة؛ استخدمها فقط لحالات بسيطة وواضحة (تُسمى أحياناً **Mixins** — فئات صغيرة تُضيف سلوكاً محدداً). إذا وجدت نفسك تحتاج هرمية وراثة معقدة جداً، فكّر في **Composition** (تضمين كائن داخل آخر) كبديل أنظف غالباً — مبدأ معروف باسم "Composition over Inheritance".

14.3 تعدد الأشكال (Polymorphism)

الفكرة الأساسية

Polymorphism (تعدد الأشكال) يعني: كائنات من فئات مختلفة يمكن التعامل معها بواجهة موحدة، لكن كل واحدة تتصرف بطريقتها الخاصة.

Python

```python
class Shape:
    def area(self) -> float:
        raise NotImplementedError("يجب تخصيص هذه الدالة في الفئة الفرعية")

class Circle(Shape):
    def __init__(self, radius: float):
        self.radius = radius

    def area(self) -> float:
        return 3.14159 * self.radius ** 2

class Rectangle(Shape):
    def __init__(self, width: float, height: float):
        self.width = width
        self.height = height

    def area(self) -> float:
        return self.width * self.height

class Triangle(Shape):
    def __init__(self, base: float, height: float):
        self.base = base
        self.height = height

    def area(self) -> float:
        return 0.5 * self.base * self.height

# هنا يكمن سحر تعدد الأشكال:
shapes: list[Shape] = [Circle(5), Rectangle(4, 6), Triangle(3, 8)]

for shape in shapes:
    print(f"{type(shape).__name__}: المساحة = {shape.area():.2f}")
```

الناتج:

```
Circle: 78.54 = المساحة
Rectangle: 24.00 = المساحة
Triangle: 12.00 = المساحة
```

لاحظ القوة هنا: الكود `shape.area()` **لا يعرف ولا يهتم** بنوع الشكل الفعلي أثناء كتابته. كل كائن «يعرف» كيف يحسب مساحته بطريقته الخاصة، والكود الذي يستخدمها يبقى موحداً وبسيطاً، بغض النظر عن إضافة أشكال جديدة لاحقاً.

> **نصيحة الخبراء:** هذا هو جوهر مبدأ **Open/Closed Principle** (أحد مبادئ SOLID التي سنلمسها أكثر في الفصل 24): الكود **مفتوح للتوسعة** (يمكنك إضافة `Pentagon` جديدة بسهولة) لكنه **مغلق للتعديل** (لا تحتاج تعديل حلقة `for` الموجودة أعلاه عند إضافة شكل جديد).

Duck Typing: فلسفة Python الفريدة

```python
class Duck:
    def make_sound(self) -> str:
        return "Quack!"

class Robot:
    def make_sound(self) -> str:
        return "Beep Boop!"

def animate(entity) -> None:
    """لا تهتم بنوع entity، فقط تحتاج أن تملك make_sound()"""
    print(entity.make_sound())

animate(Duck())    # Quack!
animate(Robot())   # Beep Boop! -- Robot ليست حيواناً إطلاقاً لكنها تعمل!
```

> **الفلسفة:** "If it walks like a duck and quacks like a duck, it's a duck" — إذا كان الكائن يملك السلوك المطلوب (method أو attribute)، Python لا تهتم بنوعه الفعلي أو علاقته بأي هرمية وراثة. هذا يختلف جذرياً عن لغات صارمة التنميط مثل Java حيث يجب أن تُعرِّف Interface صريحة. في Python، **السلوك** أهم من **النوع المُعلَن**.

14.4 الفئات المجردة (Abstract Base Classes)

المشكلة: منع إنشاء كائنات من فئة «غير مكتملة»

في مثال `Shape` أعلاه، لا معنى منطقي لإنشاء كائن `Shape()` مباشرة — إنها فكرة عامة، ليست شكلاً حقيقياً محدداً. نريد **إجبار** أي فئة فرعية على تخصيص `area()`، ومنع إنشاء `Shape` مباشرة.

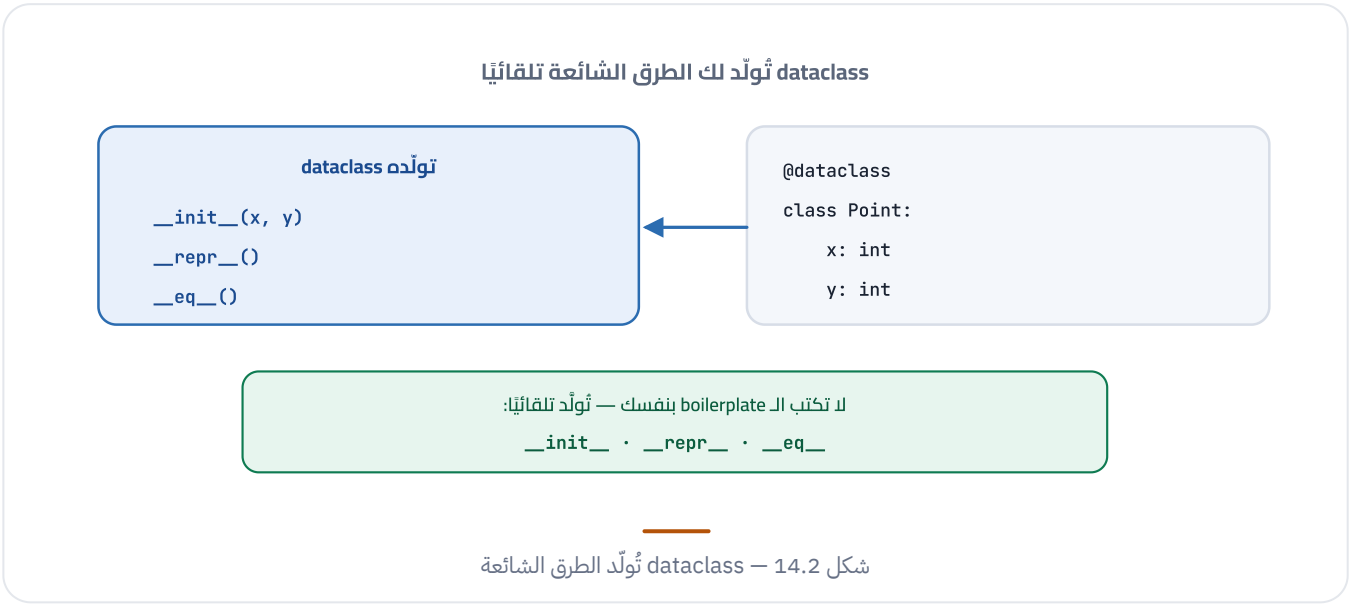
```python
```

```python
from abc import ABC, abstractmethod

class Shape(ABC):  # ترث من ABC (Abstract Base Class)
    @abstractmethod
    def area(self) -> float:
        """يجب أن تُخصَّص في كل فئة فرعية."""
        pass

    @abstractmethod
    def perimeter(self) -> float:
        """يجب أن تُخصَّص في كل فئة فرعية."""
        pass

    def describe(self) -> str:
        """method موروثة كما هي، لا حاجة لتخصيصها - عادية."""
        return f"{type(self).__name__}: مساحة={self.area():.2f}, محيط={self.perimeter():.2f}"

class Circle(Shape):
    def __init__(self, radius: float):
        self.radius = radius

    def area(self) -> float:
        return 3.14159 * self.radius ** 2

    def perimeter(self) -> float:
        return 2 * 3.14159 * self.radius

# shape = Shape()
# 💥 TypeError: Can't instantiate abstract class Shape with abstract methods area, perimeter

circle = Circle(5)
print(circle.describe())  # Circle: مساحة=78.54, محيط=31.42

class Square(Shape):
    def __init__(self, side: float):
        self.side = side
    # ⚠️ هنا: لو نسينا تعريف area() أو perimeter()

# square = Square(5)
# 💥 TypeError: Can't instantiate abstract class Square with abstract method area, perimeter
```

نصيحة الخبراء - متى تستخدم Abstract Base Classes؟ عندما تريد تعريف «عقد» (Contract) إلزامي: كل فئة فرعية **يجب** أن تُخصص دوالاً معينة، وإلا لن تستطيع إنشاء كائنات منها أصلاً. هذا يفرض انضباطاً في فرق العمل الكبيرة، ويمنع أخطاءً مثل نسيان تخصيص method ضرورية.

مقارنة مع لغات أخرى: هذا المفهوم يُشبه `interface` في Java أو `abstract class` في C++. الفرق الجوهري أن Python توفرها كمكتبة اختيارية (`abc`) وليس ميزة إلزامية في اللغة، تماشياً مع فلسفة المرونة العامة لـ Python.

Dataclasses 14.5 : اختصار كتابة الفئات



شكل 14.2 — dataclass تُولّد الطرق الشائعة

المشكلة: التكرار الممل في الفئات البسيطة

Python

```python
# الطريقة التقليدية - كثير من الكود المتكرر لفئة بسيطة تخزن بيانات فقط
class Point:
    def __init__(self, x: float, y: float):
        self.x = x
        self.y = y

    def __repr__(self) -> str:
        return f"Point(x={self.x}, y={self.y})"

    def __eq__(self, other) -> bool:
        if not isinstance(other, Point):
            return False
        return self.x == other.x and self.y == other.y
```

الحل: `@dataclass`

```python
from dataclasses import dataclass

@dataclass
class Point:
    x: float
    y: float

p1 = Point(3, 4)
p2 = Point(3, 4)

print(p1)          # Point(x=3, y=4)  -- __repr__ تلقائياً!
print(p1 == p2)    # True  -- __eq__ تلقائياً!
```

هذا كل شيء! `@dataclass` تُولّد تلقائياً `__init__`، `__repr__`، و `__eq__` بناءً على الحقول (Fields) المُعرَّفة بصيغة Type Hints، دون كتابة سطر واحد إضافي.

قيم افتراضية في Dataclasses

```python
from dataclasses import dataclass, field

@dataclass
class Student:
    name: str
    age: int = 18  # قيمة افتراضية بسيطة
    grades: list[float] = field(default_factory=list)  # للقيم القابلة للتعديل!

s1 = Student("أحمد")
s1.grades.append(90)
print(s1)  # Student(name='أحمد', age=18, grades=[90])

s2 = Student("سارة")
print(s2)  # Student(name='سارة', age=18, grades=[])  -- مستقلة تماماً!
```

⚠ **تحذير حرج:** تذكر فخ "mutable default argument" من الفصل العاشر؟ نفس المشكلة تماماً تنطبق هنا! **لا تكتب** `grades: list = []` مباشرة في dataclass:

```python
@dataclass
class Student:
    grades: list = []  # 💥 ValueError: mutable default <class 'list'> is not allowed
```

Python تمنعك صراحة من هذا الخطأ في Dataclasses (رسالة خطأ واضحة!)، وتُجبرك على استخدام `field(default_factory=list)` بدلاً منها، والذي يُنشئ قائمة **جديدة** لكل كائن.

Methods إضافية في Dataclasses

يمكنك إضافة methods عادية بجانب الحقول:

```python
from dataclasses import dataclass

@dataclass
class Rectangle:
    width: float
    height: float

    def area(self) -> float:
        return self.width * self.height

    def perimeter(self) -> float:
        return 2 * (self.width + self.height)

rect = Rectangle(5, 3)
print(rect)             # Rectangle(width=5, height=3)
print(rect.area())      # 15
```

خيارات متقدمة: `frozen` و `order`

```python
from dataclasses import dataclass

@dataclass(frozen=True)  # يجعل الكائن immutable - كالـ tuple!
class ImmutablePoint:
    x: float
    y: float

p = ImmutablePoint(3, 4)
# p.x = 10
# 💥 FrozenInstanceError: cannot assign to field 'x'

@dataclass(order=True)  # يُضيف __lt__, __le__, __gt__, __ge__ تلقائياً
class Version:
    major: int
    minor: int

v1 = Version(1, 5)
v2 = Version(2, 0)
print(v1 < v2)  # True -- تقارن الحقول بالترتيب المُعرَّف
```

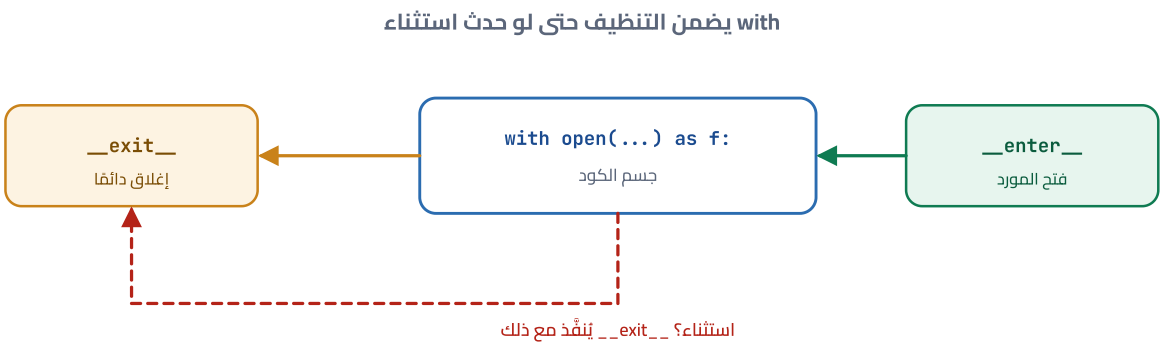
نصيحة الخبراء: استخدم `frozen=True` عندما تريد كائناً ثابتاً لا يجب تعديله بعد الإنشاء (مثل: إحداثيات، إعدادات ثابتة، مفاتيح لقاموس). هذا يمنع أخطاءً كثيرة ناتجة عن تعديل غير مقصود.

جدول مقارنة: Class عادية مقابل Dataclass

الخاصية	Class عادية	`@dataclass`
`__init__`	يدوي	تلقائي من الحقول
`__repr__`	يدوي	تلقائي، منسق بوضوح
`__eq__`	يدوي	تلقائي، يقارن كل الحقول
التحكم الكامل بالسلوك	كامل	كامل (يمكن إضافة methods)
مناسبة لـ	فئات بسلوك معقد ومنطق مخصص	فئات تُركّز بشكل أساسي على تخزين بيانات

مقارنة: `@dataclass` تشبه `record` في Java الحديثة، أو `struct` في بعض اللغات، أو حتى `namedtuple` في Python القديمة (لكن Dataclass أقوى وأكثر مرونة). القاعدة العملية: **إذا كانت فئتك تُركّز أساساً على تجميع بيانات مع القليل من السلوك، استخدم `@dataclass` دائماً** — توفر وقتاً وتقلل الأخطاء بشكل كبير.

14.6 مزيد من Magic Methods: `__call__` ، `__getitem__` ، Context Managers



شكل 14.3 — with يضمن التنظيف دائمًا

`__call__` : جعل الكائن «قابلاً للاستدعاء»

```python
class Multiplier:
    def __init__(self, factor: float):
        self.factor = factor

    def __call__(self, value: float) -> float:
        return value * self.factor

double = Multiplier(2)
print(double(5))   # 10 -- الكائن نفسه يُستدعى كدالة!
print(double(10))  # 20
```

`__getitem__`: دعم الفهرسة `obj[index]`

```python
class Playlist:
    def __init__(self, songs: list[str]):
        self.songs = songs

    def __getitem__(self, index: int) -> str:
        return self.songs[index]

    def __len__(self) -> int:
        return len(self.songs)

playlist = Playlist(["أغنية 3" ,"أغنية 2" ,"أغنية 1"])
print(playlist[0])  # الفهرسة تعمل الآن-- أغنية 1!

for song in playlist:  # __getitem__ تجعل الكائن قابلاً للتكرار تلقائياً أيضاً!
    print(song)
```

`__enter__` و `__exit__`: بناء Context Manager مخصص

تذكر `with open(...) as f:` من الفصل القادم (16)؟ يمكنك بناء كائنات تدعم نفس الصيغة:

```python
class Timer:
    """context manager يقيس زمن تنفيذ كتلة كود."""

    def __enter__(self):
        import time
        self.start = time.perf_counter()
        return self

    def __exit__(self, exc_type, exc_value, traceback):
        import time
        self.end = time.perf_counter()
        print(f"⏱ استغرق التنفيذ: {self.end - self.start:.4f} ثانية")

with Timer():
    total = sum(i ** 2 for i in range(1_000_000))
    print(f"النتيجة: {total}")
```

الناتج:

```
النتيجة: 333332833333500000
⏱ استغرق التنفيذ: 0.0842 ثانية
```

سنتعمق أكثر في Context Managers في الفصل 16 عند التعامل مع الملفات.

14.7 مشروع تطبيقي متكامل: نظام حجز مكتبة

هذا المشروع يجمع كل ما تعلمناه في الفصلين 13 و14: الوراثة، تعدد الأشكال، الفئات المجردة، Dataclasses، وMagic Methods.

Python

```python
from abc import ABC, abstractmethod
from dataclasses import dataclass, field
from datetime import datetime, timedelta
from enum import Enum, auto

# ============ الطبقة 1: نماذج البيانات (Dataclasses) ============

class ItemStatus(Enum):
    """حالة العنصر في المكتبة."""
    AVAILABLE = auto()
    BORROWED = auto()
    RESERVED = auto()

@dataclass
class Member:
    """يمثل عضواً في المكتبة."""
    member_id: int
    name: str
    borrowed_items: list[str] = field(default_factory=list)

    def __str__(self) -> str:
        return f"عضو #{self.member_id}: {self.name}"

# ============ الطبقة 2: الفئة المجردة للعناصر القابلة للاستعارة ============

class LibraryItem(ABC):
    """فئة مجردة أساسية لأي عنصر في المكتبة."""

    def __init__(self, item_id: str, title: str, loan_days: int):
        self.item_id = item_id
        self.title = title
        self.loan_days = loan_days  # عدد الأيام المسموحة للاستعارة
        self.status = ItemStatus.AVAILABLE
        self.borrowed_by: Member | None = None
        self.due_date: datetime | None = None
```

```
    @abstractmethod
    def get_category(self) -> str:
        """كل فئة فرعية يجب أن تحدد فئتها."""
        pass

    @abstractmethod
    def display_details(self) -> str:
        """كل فئة فرعية تعرض تفاصيلها الخاصة."""
        pass

    def borrow(self, member: Member) -> bool:
        """استعارة العنصر - سلوك مشترك لكل الأنواع."""
        if self.status != ItemStatus.AVAILABLE:
            print(f"❌ '{self.title}' غير متاح حالياً (الحالة: {self.status.name}).")
            return False

        self.status = ItemStatus.BORROWED
        self.borrowed_by = member
        self.due_date = datetime.now() + timedelta(days=self.loan_days)
        member.borrowed_items.append(self.item_id)

        print(f"✅ استعار {member.name} '{self.title}'. "
              f"الإرجاع المتوقع: {self.due_date.strftime('%Y-%m-%d')}")
        return True

    def return_item(self) -> None:
        """إرجاع العنصر."""
        if self.status != ItemStatus.BORROWED:
            print(f"⚠️ '{self.title}' ليس مُستعاراً أصلاً.")
            return

        if self.borrowed_by:
            self.borrowed_by.borrowed_items.remove(self.item_id)
            print(f"✅ تم إرجاع '{self.title}' من {self.borrowed_by.name}.")

        self.status = ItemStatus.AVAILABLE
        self.borrowed_by = None
        self.due_date = None

    def is_overdue(self) -> bool:
        """التحقق من تأخر الإرجاع - سلوك مشترك."""
        if self.status != ItemStatus.BORROWED or self.due_date is None:
            return False
        return datetime.now() > self.due_date

    def __str__(self) -> str:
        status_display = self.status.name
        return f"[{self.get_category()}] {self.title} ({self.item_id}) - {status_display}"

    def __eq__(self, other) -> bool:
```

```
        if not isinstance(other, LibraryItem):
            return False
        return self.item_id == other.item_id

    def __hash__(self) -> int:
        return hash(self.item_id)

# ============ الطبقة 3: الفئات الفرعية (تطبيق تعدد الأشكال) ============

class Book(LibraryItem):
    def __init__(self, item_id: str, title: str, author: str, pages: int):
        super().__init__(item_id, title, loan_days=14)  # الكتب تُستعار 14 يوماً
        self.author = author
        self.pages = pages

    def get_category(self) -> str:
        return "كتاب"

    def display_details(self) -> str:
        return f"📖 {self.title} - تأليف: {self.author} ({self.pages} صفحة)"

class DVD(LibraryItem):
    def __init__(self, item_id: str, title: str, duration_minutes: int):
        super().__init__(item_id, title, loan_days=5)  # الأفلام تُستعار 5 أيام فقط
        self.duration_minutes = duration_minutes

    def get_category(self) -> str:
        return "دي في دي"

    def display_details(self) -> str:
        return f"📀 {self.title} - المدة: {self.duration_minutes} دقيقة"

class Magazine(LibraryItem):
    def __init__(self, item_id: str, title: str, issue_number: int):
        super().__init__(item_id, title, loan_days=3)  # المجلات 3 أيام فقط
        self.issue_number = issue_number

    def get_category(self) -> str:
        return "مجلة"

    def display_details(self) -> str:
        return f"📰 {self.title} - العدد: {self.issue_number}"

# ============ الطبقة 4: استثناءات مخصصة ============

class LibraryError(Exception):
    """الفئة الأساسية لكل أخطاء نظام المكتبة."""
    pass

class ItemNotFoundError(LibraryError):
    def __init__(self, item_id: str):
```

```
        super().__init__(f"العنصر '{item_id}' غير موجود في المكتبة.")

class MemberNotFoundError(LibraryError):
    def __init__(self, member_id: int):
        super().__init__(f"العضو رقم {member_id} غير موجود.")

# ============ الطبقة 5: النظام الرئيسي ============

class Library:
    """يُدير كل عمليات المكتبة: الأعضاء، العناصر، الاستعارة، الإرجاع."""

    def __init__(self, name: str):
        self.name = name
        self.items: dict[str, LibraryItem] = {}
        self.members: dict[int, Member] = {}

    # --- إدارة العناصر ---
    def add_item(self, item: LibraryItem) -> None:
        self.items[item.item_id] = item
        print(f"📚 أُضيف '{item.title}' إلى مكتبة '{self.name}'.")

    def find_item(self, item_id: str) -> LibraryItem:
        try:
            return self.items[item_id]
        except KeyError:
            raise ItemNotFoundError(item_id)

    # --- إدارة الأعضاء ---
    def register_member(self, member: Member) -> None:
        self.members[member.member_id] = member
        print(f"👤 تم تسجيل {member.name} كعضو جديد.")

    def find_member(self, member_id: int) -> Member:
        try:
            return self.members[member_id]
        except KeyError:
            raise MemberNotFoundError(member_id)

    # --- العمليات الأساسية ---
    def borrow_item(self, item_id: str, member_id: int) -> None:
        item = self.find_item(item_id)
        member = self.find_member(member_id)
        item.borrow(member)

    def return_item(self, item_id: str) -> None:
        item = self.find_item(item_id)
        item.return_item()

    # --- تقارير (تطبيق تعدد الأشكال بوضوح) ---
    def list_all_items(self) -> None:
        print(f"\n📋 كل عناصر مكتبة '{self.name}':")
```

```
        print("-" * 60)
        for item in self.items.values():
            print(item.display_details())  # كل نوع يعرض تفاصيله الخاصة!
        print("-" * 60)

    def list_available_items(self) -> list[LibraryItem]:
        return [item for item in self.items.values()
                if item.status == ItemStatus.AVAILABLE]

    def list_overdue_items(self) -> list[LibraryItem]:
        return [item for item in self.items.values() if item.is_overdue()]

    def items_by_category(self) -> dict[str, int]:
        """عدد العناصر حسب الفئة - باستخدام polymorphism (get_category)."""
        counts: dict[str, int] = {}
        for item in self.items.values():
            category = item.get_category()
            counts[category] = counts.get(category, 0) + 1
        return counts

# ============ التشغيل التجريبي ============

if __name__ == "__main__":
    library = Library("مكتبة الأمل العامة")

    # إضافة عناصر متنوعة (تعدد الأشكال في العمل)
    library.add_item(Book("B001", "350 ,"محمد أحمد" ,"بايثون للمحترفين"))
    library.add_item(Book("B002", "Clean Code", "Robert Martin", 400))
    library.add_item(DVD("D001", "120 ,"رحلة إلى الفضاء"))
    library.add_item(Magazine("M001", "45 ,"التقنية اليوم"))

    # تسجيل أعضاء
    library.register_member(Member(1, "أحمد"))
    library.register_member(Member(2, "سارة"))

    print()
    library.list_all_items()

    print("\n--- عمليات الاستعارة ---")
    library.borrow_item("B001", 1)
    library.borrow_item("D001", 2)

    try:
        library.borrow_item("B001", 2)  # محاولة استعارة كتاب مُستعار بالفعل
    except LibraryError as e:
        print(f"❌ {e}")

    try:
        library.borrow_item("X999", 1)  # عنصر غير موجود
    except ItemNotFoundError as e:
        print(f"❌ {e}")
```

```
    print("\n--- حالة المكتبة ---")
    library.list_all_items()

    available = library.list_available_items()
    print(f"\n✅ العناصر المتاحة حالياً: {len(available)}")
    for item in available:
        print(f"  - {item}")

    print(f"\n📊 توزيع العناصر حسب الفئة: {library.items_by_category()}")

    print("\n--- إرجاع عنصر ---")
    library.return_item("B001")

    library.list_all_items()
```

الناتج المتوقع (مختصر):

```
📚 أُضيف 'بايثون للمحترفين' إلى مكتبة 'مكتبة الأمل العامة'.
📚 أُضيف 'Clean Code' إلى مكتبة 'مكتبة الأمل العامة'.
📚 أُضيف 'رحلة إلى الفضاء' إلى مكتبة 'مكتبة الأمل العامة'.
📚 أُضيف 'التقنية اليوم' إلى مكتبة 'مكتبة الأمل العامة'.
👤 تم تسجيل أحمد كعضو جديد.
👤 تم تسجيل سارة كعضو جديد.

📋 كل عناصر مكتبة 'مكتبة الأمل العامة':
------------------------------------------------------------
📖 بايثون للمحترفين - تأليف: محمد أحمد (350 صفحة)
📖 Clean Code - تأليف: Robert Martin (400 صفحة)
🎬 رحلة إلى الفضاء - المدة: 120 دقيقة
🗞️ التقنية اليوم - العدد: 45
------------------------------------------------------------

---  عمليات الاستعارة  ---
✅ استعار أحمد 'بايثون للمحترفين'. الإرجاع المتوقع: 29-01-2025
✅ استعار سارة 'رحلة إلى الفضاء'. الإرجاع المتوقع: 20-01-2025
❌ 'بايثون للمحترفين' غير متاح حالياً (الحالة: BORROWED).
❌ العنصر 'X999' غير موجود في المكتبة.

---  حالة المكتبة  ---
📋 كل عناصر مكتبة 'مكتبة الأمل العامة':
------------------------------------------------------------
📖 بايثون للمحترفين - تأليف: محمد أحمد (350 صفحة)
📖 Clean Code - تأليف: Robert Martin (400 صفحة)
🎬 رحلة إلى الفضاء - المدة: 120 دقيقة
🗞️ التقنية اليوم - العدد: 45
------------------------------------------------------------

✅ العناصر المتاحة حالياً: 2
```

```
   - [كتاب] Clean Code (B002) - AVAILABLE
   - [مجلة] التقنية اليوم (M001) - AVAILABLE

📊 توزيع العناصر حسب الفئة: {'كتاب': 2, 'دي في دي': 1, 'مجلة': 1}

---  إرجاع عنصر---
✅ تم إرجاع 'بايثون للمحترفين' من أحمد.

📋 كل عناصر مكتبة 'مكتبة الأمل العامة':
------------------------------------------------------------
📖 بايثون للمحترفين - تأليف: محمد أحمد (350 صفحة)
📖 Clean Code - تأليف: Robert Martin (400 صفحة)
🎬 رحلة إلى الفضاء - المدة: 120 دقيقة
📰 التقنية اليوم - العدد: 45
------------------------------------------------------------
```

تحليل التصميم: لاحظ كيف أن:

- `LibraryItem` فئة مجردة تفرض على كل نوع فرعي تعريف `get_category()` و `display_details()`
- السلوك المشترك (`borrow`, `return_item`, `is_overdue`) مكتوب **مرة واحدة فقط** في الفئة الأساسية
- `Library.list_all_items()` لا تحتاج معرفة نوع كل عنصر — **تعدد الأشكال** يتكفل بذلك تلقائياً
- الاستثناءات المخصصة (`ItemNotFoundError`, `MemberNotFoundError`) تجعل معالجة الأخطاء دقيقة وواضحة

✅ تحدي إضافي: إضافة نظام غرامات التأخير (الحل)

Python

```python
class LibraryItem(ABC):
    # ... (الكود السابق كما هو) ...
    FINE_PER_DAY = 2.0  # ريال لكل يوم تأخير

    def calculate_fine(self) -> float:
        """حساب الغرامة إذا كان العنصر متأخراً."""
        if not self.is_overdue() or self.due_date is None:
            return 0.0
        days_late = (datetime.now() - self.due_date).days
        return days_late * self.FINE_PER_DAY

# استخدام:
item = library.find_item("B001")
fine = item.calculate_fine()
if fine > 0:
    print(f"⚠️ غرامة تأخير: {fine} ر.س")
```

14.8 تمارين

🟢 مستوى مبتدئ

تمرين 1: أنشئ فئة أساسية `Shape` بـ method `area()` تُرجع 0 افتراضياً، وفئة فرعية `Square(Shape)` تُخصص `area()` لحساب مساحة المربع.

تمرين 2: حوّل فئة `Point` (بحقلين x, y) إلى `@dataclass`، واطبع كائنين للمقارنة.

✅ الحل

Python

```python
# تمرين 1
class Shape:
    def area(self) -> float:
        return 0

class Square(Shape):
    def __init__(self, side: float):
        self.side = side

    def area(self) -> float:
        return self.side ** 2

sq = Square(4)
print(sq.area())  # 16

# تمرين 2
from dataclasses import dataclass

@dataclass
class Point:
    x: float
    y: float

p1 = Point(1, 2)
p2 = Point(1, 2)
print(p1)          # Point(x=1, y=2)
print(p1 == p2)    # True
```

🟡 مستوى متوسط

تمرين 3: أنشئ فئة مجردة `PaymentMethod` بـ abstractmethod باسم `process(amount)`. أنشئ فئتين فرعيتين: `CreditCard` و `PayPal`، كل منهما تطبع رسالة مختلفة عند `process()`.

تمرين 4: أنشئ `@dataclass` باسم `Product` بحقول `name`, `price`, `quantity=1`. أضف method `total()` تحسب `price * quantity`. استخدم `frozen=True`.

✅ الحل

Python

```python
# تمرين 3
from abc import ABC, abstractmethod

class PaymentMethod(ABC):
    @abstractmethod
    def process(self, amount: float) -> None:
        pass

class CreditCard(PaymentMethod):
    def process(self, amount: float) -> None:
        print(f"💳 تمت معالجة {amount} عبر البطاقة الائتمانية ر.س.")

class PayPal(PaymentMethod):
    def process(self, amount: float) -> None:
        print(f"🅿️ تمت معالجة {amount} ر.س عبر PayPal.")

methods: list[PaymentMethod] = [CreditCard(), PayPal()]
for method in methods:
    method.process(100)

# تمرين 4
from dataclasses import dataclass

@dataclass(frozen=True)
class Product:
    name: str
    price: float
    quantity: int = 1

    def total(self) -> float:
        return self.price * self.quantity

p = Product("لابتوب", 3500, 2)
print(p.total())  # 7000
# p.price = 4000  # 💥 FrozenInstanceError
```

🔴 تحدي

تمرين 5: صمم نظام موظفين: فئة مجردة `Employee` بـ abstractmethod `calculate_salary()`. فئتان فرعيتان: `FullTimeEmployee` (راتب ثابت شهري) و `FreelanceEmployee` (أجر بالساعة × عدد الساعات). أضف فئة `Payroll` تحسب إجمالي رواتب قائمة موظفين مختلطة النوع (تطبيق تعدد الأشكال).

✅ الحل

Python

```python
from abc import ABC, abstractmethod

class Employee(ABC):
    def __init__(self, name: str):
        self.name = name

    @abstractmethod
    def calculate_salary(self) -> float:
        pass

    def __str__(self) -> str:
        return f"{self.name}: {self.calculate_salary():.2f} ر.س"

class FullTimeEmployee(Employee):
    def __init__(self, name: str, monthly_salary: float):
        super().__init__(name)
        self.monthly_salary = monthly_salary

    def calculate_salary(self) -> float:
        return self.monthly_salary

class FreelanceEmployee(Employee):
    def __init__(self, name: str, hourly_rate: float, hours_worked: float):
        super().__init__(name)
        self.hourly_rate = hourly_rate
        self.hours_worked = hours_worked

    def calculate_salary(self) -> float:
        return self.hourly_rate * self.hours_worked

class Payroll:
    def __init__(self):
        self.employees: list[Employee] = []

    def add_employee(self, employee: Employee) -> None:
        self.employees.append(employee)

    def total_payroll(self) -> float:
        return sum(emp.calculate_salary() for emp in self.employees)

    def print_report(self) -> None:
        print("📋 تقرير الرواتب:")
        for emp in self.employees:
            print(f"  {emp}")
```

```
        print(f"الإجمالي: {self.total_payroll():.2f} ر.س")

payroll = Payroll()
payroll.add_employee(FullTimeEmployee("أحمد", 8000))
payroll.add_employee(FreelanceEmployee("سارة", 120, 50))
payroll.print_report()

# 📋 تقرير الرواتب:
#   أحمد: 8000.00 ر.س
#   سارة: 6000.00 ر.س
# الإجمالي: 14000.00 ر.س
```

14.9 اختبر فهمك (Quiz)

❓ السؤال 1: ما وظيفة super() وأين يجب استدعاؤها عادة؟

الإجابة: `super()` تُتيح الوصول لـ methods الفئة الأساسية (Parent) من داخل الفئة الفرعية (Child). عادة تُستدعى `super().__init__(...)` كأول سطر في `__init__` الفئة الفرعية لضمان تهيئة كل الخصائص الموروثة من الفئة الأساسية قبل إضافة أي خصائص جديدة خاصة بالفئة الفرعية.

❓ السؤال 2: ما الفرق بين isinstance() و type() == عند التحقق من نوع الكائن؟

الإجابة: `isinstance(obj, Class)` تأخذ الوراثة بعين الاعتبار — تُرجع `True` إذا كان الكائن من تلك الفئة **أو من أي فئة فرعية منها**. بينما `type(obj) == Class` تتحقق من التطابق **الدقيق** فقط، وتُرجع `False` حتى لو كان الكائن من فئة فرعية. `isinstance()` هي الطريقة الموصى بها دائماً في Python.

❓ السؤال 3: لماذا لا يمكن إنشاء كائن مباشرة من فئة مجردة (Abstract Class)؟

الإجابة: لأن الفئة المجردة تحتوي على `@abstractmethod` واحدة أو أكثر بدون تنفيذ فعلي — إنها تُعرّف فقط «ما يجب أن تفعله» الفئات الفرعية، وليس «كيف». محاولة إنشاء كائن منها مباشرة تُطلق `TypeError`، لأن Python تمنع إنشاء كائنات من فئات «غير مكتملة» السلوك.

❓ السؤال 4: ما الفائدة الرئيسية من @dataclass مقارنة بكتابة الفئة يدوياً؟

الإجابة: `@dataclass` تُولّد تلقائياً `__init__`, `__repr__`, و `__eq__` بناءً على الحقول المُعرَّفة، دون الحاجة لكتابة أي من هذه الدوال يدوياً. هذا يختصر الكود بشكل كبير في الفئات التي تُركّز أساساً على تخزين بيانات، ويقلل احتمال الأخطاء الناتجة عن الكتابة اليدوية المتكررة.

❓ السؤال 5: اشرح مفهوم تعدد الأشكال (Polymorphism) بمثال من المشروع التطبيقي.

الإجابة: تعدد الأشكال يعني أن كائنات من فئات مختلفة (مثل `Book` , `DVD` , `Magazine`) يمكن التعامل معها بواجهة موحدة (مثل استدعاء `item.display_details()` على أي منها)، لكن كل نوع يُنفّذ السلوك بطريقته الخاصة. في مشروع المكتبة، `Library.list_all_items()` تستدعي `display_details()` على كل عنصر دون معرفة نوعه الفعلي، والنتيجة الصحيحة تظهر تلقائياً لكل نوع.

14.10 ملخص الفصل (Cheat Sheet)

Python

```python
# ============ الوراثة ============
class Parent:
    def __init__(self, x):
        self.x = x
    def greet(self):
        return f"Parent: {self.x}"

class Child(Parent):
    def __init__(self, x, y):
        super().__init__(x)    # استدعاء __init__ الأساسي
        self.y = y
    def greet(self):
        return f"{super().greet()}, Child: {self.y}"  # تمديد السلوك

# ============ تعدد الأشكال ============
for obj in [ClassA(), ClassB(), ClassC()]:
    obj.common_method()  # كل فئة تُنفّذ بطريقتها

# ============ الفئات المجردة ============
from abc import ABC, abstractmethod
```

```python
class Base(ABC):
    @abstractmethod
    def required_method(self): ...

# Base()  # 💥 TypeError - لا يمكن إنشاء كائن مباشرة

# ============ Dataclasses ============
from dataclasses import dataclass, field

@dataclass
class Point:
    x: float
    y: float
    tags: list = field(default_factory=list)  # ⚠️ ليس []

@dataclass(frozen=True)   # immutable
@dataclass(order=True)    # يدعم < <= >=

# ============ Magic Methods إضافية ============
def __call__(self, *args): ...      # obj(args)
def __getitem__(self, i): ...       # obj[i]
def __enter__(self): ...            # with obj as x:
def __exit__(self, *args): ...

# ============ isinstance ============
isinstance(obj, SomeClass)          # ✅ يأخذ الوراثة بعين الاعتبار
type(obj) == SomeClass              # ❌ تطابق دقيق فقط
```

المفهوم	الاستخدام الأمثل
وراثة بسيطة	عندما يوجد «هو نوع من» (Is-A) واضح: Dog is-a Animal
Composition	عندما يوجد «يملك» (Has-A): Car has-a Engine (بديل أنظف أحياناً من وراثة معقدة)
Abstract Base Class	فرض «عقد» إلزامي على كل فئة فرعية
Dataclass	فئات تُركّز على تخزين بيانات مع منطق بسيط
Duck Typing	عندما يهمك السلوك المتاح أكثر من النوع الرسمي

14.11 ماذا بعد؟

لقد أكملت الآن **الجزء الثاني بالكامل**: البرمجة الوظيفية والكائنية. تمتلك فهماً عميقاً لكيفية تنظيم الكود بطرق متقدمة، من Decorators وGenerators إلى الوراثة وتعدد الأشكال والـ Dataclasses.

لكن حتى الآن، اعتمدنا بشكل كبير على التجربة والخطأ لمعرفة أنواع البيانات المتوقعة في دوالنا. ماذا لو استطعت أن **تُصرّح** بوضوح لـ Python (ولزملائك، ولأدوات التحقق التلقائي) بالضبط ما هي الأنواع المتوقعة في كل مكان، وتكتشف الأخطاء **قبل** تشغيل الكود أصلاً؟

في **الفصل الخامس عشر: Type Hints - البرمجة الواضحة**، سنتعمق أكثر فيما بدأناه منذ الفصل الثالث: Type Hints المتقدمة، أداة `mypy` للتحقق التلقائي، وكيف تجعل Type Hints كودك أكثر أماناً ووضوحاً واحترافية — ممارسة أصبحت معياراً في كل مشاريع Python الحديثة والكبرى.

الجزء الرابع · 4 فصول

04

بايثون الحديثة

المهارات التي تفصل المبرمج العادي عن المحترف: تلميحات الأنواع، الملفات وتنسيقات البيانات، البرمجة غير المتزامنة، وإدارة المشاريع بأدوات العصر.

15 · Type Hints

16 · الملفات وتنسيقات البيانات

17 · البرمجة غير المتزامنة

18 · إدارة المشاريع كمحترف

فصل 15 / 26 · الجزء الرابع

Type Hints — نحو برمجة أكثر وضوحًا ودقة (باستخدام mypy)

18 دقيقة قراءة | 18 قسمًا | 50 مثالًا | 3,615 كلمة

في هذا الفصل

15.1 لماذا يستحق هذا الفصل اهتمامك الكامل؟
15.2 مراجعة سريعة: أين وصلنا حتى الآن؟
15.3 الأنواع المركّبة (Generic Types)
15.4 القيم الاختيارية: Optional و None
15.5 عندما يكون هناك أكثر من نوع ممكن: Union
15.6 النوع الهارب: Any
15.7 الدوال كقيم: Callable
15.8 القيم الثابتة المحدودة: Literal
15.9 القواميس المُهيكَلة: TypedDict
15.10 البرمجة العامة (Generics) باستخدام TypeVar
15.11 البروتوكولات: Protocol (Duck Typing المُهيكَل)
15.12 أداة mypy: التحقق الساكن من الأنواع
15.13 أفضل الممارسات العامة
15.14 مشروع تطبيقي: نظام إدارة مهام مُنمَّط بالكامل
15.15 تمارين
15.16 اختبر فهمك (Quiz)
15.17 ملخص الفصل (Cheat Sheet)
15.18 ماذا بعد؟

15.1 لماذا يستحق هذا الفصل اهتمامك الكامل؟

تخيّل نفسك ضمن فريق مكوّن من خمسة مبرمجين يعملون على مشروع ضخم. يكتب أحد زملائك دالة باسم `calculate_discount`، وتحتاج إلى استخدامها في كودك. تفتح تعريف الدالة فتجد:

```python
def calculate_discount(price, percentage, customer):
    ...
```

تتوالى الأسئلة في ذهنك فورًا:

- هل `price` عدد صحيح (`int`) أم عشري (`float`)؟
- هل `percentage` قيمة بين 0 و1 (مثل 0.2) أم بين 0 و100 (مثل 20)؟
- ما طبيعة `customer`؟ نص يمثّل اسمًا؟ كائن من فئة `Customer`؟ أم قاموس يحمل بيانات العميل؟
- وماذا تُرجع الدالة في النهاية؟ رقمًا؟ نصًا يمثّل الفاتورة؟

في لغة ديناميكية التنميط مثل Python، لا تجد إجابة قاطعة إلا بقراءة جسم الدالة كاملًا، أو بتجربتها وانتظار الخطأ. هذا يُهدر وقتك الثمين، ويفتح الباب أمام أخطاء قد لا تظهر إلا في بيئة الإنتاج (Production)، بعد أن يعتمد عليها آلاف المستخدمين.

والآن، تخيّل أن الدالة كُتبت على هذا النحو:

```python
def calculate_discount(price: float, percentage: float, customer: Customer) -> float:
    ...
```

بمجرد النظر إلى التوقيع (Signature)، اتضح لك كل شيء: الأسعار أرقام عشرية، النسبة رقم عشري، العميل كائن من فئة `Customer`، والناتج رقم عشري. لا حاجة لقراءة جسم الدالة، ولا مجال للتخمين.

هذا هو جوهر **Type Hints** (تلميحات الأنواع): توثيق حيّ ودقيق لكودك، يفهمه الإنسان والآلة معًا.

منذ الفصل الثالث، ونحن نكتب عبارات من هذا القبيل في كل مثال تقريبًا:

```python
age: int = 25
name: str = "محمد"

def greet(name: str) -> str:
    return f"مرحبًا، {name}!"
```

كنتَ تستخدمها دون الخوض في تفاصيلها. حان الوقت الآن لفهمها بعمق حقيقي: ما طبيعتها فعليًا؟ كيف تُستخدم مع هياكل البيانات المعقّدة؟ وكيف تُسخّر أداة باسم **mypy** للتحقق الآلي من كودك، واكتشاف الأخطاء قبل تشغيل البرنامج أصلًا؟

> **نصيحة الخبير:** لا تعتبر Type Hints «رفاهية للمحترفين فقط». شركات عملاقة مثل Google وDropbox وInstagram تعتمد عليها بكثافة في أكواد Python الإنتاجية، لأنها تقلّص الأخطاء بشكل ملموس، وتُمكّن أدوات مثل VS Code من اقتراح الكود الصحيح تلقائيًا (Autocomplete) أدق بكثير.

15.2 مراجعة سريعة: أين وصلنا حتى الآن؟

منذ الفصل الثالث، تعاملنا مع صيغة بسيطة ومباشرة:

```python
# متغيرات
age: int = 25
price: float = 19.99
name: str = "سارة"
is_active: bool = True

# دوال
def add(a: int, b: int) -> int:
    return a + b

def greet(name: str) -> None:
    print(f"مرحبًا، {name}")
```

القواعد الأساسية التي نعرفها:

العنصر	الصيغة	مثال
متغير	`name: Type = value`	`age: int = 25`
معامل دالة	`param: Type`	`def f(x: int):`
قيمة الإرجاع	`-> Type`	`def f() -> str:`
دالة لا تُرجع شيئًا	`-> None`	`def f() -> None:`

لكن حان وقت السؤال الجوهري الذي ربما يراودك منذ البداية:

هل تتحقق Python فعليًا من هذه الأنواع؟

الجواب القاطع: **لا، إطلاقًا**. جرّب هذا الكود:

```python
def add(a: int, b: int) -> int:
    return a + b

result = add("مرحبًا", "!")  # لا يوجد أي خطأ من Python!
print(result)  # "مرحبًا!"
```

شغّل هذا الكود، وستجده يعمل دون أدنى مشكلة! فسّرت Python العملية `+` على أنها دمج نصوص (Concatenation)، لأن `a` و `b` نصوص فعليًا وقت التشغيل، وتجاهلت التلميحة `int` تمامًا.

⚠️ **تحذير مهم — سوء فهم شائع:** Type Hints في Python مجرد تلميحات (Hints)، وليست آلية فرض (Enforcement). يتجاهلها المُفسِّر (Interpreter) تمامًا وقت التشغيل (باستثناء حالات نادرة متقدمة). وظيفتها الحقيقية تنحصر في:
- **القارئ البشري** (أنت، زملاؤك، أنت نفسك بعد ستة أشهر).
- **الأدوات الخارجية**، مثل المحرر (VS Code/Pylance) أو أداة تحقق ساكنة مثل mypy.

هذا يختلف جذريًا عن لغات مثل Java أو C# أو TypeScript، حيث يرفض المترجم (Compiler) تشغيل الكود إن كانت الأنواع غير متطابقة.

🔄 مقارنة: Python مقابل لغات ذات تنميط ساكن حقيقي

الخاصية	Python (مع Type Hints)	Java / C# / TypeScript
من يتحقق من الأنواع؟	أدوات خارجية اختيارية (mypy)	المترجم نفسه، إلزاميًا
هل يعمل الكود رغم خطأ في النوع؟	نعم (وقد ينهار لاحقًا)	لا، يرفض المترجم البناء أصلًا
هل Type Hints إلزامية؟	لا، اختيارية 100%	نعم (أو شبه إلزامية)
الاسم التقني	Gradual Typing (تنميط تدريجي)	Static Typing (تنميط ساكن)

يقودنا هذا إلى سؤال محوري: كيف نستفيد إذًا من الفائدة الحقيقية لـ Type Hints، ما دامت Python تتجاهلها؟

الجواب: نستعين بأداة اسمها **mypy** تقرأ كودك قبل التشغيل وتتحقق من تطابق الأنواع، ثم تُبلغك بأي أخطاء عبر رسائل نصية واضحة. سنتناولها بالتفصيل بعد أن نُتقن أولًا كتابة الأنواع نفسها.

15.3 الأنواع المركّبة (Generic Types)

تعاملنا حتى الآن مع أنواع بسيطة: `int`، `float`، `str`، `bool`، `None`. لكن ماذا عن القوائم والقواميس؟

القوائم: `list[T]`

```python
# قائمة أعداد صحيحة
numbers: list[int] = [1, 2, 3, 4, 5]

# قائمة نصوص
names: list[str] = ["أحمد", "سارة", "خالد"]

# قائمة متداخلة (قائمة من قوائم أعداد)
matrix: list[list[int]] = [[1, 2], [3, 4], [5, 6]]
```

`T` هنا رمز اصطلاحي يعني «أي نوع»، لكن في الاستخدام الفعلي نكتب النوع الحقيقي مثل `int` أو `str`.

نصيحة تاريخية: في إصدارات Python القديمة (قبل 3.9)، كان لا بدّ من استيراد `List` من وحدة `typing`:

```python
from typing import List
numbers: List[int] = [1, 2, 3]
```

منذ Python 3.9، أصبح بالإمكان استخدام `list` مباشرة بحرف صغير دون استيراد أي شيء. وبما أننا نستهدف Python 3.14 في هذا الكتاب، سنعتمد دائمًا الصيغة الحديثة (`list[int]` وليس `List[int]`).

الصناديق (Tuples): `tuple[T1, T2, ...]`

تتميّز الصناديق (Tuples) بأن كل عنصر فيها قد يكون من نوع مختلف، وعددها غالبًا ثابت:

```python
# صندوق يحتوي اسمًا (نص) وعمرًا (عدد صحيح)
person: tuple[str, int] = ("ليلى", 28)

# صندوق إحداثيات (x, y, z) - ثلاثة أعداد عشرية
point: tuple[float, float, float] = (1.5, 2.0, 3.5)
```

وإذا أردت صندوقًا بعدد متغيّر من العناصر، كلها من النوع نفسه، استخدم `...`:

```python
# صندوق بعدد غير محدد من الأعداد الصحيحة
scores: tuple[int, ...] = (90, 85, 100, 70, 60)
```

القواميس: `dict[K, V]`

```python
# قاموس: المفتاح نص، القيمة عدد صحيح
ages: dict[str, int] = {"أحمد": 30, "سارة": 25}

# قاموس أكثر تعقيدًا: المفتاح نص، القيمة قائمة نصوص
courses_by_student: dict[str, list[str]] = {
    "أحمد": ["رياضيات", "فيزياء"],
    "سارة": ["كيمياء"],
}
```

المجموعات: `set[T]`

```python
unique_ids: set[int] = {101, 102, 103}
tags: set[str] = {"برمجة", "تعليم", "python"}
```

مثال شامل يجمع كل العناصر السابقة

```python
def summarize_students(
    grades: dict[str, list[int]]
) -> dict[str, float]:
    """
    يستقبل قاموسًا بأسماء الطلاب ودرجاتهم،
    ويُرجع قاموسًا بأسماء الطلاب ومتوسط درجاتهم.
    """
    averages: dict[str, float] = {}
    for student, scores in grades.items():
        averages[student] = sum(scores) / len(scores)
    return averages

student_grades: dict[str, list[int]] = {
    "محمد": [85, 90, 78],
    "فاطمة": [92, 88, 95],
}

result: dict[str, float] = summarize_students(student_grades)
print(result)
# {'محمد': 84.33333333333333, 'فاطمة': 91.66666666666667}
```

🔄 مقارنة داخل Python: هل أكتب Type Hints لكل متغير؟

لاحظ أننا لم نكتب `student: str` و `scores: list[int]` داخل الحلقة `for student, scores in grades.items():`، وهذا مقصود وصحيح تمامًا. فعندما يكون نوع المتغير واضحًا من السياق (هنا، واضح من نوع `grades` نفسه)، فإن أدوات مثل mypy وPylance تستنتج (Infer) النوع تلقائيًا، ولا حاجة لتكرار كتابته. يُسمى هذا **Type Inference**. القاعدة الذهبية: اكتب Type Hints عند توقيعات الدوال والمتغيرات المعقّدة، ودع الأداة تستنتج الباقي بنفسها.

15.4 القيم الاختيارية: Optional و None

من أشهر الأخطاء في أي لغة برمجة محاولة التعامل مع قيمة `None` (أو `null` في لغات أخرى) وكأنها قيمة حقيقية، فينهار البرنامج بخطأ شهير اسمه في Python:

```
AttributeError: 'NoneType' object has no attribute '...'
```

المشكلة بدون Type Hints

```python
def find_user(user_id: int, users: dict[int, str]) -> str:
    return users.get(user_id)  # قد تُرجع None إن لم يوجد المستخدم!
```

هنا قد تُرجع `users.get(user_id)` قيمة `None` إن لم يكن `user_id` موجودًا، بينما يقول التوقيع `-> str`! هذا كذب في التوثيق قد يُضلل كل من يستخدم الدالة لاحقًا.

الحل: `Optional[T]` أو الصيغة الحديثة `T | None`

```python
from typing import Optional

def find_user_old_style(user_id: int, users: dict[int, str]) -> Optional[str]:
    return users.get(user_id)
```

لكن منذ Python 3.10، أصبح بإمكاننا استخدام الرمز `|` (Pipe) مباشرة، وهو الأسلوب الموصى به حاليًا لأنه أقصر وأوضح ولا يتطلب استيرادًا:

```python
def find_user(user_id: int, users: dict[int, str]) -> str | None:
    return users.get(user_id)

result: str | None = find_user(5, {1: "سارة" :2 ,"أحمد"})

if result is not None:
    print(result.upper())  # آمن تمامًا، لأننا تحققنا أولًا
else:
    print("المستخدم غير موجود")
```

تعني `str | None` حرفيًا: "إمّا نص، أو `None`". وهذا بالضبط ما يعنيه `Optional[str]` — فـ `Optional[T]` ليست سوى اختصار تاريخي لـ `T | None`.

⚠ **خطأ شائع جدًا:** لا تخلط بين `Optional[str]` ومعامل اختياري له قيمة افتراضية. هذان مفهومان مختلفان تمامًا!

```python
# معامل اختياري Optional type (له قيمة افتراضية) - لا علاقة له بـ
def greet(name: str = "ضيف") -> str:
    return f"مرحبًا، {name}"

# Optional type - القيمة قد تكون None
def find(key: str) -> str | None:
    ...
```

ويمكن أن يجتمع المفهومان معًا:

```python
def greet(name: str | None = None) -> str:
    if name is None:
        name = "ضيف"
    return f"مرحبًا، {name}"
```

15.5 عندما يكون هناك أكثر من نوع ممكن: Union

ماذا لو كانت دالة تقبل إما عددًا صحيحًا أو نصًا؟ هذا شائع جدًا، كما في حالة قراءة بيانات من مصدر خارجي (JSON، مدخلات المستخدم).

الصيغة الحديثة (موصى بها من 3.10+): `|`

```python
def format_id(value: int | str) -> str:
    """يحوّل معرّفًا (رقمًا أو نصًا) إلى نص منسّق."""
    return f"ID-{value}"

print(format_id(42))      # "ID-42"
print(format_id("A001"))  # "ID-A001"
```

الصيغة التقليدية (لا تزال شائعة في أكواد قديمة): `Union`

```python
from typing import Union

def format_id(value: Union[int, str]) -> str:
    return f"ID-{value}"
```

> **نصيحة الخبير:** بما أننا اعتمدنا Python 3.14 في هذا الكتاب، استخدم دائمًا `|` بدلًا من `Union`، فهي أقصر، ومدمجة في اللغة، ولا تحتاج استيرادًا. ستجد `Union` منتشرة في أكواد ومكتبات أقدم، لذا من المهم أن تتعرّف عليها، لكن تجنّب استخدامها في كودك الجديد.

التعامل الآمن مع Union باستخدام isinstance

عندما يكون لديك متغير بنوع مُركّب، غالبًا ما تحتاج للتحقق من نوعه الفعلي قبل التعامل معه:

```python
def process(value: int | str) -> None:
    if isinstance(value, int):
        print(f"رقم مضاعف: {value * 2}")
    else:
        print(f"نص بأحرف كبيرة: {value.upper()}")
```

تُسمّى هذه التقنية **Type Narrowing** (تضييق النوع): تبدأ بنوع عام (`int | str`)، وبعد التحقق بـ `isinstance`، تتعرّف الأداة (mypy/Pylance) داخل كل فرع من `if` على النوع الدقيق، وتمنحك اقتراحات Autocomplete صحيحة.

15.6 النوع الهارب: Any

أحيانًا، لا تستطيع (أو لا ترغب) في تحديد نوع دقيق. هنا يأتي دور `Any` :

```python
from typing import Any

def process_anything(data: Any) -> Any:
    return data
```

تعني `Any` حرفيًا: «عطّل التحقق من النوع هنا تمامًا». أي قيمة مقبولة، وأي عملية عليها مقبولة، دون أي تحذير من mypy.

⚠ **تحذير خطير:** تُعدّ `Any` بمثابة «ثقب أسود» في نظام الأنواع. كل ما تلمسه `Any` يفقد الحماية. لا تستخدمها إلا في الحالات التالية:
- عند التعامل مع بيانات حقيقية غير معروفة الشكل (مثل نتيجة `json.loads()` من مصدر خارجي).
- أثناء ترحيل كود قديم تدريجيًا إلى Type Hints (كحل مؤقت).

لا تستخدمها أبدًا كحل كسول لتفادي التفكير في النوع الصحيح. فكل استخدام لـ `Any` فرصة ضائعة للاستفادة الحقيقية من Type Hints.

🔄 مقارنة: Any مقابل object

كلاهما يقبل أي قيمة، لكن الفرق بينهما جوهري:

```python
def func_any(x: Any) -> None:
    x.anything_at_all()  # لا تحذير من mypy حتى لو الدالة غير موجودة!

def func_object(x: object) -> None:
    x.anything_at_all()  # mypy يُصدر خطأً! object لا يملك هذه الدالة
```

`object` هو النوع الأب لكل الأنواع في Python (صحيح ونوع آمن)، لكنه لا يسمح باستدعاء دوال خاصة دون التحقق أولًا (`isinstance`)، بينما `Any` يُلغي أي تحقق كليًا. القاعدة: فضّل `object` على `Any` كلما أمكن ذلك.

15.7 الدوال كقيم: Callable

تذكر من الفصلين العاشر والثاني عشر أن الدوال في Python كائنات من الدرجة الأولى (First-Class Citizens) — يمكن تمريرها كمعاملات. فكيف نكتب Type Hint لمعامل يكون دالة؟

```python
from typing import Callable

def apply_operation(
    numbers: list[int],
    operation: Callable[[int], int]
) -> list[int]:
    """يطبّق 'operation' على كل عنصر في القائمة."""
    return [operation(n) for n in numbers]

def double(x: int) -> int:
    return x * 2

def square(x: int) -> int:
    return x * x

print(apply_operation([1, 2, 3], double))  # [2, 4, 6]
print(apply_operation([1, 2, 3], square))  # [1, 4, 9]

# يعمل أيضًا مع lambda
print(apply_operation([1, 2, 3], lambda x: x + 10))  # [11, 12, 13]
```

صيغة `Callable[[Arg1Type, Arg2Type, ...], ReturnType]`:

- داخل القوسين الأولين `[...]`: قائمة بأنواع المعاملات.
- بعد الفاصلة: نوع القيمة المُرجعة.

```python
# دالة بدون معاملات تُرجع نصًا
greeting_provider: Callable[[], str]

# دالة تستقبل int و str وتُرجع bool
validator: Callable[[int, str], bool]

# دالة بأي عدد ونوع من المعاملات (تجنّب هذا إن أمكن، لكنه موجود)
from typing import Callable
any_callable: Callable[..., int]  # ... تعني "أي معاملات"
```

مثال عملي: Decorator موصوف بالكامل بـ Type Hints

Python

```python
from typing import Callable
import time
import functools

def measure_time(func: Callable[..., float]) -> Callable[..., float]:
    @functools.wraps(func)
    def wrapper(*args: int, **kwargs: int) -> float:
        start = time.perf_counter()
        result = func(*args, **kwargs)
        elapsed = time.perf_counter() - start
        print(f"استغرقت {func.__name__}: {elapsed:.4f} ثانية")
        return result
    return wrapper

@measure_time
def slow_calculation(n: int) -> float:
    return sum(i ** 2 for i in range(n)) / n

slow_calculation(1_000_000)
```

نصيحة الخبير: كتابة Type Hints دقيقة للـ Decorators (خصوصًا مع `*args` و`**kwargs`) من أصعب الجوانب في نظام الأنواع. في المشاريع الكبيرة، تُستخدم تقنيات متقدمة مثل `ParamSpec` و`TypeVar` معًا لحفظ توقيع الدالة الأصلية بدقة. لا داعي للقلق بشأن هذا التعقيد الآن — في أغلب الأحيان، التوقيع البسيط كافٍ تمامًا لمشاريعك.

15.8 القيم الثابتة المحدودة: Literal

أحيانًا، لا يكفي القول «هذا نص» — بل تريد التصريح بأن هذا النص يجب أن يكون بالضبط واحدًا من قيم محددة سلفًا. هنا يأتي دور `Literal`:

```python
from typing import Literal

def set_status(status: Literal["active", "inactive", "pending"]) -> None:
    print(f"الحالة الجديدة: {status}")

set_status("active")    # صحيح
set_status("pending")   # صحيح
set_status("deleted")   # ❌ mypy هذا يرفض! "deleted" ليست من القيم المسموحة
```

هذا أقوى بكثير من استخدام `str` العادي، لأن `str` تقبل أي نص على الإطلاق، بينما `Literal` تحصر الخيارات بدقة. يُستخدم غالبًا مع الثوابت المعروفة مسبقًا (حالات، أوضاع تشغيل، اتجاهات...).

```python
Direction = Literal["up", "down", "left", "right"]

def move(direction: Direction, steps: int = 1) -> tuple[int, int]:
    deltas: dict[Direction, tuple[int, int]] = {
        "up": (0, 1),
        "down": (0, -1),
        "left": (-1, 0),
        "right": (1, 0),
    }
    dx, dy = deltas[direction]
    return (dx * steps, dy * steps)

print(move("up", 3))  # (0, 3)
```

لاحظ أننا عرّفنا `Direction` كاسم مستعار (Type Alias) للنوع المركّب، لتجنّب تكرار كتابة `Literal["up", "down", "left", "right"]` في كل مكان.

نصيحة الخبير: يجعل تعريف أسماء مستعارة للأنواع المعقّدة (Type Aliases) كودك أنظف بكثير:

```python
UserId = int
UserName = str
UserDatabase = dict[UserId, UserName]

def get_user(db: UserDatabase, uid: UserId) -> UserName | None:
    return db.get(uid)
```

ومنذ Python 3.12، توجد صيغة مخصصة وأوضح لهذا الغرض باستخدام الكلمة المفتاحية `type`:

```python
type UserDatabase = dict[int, str]
```

15.9 القواميس المُهيكَلة: TypedDict

تذكّر القواميس من الفصل التاسع: مرنة جدًا، لكن مرونتها قد تنطوي على خطر. فماذا لو أردت قاموسًا يمثّل «بطاقة موظف» بحقول محددة ومعروفة مسبقًا (`name`، `age`، `salary`)، دون اللجوء إلى فئة (Class) كاملة لهذا الغرض البسيط؟

Python

```python
from typing import TypedDict

class Employee(TypedDict):
    name: str
    age: int
    salary: float

def print_employee(emp: Employee) -> None:
    print(f"{emp['name']} - {emp['age']} راتب - سنة {emp['salary']} دولار")

employee1: Employee = {
    "name": "ياسر",
    "age": 35,
    "salary": 5000.0,
}

print_employee(employee1)

# استخدام خاطئ يكتشفه mypy:
bad_employee: Employee = {
    "name": "سلمى",
    "age": "يجب أن يكون خطأ! ❌ #  ,"ثلاثون" int وليس str
    "salary": 4500.0,
}
```

لاحظ أن `Employee` تبدو وكأنها فئة عادية تَرث من `TypedDict`، لكنها لا تُنشئ كائنات فعلية — عند التشغيل، `employee1` قاموس عادي تمامًا (`dict`). كل الفائدة هنا مخصصة فقط لأدوات التحقق الساكن مثل mypy.

🔄 مقارنة: متى أستخدم TypedDict ومتى أستخدم dataclass؟

المعيار	TypedDict	dataclass@
النوع الفعلي وقت التشغيل	`dict` عادي	كائن (Object) حقيقي
الوصول للحقول	`emp["name"]`	`emp.name`
يحتاج JSON مباشرة (مثل API)؟	ممتاز، لأنه `dict` بالأصل	يحتاج تحويلًا يدويًا
يحتاج دوالًا/methods إضافية؟	لا يدعم ذلك	نعم، يدعم تمامًا
الأداء	أخف قليلًا	قريب جدًا

القاعدة العملية: إذا كانت بياناتك ستُحوَّل مباشرة من/إلى JSON (كما سنرى كثيرًا في الفصل القادم ولاحقًا مع FastAPI)، فـ `TypedDict` مناسب جدًا. أما إذا كنت تبني نموذج بيانات (Domain Model) حقيقي بسلوك، فاستخدم `@dataclass` أو فئة كاملة.

15.10 البرمجة العامة (Generics) باستخدام TypeVar

هذا من أقوى مفاهيم Type Hints، ويُستخدم لكتابة دوال أو فئات تعمل مع أي نوع، مع الحفاظ على ترابط الأنواع بين المدخلات والمخرجات.

المشكلة بدون Generics

```python
def get_first_element(items: list) -> Any:
    return items[0]
```

المشكلة: لا نعرف نوع العنصر الأول. فإن استدعيناها بقائمة أعداد صحيحة، يجب أن تعرف الأداة أن الناتج `int`، وإن استدعيناها بقائمة نصوص، يجب أن تعرف أن الناتج `str`. أما `Any` فتُفقدنا كل هذا الترابط.

الحل: TypeVar

```python
from typing import TypeVar

T = TypeVar("T")

def get_first_element(items: list[T]) -> T:
    return items[0]

numbers: list[int] = [10, 20, 30]
first_number = get_first_element(numbers)  # mypy يستنتج: int

words: list[str] = ["قلم" ,"كتاب"]
first_word = get_first_element(words)  # mypy يستنتج: str
```

هنا `T` ليست نوعًا محددًا، بل متغير نوع (Type Variable) — «أيًا كان النوع الذي أعطيتني إياه، سأُرجع النوع نفسه». يُسمى هذا **Generic Programming** (البرمجة العامة).

صيغة Python 3.12+ الحديثة (أبسط بكثير)

منذ Python 3.12، يمكن كتابة Generics مباشرة دون استيراد `TypeVar` صراحة، باستخدام صيغة جديدة أنظف:

```python
def get_first_element[T](items: list[T]) -> T:
    return items[0]
```

لاحظ الأقواس المربعة `[T]` مباشرة بعد اسم الدالة — هذا كل ما تحتاجه! هذه الصيغة أبسط وأقصر، وهي الموصى بها في Python 3.14.

Generics مع الفئات (Classes)

يمكن أيضًا جعل فئة كاملة عامة (Generic Class). مثال: صندوق تخزين (Box) يحتفظ بعنصر واحد من أي نوع:

```python
class Box[T]:
    """صندوق يحتفظ بعنصر واحد من أي نوع."""

    def __init__(self, content: T) -> None:
        self.content = content

    def get(self) -> T:
        return self.content

    def __repr__(self) -> str:
        return f"Box({self.content!r})"

int_box = Box(42)           # mypy يستنتج: Box[int]
str_box = Box("مرحبًا")     # mypy يستنتج: Box[str]

print(int_box.get())  # 42
print(str_box.get())  # مرحبًا

value: int = int_box.get()  # صحيح، mypy يعرف أن الناتج int
```

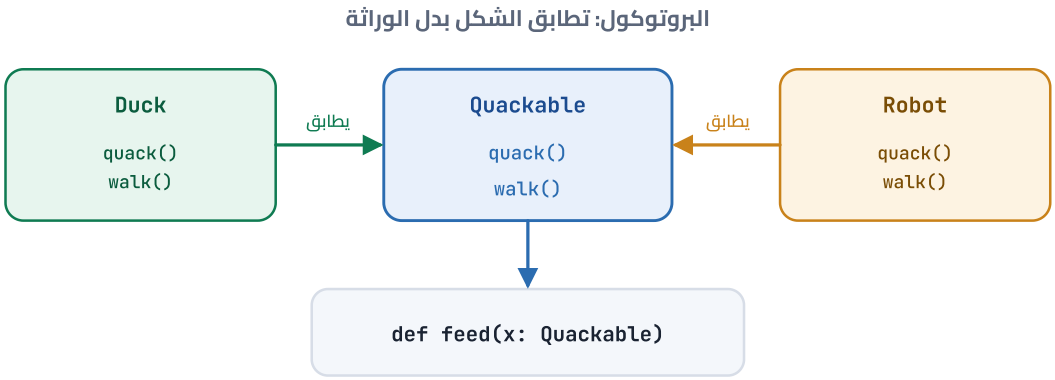
مثال عملي أكبر: مخزن بيانات عام (Generic Repository)

Python

```python
class Repository[T]:
    """مخزن بسيط يدير مجموعة من العناصر من نوع T."""

    def __init__(self) -> None:
        self._items: list[T] = []

    def add(self, item: T) -> None:
        self._items.append(item)

    def get_all(self) -> list[T]:
        return self._items.copy()

    def find(self, predicate: Callable[[T], bool]) -> T | None:
        for item in self._items:
            if predicate(item):
                return item
        return None

from typing import Callable
from dataclasses import dataclass

@dataclass
class Product:
    name: str
    price: float

product_repo = Repository[Product]()
product_repo.add(Product("لابتوب", 1200.0))
product_repo.add(Product("هاتف", 800.0))

cheap_product = product_repo.find(lambda p: p.price < 1000)
print(cheap_product)  # Product(name='هاتف', price=800.0)
```

نصيحة الخبير: قد تبدو Generics معقّدة في البداية، لكن استخدامها شائع جدًا في مكتبات حقيقية تستخدمها يوميًا: `list[int]` نفسها استخدام لـ Generic! فـ `list` فئة عامة تقبل أي نوع `T`. كلما فهمت Generics أكثر، فهمت لماذا تعمل هذه الأشياء بالطريقة التي تعمل بها.

15.11 البروتوكولات: Protocol (Duck Typing المُهيكَل)



شكل 15.1 — البروتوكول: تطابق الشكل بدل الوراثة

تذكّر من الفصل الرابع عشر مفهوم Duck Typing: «إن كان يمشي كبطة ويصدر صوت بطة، فهو بطة» — أي أن Python لا تهتم بنوع الكائن الفعلي، بل بالقدرات (الدوال) التي يمتلكها.

لكن كيف نكتب Type Hint يُعبّر عن هذه الفكرة؟ أي: "أريد أي كائن يملك دالة `.draw()`"، بصرف النظر عن فئته الأصلية؟ هنا يأتي دور `Protocol`:

```python
from typing import Protocol

class Drawable(Protocol):
    def draw(self) -> str:
        ...

class Circle:
    def draw(self) -> str:
        return "رسم دائرة ⭕"

class Square:
    def draw(self) -> str:
        return "رسم مربع ⬜"

def render(shape: Drawable) -> None:
    print(shape.draw())

render(Circle())  # صحيح! Circle تملك draw()
render(Square())  # صحيح! Square تملك draw() أيضًا
```

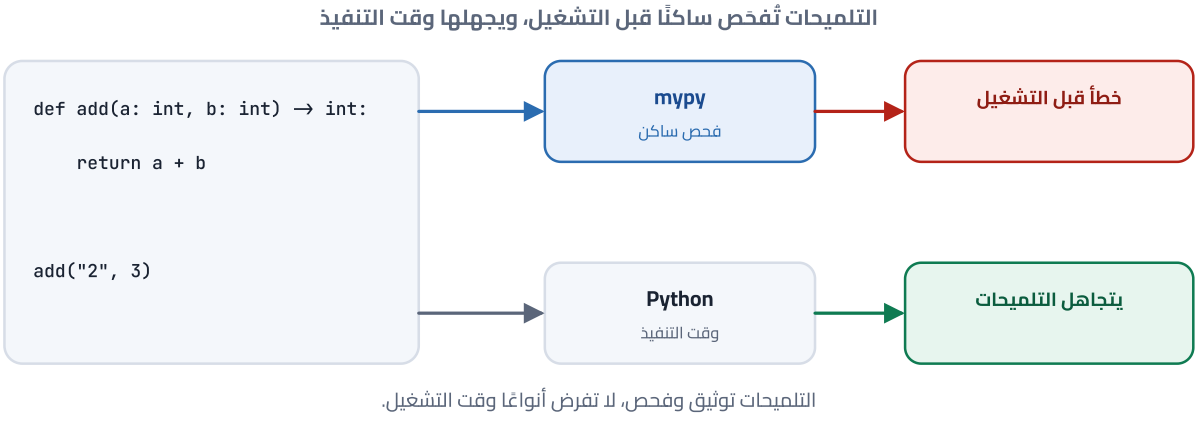
لاحظ أن `Circle` و`Square` لا تَرِثان من `Drawable` بأي شكل من الأشكال! يُسمى هذا **Structural Typing** (تنميط هيكلي): «إن كان الكائن يملك الدوال المطلوبة بنفس التوقيع، فهو يحقق البروتوكول تلقائيًا»، بخلاف الوراثة التقليدية (Nominal Typing) التي تتطلب إعلانًا صريحًا بالانتماء إلى فئة معينة.

🔄 مقارنة: Protocol مقابل ABC (Abstract Base Class)

المعيار	Protocol	ABC (الفصل 14)
هل تحتاج وراثة صريحة؟	لا	نعم، إلزامية
النمط	هيكلي (Structural) — «لو شكلك مطابق، تُقبل»	اسمي (Nominal) — «يجب أن تُعلن الانتماء»
الاستخدام الأنسب	عند التعامل مع فئات لا تتحكم بها (من مكتبات خارجية)	عند تصميم هرمية فئات خاصة بك من البداية
هل يُفرض فعليًا وقت التشغيل؟	لا (إلا مع `@runtime_checkable`)	نعم، عبر `TypeError` عند `@abstractmethod`

> **نصيحة الخبير:** استخدم `Protocol` عندما تكتب دالة تقبل «أي شيء يستطيع فعل كذا»، خصوصًا عند التعامل مع فئات من مكتبات لا تملك التحكم بها (فلا يمكنك جعلها تَرِث من ABC خاصة بك). استخدم `ABC` عندما تُصمم هرمية فئات خاصة بك من الصفر وتريد فرض تنفيذ methods معينة.

15.12 أداة mypy: التحقق الساكن من الأنواع



شكل 15.2 — الفحص الساكن مقابل وقت التشغيل

وصلنا الآن إلى الجزء الأهم عمليًا: كيف نجعل كل هذه الـ Type Hints مفيدة فعليًا، لا مجرد توثيق جميل يُقرأ ويُتجاهل؟

ما هي mypy؟

mypy أداة مستقلة (ليست جزءًا من Python نفسها) تقرأ ملفات كودك دون تشغيلها فعليًا، وتحلل كل Type Hints المكتوبة، وتبحث عن أي تضارب أو خلل منطقي في الأنواع، ثم تُصدر تقريرًا نصيًا بكل المشاكل التي رصدتها.

يُسمى هذا **Static Type Checking** (التحقق الساكن من الأنواع) — «ساكن» لأنه يحدث قبل تشغيل البرنامج، بخلاف الأخطاء التي لا تظهر إلا أثناء التشغيل (Runtime).

تثبيت mypy

باستخدام `uv` (كما اعتدنا طوال الكتاب):

Terminal

```
uv add --dev mypy
```

نستخدم `--dev` لأن mypy أداة تطوير (Development Tool)، وليست جزءًا من المكتبات التي يحتاجها برنامجك فعليًا وقت التشغيل عند المستخدم النهائي.

أول تجربة مع mypy

أنشئ ملفًا باسم `example.py` :

Python

```python
def add(a: int, b: int) -> int:
    return a + b

result = add("مرحبًا", "!")
print(result)
```

ثم شغّل mypy على هذا الملف:

Terminal

```
uv run mypy example.py
```

النتيجة:

```
example.py:5: error: Argument 1 to "add" has incompatible type "str"; expected "int"  [arg-type]
example.py:5: error: Argument 2 to "add" has incompatible type "str"; expected "int"  [arg-type]
Found 2 errors in 1 file (checked 1 source file)
```

لاحظ شيئًا مذهلًا: لم نُشغّل البرنامج فعليًا! قرأ mypy الكود فقط، وأخبرنا بوجود خطأ قبل أي تشغيل. هذا الفرق جوهري — في مشروع كبير بآلاف الأسطر، يعني هذا اكتشاف أخطاء قد لا تظهر إلا بعد أشهر في الإنتاج، خلال ثوانٍ معدودة من كتابة الكود.

> **نصيحة الخبير:** إضافة Pylance في VS Code (التي ثبّتناها في الفصل الثاني) تُجري فعليًا تحليلًا مماثلًا لحظيًا أثناء الكتابة، وتُظهر الخط الأحمر المتعرج تحت الكود الخاطئ مباشرة. أما mypy، فيُستخدم عادة بالإضافة إلى ذلك كـ«بوابة فحص نهائية» ضمن CI/CD (عمليات تحقق آلية قبل دمج الكود)، لضمان عدم تجاهل أحد لتحذيرات المحرر.

أمثلة أخطاء حقيقية تكتشفها mypy

مثال 1: نسيان حالة None

Python

```python
def find_user(user_id: int, users: dict[int, str]) -> str | None:
    return users.get(user_id)

user = find_user(5, {1: "أحمد"})
print(user.upper())  # ✗ خطأ محتمل!
```

Terminal

```
uv run mypy example.py
```

```
error: Item "None" of "str | None" has no attribute "upper"  [union-attr]
```

يُخبرنا mypy: "`user` قد يكون `None`، و`None` لا يملك دالة `.upper()`". هذا بالضبط نوع الخطأ الذي كان سيظهر فقط عند استخدام `user_id` غير موجود، ربما بعد نشر التطبيق!

التصحيح:

Python

```python
user = find_user(5, {1: "أحمد"})
if user is not None:
    print(user.upper())
else:
    print("المستخدم غير موجود")
```

مثال 2: خطأ في عدد المعاملات

```python
def greet(name: str, greeting: str = "مرحبًا") -> str:
    return f"{greeting}، {name}!"

print(greet())  # ❌ نسينا معاملًا إلزاميًا
```

```
error: Missing positional argument "name" in call to "greet"  [call-arg]
```

مثال 3: استخدام خاطئ لنوع الإرجاع

```python
def get_age() -> int:
    return "عشرون"  # ❌ نص بدل عدد صحيح
```

```
error: Incompatible return value type (got "str", expected "int")  [return-value]
```

مثال 4: خطأ في TypedDict

```python
from typing import TypedDict

class Point(TypedDict):
    x: int
    y: int

p: Point = {"x": 1}  # ❌ ناقص حقل y
```

```
error: Missing key "y" for TypedDict "Point"  [typeddict-item]
```

⚠ **تحذير:** لا يستطيع mypy اكتشاف كل الأخطاء المنطقية، فهو يتحقق فقط من تطابق الأنواع وفق ما كتبته أنت من Type Hints. إن كتبت `Any` في كل مكان، فلن يجد mypy أي شيء يُبلّغ عنه! جودة فحص mypy تتناسب طرديًا مع دقة Type Hints التي تكتبها.

تشغيل mypy على مشروع كامل

```
Terminal
```

```
# فحص ملف واحد
uv run mypy example.py

# فحص مجلد كامل (كل ملفات .py بداخله)
uv run mypy src/

# فحص المشروع كله من المجلد الرئيسي
uv run mypy .
```

إعدادات mypy: ملف pyproject.toml

بدلًا من تمرير الخيارات عبر سطر الأوامر في كل مرة، يُفضَّل ضبط mypy مرة واحدة ضمن ملف `pyproject.toml` (نفس الملف الذي يدير مشروعك مع uv):

```
TOML
```

```toml
[tool.mypy]
python_version = "3.14"
disallow_untyped_defs = true      # يرفض الدوال بدون Type Hints إطلاقًا
disallow_incomplete_defs = true   # يرفض الدوال ذات Type Hints جزئية فقط
warn_return_any = true            # يُحذّر عند إرجاع محدد Any من دالة ذات نوع إرجاع محدد
warn_unused_ignores = true        # يُحذّر عند استخدام # type: ignore بلا داعٍ
no_implicit_optional = true       # يمنع افتراض Optional تلقائيًا
strict_equality = true            # يتحقق من منطقية عمليات المقارنة==
```

أو، بدلًا من ضبط كل خيار يدويًا، يمكنك تفعيل الوضع الصارم دفعة واحدة:

```
TOML
```

```toml
[tool.mypy]
python_version = "3.14"
strict = true
```

يُفعّل `strict = true` أكثر من عشرة خيارات تحقق صارمة دفعة واحدة. هذا هو الخيار الموصى به في المشاريع الجديدة التي تطمح لأعلى مستوى من الأمان.

> **نصيحة الخبير — التبني التدريجي:** إن كنت تعمل على مشروع كبير موجود مسبقًا بدون Type Hints، لا تُفعّل `strict = true` دفعة واحدة — فستُغرقك في مئات الأخطاء! ابدأ بخيارات بسيطة، وأضف الأنواع تدريجيًا ملفًا بملف، ثم فعّل الخيارات الصارمة تباعًا. صُممت Type Hints في Python أصلًا لتكون تدريجية (Gradual) لهذا السبب بالذات.

تجاهل سطر معين عمدًا: `# type: ignore`

أحيانًا تعرف أن كودًا معينًا صحيح رغم اعتراض mypy عليه (مثلًا عند استخدام مكتبة خارجية بدون Type Hints جيدة). يمكنك تجاهل سطر محدد:

```python
import some_untyped_library  # type: ignore[import-untyped]

result = some_untyped_library.do_something()
```

> ⚠️ **تحذير:** لا تستخدم `# type: ignore` كحل سريع لتجاوز خطأ لم تفهمه! اقرأ رسالة الخطأ أولًا وحاول فهمها وإصلاحها. استخدم `# type: ignore` فقط عندما تكون متأكدًا تمامًا من أن mypy مخطئ في هذه الحالة تحديدًا (وهذا نادر نسبيًا، ويحدث غالبًا بسبب نقص معلومات في مكتبة خارجية، لا بسبب خطأ منطقي حقيقي).

15.13 أفضل الممارسات العامة

1. ابدأ من توقيعات الدوال، لا من كل متغير

```python
# ✅ جيد: التركيز على توقيع الدالة (المعاملات والإرجاع)
def calculate_total(prices: list[float], tax_rate: float) -> float:
    subtotal = sum(prices)  # mypy يستنتج أن subtotal من نوع float
    return subtotal * (1 + tax_rate)
```

لا حاجة لكتابة `subtotal: float = sum(prices)` — الاستنتاج التلقائي (Type Inference) كافٍ هنا.

2. تجنّب `Any` إلا للضرورة القصوى

كل استخدام لـ `Any` نقطة عمياء في نظام الحماية لديك.

3. استخدم `Optional` / `| None` بصراحة بدل الاعتماد على «التفاهم الضمني»

```python
# ❌ غامض: هل يمكن أن يكون None؟ لا نعرف
def find(key: str) -> str:
    ...

# ✅ واضح: نعرف بالضبط أن النتيجة قد تكون None
def find(key: str) -> str | None:
    ...
```

4. فعّل mypy منذ بداية المشروع (وليس في النهاية)

كلما أضفت Type Hints منذ البداية، أصبحت العملية طبيعية وجزءًا من كتابة الكود، لا عبئًا إضافيًا لاحقًا.

5. استخدم `reveal_type()` للتشخيص أثناء التطوير

```python
x = [1, 2, 3]
reveal_type(x)  # أداة تشخيصية خاصة بـ mypy فقط
```

```
Terminal
uv run mypy example.py
```

```
example.py:2: note: Revealed type is "builtins.list[builtins.int]"
```

هذه دالة وهمية خاصة بـ mypy فقط (لا تُستخدم فعليًا في الكود النهائي)، تُخبرك بالضبط عن النوع الذي استنتجته الأداة — مفيدة جدًا عند تصحيح مشاكل معقّدة.

جدول مرجعي شامل: كل الأنواع التي تعلمناها

الصيغة	المعنى	مثال
`int` , `float` , `str` , `bool`	أنواع أساسية	`age: int = 25`
`None`	لا شيء	`→ None`
`list[T]`	قائمة من نوع T	`list[int]`
`tuple[T1, T2]`	صندوق بأنواع محددة	`tuple[str, int]`
`tuple[T, ...]`	صندوق بعدد متغير من نفس النوع	`tuple[int, ...]`

الصيغة	المعنى	مثال
`dict[K, V]`	قاموس	`dict[str, int]`
`set[T]`	مجموعة	`set[str]`
`T | None`	اختياري (قد يكون None)	—
`T1 | T2`	إمّا هذا أو ذاك	—
`Any`	أي نوع (بدون حماية)	`Any`
`Callable[[Args], Ret]`	دالة	`Callable[[int], str]`
`Literal[...]`	قيم محددة فقط	`Literal["a", "b"]`
`TypedDict`	قاموس بحقول معروفة	`class P(TypedDict): x: int`
`[T]` / `TypeVar`	نوع عام (Generic)	`def f[T](x: T) → T`
`Protocol`	بروتوكول هيكلي	`class Drawable(Protocol): ...`

15.14 مشروع تطبيقي: نظام إدارة مهام مُنقَّط بالكامل

لنطبّق كل ما تعلمناه في مشروع متكامل: نظام بسيط لإدارة المهام (Task Manager)، مع Type Hints كاملة ودقيقة، يجتاز فحص mypy في الوضع الصارم (`strict = true`) دون أي خطأ.

المتطلبات

- كل مهمة لها: عنوان، أولوية (محدودة بقيم معينة فقط)، وحالة (معلّقة/قيد التنفيذ/منجزة).
- نظام يسمح بإضافة، البحث، تصفية، وتحديث حالة المهام.
- دعم Generic لأي نوع من «المستودعات» (Repositories) القابلة لإعادة الاستخدام.
- استخدام Protocol لتعريف «أي شيء قابل للعرض» (Displayable).

```python
"""
نظام إدارة المهام - مثال تطبيقي شامل على Type Hints
"""
from __future__ import annotations

from dataclasses import dataclass, field
```

```python
from datetime import date
from typing import Literal, Protocol, Callable

# ---------------------------------------------------------
# 1. تعريف الأنواع الأساسية باستخدام Literal و Type Alias
# ---------------------------------------------------------
type Priority = Literal["عالية", "متوسطة", "منخفضة"]
type Status = Literal["معلّقة", "قيد التنفيذ", "منجزة"]

# ---------------------------------------------------------
# 2. Protocol لـ "قابل للعرض" أي شيء
# ---------------------------------------------------------
class Displayable(Protocol):
    def display(self) -> str:
        ...

# ---------------------------------------------------------
# 3. فئة المهمة (Task) باستخدام dataclass
# ---------------------------------------------------------
@dataclass
class Task:
    title: str
    priority: Priority = "متوسطة"
    status: Status = "معلّقة"
    due_date: date | None = None
    tags: list[str] = field(default_factory=list)

    def display(self) -> str:
        due_info = f" (موعد التسليم: {self.due_date})" if self.due_date else ""
        tags_info = f" [{', '.join(self.tags)}]" if self.tags else ""
        return f"📋 {self.title} - {self.priority} - {self.status}{due_info}{tags_info}"

    def mark_as(self, new_status: Status) -> None:
        self.status = new_status

# ---------------------------------------------------------
# 4. مستودع عام (Generic Repository) لإدارة أي نوع من العناصر
# ---------------------------------------------------------
class Repository[T]:
    """مستودع عام يدير مجموعة من العناصر من أي نوع."""

    def __init__(self) -> None:
        self._items: list[T] = []

    def add(self, item: T) -> None:
        self._items.append(item)

    def remove(self, item: T) -> None:
        self._items.remove(item)

    def all(self) -> list[T]:
```

```
        return self._items.copy()

    def filter(self, predicate: Callable[[T], bool]) -> list[T]:
        return [item for item in self._items if predicate(item)]

    def find_first(self, predicate: Callable[[T], bool]) -> T | None:
        for item in self._items:
            if predicate(item):
                return item
        return None

    def count(self) -> int:
        return len(self._items)

# ---------------------------------------------------------
# 5. مدير المهام (TaskManager) - يستخدم Repository
# ---------------------------------------------------------
class TaskManager:
    def __init__(self) -> None:
        self._repo: Repository[Task] = Repository()

    def add_task(
        self,
        title: str,
        priority: Priority = "متوسطة",
        due_date: date | None = None,
        tags: list[str] | None = None,
    ) -> Task:
        task = Task(
            title=title,
            priority=priority,
            due_date=due_date,
            tags=tags if tags is not None else [],
        )
        self._repo.add(task)
        return task

    def get_by_status(self, status: Status) -> list[Task]:
        return self._repo.filter(lambda t: t.status == status)

    def get_by_priority(self, priority: Priority) -> list[Task]:
        return self._repo.filter(lambda t: t.priority == priority)

    def find_task_by_title(self, title: str) -> Task | None:
        return self._repo.find_first(lambda t: t.title == title)

    def complete_task(self, title: str) -> bool:
        task = self.find_task_by_title(title)
        if task is None:
            return False
        task.mark_as("منجزة")
```

```
        return True

    def summary(self) -> dict[Status, int]:
        statuses: list[Status] = ["معلّقة", "قيد التنفيذ", "منجزة"]
        return {
            status: len(self.get_by_status(status))
            for status in statuses
        }

    def display_all(self) -> None:
        if self._repo.count() == 0:
            print("لا توجد مهام بعد.")
            return
        for task in self._repo.all():
            print(task.display())

# ---------------------------------------------------------
# 6. دالة عامة تعمل مع أي كائن Displayable (Protocol)
# ---------------------------------------------------------
def print_report(items: list[Displayable], title: str = "تقرير") -> None:
    print(f"═══ {title} ═══")
    for item in items:
        print(item.display())
    print(f"عدد العناصر: {len(items)}")

# ---------------------------------------------------------
# 7. برنامج تجريبي
# ---------------------------------------------------------
def main() -> None:
    manager = TaskManager()

    manager.add_task(
        "كتابة تقرير شهري",
        priority="عالية",
        due_date=date(2025, 6, 30),
        tags=["عاجل", "عمل"],
    )
    manager.add_task("قراءة كتاب Python", priority="منخفضة")
    manager.add_task(
        "مراجعة الكود مع الفريق",
        priority="متوسطة",
        tags=["فريق"],
    )

    print("كل المهام:")
    manager.display_all()
    print()

    manager.complete_task("قراءة كتاب Python")

    print("المهام العالية الأولوية:")
```

```
    for task in manager.get_by_priority("عالية"):
        print(task.display())
    print()

    print("ملخص الحالات:")
    for status, count in manager.summary().items():
        print(f"  {status}: {count}")
    print()

    high_priority_tasks: list[Displayable] = list(manager.get_by_priority("عالية"))
    print_report(high_priority_tasks, title="تقرير المهام العاجلة")

if __name__ == "__main__":
    main()
```

فحص المشروع بـ mypy

```
Terminal
```

```
uv run mypy task_manager.py --strict
```

الناتج المتوقع (إن كُتب الكود بشكل صحيح تمامًا):

```
Success: no issues found in 1 source file
```

> **نصيحة الخبير:** لاحظ استخدام `from __future__ import annotations` في بداية الملف. هذا السطر (مفيد خصوصًا في إصدارات أقدم من Python) يجعل كل Type Hints تُعامَل كنصوص (Strings) أثناء التشغيل، فلا تُقيَّم فعليًا، مما يُسرّع بدء تشغيل البرنامج قليلًا ويحل مشاكل «التوقيعات الدائرية» (عندما تحتاج فئة أن تُشير لنفسها أو لفئة لاحقة في الكود). في Python 3.14 أصبح هذا السلوك قريبًا من الافتراضي في بعض الحالات، لكن إضافته صراحة لا تزال ممارسة جيدة ومتوافقة مع إصدارات أخرى.

15.15 تمارين

🟢 مستوى مبتدئ

تمرين 1: أضف Type Hints كاملة لهذه الدالة (المعاملات وقيمة الإرجاع):

```python
def calculate_area(width, height):
    return width * height
```

تمرين 2: اكتب دالة `get_grade(score: int) → str` تُرجع تقدير الطالب («ممتاز»، «جيد»، «راسب») حسب الدرجة. استخدم `Literal` لنوع الإرجاع.

تمرين 3: ما الفرق بين `Optional[int]` و `int | None`؟ وأيهما يُفضَّل استخدامه في Python 3.14؟

✅ الحل النموذجي

```python
# تمرين 1
def calculate_area(width: float, height: float) → float:
    return width * height

# تمرين 2
from typing import Literal

Grade = Literal["راسب" ,"جيد" ,"ممتاز"]

def get_grade(score: int) → Grade:
    if score ⩾ 90:
        return "ممتاز"
    elif score ⩾ 60:
        return "جيد"
    else:
        return "راسب"

# تمرين 3
# لا فرق في المعنى: كلاهما يعني "قد يكون int أو None".
# Optional[int] هي الصيغة القديمة (تحتاج: from typing import Optional).
# int | None هي الصيغة الحديثة الموصى بها في Python 3.10+ (لا تحتاج استيراد).
# في كتابنا نستخدم دائمًا: int | None
```

🟡 مستوى متوسط

تمرين 4: صحّح الأخطاء التي سيكتشفها mypy في هذا الكود:

```python
def find_product(product_id: int, catalog: dict[int, str]) -> str:
    return catalog.get(product_id)

def get_discount(price: float, percent: int) -> float:
    return price - (price * percent)

products = {1: "هاتف" :2 ,"لابتوب"}
name = find_product(3, products)
print(name.upper())
```

تمرين 5: اكتب فئة Generic باسم `Stack[T]` (مكدّس) تدعم:

- `push(item: T) -> None`
- `pop() -> T | None`
- `peek() -> T | None`
- `is_empty() -> bool`

ثم ثبّت mypy وتأكد أن الكود يجتاز الفحص بدون أخطاء.

تمرين 6: عرّف `Protocol` باسم `Comparable` يتطلب دالة `__lt__(self, other) -> bool`، ثم اكتب دالة `find_max[T: Comparable](items: list[T]) -> T` تُرجع أكبر عنصر في قائمة (دون استخدام `max()` الجاهزة).

✅ الحل النموذجي

Python

```python
# تمرين 4 - الكود المصحح
def find_product(product_id: int, catalog: dict[int, str]) -> str | None:
    return catalog.get(product_id)  # قد تكون None فعلاً

def get_discount(price: float, percent: int) -> float:
    return price - (price * (percent / 100))  # كانت تُطرح قيمة كبيرة جدًا خطأً منطقيًا

products: dict[int, str] = {1: "هاتف" ,2 :"لابتوب"}
name = find_product(3, products)

if name is not None:
    print(name.upper())
else:
    print("المنتج غير موجود")

# تمرين 5
class Stack[T]:
    def __init__(self) -> None:
        self._items: list[T] = []

    def push(self, item: T) -> None:
        self._items.append(item)

    def pop(self) -> T | None:
        if self.is_empty():
            return None
        return self._items.pop()

    def peek(self) -> T | None:
        if self.is_empty():
            return None
        return self._items[-1]

    def is_empty(self) -> bool:
        return len(self._items) == 0

stack: Stack[int] = Stack()
stack.push(1)
stack.push(2)
print(stack.pop())   # 2
print(stack.peek())  # 1

# تمرين 6
from typing import Protocol
```

```python
class Comparable(Protocol):
    def __lt__(self, other: object) -> bool:
        ...

def find_max[T: Comparable](items: list[T]) -> T:
    if not items:
        raise ValueError("القائمة فارغة")
    current_max = items[0]
    for item in items[1:]:
        if current_max < item:
            current_max = item
    return current_max

print(find_max([3, 7, 2, 9, 4]))        # 9
print(find_max(["أسد", "قطة", "بطة"]))  # قطة (حسب الترتيب الأبجدي)
```

ملاحظة: تعني `[T: Comparable]` أن «T أي نوع يُحقّق بروتوكول Comparable» — يُسمى هذا **Bounded TypeVar** (متغير نوع مقيّد).

🔴 تحدي

تمرين 7: صمّم نظام تخزين مؤقت (Cache) عام بالكامل:

1. فئة `Cache[K, V]` تخزّن أزواج مفتاح-قيمة من أي نوع.
2. دالة `get(key: K) -> V | None`.
3. دالة `set(key: K, value: V) -> None`.
4. دالة `get_or_compute(key: K, compute_fn: Callable[[], V]) -> V`: تُرجع القيمة إن وُجدت، وإلا تحسبها عبر `compute_fn`، تخزّنها، ثم تُرجعها.
5. استخدم `TypedDict` لتعريف إحصائيات الـ Cache (`hits: int`, `misses: int`) وأضف دالة `stats() -> CacheStats`.
6. شغّل mypy بوضع `--strict` وتأكد أن الكود خالٍ تمامًا من الأخطاء.

✅ الحل النموذجي

Python

```python
from typing import TypedDict, Callable

class CacheStats(TypedDict):
    hits: int
    misses: int

class Cache[K, V]:
    def __init__(self) -> None:
        self._data: dict[K, V] = {}
        self._hits: int = 0
        self._misses: int = 0

    def get(self, key: K) -> V | None:
        if key in self._data:
            self._hits += 1
            return self._data[key]
        self._misses += 1
        return None

    def set(self, key: K, value: V) -> None:
        self._data[key] = value

    def get_or_compute(self, key: K, compute_fn: Callable[[], V]) -> V:
        existing = self.get(key)
        if existing is not None:
            return existing
        new_value = compute_fn()
        self.set(key, new_value)
        return new_value

    def stats(self) -> CacheStats:
        return {"hits": self._hits, "misses": self._misses}

def expensive_computation(n: int) -> int:
    print(f"🔄 جارٍ الحساب الباهظ لـ {n}...")
    return n ** 2

cache: Cache[int, int] = Cache()

result1 = cache.get_or_compute(5, lambda: expensive_computation(5))
print(result1)  # يطبع "جارٍ الحساب..." ثم 25
```

```
result2 = cache.get_or_compute(5, lambda: expensive_computation(5))
print(result2)  # لا يطبع "جارِ الحساب" لأنها مخزّنة! يطبع 25 مباشرة

print(cache.stats())  # {'hits': 1, 'misses': 1}
```

تحقق بـ mypy:

Terminal

```
uv run mypy cache_example.py --strict
```

```
Success: no issues found in 1 source file
```

15.16 اختبر فهمك (Quiz)

❓ السؤال 1: هل تمنع Python تشغيل الكود إن كانت الأنواع غير متطابقة مع Type Hints؟

لا. تتجاهل Python Type Hints تمامًا وقت التشغيل. هي «تلميحات» فقط موجَّهة للقارئ البشري ولأدوات خارجية مثل mypy والمحررات. يجب تشغيل أداة مثل mypy بشكل منفصل لاكتشاف هذه الأخطاء قبل التشغيل.

❓ السؤال 2: ما الفرق بين `list[int]` و `List[int]`؟

لا فرق في المعنى. `List[int]` هي الصيغة القديمة التي تتطلب `from typing import List`، بينما `list[int]` هي الصيغة الحديثة المدمجة في اللغة منذ Python 3.9، ولا تحتاج استيرادًا. نستخدم دائمًا `list[int]` في هذا الكتاب.

❓ السؤال 3: متى يجب استخدام `Any` بدل تحديد نوع دقيق؟

نادرًا جدًا، فقط عند التعامل مع بيانات غير معروفة الشكل فعليًا (كنتيجة لتحليل JSON خارجي)، أو أثناء ترحيل كود قديم تدريجيًا. يجب تجنّبها قدر الإمكان لأنها تُعطّل كل حماية نظام الأنواع على القيمة المعنية.

❓ السؤال 4: ما الفرق بين `Protocol` و `ABC` (Abstract Base Class)؟

تتطلب `ABC` وراثة صريحة (Nominal Typing) — يجب أن تُعلن الفئة أنها تَرث من الفئة المجردة. أما `Protocol` فلا يتطلب أي وراثة؛ يكفي أن تمتلك الفئة الدوال المطلوبة بنفس التوقيع (Structural Typing) — «إن كان شكلك مطابقًا، فأنت تحقق البروتوكول تلقائيًا»، بصرف النظر عن هرمية الوراثة.

❓ السؤال 5: ما وظيفة `TypeVar` أو الصيغة `[T]` في تعريف دالة؟

تسمح بكتابة دوال أو فئات عامة (Generic) تعمل مع أي نوع، مع الحفاظ على «الترابط» بين نوع المدخل ونوع المخرج. مثلًا، تضمن `def first[T](items: list[T]) → T` أن ناتج الدالة هو نفس نوع عناصر القائمة المُمرَّرة، بصرف النظر عن النوع الفعلي.

❓ السؤال 6: لماذا نفضّل `str | None` على استخدام قيمة افتراضية وهمية مثل `""` للدلالة على «لا توجد قيمة»؟

لأن `""` نص صالح فعليًا وقد يكون قيمة حقيقية مقصودة (نص فارغ)، بينما يُعبّر `None` صراحة وبدون لبس عن «غياب القيمة تمامًا». استخدام `str | None` يجعل القصد واضحًا، ويجبرك (عبر mypy) على التحقق من `None` قبل استخدام القيمة، مما يمنع أخطاء شائعة.

15.17 ملخص الفصل (Cheat Sheet)

```Python
```

```python
# ===== الأنواع الأساسية =====
age: int = 25
price: float = 19.99
name: str = "سارة"
active: bool = True

# ===== الهياكل المركّبة =====
numbers: list[int] = [1, 2, 3]
point: tuple[float, float] = (1.0, 2.0)
scores: tuple[int, ...] = (1, 2, 3, 4)
ages: dict[str, int] = {"أحمد": 30}
tags: set[str] = {"a", "b"}

# ===== القيم الاختيارية والمركّبة =====
result: str | None = None          # اختياري (الصيغة الحديثة)
value: int | str = 5               # إمّا هذا أو ذاك

# ===== Any (استخدمها بحذر!) =====
from typing import Any
data: Any = {"whatever": "shape"}

# ===== Callable (الدوال كقيم) =====
from typing import Callable
operation: Callable[[int, int], int]

# ===== Literal (قيم محددة فقط) =====
from typing import Literal
Status = Literal["pending", "done"]

# ===== TypedDict (قاموس مُهيكَل) =====
from typing import TypedDict
class User(TypedDict):
    name: str
    age: int

# ===== Generics (البرمجة العامة) =====
def first[T](items: list[T]) -> T:
    return items[0]

class Box[T]:
    def __init__(self, content: T) -> None:
        self.content = content

# ===== Protocol (duck typing مُهيكَل) =====
from typing import Protocol
class Drawable(Protocol):
    def draw(self) -> str: ...

# ===== mypy =====
# التثبيت:
```

```
#   uv add --dev mypy
# التشغيل:
#   uv run mypy file.py
#   uv run mypy . --strict
# الإعداد في pyproject.toml:
#   [tool.mypy]
#   strict = true
```

المفهوم	الاستخدام السريع
`T | None`	قيمة قد تكون غائبة
`T1 | T2`	أحد نوعين
`Any`	تعطيل التحقق (نادرًا)
`Callable[[Args], Ret]`	دالة كمعامل
`Literal[...]`	قيم ثابتة محددة
`TypedDict`	قاموس بحقول معروفة
`TypeVar` / `[T]`	نوع عام (Generic)
`Protocol`	duck typing منظم
`mypy`	أداة التحقق الساكن

15.18 ماذا بعد؟

لقد أتقنت الآن أداة قوية جدًا تجعل كودك أكثر أمانًا ووضوحًا واحترافية: Type Hints المتقدمة وأداة mypy. من الآن فصاعدًا، سيستفيد كل كود تكتبه في بقية هذا الكتاب من هذا الفهم العميق — ستلاحظ توقيعات الدوال بعين جديدة، وستدرك لماذا اخترنا كتابتها بهذا الشكل بالذات منذ الفصل الثالث.

لكن حتى الآن، كانت كل برامجنا تعيش وتموت داخل الذاكرة فقط — بمجرد إغلاق البرنامج، تختفي كل البيانات. في الفصل القادم، سنتعلم كيف نحفظ البيانات بشكل دائم على القرص الصلب، وكيف نقرأها مرة أخرى: التعامل مع الملفات النصية، تنسيق JSON المنتشر في كل مكان (واجهات برمجية، إعدادات، تبادل بيانات)، ملفات CSV (الجداول)، ومكتبة `pathlib` الحديثة للتعامل الأنيق مع مسارات الملفات والمجلدات.

فصل 16 / 26 · الجزء الرابع

16

الملفات وتنسيقات البيانات

13 دقيقة قراءة | 11 قسمًا | 38 مثالًا | 2,515 كلمة

في هذا الفصل

16.1 لماذا يستحق هذا الفصل اهتمامك الكامل؟
16.2 الملفات النصية: الأساسيات
16.3 `pathlib`: الأسلوب الحديث للتعامل مع المسارات
16.4 JSON: لغة تبادل البيانات العالمية
16.5 CSV: البيانات الجدولية
16.6 أفضل الممارسات
16.7 مشروع تطبيقي: مدير جهات اتصال متكامل
16.8 تمارين
16.9 اختبر فهمك (Quiz)
16.10 ملخص الفصل (Cheat Sheet)
16.11 ماذا بعد؟

16.1 لماذا يستحق هذا الفصل اهتمامك الكامل؟

منذ بداية هذا الكتاب، كل برنامج كتبته يعاني من مشكلة واحدة قاتلة: **فقدان الذاكرة التام**. اكتب قائمة مهام، أضف إليها عشرات العناصر، أغلق البرنامج... وكل شيء يختفي. لا أثر، لا سجل، كأن شيئًا لم يكن.

هذا مقبول تمامًا للتمارين التعليمية، لكنه غير مقبول إطلاقًا لأي برنامج حقيقي. تخيّل تطبيق ملاحظات يَنسى كل ملاحظاتك بمجرد إغلاقه، أو برنامج محاسبة يفقد كل الفواتير عند إعادة تشغيل الحاسوب! هذا هو بالضبط سبب وجود **الملفات (Files)**: مساحة تخزين **دائمة** على القرص الصلب، تبقى بياناتك فيها حتى بعد إغلاق البرنامج، بل حتى بعد إطفاء الحاسوب بالكامل.

في هذا الفصل، ستتعلم:
- كيف تقرأ وتكتب ملفات نصية بأمان وبدون تسريب موارد (Resource Leaks).
- كيف تتعامل مع مسارات الملفات والمجلدات بأسلوب حديث وأنيق عبر `pathlib`.
- كيف تتبادل البيانات المُهيكَلة (قوائم، قواميس، كائنات) مع العالم الخارجي عبر **JSON** — التنسيق الأكثر انتشارًا في عالم البرمجة الحديث (تطبيقات الويب، واجهات API، ملفات الإعدادات).
- كيف تتعامل مع البيانات الجدولية (كالتي تراها في Excel) عبر **CSV**.

هذه المهارات هي **حجر الأساس** لكل ما سيأتي لاحقًا في الكتاب: عندما نبني واجهة API في الفصل 19، أو نحلّل بيانات مبيعات في الفصل 20، أو نبني أداة أتمتة في الفصل 21 — كل هذه المسارات تعتمد بشكل أساسي على قراءة وكتابة الملفات بمختلف تنسيقاتها.

> **نصيحة الخبير:** إتقان هذا الفصل يعني أن برامجك تتحول من «تجارب مؤقتة» إلى «أدوات حقيقية» يمكن الاعتماد عليها. هذا هو الخط الفاصل بين «أتعلم Python» و«أبني برامج فعلية بـ Python».

16.2 الملفات النصية: الأساسيات

فتح ملف وقراءته: الطريقة الآمنة

لفتح ملف في Python، نستخدم الدالة المدمجة `open()`. لكن هناك طريقة **صحيحة** وطريقة **خاطئة وخطيرة** لفعل ذلك.

الطريقة الخطيرة (لا تستخدمها أبدًا)

```python
file = open("notes.txt", "r", encoding="utf-8")
content = file.read()
print(content)
file.close()  # يجب أن نتذكر إغلاق الملف يدويًا!
```

المشكلة: إن حدث خطأ (استثناء) بين `open()` و `file.close()`، فإن السطر `file.close()` **لن يُنفَّذ أبدًا**، وسيبقى الملف «مفتوحًا» من منظور نظام التشغيل. هذا يُسمى **تسريب موارد (Resource Leak)** — قد يمنع برامج أخرى من الوصول للملف، ويستهلك موارد النظام دون داعٍ، وفي برنامج يفتح آلاف الملفات، قد يتسبب بانهيار كامل للنظام.

الطريقة الصحيحة: جملة `with`

```python
with open("notes.txt", "r", encoding="utf-8") as file:
    content: str = file.read()
    print(content)
# الملف يُغلق تلقائيًا هنا، حتى لو حدث خطأ بالداخل!
```

تذكر من الفصل الرابع عشر مفهوم **Context Managers** (`__enter__` و `__exit__`)؟ هذا بالضبط ما تستخدمه جملة `with` هنا: `open()` تُرجع كائن ملف يدعم بروتوكول Context Manager، فتضمن جملة `with` استدعاء `file.close()` تلقائيًا بمجرد الخروج من الكتلة، **بغض النظر عمّا حدث بالداخل** (نجاح، أو استثناء، أو `return` مبكر).

⚠ **تحذير قاطع:** استخدم جملة `with` **دائمًا ودون استثناء** عند التعامل مع الملفات. لا يوجد أي سبب مقبول لاستخدام `open()` بدون `with` في كود إنتاجي حديث.

معاملات `open()` الأساسية

```python
open(file_path, mode, encoding="utf-8")
```

المعامل	الوصف
`file_path`	مسار الملف (نص أو كائن `Path`، سنراه قريبًا)
`mode`	وضع الفتح: قراءة، كتابة، إلخ (الجدول التالي)
`encoding`	الترميز المستخدم لقراءة/كتابة النص

أوضاع الفتح (Modes)

الوضع	المعنى	إذا لم يوجد الملف	إذا وُجد الملف
`"r"`	قراءة (Read) - الافتراضي	`FileNotFoundError`	يُقرأ محتواه
`"w"`	كتابة (Write)	يُنشأ ملف جديد	**يُمحى المحتوى بالكامل!**
`"a"`	إضافة (Append)	يُنشأ ملف جديد	يُضاف المحتوى الجديد في النهاية
`"x"`	إنشاء حصري (Exclusive)	يُنشأ ملف جديد	`FileExistsError`
`"r+"`	قراءة وكتابة	`FileNotFoundError`	قراءة وتعديل بدون محو
`"rb"`, `"wb"`	نفس ما سبق، لكن بصيغة بايتات (Binary)	-	-

⚠ **خطأ شائع وخطير جدًا:** الخلط بين `"w"` و `"a"`. استخدام `"w"` على ملف موجود **يمحو محتواه بالكامل فورًا** بمجرد فتحه، حتى قبل كتابة أي شيء جديد! إن كنت تريد الإضافة إلى محتوى موجود دون فقدانه، استخدم `"a"` حصرًا.

مثال: الكتابة في ملف

```python
with open("diary.txt", "w", encoding="utf-8") as file:
    file.write("بدأت تعلم Python اليوم الأول.\n")
    file.write("كان يومًا ممتعًا جدًا!\n")
```

بعد تشغيل هذا الكود، سيظهر ملف `diary.txt` في نفس مجلد البرنامج، يحتوي على السطرين بالضبط.

> ⚠ **تحذير:** `write()` **لا تضيف** سطرًا جديدًا تلقائيًا (بعكس `print()`)! يجب عليك إضافة `\n` يدويًا إن أردت الانتقال لسطر جديد.

الإضافة إلى ملف موجود

```python
with open("diary.txt", "a", encoding="utf-8") as file:
    file.write("اليوم الثاني: تعلمت عن الملفات!\n")
```

الآن يحتوي `diary.txt` على ثلاثة أسطر (السطرين السابقين + السطر الجديد).

طرق متعددة لقراءة محتوى الملف

```python
# الطريقة 1: قراءة الملف بالكامل كنص واحد
with open("diary.txt", "r", encoding="utf-8") as file:
    content: str = file.read()
    print(content)

# الطريقة 2: قراءة سطر واحد فقط
with open("diary.txt", "r", encoding="utf-8") as file:
    first_line: str = file.readline()
    print(first_line)  # "اليوم الأول: بدأت تعلم Python.\n"

# الطريقة 3: قراءة كل الأسطر كقائمة
with open("diary.txt", "r", encoding="utf-8") as file:
    lines: list[str] = file.readlines()
    print(lines)
    # ['اليوم الأول:..  \n', 'اليوم الثاني:..  \n', ...]

# الطريقة 4 (الأفضل للملفات الكبيرة): التكرار المباشر سطرًا بسطر
with open("diary.txt", "r", encoding="utf-8") as file:
    for line in file:
        print(line.strip())  # .strip() لإزالة \n الزائدة
```

نصيحة الخبير - لماذا الطريقة 4 هي الأفضل؟ الطرق 1 و3 تُحمّل **الملف بأكمله** في الذاكرة دفعة واحدة. إن كان الملف بحجم 10 غيغابايت (سجلات خادم مثلاً)، فقد تستهلك كل ذاكرة حاسوبك وينهار البرنامج! التكرار المباشر (`for line in file:`) يقرأ **سطرًا واحدًا في كل مرة** (تمامًا مثل Generators التي تعلمناها في الفصل 12)، فيبقى استهلاك الذاكرة ثابتًا بغض النظر عن حجم الملف.

الترميز (Encoding): لماذا `encoding="utf-8"` إلزامي دائمًا؟

```python
# ❌ خطير: الاعتماد على الترميز الافتراضي لنظام التشغيل
with open("notes.txt", "r") as file:
    content = file.read()

# ✅ آمن: تحديد الترميز صراحة
with open("notes.txt", "r", encoding="utf-8") as file:
    content = file.read()
```

⚠ **تحذير حاسم خصوصًا للغة العربية:** إن لم تُحدد `encoding`، فإن Python تستخدم الترميز **الافتراضي لنظام التشغيل**، والذي يختلف بين Windows (غالبًا `cp1252` أو ما شابه) ولينكس وماك (غالبًا `utf-8`). إن كتبت ملفًا يحتوي نصًا عربيًا على نظام ويندوز بدون تحديد `encoding="utf-8"`، فقد يفشل فتحه لاحقًا على نظام آخر، أو تظهر رموز غريبة (مثل `Ø§Ù„Ø¹Ø±Ø¨ÙŠØ©`) بدلًا من النص العربي الصحيح. **القاعدة الذهبية: اكتب `encoding="utf-8"` في كل استدعاء لـ `open()`، بدون استثناء.**

معالجة أخطاء الملفات

```python
try:
    with open("missing_file.txt", "r", encoding="utf-8") as file:
        content = file.read()
except FileNotFoundError:
    print("الملف غير موجود!")
except PermissionError:
    print("لا تملك صلاحية الوصول لهذا الملف!")
```

🔄 **مقارنة: EAFP مقابل LBYL (من الفصل 11)**

```python
# أسلوب LBYL (تحقق قبل التنفيذ)
import os
if os.path.exists("file.txt"):
    with open("file.txt", "r", encoding="utf-8") as f:
        content = f.read()
else:
    print("«الملف غير موجود»")

# أسلوب EAFP (الأسلوب الـ Pythonic الموصى به)
try:
    with open("file.txt", "r", encoding="utf-8") as f:
        content = f.read()
except FileNotFoundError:
    print("«الملف غير موجود»")
```

الأسلوب الثاني أفضل لأنه **آمن من سباقات التوقيت (Race Conditions)**: قد يُحذف الملف تمامًا في اللحظة الفاصلة بين `os.path.exists()` واستدعاء `open()`!

16.3 pathlib : الأسلوب الحديث للتعامل مع المسارات

شكل 16.1 — تشريح المسار في pathlib

في الإصدارات القديمة من Python، كان التعامل مع مسارات الملفات يتم عبر وحدة `os.path`، باستخدام نصوص عادية ودمجها يدويًا. هذا الأسلوب كان عرضة لمشاكل كثيرة (خاصة اختلاف فاصل المسارات بين Windows `\` ولينكس/ماك `/`).

منذ Python 3.4، أصبح لدينا `pathlib`: وحدة حديثة تُمثّل المسارات ككائنات حقيقية (Objects)، بدلًا من مجرد نصوص. هذا هو **الأسلوب الموصى به حصريًا** في هذا الكتاب.

إنشاء كائن `Path`

```python
from pathlib import Path

# مسار نسبي
notes_path = Path("notes.txt")

# مسار بمجلدات متعددة - لاحظ استخدام / كعامل دمج!
project_path = Path("data") / "2024" / "report.txt"
print(project_path)
# على لينكس/ماك: data/2024/report.txt
# على Windows: data\2024\report.txt  (يتكيف تلقائيًا!)
```

مقارنة: `os.path` القديمة مقابل `pathlib` الحديثة

```python
# الأسلوب القديم (os.path) - لا تستخدمه في كود جديد
import os
path = os.path.join("data", "2024", "report.txt")
if os.path.exists(path):
    print(os.path.getsize(path))

# الأسلوب الحديث (pathlib) - الموصى به
from pathlib import Path
path = Path("data") / "2024" / "report.txt"
if path.exists():
    print(path.stat().st_size)
```

`pathlib` أكثر قابلية للقراءة، وتعمل بنفس الطريقة على كل أنظمة التشغيل، وتجمع كل الوظائف ذات الصلة بالمسار في كائن واحد متماسك (Object-Oriented)، بدلًا من دوال متفرقة.

خصائص ودوال مفيدة في `Path`

```python
from pathlib import Path

p = Path("data/2024/report.txt")

print(p.name)        # "report.txt" - اسم الملف كاملاً
print(p.stem)        # "report" - الاسم بدون الامتداد
print(p.suffix)      # ".txt" - الامتداد فقط
print(p.parent)      # "data/2024" - المجلد الأب
print(p.parts)       # ('data', '2024', 'report.txt') - كل الأجزاء

print(p.exists())    # True/False هل المسار موجود فعليًا؟
print(p.is_file())   # هل هو ملف؟
print(p.is_dir())    # هل هو مجلد؟
print(p.absolute())  # المسار الكامل المطلق
```

إنشاء المجلدات

```python
from pathlib import Path

data_dir = Path("data/2024")
data_dir.mkdir(parents=True, exist_ok=True)
```

- `parents=True` : ينشئ كل المجلدات الوسيطة اللازمة (هنا، `data` ثم `2024` بداخله) إن لم تكن موجودة.
- `exist_ok=True` : لا يُصدر خطأ إن كان المجلد موجودًا بالفعل (بدونها، ستحصل على `FileExistsError`).

قراءة وكتابة سريعة بدون `with` و `open()`

من أجمل ميزات `pathlib` : اختصارات مباشرة لقراءة/كتابة ملفات نصية كاملة، دون الحاجة لـ `open()` يدويًا!

```python
from pathlib import Path

notes = Path("notes.txt")

# كتابة (تُنشئ الملف تلقائيًا إن لم يكن موجودًا، وتفتح وتغلق تلقائيًا)
notes.write_text("مرحبًا من pathlib!\n", encoding="utf-8")

# قراءة
content: str = notes.read_text(encoding="utf-8")
print(content)
```

نصيحة الخبير: استخدم `read_text()` / `write_text()` للملفات البسيطة الصغيرة ذات العملية الواحدة (قراءة كاملة أو كتابة كاملة). استخدم `open()` مع `with` عندما تحتاج تحكمًا أدق (قراءة سطرًا بسطر، الإلحاق، أو عمليات متعددة على نفس الملف المفتوح).

استكشاف محتويات مجلد

Python

```python
from pathlib import Path

project_dir = Path(".")  # المجلد الحالي

# كل الملفات والمجلدات مباشرة (دون تعمق)
for item in project_dir.iterdir():
    print(item)

# البحث عن كل ملفات .py في المجلد الحالي فقط
for py_file in project_dir.glob("*.py"):
    print(py_file)

# البحث التكراري في كل المجلدات الفرعية أيضًا
for py_file in project_dir.glob("**/*.py"):
    print(py_file)

# بديل أحدث وأوضح لنفس الغرض (Python 3.12+)
for py_file in project_dir.rglob("*.py"):
    print(py_file)
```

مثال عملي: تنظيم ملفات مجلد التنزيلات حسب الامتداد

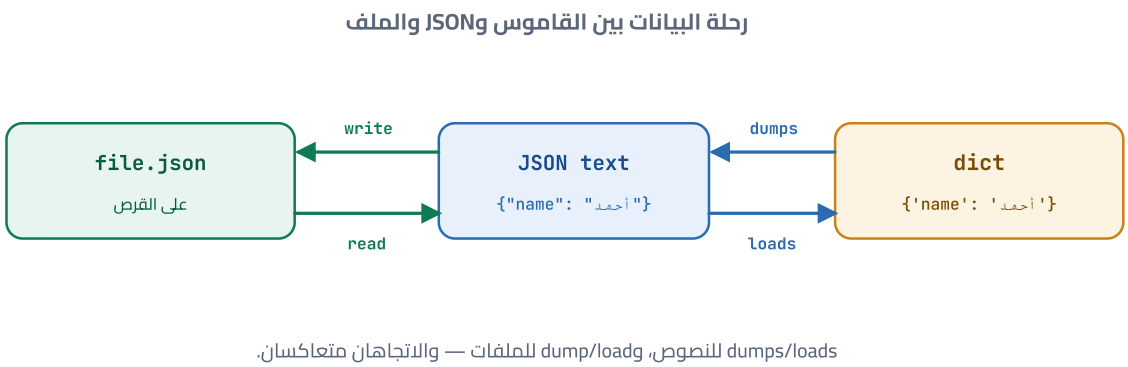
Python

```python
from pathlib import Path
import shutil

def organize_downloads(downloads_path: Path) -> None:
    """يُنظّم الملفات في مجلد حسب امتدادها، كل نوع في مجلد فرعي خاص به."""
    extension_folders: dict[str, str] = {
        ".pdf": "مستندات",
        ".jpg": "صور",
        ".png": "صور",
        ".mp4": "فيديوهات",
        ".zip": "أرشيف",
    }

    for file_path in downloads_path.iterdir():
        if file_path.is_file():
            folder_name = extension_folders.get(file_path.suffix.lower())
            if folder_name is not None:
                target_dir = downloads_path / folder_name
                target_dir.mkdir(exist_ok=True)
                shutil.move(str(file_path), str(target_dir / file_path.name))
                print(f"نُقل {file_path.name} إلى {folder_name}/")

organize_downloads(Path.home() / "Downloads")
```

هذا مثال حقيقي على **أتمتة (Automation)** بسيطة — موضوع سنتعمق فيه في الفصل 21، لكنه يُظهر بوضوح كيف تتحول معرفتك بالملفات إلى أدوات حقيقية موفّرة للوقت.

> **نصيحة الخبير:** `Path.home()` تُرجع مسار المجلد الرئيسي للمستخدم (`/home/username` على لينكس، أو `C:\Users\username` على ويندوز) بشكل متوافق تلقائيًا مع كل الأنظمة — لا تكتب هذه المسارات يدويًا أبدًا!

JSON 16.4 : لغة تبادل البيانات العالمية



شكل 16.2 — رحلة البيانات بين القاموس وJSON والملف

ما هو JSON ولماذا هو بهذه الأهمية؟

JSON (اختصار JavaScript Object Notation) هو تنسيق نصي بسيط جدًا لتمثيل البيانات المُهيكَلة، أصبح **المعيار العالمي** لتبادل البيانات بين البرامج المختلفة، خاصة في:

- واجهات برمجة التطبيقات (APIs) على الويب (سنستخدمه بكثافة في الفصل 19).
- ملفات الإعدادات (Configuration Files).
- تخزين بيانات بسيطة بدون الحاجة لقاعدة بيانات كاملة.

شكل JSON يُشبه **قواميس وقوائم Python بشكل شبه مطابق**:

```JSON
{
  "name": "أحمد",
  "age": 30,
  "is_active": true,
  "hobbies": ["برمجة" ,"قراءة"],
  "address": {
    "city": "القاهرة",
    "zip": "11511"
  },
  "spouse": null
}
```

جدول المقابلة بين JSON و Python

JSON	Python
`object {}`	`dict`
`array []`	`list`
`string "..."`	`str`
`number` (بدون فاصلة)	`int`
`number` (بفاصلة)	`float`
`false` / `true`	`False` / `True`
`null`	`None`

وحدة `json` المدمجة

لا تحتاج تثبيت أي شيء — `json` جزء من المكتبة القياسية (Standard Library) في Python.

```python
import json
```

تحويل Python إلى نص JSON: `dumps()` و `dump()`

```python
import json

user_data: dict[str, object] = {
    "name": "أحمد",
    "age": 30,
    "is_active": True,
    "hobbies": ["برمجة" ,"قراءة"],
    "spouse": None,
}

# dumps (مع s) تُرجع نصًا (string) يمكن طباعته أو إرساله
json_string: str = json.dumps(user_data, ensure_ascii=False, indent=2)
print(json_string)
```

الناتج:

JSON

```
{
  "name": "أحمد",
  "age": 30,
  "is_active": true,
  "hobbies": [
    "قراءة",
    "برمجة"
  ],
  "spouse": null
}
```

- `indent=2` : تنسيق جميل مقروء (بدونها، يُنتج JSON كله في سطر واحد متلاصق — مناسب للتخزين، غير مناسب للقراءة البشرية).
- `ensure_ascii=False` : **مهم جدًا للعربية!** بدونها، ستتحول كل الأحرف العربية إلى رموز Unicode مُرمَّزة (`\u0623\u062d\u0645\u062f`)، غير مقروءة للإنسان إطلاقًا.

⚠ **خطأ شائع جدًا عند التعامل مع العربية في JSON:** نسيان `ensure_ascii=False` . جرّب حذفها وشاهد النتيجة بنفسك — ستتفاجأ بالرموز الغريبة!

الكتابة المباشرة إلى ملف: `dump()` (بدون s)

Python

```
import json
from pathlib import Path

user_data = {"name": "سارة", "age": 25, "city": "دبي"}

with open("user.json", "w", encoding="utf-8") as file:
    json.dump(user_data, file, ensure_ascii=False, indent=2)
```

💡 **نصيحة للتذكر:** `dumps` (بحرف s، اختصار Dump String) تُرجع **نصًا**. `dump` (بدون s) تكتب **مباشرة إلى ملف مفتوح**. نفس المنطق ينطبق على `loads` / `load` في الاتجاه المعاكس.

القراءة من نص JSON: `loads()`

```python
import json

json_text = '{"name": "خالد", "age": 28}'
data: dict[str, object] = json.loads(json_text)

print(data["name"])  # "خالد"
print(type(data))    # <class 'dict'>
```

القراءة من ملف JSON: `load()` (بدون s)

```python
import json

with open("user.json", "r", encoding="utf-8") as file:
    data: dict[str, object] = json.load(file)

print(data["name"])  # "سارة"
```

مع `pathlib` : اختصار أنيق

```python
import json
from pathlib import Path

# كتابة
Path("user.json").write_text(
    json.dumps(user_data, ensure_ascii=False, indent=2),
    encoding="utf-8",
)

# قراءة
data = json.loads(Path("user.json").read_text(encoding="utf-8"))
```

التعامل مع أخطاء JSON

```python
import json

corrupted_json = '{"name": "أحمد", "age": }'  # صيغة خاطئة

try:
    data = json.loads(corrupted_json)
except json.JSONDecodeError as e:
    print(f"خطأ في تنسيق JSON: {e}")
```

Dataclasses و JSON معًا

تذكر `@dataclass` من الفصل 14. مشكلة: `json.dumps()` **لا تعرف** كيف تحوّل كائن `dataclass` مباشرة إلى JSON:

```python
from dataclasses import dataclass
import json

@dataclass
class Product:
    name: str
    price: float

product = Product("لابتوب", 1200.0)
json.dumps(product)  # ❌ TypeError: Object of type Product is not JSON serializable
```

الحل 1: `dataclasses.asdict()`

```python
from dataclasses import dataclass, asdict
import json

@dataclass
class Product:
    name: str
    price: float

product = Product("1200.0 ,"لابتوب")

product_dict: dict[str, object] = asdict(product)
json_string = json.dumps(product_dict, ensure_ascii=False, indent=2)
print(json_string)
# {
#   "name": "لابتوب",
#   "price": 1200.0
# }
```

الحل 2: تحويل عكسي (من JSON إلى Dataclass)

```python
data = json.loads(json_string)
restored_product = Product(**data)  # فك تغليف القاموس كمعاملات
print(restored_product)  # Product(name='لابتوب', price=1200.0)
```

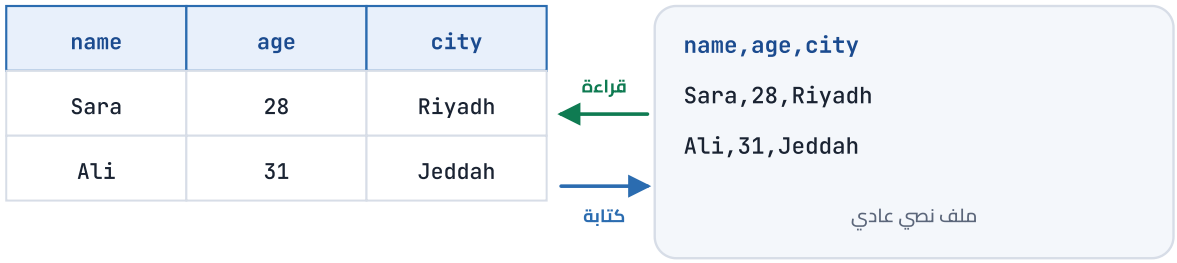
نصيحة الخبير: في مشاريع أكبر (خاصة عند بناء واجهات API في الفصل 19)، ستتعرّف على مكتبة **Pydantic** التي تؤتمت هذا التحويل بالكامل (من وإلى JSON) مع تحقق تلقائي قوي من صحة البيانات أيضًا، باستخدام Type Hints مباشرة. لكن فهم الآلية اليدوية هنا (عبر `asdict` و `**data`) ضروري لفهم ما يحدث «خلف الكواليس».

مثال متكامل: حفظ واسترجاع قائمة مهام

Python

```python
from dataclasses import dataclass, asdict
from pathlib import Path
import json

@dataclass
class Task:
    title: str
    done: bool = False

def save_tasks(tasks: list[Task], file_path: Path) -> None:
    data = [asdict(task) for task in tasks]
    file_path.write_text(
        json.dumps(data, ensure_ascii=False, indent=2),
        encoding="utf-8",
    )

def load_tasks(file_path: Path) -> list[Task]:
    if not file_path.exists():
        return []
    raw_data = json.loads(file_path.read_text(encoding="utf-8"))
    return [Task(**item) for item in raw_data]

tasks_file = Path("tasks.json")

tasks = [
    Task("تعلم JSON"),
    Task("تعلم CSV", done=True),
]
save_tasks(tasks, tasks_file)

loaded_tasks = load_tasks(tasks_file)
for task in loaded_tasks:
    status = "✅" if task.done else "⬜"
    print(f"{status} {task.title}")
```

CSV 16.5 : البيانات الجدولية



كل سطر صفٌّ، وكل فاصلة حدٌّ بين عمودين.

DictReader يربط كل صف بأسماء الأعمدة مباشرة.

شكل 16.3 — CSV: صفوف نصية تقابل جدولًا

ما هو CSV؟

(Comma-Separated Values) **CSV** هو تنسيق نصي بسيط جدًا لتمثيل بيانات جدولية (صفوف وأعمدة)، تمامًا كجدول Excel. كل سطر يمثل صفًا، والقيم مفصولة بفاصلة:

```csv
name,age,city
أحمد,30,القاهرة
سارة,25,دبي
خالد,28,الرياض
```

CSV شائع جدًا في: تصدير بيانات من Excel/Google Sheets، تبادل البيانات بين أنظمة محاسبية، وفي تحليل البيانات (سنعود إليه بعمق في الفصل 20 مع pandas).

وحدة `csv` المدمجة

الكتابة باستخدام `csv.writer`

```python
import csv

rows: list[list[str]] = [
    ["name", "age", "city"],
    ["أحمد", "30", "القاهرة"],
    ["سارة", "25", "دبي"],
]

with open("people.csv", "w", encoding="utf-8", newline="") as file:
    writer = csv.writer(file)
    writer.writerows(rows)
```

⚠ **تحذير مهم:** لاحظ `newline=""` عند فتح الملف للكتابة بصيغة CSV. بدونها، قد تظهر أسطر فارغة إضافية غير مرغوبة على نظام Windows، بسبب اختلاف طريقة تمثيل نهاية السطر بين الأنظمة. هذه قاعدة ثابتة: **دائمًا** `newline=""` **عند الكتابة بوحدة** `csv`.

القراءة باستخدام `csv.reader`

```python
import csv

with open("people.csv", "r", encoding="utf-8") as file:
    reader = csv.reader(file)
    for row in reader:
        print(row)  # كل صف عبارة عن list[str]
# ['name', 'age', 'city']
# ['أحمد', '30', 'القاهرة']
# ['سارة', '25', 'دبي']
```

⚠ **ملاحظة:** كل القيم المقروءة من CSV هي **نصوص (str) دائمًا**، حتى لو كانت تبدو أرقامًا! يجب تحويلها يدويًا (`int()`, `float()`) إن أردت استخدامها كأرقام فعلية.

الأسلوب الأفضل: `DictReader` و `DictWriter`

التعامل بالقوائم (`list[str]`) مزعج لأنك تحتاج تذكّر ترتيب الأعمدة بالأرقام (`row[0]`, `row[1]`...). الحل الأفضل: التعامل بالقواميس، حيث يصبح كل صف قاموسًا باسم العمود كمفتاح:

Python

```python
import csv

# القراءة بـ DictReader
with open("people.csv", "r", encoding="utf-8") as file:
    reader = csv.DictReader(file)
    for row in reader:
        print(row)
        # {'name': 'أحمد', 'age': '30', 'city': 'القاهرة'}
        print(f"{row['name']} يعيش في {row['city']}")
```

Python

```python
import csv

# الكتابة بـ DictWriter
people: list[dict[str, str]] = [
    {"name": "منى", "age": "22", "city": "عمّان"},
    {"name": "طارق", "age": "35", "city": "بيروت"},
]

with open("people2.csv", "w", encoding="utf-8", newline="") as file:
    fieldnames = ["name", "age", "city"]
    writer = csv.DictWriter(file, fieldnames=fieldnames)
    writer.writeheader()   # يكتب صف العناوين تلقائيًا
    writer.writerows(people)
```

نصيحة الخبير: استخدم `DictWriter` / `DictReader` دائمًا بدلًا من `writer` / `reader` العاديين، إلا في حالات نادرة جدًا لا تحتاج فيها لأسماء الأعمدة إطلاقًا. الكود أكثر وضوحًا، وأكثر مقاومة للأخطاء عند تغيير ترتيب الأعمدة مستقبلًا.

مثال متكامل: تحويل بيانات من قائمة Dataclasses إلى CSV

Python

```python
from dataclasses import dataclass, asdict, fields
import csv
from pathlib import Path

@dataclass
class Employee:
    name: str
    department: str
    salary: float

def export_to_csv(employees: list[Employee], file_path: Path) -> None:
    if not employees:
        return
    fieldnames = [f.name for f in fields(Employee)]
    with open(file_path, "w", encoding="utf-8", newline="") as file:
        writer = csv.DictWriter(file, fieldnames=fieldnames)
        writer.writeheader()
        for emp in employees:
            writer.writerow(asdict(emp))

employees = [
    Employee("6000.0 ,"تسويق" ,"ياسمين"),
    Employee("8500.0 ,"هندسة" ,"عمر"),
]

export_to_csv(employees, Path("employees.csv"))
```

مقارنة شاملة: JSON مقابل CSV

المعيار	JSON	CSV
البنية	متداخلة (Nested) - قواميس داخل قوائم داخل قواميس	مسطّحة (Flat) - صفوف وأعمدة فقط
الأنواع	يحافظ على الأنواع (رقم، نص، منطقي، null)	كل شيء نص، يحتاج تحويل يدوي
القراءة البشرية	جيدة مع `indent`	ممتازة، تفتح مباشرة في Excel
الاستخدام الأمثل	إعدادات، APIs، بيانات هرمية معقّدة	جداول بيانات، تصدير/استيراد من Excel
الحجم	أكبر نسبيًا (أسماء مفاتيح مكرّرة)	أصغر نسبيًا

16.6 أفضل الممارسات

استخدم `pathlib` دائمًا بدل نصوص المسارات الخام

```python
# ❌ تجنّب
file_path = "data" + "/" + "2024" + "/" + "report.txt"

# ✅ استخدم
from pathlib import Path
file_path = Path("data") / "2024" / "report.txt"
```

تحقق قبل الافتراض

```python
from pathlib import Path

config_path = Path("config.json")

if config_path.exists():
    config = json.loads(config_path.read_text(encoding="utf-8"))
else:
    config = {}  # قيمة افتراضية آمنة
```

استخدم `with` دائمًا، ولا تفتح ملفًا بدونها أبدًا

حدد `encoding="utf-8"` في كل استدعاء لـ `open()`

افصل منطق البرنامج عن منطق حفظ/قراءة البيانات

```python
# ✅ بنية جيدة: دوال واضحة ومسؤولة عن مهمة واحدة
def load_data(path: Path) -> list[dict[str, object]]:
    ...

def save_data(data: list[dict[str, object]], path: Path) -> None:
    ...

def process_data(data: list[dict[str, object]]) -> list[dict[str, object]]:
    ...  # المنطق الفعلي منفصل تمامًا عن I/O
```

هذا يُسهّل كثيرًا اختبار الكود لاحقًا (الفصل 25)، لأن منطق المعالجة لا يعتمد على وجود ملفات فعلية.

> ⚠ **خطأ شائع:** محاولة فتح نفس الملف للكتابة (`"w"`) **وهو مفتوح بالفعل للقراءة** في جزء آخر من البرنامج (خاصة على Windows قد يُصدر `PermissionError`). تأكد دائمًا أن جملة `with` السابقة قد انتهت (أي أن الملف أُغلق) قبل فتحه مجددًا بوضع مختلف.

16.7 مشروع تطبيقي: مدير جهات اتصال متكامل

لنبني مشروعًا يجمع كل ما تعلمناه: نظام إدارة جهات اتصال يحفظ البيانات بصيغة JSON، ويدعم التصدير إلى CSV، مع Type Hints كاملة ومعالجة أخطاء شاملة.

المتطلبات

1. كل جهة اتصال: اسم، هاتف، بريد إلكتروني، مجموعة (عائلة/عمل/أصدقاء).
2. حفظ/تحميل تلقائي من ملف JSON.
3. إضافة، بحث، حذف جهات اتصال.
4. تصدير إلى CSV لفتحه في Excel.
5. معالجة الأخطاء بشكل احترافي (ملف غير موجود، JSON تالف، إلخ).

✅ الحل النموذجي الكامل

Python

```python
"""
مدير جهات الاتصال - مشروع تطبيقي شامل
يغطي: الملفات النصية، pathlib، JSON، CSV، ومعالجة الأخطاء
"""
from __future__ import annotations

from dataclasses import dataclass, asdict, fields
from pathlib import Path
from typing import Literal
import json
import csv

type ContactGroup = Literal["عائلة", "عمل", "أصدقاء", "أخرى"]

# ---------------------------------------------------------
# نموذج البيانات
# ---------------------------------------------------------
@dataclass
class Contact:
    name: str
    phone: str
    email: str = ""
    group: ContactGroup = "أخرى"

    def display(self) -> str:
        return f"👤 {self.name} | {self.phone} | {self.email} | {self.group}"

# ---------------------------------------------------------
# استثناءات مخصصة
# ---------------------------------------------------------
class ContactNotFoundError(Exception):
    """يُرفع عند عدم العثور على جهة اتصال بالاسم المطلوب."""

# ---------------------------------------------------------
# مدير جهات الاتصال
# ---------------------------------------------------------
class ContactManager:
    def __init__(self, storage_path: Path) -> None:
        self.storage_path = storage_path
        self._contacts: list[Contact] = []
        self._load()

    # ----- التحميل والحفظ -----
    def _load(self) -> None:
```

```
        """يحمّل جهات الاتصال من ملف JSON إن وُجد، وإلا يبدأ بقائمة فارغة."""
        if not self.storage_path.exists():
            self._contacts = []
            return

        try:
            raw_text = self.storage_path.read_text(encoding="utf-8")
            raw_data = json.loads(raw_text)
            self._contacts = [Contact(**item) for item in raw_data]
        except json.JSONDecodeError:
            print("⚠ تحذير: ملف البيانات تالف. سيتم البدء بقائمة فارغة.")
            self._contacts = []
        except TypeError:
            print("⚠ تحذير: بنية البيانات غير متوافقة. سيتم البدء بقائمة فارغة.")
            self._contacts = []

    def save(self) -> None:
        """يحفظ كل جهات الاتصال الحالية إلى ملف JSON."""
        data = [asdict(contact) for contact in self._contacts]
        self.storage_path.write_text(
            json.dumps(data, ensure_ascii=False, indent=2),
            encoding="utf-8",
        )

    # ----- العمليات الأساسية -----
    def add_contact(
        self,
        name: str,
        phone: str,
        email: str = "",
        group: ContactGroup = "أخرى",
    ) -> Contact:
        contact = Contact(name=name, phone=phone, email=email, group=group)
        self._contacts.append(contact)
        self.save()
        return contact

    def find_by_name(self, name: str) -> Contact:
        for contact in self._contacts:
            if contact.name == name:
                return contact
        raise ContactNotFoundError(f"لا توجد جهة اتصال باسم '{name}'")

    def delete_contact(self, name: str) -> None:
        contact = self.find_by_name(name)  # يرفع ContactNotFoundError تلقائيًا إن لم يوجد
        self._contacts.remove(contact)
        self.save()

    def search(self, keyword: str) -> list[Contact]:
        keyword_lower = keyword.lower()
        return [
```

```
            c for c in self._contacts
            if keyword_lower in c.name.lower() or keyword_lower in c.phone
        ]

    def get_by_group(self, group: ContactGroup) -> list[Contact]:
        return [c for c in self._contacts if c.group == group]

    def all_contacts(self) -> list[Contact]:
        return self._contacts.copy()

    def count(self) -> int:
        return len(self._contacts)

    # ----- التصدير -----
    def export_to_csv(self, csv_path: Path) -> None:
        if not self._contacts:
            print("لا توجد جهات اتصال للتصدير.")
            return

        fieldnames = [f.name for f in fields(Contact)]
        with open(csv_path, "w", encoding="utf-8", newline="") as file:
            writer = csv.DictWriter(file, fieldnames=fieldnames)
            writer.writeheader()
            for contact in self._contacts:
                writer.writerow(asdict(contact))
        print(f"✅ تم تصدير {len(self._contacts)} جهة اتصال إلى {csv_path}")

    def import_from_csv(self, csv_path: Path) -> int:
        if not csv_path.exists():
            raise FileNotFoundError(f"ملف الاستيراد غير موجود: {csv_path}")

        imported_count = 0
        with open(csv_path, "r", encoding="utf-8") as file:
            reader = csv.DictReader(file)
            for row in reader:
                self._contacts.append(Contact(**row))  # type: ignore[arg-type]
                imported_count += 1
        self.save()
        return imported_count

# ---------------------------------------------------------
# برنامج تجريبي
# ---------------------------------------------------------
def main() -> None:
    manager = ContactManager(Path("contacts.json"))

    manager.add_contact("0501234567", "أحمد علي", "ahmed@example.com", "عمل")
    manager.add_contact("0559876543", "سارة محمد", group="عائلة")
    manager.add_contact("0507654321", "خالد يوسف", "khaled@example.com", "أصدقاء")

    print(f"إجمالي جهات الاتصال: {manager.count()}")
```

```
        print()

        print("كل جهات الاتصال:")
        for contact in manager.all_contacts():
            print(contact.display())
        print()

        print("البحث عن 'سارة':")
        for contact in manager.search("سارة"):
            print(contact.display())
        print()

        try:
            manager.find_by_name("غير موجود")
        except ContactNotFoundError as e:
            print(f"خطأ متوقع: {e}")
        print()

        manager.export_to_csv(Path("contacts_export.csv"))

if __name__ == "__main__":
    main()
```

فحص المشروع بـ mypy (كما تعلّمنا في الفصل 15)

```
Terminal
```

```
uv run mypy contact_manager.py --strict
```

16.8 تمارين

🟢 مستوى مبتدئ

تمرين 1: اكتب برنامجًا يطلب من المستخدم إدخال 3 أسماء، ثم يحفظها في ملف `names.txt`، كل اسم في سطر منفصل.

تمرين 2: اكتب دالة `count_lines(file_path: Path) → int` تُرجع عدد الأسطر في ملف نصي معيّن.

تمرين 3: اكتب برنامجًا يحفظ قاموسًا بسيطًا (اسم، عمر، مدينة) في ملف JSON باسم `person.json`، ثم يقرأه ويطبع محتواه.

✅ الحل النموذجي

Python

```python
# تمرين 1
names: list[str] = [input(f"أدخل الاسم {i + 1}: ") for i in range(3)]

with open("names.txt", "w", encoding="utf-8") as file:
    for name in names:
        file.write(name + "\n")

# تمرين 2
from pathlib import Path

def count_lines(file_path: Path) -> int:
    with open(file_path, "r", encoding="utf-8") as file:
        return sum(1 for _ in file)

print(count_lines(Path("names.txt")))

# تمرين 3
import json
from pathlib import Path

person: dict[str, object] = {"name": "ليلى", "age": 27, "city": "تونس"}

Path("person.json").write_text(
    json.dumps(person, ensure_ascii=False, indent=2),
    encoding="utf-8",
)

loaded = json.loads(Path("person.json").read_text(encoding="utf-8"))
print(loaded)
```

🟡 مستوى متوسط

تمرين 4: اكتب برنامجًا يقرأ ملف `sales.csv` (أنشئه بنفسك بعمودين: `product` , `amount`)، ويحسب إجمالي المبيعات (مجموع عمود `amount`)، مع معالجة احتمال عدم وجود الملف.

تمرين 5: اكتب فئة `Logger` تحتفظ بسجل أحداث، وتوفّر دالة `log(message: str) → None` تضيف سطرًا جديدًا لملف `app.log` مع الوقت الحالي (استخدم `datetime.now()`)، باستخدام وضع `"a"` (إضافة وليس كتابة).

تمرين 6: اكتب دالتين `save_config(config: dict[str, object], path: Path) → None` و `load_config(path: Path) → dict[str, object]` للتعامل مع ملف إعدادات JSON، بحيث إن لم يوجد الملف، تُرجع `load_config` قاموسًا افتراضيًا `{"theme": "light", "language": "ar"}`.

✅ الحل النموذجي

Python

```python
# تمرين 4
import csv
from pathlib import Path

def calculate_total_sales(csv_path: Path) -> float:
    if not csv_path.exists():
        print("ملف المبيعات غير موجود!")
        return 0.0

    total = 0.0
    with open(csv_path, "r", encoding="utf-8") as file:
        reader = csv.DictReader(file)
        for row in reader:
            total += float(row["amount"])
    return total

print(calculate_total_sales(Path("sales.csv")))

# تمرين 5
from pathlib import Path
from datetime import datetime

class Logger:
    def __init__(self, log_path: Path) -> None:
        self.log_path = log_path

    def log(self, message: str) -> None:
        timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
        with open(self.log_path, "a", encoding="utf-8") as file:
            file.write(f"[{timestamp}] {message}\n")

logger = Logger(Path("app.log"))
logger.log("بدأ تشغيل التطبيق")
logger.log("تم تسجيل دخول مستخدم جديد")

# تمرين 6
import json
from pathlib import Path

def save_config(config: dict[str, object], path: Path) -> None:
    path.write_text(
        json.dumps(config, ensure_ascii=False, indent=2),
        encoding="utf-8",
    )
```

```python
def load_config(path: Path) -> dict[str, object]:
    default_config: dict[str, object] = {"theme": "light", "language": "ar"}
    if not path.exists():
        return default_config
    try:
        return json.loads(path.read_text(encoding="utf-8"))
    except json.JSONDecodeError:
        return default_config

config = load_config(Path("config.json"))
print(config)
save_config({"theme": "dark", "language": "en"}, Path("config.json"))
```

🔴 تحدي

تمرين 7: ابنِ نظام «نسخ احتياطي» بسيط:

- دالة `backup_file(source: Path, backup_dir: Path) → Path` تنسخ ملفًا إلى مجلد نسخ احتياطي، مضيفةً الطابع الزمني الحالي إلى اسم النسخة (مثل `notes_20250615_143200.txt`).
- دالة `list_backups(backup_dir: Path, original_name: str) → list[Path]` تُرجع كل النسخ الاحتياطية الخاصة بملف معيّن، مرتبة من الأحدث إلى الأقدم.
- دالة `restore_latest(backup_dir: Path, original_name: str, destination: Path) → bool` تستعيد أحدث نسخة احتياطية، وتُرجع `True` إن نجحت، `False` إن لم توجد أي نسخة.
- استخدم `pathlib` حصريًا، و Type Hints كاملة، وتأكد من مرور الكود بـ mypy بوضع `--strict`.

✅ الحل النموذجي

Python

```python
from pathlib import Path
from datetime import datetime
import shutil

def backup_file(source: Path, backup_dir: Path) -> Path:
    if not source.exists():
        raise FileNotFoundError(f"الملف الأصلي غير موجود: {source}")

    backup_dir.mkdir(parents=True, exist_ok=True)

    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
    backup_name = f"{source.stem}_{timestamp}{source.suffix}"
    backup_path = backup_dir / backup_name

    shutil.copy2(source, backup_path)
    return backup_path

def list_backups(backup_dir: Path, original_name: str) -> list[Path]:
    if not backup_dir.exists():
        return []

    stem = Path(original_name).stem
    suffix = Path(original_name).suffix

    matches = list(backup_dir.glob(f"{stem}_*{suffix}"))
    return sorted(matches, key=lambda p: p.stat().st_mtime, reverse=True)

def restore_latest(backup_dir: Path, original_name: str, destination: Path) -> bool:
    backups = list_backups(backup_dir, original_name)
    if not backups:
        return False

    latest_backup = backups[0]
    shutil.copy2(latest_backup, destination)
    return True

# تجربة
source_file = Path("notes.txt")
source_file.write_text("محتوى تجريبي أول", encoding="utf-8")

backup_dir = Path("backups")

backup_file(source_file, backup_dir)
```

```
import time
time.sleep(1)  # للتأكد من اختلاف الطابع الزمني بين النسختين

source_file.write_text("محتوى تجريبي ثانٍ - مُحدَّث", encoding="utf-8")
backup_file(source_file, backup_dir)

backups = list_backups(backup_dir, "notes.txt")
print(f"عدد النسخ الاحتياطية: {len(backups)}")
for b in backups:
    print(f"  {b.name}")

success = restore_latest(backup_dir, "notes.txt", Path("restored_notes.txt"))
print(f"تمت الاستعادة بنجاح: {success}")
print(Path("restored_notes.txt").read_text(encoding="utf-8"))
```

تحقق:

```
Terminal

uv run mypy backup_system.py --strict
```

16.9 اختبر فهمك (Quiz)

❓ السؤال 1: لماذا نستخدم جملة `with` عند فتح الملفات بدلًا من `open()` و `close()` يدويًا؟

لأن `with` تضمن إغلاق الملف تلقائيًا بمجرد الخروج من الكتلة، **حتى لو حدث استثناء بالداخل**. استخدام `close()` يدويًا يعرّض البرنامج لتسريب الموارد (Resource Leak) إن حدث خطأ قبل الوصول لسطر `close()`.

❓ السؤال 2: ما الفرق بين وضعي الفتح `"w"` و `"a"`؟

`"w"` (Write) يمحو محتوى الملف بالكامل فور فتحه إن كان موجودًا، ثم يكتب من جديد. `"a"` (Append) يضيف المحتوى الجديد في نهاية الملف دون حذف أي شيء موجود مسبقًا.

❓ السؤال 3: لماذا يجب تحديد `encoding="utf-8"` دائمًا عند فتح الملفات؟

لأن الترميز الافتراضي يختلف حسب نظام التشغيل، مما قد يتسبب في ظهور رموز غريبة أو أخطاء عند التعامل مع نصوص غير إنجليزية (كالعربية) أو عند نقل الملف بين أنظمة تشغيل مختلفة. تحديد `utf-8` صراحة يضمن سلوكًا متسقًا على كل الأنظمة.

❓ السؤال 4: ما الفرق بين `json.dump()` و `json.dumps()`؟

`dump()` (بدون s) تكتب البيانات **مباشرة إلى ملف مفتوح**. `dumps()` (بحرف s، اختصار "dump string") تُرجع **نصًا (string)** يمكن طباعته أو التعامل معه يدويًا. نفس المنطق ينطبق على `load()` / `loads()` في الاتجاه المعاكس.

❓ السؤال 5: لماذا نحتاج معامل `newline=""` عند فتح ملف CSV للكتابة؟

لتجنّب ظهور أسطر فارغة إضافية غير مرغوبة، خاصة على نظام Windows، بسبب اختلاف طريقة تمثيل «نهاية السطر» بين أنظمة التشغيل. وحدة `csv` تتولى التعامل مع نهايات الأسطر بنفسها بشكل صحيح فقط عند تحديد `newline=""`.

❓ السؤال 6: لماذا تُعتبر `pathlib` أفضل من `os.path` في الكود الحديث؟

لأنها تمثّل المسارات ككائنات حقيقية (Objects) بدلًا من نصوص خام، توفّر عامل `/` أنيقًا لدمج المسارات، تتكيّف تلقائيًا مع اختلاف فاصل المسارات بين أنظمة التشغيل، وتجمع كل الوظائف ذات الصلة (فحص الوجود، القراءة، الكتابة، استخراج الاسم/الامتداد) في واجهة واحدة متماسكة وأكثر قابلية للقراءة.

16.10 ملخص الفصل (Cheat Sheet)

```Python
```

```python
# ===== فتح الملفات النصية =====
with open("file.txt", "r", encoding="utf-8") as f:
    content = f.read()            # الملف كاملاً كنص
    # أو
    lines = f.readlines()         # قائمة بكل الأسطر
    # أو (الأفضل للملفات الكبيرة)
    for line in f:
        print(line.strip())

with open("file.txt", "w", encoding="utf-8") as f:
    f.write("نص جديد\n")          # يمحو المحتوى القديم!

with open("file.txt", "a", encoding="utf-8") as f:
    f.write("إضافة\n")            # يُضيف دون حذف

# ===== pathlib =====
from pathlib import Path

p = Path("data") / "file.txt"
p.exists()          # هل موجود؟
p.is_file()         # هل ملف؟
p.mkdir(parents=True, exist_ok=True)
p.write_text("محتوى", encoding="utf-8")
content = p.read_text(encoding="utf-8")
p.name, p.stem, p.suffix, p.parent

for f in Path(".").glob("*.py"):
    print(f)

# ===== JSON =====
import json

json.dumps(data, ensure_ascii=False, indent=2)   # Python → نص JSON
json.loads(json_text)                            # نص JSON → Python
json.dump(data, file, ensure_ascii=False, indent=2)  # Python → ملف
json.load(file)                                  # ملف → Python

# مع Dataclasses
from dataclasses import asdict
json.dumps(asdict(my_object), ensure_ascii=False)

# ===== CSV =====
import csv

with open("f.csv", "w", encoding="utf-8", newline="") as file:
    writer = csv.DictWriter(file, fieldnames=["a", "b"])
    writer.writeheader()
    writer.writerows(data)
```

```python
with open("f.csv", "r", encoding="utf-8") as file:
    reader = csv.DictReader(file)
    for row in reader:
        print(row)  # dict[str, str]
```

المهمة	الأداة الموصى بها
مسارات الملفات	`pathlib.Path`
نص بسيط كامل	`write_text()` / `Path.read_text()`
قراءة تدريجية (ملفات كبيرة)	`with open(...) as f: for line in f:`
بيانات هرمية (إعدادات، APIs)	`json`
بيانات جدولية (Excel)	`DictWriter` / `csv.DictReader`
معالجة أخطاء الملفات	`try/except FileNotFoundError`

16.11 ماذا بعد؟

الآن برامجك لم تعد «تُنسى» بمجرد إغلاقها — أصبحت قادرة على حفظ بياناتها، قراءتها، وتبادلها مع العالم الخارجي بأكثر التنسيقات انتشارًا. هذه المهارة ستكون حاضرة في كل مشروع تقريبًا من الآن فصاعدًا، في كل مسارات التخصص القادمة.

لكن هناك تحدٍ آخر ينتظرنا: ماذا لو احتاج برنامجك لتحميل 100 ملف من الإنترنت، أو استدعاء واجهة API عدة مرات، أو قراءة عدة ملفات ضخمة في آن واحد؟ لو نفّذنا هذه العمليات بالطريقة التقليدية (Synchronous) التي تعلمناها حتى الآن، سينتظر برنامجك كل عملية حتى تنتهي تمامًا قبل الانتقال للتالية — مما يُهدر وقتًا ثمينًا بلا داعٍ.

في الفصل السابع عشر، سندخل عالم **البرمجة غير المتزامنة (Asynchronous Programming)**: مفهوم `async` / `await`، لماذا هو ضروري خصوصًا في برمجة الويب والشبكات، وكيف يسمح لبرنامجك بتنفيذ مهام متعددة «في نفس الوقت تقريبًا» دون الحاجة لتعقيدات الـ Threading التقليدية.

فصل 17 / 26 · الجزء الرابع

17

البرمجة غير المتزامنة - مقدمة عملية (async/await)

13 دقيقة قراءة · 13 قسمًا · 17 مثالًا · 2,568 كلمة

في هذا الفصل

17.1 لماذا يستحق هذا الفصل اهتمامك الكامل؟
17.2 فهم المشكلة أولًا: لماذا الكود العادي «بطيء» أحيانًا؟
17.3 المفاهيم الأساسية: `async def`، `await`، و Coroutines
17.4 تحقيق التزامن الحقيقي: `asyncio.gather()`
17.5 مثال عملي حقيقي: تحميل بيانات من الإنترنت بـ `aiohttp`
17.6 `asyncio.create_task()`: بدء المهام دون انتظار فوري
17.7 معالجة الأخطاء في الكود غير المتزامن
17.8 متى تستخدم Async، ومتى لا تحتاجه إطلاقًا؟
17.9 مشروع تطبيقي: مُجمِّع أسعار من عدة «متاجر» (محاكاة)
17.10 تمارين
17.11 اختبر فهمك (Quiz)
17.12 ملخص الفصل (Cheat Sheet)
17.13 ماذا بعد؟

17.1 لماذا يستحق هذا الفصل اهتمامك الكامل؟

تخيّل أنك تعمل في مطعم، وأنت النادل الوحيد. يأتيك زبون ويطلب قهوة. بدلًا من أن تذهب إلى الآلة وتضغط الزر ثم **تقف أمامها تحدّق فيها لمدة 3 دقائق كاملة** حتى تجهز القهوة، ماذا لو ذهبت واستقبلت زبونًا آخر، وأخذت طلبه، وبدأت تحضير طلب ثالث، بينما القهوة الأولى «تُحضَّر في الخلفية»؟ عندما تجهز القهوة، تعود وتقدمها. بهذه الطريقة، تخدم عشرات الزبائن في نفس الوقت الذي كنت ستخدم فيه زبونًا واحدًا فقط لو انتظرت كل طلب بالكامل قبل الانتقال للتالي.

هذا بالضبط هو جوهر **البرمجة غير المتزامنة (Asynchronous Programming)**.

في كل الفصول السابقة، كتبنا كودًا **متزامنًا (Synchronous)**: كل سطر ينتظر السطر الذي قبله حتى ينتهي تمامًا قبل أن يبدأ. هذا منطقي ومثالي عندما تكون العمليات سريعة (جمع رقمين، تكرار قائمة). لكن ماذا يحدث عندما يحتاج برنامجك إلى:

- تحميل بيانات من الإنترنت (قد تستغرق ثوانٍ، وتعتمد على سرعة الشبكة، لا على معالج حاسوبك).
- استدعاء واجهة API خارجية، وانتظار ردها.
- قراءة ملف ضخم من القرص الصلب.

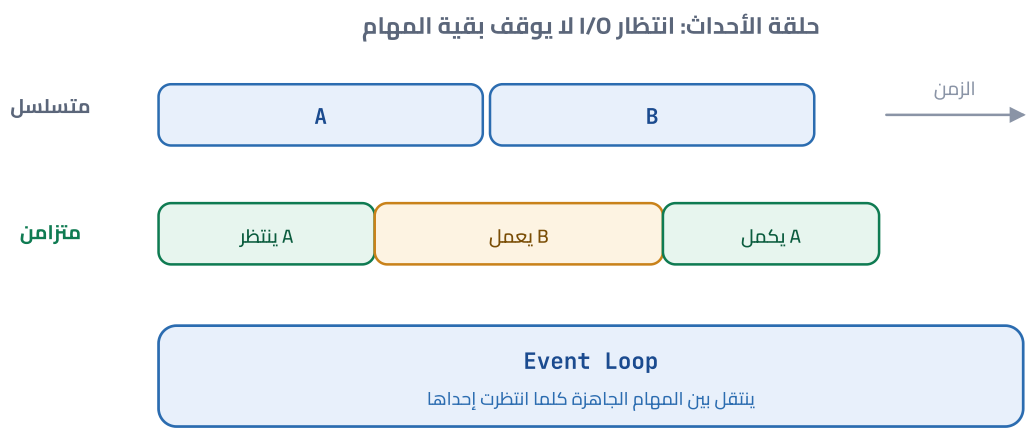
- الانتظار لإدخال من المستخدم دون تجميد باقي البرنامج.

في كل هذه الحالات، معالج حاسوبك **لا يفعل شيئًا فعليًا** أثناء الانتظار — إنه فقط «يقف ساكنًا» ينتظر استجابة من مصدر خارجي (الشبكة، القرص الصلب، إلخ). هذا النوع من الانتظار يُسمى **I/O-bound** (عملية محدودة بالإدخال/الإخراج)، بعكس **CPU-bound** (عملية محدودة بقوة المعالجة الفعلية، كحساب رياضي معقّد).

البرمجة غير المتزامنة تسمح لبرنامجك بـ: «بينما أنتظر هذه العملية البطيئة، دعني أبدأ بعملية أخرى، وسأعود للأولى عندما تجهز». هذا يُحدث فرقًا هائلاً في الأداء عند التعامل مع عشرات أو مئات العمليات المتزامنة (مثل خادم ويب يخدم آلاف الطلبات، أو برنامج يُحمّل 100 صفحة ويب في آن واحد).

> **نصيحة الخبير:** هذا الفصل هو **مقدمة عملية** وليس تغطية شاملة لكل تعقيدات البرمجة غير المتزامنة (وهي موضوع يمكن أن يملأ كتابًا كاملاً بمفرده!). هدفنا هنا أن تفهم **متى ولماذا** تستخدمها، وأن تكون قادرًا على قراءة وكتابة كود `async/await` أساسي بثقة — وهو ما ستحتاجه بكثرة في الفصل 19 عند بناء واجهات API بـ FastAPI.

17.2 فهم المشكلة أولًا: لماذا الكود العادي «بطيء» أحيانًا؟



شكل 17.1 — حلقة الأحداث: انتظار I/O لا يوقف بقية المهام

لنكتب مثالًا يُحاكي عمليات شبكة بطيئة (باستخدام `time.sleep()` كمحاكاة):

Python

```python
import time

def fetch_data(source: str, delay: float) -> str:
    """يُحاكي طلب بيانات بطيء من مصدر خارجي (مثل API أو قاعدة بيانات)."""
    print(f"🔄 بدء تحميل البيانات من {source}...")
    time.sleep(delay)  # محاكاة وقت الانتظار (شبكة، قرص صلب، إلخ)
    print(f"✅ انتهى تحميل البيانات من {source}")
    return f"بيانات من {source}"

def main() -> None:
    start = time.perf_counter()

    result1 = fetch_data("الخادم أ", 2)
    result2 = fetch_data("الخادم ب", 2)
    result3 = fetch_data("الخادم ج", 2)

    elapsed = time.perf_counter() - start
    print(f"\nالوقت الإجمالي: {elapsed:.2f} ثانية")

main()
```

الناتج:

```
🔄 بدء تحميل البيانات من الخادم أ...
✅ انتهى تحميل البيانات من الخادم أ
🔄 بدء تحميل البيانات من الخادم ب...
✅ انتهى تحميل البيانات من الخادم ب
🔄 بدء تحميل البيانات من الخادم ج...
✅ انتهى تحميل البيانات من الخادم ج

الوقت الإجمالي: 6.00 ثانية
```

لاحظ: كل عملية تنتظر الأخرى بالكامل، فالوقت الإجمالي هو **مجموع** كل الأوقات (2+2+2=6 ثوانٍ)، رغم أن هذه العمليات **لا تعتمد على بعضها البعض إطلاقًا**! لا يوجد سبب منطقي لانتظار «الخادم أ» قبل البدء بطلب «الخادم ب» — فهما مستقلان تمامًا. هذا هدر واضح للوقت.

مقارنة: لماذا لا نستخدم Threading أو Multiprocessing هنا؟

قد يسألك سؤال وجيه: «ألم نتعلم أن هناك طرقًا أخرى لتنفيذ عمليات متعددة (Threading, Multiprocessing)؟»
نعم، لكن:

- **Multiprocessing** (عمليات متعددة): مناسب للمهام الثقيلة حسابيًا (CPU-bound)، لكنه ثقيل جدًا في استهلاك الذاكرة (كل عملية تحتاج نسخة كاملة من المفسّر)، وغير مناسب لآلاف العمليات الصغيرة المتزامنة.
- **Threading** (خيوط): يسمح بالتزامن، لكنه معقّد (مشاكل تزامن الوصول للبيانات المشتركة - Race Conditions)، وفي Python تحديدًا، هناك قيد شهير اسمه **GIL** (Global Interpreter Lock) يمنع تنفيذ أكواد Python الفعلية بالتوازي الحقيقي في نفس اللحظة عبر الخيوط.
- **Async/Await**: أخف بكثير من Threading (لا حاجة لآلاف الخيوط الثقيلة)، ومصمم خصيصًا لحالات الانتظار الكثيرة (I/O-bound) مثل الشبكة، وهو الخيار الأمثل والأكثر انتشارًا حديثًا لهذا النوع من المشاكل في Python، خاصة في عالم الويب.

17.3 المفاهيم الأساسية: `async def` ، `await` ، Coroutines

الكلمة المفتاحية `async def` : تعريف دالة غير متزامنة

```python
async def say_hello() -> None:
    print("مرحبًا!")
```

بإضافة `async` قبل `def` ، تتحول الدالة إلى **Coroutine Function** (دالة مشتركة/تعاونية). عند استدعائها، **لا تُنفَّذ فورًا**:

```python
result = say_hello()
print(result)  # <coroutine object say_hello at 0x...>
# لاحظ: لم تُطبع إطلاقًا "مرحبًا!"
```

استدعاء دالة `async def` **لا يُشغّلها**، بل يُنشئ كائن **Coroutine** — وصفة جاهزة للتنفيذ، لكنها تحتاج «مُشغِّلًا» خاصًا ليبدأ تنفيذها فعليًا.

تشغيل أول Coroutine باستخدام `asyncio.run()`

```python
import asyncio

async def say_hello() -> None:
    print("مرحبًا!")

asyncio.run(say_hello())  # الآن تُطبع "مرحبًا!" فعليًا
```

`asyncio` هي وحدة مدمجة في Python (لا تحتاج تثبيتًا) توفّر كل الأدوات اللازمة للبرمجة غير المتزامنة. `asyncio.run()` هي **نقطة الدخول** الرئيسية: تُنشئ «حلقة أحداث» (Event Loop — سنشرحها قريبًا)، تُشغّل الـ Coroutine المُعطاة، ثم تُغلق كل شيء عند الانتهاء.

> **نصيحة الخبير:** `asyncio.run()` يجب أن تُستدعى **مرة واحدة فقط** في برنامجك، عادة في نقطة الدخول الرئيسية (`if __name__ == "__main__":`). لا تستدعِها من داخل Coroutine أخرى.

الكلمة المفتاحية `await`: انتظار نتيجة Coroutine أخرى

```python
import asyncio

async def greet(name: str) -> None:
    print(f"مرحبًا، {name}!")

async def main() -> None:
    await greet("أحمد")
    await greet("سارة")

asyncio.run(main())
```

`await` تعني: «نفّذ هذه الـ Coroutine، وانتظر نتيجتها، لكن أثناء الانتظار، **اسمح لعمليات أخرى بالعمل** إن كانت تنتظر أيضًا». هذا الجزء الأخير هو **جوهر** الفائدة الكاملة من `async/await`.

⚠ **قاعدة صارمة:** لا يمكنك استخدام `await` إلا داخل دالة معرّفة بـ `async def` . محاولة استخدامها خارج ذلك تُصدر `SyntaxError` .

```python
def regular_function():
    await something()  # ❌ SyntaxError: 'await' outside async function
```

محاكاة الانتظار غير المتزامن: `asyncio.sleep()`

تذكر أننا استخدمنا `time.sleep()` في المثال الأول لمحاكاة بطء الشبكة. لكن `time.sleep()` هي دالة **متزامنة (Blocking)** — تُجمّد **كل البرنامج بالكامل**، ولا تسمح لأي كود آخر بالعمل أثناء انتظارها، حتى لو كان الكود الآخر Coroutine!

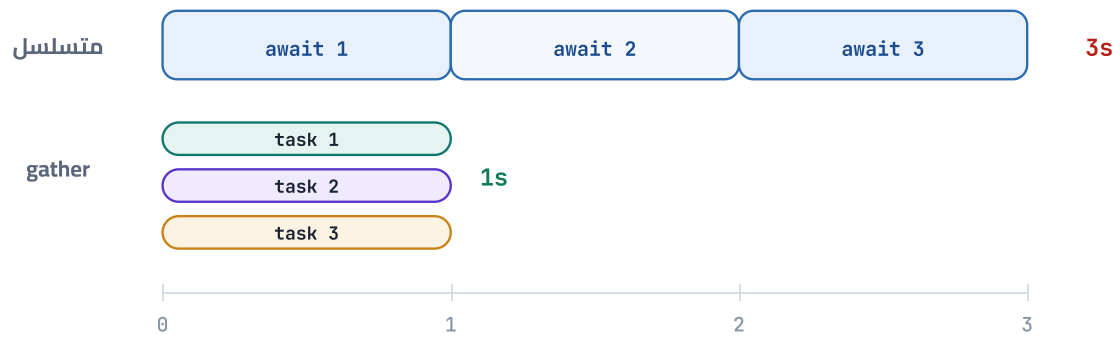
الحل: `asyncio.sleep()` — نسخة غير متزامنة من `sleep()` ، مصممة خصيصًا للعمل مع `async/await` :

```python
import asyncio
import time

async def fetch_data(source: str, delay: float) -> str:
    print(f"🔄 بدء تحميل البيانات من {source}.  ")
    await asyncio.sleep(delay)  # ← غير متزامن! يسمح لعمليات أخرى بالعمل أثناء الانتظار
    print(f"✅ انتهى تحميل البيانات من {source}")
    return f"بيانات من {source}"

async def main() -> None:
    start = time.perf_counter()

    result1 = await fetch_data("2 ,"الخادم أ")
    result2 = await fetch_data("2 ,"الخادم ب")
    result3 = await fetch_data("2 ,"الخادم ج")

    elapsed = time.perf_counter() - start
    print(f"\nالوقت الإجمالي: {elapsed:.2f} ثانية")

asyncio.run(main())
```

لكن انتظر... النتيجة هنا **لا تزال 6 ثوانٍ تقريبًا**! لماذا؟

⚠ **سوء فهم شائع جدًا للمبتدئين:** مجرد استخدام `async` / `await` **لا يجعل العمليات تعمل بالتوازي تلقائيًا!** في المثال أعلاه، كتبنا `await fetch_data(...)` ثلاث مرات **متتالية** — هذا يعني «انتظر الأولى حتى تنتهي كاملاً، **ثم** ابدأ الثانية». لا يوجد تزامن حقيقي هنا بعد! لتحقيق التزامن الفعلي، نحتاج أداة إضافية: `asyncio.gather()` .

17.4 تحقيق التزامن الحقيقي: `asyncio.gather()`



الزمن الكلي يقارب أطول مهمة، لا مجموع المهام.

شكل 17.2 — متسلسل مقابل asyncio.gather على خط زمني

Python

```python
import asyncio
import time

async def fetch_data(source: str, delay: float) -> str:
    print(f"🔄 بدء تحميل البيانات من {source}.  ")
    await asyncio.sleep(delay)
    print(f"✅ انتهى تحميل البيانات من {source}")
    return f"بيانات من {source}"

async def main() -> None:
    start = time.perf_counter()

    results = await asyncio.gather(
        fetch_data("الخادم أ", 2),
        fetch_data("الخادم ب", 2),
        fetch_data("الخادم ج", 2),
    )

    elapsed = time.perf_counter() - start
    print(f"\nالنتائج: {results}")
    print(f"الوقت الإجمالي: {elapsed:.2f} ثانية")

asyncio.run(main())
```

الناتج:

```
🔄 بدء تحميل البيانات من الخادم أ.
🔄 بدء تحميل البيانات من الخادم ب.
🔄 بدء تحميل البيانات من الخادم ج.
✅ انتهى تحميل البيانات من الخادم أ
✅ انتهى تحميل البيانات من الخادم ب
✅ انتهى تحميل البيانات من الخادم ج

النتائج: ['بيانات من الخادم أ', 'بيانات من الخادم ب', 'بيانات من الخادم ج']
الوقت الإجمالي: 2.00 ثانية
```

هذا هو الفرق المذهل! الآن الثلاث عمليات **بدأت في نفس الوقت تقريبًا**، وانتظرنا جميعها معًا، فكان الوقت الإجمالي **2 ثانية فقط** (بدلًا من 6)! هذا لأن `asyncio.gather()` تُشغّل كل الـ Coroutines الممرَّرة إليها بشكل متزامن (Concurrent)، وتنتظرها جميعًا، ثم تُرجع كل النتائج في قائمة واحدة، **بنفس ترتيب** الـ Coroutines الأصلية.

> 💡 **نصيحة الخبير:** `asyncio.gather()` هي الأداة الأكثر استخدامًا عمليًا في `asyncio`. القاعدة الذهبية: **لا تستخدم `await` بشكل متتالٍ لعمليات مستقلة عن بعضها البعض** — استخدم `gather()` لتشغيلها معًا. استخدم `await` المتتالي فقط عندما تحتاج فعليًا نتيجة العملية الأولى **قبل** أن تبدأ الثانية (اعتمادية حقيقية بين العمليات).

فهم ما يحدث «خلف الكواليس»: حلقة الأحداث (Event Loop)

لفهم السحر الذي يحدث، تخيّل **حلقة الأحداث (Event Loop)** كمدير مهام ذكي جدًا:

1. يبدأ تنفيذ `fetch_data("2 ,"الخادم أ")`، يصل إلى `await asyncio.sleep(2)`.
2. بدلًا من التجميد والانتظار، يقول Event Loop: «حسنًا، هذه المهمة ستنتظر ثانيتين، دعني أضعها جانبًا وأتفقد مهام أخرى جاهزة للعمل الآن».
3. يبدأ `fetch_data("2 ,"الخادم ب")`، ثم `fetch_data("2 ,"الخادم ج")`، بنفس المنطق.
4. بعد ثانيتين تقريبًا، تكون كل المهام الثلاث قد «انتهت من انتظارها»، فيعود Event Loop لإكمال كل واحدة بالترتيب الذي أصبحت فيه جاهزة.

هذا كله يحدث في **خيط تنفيذ واحد فقط (Single Thread)**! لا يوجد تعدد معالجات حقيقي هنا، بل **تبديل ذكي** بين المهام المنتظرة، يستغل وقت الانتظار (الذي كان سيُهدَر) لتنفيذ مهام أخرى.

> **مقارنة: التزامن (Concurrency) مقابل التوازي (Parallelism)**
> - **Concurrency (تزامن)**: عدة مهام «تتقدم معًا» خلال فترة زمنية، لكن ليس بالضرورة في نفس اللحظة الدقيقة (قد يكون التبديل بينها سريعًا جدًا فيبدو الأمر متزامنًا). هذا ما يفعله `asyncio`.
> - **Parallelism (توازي)**: عدة مهام تُنفَّذ **فعليًا في نفس اللحظة** على أنوية معالج مختلفة (هذا ما يوفره `multiprocessing`).
> - `asyncio` مثالي لمهام I/O-bound الكثيرة (شبكة، ملفات) لأن وقت الانتظار يُستغل بذكاء. لكنه **لا يُسرّع** مهام CPU-bound (حسابات ثقيلة)، لأن هذه المهام تحتاج فعلاً وقت معالج حقيقي، لا مجرد «انتظار».

17.5 مثال عملي حقيقي: تحميل بيانات من الإنترنت بـ `aiohttp`

حتى الآن استخدمنا `asyncio.sleep()` كمحاكاة. لنرَ مثالًا حقيقيًا: تحميل محتوى عدة صفحات ويب في آن واحد، باستخدام مكتبة `aiohttp` (نسخة غير متزامنة من مكتبات طلبات HTTP، بعكس `requests` التي سنستخدمها في الفصل 21 وهي متزامنة بطبيعتها).

التثبيت

```
Terminal
uv add aiohttp
```

المثال

```python
import asyncio
import time
import aiohttp

async def fetch_url(session: aiohttp.ClientSession, url: str) -> int:
    """يُحمّل صفحة ويب ويُرجع رمز الحالة (Status Code) فقط، للتبسيط."""
    async with session.get(url) as response:
        return response.status

async def main() -> None:
    urls = [
        "https://www.python.org",
        "https://www.github.com",
        "https://www.wikipedia.org",
    ]

    start = time.perf_counter()

    async with aiohttp.ClientSession() as session:
        tasks = [fetch_url(session, url) for url in urls]
        results = await asyncio.gather(*tasks)

    elapsed = time.perf_counter() - start

    for url, status in zip(urls, results):
        print(f"{url} → {status}")

    print(f"\nالوقت الإجمالي: {elapsed:.2f} ثانية")

asyncio.run(main())
```

لاحظ النقاط التالية:

- `async with` تُستخدم مع `aiohttp.ClientSession()` — نسخة غير متزامنة من `with` التي تعلمناها في الفصل 16، تعمل مع Context Managers غير متزامنة (`__aenter__` / `__aexit__`).
- `tasks = [fetch_url(session, url) for url in urls]` : قائمة من Coroutines (لم تُشغَّل بعد).
- `*tasks` : نستخدم عامل فك التغليف (من الفصل 10) لتمرير كل عناصر القائمة كمعاملات منفصلة لـ `gather()`.

تشغيل هذا البرنامج سيُحمّل الصفحات الثلاث **في نفس الوقت تقريبًا**، بدلًا من تحميلها واحدة تلو الأخرى، مما يُقلل الوقت الإجمالي بشكل ملحوظ (خاصة مع عدد أكبر من الروابط).

نصيحة الخبير: كل مكتبة I/O «عادية» (متزامنة) لها عادة «نسخة توأم» غير متزامنة: `requests` ↔ `aiohttp` أو `psycopg2` ↔ `asyncpg` لقواعد بيانات PostgreSQL، إلخ. عند العمل في بيئة `async` (مثل تطبيق FastAPI)، **يجب** استخدام النسخ غير المتزامنة من كل مكتبة، وإلا ستُجمّد استدعاءاتها المتزامنة حلقة الأحداث بالكامل وتُلغي كل فائدة `asyncio`!

⚠ **خطأ فادح وشائع جدًا:** استخدام دالة متزامنة بطيئة (مثل `time.sleep()` أو `requests.get()`) داخل Coroutine! هذا يُجمّد **حلقة الأحداث بأكملها**، فتتوقف كل المهام الأخرى المنتظرة أيضًا، مما يُلغي كل فائدة `async` تمامًا:

```python
import time

async def bad_example() -> None:
    time.sleep(2)  # ❌ خطأ فادح! يُجمّد حلقة الأحداث بأكملها

async def good_example() -> None:
    await asyncio.sleep(2)  # ✅ صحيح، يسمح بتبديل المهام
```

17.6 `asyncio.create_task()`: بدء المهام دون انتظار فوري

أحيانًا تريد أن «تبدأ» عملية غير متزامنة، لكن تريد الاستمرار في عمل آخر، والعودة لنتيجتها لاحقًا. هنا تأتي `asyncio.create_task()`:

```python
import asyncio

async def background_job(name: str, delay: float) → str:
    print(f"🚀 بدأت المهمة: {name}")
    await asyncio.sleep(delay)
    print(f"🏁 انتهت المهمة: {name}")
    return f"نتيجة {name}"

async def main() → None:
    # إنشاء المهمة تبدأ تنفيذها فورًا "في الخلفية"
    task1 = asyncio.create_task(background_job("3 ,"أ"))
    task2 = asyncio.create_task(background_job("1 ,"ب"))

    print("البرنامج يستمر بعمل أشياء أخرى بينما المهام تعمل في الخلفية.  ")
    await asyncio.sleep(0.5)
    print("لا يزال البرنامج يعمل.  ")

    # الآن ننتظر نتائج المهام عند الحاجة الفعلية إليها
    result1 = await task1
    result2 = await task2

    print(result1, result2)

asyncio.run(main())
```

الفرق بين `create_task()` و `gather()` : `create_task()` يُنشئ المهمة وتبدأ **فورًا** في الخلفية، تاركًا إياك حرًا لتنفيذ كود آخر قبل أن تنتظر نتيجتها بـ `await` لاحقًا. بينما `gather()` تُشغّل كل الـ Coroutines **وتنتظرها معًا** في نفس اللحظة دون القيام بأي شيء آخر بينهما.

17.7 معالجة الأخطاء في الكود غير المتزامن

الأخبار الجيدة: `try/except` تعمل بنفس الطريقة تمامًا مع الكود غير المتزامن!

Python

```python
import asyncio

async def risky_operation(should_fail: bool) -> str:
    await asyncio.sleep(1)
    if should_fail:
        raise ValueError("حدث خطأ أثناء العملية!")
    return "نجحت العملية"

async def main() -> None:
    try:
        result = await risky_operation(should_fail=True)
        print(result)
    except ValueError as e:
        print(f"تم اصطياد الخطأ: {e}")

asyncio.run(main())
```

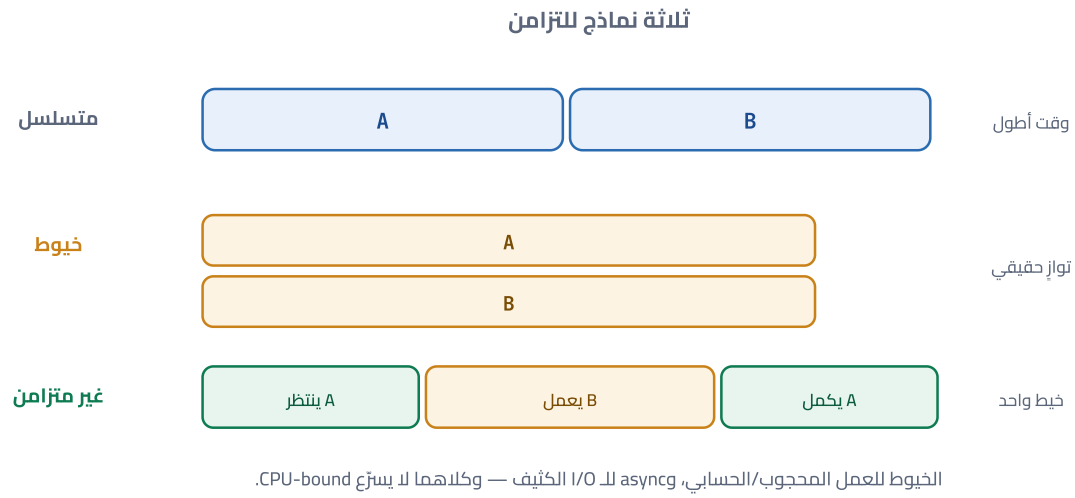
التعامل مع الأخطاء عند استخدام `gather()`

Python

```python
import asyncio

async def task_that_fails() -> str:
    await asyncio.sleep(1)
    raise ValueError("فشلت هذه المهمة!")

async def task_that_succeeds() -> str:
    await asyncio.sleep(1)
    return "نجحت هذه المهمة"

async def main() -> None:
    results = await asyncio.gather(
        task_that_succeeds(),
        task_that_fails(),
        return_exceptions=True,  # ← مهم جدًا!
    )

    for result in results:
        if isinstance(result, Exception):
            print(f"خطأ: {result}")
        else:
            print(f"نجاح: {result}")

asyncio.run(main())
```

⚠ **تحذير مهم:** بدون `return_exceptions=True`، فإن `asyncio.gather()` تتوقف فورًا وتُطلق الاستثناء الأول الذي يحدث في **أي** مهمة من المهام، حتى لو كانت المهام الأخرى لا تزال تعمل (قد تستمر في الخلفية دون أن تحصل على نتائجها). باستخدام `return_exceptions=True`، تحصل على **كل** النتائج، بما فيها كائنات الاستثناءات نفسها، لتتعامل معها بنفسك واحدة تلو الأخرى.

17.8 متى تستخدم Async، ومتى لا تحتاجه إطلاقًا؟



شكل 17.3 — متسلسل وخيوط وغير متزامن

🔄 جدول القرار: Sync أم Async؟

الموقف	الخيار المناسب	السبب
حسابات رياضية ثقيلة (معالجة صور، تدريب نماذج AI)	Sync (أو `multiprocessing`)	CPU-bound، لا يوجد «انتظار» مفيد لاستغلاله
استدعاء عشرات/مئات طلبات API	**Async**	I/O-bound، الانتظار هو عنق الزجاجة
قراءة ملف واحد صغير	Sync	لا فائدة من التعقيد الإضافي لعملية واحدة بسيطة
خادم ويب يخدم آلاف الطلبات المتزامنة	**Async** (مثل FastAPI)	يسمح بخدمة طلبات كثيرة بخيط واحد بكفاءة عالية
سكريبت بسيط يُشغَّل مرة واحدة ويقرأ ملفًا	Sync	البساطة أهم هنا من كل تحسين أداء غير ملموس
Web Scraping لمئات الصفحات (الفصل 21)	**Async**	تسريع هائل بسبب كثرة طلبات الشبكة المستقلة

⚠️ **تحذير من «فخ التعقيد غير المبرر»:** لا تستخدم `async/await` في كل مكان «لأنها حديثة وعصرية»! إن كان برنامجك يقوم بعملية واحدة فقط، أو عمليات متسلسلة تعتمد كل منها على سابقتها، فإن Async **تضيف تعقيدًا بلا أي فائدة حقيقية**. اسأل نفسك دائمًا: «هل لدي عدة عمليات I/O **مستقلة** يمكن أن تعمل معًا؟» إن كانت الإجابة لا، التزم بالكود المتزامن البسيط الذي أتقنته طوال هذا الكتاب.

💡 **نصيحة الخبير:** لا يمكنك ولا يجب عليك «تحويل» كل كودك إلى Async. عمليًا، تجد غالبًا **نقطة دخول واحدة غير متزامنة** (مثل خادم الويب في FastAPI) تستدعي من داخلها كودًا متزامنًا عاديًا للعمليات التي لا تحتاج async (كالحسابات البسيطة)، وتستخدم `await` فقط عند التعامل مع عمليات I/O فعلية (قاعدة بيانات، طلبات خارجية).

17.9 مشروع تطبيقي: مُجمِّع أسعار من عدة «متاجر» (محاكاة)

لنبني مشروعًا يُحاكي سيناريو واقعيًا جدًا: تطبيق يقارن أسعار منتج واحد من عدة متاجر إلكترونية مختلفة، عبر استدعاءات شبكة (محاكاة)، مستفيدًا من التزامن لتسريع العملية بشكل كبير.

المتطلبات

1. محاكاة استدعاء "API" لكل متجر (بأوقات استجابة مختلفة، بل وبعضها قد «يفشل»).
2. تشغيل كل الاستدعاءات بشكل متزامن باستخدام `asyncio.gather()`.
3. معالجة أخطاء المتاجر الفاشلة دون إيقاف باقي العملية.

4. عرض تقرير نهائي: أرخص سعر، أغلى سعر، والمتوسط.
5. قياس ومقارنة الوقت المستغرق بين النسخة المتزامنة (Sync) وغير المتزامنة (Async).

✅ الحل النموذجي الكامل

Python

```
"""
مُجمِّع أسعار - مقارنة أداء Sync مقابل Async
"""
from __future__ import annotations

import asyncio
import random
import time
from dataclasses import dataclass

@dataclass
class StoreResult:
    store_name: str
    price: float | None  # None يعني فشل الاستعلام
    error: str | None = None

# ---------------------------------------------------------
# محاكاة استدعاء API (غير متزامن) لمتجر واحد
# ---------------------------------------------------------
async def fetch_price_from_store(store_name: str, base_delay: float) -> StoreResult:
    """يُحاكي استعلام سعر من متجر معيّن، مع احتمال فشل عشوائي."""
    # محاكاة تذبذب في زمن الاستجابة (كالشبكة الحقيقية)
    actual_delay = base_delay + random.uniform(-0.3, 0.3)
    await asyncio.sleep(max(actual_delay, 0.1))

    # محاكاة فشل عشوائي بنسبة 20%
    if random.random() < 0.2:
        return StoreResult(store_name, price=None, error="انتهت مهلة الاتصال")

    price = round(random.uniform(80, 150), 2)
    return StoreResult(store_name, price=price)

# ---------------------------------------------------------
# النسخة غير المتزامنة: كل المتاجر معًا
# ---------------------------------------------------------
async def compare_prices_async(stores: dict[str, float]) -> list[StoreResult]:
    tasks = [
        fetch_price_from_store(name, delay)
        for name, delay in stores.items()
    ]
    return await asyncio.gather(*tasks)

# ---------------------------------------------------------
# النسخة المتزامنة التقليدية (للمقارنة فقط)
```

```
# ------------------------------------------------------------
def fetch_price_sync(store_name: str, base_delay: float) -> StoreResult:
    actual_delay = base_delay + random.uniform(-0.3, 0.3)
    time.sleep(max(actual_delay, 0.1))

    if random.random() < 0.2:
        return StoreResult(store_name, price=None, error="انتهت مهلة الاتصال")

    price = round(random.uniform(80, 150), 2)
    return StoreResult(store_name, price=price)

def compare_prices_sync(stores: dict[str, float]) -> list[StoreResult]:
    return [fetch_price_sync(name, delay) for name, delay in stores.items()]

# ------------------------------------------------------------
# تحليل وعرض النتائج
# ------------------------------------------------------------
def print_report(results: list[StoreResult]) -> None:
    print("\n--- تقرير الأسعار ---")
    successful: list[StoreResult] = []

    for result in results:
        if result.price is not None:
            print(f"  ✅ {result.store_name}: {result.price} دولار")
            successful.append(result)
        else:
            print(f"  ❌ {result.store_name}: فشل ({result.error})")

    if not successful:
        print("لم ينجح أي استعلام!")
        return

    prices = [r.price for r in successful if r.price is not None]
    cheapest = min(successful, key=lambda r: r.price or float("inf"))
    most_expensive = max(successful, key=lambda r: r.price or 0)

    print(f"\n💰 الأرخص: {cheapest.store_name} ({cheapest.price} دولار)")
    print(f"🏷️ الأغلى: {most_expensive.store_name} ({most_expensive.price} دولار)")
    print(f"📊 المتوسط: {sum(prices) / len(prices):.2f} دولار")

# ------------------------------------------------------------
# البرنامج الرئيسي: مقارنة الأداء
# ------------------------------------------------------------
async def main() -> None:
    stores: dict[str, float] = {
        "متجر الأمانة": 1.5,
        "سوق النجاح": 2.0,
        "بازار الخير": 1.0,
        "متجر السرعة": 0.8,
        "سوق التوفير": 1.8,
    }
```

```
    print("=" * 50)
    print("🐢 تشغيل النسخة المتزامنة (Sync).  ")
    start_sync = time.perf_counter()
    sync_results = compare_prices_sync(stores)
    sync_elapsed = time.perf_counter() - start_sync
    print_report(sync_results)
    print(f"\n⏱  وقت النسخة المتزامنة: {sync_elapsed:.2f} ثانية")

    print("\n" + "=" * 50)
    print("🚀 تشغيل النسخة غير المتزامنة (Async).  ")
    start_async = time.perf_counter()
    async_results = await compare_prices_async(stores)
    async_elapsed = time.perf_counter() - start_async
    print_report(async_results)
    print(f"\n⏱  وقت النسخة غير المتزامنة: {async_elapsed:.2f} ثانية")

    print("\n" + "=" * 50)
    speedup = sync_elapsed / async_elapsed if async_elapsed > 0 else 0
    print(f"🎯 التسريع المحقق: {speedup:.1f}x أسرع!")

if __name__ == "__main__":
    asyncio.run(main())
```

نتيجة متوقعة (الأرقام الدقيقة تختلف بسبب العشوائية، لكن الفارق الكبير في الوقت ثابت):

```
⏱  وقت النسخة المتزامنة: 7.21 ثانية
⏱  وقت النسخة غير المتزامنة: 2.05 ثانية
🎯 التسريع المحقق: 3.5x أسرع!
```

> 💡 لاحظ كيف أن النسخة المتزامنة استغرقت تقريبًا **مجموع** كل أوقات الانتظار، بينما النسخة غير المتزامنة استغرقت تقريبًا **وقت أطول تأخير فردي فقط** — هذا هو بالضبط الفرق الذي يجعل Async خيارًا حاسمًا في تطبيقات الويب الحديثة وأي نظام يتعامل مع عمليات شبكة متعددة.

17.10 تمارين

🟢 مستوى مبتدئ

تمرين 1: اكتب Coroutine باسم `countdown(n: int) → None` تطبع عدًا تنازليًا من `n` إلى 1، مع `await asyncio.sleep(1)` بين كل رقم والذي يليه. شغّلها باستخدام `asyncio.run()`.

تمرين 2: ما الفرق بين `await func()` و `asyncio.create_task(func())`؟ اشرح بجملة واحدة لكل منهما.

تمرين 3: صحّح هذا الكود الذي يحتوي خطأ شائعًا:

```python
async def get_data() -> str:
    return "بيانات"

def main():
    result = get_data()
    print(result)

main()
```

✅ الحل النموذجي

Python

```python
# تمرين 1
import asyncio

async def countdown(n: int) -> None:
    for i in range(n, 0, -1):
        print(i)
        await asyncio.sleep(1)
    print("انتهى!")

asyncio.run(countdown(5))

# تمرين 2
# await func(): ينفّذ الـ Coroutine وينتظر نتيجتها فورًا قبل المتابعة.
# asyncio.create_task(func()): يبدأ تنفيذ الـ Coroutine "فورًا" في الخلفية،
# ويسمح لك بمتابعة كود آخر، على أن تنتظر نتيجتها لاحقًا بـ await عند الحاجة.

# تمرين 3 - الكود المصحح
import asyncio

async def get_data() -> str:
    return "بيانات"

async def main() -> None:
    result = await get_data()  # يجب await لتنفيذ الـ Coroutine فعليًا
    print(result)

asyncio.run(main())  # يجب استدعاء main عبر asyncio.run، وليس مباشرة
```

🟡 مستوى متوسط

تمرين 4: اكتب 3 Coroutines تُحاكي تحميل: صورة (3 ثوانٍ)، فيديو (5 ثوانٍ)، ومستند (1 ثانية). شغّلها معًا بـ `asyncio.gather()`، واطبع الوقت الإجمالي. تأكد أن الوقت الإجمالي قريب من 5 ثوانٍ فقط (أطول عملية)، وليس 9 ثوانٍ (المجموع).

تمرين 5: عدّل المثال السابق بحيث تفشل عملية «تحميل الفيديو» (ترفع `ConnectionError`). استخدم `return_exceptions=True` في `gather()`، واطبع رسالة مناسبة لكل عملية، سواء نجحت أم فشلت.

تمرين 6: اكتب دالة `fetch_with_timeout(delay: float, timeout: float) → str` تستخدم `asyncio.wait_for()` لفرض مهلة قصوى على عملية `asyncio.sleep(delay)`. إن تجاوز `delay` قيمة `timeout`، يجب اصطياد `asyncio.TimeoutError` وإرجاع رسالة «انتهت المهلة».

✅ الحل النموذجي

Python

```python
# تمرين 4
import asyncio
import time

async def download_image() -> str:
    await asyncio.sleep(3)
    return "تم تحميل الصورة"

async def download_video() -> str:
    await asyncio.sleep(5)
    return "تم تحميل الفيديو"

async def download_document() -> str:
    await asyncio.sleep(1)
    return "تم تحميل المستند"

async def main() -> None:
    start = time.perf_counter()
    results = await asyncio.gather(
        download_image(), download_video(), download_document()
    )
    elapsed = time.perf_counter() - start
    print(results)
    print(f"الوقت: {elapsed:.2f} ثانية")  # قريب من 5 ثوانٍ

asyncio.run(main())

# تمرين 5
import asyncio

async def download_image() -> str:
    await asyncio.sleep(3)
    return "تم تحميل الصورة"

async def download_video() -> str:
    await asyncio.sleep(2)
    raise ConnectionError("فشل الاتصال بخادم الفيديو")

async def download_document() -> str:
    await asyncio.sleep(1)
    return "تم تحميل المستند"

async def main() -> None:
    results = await asyncio.gather(
```

```
            download_image(), download_video(), download_document(),
            return_exceptions=True,
        )
        for result in results:
            if isinstance(result, Exception):
                print(f"❌ فشل: {result}")
            else:
                print(f"✅ نجح: {result}")

asyncio.run(main())

# تمرين 6
import asyncio

async def fetch_with_timeout(delay: float, timeout: float) -> str:
    try:
        async def task() -> str:
            await asyncio.sleep(delay)
            return "تم التحميل بنجاح"
        return await asyncio.wait_for(task(), timeout=timeout)
    except asyncio.TimeoutError:
        return "انتهت المهلة"

async def main() -> None:
    print(await fetch_with_timeout(1, 3))   # "تم التحميل بنجاح"
    print(await fetch_with_timeout(5, 2))   # "انتهت المهلة"

asyncio.run(main())
```

🔴 تحدي

تمرين 7: ابنِ «نظام فحص صحة خوادم (Health Check)»:

- دالة `check_server(name: str, is_healthy: bool, delay: float) → dict[str, object]` تُحاكي فحص خادم، تُرجع قاموسًا `{"name": ..., "status": "healthy"/"down", "response_time": ...}`.
- دالة `check_all_servers(servers: list[tuple[str, bool, float]]) → list[dict[str, object]]` تفحص كل الخوادم معًا (Async)، وتُرجع النتائج.
- دالة تحليلية تطبع: عدد الخوادم السليمة، عدد المعطلة، وأبطأ خادم استجابةً من بين السليمة.
- استخدم `asyncio.wait_for()` لفرض مهلة قصوى 3 ثوانٍ على كل فحص؛ إن تجاوزها الخادم، يُعتبر "down" بسبب "timeout".

✅ الحل النموذجي

Python

```python
import asyncio
from typing import TypedDict

class ServerStatus(TypedDict):
    name: str
    status: str
    response_time: float | None

async def check_server(name: str, is_healthy: bool, delay: float) -> ServerStatus:
    async def do_check() -> float:
        await asyncio.sleep(delay)
        if not is_healthy:
            raise ConnectionError("الخادم لا يستجيب")
        return delay

    try:
        response_time = await asyncio.wait_for(do_check(), timeout=3.0)
        return {"name": name, "status": "healthy", "response_time": response_time}
    except asyncio.TimeoutError:
        return {"name": name, "status": "down", "response_time": None}
    except ConnectionError:
        return {"name": name, "status": "down", "response_time": None}

async def check_all_servers(
    servers: list[tuple[str, bool, float]]
) -> list[ServerStatus]:
    tasks = [check_server(name, healthy, delay) for name, healthy, delay in servers]
    return await asyncio.gather(*tasks)

def analyze_results(results: list[ServerStatus]) -> None:
    healthy = [r for r in results if r["status"] == "healthy"]
    down = [r for r in results if r["status"] == "down"]

    print(f"✅ الخوادم السليمة: {len(healthy)}")
    for server in healthy:
        print(f"   {server['name']} - {server['response_time']:.2f}s")

    print(f"❌ الخوادم المعطلة: {len(down)}")
    for server in down:
        print(f"   {server['name']}")

    if healthy:
        slowest = max(healthy, key=lambda r: r["response_time"] or 0)
        print(f"\n🐢 أبطأ خادم سليم: {slowest['name']} ({slowest['response_time']:.2f}s)")
```

```
async def main() -> None:
    servers = [
        ("خادم الويب الرئيسي", True, 0.5),
        ("خادم قاعدة البيانات", True, 1.2),
        ("خادم الكاش", False, 1.0),
        ("خادم البريد", True, 4.0),  # (3 ثوانٍ) سيتجاوز المهلة → down
    ]

    results = await check_all_servers(servers)
    analyze_results(results)

if __name__ == "__main__":
    asyncio.run(main())
```

17.11 اختبر فهمك (Quiz)

❓ السؤال 1: ما الفرق بين I/O-bound و CPU-bound؟ ولماذا يهم هذا الفرق عند اختيار Async؟

I/O-bound: العملية تقضي معظم وقتها في الانتظار (شبكة، قرص صلب) وليس في معالجة فعلية. CPU-bound: العملية تستهلك فعليًا قدرة المعالج (حسابات معقّدة). Async مفيد جدًا للحالة الأولى لأنه يستغل وقت الانتظار لتنفيذ مهام أخرى، لكنه لا يُسرّع الحالة الثانية لأن المعالج يعمل فعليًا طوال الوقت دون «انتظار» يمكن استغلاله.

❓ السؤال 2: ماذا يحدث عند استدعاء دالة `async def` دون استخدام `await` أو `asyncio.run()`؟

لا يُنفَّذ جسم الدالة إطلاقًا! الاستدعاء يُنشئ فقط كائن Coroutine (وصفة جاهزة للتنفيذ)، لكنه لا يُشغّل الكود الفعلي حتى يُمرَّر لـ `await` داخل Coroutine أخرى، أو إلى `asyncio.run()` كنقطة دخول رئيسية.

❓ السؤال 3: لماذا يُعتبر استخدام `time.sleep()` داخل دالة `async def` خطأً فادحًا؟

لأن `time.sleep()` دالة متزامنة (Blocking) تُجمّد **حلقة الأحداث بأكملها**، مانعة أي Coroutine أخرى من العمل أثناء الانتظار، مما يُلغي كل فائدة استخدام async أصلًا. يجب استخدام `asyncio.sleep()` بدلًا منها، لأنها تسمح لحلقة الأحداث بالتبديل إلى مهام أخرى جاهزة أثناء الانتظار.

❓ السؤال 4: ما الفرق بين تنفيذ عدة `await` متتالية وتنفيذها عبر `asyncio.gather()`؟

تنفيذ `await` متتالية (كل واحدة في سطر منفصل) يعني انتظار كل عملية بالكامل قبل بدء التي تليها — لا تزامن حقيقي، والوقت الإجمالي هو مجموع كل الأوقات. بينما `asyncio.gather()` تُشغّل كل الـ Coroutines الممرَّرة إليها في نفس الوقت تقريبًا، فيكون الوقت الإجمالي قريبًا من أطول عملية فردية فقط، وليس مجموعها.

❓ السؤال 5: متى يجب أن تتجنّب استخدام async/await في مشروعك؟

عندما لا توجد عمليات I/O متعددة ومستقلة يمكن تنفيذها بالتزامن (مثل سكريبت بسيط يقوم بعملية واحدة، أو عمليات متسلسلة يعتمد بعضها على نتيجة سابقتها بشكل كامل)، أو عند التعامل مع مهام CPU-bound ثقيلة حسابيًا، حيث لا توجد فائدة حقيقية من async وتضيف تعقيدًا غير مبرر.

17.12 ملخص الفصل (Cheat Sheet)

```python
import asyncio

# ===== تعريف وتشغيل Coroutine =====
async def my_function() -> str:
    await asyncio.sleep(1)   # انتظار غير متزامن (وليس time.sleep!)
    return "النتيجة"

asyncio.run(my_function())   # نقطة الدخول الرئيسية - تُستدعى مرة واحدة فقط

# ===== انتظار متتالٍ (عند الاعتماد الحقيقي بين العمليات) =====
async def main() -> None:
    result1 = await my_function()
    result2 = await my_function()  # يبدأ فقط بعد انتهاء الأولى
```

```python
# ===== تزامن حقيقي (عمليات مستقلة) =====
async def main() -> None:
    results = await asyncio.gather(
        my_function(),
        my_function(),
        return_exceptions=True,  # لمنع توقف الكل عند فشل واحدة
    )

# ===== بدء مهمة في الخلفية =====
async def main() -> None:
    task = asyncio.create_task(my_function())
    # ... كود آخر يعمل الآن..
    result = await task  # ننتظر النتيجة عند الحاجة فقط

# ===== فرض مهلة قصوى =====
try:
    result = await asyncio.wait_for(my_function(), timeout=2.0)
except asyncio.TimeoutError:
    print("انتهت المهلة")

# ===== معالجة الأخطاء =====
try:
    result = await risky_coroutine()
except SomeError as e:
    print(f"خطأ: {e}")
```

المفهوم	الاستخدام
`async def`	تعريف Coroutine Function
`await`	تنفيذ وانتظار Coroutine أخرى (داخل async def فقط)
`asyncio.run()`	نقطة الدخول - تُستدعى مرة واحدة
`asyncio.sleep()`	انتظار غير متزامن (بديل `time.sleep`)
`asyncio.gather()`	تشغيل عدة Coroutines بالتزامن وانتظار الكل
`asyncio.create_task()`	بدء مهمة في الخلفية فورًا
`asyncio.wait_for()`	فرض مهلة قصوى على عملية
`return_exceptions=True`	منع توقف `gather()` عند أول خطأ

17.13 ماذا بعد؟

لقد فتحت للتو بابًا أساسيًا في عالم Python الحديث: فهم متى ولماذا نحتاج التزامن، وكيف نكتبه بأمان عبر `async/await`. هذه المهارة ستكون **جوهرية** عندما نبني أول واجهة API حقيقية في الفصل 19 باستخدام FastAPI، حيث يعتمد كل شيء تقريبًا على هذا النمط.

لكن قبل أن ننتقل لبناء مشاريع حقيقية أكبر، هناك مهارة عملية أساسية لا بد من إتقانها أولًا: كيف نُنظّم مشاريعنا بشكل احترافي؟ كيف نُدير التبعيات (المكتبات الخارجية) بطريقة نظيفة وقابلة للمشاركة مع الآخرين؟ وكيف نستخدم أداة التحكم بالإصدارات **Git** — الأداة التي يعتمد عليها كل مطور محترف في العالم لحفظ تاريخ تطور كوده والتعاون مع فريقه؟

في الفصل الثامن عشر: **إدارة المشاريع كمحترف**، سنتعمق في Virtual Environments، أداة `uv` بكل إمكانياتها (التي استخدمناها بشكل أساسي طوال الكتاب)، هيكلة مشروع Python احترافية قياسية، وأساسيات Git الضرورية لكل مبرمج.

فصل 18 / 26 · الجزء الرابع

18

إدارة المشاريع كمحترف

16 دقيقة قراءة · 10 قسمًا · 7 مثالًا · 3,109 كلمة

في هذا الفصل

18.1 لماذا يستحق هذا الفصل اهتمامك الكامل؟
18.2 الجزء الأول: إدارة البيئات والتبعيات مع uv
18.3 الجزء الثاني: هيكلة مشروع احترافية
18.4 الجزء الثالث: أساسيات Git
18.5 أفضل الممارسات الشاملة
18.6 مشروع تطبيقي: تجهيز مشروع كامل من الصفر
18.7 تمارين
18.8 اختبر فهمك (Quiz)
18.9 ملخص الفصل (Cheat Sheet)
18.10 ماذا بعد؟

18.1 لماذا يستحق هذا الفصل اهتمامك الكامل؟

تخيّل أنك انتهيت للتو من بناء مشروع رائع بـ Python، وتريد مشاركته مع صديق ليجربه. ترسل له ملف `main.py` ... فيحاول تشغيله، فيظهر له:

```
ModuleNotFoundError: No module named 'requests'
```

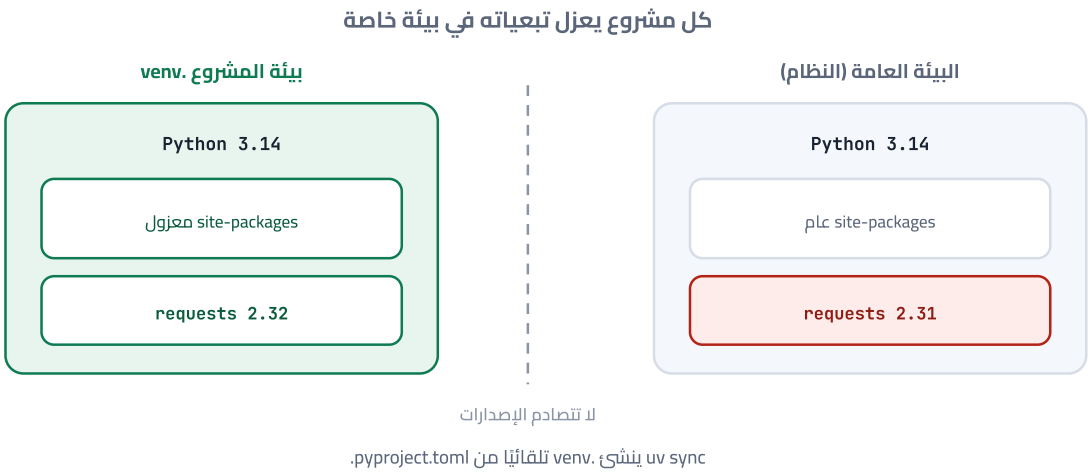
يُثبّت `requests`، يُشغّل البرنامج مجددًا، يظهر خطأ آخر من مكتبة مختلفة! بعد نصف ساعة من التثبيت اليدوي المتكرر، يكتشف أن إصدار مكتبة لديه مختلف عن إصدارك، والكود يتصرف بشكل مختلف تمامًا!

هذا بالضبط ما تحله **إدارة المشاريع الاحترافية**. وتخيّل سيناريو آخر: تعمل على ميزة جديدة في مشروعك، تُفسد شيئًا عن طريق الخطأ، وتتمنى لو تستطيع «الرجوع بالزمن» إلى النسخة التي كانت تعمل بشكل صحيح قبل ساعة. هذا بالضبط ما يحله **Git**.

هذا الفصل مختلف قليلًا عن سابقيه: لن نتعلم مفاهيم برمجية جديدة في اللغة نفسها، بل سنتعلم **الأدوات والممارسات** التي تفصل بين «شخص يكتب سكريبتات Python للمتعة» و«مبرمج محترف يبني برامج قابلة للمشاركة، والصيانة، والتطوير الجماعي». هذه المهارات ضرورية ومُسلَّم بها في كل بيئة عمل حقيقية، وستحتاجها بشدة في كل الفصول القادمة من الكتاب، خاصة عند بناء مشاريع الويب وواجهات API.

> **نصيحة الخبير:** كثير من المبرمجين المبتدئين يتجاهلون هذا الجانب لأنه «لا يبدو كبرمجة فعلية»، لكن الحقيقة أن إتقان إدارة المشاريع وGit يوفر عليك ساعات (وربما أيامًا) من الإحباط لاحقًا، ويجعلك قادرًا على العمل ضمن فريق بثقة تامة.

18.2 الجزء الأول: إدارة البيئات والتبعيات مع uv



شكل 18.1 — عزل تبعيات المشروع في .venv

لماذا نحتاج Virtual Environments؟ (مراجعة وتعميق)

تذكر من الفصل الثاني أننا تحدثنا بإيجاز عن **البيئات الافتراضية (Virtual Environments)**. الآن حان وقت فهمها بعمق حقيقي.

المشكلة: التعارض بين المشاريع

تخيّل أن لديك مشروعين على حاسوبك:
- **المشروع أ:** يحتاج `pandas` الإصدار 1.5 (لأن كودًا قديمًا فيه يعتمد على سلوك معيّن تغيّر لاحقًا).
- **المشروع ب:** يحتاج `pandas` الإصدار 2.2 (لاستخدام ميزة جديدة).

إن ثبّتَّ `pandas` «عالميًا» على حاسوبك (Globally)، فلا يمكنك أن يكون لديك إصدارين مختلفين في نفس الوقت! تثبيت الإصدار الجديد سيُفسد المشروع الأول، والعكس صحيح.

الحل: لكل مشروع **بيئته الخاصة المعزولة تمامًا**، بمكتباتها وإصداراتها الخاصة، دون أي تداخل مع مشاريع أخرى على نفس الجهاز.

```
حاسوبك
├── Python (التثبيت العام - لا نستخدمه مباشرة للمشاريع!)
├── المشروع أ/
│   └── .venv/  ← بيئة معزولة خاصة بهذا المشروع فقط (pandas 1.5)
└── المشروع ب/
    └── .venv/  ← بيئة معزولة خاصة بهذا المشروع فقط (pandas 2.2)
```

⚠ **خطأ شائع جدًا عند المبتدئين:** تثبيت كل المكتبات مباشرة على النظام العام (`pip install` بدون بيئة افتراضية). هذا يؤدي حتمًا، عاجلًا أو آجلًا، إلى تعارضات بين المشاريع، ويجعل مشاركة مشروعك مع الآخرين أمرًا شبه مستحيل (لأنك لا تعرف بالضبط ما الذي يحتاجه مشروعك فعليًا من بين عشرات المكتبات المثبتة عالميًا على جهازك فقط).

لماذا `uv` تحديدًا؟

اعتمدنا `uv` منذ الفصل الثاني كأداة إدارة الحزم والمشاريع الأساسية في هذا الكتاب. الآن دعنا نفهم بعمق لماذا هي الخيار الأمثل حاليًا، ونستكشف كل إمكانياتها.

🔄 مقارنة: تطور أدوات إدارة الحزم في Python

الأداة	الوظيفة	الحالة
`pip`	تثبيت الحزم الأساسي	قديمة، لا تزال موجودة، بطيئة نسبيًا
`venv`	إنشاء بيئات افتراضية	جزء من المكتبة القياسية، بسيطة لكن محدودة
`virtualenv`	بديل أقدم وأكثر مرونة لـ `venv`	أقل استخدامًا الآن
`pipenv`	يجمع pip + virtualenv	انخفض استخدامها كثيرًا
`poetry`	إدارة شاملة للمشاريع والتبعيات	لا تزال شائعة، لكن أبطأ من uv
`uv`	**كل ما سبق، لكن أسرع بعشرات المرات، مكتوبة بلغة Rust**	**الخيار الموصى به حاليًا (2024-2025+)**

`uv` طوّرتها شركة **Astral** (نفس الشركة التي طوّرت `ruff` الذي سنستخدمه في الفصل 24)، وهي تجمع في أداة واحدة: إدارة البيئات الافتراضية، تثبيت الحزم، إدارة إصدارات Python نفسها، وتشغيل السكريبتات — بسرعة تفوق الأدوات التقليدية بعشرات المرات بسبب كتابتها بلغة Rust عالية الأداء.

إنشاء مشروع جديد باحترافية مع uv

حتى الآن في الكتاب، كنا ننشئ ملفات `.py` منفردة. الآن سنتعلم إنشاء **مشروع متكامل** بالهيكلية القياسية الصحيحة.

```
Terminal
```

```
uv init my_project
cd my_project
```

هذا الأمر يُنشئ هيكلية كاملة:

```
my_project/
├── .venv/              # البيئة الافتراضية (تُنشأ لاحقًا عند أول uv run أو uv sync)
├── .gitignore          # ملف يُخبر Git بالملفات التي يجب تجاهلها
├── .python-version     # يُحدد إصدار Python المطلوب لهذا المشروع تحديدًا
├── pyproject.toml      # ملف التهيئة المركزي للمشروع (الأهم!)
├── README.md           # توثيق المشروع
└── main.py             # نقطة الدخول الأولية
```

نصيحة الخبير: لاحظ ملف `.python-version`. هذا الملف يضمن أن أي شخص (بما فيك أنت بعد أشهر) يعمل على هذا المشروع سيستخدم **بالضبط** نفس إصدار Python المحدد، حتى لو كانت لديه إصدارات أخرى مثبتة على جهازه. `uv` تقرأ هذا الملف تلقائيًا وتُنزّل الإصدار المطلوب إن لم يكن موجودًا أصلًا!

محتوى `pyproject.toml` الأساسي

TOML

```toml
[project]
name = "my-project"
version = "0.1.0"
description = "وصف موجز للمشروع"
readme = "README.md"
requires-python = "⩾3.14"
dependencies = []

[build-system]
requires = ["hatchling"]
build-backend = "hatchling.build"
```

هذا الملف هو **«بطاقة هوية» المشروع بالكامل**: اسمه، إصدار Python المطلوب، وكل التبعيات (المكتبات الخارجية) التي يحتاجها.

إدارة التبعيات (Dependencies)

إضافة مكتبة جديدة

Terminal

```
uv add requests
```

هذا الأمر يقوم بعدة أشياء تلقائيًا:

1. يُنشئ `.venv/` إن لم تكن موجودة.
2. يُثبّت `requests` وكل تبعياتها داخل هذه البيئة المعزولة.

3. يُضيف `requests` إلى قسم `dependencies` في `pyproject.toml`.
4. يُحدّث (أو يُنشئ) ملف `uv.lock`.

```TOML
[project]
dependencies = [
    "requests⩾2.31.0",
]
```

إضافة مكتبة بإصدار محدد

```Terminal
uv add "requests==2.31.0"    # إصدار محدد بالضبط
uv add "pandas⩾2.0,<3.0"    # نطاق إصدارات
```

حذف مكتبة

```Terminal
uv remove requests
```

تبعيات التطوير فقط: `--dev`

تذكر من الفصل 15 كيف ثبّتنا `mypy`:

```Terminal
uv add --dev mypy pytest ruff
```

```TOML
[dependency-groups]
dev = [
    "mypy⩾1.8.0",
    "pytest⩾8.0.0",
    "ruff⩾0.3.0",
]
```

> **نصيحة الخبير - لماذا نفصل تبعيات التطوير؟** أدوات مثل `mypy` و`pytest` و`ruff` تحتاجها **أنت** أثناء التطوير، لكن المستخدم النهائي الذي يُشغّل برنامجك المُنتهي **لا يحتاجها إطلاقًا**. فصلها يُبقي حزمة الإنتاج (Production) خفيفة قدر الإمكان، ويُسرّع عملية النشر (Deployment) لاحقًا (الفصل 26).

ملف `uv.lock` : الدقة المطلقة

```
Terminal
cat uv.lock | head -20
```

ملف `uv.lock` يُسجّل **بالضبط** رقم الإصدار الدقيق (حتى آخر رقم فرعي) لكل مكتبة وكل تبعياتها الفرعية (Transitive Dependencies)، مع بصمة تشفيرية (Hash) للتحقق من سلامتها. هذا يضمن أن **أي شخص** يستنسخ مشروعك سيحصل على **البيئة المطابقة تمامًا بت لبت**، بغض النظر عن متى أو أين يُشغّل `uv sync` .

> **تحذير:** لا تُعدّل ملف `uv.lock` يدويًا أبدًا! هذا الملف يُدار تلقائيًا بالكامل بواسطة `uv` . يجب إضافته إلى Git (بعكس `.venv/` التي يجب تجاهلها، كما سنرى قريبًا) حتى يحصل كل من يعمل على المشروع على نفس البيئة بالضبط.

مقارنة: `pyproject.toml` مقابل `uv.lock`

الملف	الوظيفة	مرونة الإصدار
`pyproject.toml`	يُعرّف **النطاق** المقبول (`requests ⩾ 2.31`)	مرن
`uv.lock`	يُثبّت **الإصدار الدقيق الفعلي** المُستخدم (`requests==2.31.4`)	صارم تمامًا

استنساخ مشروع موجود وتثبيت كل تبعياته دفعة واحدة

عندما تحصل على مشروع (من زميل، أو من Git)، كل ما تحتاجه:

```
Terminal
uv sync
```

هذا الأمر يقرأ `uv.lock` (أو `pyproject.toml` إن لم يوجد lock بعد) وينشئ `.venv/` ويُثبّت **بالضبط** كل ما هو مطلوب، بنفس الإصدارات تمامًا.

تشغيل الأوامر داخل البيئة: `uv run`

لاحظ أننا استخدمنا `uv run` طوال الكتاب:

```
Terminal
```

```
uv run main.py
uv run mypy .
uv run pytest
```

`uv run` تضمن تنفيذ الأمر **داخل** بيئة المشروع الافتراضية مباشرة، دون الحاجة لتفعيلها يدويًا أولًا (عبر `source .venv/bin/activate` التقليدية). هذا أبسط وأقل عرضة للأخطاء (مثل نسيان التفعيل والعمل بالخطأ على بيئة Python عامة أو بيئة مشروع آخر).

مقارنة: الطريقة التقليدية مقابل uv

```
Terminal
```

```
# الطريقة التقليدية (venv + pip)
python -m venv .venv
source .venv/bin/activate        # لينكس/ماك
.venv\Scripts\activate           # Windows
pip install requests
python main.py
deactivate

# الطريقة الحديثة (uv) - أبسط وأسرع بكثير
uv add requests
uv run main.py
```

إدارة إصدارات Python نفسها عبر uv

من الميزات القوية لـ `uv` أنها تستطيع تثبيت وإدارة **إصدارات Python ذاتها**، دون الحاجة لتثبيتها يدويًا من الموقع الرسمي:

```
Terminal
```

```
# عرض الإصدارات المتاحة للتثبيت
uv python list

# تثبيت إصدار معيّن
uv python install 3.14

# تحديد إصدار Python لمشروع معيّن
uv python pin 3.14
```

هذا الأمر الأخير يُنشئ/يُحدّث ملف `.python-version` بالقيمة `3.14`، فيضمن أن كل من يعمل على هذا المشروع (بما فيك) يستخدم بالضبط هذا الإصدار.

تشغيل سكريبتات مستقلة بتبعيات مضمّنة

ميزة فريدة من uv: يمكنك كتابة سكريبت Python **واحد** يُعرّف تبعياته الخاصة بداخله مباشرة، دون الحاجة لمشروع كامل:

Python

```python
# /// script
# dependencies = [
#   "requests",
# ]
# ///

import requests

response = requests.get("https://api.github.com")
print(response.status_code)
```

Terminal

```
uv run script.py
```

`uv` تقرأ التعليق الخاص في الأعلى، تُنشئ بيئة مؤقتة بهذه التبعية بالضبط، وتُشغّل السكريبت فورًا — مثالي لسكريبتات سريعة أو أمثلة مستقلة لا تستحق إنشاء مشروع كامل من أجلها.

جدول مرجعي شامل لأوامر uv

الأمر	الوظيفة
`uv init project_name`	إنشاء مشروع جديد
`uv add package`	إضافة تبعية
`uv add --dev package`	إضافة تبعية تطوير فقط
`uv remove package`	حذف تبعية
`uv sync`	تثبيت كل التبعيات المسجّلة بدقة
`uv run command`	تشغيل أمر داخل بيئة المشروع
`uv python install 3.14`	تثبيت إصدار Python معيّن
`uv python pin 3.14`	تحديد إصدار Python للمشروع
`uv lock`	تحديث ملف `uv.lock` يدويًا

الأمر	الوظيفة
`uv venv`	إنشاء بيئة افتراضية يدويًا (نادرًا ما نحتاجها مباشرة)

18.3 الجزء الثاني: هيكلة مشروع احترافية

هيكل مشروع احترافي

```
my-project/
   |-- pyproject.toml
   |-- README.md
   |-- .gitignore
   |-- src/
      |   `-- my_pkg/
         |       |-- __init__.py
         |       |-- main.py
         |       `-- utils.py
   `-- tests/
         `-- test_main.py
```

الإعدادات ----------
الكود ----------
الاختبارات ----------

شكل 18.2 — هيكل مشروع احترافي

هيكل "src layout" القياسي

للمشاريع الصغيرة (سكريبت واحد أو اثنين)، وضع كل شيء في المجلد الرئيسي مقبول. لكن للمشاريع الحقيقية، الهيكل القياسي الموصى به عالميًا هو **src layout**:

```
my_project/
├── .venv/
├── .gitignore
├── .python-version
├── pyproject.toml
├── uv.lock
├── README.md
├── src/
│   └── my_project/
│       ├── __init__.py
│       ├── main.py
│       ├── models.py
│       ├── utils.py
│       └── services/
│           ├── __init__.py
│           └── user_service.py
└── tests/
    ├── __init__.py
    ├── test_models.py
    └── test_user_service.py
```

لماذا `src/` تحديدًا؟

> **نصيحة الخبير:** وضع الكود داخل `src/my_project/` بدلًا من مباشرة في المجلد الرئيسي يمنع مشكلة شائعة: استيراد غير مقصود للكود «من المجلد الحالي» بدلًا من «النسخة المثبتة فعليًا» للحزمة، مما قد يُخفي أخطاءً حقيقية أثناء التطوير تظهر فقط لاحقًا عند التثبيت الفعلي للمستخدمين. هذا الهيكل أصبح المعيار شبه الرسمي المتفق عليه في مجتمع Python الحديث.

ملف `__init__.py`: ما وظيفته؟

```python
# src/my_project/__init__.py
```

هذا الملف (قد يكون فارغًا تمامًا) يُخبر Python أن هذا المجلد هو **حزمة (Package)** حقيقية يمكن استيرادها، وليس مجرد مجلد عادي. يمكن أيضًا استخدامه لتصدير عناصر معيّنة بشكل مباشر:

```python
# src/my_project/__init__.py
from my_project.models import User
from my_project.utils import format_date

__version__ = "0.1.0"
```

هذا يسمح للمستخدمين باستيراد `from my_project import User` مباشرة، بدلًا من `from my_project.models import User` الأطول.

ملف `README.md`: واجهة مشروعك الأولى

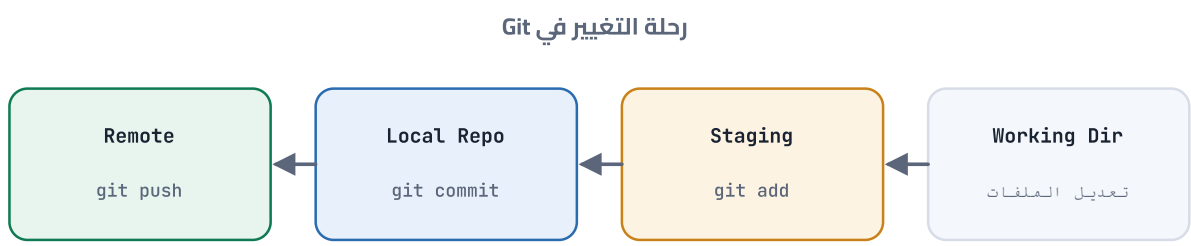
كل مشروع احترافي يبدأ بملف `README.md` يشرح:

Markdown

```
# اسم المشروع

وصف موجز وواضح لما يفعله المشروع.

## التثبيت

\`\`\`bash
uv sync
\`\`\`

## الاستخدام

\`\`\`bash
uv run main.py
\`\`\`

## المتطلبات

- Python 3.14+

## الترخيص

MIT
```

نصيحة الخبير: يُكتب `README.md` بصيغة **Markdown** — لغة تنسيق نصوص بسيطة جدًا (تعرفت عليها ضمنًا في هذا الكتاب نفسه، فكل العناوين والقوائم والصناديق التي تراها مكتوبة بـ Markdown!). `#` للعناوين، ```` ```code``` ```` للأكواد، `-` للقوائم. تُعرضها منصات مثل GitHub تلقائيًا بتنسيق جميل مقروء.

18.4 الجزء الثالث: أساسيات Git



شكل 18.3 — مراحل Git الأربع

ما هو Git ولماذا هو لا غنى عنه؟

Git هو **نظام التحكم بالإصدارات (Version Control System)** الأكثر انتشارًا في العالم. تخيّله كـ«آلة زمن» لمشروعك: يحفظ كل «لقطة» (Snapshot) من تاريخ تطور كودك، فيمكنك:

- الرجوع لأي نسخة سابقة في أي وقت.
- معرفة من غيّر أي سطر، ومتى، ولماذا.
- العمل على ميزات جديدة بشكل منعزل (Branches) دون المساس بالكود المستقر.
- التعاون مع فريق كامل على نفس الكود دون الكتابة فوق بعضهم البعض.

مقارنة: Git مقابل «حفظ نسخ يدويًا» (`project_final_FINAL` ، `project_v2_final` ، `project_v1`) كثير من المبتدئين يحفظون نسخًا متعددة من مجلدهم يدويًا كـ «نسخة احتياطية». هذا الأسلوب كارثي: لا يعرف أحد ما الفرق الدقيق بين النسخ، يستهلك مساحة تخزين هائلة (نسخ كاملة متكررة)، ولا يدعم التعاون الجماعي إطلاقًا. Git يحل كل هذه المشاكل بأناقة: يحفظ فقط **الفروقات (Diffs)** بين كل لقطة والتي تليها، مع رسائل وصفية واضحة لكل تغيير.

تثبيت Git والتهيئة الأولى

Terminal

```
# تحقق من وجود Git مسبقًا
git --version

# التهيئة لمرة واحدة فقط على جهازك (يُستخدم لتوقيع كل "اللقطات" باسمك)
git config --global user.name "اسمك"
git config --global user.email "بريدك@example.com"
```

دورة حياة Git الأساسية: المفاهيم الجوهرية

Git يعمل بثلاث «مناطق» رئيسية يجب فهمها جيدًا:

```
(Working Directory) → (Staging Area)      → (Repository)
   [تُعدّل ملفاتك]      [git add]              [git commit]
المستودع (التاريخ)          منطقة التجهيز          منطقة العمل
```

1. **Working Directory**: الملفات كما هي الآن على قرصك، قد تحتوي تعديلات غير محفوظة في Git بعد.
2. **Staging Area**: منطقة «تجهيز» مؤقتة — تُحدد فيها **أي** تعديلات تريد تضمينها في اللقطة القادمة.
3. **Repository**: التاريخ الدائم المحفوظ فعليًا — كل «لقطة» مُسجَّلة هنا للأبد (أو حتى تُحذف عمدًا).

إنشاء أول مستودع Git

Terminal

```
cd my_project
git init
```

هذا الأمر يُنشئ مجلدًا مخفيًا `.git/` يحتوي كل تاريخ المشروع (لا تلمسه يدويًا أبدًا!).

ملف `.gitignore`: ما الذي لا يجب تتبعه؟

ليس كل شيء في مشروعك يجب أن يُحفظ في Git! الملفات التي تُنشأ تلقائيًا (كالبيئة الافتراضية، أو ملفات الكاش)، أو الملفات الحساسة (كلمات مرور، مفاتيح API)، يجب **استثناؤها** تمامًا.

gitignore

```
# البيئة الافتراضية - ضخمة ويمكن إعادة إنشائها بـ uv sync
.venv/

# ملفات بايثون المُجمَّعة
__pycache__/
*.pyc

# ملفات بيئة خاصة (أسرار، مفاتيح API)
.env

# ملفات نظام التشغيل
.DS_Store
Thumbs.db

# ملفات المحرر
.vscode/
*.swp

# ملفات الاختبار والتغطية
.pytest_cache/
.mypy_cache/
htmlcov/
```

⚠ **تحذير أمني بالغ الأهمية:** لا تُضِف أبدًا ملفات تحتوي كلمات مرور أو مفاتيح API سرية إلى Git! إن حدث ذلك بالخطأ، فحتى لو حذفتها لاحقًا، **تبقى في تاريخ Git إلى الأبد** (ما لم تُعِد كتابة التاريخ بالكامل بعملية معقّدة). القاعدة الذهبية: ضع كل الأسرار في ملف `.env` منفصل، وأضفه فورًا إلى `.gitignore` **قبل** أن تكتب فيه أي سر.

💡 **نصيحة الخبير:** `uv init` تُنشئ ملف `.gitignore` مبدئي مناسب تلقائيًا عند إنشاء مشروع جديد، لكن من الجيد معرفة محتواه وتعديله حسب احتياجات مشروعك.

الدورة الأساسية: `add` ، `commit` ، `status` ، `log`

`git status` : ماذا تغيّر؟

Terminal

```
git status
```

```
On branch main
No commits yet
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        main.py
        pyproject.toml
```

يخبرك بالضبط حالة كل ملف: جديد (Untracked)، مُعدَّل (Modified)، أو مُجهَّز (Staged).

`git add`: تجهيز التغييرات

Terminal

```
git add main.py          # إضافة ملف واحد محدد
git add .                # إضافة كل الملفات المُعدَّلة/الجديدة في المجلد الحالي وما تحته
```

`git commit`: حفظ اللقطة فعليًا

Terminal

```
git commit -m "إضافة الدالة الرئيسية للبرنامج"
```

`-m` تعني «رسالة» (message) — وصف **واضح ومختصر** لما تغيّر ولماذا.

> **نصيحة الخبير - كتابة رسائل commit جيدة:**
> - ❌ سيئة: `"تعديلات"`, `"fix"`, `"تحديث"`.
> - ✅ جيدة: `"إصلاح خطأ القسمة على صفر في دالة calculate_average"`, `"إضافة ميزة تصدير CSV"`.
>
> رسالة الـ commit الجيدة تشرح **ماذا** تغيّر و**لماذا**، بحيث يفهمها أي شخص (بما فيك أنت بعد 6 أشهر) دون الحاجة لقراءة الكود بالكامل.

`git log`: استعراض التاريخ

Terminal

```
git log
```

```
commit a1b2c3d4e5f6... (HEAD → main)
Author: اسمك <بريدك@example.com>
Date:   Mon Jun 10 14:30:00 2025

    إضافة الدالة الرئيسية للبرنامج
```

```
Terminal
```

```
git log --oneline    # عرض مختصر، سطر واحد لكل commit - مفيد جدًا للاستعراض السريع
```

دورة عمل يومية نموذجية

```
Terminal
```

```
# 1. تحقق من حالة الملفات
git status

# 2. عدّل أكوادك بأي محرر (VS Code مثلاً)
#..   تعديلات..

# 3. راجع ما تغيّر بالتفصيل (اختياري لكن مفيد)
git diff

# 4. جهّز التغييرات التي أنهيتها منطقيًا
git add src/my_project/utils.py

# 5. احفظ اللقطة برسالة واضحة
git commit -m "إضافة دالة format_currency في utils"

# كرر الدورة.
```

نصيحة الخبير: اجعل كل commit يمثّل **تغييرًا منطقيًا واحدًا ومتماسكًا** (ميزة واحدة، أو إصلاح خطأ واحد)، بدلًا من تجميع عشرات التغييرات غير المترابطة في commit ضخم واحد. هذا يجعل تاريخ مشروعك أوضح بكثير، ويُسهّل كثيرًا تحديد «أي commit تسبب بهذا الخطأ» لاحقًا إن ظهرت مشكلة.

الفروع (Branches): العمل المتوازي الآمن

تخيّل أنك تعمل على ميزة جديدة كبيرة، لكن الكود الحالي (`main`) يعمل بشكل مستقر ويُستخدم فعليًا. تريد تجربة أفكار جديدة **دون المخاطرة** بكسر الكود المستقر. هنا يأتي مفهوم **الفروع (Branches)**.

```
main:      A---B---C---------F---G
                    \       /
feature:             D---E---'
```

كل فرع هو «خط تطور مستقل»، يمكن لاحقًا **دمجه (Merge)** مع الفرع الرئيسي بمجرد التأكد من أنه يعمل بشكل صحيح.

Terminal

```
# إنشاء فرع جديد والانتقال إليه مباشرة
git checkout -b feature/user-authentication

# أو بالصيغة الأحدث
git switch -c feature/user-authentication

# ... تعمل، تُعدّل، تعمل commits عادية هنا..
git add .
git commit -m "إضافة نظام تسجيل الدخول"

# الرجوع للفرع الرئيسي
git switch main

# دمج الفرع الجديد بعد التأكد من جاهزيته
git merge feature/user-authentication

# حذف الفرع بعد دمجه (لم تعد هناك حاجة إليه)
git branch -d feature/user-authentication
```

جدول: أوامر الفروع الأساسية

الأمر	الوظيفة
`git branch`	عرض كل الفروع الموجودة
`git switch -c name`	إنشاء فرع جديد والانتقال إليه
`git switch name`	الانتقال إلى فرع موجود
`git merge name`	دمج فرع معيّن في الفرع الحالي
`git branch -d name`	حذف فرع (بعد دمجه بأمان)

⚠ **تحذير شائع:** تعارضات الدمج (**Merge Conflicts**) تحدث عندما يُعدّل فرعان **نفس السطر بالضبط** بطرق مختلفة. Git لا يعرف أيهما «الصحيح»، فيطلب منك **حسم التعارض يدويًا**. لا داعي للخوف من هذا — هو جزء طبيعي من العمل الجماعي، وVS Code (بإضافة GitLens مثلاً) يُسهّل كثيرًا رؤية وحل هذه التعارضات بصريًا.

العمل مع مستودعات بعيدة: GitHub

كل ما تعلمناه حتى الآن يعمل **محليًا على جهازك فقط**. للتعاون مع الآخرين (أو لحفظ نسخة احتياطية سحابية)، نحتاج **مستودعًا بعيدًا (Remote Repository)**، وأشهر منصة لاستضافته هي **GitHub**.

ربط مشروعك المحلي بمستودع GitHub

بعد إنشاء مستودع فارغ على GitHub.com:

Terminal

```
git remote add origin https://github.com/username/my_project.git
git branch -M main
git push -u origin main
```

- `remote add origin ...` : يُخبر Git بعنوان المستودع البعيد، ويسميه `origin` (اسم اصطلاحي شائع).
- `push -u origin main` : يرفع كل الـ commits المحلية إلى GitHub، ويربط الفرع المحلي `main` بنظيره البعيد (`-u` تعني «تذكر هذا الربط للمستقبل»).

العمليات اليومية مع مستودع بعيد

Terminal

```
git push                # رفع commits جديدة محلية إلى GitHub
git pull                # تنزيل آخر تحديثات من GitHub ودمجها محليًا
git clone <url>         # نسخ مستودع كامل من GitHub إلى جهازك لأول مرة
```

نصيحة الخبير: `git clone` هي أول أمر ستستخدمه عند الانضمام لأي مشروع جماعي موجود مسبقًا على GitHub. بعدها مباشرة، نفّذ `uv sync` لتثبيت كل التبعيات المطلوبة بدقة تامة، وستكون جاهزًا للعمل خلال دقائق.

مقارنة: Git مقابل GitHub

	Git	GitHub
ما هو؟	أداة برمجية (نظام تحكم بالإصدارات)	منصة سحابية لاستضافة مستودعات Git
أين يعمل؟	محليًا على جهازك	على الإنترنت
هل ضروري؟	نعم، لا بديل عنه عمليًا	اختياري (بدائل: GitLab, Bitbucket) لكنه الأشهر

جدول مرجعي شامل لأوامر Git

الأمر	الوظيفة
`git init`	إنشاء مستودع جديد

الأمر	الوظيفة
`git status`	عرض حالة الملفات الحالية
`git add .` / `git add <file>`	تجهيز التغييرات
`git commit -m "..."`	حفظ لقطة بالتغييرات المُجهَّزة
`git log --oneline` / `git log`	عرض تاريخ الـ commits
`git diff`	عرض الفروقات التفصيلية غير المُجهَّزة بعد
`git branch`	عرض الفروع
`git switch -c <name>`	إنشاء فرع جديد والانتقال إليه
`git switch <name>`	التبديل بين الفروع
`git merge <name>`	دمج فرع في الفرع الحالي
`git remote add origin <url>`	ربط مستودع بعيد
`git push`	رفع التغييرات للمستودع البعيد
`git pull`	تنزيل ودمج تحديثات المستودع البعيد
`git clone <url>`	نسخ مستودع بعيد كاملاً محليًا

18.5 أفضل الممارسات الشاملة

Commit مبكرًا، Commit كثيرًا

لا تنتظر حتى تنتهي «من كل شيء» لعمل commit واحد ضخم. كل تغيير منطقي صغير يستحق commit خاصًا به.

لا تضع الأسرار في Git أبدًا

Terminal

```
# .env (أبدًا لا يُضاف لـ Git)
API_KEY=sk-1234567890abcdef
DATABASE_PASSWORD=super_secret_password
```

```
gitignore
```

```
# .gitignore
.env
```

اكتب رسائل commit بصيغة الأمر (اصطلاح شائع)

```
Terminal
```

```
# ✅ اصطلاح شائع عالميًا
git commit -m "Add user authentication"
git commit -m "Fix division by zero bug"

# بدلًا من
git commit -m "Added user authentication"  # صيغة الماضي
```

(هذا اصطلاح بالإنجليزية شائع عالميًا في المشاريع مفتوحة المصدر، لكن إن كان مشروعك محليًا بالعربية بالكامل، استخدم العربية بوضوح واتساق).

لا تُضف `.venv/` أبدًا إلى Git

البيئة الافتراضية ضخمة ويمكن إعادة إنشائها بالكامل من `pyproject.toml` و `uv.lock` في ثوانٍ عبر `uv sync`. إضافتها لـ Git يُضخّم المستودع دون أي فائدة.

استخدم فروعًا منفصلة للميزات الكبيرة

تجنّب العمل مباشرة على `main` للميزات الكبيرة أو التجريبية — اعمل في فرع منفصل، وادمجه فقط بعد التأكد من استقراره.

> ⚠ **خطأ شائع:** نسيان `git add` قبل `git commit`! إن نسيت، ستظهر رسالة "nothing to commit" أو سيُحفظ commit فارغ. تحقق دائمًا بـ `git status` قبل `git commit` للتأكد من أن التغييرات المطلوبة فعلاً «مُجهَّزة».

18.6 مشروع تطبيقي: تجهيز مشروع كامل من الصفر

لنطبّق كل ما تعلمناه في سيناريو متكامل: إنشاء مشروع «محول عملات» بسيط، بهيكلية احترافية كاملة، وتاريخ Git نظيف، وجاهز للرفع على GitHub.

✅ خطوات الحل الكاملة

الخطوة 1: إنشاء المشروع

Terminal

```
uv init currency_converter
cd currency_converter
uv python pin 3.14
```

الخطوة 2: إعادة الهيكلة إلى src layout

Terminal

```
mkdir -p src/currency_converter tests
touch src/currency_converter/__init__.py
touch tests/__init__.py
```

الخطوة 3: كتابة الكود الأساسي

Python src/currency_converter/__init__.py

```python
from currency_converter.converter import convert

__version__ = "0.1.0"
__all__ = ["convert"]
```

Python src/currency_converter/converter.py

```python
"""وحدة تحويل العملات."""
from __future__ import annotations

EXCHANGE_RATES: dict[str, float] = {
    "USD": 1.0,
    "EUR": 0.92,
    "SAR": 3.75,
    "EGP": 48.5,
}

class UnsupportedCurrencyError(Exception):
    """تُرفع عندما تكون العملة المطلوبة غير مدعومة."""

def convert(amount: float, from_currency: str, to_currency: str) -> float:
    """
    يحوّل مبلغًا من عملة إلى أخرى بناءً على أسعار الصرف الثابتة.

    Args:
        amount: المبلغ المراد تحويله.
        from_currency: رمز العملة الأصلية (مثل "USD").
        to_currency: رمز العملة الهدف (مثل "SAR").

    Returns:
        المبلغ المحوّل بالعملة الهدف.

    Raises:
        UnsupportedCurrencyError: إن كانت إحدى العملتين غير مدعومة.
    """
    if from_currency not in EXCHANGE_RATES:
        raise UnsupportedCurrencyError(f"العملة غير مدعومة: {from_currency}")
    if to_currency not in EXCHANGE_RATES:
        raise UnsupportedCurrencyError(f"العملة غير مدعومة: {to_currency}")

    amount_in_usd = amount / EXCHANGE_RATES[from_currency]
    return round(amount_in_usd * EXCHANGE_RATES[to_currency], 2)
```

Python src/currency_converter/main.py

```python
"""نقطة الدخول الرئيسية للبرنامج."""
from currency_converter.converter import convert, UnsupportedCurrencyError

def main() -> None:
    try:
        result = convert(100, "USD", "SAR")
        print(f"100 USD = {result} SAR")
    except UnsupportedCurrencyError as e:
        print(f"خطأ: {e}")

if __name__ == "__main__":
    main()
```

الخطوة 4: ملف `pyproject.toml` المُحدَّث

TOML

```toml
[project]
name = "currency-converter"
version = "0.1.0"
description = "محول عملات بسيط"
readme = "README.md"
requires-python = "≥3.14"
dependencies = []

[dependency-groups]
dev = [
    "mypy≥1.8.0",
    "pytest≥8.0.0",
]

[build-system]
requires = ["hatchling"]
build-backend = "hatchling.build"

[tool.mypy]
strict = true
```

الخطوة 5: كتابة `README.md`

Markdown

```
# محول العملات

أداة بسيطة لتحويل المبالغ بين عملات مختلفة بأسعار صرف ثابتة.

## التثبيت

\`\`\`bash
uv sync
\`\`\`

## الاستخدام

\`\`\`bash
uv run python -m currency_converter.main
\`\`\`

## الاختبار

\`\`\`bash
uv run pytest
uv run mypy .
\`\`\`
```

الخطوة 6: تهيئة Git

Terminal

```
git init
git add .
git commit -m "الإعداد الأولي للمشروع: هيكلية وتحويل عملات أساسي"
```

الخطوة 7: التحقق

Terminal

```
uv run mypy . --strict
# Success: no issues found

uv run python -c "from currency_converter.converter import convert; print(convert(100, 'USD', 'SAR'))"
# 375.0

git log --oneline
# a1b2c3d الإعداد الأولي للمشروع: هيكلية وتحويل عملات أساسي
```

الخطوة 8 (اختياري): الرفع على GitHub

Terminal

```
git remote add origin https://github.com/username/currency_converter.git
git branch -M main
git push -u origin main
```

الخطوة 9: إضافة ميزة جديدة في فرع منفصل

Terminal

```
git switch -c feature/add-more-currencies
```

Python

```
# تعديل converter.py - إضافة عملات جديدة
EXCHANGE_RATES: dict[str, float] = {
    "USD": 1.0,
    "EUR": 0.92,
    "SAR": 3.75,
    "EGP": 48.5,
    "AED": 3.67,   # جديد
    "KWD": 0.31,   # جديد
}
```

Terminal

```
git add .
git commit -m "إضافة دعم عملتي AED و KWD"
git switch main
git merge feature/add-more-currencies
git branch -d feature/add-more-currencies
```

18.7 تمارين

🟢 مستوى مبتدئ

تمرين 1: أنشئ مشروعًا جديدًا باسم `my_first_project` باستخدام `uv init`، ثبّت فيه مكتبة `requests`، وتحقق من ظهورها في `pyproject.toml`.

تمرين 2: في مشروعك من التمرين السابق، نفّذ `git init`، ثم اعمل أول commit برسالة مناسبة.

تمرين 3: ما الفرق بين `git add` و `git commit`؟ اشرح بجملتين.

✅ الحل النموذجي

Terminal

```
# تمرين 1
uv init my_first_project
cd my_first_project
uv add requests
cat pyproject.toml  # تحقق من وجود requests ضمن dependencies

# تمرين 2
git init
git add .
git commit -m "الإعداد الأولي للمشروع"

# تمرين 3
# git add: يُجهّز (Stage) التغييرات، أي يُحدد أيها سيدخل في اللقطة القادمة، دون حفظها نهائيًا بعد.
# git commit: يحفظ فعليًا كل التغييرات المُجهَّزة كلقطة دائمة في تاريخ المشروع، برسالة وصفية.
```

🟡 مستوى متوسط

تمرين 4: أعد هيكلة مشروع بسيط (يمكنك استخدام مشروع «مدير جهات الاتصال» من الفصل 16) إلى `src` `layout` قياسي، مع `tests/` فارغة جاهزة للفصل 25.

تمرين 5: أنشئ فرعًا جديدًا باسم `feature/export-pdf`، اعمل تعديلًا وهميًا بسيطًا (مثل إضافة دالة فارغة)، اعمل commit، ثم ادمجه في `main`.

تمرين 6: أنشئ ملف `.gitignore` مناسب لمشروع Python كامل، يتضمن: البيئة الافتراضية، ملفات الكاش، ملفات `.env`، وملفات نظام التشغيل.

✅ الحل النموذجي

Terminal

```
# تمرين 4
mkdir -p src/contact_manager tests
touch src/contact_manager/__init__.py tests/__init__.py
# نقل contact_manager.py الأصلي إلى src/contact_manager/manager.py
# وتحديث الاستيرادات بما يناسب الهيكلية الجديدة

# تمرين 5
git switch -c feature/export-pdf
# فارغة في الكود export_to_pdf إضافة دالة
git add .
git commit -m "إضافة دالة export_to_pdf (هيكل أولي)"
git switch main
git merge feature/export-pdf
git branch -d feature/export-pdf

# تمرين 6
cat > .gitignore << 'EOF'
.venv/
__pycache__/
*.pyc
.env
.DS_Store
Thumbs.db
.vscode/
.pytest_cache/
.mypy_cache/
htmlcov/
EOF
```

🔴 تحدي

تمرين 7: محاكِ سيناريو عمل جماعي كامل:
1. أنشئ مشروعًا جديدًا، واعمل commit أولي على `main`.
2. أنشئ فرعين منفصلين: `feature/login` و `feature/signup`، وفي كل منهما أضف دالة بسيطة مختلفة في نفس الملف (لنفس السطر تقريبًا) لإجبار حدوث تعارض عمدًا.
3. ادمج `feature/login` في `main` بنجاح.

4. حاول دمج `feature/signup` في `main` وشاهد رسالة التعارض (Merge Conflict).
5. افتح الملف المتعارض، لاحظ العلامات الخاصة (`<<<<<<<` , `=======` , `>>>>>>>`)، احسم التعارض يدويًا بدمج كلا التغييرين منطقيًا، ثم أكمل الدمج بـ `git add` و `git commit` .

✅ إرشادات الحل

Terminal

```
# 1. الإعداد
uv init conflict_demo
cd conflict_demo
echo "def main():
    print('البرنامج الرئيسي')
" > app.py
git init
git add .
git commit -m "الإعداد الأولي"

# 2. فرع login
git switch -c feature/login
echo "def main():
    print('ميزة تسجيل الدخول')
" > app.py
git add .
git commit -m "إضافة ميزة تسجيل الدخول"

git switch main

# فرع signup (من main، لإحداث تعارض حقيقي على نفس الأسطر)
git switch -c feature/signup
echo "def main():
    print('ميزة إنشاء حساب جديد')
" > app.py
git add .
git commit -m "إضافة ميزة إنشاء حساب"

git switch main

# 3. دمج أول فرع - ينجح بلا مشاكل
git merge feature/login

# 4. محاولة دمج الفرع الثاني - تعارض!
git merge feature/signup
# Auto-merging app.py
# CONFLICT (content): Merge conflict in app.py

# 5. فتح app.py، سيظهر شيء كهذا:
# <<<<<<< HEAD
# def main():
#     print('ميزة تسجيل الدخول')
# =======
# def main():
```

```
#     print('ميزة إنشاء حساب جديد')
# >>>>>>> feature/signup

# الحسم اليدوي - دمج كلا الميزتين منطقيًا:
cat > app.py << 'EOF'
def main():
    print('ميزة تسجيل الدخول')
    print('ميزة إنشاء حساب جديد')
EOF

git add app.py
git commit -m "حسم التعارض: دمج ميزتي تسجيل الدخول وإنشاء الحساب"
```

هذا التمرين يُحاكي بدقة الموقف الذي سيواجهه كل مبرمج يعمل ضمن فريق عاجلًا أم آجلًا — وفهمه مبكرًا يُزيل الخوف منه تمامًا.

18.8 اختبر فهمك (Quiz)

❓ السؤال 1: لماذا نحتاج بيئة افتراضية منفصلة لكل مشروع؟

لتجنّب تعارض إصدارات المكتبات بين المشاريع المختلفة. كل مشروع قد يحتاج إصدارًا مختلفًا من نفس المكتبة، والتثبيت العام (Global) على الجهاز يسمح بإصدار واحد فقط في كل مرة، مما يُسبب تعارضات حتمية عند العمل على عدة مشاريع.

❓ السؤال 2: ما الفرق بين `pyproject.toml` و `uv.lock`؟

`pyproject.toml` يُعرّف **النطاق** المقبول لكل تبعية (مثل `requests ⩾ 2.31`)، بينما `uv.lock` يُسجّل **الإصدار الدقيق الفعلي** المُستخدم فعليًا لكل تبعية وتبعياتها الفرعية، لضمان أن كل من يُشغّل `uv sync` يحصل على نفس البيئة تمامًا.

❓ السؤال 3: ما وظيفة ملف `.gitignore`؟

يُخبر Git بالملفات والمجلدات التي يجب **تجاهلها تمامًا** وعدم تتبعها، مثل البيئة الافتراضية (`.venv/`)، ملفات الكاش المؤقتة، والملفات الحساسة كـ `.env` التي تحتوي أسرارًا لا يجب رفعها لمستودع مشترك.

❓ السؤال 4: اشرح الفرق بين `git commit`، `git add`، و `git push`.

`git add` يُجهّز التغييرات (ينقلها من Working Directory إلى Staging Area). `git commit` يحفظها فعليًا كلقطة دائمة في تاريخ المستودع المحلي. `git push` يرفع هذه اللقطات المحفوظة محليًا إلى مستودع بعيد (مثل GitHub) ليراها الآخرون.

❓ السؤال 5: لماذا نستخدم الفروع (Branches) في Git؟

للعمل على ميزات جديدة أو تجريبية بشكل منعزل تمامًا عن الفرع الرئيسي المستقر (`main`)، مما يسمح بالتطوير والتجربة دون المخاطرة بكسر الكود الذي يعمل بالفعل. بعد التأكد من جاهزية الميزة، تُدمج في الفرع الرئيسي بأمان.

❓ السؤال 6: ماذا يحدث عند تعارض الدمج (Merge Conflict)، وكيف يُحل؟

يحدث عندما يُعدّل فرعان مختلفان **نفس الأسطر بالضبط** بطرق متضاربة، فلا يستطيع Git تحديد النسخة الصحيحة تلقائيًا. يُحل يدويًا بفتح الملف المتعارض، ومراجعة العلامات الخاصة (`<<<<<<<`, `=======`, `>>>>>>>`) التي تُظهر كلا النسختين، واختيار أو دمج المحتوى الصحيح يدويًا، ثم `git add` و `git commit` لإكمال عملية الدمج.

18.9 ملخص الفصل (Cheat Sheet)

Terminal

```
# ===== uv: إدارة المشروع =====
uv init project_name          # مشروع جديد
uv add package                # إضافة تبعية
uv add --dev package          # تبعية تطوير فقط
uv remove package             # حذف تبعية
uv sync                       # تثبيت كل التبعيات بدقة
uv run command                # تشغيل أمر داخل بيئة المشروع
uv python pin 3.14            # تحديد إصدار Python

# ===== هيكلية src layout =====
# my_project/
# ├── src/my_project/__init__.py
# ├── tests/
# ├── pyproject.toml
# ├── uv.lock
# ├── .gitignore
# └── README.md

# ===== Git: الأساسيات =====
git init                      # إنشاء مستودع
git status                    # عرض الحالة
git add .                     # تجهيز كل التغييرات
git commit -m "حفظ لقطة #    "رسالة واضحة
git log --oneline             # عرض التاريخ

# ===== Git: الفروع =====
git switch -c feature/name    # فرع جديد والانتقال إليه
git switch main               # العودة للفرع الرئيسي
git merge feature/name        # دمج فرع
git branch -d feature/name    # حذف فرع بعد دمجه

# ===== Git: العمل البعيد (GitHub) =====
git remote add origin <url>   # ربط مستودع بعيد
git push -u origin main       # رفع أول مرة
git push                      # رفع التحديثات
git pull                      # سحب آخر التحديثات
git clone <url>               # نسخ مستودع كامل
```

المفهوم	الأداة
عزل تبعيات المشروع	`.venv/` عبر `uv`
توثيق التبعيات الدقيقة	`uv.lock`
هيكلية احترافية	layout `src/`
تاريخ التغييرات	Git (`commit`)
عمل متوازٍ آمن	Git Branches
استضافة سحابية ومشاركة	GitHub

18.10 ماذا بعد؟

تهانينا! لقد أكملت الآن **الجزء الثالث: Python الحديثة والاحترافية** بالكامل. تمتلك الآن كل الأدوات الأساسية: Type Hints للوضوح والأمان، معالجة الملفات وتنسيقات البيانات، البرمجة غير المتزامنة للأداء، وإدارة مشاريع احترافية عبر uv وGit. هذه المهارات مجتمعة تجعلك قادرًا على الانتقال من «كتابة سكريبتات تعليمية» إلى «بناء برامج حقيقية قابلة للمشاركة والصيانة والتطوير الجماعي».

الآن، وقد اكتملت الأساسيات الاحترافية، حان الوقت لأمر مثير: **تطبيق كل ما تعلمناه على مسارات تخصص حقيقية**. في الجزء الرابع من الكتاب، سننتقل عبر خمسة مسارات مختلفة تمامًا، كل واحد يمثل مجال عمل حقيقي يستخدم Python بكثافة في سوق العمل اليوم.

سنبدأ بمسار **تطوير الويب**: سنبني معًا واجهة برمجة تطبيقات (API) كاملة لإدارة المهام، باستخدام إطار العمل الحديث **FastAPI** — الذي يعتمد بشكل أساسي على كل ما تعلمناه للتو (async/await، Type Hints، وهيكلة المشاريع)!

الجزء الخامس · 5 فصول

05

جسور التخصص

جولة عملية في خمسة مسارات مهنية: الويب، تحليل البيانات، الأتمتة، الذكاء الاصطناعي، وأدوات سطر الأوامر — كل مسار بمشروع حقيقي مكتمل.

19 · مسار تطوير الويب

20 · مسار تحليل البيانات

21 · مسار الأتمتة

22 · مسار الذكاء الاصطناعي

23 · مسار بناء أدوات سطر الأوامر

فصل 19 / 26 · الجزء الخامس

19

مسار تطوير الويب - بناء واجهة API لإدارة المهام (FastAPI)

9 دقيقة قراءة | 15 قسمًا | 18 مثالًا | 1,879 كلمة

في هذا الفصل

19.1 لماذا يستحق هذا الفصل اهتمامك الكامل؟
19.2 المفاهيم الأساسية: كيف يعمل الويب؟
19.3 لماذا FastAPI؟
19.4 أول واجهة API: «مرحبًا أيها العالم»
19.5 المعاملات: Path Parameters و Query Parameters
19.6 Pydantic: نمذجة البيانات الواردة (Request Body)
19.7 معالجة الأخطاء: `HTTPException`
19.8 دعم async بشكل كامل
19.9 تنظيم مشروع أكبر: `APIRouter`
19.10 مشروع تطبيقي متكامل: واجهة API لإدارة المهام
19.11 أفضل الممارسات
19.12 تمارين
19.13 اختبر فهمك (Quiz)
19.14 ملخص الفصل (Cheat Sheet)
19.15 ماذا بعد؟

19.1 لماذا يستحق هذا الفصل اهتمامك الكامل؟

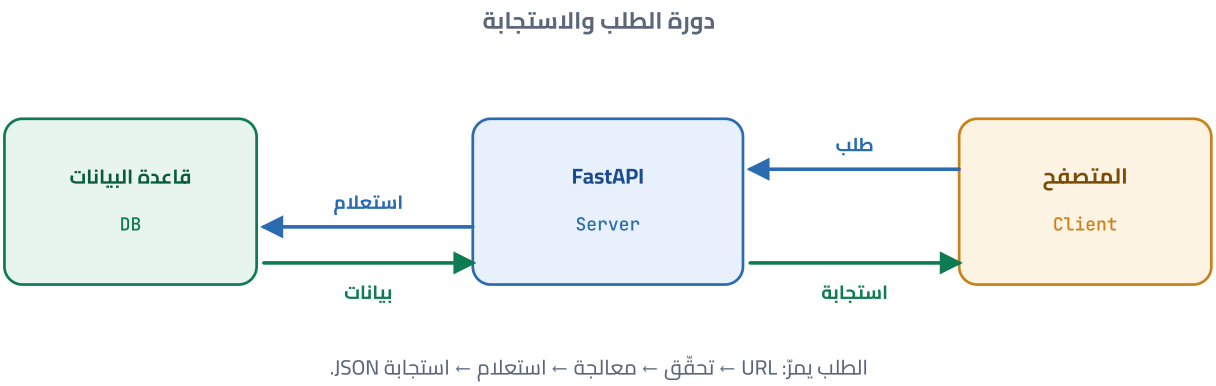
كل تطبيق تستخدمه يوميًا تقريبًا — Instagram، تطبيق البنك، منصة التوصيل، حتى الطقس على هاتفك — يعتمد على نفس الفكرة الجوهرية: تطبيق (العميل/Client) يطلب بيانات أو يُرسلها إلى خادم بعيد (Server) عبر الإنترنت، والخادم يستجيب. هذه «اللغة» التي يتحدث بها التطبيق مع الخادم تُسمى **واجهة برمجة التطبيقات (API - Application Programming Interface)**.

حتى الآن، كل برامجنا كانت «منعزلة» — تعمل على جهازك فقط. في هذا الفصل، ستبني **أول خادم حقيقي بك**: برنامج يعمل بشكل دائم، منتظرًا طلبات من أي مكان في العالم (أو من جهازك فقط أثناء التعلم)، يستقبلها، يعالجها، ويرد عليها — تمامًا كما تفعل خوادم الشركات الكبرى، لكن بمقياس أصغر وتعليمي.

سنستخدم **FastAPI**: إطار عمل (Framework) حديث لبناء واجهات API بلغة Python، أصبح في السنوات الأخيرة من أكثر الأطر شعبية وانتشارًا عالميًا، بسبب سرعته العالية، وبساطته، واعتماده المباشر والعميق على **كل** ما تعلمناه في الفصول السابقة: Type Hints (الفصل 15)، async/await (الفصل 17)، وDataclasses/Pydantic لنمذجة البيانات (امتداد لمفاهيم الفصل 14). هذا الفصل هو **تتويج عملي حقيقي** لكل رحلتك في هذا الكتاب حتى الآن.

> **نصيحة الخبير:** مسار تطوير الويب بـ Python (باستخدام FastAPI أو Django) هو أحد أكثر المسارات طلبًا في سوق العمل البرمجي عالميًا. إتقان هذا الفصل يفتح لك بابًا حقيقيًا نحو وظائف "Backend Developer" أو "Full-Stack Developer".

19.2 المفاهيم الأساسية: كيف يعمل الويب؟



شكل 19.1 — دورة الطلب والاستجابة

قبل كتابة أي كود، يجب فهم الصورة الكبرى.

نموذج العميل-الخادم (Client-Server Model)

```
┌──────────┐          طلب (Request)          ┌──────────┐
│          │ <─────────────────────────────  │          │
│  الخادم  │                                 │  العميل  │
│ (Client) │                                 │ (Server) │
│          │ <─────────────────────────────  │          │
└──────────┘          استجابة (Response)      └──────────┘
```

- **العميل (Client)**: متصفحك، تطبيق الجوال، أو حتى برنامج Python آخر — أي طرف **يطلب** شيئًا.
- **الخادم (Server)**: برنامج يعمل بشكل دائم، **ينتظر** الطلبات ويستجيب لها. هذا ما سنبنيه اليوم!

بروتوكول HTTP: لغة التواصل

HTTP (HyperText Transfer Protocol) هو «اللغة» المتفق عليها عالميًا لتبادل الطلبات والاستجابات عبر الويب. كل طلب HTTP يتكون من:

العنصر	المعنى	مثال
Method (الفعل)	ماذا أريد أن أفعل؟	`GET` ، `POST` ، `PUT` ، `DELETE`
URL (المسار)	أين؟	`/tasks/5`
Headers (ترويسات)	معلومات إضافية	نوع المحتوى، التوثيق
Body (الجسم)	البيانات المُرسَلة (إن وُجدت)	`{"title": "مهمة جديدة"}`

أفعال HTTP الأساسية (HTTP Methods)

الفعل	الغرض	مثال عملي
`GET`	**قراءة/جلب** بيانات	جلب قائمة كل المهام
`POST`	**إنشاء** مورد جديد	إضافة مهمة جديدة
`PUT` / `PATCH`	**تحديث** مورد موجود	تعديل عنوان مهمة
`DELETE`	**حذف** مورد	حذف مهمة

هذا النمط (ربط كل فعل HTTP بعملية CRUD: Create, Read, Update, Delete) يُسمى **REST** (Representational State Transfer) — وهو المعيار الأكثر انتشارًا عالميًا لتصميم واجهات API.

رموز الحالة (HTTP Status Codes)

عندما يستجيب الخادم، يُرفق **رمز حالة** يُلخّص ماذا حدث:

الفئة	المعنى	أمثلة شائعة
`2xx`	نجاح	`200 OK` ، `201 Created` ، `204 No Content`
`4xx`	خطأ من طرف العميل	`400 Bad Request` ، `404 Not Found` ، `422 Unprocessable Entity`
`5xx`	خطأ من طرف الخادم	`500 Internal Server Error`

> **نصيحة الخبير:** إن رأيت `404` أثناء تصفحك العادي للإنترنت، فأنت الآن تعرف بالضبط ماذا تعني: «الخادم فهم طلبك، لكن المورد الذي طلبته غير موجود».

19.3 لماذا FastAPI؟

🔄 مقارنة: أطر عمل الويب الشائعة في Python

الإطار	الطابع	الاستخدام الأمثل
Django	شامل جدًا (Batteries-Included) — يتضمن قاعدة بيانات، إدارة، مصادقة...	مواقع كبيرة متكاملة (كمواقع تجارة إلكترونية كاملة)
Flask	مرن وبسيط جدًا (Microframework)	مشاريع صغيرة إلى متوسطة، تحكم كامل بكل التفاصيل
FastAPI	حديث، سريع جدًا، يعتمد على Type Hints، توليد توثيق تلقائي	**واجهات API** حديثة (الأكثر ملاءمة لهذا الفصل)

لماذا FastAPI تحديدًا لتعلّم بناء APIs؟

1. **يعتمد مباشرة على Type Hints** التي أتقناها في الفصل 15 — لا مفاهيم جديدة غريبة، بل استخدام عملي لما تعرفه.
2. **تحقق تلقائي من صحة البيانات** (Validation) — إن أرسل العميل بيانات خاطئة، يرفضها FastAPI تلقائيًا برسالة خطأ واضحة، دون أن تكتب سطر تحقق واحد يدويًا.
3. **توثيق تفاعلي تلقائي** — بمجرد تشغيل الخادم، تحصل على واجهة ويب تفاعلية (Swagger UI) تعرض كل نقاط النهاية (Endpoints) ويمكنك تجربتها مباشرة من المتصفح.
4. **أداء عالٍ جدًا** — مبني على `async` / `await` (الفصل 17) من الأساس.

19.4 أول واجهة API: «مرحبًا أيها العالم»

التثبيت

```
Terminal
```

```
uv init task_api
cd task_api
uv add fastapi
uv add --dev "uvicorn[standard]"
```

- **FastAPI**: الإطار نفسه، مسؤول عن تعريف نقاط النهاية ومعالجة الطلبات.
- **Uvicorn**: خادم فعلي (ASGI Server) يُشغّل تطبيق FastAPI ويستقبل الطلبات الحقيقية عبر الشبكة.
FastAPI «يصف» كيف يُعالَج كل طلب، وUvicorn هو من «يستمع فعليًا» على منفذ الشبكة وينفّذ هذا الوصف.

الكود الأول

```python
Python  main.py

from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def read_root() -> dict[str, str]:
    return {"message": "مرحبًا بك في واجهة API!"}
```

تشغيل الخادم

```
Terminal

uv run uvicorn main:app --reload
```

- `main:app` : يعني "من ملف `main.py` ، استخدم المتغير `app` ".
- `--reload` : يُعيد تشغيل الخادم تلقائيًا كلما عدّلت الكود أثناء التطوير — مفيد جدًا، لكن **لا تستخدمها في الإنتاج الفعلي**.

افتح متصفحك على `http://127.0.0.1:8000` ، ستجد:

```
JSON

{"message": "مرحبًا بك في واجهة API!"}
```

التوثيق التفاعلي التلقائي

الآن افتح `http://127.0.0.1:8000/docs` — ستُفاجأ بواجهة تفاعلية كاملة (Swagger UI) تعرض كل نقاط النهاية، تلقائيًا، بدون أن تكتب أي سطر توثيق إضافي! يمكنك تجربة كل Endpoint مباشرة من هذه الصفحة.

> **نصيحة الخبير:** هذه الميزة وحدها (التوثيق التلقائي من `/docs`) توفر ساعات عمل كانت تُصرف تقليديًا على كتابة توثيق منفصل يصبح قديمًا بسرعة. FastAPI يُولّد هذا التوثيق مباشرة من الكود نفسه (من الـ Type Hints والتعليقات)، فيبقى دائمًا **متزامنًا ودقيقًا 100%** مع الكود الفعلي.

19.5 المعاملات: Path Parameters و Query Parameters

معاملات المسار (Path Parameters)

```python
from fastapi import FastAPI

app = FastAPI()

@app.get("/tasks/{task_id}")
def get_task(task_id: int) -> dict[str, int]:
    return {"task_id": task_id}
```

جرّب `http://127.0.0.1:8000/tasks/5` :

```json
{"task_id": 5}
```

لاحظ السحر: كتبنا `task_id: int` ، فإن حاولت زيارة `/tasks/abc` (نص بدل رقم)، سيرفض FastAPI الطلب تلقائيًا:

```json
{
  "detail": [
    {
      "type": "int_parsing",
      "loc": ["path", "task_id"],
      "msg": "Input should be a valid integer"
    }
  ]
}
```

هذا هو جوهر الفائدة العملية لـ Type Hints: لم نكتب أي كود تحقق يدوي، لكن FastAPI يتحقق تلقائيًا من النوع، ويرفض الطلبات غير الصحيحة برسالة واضحة، قبل أن يصل الطلب حتى إلى جسم دالتنا.

معاملات الاستعلام (Query Parameters)

أي معامل في توقيع الدالة **لا يظهر** في مسار الـ URL يُعتبر تلقائيًا «معامل استعلام» (Query Parameter):

```python
@app.get("/tasks")
def list_tasks(skip: int = 0, limit: int = 10) -> dict[str, int]:
    return {"skip": skip, "limit": limit}
```

جرّب: `http://127.0.0.1:8000/tasks?skip=5&limit=20`

```json
{"skip": 5, "limit": 20}
```

وإن لم تُحدد القيم: `http://127.0.0.1:8000/tasks` تستخدم القيم الافتراضية `skip=0, limit=10`.

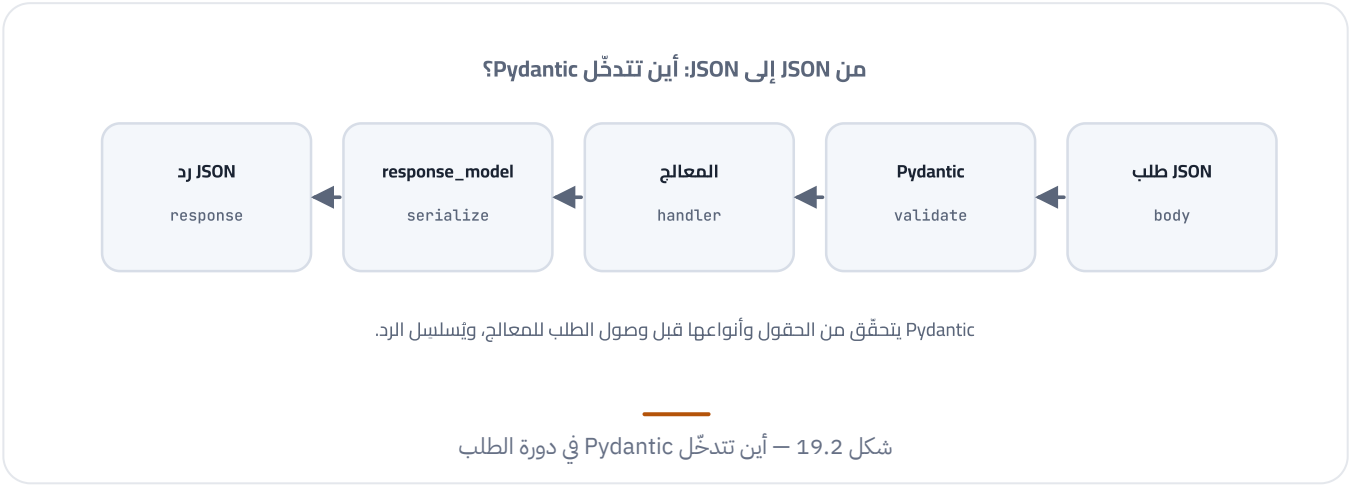
معاملات استعلام اختيارية

```python
@app.get("/search")
def search_tasks(keyword: str | None = None) -> dict[str, str | None]:
    if keyword is None:
        return {"message": "لم يُحدد كلمة بحث"}
    return {"message": f"البحث عن: {keyword}"}
```

مقارنة: Path مقابل Query Parameters

- **Path**: جزء من «هوية» المورد نفسه (`/tasks/5` ← المهمة رقم 5 تحديدًا).
- **Query**: خيارات إضافية لتصفية أو تعديل الطلب (`/tasks?status=done` ← كل المهام المنجزة).

Pydantic 19.6 : نمذجة البيانات الواردة (Request Body)



شكل 19.2 — أين تتدخّل Pydantic في دورة الطلب

حتى الآن تعاملنا مع بيانات بسيطة في المسار أو الاستعلام. لكن عند **إنشاء** مورد جديد (مثل إضافة مهمة)، نحتاج إرسال بيانات **مُهيكَلة أكثر** (عنوان، أولوية، إلخ) في «جسم الطلب» (Request Body). هنا تأتي **Pydantic** — المكتبة التي تعمل في قلب FastAPI.

تعريف نموذج بيانات (Model)

```python
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class TaskCreate(BaseModel):
    title: str
    priority: str = "متوسطة"
    done: bool = False

@app.post("/tasks")
def create_task(task: TaskCreate) -> TaskCreate:
    return task
```

لاحظ كيف تُشبه `TaskCreate` كتابة `@dataclass` من الفصل 14 — لكنها **أقوى بكثير**: `BaseModel` (من Pydantic) تُضيف **تحقق تلقائي صارم من صحة البيانات (Validation)** عند استقبال أي طلب.

جرّب إرسال طلب `POST` (عبر `/docs` أو أداة مثل `curl`) بجسم:

```json
{"title": "تعلم FastAPI"}
```

النتيجة:

```JSON
{"title": "تعلم FastAPI", "priority": "متوسطة", "done": false}
```

والآن جرّب إرسال بيانات خاطئة:

```JSON
{"priority": "عالية"}
```

(بدون `title` الإلزامي):

```JSON
{
  "detail": [
    {
      "type": "missing",
      "loc": ["body", "title"],
      "msg": "Field required"
    }
  ]
}
```

نصيحة الخبير: هذا التحقق التلقائي من صحة البيانات هو **الفائدة الأعظم** لاستخدام Pydantic/FastAPI. في أطر عمل أقدم، كان على المبرمج كتابة عشرات الأسطر يدويًا للتحقق من كل حقل (هل موجود؟ هل من النوع الصحيح؟ هل ضمن النطاق المسموح؟). هنا، التحقق **مُشتق تلقائيًا** من Type Hints التي كتبتها أصلًا لتوثيق الكود.

🔄 مقارنة: `@dataclass` (الفصل 14) مقابل Pydantic `BaseModel`

المعيار	`@dataclass`	Pydantic `BaseModel`
التحقق من الأنواع وقت التشغيل	لا (فقط عبر mypy، ساكن)	**نعم**، فعليًا عند الإنشاء
تحويل تلقائي من/إلى JSON	يدوي (`asdict`)	مدمج بالكامل (`.model_dump()` , `.model_validate_json()`)
رسائل خطأ واضحة عند بيانات خاطئة	لا، يحدث خطأ عادي لاحقًا	نعم، فورية ومفصلة
الاستخدام الأمثل	نماذج بيانات داخلية في برنامجك	بيانات قادمة من **مصدر خارجي غير موثوق** (كطلبات الويب)

قيود تحقق إضافية مع `Field`

```python
from pydantic import BaseModel, Field

class TaskCreate(BaseModel):
    title: str = Field(min_length=1, max_length=100)
    priority: str = Field(default="متوسطة", pattern="^(عالية|متوسطة|منخفضة)$")
    estimated_hours: float = Field(default=1.0, gt=0, le=100)
```

الآن إن أرسل العميل `title` فارغًا، أو `priority` بقيمة غير مدعومة، أو `estimated_hours` سالبة، يرفض FastAPI الطلب تلقائيًا برسالة دقيقة توضح بالضبط ما هو الخطأ.

نماذج الاستجابة (Response Models)

يمكننا أيضًا تحديد **شكل البيانات المُرجَعة** صراحة، لضمان عدم تسريب حقول داخلية حساسة بالخطأ:

```python
class TaskOut(BaseModel):
    id: int
    title: str
    priority: str
    done: bool

@app.post("/tasks", response_model=TaskOut)
def create_task(task: TaskCreate) -> TaskOut:
    new_id = 1  # لاحقًا سيأتي من قاعدة بيانات حقيقية
    return TaskOut(id=new_id, title=task.title, priority=task.priority, done=task.done)
```

19.7 معالجة الأخطاء: `HTTPException`

عندما يطلب العميل موردًا غير موجود، يجب أن نُرجع رمز حالة مناسبًا (`404`) ورسالة واضحة، بدلًا من انهيار الخادم بخطأ غامض.

```python
from fastapi import FastAPI, HTTPException

app = FastAPI()

tasks_db: dict[int, str] = {1: "تعلم Python", 2: "تعلم FastAPI"}

@app.get("/tasks/{task_id}")
def get_task(task_id: int) -> dict[str, str]:
    if task_id not in tasks_db:
        raise HTTPException(status_code=404, detail="المهمة غير موجودة")
    return {"title": tasks_db[task_id]}
```

جرّب `/tasks/999` :

```json
{"detail": "المهمة غير موجودة"}
```

برمز حالة `404` فعليًا (يمكن التحقق منه في تبويب Network بأدوات المطور في المتصفح، أو عبر `/docs`).

> **نصيحة الخبير:** `raise HTTPException(...)` تعمل تمامًا كأي استثناء آخر تعلمناه في الفصل 11 — FastAPI «يلتقط» هذا النوع المحدد من الاستثناءات تلقائيًا في الخلفية، ويحوّله إلى استجابة HTTP مناسبة بالرمز والرسالة التي حددتها.

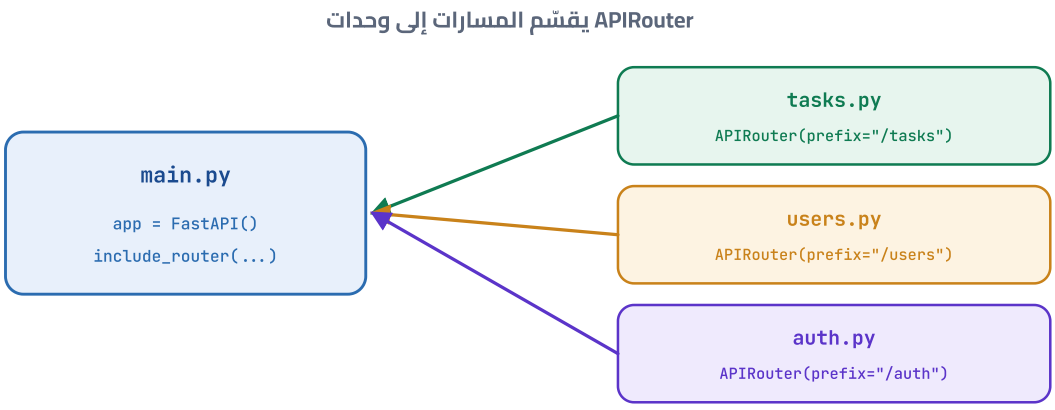
19.8 دعم async بشكل كامل

كل نقطة نهاية (Endpoint) يمكن تعريفها كـ Coroutine باستخدام `async def`، مستفيدة من كل ما تعلمناه في الفصل 17، خاصة عند التعامل مع قواعد بيانات أو طلبات خارجية بطيئة:

```python
import asyncio

@app.get("/slow-operation")
async def slow_operation() -> dict[str, str]:
    await asyncio.sleep(2)  # محاكاة عملية بطيئة (استدعاء API، إلخ، قاعدة بيانات، خارجي)
    return {"message": "اكتملت العملية البطيئة"}
```

> **تذكير من الفصل 17:** لا تستخدم `time.sleep()` أبدًا داخل دالة `async def`! هذا يُجمّد الخادم بأكمله عن خدمة أي طلب آخر أثناء الانتظار. استخدم دائمًا النسخ غير المتزامنة (`asyncio.sleep`, `httpx.AsyncClient`, إلخ).
>
> **نصيحة الخبير:** إن لم تكن نقطة النهاية تحتاج فعليًا لانتظار عملية I/O (كحساب بسيط أو قراءة من قائمة في الذاكرة)، يمكنك تعريفها كدالة عادية (`def` بدون `async`). FastAPI يتعامل بذكاء مع كلا النوعين بكفاءة.

19.9 تنظيم مشروع أكبر: APIRouter



شكل 19.3 — APIRouter يقسّم المسارات

عندما يكبر مشروعك، وضع كل نقاط النهاية في ملف `main.py` واحد يصبح غير عملي. نستخدم `APIRouter` لتقسيم الكود منطقيًا:

Python src/task_api/routers/tasks.py

```python
from fastapi import APIRouter

router = APIRouter(prefix="/tasks", tags=["Tasks"])

@router.get("/")
def list_tasks() -> list[str]:
    return ["مهمة 1", "مهمة 2"]
```

Python src/task_api/main.py

```python
from fastapi import FastAPI
from task_api.routers import tasks

app = FastAPI()
app.include_router(tasks.router)
```

هذا يُشبه تمامًا فكرة تقسيم الكود إلى ملفات منفصلة (Modules) التي تعلمناها عند هيكلة المشاريع في الفصل 18 — `APIRouter` هو ببساطة «فئة فرعية» منظّمة من نقاط النهاية، تُدمج لاحقًا في التطبيق الرئيسي.

19.10 مشروع تطبيقي متكامل: واجهة API لإدارة المهام

لنبني الآن المشروع الكامل: واجهة API حقيقية لإدارة المهام، تدعم كل عمليات CRUD، مع حفظ دائم بصيغة JSON (معتمدين على ما تعلمناه في الفصل 16)، هيكلة احترافية (الفصل 18)، وType Hints كاملة (الفصل 15).

هيكلية المشروع

```
task_api/
├── .venv/
├── pyproject.toml
├── src/
│   └── task_api/
│       ├── __init__.py
│       ├── main.py
│       ├── models.py
│       ├── storage.py
│       └── routers/
│           ├── __init__.py
│           └── tasks.py
└── tasks.json
```

✅ الحل الكامل

src/task_api/models.py

Python

```python
"""نماذج البيانات (Pydantic Models)."""
from __future__ import annotations

from typing import Literal
from pydantic import BaseModel, Field

type Priority = Literal["عالية", "متوسطة", "منخفضة"]

class TaskCreate(BaseModel):
    """البيانات المطلوبة لإنشاء مهمة جديدة."""
    title: str = Field(min_length=1, max_length=200)
    priority: Priority = "متوسطة"

class TaskUpdate(BaseModel):
    """البيانات القابلة للتحديث في مهمة موجودة (كلها اختيارية)."""
    title: str | None = Field(default=None, min_length=1, max_length=200)
    priority: Priority | None = None
    done: bool | None = None

class Task(BaseModel):
    """نموذج المهمة الكامل كما يُخزَّن ويُرجَع للعميل."""
    id: int
    title: str
    priority: Priority
    done: bool = False
```

src/task_api/storage.py

Python

```python
"""طبقة تخزين بسيطة تعتمد على ملف JSON (كما تعلمنا في الفصل 16)."""
from __future__ import annotations

import json
from pathlib import Path

from task_api.models import Task

class TaskStorage:
    def __init__(self, file_path: Path) -> None:
```

```
        self.file_path = file_path
        self._tasks: dict[int, Task] = {}
        self._next_id: int = 1
        self._load()

    def _load(self) -> None:
        if not self.file_path.exists():
            return
        try:
            raw_data = json.loads(self.file_path.read_text(encoding="utf-8"))
            self._tasks = {item["id"]: Task(**item) for item in raw_data}
            if self._tasks:
                self._next_id = max(self._tasks.keys()) + 1
        except (json.JSONDecodeError, KeyError):
            self._tasks = {}

    def _save(self) -> None:
        data = [task.model_dump() for task in self._tasks.values()]
        self.file_path.write_text(
            json.dumps(data, ensure_ascii=False, indent=2),
            encoding="utf-8",
        )

    def get_all(self) -> list[Task]:
        return list(self._tasks.values())

    def get_by_id(self, task_id: int) -> Task | None:
        return self._tasks.get(task_id)

    def create(self, title: str, priority: str) -> Task:
        task = Task(id=self._next_id, title=title, priority=priority, done=False)  # type:
ignore[arg-type]
        self._tasks[task.id] = task
        self._next_id += 1
        self._save()
        return task

    def update(self, task_id: int, **changes: object) -> Task | None:
        task = self._tasks.get(task_id)
        if task is None:
            return None
        updated_task = task.model_copy(update={k: v for k, v in changes.items() if v is not
None})
        self._tasks[task_id] = updated_task
        self._save()
        return updated_task

    def delete(self, task_id: int) -> bool:
        if task_id not in self._tasks:
            return False
        del self._tasks[task_id]
```

```
        self._save()
        return True

# نسخة واحدة مشتركة للتطبيق بالكامل (Singleton بسيط)
storage = TaskStorage(Path("tasks.json"))
```

src/task_api/routers/tasks.py

Python

```python
"""نقاط النهاية الخاصة بإدارة المهام."""
from fastapi import APIRouter, HTTPException, status

from task_api.models import Task, TaskCreate, TaskUpdate
from task_api.storage import storage

router = APIRouter(prefix="/tasks", tags=["Tasks"])

@router.get("/", response_model=list[Task])
def list_tasks(done: bool | None = None) -> list[Task]:
    """يُرجع كل المهام، مع إمكانية التصفية حسب حالة الإنجاز."""
    all_tasks = storage.get_all()
    if done is None:
        return all_tasks
    return [task for task in all_tasks if task.done == done]

@router.get("/{task_id}", response_model=Task)
def get_task(task_id: int) -> Task:
    """يُرجع مهمة واحدة بمعرّفها."""
    task = storage.get_by_id(task_id)
    if task is None:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail=f"لا توجد مهمة بالمعرّف {task_id}",
        )
    return task

@router.post("/", response_model=Task, status_code=status.HTTP_201_CREATED)
def create_task(task_data: TaskCreate) -> Task:
    """ينشئ مهمة جديدة."""
    return storage.create(title=task_data.title, priority=task_data.priority)

@router.patch("/{task_id}", response_model=Task)
def update_task(task_id: int, task_data: TaskUpdate) -> Task:
    """يُحدّث حقولًا معيّنة في مهمة موجودة."""
    updated = storage.update(
        task_id,
        title=task_data.title,
        priority=task_data.priority,
```

```
            done=task_data.done,
        )
        if updated is None:
            raise HTTPException(
                status_code=status.HTTP_404_NOT_FOUND,
                detail=f"لا توجد مهمة بالمعرّف {task_id}",
            )
        return updated

    @router.delete("/{task_id}", status_code=status.HTTP_204_NO_CONTENT)
    def delete_task(task_id: int) -> None:
        """يحذف مهمة موجودة."""
        deleted = storage.delete(task_id)
        if not deleted:
            raise HTTPException(
                status_code=status.HTTP_404_NOT_FOUND,
                detail=f"لا توجد مهمة بالمعرّف {task_id}",
            )
```

src/task_api/main.py

Python

```
"""نقطة الدخول الرئيسية لتطبيق FastAPI."""
from fastapi import FastAPI

from task_api.routers import tasks

app = FastAPI(
    title="واجهة API لإدارة المهام",
    description="مشروع تطبيقي من كتاب تعلّم Python",
    version="1.0.0",
)

app.include_router(tasks.router)

@app.get("/", tags=["Root"])
def read_root() -> dict[str, str]:
    return {"message": "مرحبًا بك في واجهة API لإدارة المهام! زُر /docs للتوثيق التفاعلي"}
```

`pyproject.toml`

```toml
[project]
name = "task-api"
version = "1.0.0"
requires-python = "⩾3.14"
dependencies = [
    "fastapi⩾0.110.0",
    "pydantic⩾2.6.0",
]

[dependency-groups]
dev = [
    "uvicorn[standard]⩾0.29.0",
    "mypy⩾1.8.0",
]

[build-system]
requires = ["hatchling"]
build-backend = "hatchling.build"
```

تشغيل المشروع

```terminal
uv run uvicorn task_api.main:app --reload
```

تجربة الـ API

Terminal

```
# إنشاء مهمة جديدة
curl -X POST http://127.0.0.1:8000/tasks/ \
  -H "Content-Type: application/json" \
  -d '{"title": "تعلم بناء APIs", "priority": "عالية"}'

# عرض كل المهام
curl http://127.0.0.1:8000/tasks/

# تحديث مهمة (وضعها كمنجزة)
curl -X PATCH http://127.0.0.1:8000/tasks/1 \
  -H "Content-Type: application/json" \
  -d '{"done": true}'

# حذف مهمة
curl -X DELETE http://127.0.0.1:8000/tasks/1
```

أو ببساطة أكبر: افتح `http://127.0.0.1:8000/docs` وجرّب كل العمليات بصريًا!

19.11 أفضل الممارسات

افصل دائمًا بين النماذج (Models) والمنطق (Logic) والمسارات (Routers)

كما في المشروع أعلاه: `models.py` للبيانات، `storage.py` للمنطق، `routers/` للمسارات — لا تضع كل شيء في `main.py` واحد ضخم.

استخدم `response_model` دائمًا

يضمن عدم تسريب بيانات داخلية حساسة بالخطأ، ويُحسّن التوثيق التلقائي في `/docs` .

استخدم رموز حالة HTTP الصحيحة

```python
status.HTTP_200_OK          # نجاح عام
status.HTTP_201_CREATED     # بعد إنشاء مورد جديد (POST)
status.HTTP_204_NO_CONTENT  # نجاح بدون محتوى مُرجَع (DELETE)
status.HTTP_404_NOT_FOUND   # المورد غير موجود
status.HTTP_422_UNPROCESSABLE_ENTITY  # بيانات غير صالحة (يتم تلقائيًا من Pydantic)
```

⚠ **خطأ شائع:** إرجاع `200 OK` لكل شيء، حتى عند إنشاء مورد جديد (الأصح `201`) أو فشل العثور عليه (الأصح `404` بدل `200` مع رسالة خطأ داخل الجسم). الالتزام بمعايير REST يجعل واجهتك متوقعة وسهلة الاستخدام لأي عميل يتعامل معها.

لا تثق أبدًا ببيانات العميل دون تحقق (هذا ما تمنحه لك Pydantic تلقائيًا)

19.12 تمارين

🟢 مستوى مبتدئ

تمرين 1: أنشئ واجهة API بسيطة بها نقطة نهاية `GET /greet/{name}` تُرجع `{"message": "مرحبًا، {name}!"}`.

تمرين 2: أضف نقطة نهاية `GET /calculate` تستقبل معاملي استعلام `a` و `b` (أعداد صحيحة)، وتُرجع ناتج جمعهما.

✅ الحل النموذجي

```python
from fastapi import FastAPI

app = FastAPI()

@app.get("/greet/{name}")
def greet(name: str) -> dict[str, str]:
    return {"message": f"مرحبًا، {name}!"}

@app.get("/calculate")
def calculate(a: int, b: int) -> dict[str, int]:
    return {"result": a + b}
```

🟡 مستوى متوسط

تمرين 3: أضف إلى مشروع «إدارة المهام» نقطة نهاية `GET /tasks/stats` تُرجع عدد المهام الكلي، عدد المنجزة، وعدد غير المنجزة.

تمرين 4: أضف حقل `due_date: date | None` إلى نموذج `Task`، مع التحقق المناسب.

🔴 تحدي

تمرين 5: أضف نظام «فئات» (Categories) للمهام: نقطة نهاية جديدة `GET /categories` و `POST /categories`، وربط كل مهمة بـ `category_id` اختياري، مع التحقق من وجود الفئة قبل قبول المهمة.

19.13 اختبر فهمك (Quiz)

❓ السؤال 1: ما الفرق بين Path Parameter و Query Parameter؟

Path Parameter جزء من مسار الـ URL نفسه ويُحدد هوية مورد معيّن (`/tasks/5`). Query Parameter يُضاف بعد `?` ويُستخدم لتصفية أو تعديل سلوك الطلب (`/tasks?done=true`).

❓ السؤال 2: لماذا نستخدم `BaseModel` من `Pydantic` بدلًا من قواميس عادية لاستقبال بيانات الطلب؟

لأن Pydantic يوفر تحققًا تلقائيًا وصارمًا من صحة البيانات (الأنواع، الحقول الإلزامية، القيود)، ويرفض الطلبات غير الصالحة تلقائيًا برسائل خطأ واضحة، دون الحاجة لكتابة أي كود تحقق يدوي.

❓ السؤال 3: ما الفرق بين رمزي الحالة 404 و 422؟

404 تعني أن المورد المطلوب غير موجود إطلاقًا (مثل مهمة بمعرّف غير موجود). 422 تعني أن البيانات المُرسَلة في جسم الطلب غير صالحة من حيث الصيغة أو النوع (مثل إرسال نص بدل رقم، أو نسيان حقل إلزامي).

19.14 ملخص الفصل (Cheat Sheet)

```python
from fastapi import FastAPI, HTTPException, status
from pydantic import BaseModel, Field

app = FastAPI()

class ItemCreate(BaseModel):
    name: str = Field(min_length=1)
    price: float = Field(gt=0)

@app.get("/items/{item_id}")
def get_item(item_id: int, q: str | None = None) -> dict:
    ...

@app.post("/items", status_code=status.HTTP_201_CREATED)
def create_item(item: ItemCreate) -> ItemCreate:
    ...

@app.patch("/items/{item_id}")
def update_item(item_id: int, item: ItemCreate) -> ItemCreate:
    ...

@app.delete("/items/{item_id}", status_code=status.HTTP_204_NO_CONTENT)
def delete_item(item_id: int) -> None:
    ...

# تشغيل:
# uv run uvicorn main:app --reload
# التوثيق: http://127.0.0.1:8000/docs
```

المفهوم	الاستخدام
`@app.get/post/put/delete`	تعريف نقطة نهاية
Path Parameter	`{id}` في المسار
Query Parameter	معامل دالة غير موجود بالمسار
`BaseModel`	نموذج بيانات الطلب/الاستجابة
`HTTPException`	إرجاع خطأ HTTP مناسب
`/docs`	توثيق تفاعلي تلقائي

19.15 ماذا بعد؟

لقد بنيت للتو واجهة API حقيقية وكاملة! في الفصل القادم، ننتقل لمسار مختلف تمامًا، لكنه بنفس الأهمية: **تحليل البيانات**، حيث سنتعلم مكتبات `pandas` و`numpy` و`matplotlib` لتحليل وتصور بيانات حقيقية، عبر مشروع تحليل مبيعات متكامل.

فصل 20 / 26 · الجزء الخامس

20

مسار تحليل البيانات (pandas, numpy, matplotlib)

11 دقيقة قراءة | 11 قسمًا | 36 مثالًا | 2,220 كلمة

في هذا الفصل

20.1 لماذا تحليل البيانات؟
20.2 تثبيت المكتبات
20.3 NumPy: أساس الحوسبة الرقمية
20.4 pandas: العمود الفقري لتحليل البيانات
20.5 Matplotlib: تحويل الأرقام إلى رسوم
20.6 مشروع تطبيقي متكامل: تحليل مبيعات متجر إلكتروني
20.7 أخطاء شائعة في تحليل البيانات
20.8 تمارين
20.9 اختبر فهمك (Quiz)
20.10 ملخص الفصل (Cheat Sheet)
20.11 ماذا بعد؟

20.1 لماذا تحليل البيانات؟

تخيّل أنك تملك ملف Excel يحتوي على **500,000 صف** من بيانات المبيعات، وتريد معرفة: ما هو المنتج الأكثر مبيعًا في كل مدينة خلال الربع الأخير؟ ما متوسط قيمة الفاتورة لكل عميل؟ هل هناك اتجاه تصاعدي أو تنازلي في المبيعات؟

في Excel، ستحتاج إلى معادلات معقدة، جداول محورية (Pivot Tables)، وربما ساعات من النقر والسحب — وقد ينهار البرنامج أصلاً مع هذا الحجم من البيانات. في Python، يمكنك الإجابة على هذه الأسئلة في **أسطر معدودة**، بسرعة فائقة، وبطريقة **قابلة لإعادة الاستخدام**: نفس الكود يعمل غدًا على بيانات جديدة دون أي تعديل.

هذا هو سبب كون **تحليل البيانات (Data Analysis)** أحد أقوى تطبيقات Python في العالم الحقيقي. شركات مثل Netflix وSpotify وAmazon تستخدم بالضبط المكتبات التي ستتعلمها في هذا الفصل لاتخاذ قرارات تجارية تساوي ملايين الدولارات.

مقارنة: Excel مقابل Python لتحليل البيانات

المعيار	Excel	Python (pandas)
حجم البيانات	يتعثر مع أكثر من ~1 مليون صف	يتعامل بسهولة مع ملايين الصفوف
التكرار	يدوي، عرضة للخطأ البشري	كود يُعاد تشغيله تلقائيًا
التوثيق	صعب تتبع «ماذا فعلت بالضبط؟»	الكود نفسه هو التوثيق
التكامل	محدود	يتكامل مع قواعد البيانات، APIs، الذكاء الاصطناعي
التعلم الآلي	غير ممكن مباشرة	يتكامل مباشرة مع scikit-learn, TensorFlow
الرسوم البيانية	جيدة للبساطة	أقوى وأكثر تخصيصًا (matplotlib, seaborn, plotly)

الخلاصة: Excel ممتاز للمهام السريعة والصغيرة. لكن عندما تكبر البيانات أو تتكرر المهمة، Python هو الخيار الاحترافي.

في هذا الفصل، سنتعلم ثلاث مكتبات تُشكّل «الثالوث المقدس» لتحليل البيانات في Python:

1. **NumPy**: الأساس الرياضي — يتعامل مع المصفوفات (Arrays) والعمليات الحسابية بسرعة فائقة.
2. **pandas**: الأداة الرئيسية — تتعامل مع البيانات الجدولية (صفوف وأعمدة) كما في Excel، لكن ببرمجة.
3. **Matplotlib**: التصور — تحويل الأرقام إلى رسوم بيانية مفهومة.

ثم سنطبق كل هذا في **مشروع متكامل لتحليل بيانات مبيعات حقيقية**.

20.2 تثبيت المكتبات

كما تعلمنا في الفصل 18، نستخدم `uv` لإدارة التبعيات:

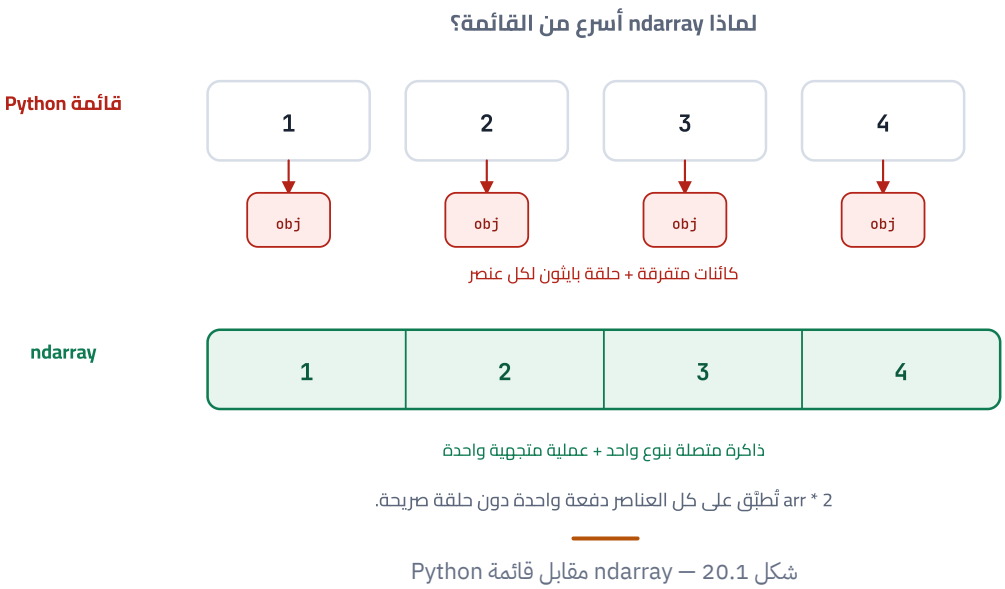
```
Terminal
uv add numpy pandas matplotlib
```

هذا الأمر سيحمّل ويثبّت المكتبات الثلاث، ويضيفها إلى `pyproject.toml`.

نصيحة خبير: غالبًا تُستخدم أسماء مستعارة (aliases) قياسية ومتعارف عليها عالميًا عند استيراد هذه المكتبات. التزم بها دائمًا — فكل مطوّر بيانات في العالم يتوقعها:

```python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

NumPy `20.3` : أساس الحوسبة الرقمية



شكل 20.1 — ndarray مقابل قائمة Python

لماذا لا نستخدم القوائم العادية؟

في الفصل 8، تعلمنا القوائم (`list`). لماذا نحتاج شيئًا آخر؟

```python
# محاولة جمع قائمتين من الأرقام عنصرًا بعنصر
prices = [10, 20, 30]
discounts = [1, 2, 3]

# هذا لن يعمل كما تتوقع!
result = prices + discounts
print(result)  # [10, 20, 30, 1, 2, 3] -- دمج وليس جمع!
```

القوائم العادية لا تدعم العمليات الرياضية «الطبيعية» بين عناصرها. لحل هذه المشكلة، نحتاج حلقة `for`:

```python
result = [p - d for p, d in zip(prices, discounts)]
print(result)  # [9, 18, 27]
```

هذا يعمل، لكنه **بطيء جدًا** مع ملايين العناصر، لأن Python يُنفّذ الحلقة عنصرًا تلو الآخر بشكل مفسَّر (interpreted).

NumPy يحل هذه المشكلة عبر هيكل بيانات جديد يُسمى `ndarray` (N-dimensional array)، مُحسَّن داخليًا بلغة C، يسمح بإجراء العمليات الحسابية على **كل العناصر دفعة واحدة** (Vectorization) بسرعة تفوق القوائم العادية بعشرات أو مئات المرات.

```python
import numpy as np

prices = np.array([10, 20, 30])
discounts = np.array([1, 2, 3])

result = prices - discounts
print(result)  # [9 18 27]
print(type(result))  # <class 'numpy.ndarray'>
```

مقارنة: list مقابل ndarray

الخاصية	`list`	`numpy.ndarray`
أنواع العناصر	مختلطة (int, str, ...)	نوع واحد موحّد (كلها int أو كلها float...)
العمليات الحسابية	تحتاج حلقات	مباشرة (Vectorized)
السرعة	أبطأ	أسرع بكثير (مُنفّذة بلغة C)
استهلاك الذاكرة	أكبر	أقل بكثير
الاستخدام	عام (أي بيانات)	حسابات رقمية/علمية

إنشاء المصفوفات

```python
import numpy as np

# من قائمة
a = np.array([1, 2, 3, 4, 5])
print(a)          # [1 2 3 4 5]
print(a.dtype)    # int64 (نوع البيانات الموحّد)

# مصفوفة ثنائية الأبعاد (مصفوفة من مصفوفات = جدول)
matrix = np.array([
    [1, 2, 3],
    [4, 5, 6],
])
print(matrix.shape)  # (2, 3) -- صفّان، 3 أعمدة

# دوال إنشاء جاهزة
zeros = np.zeros(5)           # [0. 0. 0. 0. 0.]
ones = np.ones((2, 3))        # مصفوفة 2x3 من الآحاد
range_arr = np.arange(0, 10, 2)   # [0 2 4 6 8]  (مثل range لكن تُرجع array)
linspace = np.linspace(0, 1, 5)   # [0.   0.25 0.5  0.75 1.  ] -- 5 قيم متباعدة بالتساوي
random_arr = np.random.rand(3)    # 3 أرقام عشوائية بين 0 و1
```

العمليات الحسابية (Vectorization)

هذه هي القوة الحقيقية لـ NumPy — لاحظ عدم وجود أي حلقة `for`:

```python
prices = np.array([100, 200, 300, 400])

# عملية على كل عنصر دفعة واحدة
with_tax = prices * 1.15
print(with_tax)  # [115. 230. 345. 460.]

# مقارنة تُرجع مصفوفة من Boolean
expensive = prices > 250
print(expensive)  # [False False  True  True]

# استخدام هذه المصفوفة Boolean للتصفية! (Boolean Indexing)
print(prices[expensive])  # [300 400]

# أو في سطر واحد
print(prices[prices > 250])  # [300 400]
```

إحصائيات أساسية

```python
sales = np.array([120, 340, 210, 450, 90, 380])

print(sales.sum())      # 1590 -- المجموع
print(sales.mean())     # 265.0 -- المتوسط
print(sales.max())      # 450 -- الأكبر
print(sales.min())      # 90 -- الأصغر
print(sales.std())      # الانحراف المعياري
print(sales.argmax())   # 3 -- فهرس العنصر الأكبر
print(np.median(sales))  # الوسيط
```

⚠ **تحذير: Broadcasting قد يُربك المبتدئين**

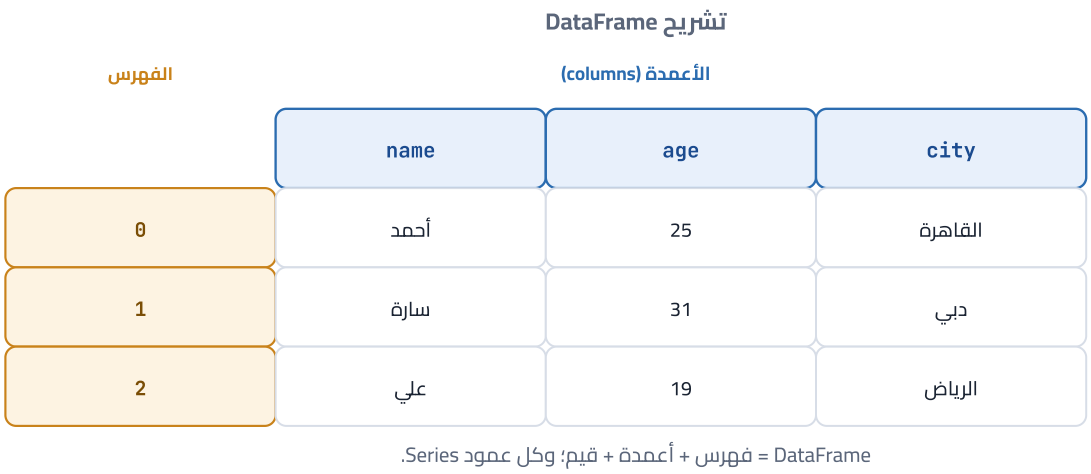
عندما تجري عملية بين مصفوفة وعدد مفرد (scalar)، يطبّق NumPy العملية على كل عنصر تلقائيًا — هذا يُسمى **Broadcasting**:

```python
arr = np.array([1, 2, 3])
print(arr + 10)  # [11 12 13] -- وليس خطأ!
```

هذا سلوك **مقصود ومفيد جدًا**، لكن يجب أن تفهمه حتى لا يفاجئك.

💡 **نصيحة خبير:** NumPy هو الأساس الذي تُبنى عليه pandas وscikit-learn وTensorFlow. لن تحتاج غالبًا لاستخدامه مباشرة بشكل مكثف في تحليل البيانات اليومي (ستستخدم pandas بدلاً منه)، لكن **فهمه ضروري** لأن pandas مبني عليه بالكامل، وستقابله باستمرار.

pandas 20.4 : العمود الفقري لتحليل البيانات



شكل 20.2 — تشريح DataFrame

إذا كان NumPy يتعامل مع أرقام في مصفوفات، فإن **pandas** يتعامل مع بيانات **جدولية** حقيقية: صفوف وأعمدة لها أسماء، أنواع بيانات مختلفة (نصوص، أرقام، تواريخ)، وقيم مفقودة — تمامًا كملف Excel أو جدول في قاعدة بيانات.

هيكلا البيانات الأساسيان

1. **`Series`** : عمود واحد من البيانات (كـ«قائمة» لكن أذكى، بفهارس مسمّاة).

```python
import pandas as pd

temperatures = pd.Series([22, 25, 19, 30], index=["الأحد", "الاثنين", "الثلاثاء", "الأربعاء"])
print(temperatures)
```

```
الأحد      22
الاثنين    25
الثلاثاء   19
الأربعاء   30
dtype: int64
```

2. **`DataFrame`** : جدول كامل — مجموعة من `Series` تشترك في نفس الفهرس. هذا هو الهيكل الذي ستستخدمه 95% من الوقت.

```python
data = {
    "اسم المنتج": ["لابتوب", "ماوس", "لوحة مفاتيح", "شاشة"],
    "السعر": [3500, 50, 120, 900],
    "الكمية": [10, 50, 30, 15],
}

df = pd.DataFrame(data)
print(df)
```

```
   اسم المنتج  السعر  الكمية
0      لابتوب   3500      10
1        ماوس     50      50
2  لوحة مفاتيح    120      30
3        شاشة    900      15
```

مقارنة: DataFrame مقابل ورقة Excel

Excel	pandas
ورقة عمل (Sheet)	`DataFrame`
عمود	`Series`
رقم الصف	`index`
اسم العمود	`columns`
جدول محوري (Pivot)	`.groupby()` + `.pivot_table()`
فلتر/تصفية	Boolean Indexing
VLOOKUP	`.merge()`

قراءة البيانات من ملفات

نادرًا ما نُنشئ DataFrame يدويًا؛ عادة نقرأه من ملف:

```python
# من CSV
df = pd.read_csv("sales.csv")

# من Excel (يتطلب: uv add openpyxl)
df = pd.read_excel("sales.xlsx", sheet_name="2024")

# من JSON
df = pd.read_json("sales.json")

# تحديد الترميز إن كانت البيانات عربية
df = pd.read_csv("sales.csv", encoding="utf-8")
```

⚠ **خطأ شائع:** نسيان `encoding="utf-8"` عند قراءة ملفات تحتوي نصوصًا عربية قد يؤدي إلى ظهور رموز غريبة (Mojibake) أو خطأ `UnicodeDecodeError`. إذا واجهت مشكلة، جرّب `encoding="utf-8-sig"` (خصوصًا للملفات المُصدَّرة من Excel على Windows).

استكشاف البيانات: أول خطوة دائمًا

بمجرد تحميل أي بيانات، نبدأ دائمًا بهذه الأوامر الاستكشافية:

```python
df.head()       # أول 5 صفوف (الافتراضي)
df.head(10)     # أول 10 صفوف
df.tail(3)      # آخر 3 صفوف

df.info()       # معلومات عامة: الأعمدة، الأنواع، القيم غير الفارغة
df.describe()   # إحصائيات (متوسط، انحراف معياري، أدنى، أقصى ...) للأعمدة الرقمية.

df.shape        # (عدد الصفوف, عدد الأعمدة) -- خاصية وليست دالة!
df.columns      # أسماء الأعمدة
df.dtypes       # نوع بيانات كل عمود
df.index        # معلومات الفهرس
```

مثال عملي:

Python

```python
import pandas as pd

df = pd.DataFrame({
    "المنتج": ["لابتوب", "ماوس", "شاشة", "لابتوب", "ماوس"],
    "السعر": [3500, 50, 900, 3600, 55],
    "الكمية": [2, 10, 3, 1, 8],
    "المدينة": ["الرياض", "جدة", "الرياض", "الدمام", "جدة"],
})

print(df.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5 entries, 0 to 4
Data columns (total 4 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   المنتج  5 non-null      object
 1   السعر   5 non-null      int64
 2   الكمية  5 non-null      int64
 3   المدينة 5 non-null      object
dtypes: int64(2), object(2)
```

الفهرسة والاختيار: `loc` و `iloc`

هذا أهم مفهوم عليك إتقانه في pandas.

Python

```python
# اختيار عمود واحد ← يُرجع Series
print(df["المنتج"])

# اختيار عدة أعمدة ← يُرجع DataFrame (لاحظ قائمة داخل قوسين)
print(df[["السعر", "المنتج"]])

# loc: الاختيار بالاسم/التسمية
print(df.loc[0])              # الصف الأول (حسب التسمية، وهنا تتطابق مع الرقم)
print(df.loc[0, "المنتج"])    # قيمة محددة
print(df.loc[0:2, "السعر"])   # نطاق من الصفوف، عمود محدد

# iloc: الاختيار بالموضع الرقمي (مثل القوائم تمامًا)
print(df.iloc[0])         # أول صف بغض النظر عن تسميته
print(df.iloc[0, 1])      # الصف 0، العمود 1
print(df.iloc[-1])        # آخر صف
```

مقارنة: `loc` مقابل `iloc`

	`loc`	`iloc`
يعتمد على	**التسمية (label)**	**الموضع (position)**
مثال	`df.loc["a"]`	`df.iloc[0]`
النطاقات	شاملة للطرفين `0:2` تشمل 2	غير شاملة للطرف الأخير، مثل `range`

التصفية (Filtering) بـ Boolean Indexing

تمامًا كما في NumPy:

```python
# كل المنتجات التي سعرها أكبر من 500
expensive = df[df["السعر"] > 500]
print(expensive)

# شروط متعددة: استخدم & و | (وليس and/or) مع أقواس إلزامية
cheap_riyadh = df[(df["السعر"] < 1000) & (df["المدينة"] == "الرياض")]
print(cheap_riyadh)

# التحقق من انتماء القيمة لقائمة
target_cities = df[df["المدينة"].isin(["الدمام", "جدة"])]

# استبعاد قيم
not_riyadh = df[df["المدينة"] != "الرياض"]
```

تحذير: `and` / `or` لا تعمل مع pandas!

```python
# ❌ سيرفع -- خطأ ValueError
df[(df[«السعر»] > 100) and (df[«الكمية»] > 5)]

# ✅ صحيح -- استخدم & و | مع أقواس حول كل شرط
df[(df[«السعر»] > 100) & (df[«الكمية»] > 5)]
```

السبب: `and` / `or` في Python تتوقع قيمة منطقية واحدة (True/False)، بينما `(df["السعر"] > 100)` تُرجع `Series` كاملة من قيم Boolean، وليس قيمة واحدة. عوامل `&` و `|` مصمّمة للعمل عنصرًا بعنصر.

إضافة وتعديل وحذف الأعمدة

```python
# إضافة عمود جديد محسوب
df["الإجمالي"] = df["السعر"] * df["الكمية"]
print(df)

# إضافة عمود بشرط باستخدام numpy.where
df["الفئة"] = np.where(df["السعر"] > 500, "غالي", "رخيص")

# تعديل عمود موجود
df["السعر"] = df["السعر"] * 1.15  # إضافة ضريبة 15%

# حذف عمود
df = df.drop(columns=["الفئة"])

# إعادة تسمية الأعمدة
df = df.rename(columns={"السعر": "سعر_الوحدة"})
```

التعامل مع القيم المفقودة (Missing Data)

البيانات الحقيقية دائمًا «فوضوية» — تحتوي على قيم مفقودة تُمثَّل في pandas بـ `NaN` (Not a Number).

```python
data = {
    "اسم": ["سارة", "أحمد", None, "خالد"],
    "العمر": [25, None, 30, 28],
}
df = pd.DataFrame(data)

print(df.isna())          # جدول Boolean يوضح أماكن القيم المفقودة
print(df.isna().sum())    # عدد القيم المفقودة في كل عمود

# حذف الصفوف التي تحتوي أي قيمة مفقودة
clean_df = df.dropna()

# حذف فقط إذا كان عمود معين فارغًا
clean_df = df.dropna(subset=["اسم"])

# تعويض القيم المفقودة بقيمة محددة
df["العمر"] = df["العمر"].fillna(df["العمر"].mean())  # بالمتوسط
df["اسم"] = df["اسم"].fillna("غير معروف")
```

> **نصيحة خبير:** القرار بين حذف القيم المفقودة (`dropna`) أو تعويضها (`fillna`) يعتمد على السياق. إذا كانت نسبة القيم المفقودة صغيرة جدًا (أقل من 5%)، الحذف غالبًا آمن. إذا كانت كبيرة، التعويض الذكي (بالمتوسط، الوسيط، أو قيمة منطقية) أفضل لتجنب فقدان بيانات ثمينة.

الفرز (Sorting)

```python
df_sorted = df.sort_values("السعر")                    # تصاعدي
df_sorted = df.sort_values("السعر", ascending=False)    # تنازلي
df_sorted = df.sort_values(["المدينة", "السعر"])         # فرز متعدد المستويات
```

التجميع: `groupby` — أقوى أداة في pandas

`groupby` يُجيب على أسئلة مثل: «ما مجموع المبيعات **لكل** مدينة؟»

```python
df = pd.DataFrame({
    "المنتج": ["لابتوب", "ماوس", "شاشة", "لابتوب", "ماوس", "شاشة"],
    "المدينة": ["الرياض", "جدة", "الرياض", "جدة", "الرياض", "جدة"],
    "الإجمالي": [1800, 450, 3600, 2700, 500, 7000],
})

# مجموع المبيعات لكل مدينة
by_city = df.groupby("المدينة")["الإجمالي"].sum()
print(by_city)
```

```
المدينة
الرياض    10150
جدة        5900
Name: الإجمالي, dtype: int64
```

```python
# عدة إحصائيات دفعة واحدة
summary = df.groupby("المدينة")["الإجمالي"].agg(["sum", "mean", "count", "max"])
print(summary)

# التجميع حسب أكثر من عمود
by_city_product = df.groupby(["المنتج" ,"المدينة"])["الإجمالي"].sum()
print(by_city_product)

# تحويل النتيجة إلى DataFrame عادي مرة أخرى
result = df.groupby("المدينة")["الإجمالي"].sum().reset_index()
```

مقارنة: `groupby` في pandas مقابل `GROUP BY` في SQL

```sql
SELECT المدينة, SUM(الإجمالي)
FROM sales
GROUP BY المدينة;
```

```python
df.groupby(«المدينة»)[«الإجمالي»].sum()
```

الفكرة متطابقة تمامًا! إذا كنت تعرف SQL، فهم `groupby` سيكون سهلاً جدًا.

الجداول المحورية: `pivot_table`

```python
pivot = df.pivot_table(
    values="الإجمالي",
    index="المدينة",
    columns="المنتج",
    aggfunc="sum",
    fill_value=0,
)
print(pivot)
```

```
المنتج    لابتوب  ماوس  شاشة
المدينة
الرياض      7000   450   2700
جدة         3600   500   1800
```

تمامًا كجدول Pivot في Excel، لكن بسطر واحد وقابل لإعادة الاستخدام على أي بيانات.

دمج البيانات: `merge` و `concat`

Python

```
customers = pd.DataFrame({
    "customer_id": [1, 2, 3],
    "["خالد" ,"سارة" ,"أحمد"] :"اسم",
})

orders = pd.DataFrame({
    "order_id": [101, 102, 103, 104],
    "customer_id": [1, 2, 1, 3],
    "[300 ,120 ,450 ,200] :"المبلغ",
})

# الدمج (مثل JOIN في SQL)
merged = pd.merge(orders, customers, on="customer_id", how="left")
print(merged)

# دمج/تكديس DataFrames (مثل إلحاق ملفات شهر بشهر فوق بعضها)
jan_sales = pd.DataFrame({"[3500] :"الإجمالي" ,["لابتوب"] :"المنتج"})
feb_sales = pd.DataFrame({"[50] :"الإجمالي" ,["ماوس"] :"المنتج"})
all_sales = pd.concat([jan_sales, feb_sales], ignore_index=True)
```

جدول مرجعي: أنواع `how` في `merge`

النوع	الوصف
`"inner"` (افتراضي)	فقط الصفوف المتطابقة في الجدولين
`"left"`	كل صفوف الجدول الأيسر + المطابقة من الأيمن
`"right"`	كل صفوف الجدول الأيمن + المطابقة من الأيسر
`"outer"`	كل الصفوف من الجدولين (مع `NaN` حيث لا تطابق)

التعامل مع التواريخ

```python
df = pd.DataFrame({
    "التاريخ": ["10-03-2024", "20-02-2024", "15-01-2024"],
    "المبيعات": [1200, 1500, 1000],
})

# تحويل النص إلى تاريخ حقيقي
df["التاريخ"] = pd.to_datetime(df["التاريخ"])

# استخراج معلومات من التاريخ
df["الشهر"] = df["التاريخ"].dt.month
df["اسم_اليوم"] = df["التاريخ"].dt.day_name()

print(df)
```

تصدير البيانات

```python
df.to_csv("output.csv", index=False, encoding="utf-8-sig")
df.to_excel("output.xlsx", index=False)
df.to_json("output.json", orient="records", force_ascii=False, indent=2)
```

نصيحة خبير: استخدم دائمًا `index=False` عند التصدير إلى CSV/Excel، وإلا ستحصل على عمود إضافي غير مرغوب فيه يحتوي أرقام الفهرس الداخلية لـ pandas.

Matplotlib 20.5 : تحويل الأرقام إلى رسوم

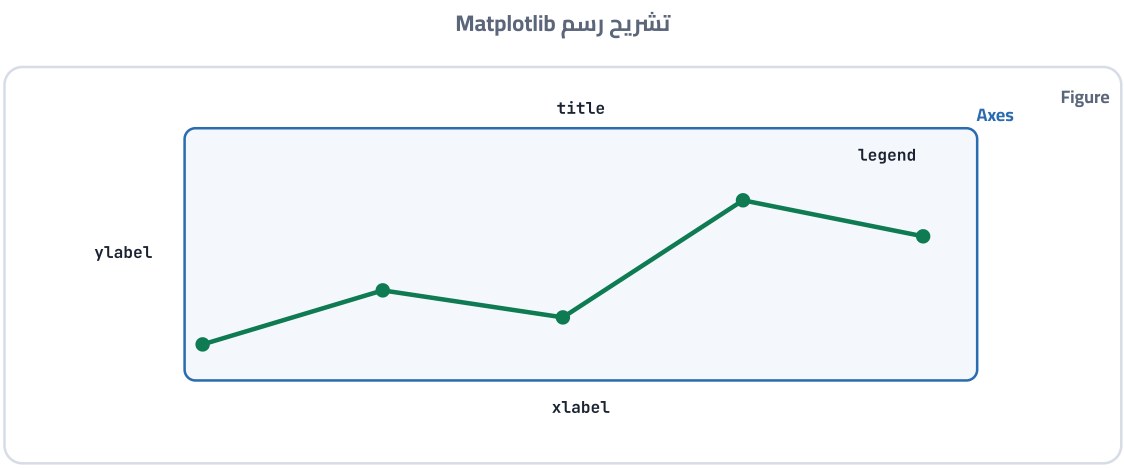


Figure الحاوية، Axes منطقة الرسم، ثم الخطوط والعناوين والمفتاح.

شكل 20.3 — تشريح رسم Matplotlib

الأرقام في جدول صعبة الفهم بصريًا. رسم بياني واحد قد يوضح اتجاهًا يستغرق فهمه من الجدول دقائق طويلة.

الرسم الأساسي

```python
import matplotlib.pyplot as plt

months = ["يناير", "فبراير", "مارس", "أبريل", "مايو"]
sales = [1200, 1900, 1500, 2200, 2800]

plt.plot(months, sales)
plt.title("المبيعات الشهرية")
plt.xlabel("الشهر")
plt.ylabel("المبيعات (ر.س)")
plt.show()  # يفتح نافذة لعرض الرسم
```

⚠ **تحذير: دعم اللغة العربية في matplotlib**

matplotlib لا يدعم اتجاه الكتابة من اليمين لليسار (RTL) أو ربط الحروف العربية بشكل جيد افتراضيًا — قد تظهر الحروف منفصلة أو معكوسة. للحصول على نص عربي صحيح، استخدم مكتبة `arabic-reshaper` مع `python-bidi`:

```python
import arabic_reshaper
from bidi.algorithm import get_display

def fix_arabic(text: str) -> str:
    reshaped = arabic_reshaper.reshape(text)
    return get_display(reshaped)

plt.title(fix_arabic("المبيعات الشهرية"))
```

في الأمثلة التالية سنستخدم أحيانًا تسميات إنجليزية للتبسيط، لكن اعتمد هذا الحل في مشاريعك الحقيقية عند الحاجة لنصوص عربية في الرسوم.

أنواع الرسوم الشائعة

Python

```python
import matplotlib.pyplot as plt

categories = ["لابتوب", "ماوس", "شاشة", "لوحة مفاتيح"]
values = [45, 120, 30, 80]

# رسم بياني خطي -- مناسب للاتجاهات عبر الزمن
plt.plot(categories, values, marker="o")
plt.show()

# رسم بياني شريطي (Bar Chart) -- مناسب للمقارنة بين فئات
plt.bar(categories, values, color="steelblue")
plt.show()

# رسم دائري (Pie Chart) -- مناسب لإظهار النسب من الكل
plt.pie(values, labels=categories, autopct="%1.1f%%")
plt.show()

# مخطط التشتت (Scatter Plot) -- مناسب لإظهار العلاقة بين متغيرين
prices = [3500, 50, 900, 120]
plt.scatter(prices, values)
plt.xlabel("السعر")
plt.ylabel("الكمية المباعة")
plt.show()

# المدرج التكراري (Histogram) -- مناسب لتوزيع البيانات
import numpy as np
ages = np.random.normal(30, 5, 1000)  # 1000 عمر عشوائي حول متوسط 30
plt.hist(ages, bins=20, color="coral", edgecolor="black")
plt.show()
```

جدول مرجعي: أي رسم تختار؟

نوع البيانات	السؤال	الرسم المناسب
تطور عبر الزمن	هل المبيعات تزداد؟	(خطي) `plt.plot()`
مقارنة فئات	أي منتج الأكثر مبيعًا؟	(شريطي) `plt.bar()`
نسب من كل	ما حصة كل قسم من الميزانية؟	(دائري) `plt.pie()`
علاقة بين متغيرين	هل السعر يؤثر على الكمية المباعة؟	(تشتت) `plt.scatter()`
توزيع قيم	كيف تتوزع أعمار العملاء؟	(مدرج تكراري) `plt.hist()`

تخصيص الرسوم

```python
plt.figure(figsize=(10, 6))  # حجم الرسم (عرض, ارتفاع) بالبوصة

plt.plot(months, sales, marker="o", linestyle="--", color="green", linewidth=2, label="2024")
plt.title("المبيعات الشهرية", fontsize=16, fontweight="bold")
plt.xlabel("الشهر")
plt.ylabel("المبيعات (﷼)")
plt.legend()         # إظهار الـ label الموجود أعلاه
plt.grid(True, alpha=0.3)   # شبكة خفيفة في الخلفية
plt.tight_layout()  # ترتيب تلقائي لمنع تداخل العناصر

plt.savefig("sales_chart.png", dpi=300)  # حفظ كصورة (بالإضافة إلى/بدلاً من show)
plt.show()
```

رسوم متعددة في شكل واحد: `subplots`

```python
fig, axes = plt.subplots(1, 2, figsize=(12, 5))  # صف واحد، عمودان

axes[0].plot(months, sales, marker="o")
axes[0].set_title("خطي")

axes[1].bar(months, sales, color="orange")
axes[1].set_title("شريطي")

plt.tight_layout()
plt.show()
```

الرسم مباشرة من pandas

ميزة مريحة جدًا: DataFrame وSeries لديهما دالة `.plot()` مدمجة تستخدم matplotlib خلف الكواليس:

```python
import pandas as pd

df = pd.DataFrame({
    "الشهر": ["يناير", "فبراير", "مارس"],
    "المبيعات": [1200, 1900, 1500],
})

df.plot(x="الشهر", y="المبيعات", kind="bar", title="المبيعات الشهرية")
plt.show()

# أو مباشرة من نتيجة groupby
df.groupby("الشهر")["المبيعات"].sum().plot(kind="pie", autopct="%1.1f%%")
plt.show()
```

نصيحة خبير: للتحليل الاستكشافي السريع (أنت فقط تستكشف البيانات)، استخدم `.plot()` من pandas — أسرع وأقل كتابة. للرسوم النهائية التي ستُعرض أو تُنشر في تقرير، استخدم matplotlib مباشرة للحصول على تحكم كامل في التفاصيل.

مقارنة: matplotlib مقابل مكتبات تصور أخرى

المكتبة	الاستخدام الأمثل
matplotlib	الأساس، تحكم كامل، لكن أكثر كتابة للتفاصيل الجميلة
seaborn	مبنية فوق matplotlib، رسوم إحصائية جاهزة وأجمل افتراضيًا
plotly	رسوم تفاعلية (تكبير، تمرير الفأرة) مناسبة لتطبيقات الويب

كمبتدئ، أتقن matplotlib أولاً — فهمها يجعل تعلم الباقي أسهل بكثير لأنها الأساس الذي تُبنى عليه.

20.6 مشروع تطبيقي متكامل: تحليل مبيعات متجر إلكتروني

سنبني الآن سكربت تحليل بيانات متكامل لمتجر إلكتروني وهمي، يغطي: تحميل البيانات، تنظيفها، تحليلها من زوايا متعددة، وإنتاج رسوم بيانية احترافية.

هيكلية المشروع

```
sales_analysis/
├── pyproject.toml
├── data/
│   └── sales_data.csv
├── output/              (سيُنشأ تلقائيًا للرسوم البيانية)
└── src/
    └── sales_analysis/
        ├── __init__.py
        ├── data_loader.py
        ├── analysis.py
        ├── visualization.py
        └── main.py
```

Terminal

```
uv init sales_analysis
cd sales_analysis
uv add pandas numpy matplotlib
mkdir -p src/sales_analysis data output
touch src/sales_analysis/__init__.py
```

الخطوة 1: توليد بيانات تجريبية واقعية

في مشروع حقيقي، ستقرأ ملف CSV جاهز لديك. هنا، سنولّد بيانات واقعية عشوائية لنتدرب عليها — ضعها في `data/generate_data.py` (سكربت مساعد منفصل، نُشغّله مرة واحدة فقط):

Python

```python
"""سكربت مساعد لتوليد بيانات مبيعات تجريبية واقعية."""

import numpy as np
import pandas as pd

np.random.seed(42)  # لضمان نتائج قابلة للإعادة

n_records = 1000

products = ["لابتوب", "ماوس", "لوحة مفاتيح", "شاشة", "سماعات", "طابعة"]
product_prices = {
    "لابتوب": 3500, "ماوس": 50, "لوحة مفاتيح": 120,
    "شاشة": 900, "سماعات": 150, "طابعة": 600,
}
cities = ["الرياض", "جدة", "الدمام", "مكة", "المدينة"]

data = {
    "order_id": range(1, n_records + 1),
    "التاريخ": pd.date_range("2024-01-01", "2024-12-31", periods=n_records),
    "المنتج": np.random.choice(products, n_records),
    "المدينة": np.random.choice(cities, n_records, p=[0.35, 0.25, 0.15, 0.15, 0.10]),
    "الكمية": np.random.randint(1, 6, n_records),
}

df = pd.DataFrame(data)
df["سعر_الوحدة"] = df["المنتج"].map(product_prices)
df["الإجمالي"] = df["سعر_الوحدة"] * df["الكمية"]

# إدخال بعض القيم المفقودة عمدًا (كما يحدث في البيانات الحقيقية)
missing_indices = np.random.choice(df.index, size=20, replace=False)
df.loc[missing_indices, "المدينة"] = None

df.to_csv("data/sales_data.csv", index=False, encoding="utf-8-sig")
print(f"تم توليد {n_records} سجل مبيعات في data/sales_data.csv")
```

Terminal

```
uv run python data/generate_data.py
```

الخطوة 2: `data_loader.py` — تحميل وتنظيف البيانات

Python

```python
"""وحدة تحميل وتنظيف بيانات المبيعات."""

from pathlib import Path

import pandas as pd

def load_sales_data(file_path: str | Path) -> pd.DataFrame:
    """تحميل بيانات المبيعات من ملف CSV.

    Args:
        file_path: مسار ملف CSV.

    Returns:
        DataFrame يحتوي بيانات المبيعات الخام.
    """
    df = pd.read_csv(file_path, encoding="utf-8-sig", parse_dates=["التاريخ"])
    return df

def clean_sales_data(df: pd.DataFrame) -> pd.DataFrame:
    """تنظيف بيانات المبيعات: معالجة القيم المفقودة والتكرارات.

    Args:
        df: DataFrame الخام.

    Returns:
        DataFrame نظيف جاهز للتحليل.
    """
    clean_df = df.copy()

    # تعويض المدن المفقودة بقيمة واضحة بدلاً من حذف الصفوف (نحتفظ بالبيانات)
    clean_df["المدينة"] = clean_df["المدينة"].fillna("غير محدد")

    # حذف أي صفوف مكررة بالكامل إن وُجدت
    clean_df = clean_df.drop_duplicates()

    # إضافة أعمدة مشتقة مفيدة للتحليل
    clean_df["الشهر"] = clean_df["التاريخ"].dt.month
    clean_df["اسم_الشهر"] = clean_df["التاريخ"].dt.month_name()
    clean_df["ربع_السنة"] = clean_df["التاريخ"].dt.quarter

    return clean_df
```

الخطوة 3: `analysis.py` — دوال التحليل

```python
```

```python
"""وحدة تحليل بيانات المبيعات."""

import pandas as pd

def total_revenue(df: pd.DataFrame) -> float:
    """حساب إجمالي الإيرادات."""
    return float(df["الإجمالي"].sum())

def revenue_by_product(df: pd.DataFrame) -> pd.Series:
    """إجمالي الإيرادات لكل منتج، مرتبة تنازليًا."""
    return df.groupby("المنتج")["الإجمالي"].sum().sort_values(ascending=False)

def revenue_by_city(df: pd.DataFrame) -> pd.Series:
    """إجمالي الإيرادات لكل مدينة، مرتبة تنازليًا."""
    return df.groupby("المدينة")["الإجمالي"].sum().sort_values(ascending=False)

def monthly_trend(df: pd.DataFrame) -> pd.Series:
    """اتجاه الإيرادات الشهري عبر العام."""
    return df.groupby("الشهر")["الإجمالي"].sum().sort_index()

def top_n_products_by_quantity(df: pd.DataFrame, n: int = 5) -> pd.Series:
    """أكثر n منتجات مبيعًا من حيث الكمية."""
    return df.groupby("المنتج")["الكمية"].sum().sort_values(ascending=False).head(n)

def average_order_value(df: pd.DataFrame) -> float:
    """متوسط قيمة الطلب الواحد."""
    return float(df["الإجمالي"].mean())

def city_product_matrix(df: pd.DataFrame) -> pd.DataFrame:
    """جدول محوري: الإيرادات لكل منتج في كل مدينة."""
    return df.pivot_table(
        values="الإجمالي",
        index="المدينة",
        columns="المنتج",
        aggfunc="sum",
        fill_value=0,
    )

def generate_summary_report(df: pd.DataFrame) -> dict[str, float | int]:
    """توليد ملخص إحصائي شامل للبيانات."""
    return {
        "إجمالي_الإيرادات": total_revenue(df),
        "عدد_الطلبات": len(df),
        "متوسط_قيمة_الطلب": average_order_value(df),
        "أعلى_منتج_مبيعًا": str(revenue_by_product(df).index[0]),
        "أعلى_مدينة_مبيعًا": str(revenue_by_city(df).index[0]),
    }
```

> **نصيحة خبير:** لاحظ أن كل دالة تحليل تأخذ `DataFrame` وتُرجع نتيجة محددة — هذا نمط تصميم ممتاز (دوال نقية Pure Functions كما تعلمنا في الفصل 10) يجعل الكود **قابلاً للاختبار** بسهولة (سنتعلم `pytest` في الفصل 25) وقابلاً لإعادة الاستخدام على أي بيانات مشابهة.

الخطوة 4: `visualization.py` — الرسوم البيانية

Python

```python
"""وحدة توليد الرسوم البيانية لتحليل المبيعات."""

from pathlib import Path

import matplotlib.pyplot as plt
import pandas as pd

OUTPUT_DIR = Path("output")

def _ensure_output_dir() -> None:
    """التأكد من وجود مجلد الإخراج."""
    OUTPUT_DIR.mkdir(exist_ok=True)

def plot_revenue_by_product(revenue: pd.Series) -> None:
    """رسم بياني شريطي لإيرادات المنتجات."""
    _ensure_output_dir()
    plt.figure(figsize=(10, 6))
    revenue.plot(kind="bar", color="steelblue", edgecolor="black")
    plt.title("Revenue by Product", fontsize=14, fontweight="bold")
    plt.xlabel("Product")
    plt.ylabel("Revenue (SAR)")
    plt.xticks(rotation=45)
    plt.tight_layout()
    plt.savefig(OUTPUT_DIR / "revenue_by_product.png", dpi=200)
    plt.close()

def plot_revenue_by_city_pie(revenue: pd.Series) -> None:
    """رسم دائري لتوزيع الإيرادات حسب المدينة."""
    _ensure_output_dir()
    plt.figure(figsize=(8, 8))
    plt.pie(revenue.values, labels=revenue.index, autopct="%1.1f%%", startangle=90)
    plt.title("Revenue Distribution by City", fontsize=14, fontweight="bold")
    plt.tight_layout()
    plt.savefig(OUTPUT_DIR / "revenue_by_city.png", dpi=200)
    plt.close()

def plot_monthly_trend(trend: pd.Series) -> None:
    """رسم خطي لاتجاه المبيعات الشهري."""
    _ensure_output_dir()
    plt.figure(figsize=(10, 6))
```

```
    plt.plot(trend.index, trend.values, marker="o", linewidth=2, color="darkgreen")
    plt.title("Monthly Revenue Trend", fontsize=14, fontweight="bold")
    plt.xlabel("Month")
    plt.ylabel("Revenue (SAR)")
    plt.grid(True, alpha=0.3)
    plt.xticks(range(1, 13))
    plt.tight_layout()
    plt.savefig(OUTPUT_DIR / "monthly_trend.png", dpi=200)
    plt.close()

def plot_city_product_heatmap(matrix: pd.DataFrame) -> None:
    """رسم Heatmap المدينة/المنتج كمصفوفة."""
    _ensure_output_dir()
    plt.figure(figsize=(10, 6))
    plt.imshow(matrix.values, cmap="YlOrRd", aspect="auto")
    plt.colorbar(label="Revenue (SAR)")
    plt.xticks(range(len(matrix.columns)), matrix.columns, rotation=45)
    plt.yticks(range(len(matrix.index)), matrix.index)
    plt.title("Revenue Heatmap: City vs Product", fontsize=14, fontweight="bold")
    plt.tight_layout()
    plt.savefig(OUTPUT_DIR / "city_product_heatmap.png", dpi=200)
    plt.close()
```

> **نصيحة خبير:** استخدمنا `plt.close()` بعد كل `savefig()` بدلاً من `plt.show()`. هذا ضروري في السكربتات التي تنتج عدة رسوم متتالية، لأنه يُحرر الذاكرة ويمنع تراكب الرسوم فوق بعضها بالخطأ.

الخطوة 5: `main.py` — تجميع كل شيء

Python

```
"""نقطة الدخول الرئيسية لتحليل بيانات المبيعات."""

from sales_analysis.analysis import (
    city_product_matrix,
    generate_summary_report,
    monthly_trend,
    revenue_by_city,
    revenue_by_product,
    top_n_products_by_quantity,
)
from sales_analysis.data_loader import clean_sales_data, load_sales_data
from sales_analysis.visualization import (
    plot_city_product_heatmap,
    plot_monthly_trend,
    plot_revenue_by_city_pie,
    plot_revenue_by_product,
)
```

```
def print_report(summary: dict) → None:
    """طباعة تقرير نصي منسّق في الطرفية."""
    print("=" * 50)
    print(" " * 15 + "تقرير تحليل المبيعات")
    print("=" * 50)
    for key, value in summary.items():
        label = key.replace("_", " ")
        if isinstance(value, float):
            print(f"{label}: {value:,.2f}")
        else:
            print(f"{label}: {value}")
    print("=" * 50)

def main() → None:
    """تشغيل خط التحليل الكامل."""
    print("📊 تحميل البيانات...")
    raw_df = load_sales_data("data/sales_data.csv")

    print("🧹 تنظيف البيانات...")
    df = clean_sales_data(raw_df)
    print(f"   تم تحميل {len(df)} سجل بنجاح.\n")

    print("📈 تحليل البيانات...")
    summary = generate_summary_report(df)
    print_report(summary)

    print("\n🏆 أفضل 5 منتجات من حيث الكمية المباعة:")
    print(top_n_products_by_quantity(df, 5))

    print("\n🎨 توليد الرسوم البيانية...")
    plot_revenue_by_product(revenue_by_product(df))
    plot_revenue_by_city_pie(revenue_by_city(df))
    plot_monthly_trend(monthly_trend(df))
    plot_city_product_heatmap(city_product_matrix(df))

    print("✅ تم حفظ كل الرسوم البيانية في مجلد output/")

if __name__ == "__main__":
    main()
```

تشغيل المشروع:

```
Terminal
uv run python -m sales_analysis.main
```

الناتج المتوقع في الطرفية:

```
📊 تحميل البيانات.
🧹 تنظيف البيانات.
   تم تحميل 1000 سجل بنجاح.

==================================================
               تقرير تحليل المبيعات
==================================================
إجمالي الإيرادات: 1,245,780.00
عدد الطلبات: 1000
متوسط قيمة الطلب: 1,245.78
أعلى منتج مبيعًا: لابتوب
أعلى مدينة مبيعًا: الرياض
==================================================

🏆 أفضل 5 منتجات من حيث الكمية المباعة:
المنتج
312          ماوس
298    لوحة مفاتيح
...

🎨 توليد الرسوم البيانية.
✅ تم حفظ كل الرسوم البيانية في مجلد output/
```

ستجد الآن في مجلد `output/` أربع صور PNG احترافية جاهزة لوضعها في أي تقرير أو عرض تقديمي!

✅ الحل الكامل للمشروع (جميع الملفات مجتمعة)

`pyproject.toml` (مُولّد تلقائيًا عبر `uv init` + `uv add`، لا حاجة لتعديله يدويًا).

`src/sales_analysis/data_loader.py`، `analysis.py`، `visualization.py`، `main.py`:
كما وردت بالكامل أعلاه دون أي تعديل — هذه هي الحل النموذجي الكامل.

للتحقق من عمل كل شيء معًا:

```
Terminal
uv run python data/generate_data.py   # توليد البيانات (مرة واحدة)
uv run python -m sales_analysis.main  # تشغيل التحليل الكامل
```

نقاط تصميم مهمة في هذا الحل:

1. فصل الاهتمامات (Separation of Concerns): تحميل البيانات - تحليلها - رسمها - عرضها — كل مسؤولية في ملف منفصل.
2. كل دالة تحليل هي دالة نقية (Pure Function): تأخذ DataFrame وتُرجع نتيجة، دون آثار جانبية.
3. استخدام `Path` بدلاً من نصوص خام للمسارات (كما تعلمنا في الفصل 16).
4. Type Hints في كل الدوال (كما تعلمنا في الفصل 15) — هذا يجعل الكود أوضح ويسمح لـ mypy باكتشاف الأخطاء.

20.7 أخطاء شائعة في تحليل البيانات

⚠️ **تحذير 1: `SettingWithCopyWarning`**

```python
subset = df[df['السعر'] > 100]
subset['جديد'] = تحذير! قد لا يعمل كما تتوقع ⚠️ # 1
```

السبب: `subset` قد يكون «عرضًا» (view) وليس نسخة مستقلة من `df`. الحل:

```python
subset = df[df['السعر'] > 100].copy()
subset['جديد'] = آمن الآن ✅ # 1
```

⚠️ **تحذير 2: الخلط بين `.mean()` على عمود يحتوي نصوصًا**

إذا حاولت حساب `.mean()` على عمود يحتوي قيمًا غير رقمية (نصوص مختلطة مع أرقام)، ستحصل على خطأ `TypeError`. تحقق دائمًا من `df.dtypes` قبل إجراء عمليات حسابية.

⚠️ **تحذير 3: مقارنة القيم المفقودة بـ `==`**

```python
df[df['العمود'] == None]  # ❌ لا يعمل أبدًا! NaN ≠ NaN رياضيًا
df[df['العمود'].isna()]   # ✅ الطريقة الصحيحة
```

💡 **نصيحة خبير شاملة:** اجعل قاعدة «استكشف قبل أن تحلل» عادة ثابتة: عند كل بيانات جديدة، ابدأ دائمًا بـ `df.head()`، `df.info()`، `df.describe()`، `df.isna().sum()` قبل أي تحليل آخر. هذه العادة البسيطة تمنع 80% من الأخطاء الشائعة في تحليل البيانات.

20.8 تمارين

🟢 مستوى مبتدئ

1. أنشئ `DataFrame` يحتوي بيانات 5 طلاب (اسم، درجة في مادتين). احسب متوسط كل مادة باستخدام `.mean()`.
2. استخدم NumPy لإنشاء مصفوفة من 10 أرقام عشوائية بين 1 و100، ثم اطبع الأكبر والأصغر والمتوسط.
3. ارسم رسمًا بيانيًا شريطيًا بسيطًا يوضح عدد ساعات الدراسة لـ 5 أيام من الأسبوع.

✅ الحل المقترح

Python

```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# 1
students = pd.DataFrame({
    "الاسم": ["أحمد", "سارة", "خالد", "منى", "يوسف"],
    "الرياضيات": [88, 95, 78, 92, 85],
    "العلوم": [79, 91, 82, 85, 90],
})
print(students["الرياضيات"].mean())
print(students["العلوم"].mean())

# 2
numbers = np.random.randint(1, 101, 10)
print(numbers)
print("الأكبر:", numbers.max())
print("الأصغر:", numbers.min())
print("المتوسط:", numbers.mean())

# 3
days = ["السبت", "الأحد", "الاثنين", "الثلاثاء", "الأربعاء"]
hours = [2, 3, 1.5, 4, 2.5]
plt.bar(days, hours, color="purple")
plt.title("ساعات الدراسة")
plt.show()
```

🟡 مستوى متوسط

4. لديك DataFrame يحتوي بيانات موظفين (اسم، قسم، راتب). استخدم `groupby` لحساب متوسط الراتب في كل قسم، ثم ارسم النتيجة كرسم شريطي.
5. أنشئ DataFrame يحتوي قيمًا مفقودة عمدًا في عمود «العمر»، ثم عوّضها بالوسيط (median) بدلاً من المتوسط، واشرح سبب اختيارك.

✅ الحل المقترح

Python

```python
import pandas as pd
import matplotlib.pyplot as plt

# 4
employees = pd.DataFrame({
    "["عمر" ,"ليلى" ,"سعد" ,"فاطمة" ,"علي"] :"اسم",
    "["مالية" ,"تقنية" ,"مبيعات" ,"تقنية" ,"مبيعات"] :"القسم",
    "[9500 ,13000 ,8500 ,12000 ,8000] :"الراتب",
})
avg_salary = employees.groupby("القسم")["الراتب"].mean()
print(avg_salary)
avg_salary.plot(kind="bar", color="teal")
plt.title("متوسط الراتب حسب القسم")
plt.ylabel("الراتب")
plt.tight_layout()
plt.show()

# 5
data = pd.DataFrame({"العمر": [30 ,25, None, 45, 28, None, 90]})
# 90 الوسيط أفضل من المتوسط هنا لأن القيمة (قيمة متطرفة/Outlier)
# تؤثر بشدة على المتوسط لكن تأثيرها على الوسيط محدود جدًا
data["العمر"] = data["العمر"].fillna(data["العمر"].median())
print(data)
```

🔴 مستوى تحدي

6. وسّع مشروع تحليل المبيعات: أضف دالة جديدة `best_selling_month_per_product()` تحدد الشهر الأفضل مبيعًا لكل منتج على حدة (استخدم `groupby` بعمودين و `idxmax`). أضف أيضًا رسمًا بيانيًا جديدًا `plot_quarterly_comparison()` يقارن مبيعات الأرباع الأربعة للسنة بأعمدة متجاورة لكل مدينة (استخدم `pivot_table` ثم `.plot(kind="bar")`).

✅ الحل المقترح

Python

```python
# إضافة إلى analysis.py
def best_selling_month_per_product(df: pd.DataFrame) -> pd.Series:
    """الشهر الأفضل مبيعًا لكل منتج."""
    grouped = df.groupby(["الشهر", "المنتج"])["الإجمالي"].sum()
    return grouped.groupby(level="المنتج").idxmax()

# إضافة إلى visualization.py
def plot_quarterly_comparison(df: pd.DataFrame) -> None:
    """مقارنة مبيعات الأرباع الأربعة لكل مدينة."""
    _ensure_output_dir()
    pivot = df.pivot_table(
        values="الإجمالي", index="ربع_السنة", columns="المدينة",
        aggfunc="sum", fill_value=0,
    )
    pivot.plot(kind="bar", figsize=(10, 6))
    plt.title("Quarterly Revenue Comparison by City")
    plt.xlabel("Quarter")
    plt.ylabel("Revenue (SAR)")
    plt.legend(title="City")
    plt.tight_layout()
    plt.savefig(OUTPUT_DIR / "quarterly_comparison.png", dpi=200)
    plt.close()
```

20.9 اختبر فهمك (Quiz)

1. ما الفرق الجوهري بين `list` العادية و `numpy.ndarray`؟

`ndarray` تتطلب أن تكون كل عناصرها من نفس النوع (موحدة)، وتدعم العمليات الحسابية مباشرة على كل العناصر دفعة واحدة (Vectorization) دون حلقات، وهي أسرع بكثير وأقل استهلاكًا للذاكرة لأنها منفذة بلغة C داخليًا. القوائم العادية مرنة (تقبل أنواعًا مختلطة) لكنها أبطأ في الحسابات الرقمية.

2. لماذا تفشل `df[(df["a"] > 5) and (df["b"] < 10)]` وما البديل الصحيح؟

لأن `and` / `or` في Python تتوقع قيمة Boolean مفردة، بينما الشرط `(df["a"] > 5)` يُرجع `Series` كاملة من قيم True/False وليس قيمة واحدة. البديل الصحيح هو استخدام `&` و`|` مع وضع أقواس حول كل شرط فرعي: `df[(df["a"] > 5) & (df["b"] < 10)]`.

3. متى تستخدم `loc` ومتى تستخدم `iloc`؟

`loc` يُستخدم للاختيار بناءً على **التسمية/الاسم** (label-based)، مثل اسم العمود أو قيمة الفهرس الفعلية. `iloc` يُستخدم للاختيار بناءً على **الموضع الرقمي** (position-based) تمامًا كالقوائم العادية (0، 1، 2...)، بغض النظر عن تسميات الفهرس.

4. ما وظيفة `groupby` ومتى تستخدمها؟

`groupby` يُستخدم لتقسيم البيانات إلى مجموعات بناءً على قيمة عمود (أو أكثر)، ثم تطبيق دالة تجميعية (sum, mean, count...) على كل مجموعة على حدة. يُستخدم للإجابة على أسئلة مثل «ما إجمالي المبيعات لكل مدينة؟» وهو مكافئ لـ `GROUP BY` في SQL.

5. لماذا نستخدم `.copy()` أحيانًا بعد تصفية DataFrame؟

لتجنب تحذير `SettingWithCopyWarning` والسلوك غير المتوقع الناتج عن أن الجزء المُصفّى قد يكون «عرضًا» (view) مرتبطًا بالبيانات الأصلية بدلاً من نسخة مستقلة. استخدام `.copy()` يضمن أننا نعمل على نسخة منفصلة تمامًا وآمنة للتعديل.

6. ما الفرق بين `plt.show()` و `plt.savefig()`؟

`plt.show()` يفتح نافذة تفاعلية لعرض الرسم على الشاشة مباشرة، بينما `plt.savefig()` يحفظ الرسم كملف صورة (PNG, JPG, PDF...) على القرص دون بالضرورة عرضه، وهذا مفيد جدًا في السكربتات الآلية التي تولّد عدة رسوم دون تدخل بشري.

20.10 ملخص الفصل (Cheat Sheet)

Python

```python
# ===== NumPy =====
import numpy as np
arr = np.array([1, 2, 3])
arr.sum(); arr.mean(); arr.max(); arr.min(); arr.std()
filtered = arr[arr > 1]                        # Boolean Indexing

# ===== pandas: القراءة والاستكشاف =====
import pandas as pd
df = pd.read_csv("file.csv", encoding="utf-8")
df.head(); df.tail(); df.info(); df.describe()
df.shape; df.columns; df.dtypes
df.isna().sum()                                # القيم المفقودة

# ===== pandas: الاختيار والتصفية =====
df["col"]                                      # عمود ← Series
df[["col1", "col2"]]                           # أعمدة ← DataFrame
df.loc[0, "col"]                               # بالتسمية
df.iloc[0, 1]                                  # بالموضع
df[(df["a"] > 5) & (df["b"] < 10)]             # تصفية بشروط (& | وليس and or)

# ===== pandas: التعديل =====
df["new_col"] = df["a"] * df["b"]
df = df.drop(columns=["col"])
df = df.rename(columns={"old": "new"})
df["col"] = df["col"].fillna(df["col"].mean())
df = df.dropna(subset=["col"])
df = df.sort_values("col", ascending=False)

# ===== pandas: التجميع والدمج =====
df.groupby("col")["val"].sum()
df.groupby("col")["val"].agg(["sum", "mean", "count"])
df.pivot_table(values="val", index="r", columns="c", aggfunc="sum")
pd.merge(df1, df2, on="key", how="left")
pd.concat([df1, df2], ignore_index=True)

# ===== pandas: التصدير =====
df.to_csv("out.csv", index=False, encoding="utf-8-sig")

# ===== Matplotlib =====
import matplotlib.pyplot as plt
plt.plot(x, y); plt.bar(x, y); plt.pie(values, labels=labels)
plt.scatter(x, y); plt.hist(data, bins=20)
plt.title(""); plt.xlabel(""); plt.ylabel(""); plt.legend()
plt.savefig("chart.png", dpi=200); plt.show(); plt.close()
```

المهمة	الكود
تحميل CSV	`pd.read_csv(path, encoding="utf-8")`
فحص سريع	`df.head()` ، `df.info()` ، `df.describe()`
تصفية	`df[df["col"] > x]`
تجميع	`df.groupby("col")["val"].sum()`
جدول محوري	`df.pivot_table(...)`
رسم سريع	`df.plot(kind="bar")`

20.11 ماذا بعد؟

لقد اكتسبت للتو قدرة قوية جدًا: تحويل بيانات خام فوضوية إلى رؤى وقرارات واضحة، مدعومة برسوم بيانية احترافية. هذه المهارة بالذات هي من أكثر المهارات طلبًا في سوق العمل التقني اليوم.

في الفصل القادم، سننتقل إلى مسار مختلف تمامًا لكنه بنفس الإثارة: **الأتمتة (Automation)**. سنتعلم كيف نجعل Python «يتصفح» الإنترنت نيابة عنا باستخدام مكتبتي `requests` و `BeautifulSoup`، لاستخراج بيانات من صفحات الويب تلقائيًا — مهارة تُعرف بـ **Web Scraping**، وتُستخدم في كل شيء من مراقبة الأسعار إلى جمع بيانات البحث.

فصل 21 / 26 · الجزء الخامس

21

مسار الأتمتة — استخراج البيانات من الويب (Web Scraping)

11 دقيقة قراءة | 15 قسمًا | 25 مثالًا | 2,265 كلمة

في هذا الفصل

21.1 لماذا الأتمتة؟ ولماذا Web Scraping تحديدًا؟
21.2 قبل أن نبدأ: الأخلاقيات والقانونية في Web Scraping
21.3 أساسيات HTML التي تحتاجها (مراجعة سريعة)
21.4 مكتبة `requests`: طلب الصفحات
21.5 مكتبة BeautifulSoup: تحليل واستخراج البيانات
21.6 نمط العمل الكامل: من الطلب إلى البيانات المنظّمة
21.7 التعامل مع التصفّح متعدد الصفحات (Pagination)
21.8 حفظ البيانات المُستخرجة
21.9 معالجة الأخطاء بصلابة (Robust Scraping)
21.10 ماذا عن المواقع التي تعتمد على JavaScript؟
21.11 مشروع تطبيقي متكامل: مستخرج بيانات كتب شامل
21.12 تمارين
21.13 اختبر فهمك (Quiz)
21.14 ملخص الفصل (Cheat Sheet)
21.15 ماذا بعد؟

21.1 لماذا الأتمتة؟ ولماذا Web Scraping تحديدًا؟

تخيّل أنك تريد مراقبة سعر منتج معيّن في متجر إلكتروني كل يوم، أو جمع كل عناوين الأخبار من موقع معيّن، أو تتبّع تقييمات منتج على مئات الصفحات. فتح المتصفح يدويًا ونسخ البيانات واحدة تلو الأخرى قد يستغرق ساعات — وهو عمل مملّ ومتكرر بالضبط من النوع الذي صُمم الحاسوب لتوفيره عنا.

الأتمتة (Automation) هي استخدام البرمجة لتنفيذ مهام متكررة تلقائيًا دون تدخل بشري. و**استخراج بيانات الويب (Web Scraping)** هو أحد أكثر تطبيقات الأتمتة شيوعًا وفائدة: كتابة برنامج «يزور» صفحات الويب نيابة عنك، «يقرأ» محتواها، ويستخرج منها البيانات التي تهمك بشكل منظم.

مقارنة: كيف يحصل الإنسان على البيانات مقابل كيف يحصل عليها السكربت

الإنسان	سكربت Web Scraping
يفتح المتصفح، يقرأ الصفحة بعينه	يرسل طلب HTTP، يستقبل HTML خام
ينسخ البيانات يدويًا في Excel	يحلل HTML ويستخرج البيانات تلقائيًا
دقائق لكل صفحة	أجزاء من الثانية لكل صفحة
يتعب بعد 50 صفحة	يمكنه معالجة 50,000 صفحة بنفس الجهد
عرضة للخطأ البشري	دقيق طالما الكود صحيح

استخدامات Web Scraping الحقيقية تشمل: مراقبة أسعار المنافسين، جمع بيانات للتدريب على نماذج الذكاء الاصطناعي، تتبّع فرص العمل الجديدة، أرشفة الأخبار، البحث الأكاديمي، ومراقبة توفر المنتجات.

في هذا الفصل، سنتعلم مكتبتين أساسيتين:

1. `requests` : لإرسال طلبات HTTP والحصول على محتوى الصفحات (تمامًا كما يفعل المتصفح، لكن بدون واجهة رسومية).
2. `BeautifulSoup` : لتحليل محتوى HTML المُستلم واستخراج البيانات المحددة منه بسهولة.

ثم سنطبّق كل هذا في **مشروع متكامل لاستخراج بيانات كتب من موقع تدريبي**.

21.2 قبل أن نبدأ: الأخلاقيات والقانونية في Web Scraping

هذا قسم **إلزامي القراءة** قبل أي سطر كود.

Web Scraping قوة كبيرة، ومعها مسؤولية كبيرة. إليك القواعد الذهبية:

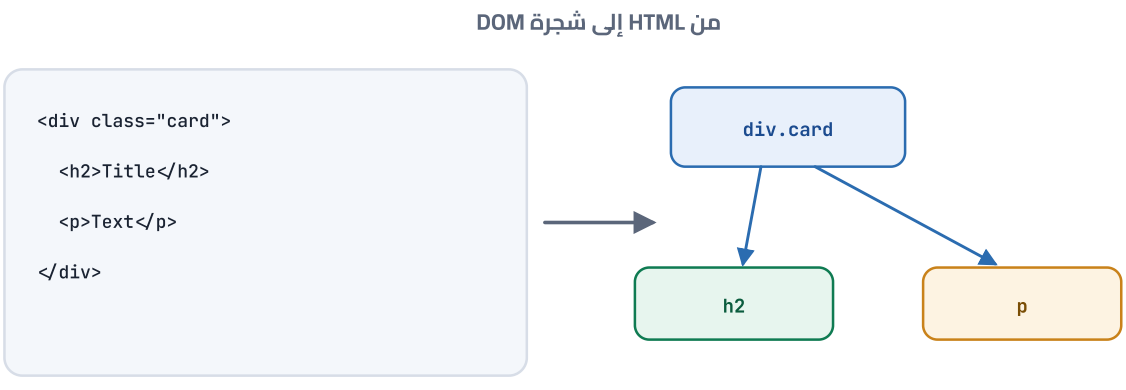
⚠ **تحذير مهم جدًا: قواعد السلوك الأخلاقي في Web Scraping**

1. **افحص ملف** `robots.txt` دائمًا قبل استخراج بيانات من أي موقع. يوجد عادة على `example.com/robots.txt` ويوضّح ما يُسمح وما لا يُسمح باستخراجه آليًا.
2. **اقرأ شروط الاستخدام (Terms of Service)** للموقع — بعض المواقع تمنع صراحة الاستخراج الآلي.
3. **لا تُثقل الخادم بالطلبات**: أضف تأخيرًا (`time.sleep()`) بين الطلبات المتتالية. إرسال آلاف الطلبات في ثوانٍ قد يُعتبر هجوم حرمان من الخدمة (DoS) حتى لو لم تقصد ذلك!
4. **لا تستخرج بيانات شخصية** أو محمية بحقوق نشر دون إذن.
5. **فضّل استخدام API رسمي** إن وُجد — معظم المواقع الكبرى (Twitter/X، Reddit، GitHub...) توفّر APIs رسمية أنظف وأكثر استقرارًا من Web Scraping.
6. **عرّف نفسك**: أرسل `User-Agent` واضحًا يوضّح من أنت، بدلاً من انتحال متصفح حقيقي خداعًا.

في كل أمثلة هذا الفصل سنستخدم موقع **books.toscrape.com**، وهو موقع **تدريبي مُصمَّم خصيصًا وبشكل قانوني وآمن** لممارسة Web Scraping دون أي قلق أخلاقي أو قانوني.

💡 **نصيحة خبير:** تحقق دائمًا من `robots.txt` برمجيًا إن أردت الأتمتة الكاملة، عبر مكتبة Python المدمجة `urllib.robotparser`. لكن في مشاريعك الحقيقية، القراءة اليدوية لشروط الموقع كافية ومهمة.

21.3 أساسيات HTML التي تحتاجها (مراجعة سريعة)



شكل 21.1 — من HTML إلى شجرة DOM

لكي نستخرج بيانات من صفحة ويب، يجب أن نفهم هيكلها الأساسي. **HTML** (HyperText Markup Language) هي اللغة التي تُبنى بها صفحات الويب، وتتكوّن من **عناصر (Elements)** محاطة بـ **وسوم (Tags)**:

HTML

```
<html>
  <body>
    <div class="product">
      <h2 class="title">لابتوب Dell</h2>
      <p class="price">2500 ﷼</p>
      <a href="/product/1">عرض التفاصيل</a>
    </div>
    <div class="product">
      <h2 class="title">ماوس لاسلكي</h2>
      <p class="price">80 ﷼</p>
      <a href="/product/2">عرض التفاصيل</a>
    </div>
  </body>
</html>
```

مفاهيم أساسية يجب معرفتها:

المصطلح	الوصف	مثال
Tag (وسم)	يحدد نوع العنصر	`<div>` , `<p>` , `<h2>` , `<a>`
Attribute (خاصية)	معلومات إضافية عن العنصر	`class="product"` , `href="/product/1"`
class	يُستخدم غالبًا للتنسيق، ومفيد جدًا للاستهداف في الاستخراج	`class="price"`
id	معرّف فريد لعنصر واحد في الصفحة	`id="main-header"`
Nesting (التداخل)	العناصر تحتوي عناصر أخرى بداخلها (هيكل شجري)	`<div>` يحتوي `<h2>` و `<p>`

نصيحة خبير: أفضل صديق لك في Web Scraping هو **أدوات المطوّر في المتصفح** (اضغط `F12` أو كليك يمين → «فحص العنصر» / Inspect). استخدمها دائمًا لتحديد الوسم والـ class الدقيقين للبيانات التي تريد استخراجها **قبل** كتابة أي كود.

21.4 مكتبة `requests`: طلب الصفحات

التثبيت

```
Terminal
uv add requests
uv add --dev beautifulsoup4 lxml
```

(`lxml` هو محلّل HTML سريع نستخدمه مع BeautifulSoup).

أول طلب GET

```python
import requests

response = requests.get("https://books.toscrape.com/")

print(response.status_code)    # 200 (كما تعلمنا في الفصل 19 يعني نجاح
print(response.text[:500])     # أول 500 حرف من محتوى HTML الخام
```

فحص حالة الاستجابة

```python
import requests

response = requests.get("https://books.toscrape.com/")

if response.status_code == 200:
    print("✅ نجح الطلب!")
elif response.status_code == 404:
    print("❌ الصفحة غير موجودة")
else:
    print(f"⚠️ حدث خطأ: {response.status_code}")

# طريقة أنظف وأكثر احترافية
try:
    response.raise_for_status()  # يرفع استثناء تلقائيًا إذا كان status_code خطأ
    print("✅ نجح الطلب!")
except requests.HTTPError as e:
    print(f"❌ خطأ HTTP: {e}")
```

مقارنة: `status_code` اليدوي مقابل `raise_for_status()`

الطريقة	السلوك
`if response.status_code == 200`	يتطلب تحققًا يدويًا من كل كود محتمل
`response.raise_for_status()`	يرفع `requests.HTTPError` تلقائيًا لأي كود 4xx أو 5xx، فتتعامل معه بـ `try/except` كما تعلمنا في الفصل 11

إضافة Headers (مهم جدًا)

بعض المواقع ترفض الطلبات التي لا تحمل `User-Agent` يشبه متصفحًا حقيقيًا:

```python
import requests

headers = {
    "User-Agent": "Mozilla/5.0 (Educational Scraper Bot - contact@example.com)"
}

response = requests.get("https://books.toscrape.com/", headers=headers)
print(response.status_code)
```

تحديد مهلة زمنية (Timeout) — إلزامي دائمًا

```python
import requests

try:
    response = requests.get(
        "https://books.toscrape.com/",
        timeout=10,  # إذا لم يرد الخادم خلالها، يُرفع استثناء -- ثوانٍ
    )
except requests.Timeout:
    print("⏱️ انتهت المهلة الزمنية -- الخادم بطيء جدًا أو لا يستجيب")
except requests.ConnectionError:
    print("🔌 تعذّر الاتصال -- تحقق من الإنترنت أو الرابط")
```

تحذير شائع جدًا: نسيان `timeout`

بدون `timeout`، إذا توقف الخادم عن الاستجابة، سيبقى برنامجك **معلّقًا إلى الأبد** (hang) دون أي رسالة خطأ. ضع `timeout` في **كل** طلب `requests` تكتبه، بلا استثناء — هذه قاعدة احترافية لا نقاش فيها.

معاملات الاستعلام (Query Parameters) و POST

```python
import requests

# GET مع query parameters (مثل الفصل 19)
response = requests.get(
    "https://api.example.com/search",
    params={"q": "python", "page": 1},
    timeout=10,
)
# الرابط الفعلي سيكون: https://api.example.com/search?q=python&page=1

# POST لإرسال بيانات (مثلاً تسجيل الدخول أو إرسال نموذج)
response = requests.post(
    "https://api.example.com/login",
    json={"username": "ahmad", "password": "secret"},
    timeout=10,
)
```

التعامل مع استجابات JSON

كثير من المواقع الحديثة تستخدم APIs تُرجع JSON مباشرة (بدون HTML)، وهذا أسهل بكثير من استخراج HTML:

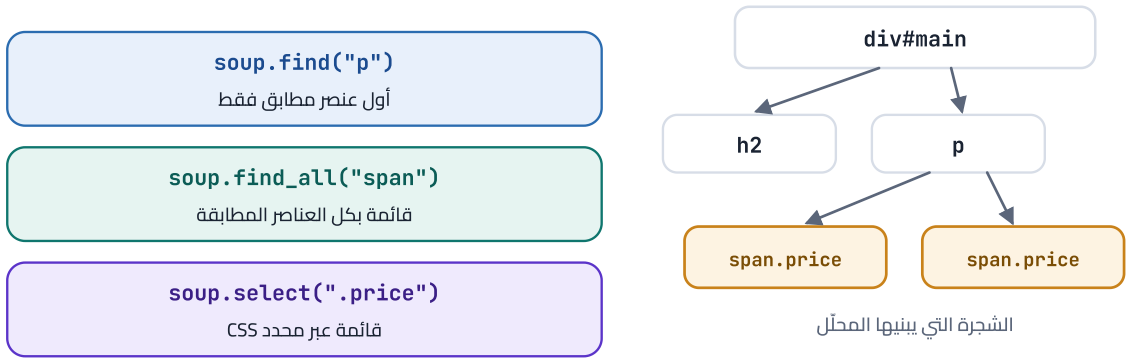
```python
import requests

response = requests.get("https://api.github.com/users/python", timeout=10)
data = response.json()  # تحويل تلقائي من JSON إلى dict/list بايثون

print(data["name"])
print(data["public_repos"])
```

نصيحة خبير: قبل أن تلجأ لتحليل HTML المعقّد، تحقّق دائمًا إن كان الموقع يوفّر **API رسمي** أو حتى API داخلي غير موثّق تستخدمه صفحة الويب نفسها (يمكنك اكتشافه عبر تبويب "Network" في أدوات المطوّر، والبحث عن طلبات تُرجع JSON). التعامل مع JSON مباشرة أسهل وأكثر استقرارًا بكثير من استخراج HTML.

استخدام `Session` لعدة طلبات متتالية

```python
import requests

# Session يحافظ تلقائيًا على الكوكيز بين الطلبات (مفيد بعد تسجيل الدخول مثلاً)
with requests.Session() as session:
    session.headers.update({"User-Agent": "Educational Bot"})

    login_response = session.post(
        "https://example.com/login",
        data={"user": "ahmad", "pass": "1234"},
        timeout=10,
    )

    # هذا الطلب سيستخدم نفس الكوكيز تلقائيًا
    profile_response = session.get("https://example.com/profile", timeout=10)
```

21.5 مكتبة BeautifulSoup: تحليل واستخراج البيانات



find يعيد عنصرًا أو None، أما find_all فيعيد قائمة قد تكون فارغة.

شكل 21.2 — find وfind_all وselect على شجرة العناصر

بعد الحصول على محتوى HTML الخام عبر `requests`، نحتاج طريقة ذكية لاستخراج البيانات منه. هنا يأتي دور **BeautifulSoup**.

الإعداد الأساسي

```python
import requests
from bs4 import BeautifulSoup

response = requests.get("https://books.toscrape.com/", timeout=10)
soup = BeautifulSoup(response.text, "lxml")  # "lxml" هو المحلل (parser) المستخدم

print(soup.title)        # <title>All products | Books to Scrape...</title>
print(soup.title.text)   # All products | Books to Scrape - Sandbox
```

مقارنة: المحلّلات (Parsers) المتاحة في BeautifulSoup

Parser	السرعة	يحتاج تثبيت خارجي	ملاحظات
`"html.parser"`	متوسطة	لا (مدمج في Python)	خيار جيد افتراضي
`"lxml"`	سريع جدًا	نعم (`uv add lxml`)	الأفضل للاستخدام الاحترافي
`"html5lib"`	بطيء	نعم	الأكثر تسامحًا مع HTML غير صحيح

البحث عن العناصر: `find` و `find_all`

```python
# العثور على أول عنصر مطابق
first_product = soup.find("article", class_="product_pod")

# العثور على كل العناصر المطابقة  ←تُرجع قائمة
all_products = soup.find_all("article", class_="product_pod")
print(f"عدد المنتجات: {len(all_products)}")

# البحث بالوسم فقط
all_headings = soup.find_all("h3")

# البحث بخاصية id
header = soup.find(id="messages")

# البحث بعدة شروط
links = soup.find_all("a", href=True)  # كل روابط <a> التي تملك خاصية href
```

⚠️ **تحذير: `class` كلمة محجوزة في Python!**
لا يمكنك كتابة `soup.find("div", class="price")` لأن `class` كلمة مفتاحية في بايثون (تُستخدم لتعريف الأصناف كما في الفصل 13). الحل: استخدم `class_` بشرطة سفلية:

```python
soup.find("div", class_="price")   # ✅ صحيح
```

استخراج النص والخصائص

```python
product = soup.find("article", class_="product_pod")

title_tag = product.find("h3").find("a")
title = title_tag["title"]           # الوصول لخاصية HTML (title attribute)
# أو
title = title_tag.get("title")       # يُرجع None أكثر أمانًا -- إن لم توجد بدلاً من خطأ

price_tag = product.find("p", class_="price_color")
price_text = price_tag.text          # 51.77£" :النص الظاهر للمستخدم"
price_text_stripped = price_tag.get_text(strip=True)  # يزيل المسافات الزائدة تلقائيًا
```

💡 **نصيحة خبير:** استخدم دائمًا `.get("attribute")` بدلاً من `["attribute"]` عند استخراج خصائص HTML، لأن `.get()` تُرجع `None` بأمان إذا لم تكن الخاصية موجودة، بينما `[]` يرفع `KeyError` ويُوقف برنامجك. هذا تطبيق مباشر لمبدأ «التعامل الدفاعي مع البيانات» الذي تعلمناه في الفصل 11.

محددات CSS: `select` و `select_one` (الطريقة الأكثر قوة ومرونة)

إذا كنت تعرف CSS (المستخدمة لتنسيق صفحات الويب)، يمكنك استخدام نفس الصياغة للاستخراج:

```python
# محدد CSS: وسم.class
titles = soup.select("h3 a")              # كل وسم <a> داخل <h3>
prices = soup.select("p.price_color")      # كل <p> بخاصية class="price_color"
first_price = soup.select_one("p.price_color")  # أول عنصر مطابق فقط

# محددات أكثر تعقيدًا
all_product_links = soup.select("article.product_pod h3 a")

for link in all_product_links:
    print(link.get("title"), "→", link.get("href"))
```

مقارنة: `find` / `find_all` مقابل `select` / `select_one`

الطريقة	الصياغة	متى تستخدمها
`find("div", class_="x")`	صياغة BeautifulSoup الخاصة	عند الحاجة لمرونة برمجية (شروط Python)
`select("div.x")`	صياغة CSS القياسية	أسرع كتابة إذا كنت تعرف CSS، ويدعم محددات معقدة مثل `div > p:first-child`

توصية: تعلّم كلتا الطريقتين، لكن `select` غالبًا أسرع وأوضح للمحددات المعقدة.

التنقل في الشجرة (Tree Navigation)

```python
product = soup.find("article", class_="product_pod")

print(product.parent)          # العنصر الأب
print(product.find_next_sibling())  # العنصر الشقيق التالي
print(product.contents)        # كل العناصر الفرعية المباشرة كقائمة
```

21.6 نمط العمل الكامل: من الطلب إلى البيانات المنظّمة

شكل 21.3 — من الطلب إلى البيانات المنظّمة

لنجمع كل شيء في مثال متكامل يستخرج بيانات كتاب واحد من صفحة تجريبية:

Python

```python
import requests
from bs4 import BeautifulSoup

url = "https://books.toscrape.com/"
headers = {"User-Agent": "Educational Scraper (learning purposes)"}

response = requests.get(url, headers=headers, timeout=10)
response.raise_for_status()

soup = BeautifulSoup(response.text, "lxml")

books = soup.select("article.product_pod")
print(f"تم العثور على {len(books)} كتاب في الصفحة")

for book in books[:3]:  # أول 3 كتب فقط للعرض
    title = book.select_one("h3 a").get("title")
    price = book.select_one("p.price_color").get_text(strip=True)
    availability = book.select_one("p.instock.availability").get_text(strip=True)
    rating_classes = book.select_one("p.star-rating")["class"]
    rating = [c for c in rating_classes if c != "star-rating"][0]  # مثل "Three"

    print(f"📖 {title}")
    print(f"   السعر: {price}")
    print(f"   التوفر: {availability}")
    print(f"   التقييم: {rating}")
    print("-" * 40)
```

```
تم العثور على 20 كتاب في الصفحة
📖 A Light in the Attic
   السعر: £51.77
   التوفر: In stock
   التقييم: Three
----------------------------------------
...
```

21.7 التعامل مع التصفّح متعدد الصفحات (Pagination)

معظم المواقع الحقيقية توزّع البيانات على عدة صفحات. يجب أتمتة الانتقال بينها:

Python

```python
import time

import requests
from bs4 import BeautifulSoup

BASE_URL = "https://books.toscrape.com/catalogue/page-{}.html"
headers = {"User-Agent": "Educational Scraper"}

all_titles = []
page = 1

while True:
    url = BASE_URL.format(page)
    response = requests.get(url, headers=headers, timeout=10)

    if response.status_code == 404:
        print(f"انتهت الصفحات عند الصفحة {page - 1}")
        break

    soup = BeautifulSoup(response.text, "lxml")
    books = soup.select("article.product_pod")

    for book in books:
        title = book.select_one("h3 a").get("title")
        all_titles.append(title)

    print(f"✅ تمت معالجة الصفحة {page} ({len(books)} كتاب)")
    page += 1
    time.sleep(1)  # ⚠️ تأخير بين الطلبات إلزامي -- احترام الخادم

print(f"\nإجمالي الكتب المُستخرجة: {len(all_titles)}")
```

⚠️ **تحذير: لماذا `time.sleep(1)` ضرورية؟**

بدون تأخير، سكربتك قد يُرسل عشرات أو مئات الطلبات في الثانية الواحدة — هذا:

1. قد يُحمّل خادم الموقع بشكل غير عادل (غير أخلاقي).
2. قد يؤدي لحظر عنوان IP الخاص بك تلقائيًا (Rate Limiting من جانب الموقع).
3. في الحالات القصوى، قد يُعتبر هجومًا إلكترونيًا من الناحية القانونية.

قاعدة عامة: ثانية واحدة على الأقل بين الطلبات لموقع صغير، وأكثر من ذلك للمواقع الكبيرة حسب سياستها المعلنة في `robots.txt`.

21.8 حفظ البيانات المُستخرجة

البيانات المُستخرجة بلا فائدة إن بقيت في الذاكرة فقط. نحفظها كما تعلمنا في الفصل 16:

```python
import csv

data = [
    {"title": "A Light in the Attic", "price": "£51.77", "rating": "Three"},
    {"title": "Tipping the Velvet", "price": "£53.74", "rating": "One"},
]

with open("books.csv", "w", newline="", encoding="utf-8") as f:
    writer = csv.DictWriter(f, fieldnames=["title", "price", "rating"])
    writer.writeheader()
    writer.writerows(data)
```

أو مباشرة عبر pandas (كما تعلمنا في الفصل 20) لتحليل فوري:

```python
import pandas as pd

df = pd.DataFrame(data)
df.to_csv("books.csv", index=False, encoding="utf-8-sig")
print(df.describe())
```

> **نصيحة خبير:** هذا هو **الجسر الحقيقي بين المسارات**: استخرج البيانات بـ `requests` / `BeautifulSoup`، نظّمها وحلّلها بـ `pandas`، وتصوّرها بـ `matplotlib`. هذه هي خطوط الأنابيب (Pipelines) الحقيقية المستخدمة في الصناعة.

21.9 معالجة الأخطاء بصلابة (Robust Scraping)

البيانات الحقيقية على الويب فوضوية: صفحات مفقودة، عناصر غير موجودة أحيانًا، اتصال بطيء. سكربت احترافي يجب أن يتحمّل كل هذا دون أن ينهار:

```python
```

```python
import time

import requests
from bs4 import BeautifulSoup

def fetch_page(url: str, retries: int = 3) -> requests.Response | None:
    """جلب صفحة مع إعادة محاولة عند الفشل.

    Args:
        url: الرابط المطلوب جلبه.
        retries: عدد محاولات إعادة الطلب عند الفشل.

    Returns:
        كائن Response عند النجاح، أو None عند فشل كل المحاولات.
    """
    headers = {"User-Agent": "Educational Scraper"}

    for attempt in range(1, retries + 1):
        try:
            response = requests.get(url, headers=headers, timeout=10)
            response.raise_for_status()
            return response
        except requests.Timeout:
            print(f"⏱ محاولة {attempt}: انتهت المهلة الزمنية لـ {url}")
        except requests.HTTPError as e:
            print(f"❌ محاولة {attempt}: خطأ HTTP {e}")
        except requests.ConnectionError:
            print(f"🔌 محاولة {attempt}: تعذّر الاتصال بـ {url}")

        if attempt < retries:
            time.sleep(2 * attempt)  # تأخير متزايد (Exponential backoff مبسّط)

    print(f"🚫 فشلت كل المحاولات لـ {url}")
    return None

def extract_book_data(book_element) -> dict[str, str] | None:
    """استخراج بيانات كتاب واحد بأمان، مع التعامل مع عناصر مفقودة.

    Args:
        book_element: عنصر BeautifulSoup يمثّل بطاقة كتاب واحدة.

    Returns:
        قاموس ببيانات الكتاب، أو None إذا كانت البيانات الأساسية مفقودة.
    """
    title_tag = book_element.select_one("h3 a")
    price_tag = book_element.select_one("p.price_color")

    if title_tag is None or price_tag is None:
        return None  # تخطَّ هذا العنصر بدلاً من رفع خطأ يوقف البرنامج كله
```

```
    return {
        "title": title_tag.get("title", "غير معروف"),
        "price": price_tag.get_text(strip=True),
    }
```

نصيحة خبير: لاحظ النمط هنا: بدلاً من ترك البرنامج ينهار عند أول خطأ (كصفحة مفقودة أو عنصر HTML غير متوقع)، نتعامل مع كل حالة فشل بلطف ونُكمل العمل. هذا مبدأ أساسي في الأتمتة الاحترافية: **السكربت الذي يعمل طوال الليل بدون إشراف بشري يجب أن يتحمّل الأخطاء الجزئية دون توقف كامل.**

21.10 ماذا عن المواقع التي تعتمد على JavaScript؟

مكتبة `requests` تجلب HTML **الخام** فقط كما يُرسله الخادم، دون تنفيذ أي JavaScript. لكن كثيرًا من المواقع الحديثة (تطبيقات React, Vue...) تبني محتواها **ديناميكيًا بعد تحميل الصفحة** عبر JavaScript، فتجد `requests` يُرجع صفحة شبه فارغة.

مقارنة: الأدوات حسب نوع الموقع

نوع الموقع	الأداة المناسبة	السبب
HTML ثابت (المحتوى موجود في المصدر مباشرة)	`requests` + `BeautifulSoup`	سريع وخفيف، لا حاجة لمتصفح حقيقي
يعتمد على JavaScript (المحتوى يُبنى ديناميكيًا)	**Selenium** أو **Playwright**	يُشغّلان متصفحًا حقيقيًا (أو بدون واجهة) وينتظران تحميل JavaScript قبل الاستخراج

كيف تعرف أيّهما تحتاج؟ افتح الصفحة في المتصفح، ثم اضغط كليك يمين ← «عرض مصدر الصفحة» (View Page Source، وليس Inspect). إذا رأيت البيانات التي تريدها موجودة في المصدر، فـ `requests` كافية. إذا كان المصدر شبه فارغ وكل المحتوى يظهر فقط بعد التحميل، تحتاج Selenium/Playwright.

هذا الفصل يركّز على `requests` / `BeautifulSoup` لأنهما الأساس الذي يجب إتقانه أولاً، وهما كافيان لغالبية الحالات العملية والتدريبية.

21.11 مشروع تطبيقي متكامل: مستخرج بيانات كتب شامل

سنبني أداة أتمتة كاملة تستخرج بيانات كل الكتب من موقع `books.toscrape.com` عبر جميع صفحاته، تحفظها بشكل منظّم، ثم تنتج تحليلًا سريعًا عنها.

هيكلية المشروع

```
book_scraper/
├── pyproject.toml
├── output/
└── src/
    └── book_scraper/
        ├── __init__.py
        ├── fetcher.py
        ├── parser.py
        ├── storage.py
        └── main.py
```

Terminal

```
uv init book_scraper
cd book_scraper
uv add requests beautifulsoup4 lxml pandas
mkdir -p src/book_scraper output
touch src/book_scraper/__init__.py
```

`src/book_scraper/fetcher.py` — جلب الصفحات بصلابة

Python

```python
"""وحدة جلب صفحات الويب مع معالجة الأخطاء وإعادة المحاولة."""

import time

import requests

USER_AGENT = "Educational Book Scraper (learning purposes, github.com/example)"
REQUEST_TIMEOUT = 10
DELAY_BETWEEN_REQUESTS = 1.0  # احترام الخادم -- ثانية

def fetch_page(url: str, retries: int = 3) -> str | None:
    """جلب محتوى HTML عند الفشل مع إعادة محاولة للصفحة.

    Args:
        url: رابط الصفحة المطلوبة.
        retries: عدد محاولات إعادة الطلب عند حدوث خطأ مؤقت.

    Returns:
        محتوى HTML أو None كنص، إذا فشلت كل المحاولات أو الصفحة غير موجودة.
    """
    headers = {"User-Agent": USER_AGENT}

    for attempt in range(1, retries + 1):
```

```
        try:
            response = requests.get(url, headers=headers, timeout=REQUEST_TIMEOUT)

            if response.status_code == 404:
                return None  # ليس خطأ، بل إشارة توقف طبيعية -- نهاية الصفحات

            response.raise_for_status()
            return response.text

        except requests.Timeout:
            print(f"⏱️ [محاولة {attempt}/{retries}] انتهت المهلة الزمنية: {url}")
        except requests.ConnectionError:
            print(f"🔌 [محاولة {attempt}/{retries}] تعذّر الاتصال: {url}")
        except requests.HTTPError as e:
            print(f"❌ [محاولة {attempt}/{retries}] خطأ HTTP: {e}")

        if attempt < retries:
            time.sleep(2 * attempt)

    print(f"🚫 فشلت كل المحاولات لجلب: {url}")
    return None

def polite_delay() -> None:
    """تأخير مهذّب بين الطلبات المتتالية لاحترام خادم الموقع."""
    time.sleep(DELAY_BETWEEN_REQUESTS)
```

`src/book_scraper/parser.py` — تحليل HTML واستخراج البيانات

Python

```
"""وحدة تحليل صفحات الكتب واستخراج البيانات منها."""

from dataclasses import dataclass

from bs4 import BeautifulSoup

RATING_WORDS_TO_NUMBERS = {
    "One": 1, "Two": 2, "Three": 3, "Four": 4, "Five": 5,
}

@dataclass
class Book:
    """يمثّل بيانات كتاب واحد مُستخرجة من الموقع."""

    title: str
    price: float
    rating: int
    availability: str
```

```
def _parse_price(price_text: str) → float:
    """تحويل نص السعر مثل '51.77£' إلى رقم عشري."""
    cleaned = price_text.replace("£", "").strip()
    return float(cleaned)

def _parse_rating(star_rating_tag) → int:
    """استخراج رقم التقييم من اسم الـ class (مثل 'Three' → 3)."""
    classes = star_rating_tag.get("class", [])
    for cls in classes:
        if cls in RATING_WORDS_TO_NUMBERS:
            return RATING_WORDS_TO_NUMBERS[cls]
    return 0

def extract_books_from_page(html: str) → list[Book]:
    """استخراج كل الكتب من صفحة HTML واحدة.

    Args:
        html: محتوى HTML الخام للصفحة.

    Returns:
        قائمة من كائنات Book، مع تخطي أي عنصر بيانات ناقصة.
    """
    soup = BeautifulSoup(html, "lxml")
    book_elements = soup.select("article.product_pod")

    books: list[Book] = []

    for element in book_elements:
        title_tag = element.select_one("h3 a")
        price_tag = element.select_one("p.price_color")
        rating_tag = element.select_one("p.star-rating")
        availability_tag = element.select_one("p.instock.availability")

        if not all([title_tag, price_tag, rating_tag, availability_tag]):
            continue  # تخطَّ أي كتاب ببيانات ناقصة بدلاً من إيقاف البرنامج

        book = Book(
            title=title_tag.get("title", "غير معروف"),
            price=_parse_price(price_tag.get_text(strip=True)),
            rating=_parse_rating(rating_tag),
            availability=availability_tag.get_text(strip=True),
        )
        books.append(book)

    return books

def has_next_page(html: str) → bool:
    """التحقق من وجود صفحة تالية في نظام التصفّح."""
    soup = BeautifulSoup(html, "lxml")
    return soup.select_one("li.next") is not None
```

نصيحة خبير: استخدمنا `dataclass` (كما تعلمنا في الفصل 14) لتمثيل `Book` بدلاً من `dict` عادي. هذا يمنحنا Type Hints واضحة، اكتشاف أخطاء مبكر عبر mypy، وكودًا أسهل قراءة من `book["price"]` إلى `book.price`.

`src/book_scraper/storage.py` — حفظ البيانات

Python

```
"""وحدة حفظ بيانات الكتب المُستخرجة."""

from dataclasses import asdict
from pathlib import Path

import pandas as pd

from book_scraper.parser import Book

def save_books_to_csv(books: list[Book], file_path: str | Path) -> None:
    """حفظ قائمة الكتب في ملف CSV باستخدام pandas.

    Args:
        books: قائمة كائنات Book المُستخرجة.
        file_path: مسار ملف الحفظ.
    """
    records = [asdict(book) for book in books]
    df = pd.DataFrame(records)
    df.to_csv(file_path, index=False, encoding="utf-8-sig")
```

`src/book_scraper/main.py` — تجميع كل شيء

Python

```
"""نقطة الدخول الرئيسية لأداة استخراج بيانات الكتب."""

from pathlib import Path

import pandas as pd

from book_scraper.fetcher import fetch_page, polite_delay
from book_scraper.parser import Book, extract_books_from_page, has_next_page
from book_scraper.storage import save_books_to_csv

BASE_URL = "https://books.toscrape.com/catalogue/page-{}.html"
OUTPUT_FILE = Path("output/books.csv")
MAX_PAGES = 5  # نحدد سقفًا احترازيًا لتجنب حلقة لا نهائية في حال خطأ غير متوقع

def scrape_all_books() -> list[Book]:
```

```
    """استخراج كل الكتب من كل الصفحات المتاحة."""

    Returns:
        قائمة شاملة بكل الكتب المُستخرجة من كل الصفحات.
    """
    all_books: list[Book] = []
    page_number = 1

    while page_number <= MAX_PAGES:
        url = BASE_URL.format(page_number)
        print(f"📄 جلب الصفحة {page_number}...")

        html = fetch_page(url)
        if html is None:
            print("🏁 توقف. لا مزيد من الصفحات أو حدث خطأ غير قابل للتعافي.")
            break

        books_in_page = extract_books_from_page(html)
        all_books.extend(books_in_page)
        print(f"   ✅ تم استخراج {len(books_in_page)} كتاب")

        if not has_next_page(html):
            print("🏁 تم الوصول لآخر صفحة.")
            break

        page_number += 1
        polite_delay()

    return all_books

def print_analysis(books: list[Book]) -> None:
    """طباعة تحليل سريع لبيانات الكتب المُستخرجة."""
    if not books:
        print("لا توجد بيانات للتحليل.")
        return

    df = pd.DataFrame([{"title": b.title, "price": b.price, "rating": b.rating} for b in books])

    print("\n" + "=" * 50)
    print(" " * 12 + "تحليل بيانات الكتب المُستخرجة")
    print("=" * 50)
    print(f"إجمالي الكتب: {len(df)}")
    print(f"متوسط السعر: £{df['price'].mean():.2f}")
    print(f"أغلى كتاب: £{df['price'].max():.2f}")
    print(f"أرخص كتاب: £{df['price'].min():.2f}")
    print(f"متوسط التقييم: {df['rating'].mean():.1f} / 5")
    print("=" * 50)

def main() -> None:
    """تشغيل عملية الاستخراج الكاملة."""
    print("🚀 بدء استخراج بيانات الكتب من books.toscrape.com\n")
```

```
    books = scrape_all_books()

    print(f"\n💾 حفظ {len(books)} كتاب في {OUTPUT_FILE}.  ")
    OUTPUT_FILE.parent.mkdir(exist_ok=True)
    save_books_to_csv(books, OUTPUT_FILE)

    print_analysis(books)
    print(f"\n✅ اكتمل! راجع الملف: {OUTPUT_FILE}")

if __name__ == "__main__":
    main()
```

تشغيل المشروع:

Terminal

```
uv run python -m book_scraper.main
```

الناتج المتوقع:

```
🚀 بدء استخراج بيانات الكتب من books.toscrape.com

📄 جلب الصفحة 1...
   ✅ تم استخراج 20 كتاب
📄 جلب الصفحة 2...
   ✅ تم استخراج 20 كتاب
...

💾 حفظ 100 كتاب في output/books.csv...

==================================================
            تحليل بيانات الكتب المُستخرجة
==================================================
إجمالي الكتب: 100
متوسط السعر: £36.47
أغلى كتاب: £59.99
أرخص كتاب: £10.00
متوسط التقييم: 2.9 / 5
==================================================

✅ اكتمل! راجع الملف: output/books.csv
```

✅ الحل الكامل للمشروع (جميع الملفات مجتمعة)

جميع ملفات `fetcher.py` ، `parser.py` ، `storage.py` ، `main.py` كما وردت بالكامل أعلاه تُشكّل الحل النموذجي الكامل. تثبيت التبعيات:

```
Terminal
uv add requests beautifulsoup4 lxml pandas
```

نقاط تصميم مهمة في هذا الحل:

1. **فصل المسؤوليات**: `fetcher.py` (الشبكة) - `parser.py` (التحليل) - `storage.py` (الحفظ) - `main.py` (التنسيق).
2. **الصلابة (Robustness)**: كل دالة تتعامل بلطف مع الفشل الجزئي (عنصر مفقود، صفحة غير موجودة) بدلاً من إيقاف البرنامج بالكامل.
3. **احترام الخادم**: `polite_delay()` بين كل طلب وآخر، و `User-Agent` واضح يُعرّف عن طبيعة الزيارة.
4. **سقف أمان** (`MAX_PAGES`): يمنع حلقة لا نهائية محتملة إذا حدث خطأ منطقي غير متوقع في اكتشاف «آخر صفحة».
5. **ربط مباشر بالفصل 20**: استخدمنا pandas للتحليل الفوري للبيانات المُستخرجة — توضيحًا عمليًا لكيفية تكامل مسارات الكتاب المختلفة معًا.

21.12 تمارين

🟢 مستوى مبتدئ

1. استخدم `requests` لجلب صفحة `https://books.toscrape.com/` واطبع `status_code` و `len(response.text)` .
2. استخدم BeautifulSoup لاستخراج **نص** عنوان الصفحة (`<title>`) من نفس الصفحة.
3. استخرج كل روابط `<a>` الموجودة في الصفحة الرئيسية واطبع أول 5 منها.

✅ الحل المقترح

```python
import requests
from bs4 import BeautifulSoup

response = requests.get("https://books.toscrape.com/", timeout=10)
print(response.status_code)
print(len(response.text))

soup = BeautifulSoup(response.text, "lxml")
print(soup.title.get_text(strip=True))

links = soup.find_all("a", href=True)
for link in links[:5]:
    print(link["href"])
```

🟡 مستوى متوسط

4. عدّل مشروع الكتب ليستخرج أيضًا **فئة المنتج (category)** من رابط صفحة الكتاب (تحتاج زيارة صفحة كل كتاب على حدة — رابط صفحة الكتاب موجود في `h3 a` كخاصية `href`). احذر: هذا يتطلب طلب HTTP إضافي لكل كتاب، فلا تنسَ `polite_delay()`!
5. اكتب دالة `filter_books_by_rating(books, min_rating)` تُرجع فقط الكتب التي تقييمها أكبر من أو يساوي قيمة معينة.

✅ الحل المقترح

Python

```python
# تمرين 5 Book -- سهل التطبيق مباشرة على كائنات
def filter_books_by_rating(books: list[Book], min_rating: int) → list[Book]:
    """إرجاع الكتب التي تقييمها أكبر من أو يساوي الحد الأدنى المطلوب."""
    return [book for book in books if book.rating ⩾ min_rating]

# مثال استخدام
high_rated = filter_books_by_rating(books, min_rating=4)
print(f"عدد الكتب ذات التقييم 4+: {len(high_rated)}")
```

(تمرين 4 يتطلب دمج زيارة صفحة فرعية لكل كتاب، مشابه تمامًا لنمط `fetch_page` الموجود في المشروع، مع استخلاص `href` من `title_tag` وزيارته، ثم البحث عن عنصر الفئة في شريط التنقل Breadcrumb الخاص بصفحة الكتاب.)

🔴 مستوى تحدي

6. حوّل مشروع استخراج الكتب إلى **نسخة غير متزامنة (async)** باستخدام `aiohttp` (كما تعلمنا في الفصل 17) لجلب عدة صفحات بالتوازي بدلاً من التسلسل، **مع الحفاظ على تأخير مناسب يحترم الخادم** (مثلاً عبر `asyncio.Semaphore` لتحديد عدد الطلبات المتزامنة القصوى بـ 3 طلبات فقط في نفس الوقت).

✅ الحل المقترح (تصور مبسّط)

Python

```python
import asyncio

import aiohttp

SEMAPHORE_LIMIT = 3  # عدد الطلبات المتزامنة القصوى -- احترامًا للخادم

async def fetch_page_async(session: aiohttp.ClientSession, url: str, sem: asyncio.Semaphore)
→ str | None:
    """جلب صفحة بشكل غير متزامن مع تحديد التزامن عبر Semaphore."""
    async with sem:
        try:
            async with session.get(url, timeout=aiohttp.ClientTimeout(total=10)) as response:
                if response.status == 404:
                    return None
                response.raise_for_status()
                await asyncio.sleep(1)  # تأخير مهذّب حتى في الوضع المتزامن
                return await response.text()
        except aiohttp.ClientError as e:
            print(f"خطأ: {e}")
            return None

async def scrape_pages_async(urls: list[str]) → list[str | None]:
    """جلب عدة صفحات بالتوازي مع تحديد عدد الطلبات المتزامنة."""
    sem = asyncio.Semaphore(SEMAPHORE_LIMIT)
    async with aiohttp.ClientSession() as session:
        tasks = [fetch_page_async(session, url, sem) for url in urls]
        return await asyncio.gather(*tasks)

# urls = [BASE_URL.format(i) for i in range(1, 6)]
# results = asyncio.run(scrape_pages_async(urls))
```

ملاحظة مهمة: `asyncio.Semaphore(3)` يضمن ألا يُرسل البرنامج أكثر من 3 طلبات في نفس اللحظة، رغم كونه «متزامنًا» من حيث البنية — هذا توازن جيد بين السرعة واحترام الخادم.

21.13 اختبر فهمك (Quiz)

1. ما الفرق بين وظيفة `requests` ووظيفة `BeautifulSoup`؟

`requests` مسؤولة عن **جلب** محتوى الصفحة عبر إرسال طلب HTTP واستقبال الاستجابة (HTML خام كنص). `BeautifulSoup` مسؤولة عن **تحليل** هذا النص الخام وتحويله إلى هيكل يمكن البحث فيه واستخراج بيانات محددة منه بسهولة. الأولى تتعامل مع الشبكة، والثانية تتعامل مع تحليل البنية.

2. لماذا يجب دائمًا تحديد `timeout` في طلبات `requests`؟

لأنه بدون `timeout`، إذا توقف الخادم البعيد عن الاستجابة لأي سبب، سيبقى البرنامج معلّقًا إلى الأبد دون أي رسالة خطأ أو طريقة للتعافي تلقائيًا. `timeout` يضمن أن البرنامج يرفع استثناءً قابلًا للمعالجة بعد مدة معقولة.

3. لماذا نكتب `class_="product"` بدلاً من `"class="product"` في BeautifulSoup؟

لأن `class` كلمة مفتاحية محجوزة في لغة Python نفسها (تُستخدم لتعريف الأصناف Classes)، فاستخدامها كاسم معامل مباشرة يُسبب خطأ صياغي. BeautifulSoup يستخدم `class_` (بشرطة سفلية) كحل بديل لتجنب هذا التعارض.

4. اذكر 3 قواعد أخلاقية يجب اتباعها قبل استخراج بيانات أي موقع.

أي ثلاث من: فحص `robots.txt`، قراءة شروط الاستخدام، إضافة تأخير بين الطلبات لعدم إثقال الخادم، عدم استخراج بيانات شخصية أو محمية بحقوق نشر، استخدام API رسمي إن وُجد بدلاً من Web Scraping، وإرسال `User-Agent` واضح يُعرّف عن الزيارة.

5. متى تحتاج Selenium أو Playwright بدلاً من requests + BeautifulSoup؟

عندما يعتمد الموقع على JavaScript لبناء محتواه ديناميكيًا بعد تحميل الصفحة الأولي (كما في تطبيقات React أو Vue)، بحيث لا يحتوي HTML الخام الذي يُرجعه `requests` على البيانات المطلوبة فعليًا. Selenium/Playwright يُشغّلان متصفحًا حقيقيًا (أو بدون واجهة) وينتظران تنفيذ JavaScript قبل استخراج المحتوى النهائي.

6. ما الفرق بين `find_all`/`find` و `select_one`/`select` في BeautifulSoup؟

كلاهما يُستخدم للبحث عن عناصر HTML، لكن `find_all` / `find` يستخدمان صياغة BeautifulSoup الخاصة (وسم + خصائص Python)، بينما `select_one` / `select` يستخدمان صياغة محددات CSS القياسية، وهي غالبًا أقصر وأقوى للتعبير عن علاقات معقدة بين العناصر (مثل "كل `a` داخل `h3` داخل `article.product_pod`").

21.14 ملخص الفصل (Cheat Sheet)

Python

```python
# ===== requests: الطلبات الأساسية =====
import requests
response = requests.get(url, headers={"User-Agent": "..."}, timeout=10)
response.status_code         # 200, 404, 500...
response.text                 # محتوى HTML/نص خام
response.json()               # تحويل تلقائي من JSON
response.raise_for_status()   # يرفع استثناء عند 4xx/5xx

try:
    response = requests.get(url, timeout=10)
except requests.Timeout:
    ...
except requests.ConnectionError:
    ...

# ===== BeautifulSoup: التحليل =====
from bs4 import BeautifulSoup
soup = BeautifulSoup(response.text, "lxml")

soup.find("div", class_="price")       # أول عنصر مطابق
soup.find_all("article")               # كل العناصر المطابقة ← قائمة
soup.select("article.product_pod h3 a")  # محدد CSS ← قائمة
soup.select_one("title")               # محدد CSS ← أول عنصر

tag.text / tag.get_text(strip=True)    # النص الظاهر
tag.get("href")                        # خاصية HTML بأمان (None إن لم توجد)
tag["href"]                            # خاصية HTML (يرفع خطأ إن لم توجد)

# ===== الأتمتة الآمنة =====
import time
time.sleep(1)                          # تأخير بين الطلبات (إلزامي أخلاقيًا)
```

المبدأ الأخلاقي	التطبيق العملي
احترام الخادم	`time.sleep()` بين الطلبات
الشفافية	`User-Agent` واضح
الأمان	`timeout` في كل طلب
الصلابة	`try/except` + إعادة محاولة
احترام القواعد	فحص `robots.txt` وشروط الاستخدام

21.15 ماذا بعد؟

لقد تعلّمت للتو كيف تجعل Python «يتصفح» الإنترنت نيابة عنك، ويستخرج بيانات منظّمة من فوضى صفحات HTML — وربطت هذه المهارة مباشرة بما تعلمته في الفصل السابق عبر تحليل البيانات المُستخرجة بـ pandas. هذا هو بالضبط نمط العمل الحقيقي في المشاريع الاحترافية: جمع البيانات ← تنظيفها ← تحليلها.

في الفصل القادم، ننتقل إلى مسار مثير آخر: **نافذتنا الأولى على الذكاء الاصطناعي**. سنتعرّف على مكتبة `scikit-learn`، ونتعلّم كيف «يتعلّم» الحاسوب من البيانات لاتخاذ تنبؤات، عبر بناء أول نموذج تعلّم آلي بسيط خطوة بخطوة.

فصل 22 / 26 · الجزء الخامس

22

مسار الذكاء الاصطناعي — نافذة أولى (scikit-learn)

10 دقيقة قراءة · 14 قسمًا · 17 مثالًا · 2,084 كلمة

في هذا الفصل

22.1 لماذا هذا الفصل مختلف عن كل ما سبق؟
22.2 المفاهيم الأساسية (بدون رياضيات معقدة)
22.3 التثبيت وأول نموذج
22.4 خط أنابيب تعلّم آلي كامل: التصنيف
22.5 خوارزميات شائعة أخرى (نظرة سريعة)
22.6 تحضير البيانات: الخطوة الأهم (غالبًا)
22.7 مقاييس التقييم (Evaluation Metrics)
22.8 حفظ النموذج واستخدامه لاحقًا
22.9 مشروع تطبيقي متكامل: نظام تنبؤ بأسعار المنازل
22.10 حدود هذا الفصل وأين تذهب بعده
22.11 تمارين
22.12 اختبر فهمك (Quiz)
22.13 ملخص الفصل (Cheat Sheet)
22.14 ماذا بعد؟

22.1 لماذا هذا الفصل مختلف عن كل ما سبق؟

في كل الفصول السابقة، كنت أنت من يكتب **كل القواعد**: «إذا كان السعر أكبر من 100، اعتبره غاليًا»، «إذا كانت الدرجة أكبر من 60، فالطالب ناجح». البرنامج ينفّذ تمامًا ما أخبرته به، لا أكثر ولا أقل.

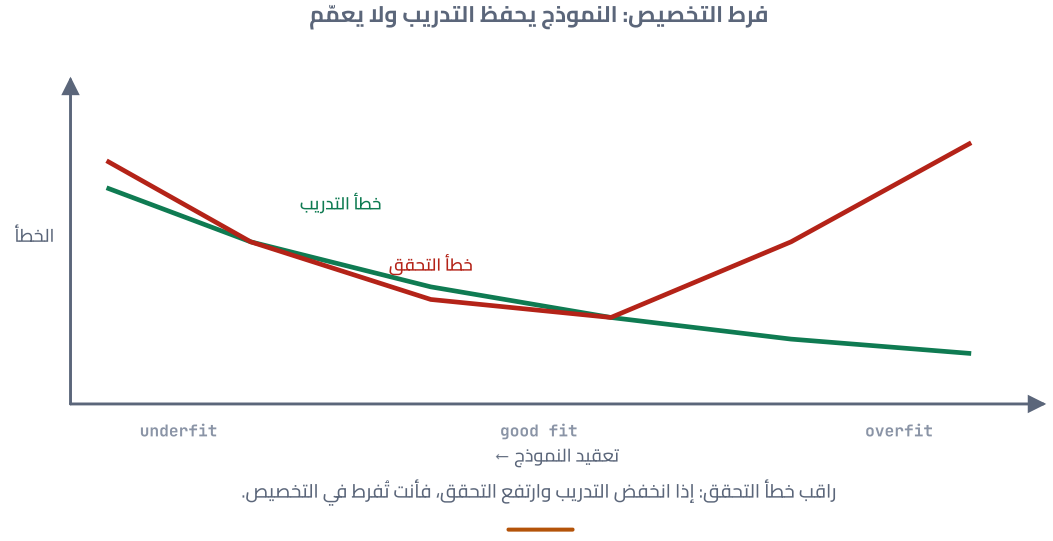
التعلّم الآلي (Machine Learning) يقلب هذا المنطق رأسًا على عقب: بدلاً من أن تكتب أنت القواعد، تُعطي الحاسوب **أمثلة كثيرة** من المدخلات والنتائج الصحيحة، ويقوم هو باستنتاج القواعد بنفسه. ثم يستخدم هذه القواعد المُستنتَجة للتنبؤ بنتائج جديدة لم يرها من قبل.

مقارنة: البرمجة التقليدية مقابل التعلّم الآلي

البرمجة التقليدية	التعلّم الآلي
المدخلات: بيانات + قواعد (كتبتها أنت)	المدخلات: بيانات + نتائج معروفة
المخرجات: نتائج	المخرجات: قواعد (نموذج) يتعلّمها الحاسوب
مثال: `if temp > 30: print("حار")`	مثال: أعطِ الحاسوب 10,000 يوم (درجة حرارة، رطوبة، رياح) مع تصنيف «حار/بارد» لكل يوم، ودعه يكتشف النمط بنفسه
مناسب لـ قواعد واضحة ومعروفة مسبقًا	
التعلم الآلي مناسب لـ أنماط معقدة يصعب كتابتها كقواعد صريحة (كالتعرف على الوجوه، التنبؤ بالأسعار، فلترة البريد المزعج)	

هذا الفصل **ليس** دورة متعمقة في الذكاء الاصطناعي (هذا يستحق كتابًا كاملاً بذاته)، بل **نافذة عملية أولى** تُعطيك فهمًا صحيحًا للمفاهيم الأساسية، وتُمكّنك من بناء أول نموذج تعلّم آلي حقيقي يعمل، عبر مكتبة **scikit-learn** — المكتبة الأشهر والأكثر استخدامًا لتعلّم الآلة «التقليدي» (غير الشبكات العصبية العميقة) في بايثون.

22.2 المفاهيم الأساسية (بدون رياضيات معقدة)



شكل 22.1 — فرط التخصيص: التدريب مقابل التحقق

أنواع التعلّم الآلي

النوع	الوصف	مثال
التعلّم بإشراف (Supervised Learning)	نُعطي النموذج أمثلة بها «الإجابة الصحيحة» معروفة مسبقًا	توقّع سعر منزل بناءً على مساحته (السعر معروف لمنازل سابقة)
التعلّم بدون إشراف (Unsupervised Learning)	لا توجد «إجابة صحيحة»، نكتشف أنماطًا مخفية	تجميع العملاء في شرائح متشابهة دون تصنيف مسبق
تعلّم التعزيز (Reinforcement Learning)	النموذج يتعلّم عبر التجربة والخطأ بمكافآت وعقوبات	تدريب برنامج للعب الشطرنج

في هذا الفصل سنركّز على **التعلّم بإشراف**، وهو الأكثر شيوعًا وسهولة للبدء، وينقسم بدوره إلى نوعين:

النوع	ماذا يتنبأ؟	مثال
الانحدار (Regression)	**رقمًا مستمرًا**	سعر منزل، درجة حرارة، راتب متوقع
التصنيف (Classification)	**فئة من ضمن خيارات محددة**	بريد مزعج/ليس مزعجًا، نوع زهرة، هل سيُسدّد القرض؟

المصطلحات الأساسية

المصطلح	المعنى
Features (الخصائص/المتغيرات المستقلة)	المعلومات المُدخَلة التي نستخدمها للتنبؤ (مساحة المنزل، عدد الغرف...)
Label / Target (التسمية/الهدف)	القيمة التي نريد التنبؤ بها (السعر)
Training (التدريب)	عملية «تعليم» النموذج من البيانات
Model (النموذج)	النتيجة النهائية بعد التدريب — «القواعد» التي تعلّمها الحاسوب
Prediction (التنبؤ)	استخدام النموذج المُدرَّب على بيانات جديدة لم يرها من قبل

لماذا نقسّم البيانات إلى تدريب واختبار؟

هذا من أهم المفاهيم في كل هذا المجال. لو درّبنا النموذج واختبرناه على **نفس** البيانات، فسنحصل على نتيجة مضلِّلة — تمامًا كطالب يحفظ إجابات امتحان رآه مسبقًا: سيحصل على علامة كاملة، لكن هذا لا يعني أنه **فهم** المادة.

لذلك نقسّم البيانات دائمًا:
- **بيانات التدريب (Training Set)**: عادة 70-80% من البيانات، يتعلّم منها النموذج.
- **بيانات الاختبار (Test Set)**: 20-30% المتبقية، «مخفية» عن النموذج أثناء التدريب، نستخدمها لقياس أداءه الحقيقي على بيانات لم يرها من قبل.

> ⚠ **تحذير: الإفراط في التخصيص (Overfitting)**
> إذا كان النموذج «يحفظ» بيانات التدريب بدلاً من «تعلّم النمط العام»، سيحصل على دقة ممتازة على بيانات التدريب لكن أداء سيئ جدًا على بيانات جديدة. هذا يُسمى **Overfitting** (الإفراط في التخصيص)، وهو أشهر مشكلة في تعلّم الآلة. **نقيضه** هو **Underfitting** (نقص التخصيص): النموذج بسيط جدًا لدرجة لا يستطيع حتى فهم بيانات التدريب نفسها.
>
> **علامة الخطر:** دقة عالية جدًا على بيانات التدريب (مثلاً 99%) مقابل دقة ضعيفة على بيانات الاختبار (مثلاً 60%) — هذا دليل واضح على Overfitting.

22.3 التثبيت وأول نموذج

Terminal

```
uv add scikit-learn pandas numpy matplotlib
```

مثال 1: انحدار خطي بسيط (Linear Regression)

لنبدأ بأبسط مثال ممكن: التنبؤ بسعر منزل بناءً على مساحته فقط.

Python

```python
import numpy as np
from sklearn.linear_model import LinearRegression

# بيانات تدريب: مساحة المنزل (م²) ← السعر (بالآلاف)
# Features يجب أن تكون دائمًا مصفوفة ثنائية الأبعاد: [[قيمة], [قيمة], ...]
sizes = np.array([[50], [70], [90], [120], [150], [200]])
prices = np.array([150, 200, 250, 320, 400, 500])  # Target أحادي البعد

# إنشاء النموذج وتدريبه
model = LinearRegression()
model.fit(sizes, prices)  # "fit" = التدريب

# التنبؤ بسعر منزل جديد مساحته 100 م²
new_size = np.array([[100]])
predicted_price = model.predict(new_size)

print(f"السعر المتوقع لمنزل 100م²: {predicted_price[0]:.1f} ألف")

# معاملات النموذج المُتعلَّمة (المعادلة: price = slope * size + intercept)
print(f"الميل (slope): {model.coef_[0]:.2f}")
print(f"التقاطع (intercept): {model.intercept_:.2f}")
```

```
السعر المتوقع لمنزل 100م²: 266.7 ألف
الميل (slope): 2.31
التقاطع (intercept): 35.00
```

لاحظ السحر هنا: لم نكتب أبدًا `price = size * 2.31 + 35` بأنفسنا — النموذج **اكتشف** هذه المعادلة بنفسه من الأمثلة!

نصيحة خبير: scikit-learn يتبع دائمًا نفس النمط الثابت لكل نموذج، بغض النظر عن تعقيده:

```python
model = SomeAlgorithm()      # 1. اختر الخوارزمية
model.fit(X_train, y_train)  # 2. درّب النموذج
model.predict(X_new)         # 3. تنبأ ببيانات جديدة
```

بمجرد إتقان هذا النمط، يمكنك استخدام عشرات الخوارزميات المختلفة بنفس الطريقة تقريبًا.

22.4 خط أنابيب تعلّم آلي كامل: التصنيف

شكل 22.2 — خط أنابيب التعلّم الآلي

لنبنِ الآن مثالًا أكثر واقعية: تصنيف. سنستخدم بيانات **Iris** الشهيرة (مجموعة بيانات كلاسيكية عن زهور، مدمجة في scikit-learn) للتنبؤ بنوع الزهرة بناءً على قياساتها.

Python

```python
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

# 1. تحميل البيانات
iris = load_iris()
X = iris.data      # Features: (أعمدة 4) طول/عرض البتلة والكأس
y = iris.target    # Target: (2 ,1 ,0) نوع الزهرة

print(pd.DataFrame(X, columns=iris.feature_names).head())
print("الأنواع:", iris.target_names)

# 2. تقسيم البيانات إلى تدريب واختبار
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,       # 20% للاختبار
    random_state=42,     # لضمان نفس التقسيم في كل مرة (قابلية إعادة الإنتاج)
)

print(f"بيانات التدريب: {len(X_train)} عينة")
print(f"بيانات الاختبار: {len(X_test)} عينة")

# 3. اختيار الخوارزمية وتدريب النموذج
model = KNeighborsClassifier(n_neighbors=3)
model.fit(X_train, y_train)

# 4. التنبؤ على بيانات الاختبار
predictions = model.predict(X_test)

# 5. تقييم الأداء
accuracy = accuracy_score(y_test, predictions)
print(f"الدقة: {accuracy:.2%}")
```

```
   sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)
0                5.1               3.5                1.4               0.2
...
الأنواع: ['setosa' 'versicolor' 'virginica']
بيانات التدريب: 120 عينة
بيانات الاختبار: 30 عينة
الدقة: 100.00%
```

> **نصيحة خبير:** `random_state`
> معظم دوال scikit-learn التي تحتوي عشوائية (كـ `train_test_split`) تقبل معامل `random_state`. تثبيته على رقم معيّن (أي رقم، الشائع هو `42`) يضمن حصولك على **نفس النتائج بالضبط** في كل مرة تُشغّل فيها الكود — وهذا ضروري للتجربة العلمية القابلة لإعادة الإنتاج (Reproducibility).

كيف يعمل خوارزمية K-Nearest Neighbors (KNN) ببساطة؟

شرح مفاهيمي سريع لما حدث للتو: `KNeighborsClassifier(n_neighbors=3)` تعمل بفكرة بسيطة جدًا: لتصنيف نقطة بيانات جديدة، انظر إلى **أقرب 3 نقاط** لها في بيانات التدريب (حسب المسافة الرياضية بين القياسات)، وخذ **الفئة الأكثر تكرارًا** بينها كتنبؤ. لا حاجة لفهم الرياضيات بعمق الآن — المهم أن تفهم الفكرة العامة: «احكم على الشيء الجديد بناءً على أشباهه المعروفين».

22.5 خوارزميات شائعة أخرى (نظرة سريعة)

scikit-learn يوفّر عشرات الخوارزميات، وكلها تُستخدم بنفس نمط `fit` / `predict`. إليك أشهرها:

الخوارزمية	النوع	الاستخدام الأمثل
`LinearRegression`	انحدار	علاقات خطية بسيطة
`LogisticRegression`	تصنيف (رغم الاسم!)	تصنيف ثنائي بسيط (نعم/لا)
`KNeighborsClassifier`	تصنيف	بيانات صغيرة-متوسطة، سهل الفهم
`DecisionTreeClassifier`	تصنيف/انحدار	قابل للتفسير بصريًا (كشجرة قرارات)
`RandomForestClassifier`	تصنيف/انحدار	دقة عالية عمومًا، مقاوم لـ Overfitting
`SVC` (Support Vector Machine)	تصنيف	فعّال في أبعاد عالية

مثال سريع على استبدال الخوارزمية بسهولة تامة (هذه هي قوة النمط الموحّد لـ scikit-learn):

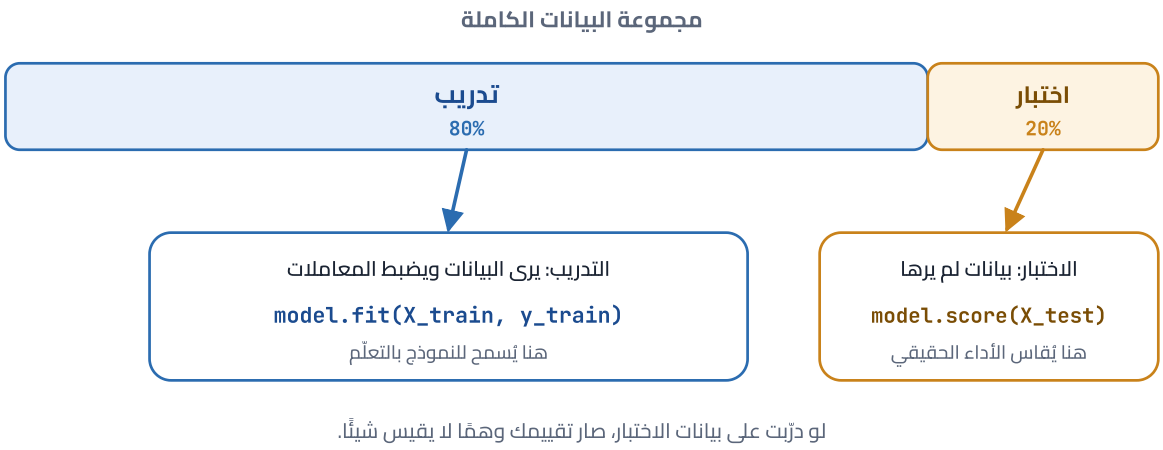
```python
from sklearn.ensemble import RandomForestClassifier

# نفس البيانات بالضبط، فقط غيّرنا الخوارزمية
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)
predictions = model.predict(X_test)
print(f"دقة Random Forest: {accuracy_score(y_test, predictions):.2%}")
```

مقارنة: كيف تختار الخوارزمية المناسبة؟

إذا كانت أولويتك	اختر
البساطة والسرعة للبدء	`LogisticRegression` ، `KNeighborsClassifier`
قابلية التفسير (فهم سبب القرار)	`DecisionTreeClassifier`
أعلى دقة ممكنة	`RandomForestClassifier` ، Gradient Boosting
بيانات كبيرة جدًا	خوارزميات خطية أو نماذج تعلّم عميق متخصصة (خارج نطاق هذا الفصل)

نصيحة عملية: لا يوجد «أفضل خوارزمية» مطلقة — جرّب عدة خوارزميات على بياناتك وقارن أداءها فعليًا.

22.6 تحضير البيانات: الخطوة الأهم (غالبًا)



شكل 22.3 — تقسيم البيانات إلى تدريب واختبار

في الحياة الحقيقية، 80% من وقت أي مشروع تعلّم آلي يُصرَف على **تحضير البيانات**، وليس على اختيار الخوارزمية.

التعامل مع البيانات النصية (Encoding)

النماذج الرياضية تفهم أرقامًا فقط. يجب تحويل أي عمود نصي إلى أرقام:

```python
import pandas as pd
from sklearn.preprocessing import LabelEncoder, OneHotEncoder

df = pd.DataFrame({
    "المدينة": ["الرياض", "جدة", "الرياض", "الدمام"],
    "السعر": [100, 150, 120, 90],
})

# الطريقة 1: Label Encoding -- يحوّل كل فئة لرقم (مناسب للفئات المرتبة فقط!)
encoder = LabelEncoder()
df["المدينة_مشفرة"] = encoder.fit_transform(df["المدينة"])
print(df)

# الطريقة 2: One-Hot Encoding -- عمود منفصل لكل فئة (الأنسب غالبًا للفئات غير المرتبة)
df_encoded = pd.get_dummies(df, columns=["المدينة"])
print(df_encoded)
```

⚠ **تحذير: متى تستخدم Label Encoding ومتى One-Hot Encoding؟**

`LabelEncoder` يحوّل «الرياض»→0، «جدة»→1، «الدمام»→2. المشكلة: النموذج قد **يفترض خطأً** أن هناك ترتيبًا رياضيًا بينها (كأن الدمام < جدة < الرياض)، وهذا غير منطقي لفئات لا تحمل ترتيبًا طبيعيًا (كالمدن). **القاعدة العامة**: استخدم `pd.get_dummies` / `OneHotEncoder` للفئات **غير المرتبة** (مدن، ألوان، أنواع منتجات)، واستخدم `LabelEncoder` فقط للفئات **المرتبة فعليًا** (ضعيف < متوسط < ممتاز).

قياس البيانات (Feature Scaling)

بعض الخوارزميات (مثل KNN وSVM) حساسة جدًا لمقياس الأرقام. إذا كان عمود «السعر» يتراوح بين 0-100000 وعمود «عدد الغرف» بين 1-10، فإن الفرق الهائل في المقياس يُضلّل النموذج:

```python
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

# fit_transform على بيانات التدريب فقط
X_train_scaled = scaler.fit_transform(X_train)

# transform فقط (وليس fit) على بيانات الاختبار!
X_test_scaled = scaler.transform(X_test)
```

⚠ **تحذير مهم جدًا: تسرّب البيانات (Data Leakage)**
لاحظ أننا استخدمنا `fit_transform` على بيانات التدريب فقط، و `transform` فقط (بدون `fit`) على بيانات الاختبار. إذا استخدمت `fit_transform` على بيانات الاختبار أيضًا، فأنت «تسرّب» معلومات إحصائية من بيانات الاختبار (التي يُفترض أنها «غير مرئية») إلى عملية التحضير، مما يُعطي تقييمًا مضلِّلًا للأداء الحقيقي للنموذج. **القاعدة الذهبية:** كل ما تتعلمه (`fit`) يجب أن يأتي من بيانات التدريب فقط.

استخدام `Pipeline` لتنظيم كل الخطوات

Python

```python
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier

# يُطبّق كل الخطوات تلقائيًا وبالترتيب الصحيح الـ Pipeline
pipeline = Pipeline([
    ("scaler", StandardScaler()),
    ("classifier", KNeighborsClassifier(n_neighbors=3)),
])

pipeline.fit(X_train, y_train)          # يُطبّق scaler ثم classifier تلقائيًا
predictions = pipeline.predict(X_test)  # بدون تسرّب بيانات -- ونفس الشيء هنا!
```

💡 **نصيحة خبير:** `Pipeline` ليست مجرد أناقة في الكود — إنها **تحمي تلقائيًا** من مشكلة تسرّب البيانات، وتجعل كامل خط المعالجة قابلاً للحفظ والنشر كوحدة واحدة.

22.7 مقاييس التقييم (Evaluation Metrics)

للتصنيف (Classification)

```python
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report,
)

accuracy = accuracy_score(y_test, predictions)        # نسبة التنبؤات الصحيحة
precision = precision_score(y_test, predictions, average="macro")  # دقة التنبؤات الإيجابية
recall = recall_score(y_test, predictions, average="macro")        # نسبة اكتشاف الحالات الإيجابية الفعلية
f1 = f1_score(y_test, predictions, average="macro")   # متوسط توافقي بين precision وrecall

print(classification_report(y_test, predictions))  # تقرير شامل بكل المقاييس دفعة واحدة
print(confusion_matrix(y_test, predictions))       # مصفوفة توضح أنواع الأخطاء بالتفصيل
```

⚠ **تحذير: الدقة (Accuracy) وحدها قد تكون مضلِّلة!**
تخيّل نموذجًا للكشف عن احتيال بطاقات ائتمان، حيث 99% من المعاملات شرعية و1% فقط احتيال. نموذج «غبي» يتنبأ دائمًا بـ «شرعي» سيحقق **دقة 99%**! لكنه عديم الفائدة تمامًا — فهو لم يكتشف أي حالة احتيال. في مثل هذه الحالات (**البيانات غير المتوازنة - Imbalanced Data**)، يجب الاعتماد على `precision` ، `recall` ، و`f1_score` بدلاً من `accuracy` وحدها.

للانحدار (Regression)

```python
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

mae = mean_absolute_error(y_test, predictions)   # متوسط الخطأ المطلق
mse = mean_squared_error(y_test, predictions)    # متوسط مربع الخطأ (يُعاقب الأخطاء الكبيرة أكثر)
r2 = r2_score(y_test, predictions)               # نسبة التباين المُفسَّر (0 إلى 1، كلما اقترب من 1 كان أفضل)

print(f"متوسط الخطأ المطلق: {mae:.2f}")
print(f"R²: {r2:.3f}")
```

22.8 حفظ النموذج واستخدامه لاحقًا

تدريب نموذج قد يستغرق وقتًا طويلًا. لا نريد إعادة تدريبه في كل مرة — نحفظه كملف، ونعيد تحميله عند الحاجة:

```python
import joblib

# حفظ النموذج المُدرَّب
joblib.dump(pipeline, "model.joblib")

# تحميله لاحقًا (في سكربت آخر، أو حتى بعد أسابيع)
loaded_model = joblib.load("model.joblib")
new_prediction = loaded_model.predict(X_new)
```

نصيحة خبير: هذا هو بالضبط ما يحدث خلف الكواليس في التطبيقات الحقيقية: نموذج يُدرَّب **مرة واحدة** (قد يستغرق ساعات)، يُحفَظ كملف، ثم يُحمَّل داخل واجهة API (تذكّر الفصل 19! FastAPI) لتقديم تنبؤات فورية للمستخدمين دون إعادة التدريب في كل طلب.

22.9 مشروع تطبيقي متكامل: نظام تنبؤ بأسعار المنازل

سنبني مشروعًا متكاملًا: تحميل بيانات، تحضيرها، تدريب عدة نماذج ومقارنتها، واختيار الأفضل، مع حفظه للاستخدام اللاحق.

هيكلية المشروع

```
house_price_predictor/
├── pyproject.toml
├── data/
│   └── houses.csv
├── models/
└── src/
    └── house_price_predictor/
        ├── __init__.py
        ├── data_prep.py
        ├── training.py
        └── main.py
```

Terminal

```
uv init house_price_predictor
cd house_price_predictor
uv add scikit-learn pandas numpy joblib
mkdir -p src/house_price_predictor data models
touch src/house_price_predictor/__init__.py
```

توليد بيانات تجريبية: `data/generate_data.py`

Python

```python
"""سكربت توليد بيانات منازل تجريبية واقعية."""

import numpy as np
import pandas as pd

np.random.seed(42)
n = 500

sizes = np.random.uniform(50, 400, n)               # المساحة بالمتر المربع
rooms = np.random.randint(1, 8, n)                   # عدد الغرف
age = np.random.uniform(0, 40, n)                    # عمر المبنى بالسنوات
city = np.random.choice(["الدمام" ,"جدة" ,"الرياض"], n, p=[0.5, 0.3, 0.2])

city_premium = {"الدمام" :1.0 ,"جدة" :1.15 ,"الرياض" :1.3}
base_price = (
    sizes * 8
    + rooms * 50
    - age * 10
    + np.random.normal(0, 100, n)  # ضوضاء عشوائية واقعية
)
price = base_price * [city_premium[c] for c in city]
price = np.clip(price, 100, None)  # لا أسعار سالبة

df = pd.DataFrame({
    "المساحة": sizes.round(1),
    "عدد_الغرف": rooms,
    "عمر_المبنى": age.round(1),
    "المدينة": city,
    "السعر": price.round(0),
})

df.to_csv("data/houses.csv", index=False, encoding="utf-8-sig")
print(f"تم توليد {n} عقار في data/houses.csv")
```

Terminal

```
uv run python data/generate_data.py
```

src/house_price_predictor/data_prep.py

Python

```python
"""وحدة تحضير بيانات المنازل للتدريب."""

import pandas as pd
from sklearn.model_selection import train_test_split

FEATURE_COLUMNS = ["المدينة" ,"عمر_المبنى" ,"عدد_الغرف" ,"المساحة"]
TARGET_COLUMN = "السعر"

def load_data(file_path: str) -> pd.DataFrame:
    """تحميل بيانات المنازل من ملف CSV."""
    return pd.read_csv(file_path, encoding="utf-8-sig")

def prepare_features(df: pd.DataFrame) -> pd.DataFrame:
    """تحويل الأعمدة النصية إلى صيغة رقمية مناسبة للنموذج.

    نستخدم One-Hot Encoding للمدينة لأنها فئة غير مرتبة.
    """
    return pd.get_dummies(df[FEATURE_COLUMNS], columns=["المدينة"])

def split_data(
    df: pd.DataFrame, test_size: float = 0.2, random_state: int = 42
):
    """تقسيم البيانات إلى مجموعتي تدريب واختبار.

    Returns:
        X_train, X_test, y_train, y_test
    """
    X = prepare_features(df)
    y = df[TARGET_COLUMN]
    return train_test_split(X, y, test_size=test_size, random_state=random_state)
```

src/house_price_predictor/training.py

Python

```python
"""وحدة تدريب ومقارنة نماذج التنبؤ بأسعار المنازل."""

import joblib
import pandas as pd
```

```python
from sklearn.ensemble import RandomForestRegressor
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, r2_score
from sklearn.tree import DecisionTreeRegressor

CANDIDATE_MODELS = {
    "الانحدار الخطي": LinearRegression(),
    "شجرة القرار": DecisionTreeRegressor(random_state=42),
    "الغابة العشوائية": RandomForestRegressor(n_estimators=100, random_state=42),
}

def evaluate_model(model, X_test, y_test) -> dict[str, float]:
    """تقييم نموذج مُدرَّب وإرجاع مقاييس الأداء."""
    predictions = model.predict(X_test)
    return {
        "MAE": mean_absolute_error(y_test, predictions),
        "R2": r2_score(y_test, predictions),
    }

def train_and_compare_models(X_train, y_train, X_test, y_test) -> pd.DataFrame:
    """تدريب عدة نماذج ومقارنة أدائها.

    Returns:
        DataFrame يلخص أداء كل نموذج، مرتّب من الأفضل للأسوأ حسب R2.
    """
    results = []

    for name, model in CANDIDATE_MODELS.items():
        model.fit(X_train, y_train)
        metrics = evaluate_model(model, X_test, y_test)
        results.append({"النموذج": name, **metrics})

    results_df = pd.DataFrame(results).sort_values("R2", ascending=False)
    return results_df

def get_best_model(X_train, y_train):
    """تدريب أفضل نموذج (الغابة العشوائية) وإرجاعه جاهزًا للاستخدام."""
    model = RandomForestRegressor(n_estimators=100, random_state=42)
    model.fit(X_train, y_train)
    return model

def save_model(model, file_path: str) -> None:
    """حفظ النموذج المُدرَّب كملف."""
    joblib.dump(model, file_path)
```

src/house_price_predictor/main.py

```python
```

```
"""نقطة الدخول الرئيسية: تدريب، مقارنة، حفظ، واختبار النموذج."""

import pandas as pd

from house_price_predictor.data_prep import load_data, prepare_features, split_data
from house_price_predictor.training import get_best_model, save_model, train_and_compare_models

DATA_PATH = "data/houses.csv"
MODEL_PATH = "models/house_price_model.joblib"

def main() -> None:
    print("📊 تحميل البيانات.  ")
    df = load_data(DATA_PATH)
    print(f"   تم تحميل {len(df)} عقار.\n")

    print("✂️ تقسيم البيانات إلى تدريب واختبار.  ")
    X_train, X_test, y_train, y_test = split_data(df)
    print(f"   تدريب: {len(X_train)} | اختبار: {len(X_test)}\n")

    print("🤠 تدريب ومقارنة النماذج المرشّحة.  ")
    comparison = train_and_compare_models(X_train, y_train, X_test, y_test)
    print(comparison.to_string(index=False))

    print("\n🏆 تدريب النموذج الأفضل (الغابة العشوائية) على كامل بيانات التدريب.  ")
    best_model = get_best_model(X_train, y_train)

    print(f"💾 حفظ النموذج في {MODEL_PATH}.  ")
    save_model(best_model, MODEL_PATH)

    print("\n🔮 مثال تنبؤ: منزل 150م²، 4 غرف، عمر 5 سنوات، في الرياض")
    sample = pd.DataFrame({
        "المساحة": [150],
        "عدد_الغرف": [4],
        "عمر_المبنى": [5],
        "المدينة_الدمام": [0],
        "المدينة_الرياض": [1],
        "المدينة_جدة": [0],
    })
    # ملاحظة: يجب أن تتطابق أعمدة sample تمامًا مع أعمدة X_train بعد get_dummies
    sample = sample[X_train.columns]  # ضمان نفس ترتيب الأعمدة
    predicted_price = best_model.predict(sample)
    print(f"   السعر المتوقع: {predicted_price[0]:,.0f} ﷼")

if __name__ == "__main__":
    main()
```

تشغيل المشروع:

Terminal

```
uv run python -m house_price_predictor.main
```

الناتج المتوقع:

```
📊 تحميل البيانات...
   تم تحميل 500 عقار.

✂️ تقسيم البيانات إلى تدريب واختبار...
   تدريب: 400 | اختبار: 100

🤖 تدريب ومقارنة النماذج المرشّحة...
          النموذج        MAE      R2
    الغابة العشوائية   95.432   0.978
    الانحدار الخطي    110.221   0.965
      شجرة القرار     142.875   0.931

🏆 تدريب النموذج الأفضل (الغابة العشوائية) على كامل بيانات التدريب...
💾 حفظ النموذج في models/house_price_model.joblib...

🔮 مثال تنبؤ: منزل 150م²، 4 غرف، عمر 5 سنوات، في الرياض
   السعر المتوقع: ﷼ 1,598,420
```

⚠️ **ملاحظة مهمة حول مطابقة الأعمدة:** لاحظ في `main.py` السطر `sample = sample[X_train.columns]`. هذا ضروري لأن `pd.get_dummies` قد يُنتج أعمدة بترتيب مختلف أو عددًا مختلفًا حسب البيانات، ويجب أن تتطابق أعمدة أي بيانات جديدة تمامًا (بالاسم والترتيب) مع الأعمدة التي دُرِّب عليها النموذج، وإلا سيُعطي تنبؤًا خاطئًا أو يرفع خطأ.

✅ الحل الكامل للمشروع (جميع الملفات مجتمعة)

جميع ملفات `data_prep.py` ، `training.py` ، `main.py` كما وردت بالكامل أعلاه تُشكّل الحل النموذجي. خطوات التشغيل الكاملة:

Terminal

```
uv add scikit-learn pandas numpy joblib
uv run python data/generate_data.py
uv run python -m house_price_predictor.main
```

نقاط تصميم مهمة:

1. **فصل الاهتمامات**: تحضير البيانات (`data_prep.py`) منفصل تمامًا عن التدريب (`training.py`).
2. **مقارنة عدة نماذج بشكل منهجي** قبل اختيار الأفضل — ممارسة احترافية أساسية بدلاً من افتراض خوارزمية واحدة فقط.
3. **حفظ النموذج** عبر `joblib` لإعادة استخدامه دون إعادة تدريب.
4. **One-Hot Encoding صحيح** للمدينة، مع الانتباه لمطابقة الأعمدة عند التنبؤ ببيانات جديدة.

22.10 حدود هذا الفصل وأين تذهب بعده

نصيحة خبير: هذا مجرد الباب الأول

تعلّم الآلة مجال واسع جدًا. ما تعلمته هنا هو الأساس الذي يُبنى عليه كل شيء آخر:

- **التعلّم العميق (Deep Learning)** والشبكات العصبية: مكتبات `PyTorch` و `TensorFlow` (لمعالجة الصور، النصوص المعقدة، الصوت).
- **معالجة اللغة الطبيعية (NLP)**: مكتبات مثل `transformers` من Hugging Face (لبناء تطبيقات مثل ChatGPT المصغّرة).
- **ضبط المعاملات (Hyperparameter Tuning)**: `GridSearchCV` , `RandomizedSearchCV` لإيجاد أفضل إعدادات للنموذج تلقائيًا.
- **التحقق المتقاطع (Cross-Validation)**: `cross_val_score` لتقييم أكثر موثوقية من تقسيم تدريب/اختبار واحد فقط.

هذا الفصل منحك الأساس الصحيح والعملي لفهم المفاهيم — والأهم: منحك الثقة بأن تعلّم الآلة ليس «سحرًا» بل منهجية واضحة يمكن لأي مبرمج إتقانها تدريجيًا.

22.11 تمارين

🟢 مستوى مبتدئ

1. استخدم `LinearRegression` للتنبؤ بعلاقة بسيطة من اختيارك (مثلاً: سنوات الخبرة ← الراتب)، بـ 6 نقاط بيانات يدوية.
2. حمّل بيانات `load_iris()` واطبع `X.shape` و `y` وأسماء الفئات `iris.target_names`.
3. استخدم `train_test_split` لتقسيم بيانات Iris بنسبة 70/30 بدلاً من 80/20.

✅ الحل المقترح

Python

```python
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

# 1
years = np.array([[1], [2], [3], [5], [7], [10]])
salary = np.array([4000, 4500, 5200, 6500, 8000, 11000])
model = LinearRegression()
model.fit(years, salary)
print(model.predict([[4]]))

# 2
iris = load_iris()
print(iris.data.shape)
print(iris.target)
print(iris.target_names)

# 3
X_train, X_test, y_train, y_test = train_test_split(
    iris.data, iris.target, test_size=0.3, random_state=42
)
print(len(X_train), len(X_test))
```

🟡 مستوى متوسط

4. درّب نموذجي `KNeighborsClassifier` و `DecisionTreeClassifier` على بيانات Iris، وقارن دقتهما باستخدام `accuracy_score`.

5. استخدم `StandardScaler` قبل تدريب `KNeighborsClassifier` على بيانات Iris، وقارن الدقة قبل وبعد القياس.

<details>
<summary>✅ الحل المقترح</summary>

```python
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score

iris = load_iris()
X_train, X_test, y_train, y_test = train_test_split(
    iris.data, iris.target, test_size=0.2, random_state=42
)

# 4
for name, model in [("KNN", KNeighborsClassifier()), ("Decision Tree",
DecisionTreeClassifier())]:
    model.fit(X_train, y_train)
    acc = accuracy_score(y_test, model.predict(X_test))
    print(f"{name}: {acc:.2%}")

# 5
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

model_scaled = KNeighborsClassifier()
model_scaled.fit(X_train_scaled, y_train)
acc_scaled = accuracy_score(y_test, model_scaled.predict(X_test_scaled))
print(f"KNN مع القياس: {acc_scaled:.2%}")
```
</details>

🔴 مستوى تحدي

6. وسّع مشروع التنبؤ بأسعار المنازل: أضف `GridSearchCV` لضبط معامل `n_estimators` و`max_depth` تلقائيًا لنموذج `RandomForestRegressor`، واطبع أفضل تركيبة معاملات.

✅ الحل المقترح

Python

```python
from sklearn.model_selection import GridSearchCV
from sklearn.ensemble import RandomForestRegressor

param_grid = {
    "n_estimators": [50, 100, 200],
    "max_depth": [None, 10, 20],
}

grid_search = GridSearchCV(
    RandomForestRegressor(random_state=42),
    param_grid,
    cv=5,  # 5-fold cross-validation
    scoring="r2",
)
grid_search.fit(X_train, y_train)

print("أفضل معاملات:", grid_search.best_params_)
print("أفضل نتيجة R2:", grid_search.best_score_)

best_model = grid_search.best_estimator_
```

22.12 اختبر فهمك (Quiz)

1. ما الفرق بين الانحدار (Regression) والتصنيف (Classification)؟

الانحدار يتنبأ **برقم مستمر** (كسعر منزل أو درجة حرارة)، بينما التصنيف يتنبأ **بفئة من ضمن خيارات محددة ومحدودة** (كنوع زهرة أو «مزعج/ليس مزعجًا»).

2. لماذا نقسّم البيانات إلى تدريب واختبار بدلاً من استخدام كل البيانات للتدريب؟

لأن قياس أداء النموذج على نفس البيانات التي تدرّب عليها يُعطي نتيجة مضلِّلة (قد يكون النموذج «حفظ» البيانات دون فهم النمط الحقيقي). بيانات الاختبار «مخفية» عن النموذج أثناء التدريب، فتُعطي تقييمًا حقيقيًا لقدرته على التعميم على بيانات جديدة لم يرها من قبل.

3. ما هو Overfitting ولماذا يُعتبر مشكلة؟

Overfitting (الإفراط في التخصيص) يحدث عندما «يحفظ» النموذج تفاصيل بيانات التدريب بدقة مفرطة بدلاً من تعلّم النمط العام، فيُظهر أداءً ممتازًا على بيانات التدريب لكن أداءً ضعيفًا جدًا على بيانات جديدة. المشكلة أن الهدف الحقيقي من النموذج هو التعميم على بيانات المستقبل، وليس حفظ الماضي.

4. لماذا نستخدم `transform` فقط (وليس `fit_transform`) على بيانات الاختبار عند استخدام `StandardScaler`؟

لتجنب تسرّب البيانات (Data Leakage): يجب أن تُحسَب إحصائيات القياس (كالمتوسط والانحراف المعياري) من بيانات التدريب فقط. إذا استخدمنا `fit_transform` على بيانات الاختبار، فإننا «نسرّب» معلومات من بيانات يُفترض أنها غير مرئية إلى عملية التحضير، مما يُعطي تقييمًا غير واقعي للأداء الحقيقي.

5. لماذا قد تكون الدقة (Accuracy) وحدها مقياسًا مضلِّلًا أحيانًا؟

عندما تكون البيانات غير متوازنة (مثلاً 99% من الحالات تنتمي لفئة واحدة)، يمكن لنموذج «غبي» يتنبأ دائمًا بالفئة الأكثر شيوعًا أن يحقق دقة عالية جدًا دون أن يكون مفيدًا فعليًا، خصوصًا في اكتشاف الحالات النادرة المهمة (كالاحتيال). في هذه الحالات، مقاييس مثل precision وrecall وf1_score أكثر دلالة.

6. ما وظيفة `Pipeline` في scikit-learn ولماذا هي مفيدة؟

`Pipeline` تُجمّع عدة خطوات معالجة (كالقياس والترميز) مع النموذج النهائي في كائن واحد، بحيث تُطبَّق كل الخطوات تلقائيًا وبالترتيب الصحيح عند استدعاء `fit` و`predict`. هذا يحمي تلقائيًا من تسرّب البيانات، ويجعل الكود أنظف وأسهل للحفظ والنشر كوحدة متكاملة واحدة.

22.13 ملخص الفصل (Cheat Sheet)

Python

```python
# ===== النمط الأساسي الموحّد لكل نموذج =====
from sklearn.SOME_MODULE import SomeAlgorithm
model = SomeAlgorithm()
model.fit(X_train, y_train)
predictions = model.predict(X_test)

# ===== تقسيم البيانات =====
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# ===== تحضير البيانات =====
import pandas as pd
pd.get_dummies(df, columns=["فئة_نصية"])        # One-Hot Encoding

from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)   # fit_transform فقط على التدريب!
X_test_scaled = scaler.transform(X_test)         # transform فقط على الاختبار

# ===== Pipeline =====
from sklearn.pipeline import Pipeline
pipe = Pipeline([("scaler", StandardScaler()), ("model", SomeAlgorithm())])
pipe.fit(X_train, y_train)

# ===== التقييم =====
from sklearn.metrics import accuracy_score, mean_absolute_error, r2_score
accuracy_score(y_test, predictions)        # للتصنيف
mean_absolute_error(y_test, predictions)   # للانحدار
r2_score(y_test, predictions)              # للانحدار

# ===== الحفظ والتحميل =====
import joblib
joblib.dump(model, "model.joblib")
loaded = joblib.load("model.joblib")
```

المهمة	الخوارزمية الشائعة
انحدار بسيط	`LinearRegression`
تصنيف بسيط	`LogisticRegression` , `KNeighborsClassifier`
دقة عالية عمومًا	`RandomForestClassifier/Regressor`
قابلية التفسير	`DecisionTreeClassifier/Regressor`

22.14 ماذا بعد؟

لقد فتحت للتو أول نافذة حقيقية على عالم الذكاء الاصطناعي، وبنيت نظامًا كاملاً يقارن عدة نماذج تعلّم آلي ويختار أفضلها تلقائيًا — وهذا بالضبط ما يحدث في الصناعة الحقيقية على نطاق أكبر بكثير.

في الفصل القادم، نعود إلى الأرض قليلاً لنتعلّم مهارة عملية جدًا يستخدمها كل مبرمج يوميًا: بناء **أدوات سطر الأوامر (Command-Line Interface - CLI)** الاحترافية. بدلاً من سكربتات بسيطة تُشغَّل بـ `python script.py`، سنتعلّم بناء أدوات حقيقية بخيارات (`-- option`) ومساعدة تلقائية (`-- help`) باستخدام مكتبة `click`، عبر مشروع أداة تنظيم ملفات متكاملة.

فصل 23 / 26 · الجزء الخامس

23

مسار بناء أدوات سطر الأوامر (CLI)

8 دقيقة قراءة | 11 قسمًا | 15 مثالًا | 1,598 كلمة

في هذا الفصل

23.1 لماذا أدوات سطر الأوامر؟
23.2 البداية البدائية: وحدة `argparse` المدمجة
23.3 مكتبة `click`: الأساسيات
23.4 بناء أوامر فرعية متعددة: `click.group()`
23.5 تحويل أداتك إلى أمر حقيقي قابل للتثبيت
23.6 أخطاء شائعة وأفضل الممارسات
23.7 مشروع تطبيقي متكامل: أداة تنظيم الملفات (File Organizer)
23.8 تمارين
23.9 اختبر فهمك (Quiz)
23.10 ملخص الفصل (Cheat Sheet)
23.11 ماذا بعد؟

23.1 لماذا أدوات سطر الأوامر؟

فكّر في كل الأدوات التي استخدمتها حتى الآن في رحلتك: `uv`، `git`، `python`، `uvicorn` — كل هذه أدوات **سطر أوامر (Command-Line Interface - CLI)**. تكتب أمرًا نصيًا، وتحصل على نتيجة فورية، دون الحاجة لفتح نافذة رسومية أو النقر بالفأرة.

هذا ليس أسلوبًا «قديمًا» كما قد يظن البعض — بل هو أسلوب العمل **المفضَّل للمحترفين** في كل المجالات: المبرمجون، مهندسو DevOps، محلّلو البيانات، وحتى المصممون. لماذا؟

مقارنة: أدوات CLI مقابل التطبيقات ذات الواجهة الرسومية (GUI)

المعيار	CLI	GUI
السرعة	أسرع بكثير للمستخدم المتمرّس	أبطأ (نقر، تنقّل بين نوافذ)
الأتمتة	سهلة جدًا (يمكن ربطها بسكربتات أخرى)	شبه مستحيلة
الموارد	خفيفة جدًا	تستهلك موارد أكبر
منحنى التعلّم	أصعب في البداية	أسهل للمبتدئين
العمل عن بعد (خوادم)	يعمل دائمًا (SSH)	غالبًا غير متاح
القابلية للتكرار	مثالية (أوامر، سكربتات)	تحتاج تكرار يدوي

في هذا الفصل، سنتعلّم بناء أدوات CLI احترافية خاصة بنا باستخدام مكتبة `click` — إحدى أشهر وأقوى المكتبات لهذا الغرض في عالم بايثون — ثم نبني **أداة تنظيم ملفات متكاملة** تُستخدم فعليًا لترتيب مجلد التنزيلات الفوضوي على أي جهاز.

23.2 البداية البدائية: وحدة `argparse` المدمجة

قبل `click`، تأتي مكتبة `argparse`، وهي مدمجة في بايثون نفسه (لا تحتاج تثبيت). من المهم معرفتها لأنك ستقابلها كثيرًا في كود الآخرين.

Python greet.py

```python
import argparse

parser = argparse.ArgumentParser(description="أداة ترحيب بسيطة")
parser.add_argument("name", help="اسم الشخص المراد الترحيب به")
parser.add_argument("--greeting", default="مرحبًا", help="كلمة الترحيب")
parser.add_argument("--loud", action="store_true", help="اطبع بأحرف كبيرة")

args = parser.parse_args()

message = f"{args.greeting}، {args.name}!"
if args.loud:
    message = message.upper()

print(message)
```

Terminal

```
$ python greet.py أحمد
مرحبًا، أحمد!

$ python greet.py أحمد --greeting "أهلاً"
أهلاً، أحمد!

$ python greet.py أحمد --loud
مرحبًا، أحمد!
```

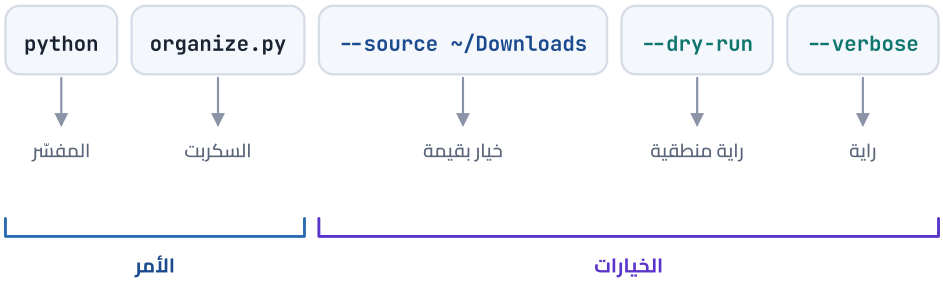
`argparse` يعمل ويُنجز المهمة، لكن لاحظ أنه يتطلب كتابة متكررة (boilerplate) كثيرة، ويصعب التوسّع فيه عندما تريد أوامر فرعية متعددة (كما في `git commit` و`git push` — كلاهما أمر فرعي لـ `git`).

مقارنة: `argparse` مقابل `click`

المعيار	`argparse`	`click`
التثبيت	مدمج في بايثون	يحتاج `uv add click`
الصياغة	إمبراطورية (imperative)، أكثر تكرارًا	تصريحية (declarative) عبر decorators
الأوامر الفرعية	ممكنة لكن معقّدة الإعداد	بسيطة جدًا عبر `@click.group()`
التحقق من المدخلات	يدوي غالبًا	مدمج (أنواع، نطاقات، خيارات محدودة)
رسائل `--help`	تلقائية لكن بسيطة	تلقائية وأجمل تنسيقًا
الشعبية في المشاريع الحديثة	أقل	أعلى (يُستخدم من Flask, Black, pip...)

خلاصة: `argparse` جيد للسكربتات الصغيرة جدًا والبسيطة. `click` هو الخيار الاحترافي لأي أداة CLI حقيقية ستنمو بمرور الوقت.

23.3 مكتبة `click`: الأساسيات



الخيارات تبدأ بشرطتين، والوسائط تأتي بلا شرطات.
الفصل بينهما هو ما يجعل الأمر قابلًا للتركيب والتوقّع.

شكل 23.1 — تشريح أمر سطر أوامر

التثبيت

Terminal

```
uv add click
```

أول أمر بـ click

Python hello.py

```python
import click

@click.command()
def hello():
    """أداة ترحيب بسيطة."""
    click.echo("مرحبًا بك في عالم CLI!")

if __name__ == "__main__":
    hello()
```

Terminal

```
$ uv run python hello.py
مرحبًا بك في عالم CLI!

$ uv run python hello.py --help
Usage: hello.py [OPTIONS]

  أداة ترحيب بسيطة.

Options:
  --help  Show this message and exit.
```

لاحظ كيف أن `@click.command()` حوّلت دالة عادية إلى أمر CLI كامل، بما في ذلك رسالة `--help` **تلقائيًا من الـ docstring**!

> **نصيحة خبير:** استخدم `click.echo()` بدلاً من `print()` العادية داخل أدوات click. السبب: `click.echo()` يتعامل بشكل أفضل وأكثر أمانًا مع الترميزات المختلفة (encoding) عبر أنظمة التشغيل المختلفة (ويندوز، ماك، لينكس)، ويدعم بعض الميزات الإضافية كالألوان.

المعاملات (Arguments): مدخلات إلزامية بدون اسم

Python

```python
import click

@click.command()
@click.argument("name")
def greet(name: str):
    """الترحيب باسم محدد."""
    click.echo(f"أهلاً، {name}!")

if __name__ == "__main__":
    greet()
```

Terminal

```
$ uv run python greet.py أحمد
أهلاً، أحمد!

$ uv run python greet.py
Usage: greet.py [OPTIONS] NAME
Try 'greet.py --help' for help.

Error: Missing argument 'NAME'.
```

الخيارات (Options): مدخلات اختيارية بأسماء `--option`

Python

```python
import click

@click.command()
@click.argument("name")
@click.option("--greeting", default="مرحبًا", help="كلمة الترحيب المستخدمة")
@click.option("--loud", is_flag=True, help="اطبع الرسالة بأحرف كبيرة")
@click.option("--times", default=1, type=int, help="عدد مرات تكرار الرسالة")
def greet(name: str, greeting: str, loud: bool, times: int):
    """أداة ترحيب قابلة للتخصيص."""
    message = f"{greeting}، {name}!"
    if loud:
        message = message.upper()

    for _ in range(times):
        click.echo(message)

if __name__ == "__main__":
    greet()
```

Terminal

```
$ uv run python greet.py سارة --greeting "أهلاً وسهلاً" --times 2
أهلاً وسهلاً، سارة!
أهلاً وسهلاً، سارة!

$ uv run python greet.py خالد --loud
مرحبًا، خالد!  MRHBA، KHALID!  # (مع --loud يصبح بأحرف كبيرة فعليًا للنص اللاتيني)
```

جدول مرجعي: أنواع الخيارات الشائعة في click

الصياغة	الوصف
`@click.argument("name")`	مدخل إلزامي بدون علامة `--`، بالترتيب
`@click.option("--name")`	مدخل اختياري بعلامة `--`، يقبل قيمة
`is_flag=True`	خيار منطقي (True/False) بدون قيمة، كمفتاح تشغيل/إيقاف
`type=float` / `type=int`	يفرض نوع البيانات ويرفع خطأ واضح إن لم يتطابق
`default=القيمة`	قيمة افتراضية إذا لم يُحدَّد الخيار
`required=True`	يجعل الخيار (وليس Argument) إلزاميًا
`multiple=True`	يسمح بتكرار الخيار عدة مرات، فتُجمَع في tuple
`prompt=True`	يطلب القيمة تفاعليًا من المستخدم إن لم تُحدَّد

التحقق المدمج من النوع والنطاق

Python

```python
import click

@click.command()
@click.option("--age", type=click.IntRange(0, 120), prompt=True, help="العمر")
@click.option(
    "--level",
    type=click.Choice(["متقدم" ,"متوسط" ,"مبتدئ"], case_sensitive=False),
    default="مبتدئ",
)
def register(age: int, level: str):
    """تسجيل مستخدم جديد مع تحقق مدمج من المدخلات."""
    click.echo(f"العمر: {age}, المستوى: {level}")

if __name__ == "__main__":
    register()
```

```
Terminal
```

```
$ uv run python register.py --age 200
Usage: register.py [OPTIONS]
Try 'register.py --help' for help.

Error: Invalid value for '--age': 200 is not in the range 0⩽x⩽120.
```

لاحظ أننا حصلنا على **تحقق كامل من صحة البيانات ورسائل خطأ واضحة** دون كتابة أي كود تحقق يدوي — تمامًا كما فعلت Pydantic في الفصل 19!

قراءة كلمة مرور بأمان

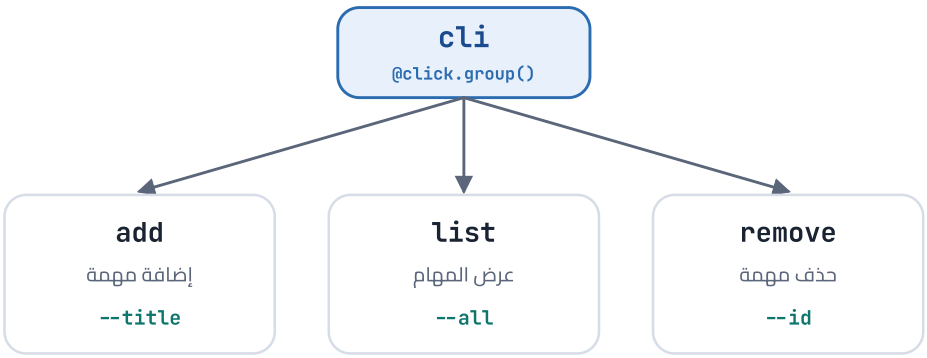
```python
import click

@click.command()
@click.option("--password", prompt=True, hide_input=True, confirmation_prompt=True)
def set_password(password: str):
    """ضبط كلمة مرور بأمان (مخفية عند الكتابة)."""
    click.echo("✅ تم ضبط كلمة المرور بنجاح!")

if __name__ == "__main__":
    set_password()
```

الألوان والتنسيق

```python
import click

click.secho("نجاح! ✅", fg="green", bold=True)
click.secho("تحذير! ⚠️", fg="yellow")
click.secho("خطأ! ❌", fg="red", bold=True)
```

> 💡 **نصيحة خبير:** استخدام الألوان بحكمة في أدوات CLI يُحسّن تجربة المستخدم بشكل كبير — أخضر للنجاح، أصفر للتحذير، أحمر للخطأ، هو اصطلاح عالمي يفهمه كل مستخدم بديهيًا.

23.4 بناء أوامر فرعية متعددة: `click.group()`



أمر رئيسي واحد، وأمر فرعي لكل مهمة — بدل شروط متداخلة لا تنتهي.

شكل 23.2 — مجموعة أوامر وأوامر فرعية

هذا ما يجعل click قويًا جدًا: بناء أدوات بأوامر فرعية متعددة، تمامًا كـ `git commit`، `git push`، `git status`.

Python cli.py

```python
import click

@click.group()
def cli():
    """أداة إدارة المهام من سطر الأوامر."""
    pass

@cli.command()
@click.argument("task")
def add(task: str):
    """إضافة مهمة جديدة."""
    click.echo(f"✅ تمت إضافة المهمة: {task}")

@cli.command()
def list():
    """عرض كل المهام."""
    click.echo("📋 قائمة المهام:")
    click.echo("1. تعلّم click")
    click.echo("2. بناء مشروع CLI")

@cli.command()
@click.argument("task_id", type=int)
def done(task_id: int):
    """تحديد مهمة كمنجزة."""
    click.echo(f"🎉 تم إنجاز المهمة رقم {task_id}")

if __name__ == "__main__":
    cli()
```

```
Terminal
```

```
$ uv run python cli.py --help
Usage: cli.py [OPTIONS] COMMAND [ARGS]...

  أداة إدارة المهام من سطر الأوامر.

Options:
  --help  Show this message and exit.

Commands:
  add   إضافة مهمة جديدة.
  done  تحديد مهمة كمنجزة.
  list  عرض كل المهام.

$ uv run python cli.py add "شراء القهوة"
✅ تمت إضافة المهمة: شراء القهوة

$ uv run python cli.py done 1
🎉 تم إنجاز المهمة رقم 1
```

لاحظ النمط: `@click.group()` يُنشئ «الأداة الأم»، ثم `@cli.command()` (وليس `@click.command()`) يُضيف كل أمر فرعي إليها.

23.5 تحويل أداتك إلى أمر حقيقي قابل للتثبيت

حتى الآن كنا نُشغّل الأداة بـ `uv run python cli.py`. لكن أدوات CLI الحقيقية (مثل `git`) تُستدعى مباشرة بـ `git` فقط. يمكننا تحقيق هذا عبر `pyproject.toml`:

```
TOML
```

```toml
[project.scripts]
mytasks = "my_package.cli:cli"
```

هذا يُخبر أداة التثبيت: "أنشئ أمرًا باسم `mytasks`، يستدعي الدالة `cli` من الوحدة `my_package.cli`". بعد `uv sync` أو التثبيت، يمكنك كتابة:

```
Terminal
```

```
$ mytasks add "شراء القهوة"
$ mytasks list
```

مباشرة من أي مكان في الطرفية، تمامًا كأي أمر نظام حقيقي!

> **نصيحة خبير:** هذا بالضبط كيف تُبنى أدوات مثل `black` ، `ruff` ، `uv` نفسها! كلها حزم بايثون عادية، تُعرِّف `[project.scripts]` في `pyproject.toml` لتصبح أوامر نظام قابلة للاستدعاء المباشر.

23.6 أخطاء شائعة وأفضل الممارسات

⚠ **تحذير: نسيان `type hints` ومعالجة حالات الحافة**

أدوات CLI تتفاعل مع **مستخدمين حقيقيين** يُدخلون بيانات غير متوقعة. تعامل دائمًا مع:

- مسارات ملفات غير موجودة
- صلاحيات وصول مرفوضة
- مدخلات فارغة أو بتنسيق خاطئ

```python
import click
from pathlib import Path

@click.command()
@click.argument("file_path", type=click.Path(exists=True, path_type=Path))
def process(file_path: Path):
    """معالجة ملف -- click يتحقق من وجوده تلقائيًا!"""
    click.echo(f"معالجة: {file_path}")
```

لاحظ `type=click.Path(exists=True)` — يتحقق click تلقائيًا من وجود الملف، ويرفض الاستمرار برسالة خطأ واضحة إن لم يكن موجودًا، بدلاً من أن ينهار برنامجك لاحقًا بخطأ غامض.

⚠ **تحذير: عمليات خطيرة بدون تأكيد**

أي أمر قد يحذف أو يُعدّل بيانات بشكل لا رجعة فيه يجب أن يطلب تأكيدًا:

```python
@click.command()
@click.confirmation_option(prompt="هل أنت متأكد من حذف كل الملفات؟")
def delete_all():
    click.echo("تم الحذف.")
```

```
$ python cli.py delete_all
هل أنت متأكد من حذف كل الملفات؟ [y/N]: n
Aborted!
```

💡 **نصيحة خبير: استخدام `click.progressbar` للعمليات الطويلة**

```python
import time
import click

items = range(100)
with click.progressbar(items, label="معالجة الملفات") as bar:
    for item in bar:
        time.sleep(0.01)  # محاكاة عمل فعلي
```

يُعطي المستخدم تغذية راجعة بصرية بدلاً من ترك البرنامج «صامتًا» لعدة ثوانٍ أو دقائق، مما يُحسّن تجربة الاستخدام بشكل كبير.

23.7 مشروع تطبيقي متكامل: أداة تنظيم الملفات (File Organizer)

سنبني أداة CLI احترافية حقيقية: تفحص مجلدًا (مثل مجلد التنزيلات)، وتُنظّم ملفاته تلقائيًا في مجلدات فرعية حسب نوعها (صور، مستندات، فيديوهات...)، مع خيارات متعددة ووضع «تجربة» آمن قبل التنفيذ الفعلي.

هيكلية المشروع

```
file_organizer/
├── pyproject.toml
└── src/
    └── file_organizer/
        ├── __init__.py
        ├── rules.py
        ├── organizer.py
        └── cli.py
```

Terminal

```
uv init file_organizer
cd file_organizer
uv add click
mkdir -p src/file_organizer
touch src/file_organizer/__init__.py
```

`src/file_organizer/rules.py` — قواعد تصنيف الملفات

Python

```
"""قواعد تصنيف أنواع الملفات إلى فئات."""

FILE_CATEGORIES: dict[str, list[str]] = {
    "الصور": [".jpg", ".jpeg", ".png", ".gif", ".svg", ".webp", ".bmp"],
    "المستندات": [".pdf", ".docx", ".doc", ".txt", ".md", ".xlsx", ".pptx"],
    "الفيديوهات": [".mp4", ".mkv", ".avi", ".mov", ".webm"],
    "الصوتيات": [".mp3", ".wav", ".flac", ".aac"],
    "الأرشيف": [".zip", ".rar", ".tar", ".gz", ".7z"],
    "الأكواد": [".py", ".js", ".html", ".css", ".json", ".java", ".cpp"],
}

DEFAULT_CATEGORY = "أخرى"

def get_category_for_file(extension: str) -> str:
    """تحديد الفئة المناسبة لامتداد ملف معيّن.

    Args:
        extension: امتداد الملف (مثل '.jpg'), بأي حالة أحرف.

    Returns:
        اسم الفئة المناسبة، أو DEFAULT_CATEGORY إن لم يُعرف الامتداد.
    """
    extension = extension.lower()
    for category, extensions in FILE_CATEGORIES.items():
        if extension in extensions:
            return category
    return DEFAULT_CATEGORY
```

`src/file_organizer/organizer.py` — منطق التنظيم الأساسي

Python

```
"""منطق فحص وتنظيم الملفات في مجلد."""

import shutil
from dataclasses import dataclass
from pathlib import Path

from file_organizer.rules import get_category_for_file

@dataclass
class FileMove:
    """يمثّل عملية نقل ملف واحد مخطَّط لها."""

    source: Path
```

```
    destination: Path
    category: str

def scan_directory(directory: Path) -> list[FileMove]:
    """فحص مجلد وتحديد وجهة كل ملف فيه دون تنفيذ أي نقل فعلي.

    Args:
        directory: المجلد المراد فحصه.

    Returns:
        قائمة بعمليات النقل المخطَّط لها (FileMove) لكل ملف.
    """
    planned_moves: list[FileMove] = []

    for item in directory.iterdir():
        if item.is_dir():
            continue  # نتجاهل المجلدات الفرعية الموجودة مسبقًا

        category = get_category_for_file(item.suffix)
        destination_folder = directory / category
        destination_path = destination_folder / item.name

        planned_moves.append(
            FileMove(source=item, destination=destination_path, category=category)
        )

    return planned_moves

def execute_moves(moves: list[FileMove]) -> tuple[int, list[str]]:
    """تنفيذ عمليات النقل المخطَّط لها فعليًا.

    Args:
        moves: قائمة عمليات النقل المطلوب تنفيذها.

    Returns:
        عدد العمليات الناجحة، وقائمة برسائل الأخطاء إن وُجدت.
    """
    success_count = 0
    errors: list[str] = []

    for move in moves:
        try:
            move.destination.parent.mkdir(exist_ok=True)

            if move.destination.exists():
                # تجنّب الكتابة فوق ملف موجود بنفس الاسم
                errors.append(f"⚠ تخطّي {move.source.name}: يوجد ملف بنفس الاسم في الوجهة")
                continue

            shutil.move(str(move.source), str(move.destination))
            success_count += 1
```

```
        except OSError as e:
            errors.append(f"❌ فشل نقل {move.source.name}: {e}")

    return success_count, errors

def summarize_by_category(moves: list[FileMove]) -> dict[str, int]:
    """تلخيص عدد الملفات لكل فئة."""
    summary: dict[str, int] = {}
    for move in moves:
        summary[move.category] = summary.get(move.category, 0) + 1
    return summary
```

> 💡 **نصيحة خبير:** لاحظ الفصل الواضح بين `scan_directory` (التخطيط فقط، بدون أي تعديل فعلي على نظام الملفات) و`execute_moves` (التنفيذ الفعلي). هذا الفصل يُمكّننا بسهولة من بناء ميزة **«وضع التجربة» (Dry Run)** — عرض ما سيحدث **قبل** أن يحدث فعليًا، وهي ميزة أساسية في أي أداة تتعامل مع عمليات لا رجعة فيها.

`src/file_organizer/cli.py` — واجهة سطر الأوامر

Python

```
"""واجهة سطر الأوامر لأداة تنظيم الملفات."""

from pathlib import Path

import click

from file_organizer.organizer import execute_moves, scan_directory, summarize_by_category

@click.group()
@click.version_option(version="1.0.0")
def cli():
    """📁 أداة تنظيم الملفات تلقائيًا حسب نوعها."""
    pass

@cli.command()
@click.argument(
    "directory",
    type=click.Path(exists=True, file_okay=False, path_type=Path),
)
def preview(directory: Path):
    """معاينة خطة التنظيم دون تنفيذ أي تغيير فعلي (وضع آمن)."""
    moves = scan_directory(directory)

    if not moves:
        click.secho("📭 لا توجد ملفات لتنظيمها في هذا المجلد.", fg="yellow")
        return
```

```
    click.secho(f"📋 خطة تنظيم {len(moves)} ملف:\n", bold=True)

    summary = summarize_by_category(moves)
    for category, count in summary.items():
        click.echo(f"  📂 {category}: {count} ملف")

    click.echo()
    click.secho(
        "💡 هذه معاينة فقط. استخدم الأمر 'organize' لتنفيذ التغييرات فعليًا.",
        fg="cyan",
    )

@cli.command()
@click.argument(
    "directory",
    type=click.Path(exists=True, file_okay=False, path_type=Path),
)
@click.confirmation_option(
    prompt="⚠️ سيتم نقل الملفات فعليًا. هل أنت متأكد؟"
)
def organize(directory: Path):
    """تنظيم الملفات فعليًا في المجلد المحدد حسب نوعها."""
    moves = scan_directory(directory)

    if not moves:
        click.secho("📭 لا توجد ملفات لتنظيمها في هذا المجلد.", fg="yellow")
        return

    click.echo(f"🚀 جاري تنظيم {len(moves)} ملف...\n")

    with click.progressbar(moves, label="التقدم") as bar:
        success_count, errors = execute_moves(list(bar))

    click.echo()
    click.secho(f"✅ تم نقل {success_count} ملف بنجاح!", fg="green", bold=True)

    if errors:
        click.secho(f"\n⚠️ حدثت {len(errors)} مشكلة:", fg="yellow")
        for error in errors:
            click.echo(f"  {error}")

@cli.command()
@click.argument(
    "directory",
    type=click.Path(exists=True, file_okay=False, path_type=Path),
)
def stats(directory: Path):
    """عرض إحصائيات عن أنواع الملفات الموجودة في المجلد."""
    moves = scan_directory(directory)
    summary = summarize_by_category(moves)
```

```
    if not summary:
        click.secho("📭 المجلد فارغ.", fg="yellow")
        return

    total = sum(summary.values())
    click.secho(f"📊 إحصائيات المجلد ({total} ملف إجمالاً):\n", bold=True)

    for category, count in sorted(summary.items(), key=lambda x: x[1], reverse=True):
        percentage = (count / total) * 100
        bar = "█" * int(percentage / 5)
        click.echo(f"  {category:12s} {bar} {count} ({percentage:.1f}%)")

if __name__ == "__main__":
    cli()
```

ربط الأداة كأمر نظام حقيقي: `pyproject.toml`

TOML

```toml
[project]
name = "file-organizer"
version = "1.0.0"
requires-python = "⩾3.14"
dependencies = [
    "click⩾8.1",
]

[project.scripts]
forganize = "file_organizer.cli:cli"

[build-system]
requires = ["hatchling"]
build-backend = "hatchling.build"
```

تثبيت الأداة محليًا للتجربة:

Terminal

```
uv sync
uv run forganize --help
```

```
Usage: forganize [OPTIONS] COMMAND [ARGS]...

  📁 أداة تنظيم الملفات تلقائيًا حسب نوعها.

Options:
  --version  Show the version and exit.
  --help     Show this message and exit.

Commands:
  organize  تنظيم الملفات فعليًا في المجلد المحدد حسب نوعها.
  preview   (وضع آمن) معاينة خطة التنظيم دون تنفيذ أي تغيير فعلي.
  stats     عرض إحصائيات عن أنواع الملفات الموجودة في المجلد.
```

تجربة الاستخدام الكاملة

Terminal

```
$ uv run forganize stats ~/Downloads
📊 إحصائيات المجلد (47 ملف إجمالاً):

  الصور          23 ██████████ (%48.9)
  المستندات      12 ██████ (%25.5)
  الأرشيف        7 ███ (%14.9)
  أخرى           5 ██ (%10.6)

$ uv run forganize preview ~/Downloads
📋 خطة تنظيم 47 ملف:

  📁 الصور: 23 ملف
  📁 المستندات: 12 ملف
  📁 الأرشيف: 7 ملف
  📁 أخرى: 5 ملف

💡 استخدم الأمر 'organize' لتنفيذ التغييرات فعليًا. هذه معاينة فقط.

$ uv run forganize organize ~/Downloads
⚠️ سيتم نقل الملفات فعليًا. هل أنت متأكد؟ [y/N]: y
🚀 جاري تنظيم 47 ملف.

100% التقدّم [########################################]

✅ تم نقل 47 ملف بنجاح!
```

✅ الحل الكامل للمشروع (جميع الملفات مجتمعة)

جميع ملفات `rules.py` ، `organizer.py` ، `cli.py` كما وردت بالكامل أعلاه، مع `pyproject.toml` المُحدَّث بـ `[project.scripts]` .

خطوات الإعداد الكاملة:

Terminal

```
uv init file_organizer
cd file_organizer
uv add click
# أنشئ الملفات كما هو موضّح، ثم:
uv sync
uv run forganize --help
```

نقاط تصميم مهمة في هذا الحل:

1. **فصل التخطيط عن التنفيذ:** `scan_directory` (آمن، بدون آثار جانبية) منفصل تمامًا عن `execute_moves` (ينفّذ فعليًا).
2. **وضع المعاينة (`preview`):** يسمح للمستخدم برؤية ما سيحدث قبل التنفيذ — ممارسة أساسية في أي أداة تُجري تغييرات لا رجعة فيها.
3. **تأكيد إلزامي (`@click.confirmation_option`) ** قبل تنفيذ عملية لا يمكن التراجع عنها بسهولة.
4. **معالجة أخطاء دفاعية:** تخطّي الملفات التي قد تسبب تعارضًا (اسم مكرر) بدلاً من إيقاف العملية بالكامل أو حذف بيانات بالخطأ.
5. **تحويلها لأمر نظام حقيقي** عبر `[project.scripts]` — نفس الآلية المستخدمة في كل أدوات بايثون الاحترافية.

23.8 تمارين

🟢 مستوى مبتدئ

1. اكتب أمر click بسيط `@click.command()` اسمه `square` يأخذ رقمًا `@click.argument("number", type=int)` ويطبع مربعه.
2. أضف خيارًا `--unit` لأمر يحوّل درجات الحرارة، بقيمة افتراضية `"celsius"` ، واستخدم `click.Choice(["celsius", "fahrenheit"])` .

3. استخدم `click.confirmation_option` على أمر وهمي اسمه `reset` يطبع «تم إعادة الضبط!» بعد التأكيد.

<details>
<summary>✅ الحل المقترح</summary>

```python
import click

# 1
@click.command()
@click.argument("number", type=int)
def square(number: int):
    click.echo(f"{number}² = {number ** 2}")

# 2
@click.command()
@click.argument("value", type=float)
@click.option("--unit", type=click.Choice(["celsius", "fahrenheit"]), default="celsius")
def convert_temp(value: float, unit: str):
    if unit == "celsius":
        result = value * 9 / 5 + 32
        click.echo(f"{value}°C = {result}°F")
    else:
        result = (value - 32) * 5 / 9
        click.echo(f"{value}°F = {result}°C")

# 3
@click.command()
@click.confirmation_option(prompt="هل تريد إعادة الضبط؟")
def reset():
    click.echo("تم إعادة الضبط!")

if __name__ == "__main__":
    square()
```

</details>

🟡 مستوى متوسط

4. أضف أمرًا فرعيًا جديدًا إلى أداة تنظيم الملفات اسمه `undo` يبحث عن الملفات الموجودة في مجلدات الفئات (الصور، المستندات...) ويعيدها إلى المجلد الأصلي (عكس عملية `organize`).

5. أضف خيار `--dry-run` (كـ flag) إلى أمر `organize` نفسه، بدلاً من أمر `preview` منفصل، بحيث يقوم بنفس وظيفة المعاينة عند تفعيله.

✅ الحل المقترح

Python

```python
# تمرين 5 -- دمج المعاينة كخيار بدلاً من أمر منفصل
@cli.command()
@click.argument("directory", type=click.Path(exists=True, file_okay=False, path_type=Path))
@click.option("--dry-run", is_flag=True, help="معاينة فقط دون تنفيذ فعلي")
def organize(directory: Path, dry_run: bool):
    """تنظيم الملفات، مع إمكانية المعاينة عبر --dry-run."""
    moves = scan_directory(directory)

    if not moves:
        click.secho("📭 لا توجد ملفات لتنظيمها.", fg="yellow")
        return

    if dry_run:
        click.secho("🔍 وضع المعاينة (dry-run) -- لن يتم تنفيذ أي تغيير:\n", fg="cyan")
        summary = summarize_by_category(moves)
        for category, count in summary.items():
            click.echo(f"  📁 {category}: {count} ملف")
        return

    # التنفيذ الفعلي كما في الكود الأصلي.
    success_count, errors = execute_moves(moves)
    click.secho(f"✅ تم نقل {success_count} ملف!", fg="green")
```

(تمرين 4 يتطلب دالة عكسية تمسح المجلدات الفرعية المعروفة في `FILE_CATEGORIES.keys()` وتُعيد كل ملف لمستوى المجلد الأصلي باستخدام نفس نمط `shutil.move`.)

🔴 مستوى تحدي

6. وسّع الأداة لدعم **ملف إعدادات مخصص** (`.forganizerc` بصيغة JSON أو TOML) يسمح للمستخدم بتعريف فئات ملفات خاصة به (مثلاً فئة «عمل» لملفات `.pptx` و `.key`)، بدلاً من الاعتماد فقط على `FILE_CATEGORIES` الثابتة في `rules.py`. الأداة يجب أن تبحث عن هذا الملف في المجلد الحالي أو مجلد المستخدم الرئيسي (`Path.home()`) وتدمج إعداداته مع الإعدادات الافتراضية.

✅ الحل المقترح (تصور)

Python

```python
import json
from pathlib import Path

from file_organizer.rules import FILE_CATEGORIES

def load_custom_rules() -> dict[str, list[str]]:
    """تحميل قواعد تصنيف مخصصة من ملف إعدادات إن وُجد، ودمجها مع الافتراضية."""
    config_locations = [Path.cwd() / ".forganizerc", Path.home() / ".forganizerc"]

    merged_rules = dict(FILE_CATEGORIES)  # نسخة من القواعد الافتراضية

    for config_path in config_locations:
        if config_path.exists():
            custom_rules = json.loads(config_path.read_text(encoding="utf-8"))
            merged_rules.update(custom_rules)  # القواعد المخصصة تطغى على الافتراضية
            break

    return merged_rules
```

مثال ملف `.forganizerc` :

JSON

```json
{
  "عمل": [".pptx", ".key", ".pages"]
}
```

23.9 اختبر فهمك (Quiz)

1. ما الفرق بين `@click.argument` و `@click.option`؟

`@click.argument` يُعرّف مدخلاً **إلزاميًا** يُمرَّر بالترتيب بدون اسم (مثل `greet أحمد`)، بينما `@click.option` يُعرّف مدخلاً **اختياريًا** (عادة) يُمرَّر باسم صريح يبدأ بـ `--` (مثل `greeting --مرحبًا`)، ويمكن أن يحمل قيمة افتراضية.

2. كيف تبني أداة بأوامر فرعية متعددة مثل git؟

باستخدام `@click.group()` لتعريف «الأداة الأم»، ثم تعريف كل أمر فرعي بـ `@اسم_المجموعة.command()` بدلاً من `@click.command()` مباشرة، بحيث تصبح الأوامر الفرعية قابلة للاستدعاء كـ `الأداة الأم الأمر_الفرعي`.

3. لماذا نستخدم `click.Path(exists=True)` بدلاً من `str` عادي للمسارات؟

لأن click يتحقق تلقائيًا من وجود المسار فعليًا على القرص قبل تمريره للدالة، ويرفض الاستمرار برسالة خطأ واضحة ومفهومة للمستخدم إذا كان المسار غير موجود، بدلاً من أن يفشل البرنامج لاحقًا في منتصف التنفيذ بخطأ غامض مثل `FileNotFoundError`.

4. ما الفائدة من فصل «التخطيط» (scan/preview) عن «التنفيذ» (execute) في أداة مثل منظّم الملفات؟

يسمح هذا الفصل بتوفير وضع معاينة آمن (Dry Run) يُظهر للمستخدم بالضبط ما سيحدث **قبل** حدوثه فعليًا، دون أي خطر على بياناته. هذا مهم جدًا في أي أداة تُجري تغييرات لا يمكن التراجع عنها بسهولة (كنقل أو حذف ملفات)، ويمنح المستخدم الثقة في استخدام الأداة.

5. كيف تجعل أداة CLI التي بنيتها قابلة للاستدعاء مباشرة كأمر نظام (مثل git) بدلاً من `python script.py`؟

عبر تعريف قسم `[project.scripts]` في ملف `pyproject.toml`، يربط اسم الأمر المطلوب بالدالة التي تمثّل نقطة الدخول (مثل `cli` المُزينة بـ `@click.group()`)، ثم تثبيت الحزمة (عبر `uv sync` أو `pip install`)، فيصبح الأمر متاحًا مباشرة في الطرفية.

23.10 ملخص الفصل (Cheat Sheet)

Python

```python
import click

# ===== أمر بسيط =====
@click.command()
def hello():
    """وصف الأمر (يظهر في --help)."""
    click.echo("مرحبًا!")

# ===== Argument مقابل Option =====
@click.argument("name")                          # إلزامي، بدون --
@click.option("--greeting", default="مرحبًا")    # اختياري، بـ --
@click.option("--loud", is_flag=True)             # علم منطقي True/False
@click.option("--age", type=click.IntRange(0, 120))
@click.option("--level", type=click.Choice(["a", "b"]))

# ===== أوامر فرعية (مثل git) =====
@click.group()
def cli():
    pass

@cli.command()
def sub_command():
    pass

# ===== ميزات مفيدة =====
click.secho("نص ملوّن", fg="green", bold=True)
click.confirmation_option(prompt="متأكد؟")
click.progressbar(items, label="جاري العمل")
type=click.Path(exists=True, path_type=Path)

# ===== تحويلها لأمر نظام =====
# في pyproject.toml:
# [project.scripts]
# mytool = "my_package.cli:cli"
```

المهمة	الحل
مدخل إلزامي	`@click.argument("x")`
مدخل اختياري بقيمة افتراضية	`@click.option("--x", default=...)`
علم تشغيل/إيقاف	`@click.option("--x", is_flag=True)`
تحقق من نوع/نطاق	`type=click.IntRange(...)` , `type=int`
أوامر فرعية	`@cli.command()` + `@click.group()`
تأكيد قبل عملية خطيرة	`@click.confirmation_option()`
شريط تقدّم	`click.progressbar(...)`

23.11 ماذا بعد؟

لقد بنيت الآن أداة CLI احترافية وحقيقية — من النوع الذي يمكنك نشره فعليًا واستخدامه يوميًا، أو مشاركته مع زملائك. بهذا، تكون قد أتممت **الجزء الرابع: جسور التخصص** بالكامل، وتعرّفت عمليًا على خمسة مسارات مختلفة لبايثون: الويب، تحليل البيانات، الأتمتة، الذكاء الاصطناعي، وأدوات سطر الأوامر.

الآن، ننتقل إلى **الجزء الخامس: الاحتراف والجودة** — حيث سنرتقي بكل ما تعلمته إلى مستوى احترافي حقيقي. نبدأ بالفصل القادم: **كتابة كود نظيف (Clean Code)**، حيث سنتعلّم معايير PEP 8 بعمق، أدوات الفحص الآلي مثل `black` و `ruff` ، ومبادئ SOLID المبسّطة لكتابة كود يفهمه ويُحبّه كل من يقرأه بعدك — بما في ذلك أنت نفسك بعد ستة أشهر!

الجزء السادس · 3 فصول

06

الاحتراف والنشر

كيف تكتب كودًا يقرأه الآخرون بسرور، كيف تثق فيه باختبارات آلية، وكيف تُحوّله من ملف على جهازك إلى منتج حقيقي.

24 · كتابة كود نظيف
25 · الاختبار
26 · من الكود إلى المنتج

فصل 24 / 26 · الجزء السادس

24

كتابة كود نظيف (Clean Code)

9 دقيقة قراءة · 12 قسمًا · 27 مثالًا · 1,807 كلمة

في هذا الفصل

24.1 لماذا «الكود الذي يعمل» لا يكفي؟
24.2 PEP 8: دليل الأسلوب الرسمي لبايثون
24.3 التنسيق الآلي: `black`
24.4 الفحص الآلي للأخطاء والجودة: `ruff`
24.5 مبادئ أساسية لكتابة دوال وأصناف نظيفة
24.6 مقدمة مبسّطة لمبادئ SOLID
24.7 ممارسات إضافية للكود النظيف
24.8 نموذج عملي: قبل وبعد (Refactoring كامل)
24.9 تمارين
24.10 اختبر فهمك (Quiz)
24.11 ملخص الفصل (Cheat Sheet)
24.12 ماذا بعد؟

24.1 لماذا «الكود الذي يعمل» لا يكفي؟

حتى الآن، ركّزنا على هدف واحد رئيسي: **أن يعمل الكود ويُنتج النتيجة الصحيحة**. لكن هناك سؤال آخر لا يقل أهمية: **هل يستطيع إنسان آخر (أو أنتَ نفسك بعد 6 أشهر) أن يفهم هذا الكود بسهولة؟**

تخيّل هذا الموقف الشائع جدًا في حياة كل مبرمج: تفتح ملفًا كتبته بنفسك قبل نصف عام، لحل مشكلة بسيطة فيه، فتجد نفسك تحدّق في الشاشة لعشر دقائق محاولًا فهم «ماذا كنتُ أفكر هنا؟!». هذا ليس نقصًا في الذكاء — إنه نتيجة طبيعية لكتابة **كود غير نظيف**.

> **مقولة شهيرة في عالم البرمجة:**
> *«اكتب الكود وكأن الشخص الذي سيصونه هو قاتل مختل يعرف أين تسكن.»* — مطوّر مجهول (نصيحة مبالغ فيها، لكن الفكرة واضحة: اجعل كودك مفهومًا بلا جهد)

الكود النظيف (Clean Code) ليس ترفًا أو مثالية زائدة — إنه ما يُفرّق بين مبرمج هاوٍ ومبرمج محترف. الشركات الكبرى لا تُقيّم المبرمجين فقط بناءً على «هل يحل المشكلة؟»، بل «هل يحلها بطريقة واضحة، قابلة للصيانة، ولا تُسبب كوارث لاحقة؟».

في هذا الفصل، سنتعلّم:

1. **PEP 8**: دليل الأسلوب الرسمي للغة بايثون.
2. أدوات الفحص والتنسيق الآلي: `black` و `ruff`.
3. مبادئ **SOLID** المبسّطة لتصميم كود قابل للتوسّع والصيانة.
4. ممارسات عملية لتسمية المتغيرات، تنظيم الدوال، وكتابة تعليقات مفيدة.

PEP 8 24.2 : دليل الأسلوب الرسمي لبايثون

PEP تعني "Python Enhancement Proposal" (اقتراح تحسين بايثون) — وهي وثائق رسمية تصف تغييرات أو معايير للغة. **PEP 8** تحديدًا هي الوثيقة الأشهر على الإطلاق: دليل الأسلوب القياسي لكتابة كود بايثون. لقد كنا نتبع معظم قواعده طوال الكتاب دون أن نسمّيه صراحة — الآن حان وقت التوثيق الرسمي.

قواعد التسمية (Naming Conventions)

النوع	الأسلوب	مثال
المتغيرات والدوال	`snake_case`	`user_name` ، `calculate_total()`
الثوابت	`UPPER_SNAKE_CASE`	`MAX_RETRIES` ، `PI`
الأصناف (Classes)	`PascalCase`	`UserAccount` ، `TaskManager`
الخاصيات المحمية	`_single_leading_underscore`	`_internal_cache`
الخاصيات الخاصة	`__double_leading_underscore`	`__secret_key`
الوحدات (ملفات .py)	`snake_case` قصير	`data_loader.py`
أسماء غير مستخدمة	`_` أو `_unused`	`for _ in range(10):`

⚠ **تحذير: تجنّب الأسماء المضلِّلة أو المبهمة**

```python
# ❌ سيء جدًا -- لا يخبرنا بشيء
def calc(x, y, z):
    return x * y - z

# ✅ ممتاز -- الاسم نفسه يشرح الغرض
def calculate_net_price(unit_price: float, quantity: int, discount: float) -> float:
    return unit_price * quantity - discount
```

الكود الجيد **يُقرأ كالنثر**. إذا احتجت تعليقًا لشرح ماذا تفعل دالة أو متغيّر، فكّر أولًا: هل يمكن أن يشرح الاسم نفسه هذا بدلاً من التعليق؟

المسافات والتباعد

```python
# ✅ صحيح: مسافة واحدة حول عوامل التشغيل
total = price + tax

# ❌ خطأ: بدون مسافات، أو مسافات زائدة
total=price+tax
total  =  price  +  tax

# ✅ صحيح: بدون مسافة قبل الفاصلة، مسافة بعدها
def greet(name, age):
    pass

# ✅ صحيح: بدون مسافات حول = في المعاملات الافتراضية (دون type hint)
def greet(name="أحمد"):
    pass

# ✅ نضيف مسافات حول =، لكن مع type hint
def greet(name: str = "أحمد"):
    pass
```

طول السطر

PEP 8 يوصي بحد أقصى **79 حرفًا** للسطر تقليديًا، لكن المعيار الشائع اليوم في المشاريع الحديثة هو **88 حرفًا** (وهو الافتراضي في أداة `black` التي سنتعلمها بعد قليل). الفكرة: الأسطر الطويلة جدًا يصعب قراءتها، خصوصًا عند عرض ملفين جنبًا إلى جنب.

```python
# ❌ سطر طويل جدًا، يصعب قراءته دفعة واحدة
result = calculate_total_price(unit_price, quantity, discount_percentage, tax_rate, shipping_cost)

# ✅ مقسّم بشكل مقروء
result = calculate_total_price(
    unit_price,
    quantity,
    discount_percentage,
    tax_rate,
    shipping_cost,
)
```

الاستيراد (Imports)

```python
# ✅ الترتيب الموصى به: مكتبات قياسية، ثم مكتبات خارجية، ثم وحدات المشروع
import json
import os
from pathlib import Path

import pandas as pd
import requests

from my_project.models import User
from my_project.utils import format_date

# ❌ تجنّب الاستيراد الشامل -- يُخفي مصدر الأسماء ويُسبب تعارضات
from os import *

# ✅ استورد ما تحتاجه فقط بوضوح
from os import getcwd, path
```

> **نصيحة خبير:** لا تحتاج لحفظ كل قواعد PEP 8 عن ظهر قلب — هذا بالضبط ما تفعله الأدوات الآلية التي سنتعلمها الآن تلقائيًا. المهم أن تفهم **الفلسفة**: الوضوح، الاتساق، والقابلية للقراءة.

24.3 التنسيق الآلي: `black`

بدلاً من الجدال حول «أين أضع الفاصلة؟» أو «هل أستخدم علامات اقتباس مفردة أم مزدوجة؟»، تحلّ أداة `black` هذا الجدال نهائيًا: إنها «منسّق عنيد» (Opinionated Formatter) يُعيد كتابة كودك تلقائيًا بأسلوب موحّد وثابت، بلا نقاش.

التثبيت والاستخدام

```terminal
uv add --dev black
```

Python

```python
# قبل black (ملف messy.py)
def   calculate(x,y) :
    result=x+y
    if result>10:
        return   'كبير'
    else:
        return "صغير"
```

Terminal

```
uv run black messy.py
```

Python

```python
# بعد black -- تلقائيًا!
def calculate(x, y):
    result = x + y
    if result > 10:
        return "كبير"
    else:
        return "صغير"
```

لاحظ: `black` وحّد علامات الاقتباس إلى مزدوجة، أصلح المسافات، ونظّم كل شيء — **دون أن تكتب قاعدة واحدة بنفسك**.

Terminal

```
# فحص الملفات بدون تعديلها فعليًا (مفيد في CI/CD)
uv run black --check .

# معاينة التغييرات المقترحة دون تطبيقها
uv run black --diff .

# تنسيق كل ملفات المشروع
uv run black .
```

نصيحة خبير: اضبط VS Code (الإضافة التي ثبّتناها في الفصل 2) على «التنسيق عند الحفظ» (Format on Save) باستخدام `black`. بهذا، كل مرة تحفظ فيها ملفًا، يُنسَّق تلقائيًا دون أي جهد واعٍ منك.

مقارنة: لماذا «عنيد» (Opinionated) أمر جيد؟

أداة تنسيق مرنة (إعدادات كثيرة)	`black` (عنيد، إعدادات قليلة)
كل فريق يختار أسلوبًا مختلفًا	أسلوب موحّد عالميًا بين كل مشاريع بايثون
نقاشات لا تنتهي في مراجعات الكود حول الأسلوب	لا نقاش — الأداة تقرر
وقت ضائع في التنسيق اليدوي	وقت التنسيق = صفر

هذا بالضبط سبب نجاح `black`: إنه يُنهي «حرب الأساليب» بشكل نهائي.

24.4 الفحص الآلي للأخطاء والجودة: `ruff`

بينما `black` يُنسّق **شكل** الكود، أداة `ruff` (Linter) تفحص **جودة ومنطق** الكود: متغيرات غير مستخدمة، استيراد لا داعي له، أخطاء منطقية محتملة، ومخالفات PEP 8 التي لا يُصلحها `black`.

مقارنة: Formatter مقابل Linter

	Formatter (مثل `black`)	Linter (مثل `ruff`)
يهتم بـ	الشكل (مسافات، أسطر، اقتباسات)	المحتوى (أخطاء محتملة، كود ميت، أنماط سيئة)
مثال	إعادة ترتيب المسافات	تحذير: "المتغيّر `x` مُعرَّف لكن غير مستخدم أبدًا"
يُعدّل الكود تلقائيًا؟	نعم، دائمًا	أحيانًا (عبر `--fix`)

لماذا `ruff` تحديدًا؟

`ruff` أداة حديثة مكتوبة بلغة Rust، وأصبحت المعيار الفعلي (de facto standard) في مجتمع بايثون الحديث لأنها **أسرع بعشرات المرات** من الأدوات التقليدية (مثل `flake8` و`pylint`)، وتجمع وظائف عدة أدوات قديمة في أداة واحدة سريعة.

التثبيت والاستخدام

Terminal

```
uv add --dev ruff
```

Python problematic.py

```python
import os  # مستورد لكن غير مستخدم
import sys

def process_data(data):
    unused_variable = 42  # معرّف لكن غير مستخدم
    result = []
    for item in data:
        if item == None:  # يجب استخدام "is None" وليس "== None"
            continue
        result.append(item)
    return result
```

Terminal

```
uv run ruff check problematic.py
```

```
problematic.py:1:8: F401 `os` imported but unused
problematic.py:8:5: F841 local variable `unused_variable` is assigned to but never used
problematic.py:11:16: E711 comparison to None should be 'if cond is None:'
Found 3 errors.
```

Terminal

```
# إصلاح تلقائي لما يمكن إصلاحه آليًا
uv run ruff check --fix problematic.py
```

إعدادات `ruff` في `pyproject.toml`

```toml
[tool.ruff]
line-length = 88
target-version = "py314"

[tool.ruff.lint]
select = [
    "E",   # pycodestyle errors
    "F",   # pyflakes (أخطاء منطقية)
    "I",   # ترتيب الاستيراد (isort)
    "N",   # قواعد تسمية PEP 8
    "UP",  # ترقية تلقائية لصياغات بايثون حديثة
]
```

نصيحة خبير: شغّل `black` و `ruff` معًا دائمًا كجزء من روتين عملك اليومي:

```
uv run ruff check --fix .   # إصلاح المشاكل المنطقية أولاً
uv run black .              # تنسيق الشكل أخيرًا
```

هذا الثنائي (`ruff` + `black`) هو المعيار شبه الرسمي في معظم مشاريع بايثون الاحترافية الحديثة اليوم.

24.5 مبادئ أساسية لكتابة دوال وأصناف نظيفة

مبدأ المسؤولية الواحدة (دالة واحدة = مهمة واحدة)

```python
# ❌ سيء: دالة واحدة تفعل أشياء كثيرة جدًا
def process_order(order_data):
    # التحقق من الصحة
    if not order_data.get("items"):
        raise ValueError("لا توجد عناصر")

    # حساب الإجمالي
    total = sum(item["price"] * item["qty"] for item in order_data["items"])

    # تطبيق الخصم
    if order_data.get("coupon"):
        total *= 0.9
```

```
    # حفظ في قاعدة البيانات
    print(f"حفظ الطلب بإجمالي {total}")

    # إرسال بريد إلكتروني
    print("إرسال تأكيد عبر البريد.  ")

    return total

# ✅ جيد: كل دالة مسؤولة عن شيء واحد فقط
def validate_order(order_data: dict) -> None:
    """التحقق من صحة بيانات الطلب."""
    if not order_data.get("items"):
        raise ValueError("لا توجد عناصر في الطلب")

def calculate_total(items: list[dict]) -> float:
    """حساب إجمالي سعر العناصر."""
    return sum(item["price"] * item["qty"] for item in items)

def apply_discount(total: float, has_coupon: bool) -> float:
    """تطبيق خصم %10 إذا وُجد كوبون."""
    return total * 0.9 if has_coupon else total

def save_order(total: float) -> None:
    """حفظ الطلب في قاعدة البيانات."""
    print(f"حفظ الطلب بإجمالي {total}")

def send_confirmation_email() -> None:
    """إرسال بريد تأكيد الطلب."""
    print("إرسال تأكيد عبر البريد.  ")

def process_order(order_data: dict) -> float:
    """تنسيق عملية معالجة الطلب الكاملة."""
    validate_order(order_data)
    total = calculate_total(order_data["items"])
    total = apply_discount(total, bool(order_data.get("coupon")))
    save_order(total)
    send_confirmation_email()
    return total
```

لماذا هذا أفضل؟

- كل دالة **قابلة للاختبار بمفردها** (سنتعلّم `pytest` في الفصل القادم).
- يسهل **إعادة استخدام** أي دالة في سياق آخر (مثلاً `calculate_total` في تقرير مختلف).
- عند حدوث خطأ، تعرف **بالضبط** أين تبحث.
- القراءة من `process_order` تُشبه قراءة فهرس كتاب: واضحة ومباشرة.

تجنّب «الأرقام السحرية» (Magic Numbers) والنصوص الحرفية المتكررة

Python

```
# ❌ سيء: ما معنى 0.15؟ ولماذا 3؟
def calculate_price(amount):
    if amount > 1000:
        return amount * 0.85
    return amount * 0.95

for attempt in range(3):
    try_connection()

# ✅ جيد: الثوابت المُسمّاة تشرح نفسها
BULK_DISCOUNT_THRESHOLD = 1000
BULK_DISCOUNT_RATE = 0.85
STANDARD_DISCOUNT_RATE = 0.95
MAX_CONNECTION_ATTEMPTS = 3

def calculate_price(amount: float) -> float:
    if amount > BULK_DISCOUNT_THRESHOLD:
        return amount * BULK_DISCOUNT_RATE
    return amount * STANDARD_DISCOUNT_RATE

for attempt in range(MAX_CONNECTION_ATTEMPTS):
    try_connection()
```

Guard Clauses بدلاً من التداخل العميق

تعلمنا هذا في الفصل 6 و10، لكنه يستحق التذكير كممارسة من ممارسات الكود النظيف الأساسية:

Python

```python
# ❌ سيء: تداخل عميق (Nested Ifs) صعب التتبع
def get_discount(user):
    if user is not None:
        if user.is_active:
            if user.membership == "premium":
                return 0.2
            else:
                return 0.1
        else:
            return 0
    else:
        return 0

# ✅ جيد: Guard Clauses -- خروج مبكر، أقل تداخلًا
def get_discount(user) -> float:
    if user is None:
        return 0
    if not user.is_active:
        return 0
    if user.membership == "premium":
        return 0.2
    return 0.1
```

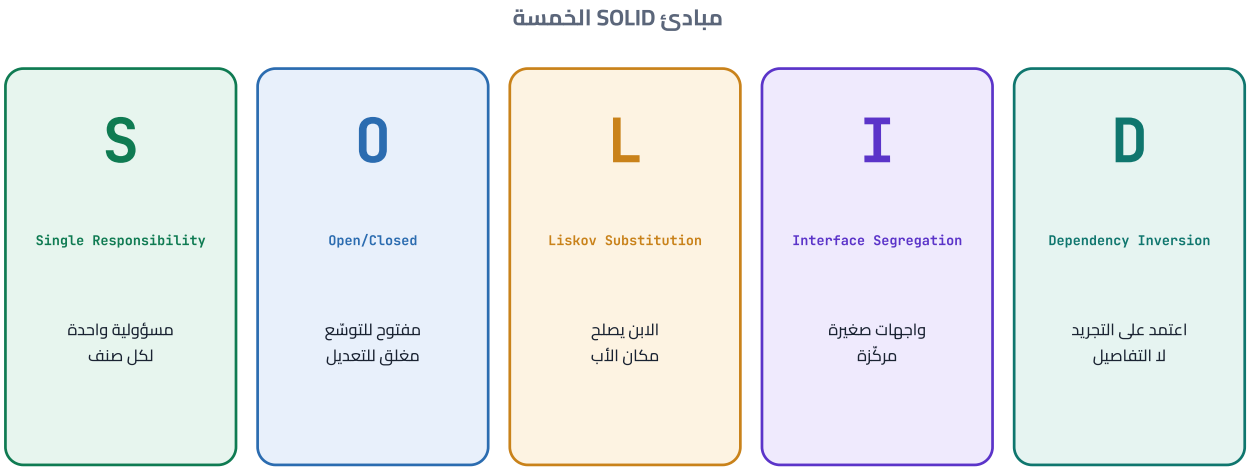
التعليقات: متى تكتبها ومتى تتجنّبها

Python

```python
# ❌ تعليق زائد لا يُضيف أي معلومة
x = x + 1  # إضافة 1 إلى x

# ❌ تعليق يشرح "ماذا" بدلاً من "لماذا" (الكود نفسه يوضّح "ماذا")
# نحسب السعر الإجمالي بضرب السعر في الكمية
total = price * quantity

# ✅ تعليق مفيد يشرح "لماذا" -- معلومة لا يمكن استنتاجها من الكود نفسه
# نستخدم هامش أمان 5% لتعويض تقلبات أسعار الصرف اليومية
total = price * quantity * 1.05

# ✅ تعليق يشرح قرارًا غير بديهي
# نستخدم format التاريخ القديم هنا للتوافق مع النظام القديم (legacy system) ملاحظة:
date_string = date.strftime("%d/%m/%Y")
```

> **نصيحة خبير ذهبية:** «التعليق الجيد يشرح **لماذا**، والكود الجيد يشرح **ماذا**.» إذا وجدت نفسك تكتب تعليقًا لشرح «ماذا يفعل هذا السطر»، فكّر أولًا في إعادة تسمية المتغيرات أو الدوال لتكون أوضح، بدلاً من الاعتماد على التعليق.

24.6 مقدمة مبسّطة لمبادئ SOLID



شكل 24.1 — مبادئ SOLID الخمسة

SOLID هي اختصار لخمسة مبادئ لتصميم كود كائني التوجه (OOP، كما تعلمنا في الفصلين 13 و14) قابل للصيانة والتوسّع. لن نتعمّق في كل التفاصيل الأكاديمية، بل نفهم **الفكرة العملية** وراء كل مبدأ.

S — مبدأ المسؤولية الواحدة (Single Responsibility Principle)

«يجب أن يكون لكل صنف (class) سبب واحد فقط للتغيير.»

Python

```python
# ❌ ينتهك SRP: صنف واحد يفعل أشياء كثيرة غير مرتبطة
class Report:
    def __init__(self, data: list[dict]):
        self.data = data

    def calculate_summary(self) -> dict:
        return {"total": sum(d["value"] for d in self.data)}

    def save_to_file(self, path: str) -> None:  # مسؤولية مختلفة: التخزين!
        with open(path, "w") as f:
            f.write(str(self.data))

    def send_email(self, address: str) -> None:  # مسؤولية مختلفة: الاتصال!
        print(f"إرسال التقرير إلى {address}")

# ✅ يتبع SRP: كل صنف مسؤول عن شيء واحد
class Report:
    def __init__(self, data: list[dict]):
        self.data = data

    def calculate_summary(self) -> dict:
        return {"total": sum(d["value"] for d in self.data)}

class ReportStorage:
    @staticmethod
    def save_to_file(report: Report, path: str) -> None:
        with open(path, "w") as f:
            f.write(str(report.data))

class ReportMailer:
    @staticmethod
    def send_email(report: Report, address: str) -> None:
        print(f"إرسال التقرير إلى {address}")
```

الفائدة العملية: إذا تغيّرت طريقة الحفظ (من ملف إلى قاعدة بيانات)، تُعدّل `ReportStorage` فقط، دون المخاطرة بكسر منطق حساب التقرير أو الإرسال.

O — مبدأ الفتح والإغلاق (Open/Closed Principle)

«يجب أن يكون الكود مفتوحًا للتوسّع، مغلقًا للتعديل.» أي: يمكنك إضافة سلوك جديد **دون تعديل الكود الموجود والمُختبَر مسبقًا**.

Python

```python
from abc import ABC, abstractmethod

# ✅ يتبع OCP: إضافة طريقة دفع جديدة لا تتطلب تعديل أي كود قديم
class PaymentMethod(ABC):
    @abstractmethod
    def pay(self, amount: float) -> str:
        ...

class CreditCardPayment(PaymentMethod):
    def pay(self, amount: float) -> str:
        return f"دفع {amount} عبر بطاقة الائتمان"

class PayPalPayment(PaymentMethod):
    def pay(self, amount: float) -> str:
        return f"دفع {amount} عبر PayPal"

# لإضافة طريقة دفع جديدة، أضف صنفًا جديدًا فقط -- بدون لمس الكود القديم!
class CryptoPayment(PaymentMethod):
    def pay(self, amount: float) -> str:
        return f"دفع {amount} عبر العملات الرقمية"

def checkout(method: PaymentMethod, amount: float) -> None:
    print(method.pay(amount))

checkout(CreditCardPayment(), 100)
checkout(CryptoPayment(), 50)
```

هذا يُذكّرنا بـ Abstract Base Classes وتعدد الأشكال (Polymorphism) التي تعلمناها في الفصل 14 — وهي الأداة التقنية الأساسية لتطبيق هذا المبدأ.

L — مبدأ استبدال ليسكوف (Liskov Substitution Principle)

«يجب أن يكون بإمكان استبدال أي صنف فرعي بصنفه الأساسي دون كسر البرنامج.»

```python
# ❌ الصنف الفرعي يُغيّر السلوك المتوقع بشكل مفاجئ: ينتهك LSP
class Bird:
    def fly(self) -> str:
        return "أطير في السماء"

class Penguin(Bird):
    def fly(self) -> str:
        raise NotImplementedError("البطريق لا يطير!")  # مفاجأة سيئة لأي كود يتوقع fly()

# ✅ إعادة تصميم الهرمية لتعكس الواقع بدقة: يتبع LSP
class Bird(ABC):
    @abstractmethod
    def move(self) -> str:
        ...

class FlyingBird(Bird):
    def move(self) -> str:
        return "أطير في السماء"

class Penguin(Bird):
    def move(self) -> str:
        return "أسبح في الماء"  # سلوك صحيح ومتوقع، بدون كسر توقعات المستخدم
```

الدرس العملي: إذا وجدت نفسك تكتب `if isinstance(bird, Penguin): ... else: ...` للتعامل مع حالة استثنائية لصنف فرعي، فهذا مؤشر قوي على انتهاك LSP — عيد التفكير في التصميم الهرمي.

I — مبدأ فصل الواجهات (Interface Segregation Principle)

«لا تُجبر صنفًا على تطبيق دوال لا يحتاجها.»

Python

```python
# ❌ ينتهك ISP: واجهة ضخمة تُجبر كل شيء على تطبيق كل شيء
class Worker(ABC):
    @abstractmethod
    def work(self) -> str: ...
    @abstractmethod
    def eat(self) -> str: ...

class RobotWorker(Worker):
    def work(self) -> str:
        return "أعمل 24 ساعة"

    def eat(self) -> str:
        raise NotImplementedError("الروبوت لا يأكل!")  # إجبار غير منطقي

# ✅ يتبع ISP: واجهات صغيرة ومتخصصة، يطبّق كل صنف ما يحتاجه فقط
class Workable(ABC):
    @abstractmethod
    def work(self) -> str: ...

class Eatable(ABC):
    @abstractmethod
    def eat(self) -> str: ...

class HumanWorker(Workable, Eatable):
    def work(self) -> str:
        return "أعمل 8 ساعات"

    def eat(self) -> str:
        return "أتناول الغداء"

class RobotWorker(Workable):  # أبدًا لا حاجة لتطبيق eat()
    def work(self) -> str:
        return "أعمل 24 ساعة"
```

D — مبدأ انعكاس الاعتماديات (Dependency Inversion Principle)

«اعتمد على التجريد (Abstractions)، وليس على التفاصيل الملموسة (Concrete Implementations).»

Python

```python
# ❌ ينتهك DIP: الصنف مرتبط مباشرة بتفصيل محدد (MySQL)
class UserRepository:
    def __init__(self):
        self.db = MySQLDatabase()  # اعتماد مباشر وصلب على تفصيل محدد

    def get_user(self, user_id: int):
        return self.db.query(f"SELECT * FROM users WHERE id={user_id}")

# ✅ يتبع DIP: الاعتماد على تجريد (Protocol كما تعلمنا في الفصل 15)
from typing import Protocol

class Database(Protocol):
    def query(self, sql: str) -> list: ...

class UserRepository:
    def __init__(self, db: Database):  # نعتمد على أي شيء يطابق الواجهة
        self.db = db

    def get_user(self, user_id: int):
        return self.db.query(f"SELECT * FROM users WHERE id={user_id}")

# الآن يمكن تمرير MySQL أو PostgreSQL أو حتى قاعدة بيانات وهمية للاختبار
repo = UserRepository(db=MySQLDatabase())
repo_test = UserRepository(db=FakeDatabaseForTesting())
```

الفائدة العملية الكبرى: هذا التصميم يجعل **الاختبار** (الفصل القادم) أسهل بكثير — يمكنك تمرير قاعدة بيانات وهمية (Mock) للاختبار بدلاً من الاتصال بقاعدة بيانات حقيقية في كل اختبار.

> **نصيحة خبير:** مبادئ SOLID ليست قواعد صارمة يجب تطبيقها دائمًا وفي كل مكان — تطبيقها المفرط على مشروع صغير قد يُعقّده دون داعٍ (يُسمى هذا أحيانًا Over-engineering). **استخدمها كأدوات تفكير** عند الشعور أن كودًا معيّنًا أصبح صعب الصيانة أو التوسيع، وليس كقائمة تحقق إلزامية لكل سطر تكتبه.

24.7 ممارسات إضافية للكود النظيف

DRY: لا تُكرّر نفسك (Don't Repeat Yourself)

Python

```python
# ❌ تكرار -- نفس المنطق مكرر 3 مرات
def calculate_circle_area(radius):
    return 3.14159 * radius ** 2

def calculate_circle_circumference(radius):
    return 2 * 3.14159 * radius

def calculate_sphere_volume(radius):
    return (4/3) * 3.14159 * radius ** 3

# ✅ استخراج الثابت المشترك
import math  # استخدام math.pi بدلاً من كتابة الرقم يدويًا في كل مكان

def calculate_circle_area(radius: float) -> float:
    return math.pi * radius ** 2

def calculate_circle_circumference(radius: float) -> float:
    return 2 * math.pi * radius
```

KISS: أبقِه بسيطًا وغبيًا (Keep It Simple, Stupid)

Python

```python
# ❌ معقّد دون داعٍ
def is_even(n: int) -> bool:
    return True if n % 2 == 0 else False

# ✅ بسيط ومباشر
def is_even(n: int) -> bool:
    return n % 2 == 0
```

⚠️ تحذير: الذكاء المفرط عدو الوضوح

```python
# ❌ «ذكي» جدًا لكن غير مقروء
result = [x for x in range(100) if not any(x % i == 0 for i in range(2, int(x**0.5)+1)) and x > 1]

# ✅ أوضح، حتى لو أطول قليلاً
def is_prime(n: int) -> bool:
    if n < 2:
        return False
    return all(n % i != 0 for i in range(2, int(n**0.5) + 1))

result = [x for x in range(100) if is_prime(x)]
```

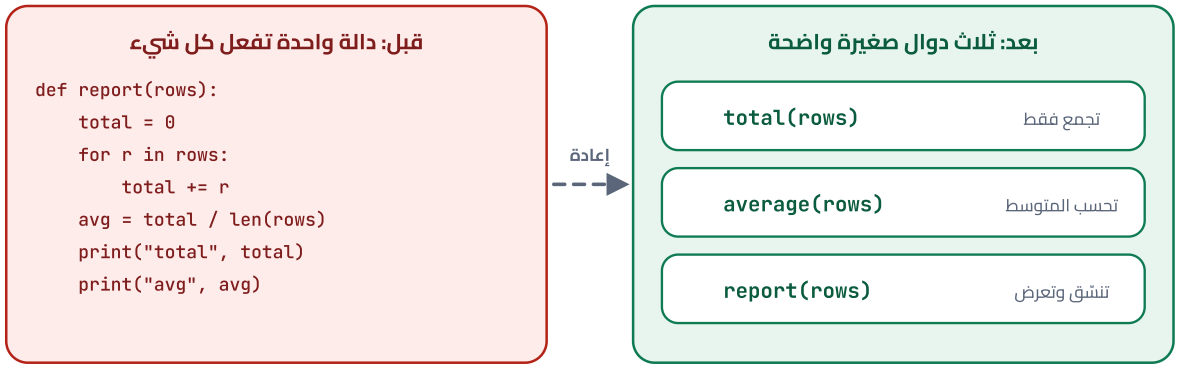
الكود «الذكي جدًا» الذي يتباهى بحل كل شيء في سطر واحد غالبًا ما يكون عبئًا على كل من يقرأه لاحقًا — بما فيهم أنت. **الوضوح دائمًا أهم من الإيجاز.**

YAGNI: لن تحتاجه (You Aren't Gonna Need It)

لا تبني مرونة أو ميزات «قد تحتاجها مستقبلاً» دون حاجة فعلية حالية. هذا يُعقّد الكود دون فائدة حقيقية.

```python
# ❌ مبالغة: نظام إعدادات معقّد لميزة بسيطة قد لا تُستخدم أبدًا
class ConfigurableGreeter:
    def __init__(self, language="ar", formal=False, emoji_support=True,
                 max_length=100, custom_templates=None, plugins=None):
        # ... 20 سطرًا من المرونة غير المستخدمة فعليًا

# ✅ ابدأ بسيطًا، وأضف المرونة فقط عند الحاجة الفعلية
def greet(name: str) -> str:
    return f"مرحبًا، {name}!"
```

24.8 نموذج عملي: قبل وبعد (Refactoring كامل)



شكل 24.2 — إعادة الهيكلة: من دالة واحدة إلى ثلاث

لنرَ مثالًا متكاملًا يُطبّق كل ما تعلمناه في هذا الفصل:

Python

```
# ===== قبل: كود يعمل لكنه غير نظيف =====
def proc(l):
    r = []
    for i in l:
        if i['type'] == 'A':
            if i['amt'] > 100:
                r.append({'id': i['id'], 'final': i['amt'] * 0.9})
            else:
                r.append({'id': i['id'], 'final': i['amt']})
        elif i['type'] == 'B':
            r.append({'id': i['id'], 'final': i['amt'] * 0.95})
    return r
```

Python

```
# ===== بعد: نظيف، واضح، وقابل للصيانة =====

TYPE_A = "A"
TYPE_B = "B"

TYPE_A_BULK_THRESHOLD = 100
TYPE_A_BULK_DISCOUNT = 0.9
TYPE_B_STANDARD_DISCOUNT = 0.95

def calculate_type_a_discount(amount: float) -> float:
```

```
    """حساب السعر النهائي لعناصر النوع A (خصم إضافي للكميات الكبيرة)."""
    if amount > TYPE_A_BULK_THRESHOLD:
        return amount * TYPE_A_BULK_DISCOUNT
    return amount

def calculate_type_b_discount(amount: float) -> float:
    """حساب السعر النهائي لعناصر النوع B (خصم ثابت)."""
    return amount * TYPE_B_STANDARD_DISCOUNT

def process_items(items: list[dict]) -> list[dict]:
    """معالجة قائمة عناصر وحساب السعر النهائي لكل منها حسب نوعه.

    Args:
        items: قائمة قواميس، كل منها يحتوي 'id', 'type', 'amt'.

    Returns:
        قائمة قواميس تحتوي 'id' و'final' (السعر النهائي بعد الخصم).
    """
    processed_items = []

    for item in items:
        if item["type"] == TYPE_A:
            final_price = calculate_type_a_discount(item["amt"])
        elif item["type"] == TYPE_B:
            final_price = calculate_type_b_discount(item["amt"])
        else:
            continue  # تخطّ الأنواع غير المعروفة بدلاً من كسر البرنامج

        processed_items.append({"id": item["id"], "final": final_price})

    return processed_items
```

لاحظ الفرق الهائل في القابلية للفهم، رغم أن **المنطق الأساسي لم يتغيّر إطلاقًا**!

24.9 تمارين

🟢 مستوى مبتدئ

1. ثبّت `black` و `ruff` في مشروع تجريبي، وشغّلهما على كود به مشاكل تنسيق واضحة (مسافات عشوائية، علامات اقتباس مختلطة).
2. أعد تسمية هذه المتغيرات لتكون أوضح: `d` , `tmp` , `lst2` , `flag1` .
3. استخرج «الأرقام السحرية» في الكود التالي إلى ثوابت مُسمّاة:

```python
def calculate_shipping(weight):
    if weight > 20:
        return weight * 5.5
    return weight * 3.2
```

✅ الحل المقترح

```python
# 2
# d → user_data
# tmp → temporary_file_path
# lst2 → filtered_products
# flag1 → is_user_verified

# 3
HEAVY_PACKAGE_THRESHOLD_KG = 20
HEAVY_PACKAGE_RATE = 5.5
STANDARD_RATE = 3.2

def calculate_shipping(weight: float) -> float:
    if weight > HEAVY_PACKAGE_THRESHOLD_KG:
        return weight * HEAVY_PACKAGE_RATE
    return weight * STANDARD_RATE
```

🟡 مستوى متوسط

4. أعد هيكلة (refactor) هذه الدالة لتطبيق مبدأ المسؤولية الواحدة (قسّمها لعدة دوال أصغر):

```python
def handle_user_signup(email, password):
    if "@" not in email:
        raise ValueError("بريد غير صالح")
    if len(password) < 8:
        raise ValueError("كلمة مرور قصيرة")
    hashed = password[::-1]  # تشفير وهمي للتبسيط
    print(f"حفظ المستخدم: {email}")
    print(f"إرسال بريد ترحيبي إلى: {email}")
    return {"email": email, "password": hashed}
```

5. طبّق Guard Clauses على هذه الدالة لتقليل التداخل:

```python
def get_shipping_cost(order):
    if order is not None:
        if order.weight > 0:
            if order.destination == "local":
                return order.weight * 2
            else:
                return order.weight * 5
    return 0
```

✅ الحل المقترح

Python

```python
# 4
def validate_email(email: str) -> None:
    if "@" not in email:
        raise ValueError("بريد غير صالح")

def validate_password(password: str) -> None:
    if len(password) < 8:
        raise ValueError("كلمة مرور قصيرة")

def hash_password(password: str) -> str:
    return password[::-1]  # تشفير وهمي للتبسيط

def save_user(email: str, hashed_password: str) -> None:
    print(f"حفظ المستخدم: {email}")

def send_welcome_email(email: str) -> None:
    print(f"إرسال بريد ترحيبي إلى: {email}")

def handle_user_signup(email: str, password: str) -> dict:
    validate_email(email)
    validate_password(password)
    hashed = hash_password(password)
    save_user(email, hashed)
    send_welcome_email(email)
    return {"email": email, "password": hashed}

# 5
def get_shipping_cost(order) -> float:
    if order is None:
        return 0
    if order.weight <= 0:
        return 0
    if order.destination == "local":
        return order.weight * 2
    return order.weight * 5
```

🔴 مستوى تحدي

6. صمّم نظام إشعارات (Notification System) يطبّق مبدأي **Open/Closed** و **Dependency Inversion**: يجب أن تكون إضافة قناة إشعار جديدة (مثل SMS أو Push Notification) ممكنة **دون تعديل أي كود قديم موجود**، وأن تعتمد الدالة الرئيسية على تجريد (Protocol أو ABC) بدلاً من تفاصيل ملموسة.

✅ الحل المقترح

Python

```python
from abc import ABC, abstractmethod

class NotificationChannel(ABC):
    """تجريد (Abstraction) لأي قناة إشعار."""

    @abstractmethod
    def send(self, message: str, recipient: str) -> None:
        ...

class EmailNotification(NotificationChannel):
    def send(self, message: str, recipient: str) -> None:
        print(f"📧 إرسال بريد إلى {recipient}: {message}")

class SMSNotification(NotificationChannel):
    def send(self, message: str, recipient: str) -> None:
        print(f"📱 إرسال رسالة نصية إلى {recipient}: {message}")

# إضافة قناة جديدة (مبدأ Open/Closed) -- بدون أي تعديل على الكود القديم
class PushNotification(NotificationChannel):
    def send(self, message: str, recipient: str) -> None:
        print(f"🔔 إرسال إشعار دفع إلى {recipient}: {message}")

class NotificationService:
    """تعتمد على التجريد فقط -- مبدأ Dependency Inversion."""

    def __init__(self, channels: list[NotificationChannel]):
        self.channels = channels

    def notify_all(self, message: str, recipient: str) -> None:
        for channel in self.channels:
            channel.send(message, recipient)

# الاستخدام
service = NotificationService(
    channels=[EmailNotification(), SMSNotification(), PushNotification()]
)
service.notify_all("طلبك في الطريق!", "ahmad@example.com")
```

24.10 اختبر فهمك (Quiz)

1. ما الفرق بين وظيفة `black` ووظيفة `ruff`؟

`black` أداة تنسيق (Formatter) تهتم بـ**شكل** الكود (مسافات، علامات اقتباس، تقسيم الأسطر) وتُعدّله تلقائيًا بأسلوب موحّد وثابت. `ruff` أداة فحص (Linter) تهتم بـ**جودة ومنطق** الكود، فتكتشف أخطاء محتملة مثل متغيرات غير مستخدمة، استيراد زائد، ومخالفات أسلوبية لا يُصلحها التنسيق وحده.

2. ما الفرق بين التعليق الجيد والتعليق غير الضروري؟

التعليق غير الضروري يشرح «ماذا يفعل الكود» رغم أن الكود نفسه واضح بما يكفي لفهم ذلك. التعليق الجيد يشرح «لماذا» اتُّخذ قرار معيّن — معلومة سياقية لا يمكن استنتاجها من قراءة الكود وحده (كسبب استخدام قيمة معيّنة، أو قيد تقني خارجي).

3. اشرح مبدأ المسؤولية الواحدة (SRP) بمثال من حياتك اليومية بعيدًا عن البرمجة.

مثال: سكين المطبخ مسؤول فقط عن التقطيع، وليس عن الطبخ أو الغسيل أيضًا. إذا كانت أداة واحدة «تفعل كل شيء»، يصبح إصلاحها أو تحسينها لمهمة واحدة محفوفًا بمخاطر التأثير على المهام الأخرى. بالمثل، الصنف الذي يتعامل مع البيانات ويحفظها ويرسل بريدًا إلكترونيًا في آنٍ واحد يصعب صيانته وتعديله بأمان.

4. ما الفرق بين مبدأ DRY ومبدأ YAGNI؟

DRY («لا تُكرّر نفسك») يحثّ على استخراج المنطق المتكرر إلى مكان واحد بدلاً من نسخه في عدة أماكن. YAGNI («لن تحتاجه») يحثّ على عدم بناء مرونة أو ميزات إضافية لحالات استخدام مستقبلية افتراضية لم تظهر الحاجة الفعلية لها بعد. كلاهما يهدف لتبسيط الكود، لكن DRY يركّز على التكرار الحالي، وYAGNI يركّز على التعقيد المستقبلي غير الضروري.

5. لماذا يُعتبر الاعتماد على Protocol/ABC (التجريد) أفضل من الاعتماد المباشر على صنف ملموس محدد (مبدأ DIP)؟

لأن الاعتماد على تجريد يسمح باستبدال التفاصيل الملموسة بسهولة (كتغيير قاعدة بيانات، أو استخدام نسخة وهمية للاختبار) دون تعديل الكود الذي يستخدم ذلك التجريد. هذا يجعل الكود أكثر مرونة وأسهل اختبارًا، لأنه لا يكون «متجمّدًا» على تفصيل تقني واحد.

24.11 ملخص الفصل (Cheat Sheet)

Terminal

```
# ===== الأدوات =====
uv add --dev black ruff

uv run black .                  # تنسيق تلقائي لكل الملفات
uv run black --check .          # فحص دون تعديل (لـ CI/CD)

uv run ruff check .             # فحص المشاكل المنطقية والأسلوبية
uv run ruff check --fix .       # إصلاح تلقائي لما يمكن إصلاحه
```

Python

```
# ===== قواعد تسمية PEP 8 =====
# متغيرات/دوال  ←  snake_case
# ثوابت         ←  UPPER_SNAKE_CASE
# أصناف         ←  PascalCase
# خاص           ←  _single_underscore / __double_underscore
```

المبدأ	الفكرة بإيجاز
SRP	صنف واحد = مسؤولية واحدة
OCP	وسّع بإضافة كود جديد، لا بتعديل القديم
LSP	الصنف الفرعي يجب أن يحترم عقد الصنف الأساسي
ISP	واجهات صغيرة ومتخصصة، لا واجهات ضخمة شاملة
DIP	اعتمد على التجريد، لا على التفاصيل الملموسة
DRY	لا تُكرّر نفسك
KISS	أبقِه بسيطًا
YAGNI	لا تبنِ ما لا تحتاجه فعليًا الآن

24.12 ماذا بعد؟

لقد تعلّمت الآن كيف تُميّز بين «كود يعمل» و«كود احترافي» — فرقٌ جوهري يُعرّف مسيرتك المهنية كمبرمج. لكن يبقى سؤال حاسم: **كيف تتأكد أن كودك «يعمل» فعلاً، وأنه سيستمر في العمل بشكل صحيح حتى بعد تعديله لاحقًا؟**

الجواب هو **الاختبار الآلي (Automated Testing)**. في الفصل القادم، سنتعلّم مكتبة `pytest` — المعيار الذهبي للاختبار في بايثون — لكتابة اختبارات تتحقق تلقائيًا من صحة كودك، تكتشف الأخطاء قبل أن يراها المستخدم، وتمنحك الثقة الكاملة لتعديل الكود دون خوف من كسر شيء ما دون أن تعلم.

فصل 25 / 26 · الجزء السادس

25

الاختبار (Testing) باستخدام pytest

10 دقيقة قراءة | 16 قسمًا | 25 مثالًا | 2,072 كلمة

في هذا الفصل

25.1 لماذا نختبر الكود؟
25.2 المفاهيم الأساسية للاختبار
25.3 البدء مع pytest
25.4 `assert`: قلب كل اختبار
25.5 Fixtures: تجهيز بيانات الاختبار بكفاءة
25.6 Parametrize: اختبار حالات متعددة بكفاءة
25.7 اختبار الأصناف (Classes) والكود الكائني
25.8 قياس تغطية الاختبارات (Coverage)
25.9 اختبار الكود غير المتزامن (async)
25.10 Mocking: محاكاة الأجزاء الخارجية
25.11 مشروع تطبيقي متكامل: اختبار نظام إدارة مخزون
25.12 متى تكتب الاختبارات؟ (TDD بإيجاز)
25.13 تمارين
25.14 اختبر فهمك (Quiz)
25.15 ملخص الفصل (Cheat Sheet)
25.16 ماذا بعد؟

25.1 لماذا نختبر الكود؟

تخيّل هذا السيناريو: لديك دالة `calculate_discount()` تعمل بشكل ممتاز منذ أشهر. يأتي زميلك ويُعدّل دالة أخرى في نفس الملف لإضافة ميزة جديدة، دون أن يعلم أن تعديله الصغير كسر منطق الخصم بطريقة غير متوقعة. لا أحد يلاحظ المشكلة حتى يشتكي عميل حقيقي من حساب خاطئ لفاتورته — بعد أسابيع من نشر الكود!

هذا بالضبط ما تحلّه **الاختبارات الآلية (Automated Tests)**: برامج صغيرة تتحقق تلقائيًا من أن كودك يعمل كما هو متوقع، وتُشغَّل في ثوانٍ معدودة، وتُخبرك **فورًا** إذا كسر أي تعديل جديد شيئًا كان يعمل سابقًا.

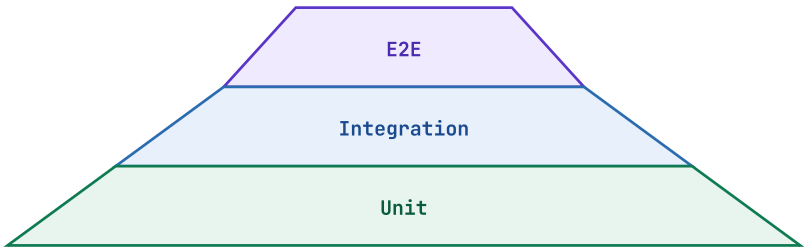
مقارنة: الاختبار اليدوي مقابل الاختبار الآلي

المعيار	الاختبار اليدوي	الاختبار الآلي (pytest)
السرعة	دقائق إلى ساعات	ثوانٍ
التكرار	يُنسى غالبًا، أو يُهمَل لضيق الوقت	يُشغَّل تلقائيًا في كل مرة (حتى قبل كل Commit)
التغطية	جزئية، تعتمد على مزاج المختبِر	شاملة ومتسقة
الثقة عند التعديل	قلق دائم من «هل كسرتُ شيئًا؟»	ثقة: «لو كسرتُ شيئًا، سأعرف فورًا»
التوثيق الضمني	لا يوجد	الاختبارات نفسها توثّق كيف يُفترض أن يعمل الكود

في هذا الفصل، سنتعلّم `pytest` — المكتبة المعيارية والأكثر استخدامًا للاختبار في عالم بايثون — من الأساسيات حتى بناء مجموعة اختبارات حقيقية لمشروع كامل.

25.2 المفاهيم الأساسية للاختبار

هرم الاختبارات: القاعدة الأكبر والأسرع



كلما صعدت: أقل عددًا، أبطأ، وأقرب من تجربة المستخدم.

اكتب الكثير من اختبارات الوحدة، وبعض التكامل، وقليلًا من E2E.

شكل 25.1 — هرم الاختبارات

أنواع الاختبارات (نظرة عامة)

النوع	ماذا يختبر؟	مثال
اختبار الوحدة (Unit Test)	دالة أو صنف **واحد منفرد**، بمعزل عن باقي النظام	هل `calculate_total(10, 2)` تُرجع `20`؟
اختبار التكامل (Integration Test)	تفاعل عدة أجزاء من النظام معًا	هل يعمل حفظ البيانات في قاعدة البيانات الفعلية بشكل صحيح؟
اختبار النظام الشامل (End-to-End)	النظام كاملاً من منظور المستخدم النهائي	هل يمكن للمستخدم تسجيل الدخول، إضافة منتج، وإتمام الشراء؟

في هذا الفصل سنركّز بشكل أساسي على **اختبارات الوحدة**، لأنها الأساس الذي يُبنى عليه كل شيء آخر، وهي الأسرع والأسهل للبدء بها.

هيكل الاختبار النموذجي: Arrange-Act-Assert (AAA)

كل اختبار جيد يتبع عادة نمطًا من ثلاث خطوات:

1. **Arrange (التحضير)**: جهّز البيانات والظروف اللازمة للاختبار.
2. **Act (التنفيذ)**: نفّذ الكود الذي تريد اختباره.
3. **Assert (التحقق)**: تحقّق أن النتيجة مطابقة لما تتوقعه.

```python
def test_addition():
    # Arrange (التحضير)
    a = 5
    b = 3

    # Act (التنفيذ)
    result = a + b

    # Assert (التحقق)
    assert result == 8
```

25.3 البدء مع pytest

التثبيت

```
Terminal
uv add --dev pytest
```

أول اختبار

لنفترض لدينا ملف `calculator.py` بدالة بسيطة:

```python
Python  src/calculator.py
def add(a: float, b: float) -> float:
    """جمع رقمين."""
    return a + b

def divide(a: float, b: float) -> float:
    """قسمة رقمين، مع رفع استثناء عند القسمة على صفر."""
    if b == 0:
        raise ValueError("لا يمكن القسمة على صفر")
    return a / b
```

نكتب ملف الاختبار في `tests/test_calculator.py` :

```python
Python  tests/test_calculator.py
from calculator import add, divide

def test_add_two_positive_numbers():
    result = add(2, 3)
    assert result == 5

def test_add_negative_numbers():
    result = add(-2, -3)
    assert result == -5

def test_divide_normal_case():
    result = divide(10, 2)
    assert result == 5
```

تشغيل الاختبارات:

Terminal

```
uv run pytest
```

```
======================== test session starts ========================
collected 3 items

tests/test_calculator.py ...                                  [100%]

========================= 3 passed in 0.02s =========================
```

نصيحة خبير: قواعد تسمية صارمة يعتمد عليها pytest
يكتشف pytest الاختبارات تلقائيًا بناءً على قواعد تسمية محددة — **احفظها جيدًا**:
- ملفات الاختبار: يجب أن تبدأ بـ `test_` أو تنتهي بـ `_test.py` (مثل `test_calculator.py`)
- دوال الاختبار: يجب أن تبدأ بـ `test_` (مثل `def test_add_numbers():`)
- إذا نسيت هذه البادئة، سيتجاهل pytest اختبارك **بصمت** دون أي تحذير!

تنظيم المشروع مع الاختبارات

```
my_project/
├── pyproject.toml
├── src/
│   └── calculator.py
└── tests/
    ├── __init__.py
    └── test_calculator.py
```

نصيحة خبير: ضَع ملفات الاختبار في مجلد `tests/` منفصل عن الكود المصدري في `src/` . هذا يُبقي المشروع منظّمًا، ويُسهّل استبعاد الاختبارات عند تحزيم (packaging) المشروع للنشر النهائي.

25.4 `assert` : قلب كل اختبار

كما تعلمنا في الفصل 11، `assert` تتحقق من شرط وترفع `AssertionError` إن لم يتحقق. pytest يستخدم `assert` العادية في بايثون، لكنه **يُحسّن رسائل الخطأ تلقائيًا** لتكون واضحة جدًا عند الفشل:

Python

```python
def test_add_fails_example():
    result = add(2, 2)
    assert result == 5  # سيفشل عمدًا لنرى رسالة الخطأ
```

Terminal

```
uv run pytest tests/test_calculator.py::test_add_fails_example -v
```

```
FAILED tests/test_calculator.py::test_add_fails_example - assert 4 == 5
    def test_add_fails_example():
        result = add(2, 2)
>       assert result == 5
E       assert 4 == 5
```

لاحظ كيف أن pytest أخبرنا بالضبط: «النتيجة الفعلية كانت 4، لكننا توقعنا 5» — دون أي كتابة إضافية من جانبنا!

أنواع مختلفة من التحقق

Python

```python
def test_various_assertions():
    assert 5 > 3                          # مقارنة
    assert "hello" in "hello world"       # الاحتواء
    assert [1, 2, 3] == [1, 2, 3]         # تساوي القوائم
    assert isinstance(5, int)             # النوع
    assert not False                      # النفي
    assert {"a": 1} == {"a": 1}           # تساوي القواميس
```

اختبار الاستثناءات: `pytest.raises`

Python

```python
import pytest
from calculator import divide

def test_divide_by_zero_raises_error():
    with pytest.raises(ValueError):
        divide(10, 0)

def test_divide_by_zero_error_message():
    with pytest.raises(ValueError, match="لا يمكن القسمة على صفر"):
        divide(10, 0)
```

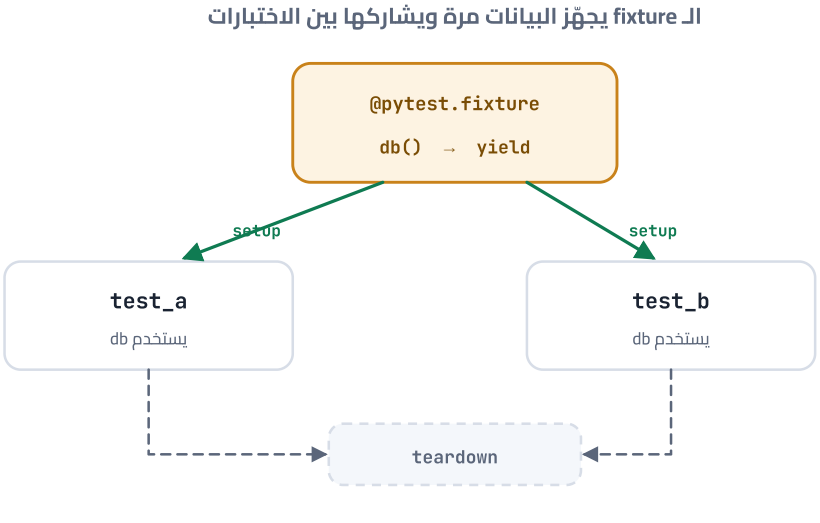
> ⚠️ **تحذير: اختبار الحالة السعيدة فقط لا يكفي**
> من أشيع الأخطاء عند كتابة الاختبارات هو اختبار **فقط** الحالات التي «تعمل بشكل صحيح» (Happy Path)، وتجاهل الحالات الاستثنائية والحدّية (Edge Cases). اختبار جيد يغطي:
> - الحالة الطبيعية (`divide(10, 2)`)
> - الحالات الحدّية (`divide(0, 5)` , قيم سالبة)
> - حالات الخطأ المتوقعة (`divide(10, 0)`)

اختبار القيم التقريبية (الأعداد العشرية)

تذكّر من الفصل 5 أن الأعداد العشرية قد تحمل أخطاء دقة صغيرة:

```python
def test_float_addition_naive():
    assert 0.1 + 0.2 == 0.3  # ❌ قد يفشل! بسبب دقة الفاصلة العائمة

def test_float_addition_correct():
    assert 0.1 + 0.2 == pytest.approx(0.3)  # ✅ يتحقق بتسامح معقول من الفروق الدقيقة جدًا
```

25.5 Fixtures : تجهيز بيانات الاختبار بكفاءة



شكل 25.2 — الـ fixture يجهّز ويشارك

غالبًا ما تحتاج عدة اختبارات إلى نفس «التحضير» (Arrange). بدلاً من تكرار نفس الكود في كل دالة اختبار، نستخدم **Fixtures** — دوال خاصة تُحضّر البيانات أو الموارد وتُمررها تلقائيًا للاختبارات التي تحتاجها.

```python
import pytest

@pytest.fixture
def sample_cart():
    """Fixture تُجهّز سلة تسوّق تجريبية لاستخدامها في عدة اختبارات."""
    return {
        "items": [
            {"name": "لابتوب", "price": 3500, "qty": 1},
            {"name": "ماوس", "price": 50, "qty": 2},
        ],
    }

def test_cart_has_two_item_types(sample_cart):
    assert len(sample_cart["items"]) == 2

def test_cart_total_quantity(sample_cart):
    total_qty = sum(item["qty"] for item in sample_cart["items"])
    assert total_qty == 3
```

لاحظ: كل دالة اختبار تحتاج `sample_cart` تستقبلها ببساطة **كمعامل بنفس الاسم**، وpytest يتكفّل تلقائيًا باستدعاء الـ fixture وتمرير نتيجتها.

> **نصيحة خبير:** هذا يُشبه تمامًا مفهوم **Dependency Injection** الذي رأيناه في الفصل 24 (مبدأ DIP) — الاختبار «يطلب» ما يحتاجه، وpytest «يُحضره» له، دون أن يهتم الاختبار بكيفية إنشائه.

نطاق الـ Fixtures (Scope)

```python
@pytest.fixture(scope="function")  # الافتراضي: يُعاد إنشاؤها لكل اختبار
def fresh_data():
    return []

@pytest.fixture(scope="module")  # تُنشأ مرة واحدة فقط لكل ملف اختبار
def database_connection():
    print("🔌 فتح اتصال بقاعدة البيانات.  ")
    connection = "وهمي-اتصال"
    yield connection  # القيمة التي تُمرَّر للاختبارات
    print("🔌 إغلاق الاتصال.  ")  # كود التنظيف، يُنفَّذ بعد انتهاء كل الاختبارات
```

جدول مرجعي: أنواع نطاق الـ Fixtures

Scope	يُنشأ	الاستخدام المناسب
`function` (افتراضي)	قبل كل دالة اختبار	البيانات البسيطة الخفيفة
`class`	مرة لكل صنف اختبار	بيانات مشتركة بين اختبارات مترابطة
`module`	مرة لكل ملف اختبار	موارد أثقل (اتصال قاعدة بيانات وهمية)
`session`	مرة واحدة لكل تشغيل pytest بالكامل	موارد باهظة جدًا (نادرة الاستخدام)

Fixtures مع `yield` للتنظيف (Setup/Teardown)

```python
import pytest
from pathlib import Path

@pytest.fixture
def temp_file(tmp_path):
    """إنشاء ملف مؤقت للاختبار، وحذفه تلقائيًا بعد الانتهاء."""
    file_path = tmp_path / "test_data.txt"
    file_path.write_text("بيانات تجريبية", encoding="utf-8")

    yield file_path  # الاختبار يستخدم هذه القيمة

    # هذا الجزء يُنفَّذ تلقائيًا بعد انتهاء الاختبار (تنظيف)
    print(f"تنظيف: {file_path}")

def test_file_content(temp_file):
    content = temp_file.read_text(encoding="utf-8")
    assert content == "بيانات تجريبية"
```

نصيحة خبير: `tmp_path` هو fixture **مدمج** في pytest نفسه! ينشئ مجلدًا مؤقتًا فريدًا لكل اختبار، ويحذفه تلقائيًا لاحقًا — مثالي لاختبار دوال تتعامل مع الملفات (كما تعلمنا في الفصل 16) دون ترك «فوضى» في نظام الملفات الحقيقي.

25.6 Parametrize : اختبار حالات متعددة بكفاءة

بدلاً من كتابة دالة اختبار منفصلة لكل حالة، نستخدم `@pytest.mark.parametrize` لتشغيل نفس الاختبار بمدخلات مختلفة:

Python

```python
import pytest
from calculator import add

@pytest.mark.parametrize(
    "a, b, expected",
    [
        (2, 3, 5),
        (-1, 1, 0),
        (0, 0, 0),
        (-5, -5, -10),
        (100, 200, 300),
    ],
)
def test_add_multiple_cases(a, b, expected):
    assert add(a, b) == expected
```

Terminal

```
uv run pytest -v
```

```
tests/test_calculator.py::test_add_multiple_cases[2-3-5] PASSED
tests/test_calculator.py::test_add_multiple_cases[-1-1-0] PASSED
tests/test_calculator.py::test_add_multiple_cases[0-0-0] PASSED
tests/test_calculator.py::test_add_multiple_cases[-5--5--10] PASSED
tests/test_calculator.py::test_add_multiple_cases[100-200-300] PASSED
```

5 اختبارات من دالة واحدة فقط! هذا يُقلّل التكرار بشكل كبير (تطبيق مباشر لمبدأ DRY من الفصل 24).

> **نصيحة خبير:** استخدم `parametrize` بسخاء، خصوصًا لاختبار **الحالات الحدّية** (قيم صفرية، سالبة، فارغة، قصوى) بكفاءة دون تكرار الكود.

25.7 اختبار الأصناف (Classes) والكود الكائني

Python src/bank_account.py

```python
class InsufficientFundsError(Exception):
    """يُرفع عند محاولة سحب مبلغ أكبر من الرصيد المتاح."""
    pass

class BankAccount:
    def __init__(self, owner: str, balance: float = 0):
        self.owner = owner
        self.balance = balance

    def deposit(self, amount: float) -> None:
        if amount <= 0:
            raise ValueError("مبلغ الإيداع يجب أن يكون موجبًا")
        self.balance += amount

    def withdraw(self, amount: float) -> None:
        if amount > self.balance:
            raise InsufficientFundsError("الرصيد غير كافٍ")
        self.balance -= amount
```

Python tests/test_bank_account.py

```python
import pytest
from bank_account import BankAccount, InsufficientFundsError

@pytest.fixture
def account():
    """حساب بنكي تجريبي برصيد ابتدائي 100."""
    return BankAccount(owner="أحمد", balance=100)

class TestDeposit:
    """تجميع كل اختبارات عملية الإيداع في صنف واحد."""

    def test_deposit_increases_balance(self, account):
        account.deposit(50)
        assert account.balance == 150

    def test_deposit_negative_amount_raises_error(self, account):
        with pytest.raises(ValueError):
            account.deposit(-10)

    def test_deposit_zero_raises_error(self, account):
        with pytest.raises(ValueError):
            account.deposit(0)

class TestWithdraw:
    """تجميع كل اختبارات عملية السحب في صنف واحد."""

    def test_withdraw_decreases_balance(self, account):
        account.withdraw(30)
        assert account.balance == 70

    def test_withdraw_more_than_balance_raises_error(self, account):
        with pytest.raises(InsufficientFundsError):
            account.withdraw(200)

    def test_withdraw_exact_balance_succeeds(self, account):
        account.withdraw(100)
        assert account.balance == 0
```

نصيحة خبير: تجميع الاختبارات المترابطة داخل أصناف (`class TestDeposit:`) يُنظّم مجموعة الاختبارات بوضوح، خصوصًا في المشاريع الكبيرة. لاحظ أن أسماء هذه الأصناف يجب أن تبدأ بـ `Test` (وليس `test_`) ليكتشفها pytest تلقائيًا.

25.8 قياس تغطية الاختبارات (Coverage)

شكل 25.3 — التغطية: الأسطر المُغطّاة

تغطية الاختبار (Test Coverage) تقيس: **أي نسبة من كودك** تُنفَّذ فعليًا عند تشغيل الاختبارات؟

```
Terminal
uv add --dev pytest-cov
uv run pytest --cov=src --cov-report=term-missing
```

```
---------- coverage: platform linux, python 3.14.0 -----------
Name                      Stmts   Miss  Cover   Missing
-----------------------------------------------------------
src/calculator.py             8      0   100%
src/bank_account.py          15      2    87%   23-24
-----------------------------------------------------------
TOTAL                        23      2    91%
```

هذا يُخبرنا بالضبط: الأسطر 23-24 في `bank_account.py` **لم تُختبر أبدًا**. يمكننا الآن إضافة اختبار يغطيها تحديدًا.

> ⚠ **تحذير: التغطية 100% ليست الهدف النهائي**
> تغطية 100% تعني فقط أن كل سطر **نُفِّذ** مرة واحدة على الأقل أثناء الاختبارات — وليس بالضرورة أن كل **سيناريو منطقي** اختبرناه بعمق. يمكن أن تحصل على تغطية 100% مع اختبارات سطحية جدًا لا تتحقق من الحالات الحدّية أو المنطق الدقيق. **الهدف الحقيقي** هو اختبارات ذات معنى، وليس مجرد رقم تغطية مرتفع. استخدم نسبة التغطية ك **دليل** لاكتشاف الكود غير المُختبَر إطلاقًا، وليس كهدف نهائي بحد ذاته.

25.9 اختبار الكود غير المتزامن (async)

تذكّر الفصل 17؟ اختبار دوال `async` يتطلب إضافة بسيطة:

Terminal

```
uv add --dev pytest-asyncio
```

Python src/async_utils.py

```python
import asyncio

async def fetch_data() -> str:
    await asyncio.sleep(0.1)
    return "بيانات"
```

Python tests/test_async_utils.py

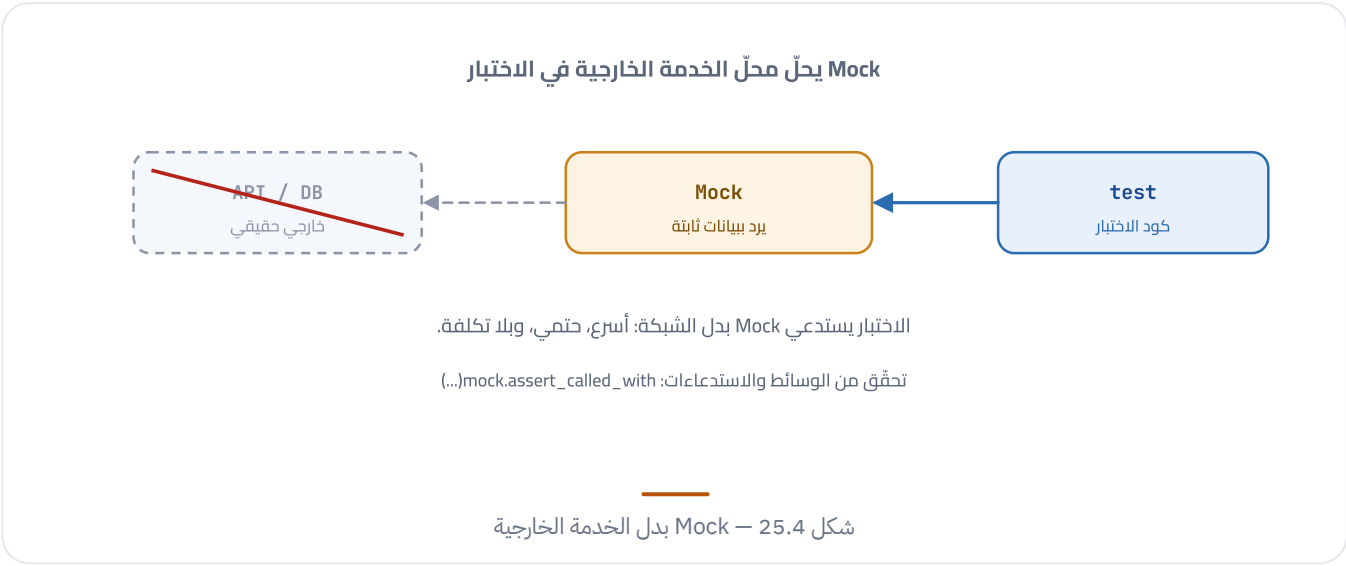
```python
import pytest
from async_utils import fetch_data

@pytest.mark.asyncio
async def test_fetch_data():
    result = await fetch_data()
    assert result == "بيانات"
```

في `pyproject.toml`، يجب تفعيل الوضع التلقائي:

TOML

```toml
[tool.pytest.ini_options]
asyncio_mode = "auto"
```

Mocking 25.10 : محاكاة الأجزاء الخارجية



شكل 25.4 — Mock بدل الخدمة الخارجية

ماذا لو أردنا اختبار دالة تعتمد على طلب شبكة حقيقي (كما تعلمنا في الفصل 21، Web Scraping)؟ لا نريد أن تعتمد اختباراتنا على اتصال إنترنت فعلي (بطيء، وغير مضمون النجاح دائمًا). الحل: **Mocking** — استبدال الجزء الخارجي بـ«نسخة وهمية» تتصرف كما نُريدها بالضبط.

Python src/weather.py

```python
import requests

def get_temperature(city: str) -> float:
    """جلب درجة الحرارة الحالية لمدينة عبر API خارجي."""
    response = requests.get(f"https://api.weather.example.com/{city}", timeout=10)
    data = response.json()
    return data["temperature"]
```

Python tests/test_weather.py

```python
from unittest.mock import MagicMock, patch

from weather import get_temperature

def test_get_temperature_with_mock():
    # استبدال requests.get بنسخة وهمية لا تتصل فعليًا بالإنترنت
    with patch("weather.requests.get") as mock_get:
        mock_response = MagicMock()
        mock_response.json.return_value = {"temperature": 25.5}
        mock_get.return_value = mock_response

        result = get_temperature("الرياض")

        assert result == 25.5
        mock_get.assert_called_once()  # التحقق من أن الدالة استُدعيت فعلاً
```

نصيحة خبير: استخدم Mocking لأي اعتماد خارجي بطيء، غير موثوق، أو مكلف: طلبات الشبكة، قواعد البيانات، إرسال بريد إلكتروني فعلي، أو حتى `datetime.now()` عند اختبار منطق يعتمد على الوقت الحالي. هذا يجعل اختباراتك **سريعة وموثوقة ومستقلة تمامًا** عن العالم الخارجي.

مقارنة: اختبار حقيقي مقابل اختبار بـ Mock

	اختبار حقيقي (بدون Mock)	اختبار بـ Mock
السرعة	بطيء (ينتظر الشبكة فعليًا)	سريع جدًا (فوري)
الموثوقية	يفشل إذا كان الإنترنت معطلاً أو الـ API متوقفًا	يعمل دائمًا بشكل متوقع
الواقعية	يختبر التكامل الفعلي	يفترض سلوك الـ API، قد يُخفي مشاكل حقيقية
الاستخدام المناسب	اختبارات تكامل قليلة ومتعمَّدة	معظم اختبارات الوحدة

25.11 مشروع تطبيقي متكامل: اختبار نظام إدارة مخزون

سنكتب اختبارات شاملة لنظام إدارة مخزون بسيط، مغطّين كل ما تعلمناه: parametrize، fixtures، اختبار استثناءات، mocking، وتنظيم الاختبارات في أصناف.

هيكلية المشروع

```
inventory_system/
├── pyproject.toml
├── src/
│   └── inventory/
│       ├── __init__.py
│       ├── models.py
│       └── inventory_manager.py
└── tests/
    ├── __init__.py
    ├── conftest.py
    └── test_inventory_manager.py
```

Terminal

```
uv init inventory_system
cd inventory_system
uv add --dev pytest pytest-cov
mkdir -p src/inventory tests
touch src/inventory/__init__.py tests/__init__.py
```

`src/inventory/models.py`

```python
"""نماذج بيانات نظام إدارة المخزون."""

from dataclasses import dataclass

@dataclass
class Product:
    """يمثّل منتجًا في المخزون."""

    sku: str
    name: str
    price: float
    quantity: int

class ProductNotFoundError(Exception):
    """يُرفع عند البحث عن منتج غير موجود."""
    pass

class InsufficientStockError(Exception):
    """يُرفع عند محاولة بيع كمية أكبر من المتوفر."""
    pass

class DuplicateSKUError(Exception):
    """يُرفع عند محاولة إضافة منتج برمز SKU موجود مسبقًا."""
    pass
```

`src/inventory/inventory_manager.py`

```python
"""منطق إدارة المخزون: إضافة، بيع، واستعلام عن المنتجات."""

from inventory.models import (
    DuplicateSKUError,
    InsufficientStockError,
    Product,
    ProductNotFoundError,
)

class InventoryManager:
    """يدير مجموعة من المنتجات داخل المخزون."""

    def __init__(self):
        self._products: dict[str, Product] = {}

    def add_product(self, product: Product) -> None:
```

```
        """إضافة منتج جديد إلى المخزون."""

        Raises:
            DuplicateSKUError: إذا كان رمز SKU مستخدمًا مسبقًا.
        """
        if product.sku in self._products:
            raise DuplicateSKUError(f"المنتج {product.sku} موجود بالفعل")
        self._products[product.sku] = product

    def get_product(self, sku: str) -> Product:
        """الحصول على منتج بواسطة رمزه."""

        Raises:
            ProductNotFoundError: إذا لم يوجد منتج بهذا الرمز.
        """
        if sku not in self._products:
            raise ProductNotFoundError(f"المنتج {sku} غير موجود")
        return self._products[sku]

    def sell(self, sku: str, quantity: int) -> float:
        """بيع كمية من منتج، وإرجاع الإيراد الناتج."""

        Raises:
            ProductNotFoundError: إذا لم يوجد المنتج.
            ValueError: إذا كانت الكمية المطلوبة غير موجبة.
            InsufficientStockError: إذا تجاوزت الكمية المطلوبة المخزون المتاح.
        """
        if quantity <= 0:
            raise ValueError("الكمية يجب أن تكون موجبة")

        product = self.get_product(sku)

        if quantity > product.quantity:
            raise InsufficientStockError(
                f"الكمية المتوفرة ({product.quantity}) أقل من المطلوبة ({quantity})"
            )

        product.quantity -= quantity
        return product.price * quantity

    def total_inventory_value(self) -> float:
        """حساب القيمة الإجمالية لكل المخزون الحالي."""
        return sum(p.price * p.quantity for p in self._products.values())

    def low_stock_products(self, threshold: int = 5) -> list[Product]:
        """إرجاع قائمة المنتجات التي كميتها أقل من أو تساوي حدًا معيّنًا."""
        return [p for p in self._products.values() if p.quantity <= threshold]
```

`tests/conftest.py` — Fixtures مشتركة بين كل ملفات الاختبار

Python

```python
"""Fixtures مشتركة متاحة تلقائيًا لكل ملفات الاختبار في هذا المجلد."""

import pytest

from inventory.inventory_manager import InventoryManager
from inventory.models import Product

@pytest.fixture
def empty_manager():
    """مدير مخزون فارغ تمامًا."""
    return InventoryManager()

@pytest.fixture
def manager_with_products(empty_manager):
    """مدير مخزون مُعبّأ بمنتجات تجريبية جاهزة للاختبارات."""
    empty_manager.add_product(Product(sku="LAP001", name="لابتوب", price=3500, quantity=10))
    empty_manager.add_product(Product(sku="MOU001", name="ماوس", price=50, quantity=100))
    empty_manager.add_product(Product(sku="KEY001", name="لوحة مفاتيح", price=120, quantity=3))
    return empty_manager
```

نصيحة خبير: ملف `conftest.py` ملف خاص يكتشفه pytest تلقائيًا — أي fixture مُعرَّفة فيه تصبح متاحة **تلقائيًا** في كل ملفات الاختبار داخل نفس المجلد (وما دونه)، دون الحاجة لاستيرادها يدويًا. هذا هو المكان القياسي لوضع الـ fixtures المشتركة بين عدة ملفات اختبار.

`tests/test_inventory_manager.py`

Python

```python
"""اختبارات شاملة لنظام إدارة المخزون."""

import pytest

from inventory.models import (
    DuplicateSKUError,
    InsufficientStockError,
    Product,
    ProductNotFoundError,
)

class TestAddProduct:
    """اختبارات إضافة منتج جديد."""

    def test_add_product_succeeds(self, empty_manager):
```

```
        product = Product(sku="TEST01", name="منتج تجريبي", price=100, quantity=5)
        empty_manager.add_product(product)

        retrieved = empty_manager.get_product("TEST01")
        assert retrieved.name == "منتج تجريبي"

    def test_add_duplicate_sku_raises_error(self, manager_with_products):
        duplicate = Product(sku="LAP001", name="لابتوب آخر", price=4000, quantity=5)

        with pytest.raises(DuplicateSKUError):
            manager_with_products.add_product(duplicate)

class TestGetProduct:
    """اختبارات استرجاع منتج."""

    def test_get_existing_product(self, manager_with_products):
        product = manager_with_products.get_product("MOU001")
        assert product.name == "ماوس"
        assert product.price == 50

    def test_get_nonexistent_product_raises_error(self, manager_with_products):
        with pytest.raises(ProductNotFoundError):
            manager_with_products.get_product("NOTEXIST")

class TestSellProduct:
    """اختبارات عملية البيع."""

    def test_sell_reduces_quantity(self, manager_with_products):
        manager_with_products.sell("LAP001", 3)
        remaining = manager_with_products.get_product("LAP001")
        assert remaining.quantity == 7

    def test_sell_returns_correct_revenue(self, manager_with_products):
        revenue = manager_with_products.sell("MOU001", 10)
        assert revenue == 500  # 50 * 10

    def test_sell_nonexistent_product_raises_error(self, manager_with_products):
        with pytest.raises(ProductNotFoundError):
            manager_with_products.sell("NOTEXIST", 1)

    @pytest.mark.parametrize("invalid_quantity", [0, -1, -100])
    def test_sell_non_positive_quantity_raises_error(
        self, manager_with_products, invalid_quantity
    ):
        with pytest.raises(ValueError):
            manager_with_products.sell("LAP001", invalid_quantity)

    def test_sell_more_than_available_raises_error(self, manager_with_products):
        with pytest.raises(InsufficientStockError):
            manager_with_products.sell("KEY001", 1000)
```

```python
    def test_sell_exact_remaining_quantity_succeeds(self, manager_with_products):
        """حالة حدّية: بيع الكمية المتبقية بالضبط يجب أن ينجح (وليس يرفض)."""
        manager_with_products.sell("KEY001", 3)  # الكمية المتوفرة بالضبط
        remaining = manager_with_products.get_product("KEY001")
        assert remaining.quantity == 0

class TestInventoryReports:
    """اختبارات تقارير المخزون الإجمالية."""

    def test_total_inventory_value(self, manager_with_products):
        # (3500*10) + (50*100) + (120*3) = 35000 + 5000 + 360 = 40360
        assert manager_with_products.total_inventory_value() == 40360

    def test_empty_inventory_value_is_zero(self, empty_manager):
        assert empty_manager.total_inventory_value() == 0

    def test_low_stock_products_default_threshold(self, manager_with_products):
        low_stock = manager_with_products.low_stock_products()
        skus = [p.sku for p in low_stock]
        assert "KEY001" in skus      # كمية 3، أقل من الحد الافتراضي 5
        assert "LAP001" not in skus  # كمية 10، أعلى من الحد

    def test_low_stock_products_custom_threshold(self, manager_with_products):
        low_stock = manager_with_products.low_stock_products(threshold=15)
        skus = [p.sku for p in low_stock]
        assert "LAP001" in skus  # كمية 10، أقل من الحد المخصص 15
```

تشغيل المجموعة الكاملة مع تقرير التغطية

Terminal

```
uv run pytest --cov=src --cov-report=term-missing -v
```

```
tests/test_inventory_manager.py::TestAddProduct::test_add_product_succeeds PASSED
tests/test_inventory_manager.py::TestAddProduct::test_add_duplicate_sku_raises_error PASSED
tests/test_inventory_manager.py::TestGetProduct::test_get_existing_product PASSED
tests/test_inventory_manager.py::TestGetProduct::test_get_nonexistent_product_raises_error PASSED
tests/test_inventory_manager.py::TestSellProduct::test_sell_reduces_quantity PASSED
tests/test_inventory_manager.py::TestSellProduct::test_sell_returns_correct_revenue PASSED
tests/test_inventory_manager.py::TestSellProduct::test_sell_nonexistent_product_raises_error PASSED
tests/test_inventory_manager.py::TestSellProduct::test_sell_non_positive_quantity_raises_error[0]
PASSED
tests/test_inventory_manager.py::TestSellProduct::test_sell_non_positive_quantity_raises_error[-1]
PASSED
tests/test_inventory_manager.py::TestSellProduct::test_sell_non_positive_quantity_raises_error[-100
] PASSED
tests/test_inventory_manager.py::TestSellProduct::test_sell_more_than_available_raises_error PASSED
tests/test_inventory_manager.py::TestSellProduct::test_sell_exact_remaining_quantity_succeeds
PASSED
tests/test_inventory_manager.py::TestInventoryReports::test_total_inventory_value PASSED
tests/test_inventory_manager.py::TestInventoryReports::test_empty_inventory_value_is_zero PASSED
tests/test_inventory_manager.py::TestInventoryReports::test_low_stock_products_default_threshold
PASSED
tests/test_inventory_manager.py::TestInventoryReports::test_low_stock_products_custom_threshold
PASSED

---------- coverage: platform linux, python 3.14.0 -----------
Name                                  Stmts   Miss  Cover
-------------------------------------------------------
src/inventory/models.py                  14      0   100%
src/inventory/inventory_manager.py       28      0   100%
-------------------------------------------------------
TOTAL                                    42      0   100%

========================= 16 passed in 0.05s =========================
```

تغطية 100%، و16 اختبارًا يُغطّون الحالات الطبيعية **والحدّية والاستثنائية** — بثقة كاملة أن هذا النظام يعمل كما هو متوقع.

✅ الحل الكامل للمشروع (جميع الملفات مجتمعة)

جميع ملفات `models.py`، `inventory_manager.py`، `conftest.py`، و`test_inventory_manager.py` كما وردت بالكامل أعلاه تُشكّل الحل النموذجي الكامل.

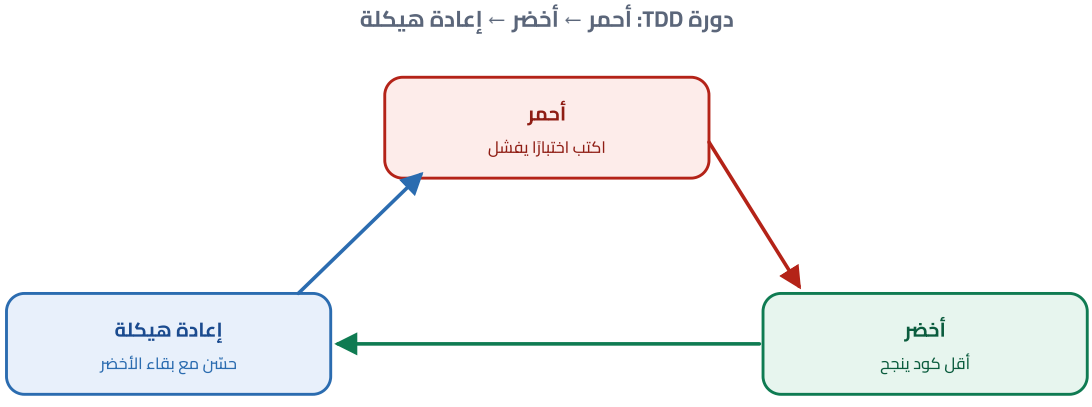
Terminal

```
uv add --dev pytest pytest-cov
uv run pytest --cov=src --cov-report=term-missing -v
```

نقاط تصميم مهمة في هذا الحل:

1. `conftest.py` **للـ Fixtures المشتركة**: يتجنّب تكرار إعداد بيانات الاختبار في كل ملف.
2. **تنظيم الاختبارات في أصناف** (`TestAddProduct`, `TestSellProduct`...) يعكس بنية الدوال المُختبَرة ويُسهّل التنقّل.
3. **تغطية شاملة للحالات**: الحالة الطبيعية، الحدّية (الكمية المتبقية بالضبط)، والاستثنائية (منتج غير موجود، كمية سالبة، مخزون غير كافٍ) لكل دالة.
4. **استخدام** `parametrize` لاختبار عدة قيم غير صالحة بكفاءة دون تكرار الكود.
5. **استثناءات مخصصة واضحة** (كما تعلمنا في الفصل 11) تجعل اختبار حالات الخطأ دقيقًا ومعبّرًا.

25.12 متى تكتب الاختبارات؟ (TDD بإيجاز)



شكل 25.5 — دورة TDD: أحمر ← أخضر ← إعادة هيكلة

نصيحة خبير: التطوير الموجَّه بالاختبار (Test-Driven Development - TDD)

منهجية TDD تقترح كتابة **الاختبار أولاً**، قبل كتابة الكود الفعلي، بدورة من ثلاث خطوات:

1. 🔴 **Red**: اكتب اختبارًا يفشل (لأن الكود غير موجود بعد).
2. 🟢 **Green**: اكتب أبسط كود ممكن لجعل الاختبار ينجح.
3. 🔵 **Refactor**: حسّن الكود (تطبيق مبادئ الفصل 24) مع إبقاء الاختبار ناجحًا.

هذه المنهجية ليست إلزامية للجميع، لكنها تُجبرك على التفكير بوضوح في «ماذا يُفترض أن يفعل الكود بالضبط» **قبل** الغرق في التفاصيل التقنية. جرّبها على مشاريعك الصغيرة القادمة لترى إن كانت تناسب أسلوبك.

⚠ تحذير: لا تنتظر «حتى ينتهي المشروع» لكتابة الاختبارات

الاعتقاد الشائع الخاطئ: «سأكتب الاختبارات بعد أن أنهي كل الميزات.» في الواقع، هذا نادرًا ما يحدث — الضغط لتسليم ميزات جديدة يجعل الاختبارات تُؤجَّل إلى الأبد. الممارسة الأفضل: اكتب اختبارًا **لكل دالة جديدة مهمة فور الانتهاء منها**، كجزء طبيعي من عملية التطوير، وليس كخطوة منفصلة «لاحقة».

25.13 تمارين

🟢 مستوى مبتدئ

1. اكتب دالة `is_palindrome(text: str) → bool` تتحقق إن كانت الكلمة تُقرأ نفسها من الاتجاهين، ثم اكتب 4 اختبارات لها (حالة صحيحة، حالة خاطئة، نص فارغ، نص بحرف واحد).
2. استخدم `pytest.raises` لاختبار أن دالة `calculate_average(numbers: list)` ترفع `ValueError` عند استقبال قائمة فارغة.
3. استخدم `@pytest.mark.parametrize` لاختبار دالة `is_even(n: int) → bool` بـ 5 قيم مختلفة (زوجية وفردية، موجبة وسالبة، صفر).

✅ الحل المقترح

Python

```python
# دوال للاختبار (ضعها في ملف منفصل src/utils.py)
def is_palindrome(text: str) -> bool:
    cleaned = text.lower().replace(" ", "")
    return cleaned == cleaned[::-1]

def calculate_average(numbers: list) -> float:
    if not numbers:
        raise ValueError("القائمة لا يمكن أن تكون فارغة")
    return sum(numbers) / len(numbers)

def is_even(n: int) -> bool:
    return n % 2 == 0

# الاختبارات
import pytest
from utils import is_palindrome, calculate_average, is_even

def test_is_palindrome_true_case():
    assert is_palindrome("مدم") is True

def test_is_palindrome_false_case():
    assert is_palindrome("بيت") is False

def test_is_palindrome_empty_string():
    assert is_palindrome("") is True

def test_is_palindrome_single_char():
    assert is_palindrome("أ") is True

def test_calculate_average_empty_list_raises_error():
    with pytest.raises(ValueError):
        calculate_average([])

@pytest.mark.parametrize("n, expected", [
    (2, True), (3, False), (0, True), (-4, True), (-7, False),
])
def test_is_even(n, expected):
    assert is_even(n) == expected
```

🟡 مستوى متوسط

4. أضف fixture `sample_products` تُرجع قائمة من 3 منتجات تجريبية، ثم اكتب اختبارًا لدالة تحسب متوسط أسعارها.

5. اكتب اختبارًا يستخدم `unittest.mock.patch` لمحاكاة دالة `datetime.now()` بحيث تُرجع تاريخًا ثابتًا، لاختبار دالة تعتمد على التاريخ الحالي بشكل متسق (بدون اعتماد على التاريخ الفعلي وقت تشغيل الاختبار).

<details>
<summary>✅ الحل المقترح</summary>

Python

```python
# 4
import pytest

@pytest.fixture
def sample_products():
    return [
        {"name": "أ", "price": 100},
        {"name": "ب", "price": 200},
        {"name": "ج", "price": 300},
    ]

def average_price(products: list[dict]) -> float:
    return sum(p["price"] for p in products) / len(products)

def test_average_price(sample_products):
    assert average_price(sample_products) == 200

# 5
from datetime import datetime
from unittest.mock import patch

def get_current_year() -> int:
    return datetime.now().year

def test_get_current_year_with_fixed_date():
    fixed_date = datetime(2030, 1, 1)
    with patch("__main__.datetime") as mock_datetime:
        mock_datetime.now.return_value = fixed_date
        assert get_current_year() == 2030
```

</details>

🔴 مستوى تحدي

6. وسّع مشروع إدارة المخزون: أضف دالة جديدة `restock(sku: str, quantity: int)` تزيد كمية منتج موجود، مع رفع `ValueError` إذا كانت الكمية المُضافة غير موجبة. اكتب مجموعة اختبارات كاملة لها (الحالة الطبيعية، الكمية الصفرية/السالبة، منتج غير موجود)، واحرص على تحقيق تغطية 100% لهذه الدالة الجديدة تحديدًا.

✅ الحل المقترح

Python

```python
# إضافة إلى inventory_manager.py
def restock(self, sku: str, quantity: int) -> None:
    """زيادة كمية منتج موجود في المخزون.

    Raises:
        ProductNotFoundError: إذا لم يوجد المنتج.
        ValueError: إذا كانت الكمية المُضافة غير موجبة.
    """
    if quantity <= 0:
        raise ValueError("كمية التزويد يجب أن تكون موجبة")

    product = self.get_product(sku)  # يرفع ProductNotFoundError تلقائيًا إن لم يوجد
    product.quantity += quantity

# الاختبارات
class TestRestock:
    def test_restock_increases_quantity(self, manager_with_products):
        manager_with_products.restock("KEY001", 10)
        product = manager_with_products.get_product("KEY001")
        assert product.quantity == 13

    @pytest.mark.parametrize("invalid_quantity", [0, -1, -50])
    def test_restock_non_positive_raises_error(self, manager_with_products,
invalid_quantity):
        with pytest.raises(ValueError):
            manager_with_products.restock("KEY001", invalid_quantity)

    def test_restock_nonexistent_product_raises_error(self, manager_with_products):
        with pytest.raises(ProductNotFoundError):
            manager_with_products.restock("NOTEXIST", 10)
```

25.14 اختبر فهمك (Quiz)

1. ما معنى نمط Arrange-Act-Assert وما فائدته؟

هو نمط تنظيمي لكتابة الاختبارات بثلاث خطوات واضحة: Arrange (تحضير البيانات والظروف اللازمة)، Act (تنفيذ الكود المراد اختباره)، Assert (التحقق من أن النتيجة مطابقة للمتوقع). فائدته أنه يجعل كل اختبار واضح البنية وسهل القراءة والفهم، ويفصل بوضوح بين «ماذا نحضّر» و«ماذا نختبر فعليًا».

2. ما وظيفة الـ Fixtures في pytest ولماذا هي مفيدة؟

الـ Fixtures دوال خاصة تُحضّر بيانات أو موارد (كاتصال وهمي، كائن تجريبي، ملف مؤقت) وتُمرّرها تلقائيًا لدوال الاختبار التي تحتاجها، عبر تمريرها كمعامل بنفس الاسم. فائدتها الأساسية هي تجنّب تكرار كود «التحضير» في كل اختبار على حدة، وتوحيد طريقة إعداد البيانات المشتركة بين عدة اختبارات.

3. متى يجب استخدام Mocking بدلاً من استدعاء الكود الحقيقي مباشرة في الاختبار؟

عند الاعتماد على موارد خارجية بطيئة، غير موثوقة، أو مكلفة، مثل طلبات الشبكة الحقيقية، قواعد البيانات الفعلية، أو خدمات خارجية (كإرسال بريد إلكتروني). استخدام Mocking يجعل الاختبارات سريعة وموثوقة ومستقلة تمامًا عن توفر هذه الموارد الخارجية وقت تشغيل الاختبار.

4. لماذا تغطية الاختبار 100% ليست دليلًا كافيًا على أن الكود خالٍ من الأخطاء؟

لأن التغطية تقيس فقط أن كل سطر من الكود تم **تنفيذه** مرة واحدة على الأقل أثناء الاختبارات، لكنها لا تضمن أن كل **سيناريو منطقي ممكن** (بما فيه الحالات الحدّية والاستثنائية) قد اختُبر بعمق. يمكن تحقيق تغطية 100% باختبارات سطحية جدًا تتجاهل حالات مهمة.

5. ما الفرق بين استخدام `@pytest.mark.parametrize` وكتابة عدة دوال اختبار منفصلة لنفس المنطق؟

`parametrize` يسمح بتشغيل نفس منطق الاختبار بعدة مجموعات من المدخلات والنتائج المتوقعة من خلال دالة اختبار واحدة فقط، مما يقلل تكرار الكود (تطبيق مبدأ DRY) ويجعل إضافة حالة اختبار جديدة بسيطة (مجرد سطر جديد في القائمة)، بدلاً من نسخ ولصق دالة اختبار كاملة لكل حالة جديدة.

25.15 ملخص الفصل (Cheat Sheet)

Python

```python
# ===== الأساسيات =====
def test_something():                    # يجب أن يبدأ بـ test_
    assert actual == expected

# ===== اختبار الاستثناءات =====
import pytest
with pytest.raises(ValueError):
    risky_function()
with pytest.raises(ValueError, match="رسالة محددة"):
    risky_function()

# ===== الأعداد العشرية =====
assert 0.1 + 0.2 == pytest.approx(0.3)

# ===== Fixtures =====
@pytest.fixture
def my_data():
    return {"key": "value"}

def test_uses_fixture(my_data):
    assert my_data["key"] == "value"

@pytest.fixture
def resource_with_cleanup():
    resource = setup()
    yield resource
    teardown(resource)

# ===== Parametrize =====
@pytest.mark.parametrize("a, b, expected", [(1,1,2), (2,2,4)])
def test_add(a, b, expected):
    assert add(a, b) == expected
```

```python
# ===== Mocking =====
from unittest.mock import patch, MagicMock
with patch("module.function_name") as mock_func:
    mock_func.return_value = "قيمة وهمية"

# ===== أوامر التشغيل =====
# uv run pytest                              تشغيل كل الاختبارات
# uv run pytest -v                           تفصيلي
# uv run pytest tests/test_x.py::test_y      اختبار محدد
# uv run pytest --cov=src                    مع تقرير التغطية
# uv run pytest -k "keyword"                 فلترة بالاسم
```

الملف/المفهوم	الدور
`test_*.py`	ملفات الاختبار (تُكتشف تلقائيًا)
`conftest.py`	fixtures مشتركة بين ملفات الاختبار
`pytest.fixture`	تحضير بيانات/موارد للاختبارات
`pytest.raises`	اختبار أن كودًا يرفع استثناءً متوقعًا
`pytest.mark.parametrize`	تشغيل نفس الاختبار بمدخلات متعددة
`unittest.mock.patch`	محاكاة اعتماديات خارجية

25.16 ماذا بعد؟

لقد تعلّمت الآن المهارة التي تُميّز المبرمج المحترف عن الهاوي بشكل حاسم: **الثقة في كودك**، مدعومة باختبارات آلية تتحقق من صحته باستمرار. مع كل ما تعلمته من كود نظيف (الفصل 24) واختبارات شاملة (هذا الفصل)، أصبح لديك الآن الأساس الكامل لكتابة برمجيات احترافية جاهزة للعالم الحقيقي.

في الفصل الأخير من رحلتنا، سنُكمل الصورة: كيف تُحوّل مشروعك من مجرد «كود يعمل على جهازك» إلى **منتج حقيقي** يمكن للآخرين تثبيته واستخدامه؟ سنتعلّم أساسيات تحزيم المشاريع (Packaging)، نظرة سريعة على مفهوم النشر (Deployment)، ثم نختم بخارطة طريق شاملة لما بعد هذا الكتاب — إلى أين تذهب من هنا في رحلتك كمبرمج بايثون.

فصل 26 / 26 · الجزء السادس

26

من الكود إلى المنتج (Packaging، Deployment، وخارطة الطريق)

16 دقيقة قراءة | 18 قسمًا | 7 مثالًا | 3,168 كلمة

في هذا الفصل

26.1 مقدمة: اللحظة التي ينتظرها كل مبرمج
26.2 من «سكربت» إلى «منتج»: الفرق الجوهري
26.3 تشريح `pyproject.toml` الكامل
26.4 كتابة الكود القابل للنشر
26.5 بناء الحزمة (Build)
26.6 النشر على PyPI (و TestPyPI أولاً)
26.7 ملفات لا غنى عنها في أي حزمة احترافية
26.8 لمحة عن Deployment: من جهازك إلى الإنترنت
26.9 متغيرات البيئة وإدارة الأسرار
26.10 مقدمة إلى Docker: «يعمل على جهازي» تصبح من الماضي
26.11 كتابة Dockerfile لتطبيق FastAPI
26.12 لمحة سريعة: أين تُشغّل الحاوية فعليًا؟
26.13 لمحة عن CI/CD: الأتمتة الكاملة
26.14 مشروع تطبيقي متكامل: من مكتبة محلية إلى منتج منشور بالكامل
26.15 تمارين
26.16 اختبر فهمك (Quiz)
26.17 ملخص الفصل (Cheat Sheet)
26.18 خاتمة: نهاية الكتاب... وبداية رحلتك الحقيقية

26.1 مقدمة: اللحظة التي ينتظرها كل مبرمج

تخيّل هذا المشهد: أمضيت أسابيع في بناء أداة رائعة — ربما مكتبة لتحليل النصوص العربية، أو واجهة API لإدارة المهام، أو أداة سطر أوامر لتنظيم الملفات. الكود يعمل بشكل ممتاز على جهازك. الاختبارات كلها خضراء. الكود نظيف ومنظم وفق كل ما تعلمته في الفصلين الماضيين.

ثم يسألك صديق: «رائع! كيف أستخدمها أنا؟»

وهنا تقف صامتًا. لأن الإجابة الصادقة هي: *«يجب أن تنسخ الكود، تثبّت نفس الإصدار من بايثون، تثبّت نفس المكتبات يدويًا، وتدعو ألا يتعارض شيء مع إعدادات جهازك.»* هذه ليست إجابة مُرضية لمنتج حقيقي.

الفجوة بين **«كود يعمل عندي»** و **«منتج يستخدمه الآخرون»** هي بالضبط ما سنردمه في هذا الفصل الأخير. سنتعلم:

1. كيف نُحوّل مجلد الكود إلى **حزمة (Package)** رسمية يمكن تثبيتها بأمر واحد (`pip install` أو `uv add`)، وكيف ننشرها على **PyPI** (المتجر الرسمي لحزم بايثون).

2. لمحة عملية عن **Deployment**: كيف نأخذ تطبيق ويب ونُشغّله على خادم حقيقي يصل إليه أي شخص عبر الإنترنت، باستخدام **Docker** — المعيار العالمي لتعبئة التطبيقات.
3. وأخيرًا، **خارطة طريق** تُجيب على السؤال الذي سيدور في ذهنك بعد إغلاق هذا الكتاب: *«تعلّمتُ كل هذا... والآن ماذا؟»*

هذا الفصل مختلف قليلًا عن سابقيه؛ فهو لا يُعلّمك «لغة» بقدر ما يُعلّمك «طريق الخروج إلى العالم الحقيقي». لنبدأ.

26.2 من «سكربت» إلى «منتج»: الفرق الجوهري

حتى الآن، كل أكوادنا كانت تعمل بصيغة: افتح VS Code، شغّل `python main.py` أو `uv run main.py`، وشاهد النتيجة. هذا ممتاز للتعلم، لكنه غير كافٍ لثلاثة أسباب:

المشكلة	الشرح
قابلية التوزيع	لا يمكنك إرسال مجلد المشروع لصديق ويتوقع أن يعمل فورًا؛ قد يفتقد مكتبات أو يملك إصدار بايثون مختلف
قابلية إعادة الاستخدام	إذا أردت استخدام دالة من مشروعك في مشروع آخر، يجب نسخ الكود يدويًا — لا توجد طريقة لـ«استيرادها» كمكتبة
الوصول عبر الإنترنت	تطبيقات الويب (مثل FastAPI من الفصل 19) تحتاج لتعمل على خادم متاح 24 ساعة، وليس فقط على جهازك المحلي

الحل لهذه المشاكل الثلاث هو بالترتيب: **Packaging** (للمشكلتين الأولى والثانية)، و **Deployment** (للمشكلة الثالثة).

> **نصيحة الخبير:** لا تحتاج كل مشروع للتحزيم أو النشر. إذا كان المشروع سكربتًا شخصيًا بسيطًا تستخدمه أنت فقط، فلا داعي للتعقيد. التحزيم والنشر يستحقان الجهد عندما يكون هدفك مشاركة عملك مع الآخرين — سواء كانوا مستخدمين أو مطورين آخرين.

26.3 تشريح `pyproject.toml` الكامل

تشريح pyproject.toml

```
[build-system]
build-backend = "hatchling"

[project]
name = "my-pkg"

dependencies
["requests⩾2"]

[project.scripts]
mycli = "pkg.cli:main"
```

الملف الواحد يجمع البناء والبيانات والتبعيات والأوامر.

شكل 26.1 — تشريح pyproject.toml

تعرّفنا على `pyproject.toml` بشكل سريع في الفصل 18 كملف تُنشئه أداة `uv` تلقائيًا. الآن حان وقت فهمه بعمق، لأنه **جواز سفر** مشروعك إلى العالم الخارجي.

لنفترض أننا بدأنا مشروع مكتبة جديدة:

Terminal

```
uv init --lib textkit
cd textkit
```

مقارنة: `uv init --lib` مقابل الافتراضي

في الفصل 18 استخدمنا `uv init` العادي لإنشاء تطبيق (Application). أما `uv init --lib` فيُنشئ هيكلية **مكتبة (Library)** جاهزة للنشر، بفارق جوهري: الكود يوضع مباشرة تحت `src/textkit/` مع ملف `__init__.py` يُصدّر الدوال العامة، وهذا هو الشكل القياسي لأي حزمة تُنشر على PyPI.

الهيكل الناتج:

```
textkit/
├── pyproject.toml
├── README.md
├── src/
│   └── textkit/
│       ├── __init__.py
│       └── py.typed
└── tests/
```

> **ما هو `py.typed`؟** ملف فارغ تمامًا، لكنه علامة رسمية (PEP 561) تُخبر أدوات مثل `mypy` و Pylance أن حزمتك تحتوي Type Hints يمكن الوثوق بها عند استيرادها في مشروع آخر. أي مكتبة احترافية حديثة يجب أن تتضمنه.

الآن لنفتح `pyproject.toml` ونفهم كل قسم فيه:

TOML

```toml
[project]
name = "textkit"
version = "0.1.0"
description = "مجموعة أدوات بسيطة لمعالجة النصوص العربية والإنجليزية"
readme = "README.md"
requires-python = "⩾3.12"
license = { text = "MIT" }
authors = [
    { name = "اسمك هنا", email = "you@example.com" }
]
keywords = ["text", "nlp", "arabic", "utilities"]
classifiers = [
    "Programming Language :: Python :: 3",
    "Programming Language :: Python :: 3.12",
    "License :: OSI Approved :: MIT License",
    "Operating System :: OS Independent",
    "Intended Audience :: Developers",
]
dependencies = []

[project.optional-dependencies]
dev = [
    "pytest⩾8.0",
    "pytest-cov⩾5.0",
    "ruff⩾0.5",
    "black⩾24.0",
    "mypy⩾1.10",
]

[project.urls]
Homepage = "https://github.com/username/textkit"
Repository = "https://github.com/username/textkit"
Issues = "https://github.com/username/textkit/issues"

[build-system]
requires = ["hatchling"]
build-backend = "hatchling.build"
```

لنُفكك كل جزء:

القسم	الغرض
`[project]`	البيانات الوصفية الأساسية: الاسم، الإصدار، الوصف، المؤلف
`requires-python`	أقل إصدار بايثون متوافق — pip سيرفض التثبيت على إصدار أقدم
`classifiers`	وسوم معيارية يستخدمها PyPI للفهرسة والبحث (القائمة الكاملة في pypi.org/classifiers)
`dependencies`	المكتبات المطلوبة *لتشغيل* الحزمة (فارغة هنا لأن مكتبتنا بسيطة)
`[project.optional-dependencies]`	مجموعات تبعيات اختيارية، مثل `dev` للتطوير فقط
`[project.urls]`	روابط تظهر في صفحة الحزمة على PyPI
`[build-system]`	يُحدد الأداة المسؤولة عن *بناء* الحزمة (هنا: `hatchling`، وهي الافتراضية مع uv)

⚠ **خطأ شائع: نسيان رفع رقم الإصدار**
إذا حاولت النشر على PyPI برقم إصدار سبق استخدامه (مثل `0.1.0` مرتين)، سيُرفض الطلب فورًا — PyPI **لا يسمح** بالكتابة فوق إصدار منشور سابقًا، حتى لو حذفته. هذا إجراء أمان مقصود لحماية من يعتمدون على حزمتك. الحل: اتبع **الترقيم الدلالي (Semantic Versioning)** دائمًا.

الترقيم الدلالي (Semantic Versioning)

الصيغة القياسية هي `MAJOR.MINOR.PATCH` (مثل `2.4.1`):

الجزء	يتغيّر متى؟	مثال
MAJOR	تغييرات جذرية تكسر التوافق مع الإصدارات السابقة	`1.0.0 → 2.0.0`
MINOR	إضافة ميزة جديدة متوافقة مع القديم	`1.2.0 → 1.3.0`
PATCH	إصلاح خطأ (Bug Fix) دون تغيير السلوك المتوقع	`1.2.3 → 1.2.4`

💡 **نصيحة الخبير:** قبل الوصول إلى `1.0.0`، تُعتبر الحزمة في طور «عدم استقرار الواجهة» بشكل ضمني — استخدم `0.x.y` للإشارة إلى أن الواجهة ما تزال قابلة للتغيير الكبير. أغلب المكتبات الشهيرة (بما فيها FastAPI لسنوات طويلة) بقيت تحت `0.x` حتى اكتمل نضجها.

26.4 كتابة الكود القابل للنشر

لنملأ مكتبتنا `textkit` بمحتوى حقيقي. في `src/textkit/formatters.py`:

Python

```
"""أدوات معالجة وتنسيق النصوص."""

import re

def slugify(text: str) -> str:
    """
    يحوّل نصًا إلى صيغة "slug" مناسبة للروابط.

  >>>   slugify("Hello World!")
    'hello-world'
    >>> slugify("  Python 3.14  Guide  ")
    'python-3-14-guide'
    """
    text = text.strip().lower()
    text = re.sub(r"[^\w\s-]", "", text)
    text = re.sub(r"[\s_]+", "-", text)
    return text.strip("-")

def truncate(text: str, max_length: int, suffix: str = "...") -> str:
    """
    يختصر نصًا طويلاً ويضيف لاحقة إن تجاوز الطول المحدد.

  >>>   truncate("Python is amazing", 10)
    'Python is...'
    """
    if len(text) ⩽ max_length:
        return text
    return text[:max_length].rstrip() + suffix

def word_count(text: str) -> int:
    """يُحصي عدد الكلمات في نص."""
    return len(text.split())

def is_palindrome(text: str) -> bool:
    """
    يتحقق إن كان النص متناظرًا (يُقرأ نفسه عكسيًا).

  >>>   is_palindrome("Was it a car or a cat I saw")
    True
    """
    cleaned = re.sub(r"[^\w]", "", text.lower())
    return cleaned == cleaned[::-1]
```

لاحظ استخدام الأمثلة بصيغة **Doctest** (الأسطر التي تبدأ بـ `>>>` داخل الـ docstring). هذه ليست مجرد توثيق — يمكن تشغيلها فعليًا كاختبارات عبر `python -m doctest`، وهي أداة توثيق قوية نادرًا ما يستغلها المبتدئون.

الآن في `src/textkit/__init__.py`، نُصدّر الدوال العامة بوضوح:

```python
"""textkit: مجموعة أدوات بسيطة لمعالجة النصوص."""

from textkit.formatters import is_palindrome, slugify, truncate, word_count

__version__ = "0.1.0"

__all__ = [
    "slugify",
    "truncate",
    "word_count",
    "is_palindrome",
]
```

نصيحة الخبير: لماذا `__all__`؟ هذه القائمة تُحدد بوضوح ما الذي يُعتبر «واجهة عامة» للمكتبة عند كتابة `from textkit import *`. أكثر أهمية من ذلك: محررات الأكواد (مثل Pylance) تستخدمها لاقتراح الأسماء الصحيحة للمستخدم، مما يجعل تجربة استخدام مكتبتك أكثر احترافية.

مقارنة: `__version__` داخل الكود مقابل `pyproject.toml`
لاحظ أننا كتبنا رقم الإصدار في مكانين (`pyproject.toml` و `__init__.py`). هذا تكرار قد يُسبب تضاربًا إن نسيت تحديث أحدهما. الحل الاحترافي: استخدام أدوات مثل `hatch-vcs` التي تستنتج الإصدار تلقائيًا من Git tags، أو ببساطة الاكتفاء بمصدر واحد والقراءة منه ديناميكيًا عبر `importlib.metadata.version("textkit")`. للمشاريع الصغيرة، التكرار اليدوي مقبول طالما تتذكره في كل إصدار.

الآن نكتب اختبارات (كما تعلمنا في الفصل 25) في `tests/test_formatters.py`:

Python

```python
import pytest

from textkit import is_palindrome, slugify, truncate, word_count

class TestSlugify:
    def test_simple_sentence(self) -> None:
        assert slugify("Hello World!") == "hello-world"

    def test_multiple_spaces(self) -> None:
        assert slugify("  Python   Guide  ") == "python-guide"

    def test_already_slug(self) -> None:
        assert slugify("already-a-slug") == "already-a-slug"

class TestTruncate:
    @pytest.mark.parametrize(
        "text, max_length, expected",
        [
            ("Python", 10, "Python"),
            ("Python is amazing", 10, "Python is..."),
            ("", 5, ""),
        ],
    )
    def test_truncate_cases(
        self, text: str, max_length: int, expected: str
    ) -> None:
        assert truncate(text, max_length) == expected

def test_word_count() -> None:
    assert word_count("Python is fun") == 3

def test_is_palindrome() -> None:
    assert is_palindrome("A man a plan a canal Panama") is True
    assert is_palindrome("Hello") is False
```

نُشغّل الاختبارات ونتأكد من النجاح قبل أي خطوة أخرى:

Terminal

```
uv run pytest --cov=src --cov-report=term-missing
```

> ⚠ **تحذير: لا تنشر حزمة بدون اختبارات**
> النشر على PyPI يعني أن **أي شخص في العالم** قد يعتمد على حزمتك في مشروعه. حزمة بدون اختبارات هي قنبلة موقوتة — أي تعديل بسيط لاحقًا قد يكسر شيئًا لم تنتبه له. اجعل تشغيل الاختبارات خطوة إلزامية قبل كل نشر جديد (وسنرى لاحقًا كيف نُؤتمت هذا عبر CI/CD).

26.5 بناء الحزمة (Build)

«البناء» يعني تحويل كود المصدر إلى صيغتين موزّعتين قياسيًا:

الصيغة	الامتداد	الوصف
Source Distribution (sdist)	`.tar.gz`	أرشيف يحتوي الكود المصدري كاملاً، يحتاج بناءً محليًا عند التثبيت
Wheel	`.whl`	صيغة «جاهزة للتثبيت» مُجمّعة مسبقًا — أسرع في التثبيت، وهي المفضّلة اليوم

مع `uv`، البناء أمر واحد:

```
Terminal
uv build
```

الناتج يظهر في مجلد `dist/`:

```
dist/
├── textkit-0.1.0-py3-none-any.whl
└── textkit-0.1.0.tar.gz
```

> **نصيحة الخبير:** يمكنك تثبيت الحزمة محليًا من ملف الـ wheel مباشرة للتأكد من أنها تعمل قبل نشرها للعالم:
>
> ```
> Terminal
> uv pip install dist/textkit-0.1.0-py3-none-any.whl
> ```
>
> جرّب هذا دائمًا في بيئة افتراضية نظيفة (مشروع فارغ جديد) — فقد تنسى إضافة تبعية في `pyproject.toml` وتعمل الحزمة «بالصدفة» على جهازك لأنها مثبّتة أصلاً لغرض آخر.

26.6 النشر على PyPI (و TestPyPI أولاً)

PyPI (Python Package Index، يُنطق «باي-بي-آي») هو المستودع الرسمي الذي يسحب منه `pip` و `uv` كل الحزم. عندما تكتب `uv add requests`، فإن `requests` تُسحب من PyPI.

> ⚠️ **تحذير حاسم: استخدم TestPyPI أولاً دائمًا**
> PyPI الحقيقي لا يسمح بحذف أو إعادة استخدام أرقام الإصدارات. قبل نشر أي شيء حقيقي، تدرّب على **TestPyPI** — نسخة طبق الأصل مخصصة للتجربة والتعلم، متاحة على test.pypi.org.

الخطوات العملية

1. إنشاء حساب على pypi.org و test.pypi.org (حسابان منفصلان).

2. إنشاء API Token من إعدادات الحساب (Account Settings → API tokens) بدلاً من استخدام كلمة المرور مباشرة — هذه هي الطريقة الآمنة المعتمدة اليوم.

3. النشر على TestPyPI للتجربة:

```
Terminal
uv publish --publish-url https://test.pypi.org/legacy/ --token pypi-xxxxxxxx
```

4. التحقق من التثبيت من TestPyPI:

```
Terminal
uv pip install --index-url https://test.pypi.org/simple/ textkit
```

5. عند الرضا التام، النشر الحقيقي على PyPI:

```
Terminal
uv publish --token pypi-xxxxxxxx
```

وبعدها، أي شخص في العالم يمكنه كتابة:

```
Terminal
uv add textkit
# أو
pip install textkit
```

وستُصبح مكتبتك جزءًا من النظام البيئي العالمي لبايثون! 🥕

نصيحة الخبير: لا تضع الـ Token في الكود مباشرة

لا تكتب التوكن في أمر الطرفية بشكل دائم (قد يُحفظ في سجل الأوامر `history`). الممارسة الأفضل هي تخزينه كمتغير بيئة:

```
Terminal
export UV_PUBLISH_TOKEN="pypi-xxxxxxxx"
uv publish
```

أو — الأفضل من ذلك كله — تفويض النشر لـ GitHub Actions (سنراه في القسم 26.9) بحيث لا يلمس التوكن جهازك الشخصي إطلاقًا.

مقارنة: `uv publish` مقابل `twine`

قبل ظهور `uv`، كانت الأداة القياسية للنشر هي `twine` (`pip install twine` ثم `twine upload dist/*`). ما تزال تعمل وتُستخدم في مشاريع كثيرة قديمة، لكن `uv publish` تدمج البناء والنشر في تجربة واحدة أبسط ومتوافقة مع فلسفة أدوات uv الموحّدة.

26.7 ملفات لا غنى عنها في أي حزمة احترافية

الملف	الغرض
`README.md`	أول شيء يراه الزائر على PyPI أو GitHub — يجب أن يشرح: ما هي المكتبة، كيف تُثبَّت، مثال استخدام سريع
`LICENSE`	رخصة الاستخدام القانونية. بدونها، من الناحية القانونية لا يحق لأحد استخدام كودك رسميًا!
`CHANGELOG.md`	سجل بالتغييرات في كل إصدار، يساعد المستخدمين على فهم ما تغيّر عند الترقية
`.gitignore`	يمنع رفع ملفات غير مرغوبة لـ Git (`__pycache__`, `.venv`, `dist/`, إلخ)

مثال `LICENSE` (رخصة MIT، الأكثر شيوعًا للمشاريع مفتوحة المصدر)

```
MIT License

Copyright (c) 2025 اسمك هنا

Permission is hereby granted, free of charge, to any person obtaining a copy
of this software and associated documentation files (the "Software"), to
deal in the Software without restriction, including without limitation the
rights to use, copy, modify, merge, publish, distribute, sublicense, and/or
sell copies of the Software...
```

نصيحة الخبير: لا تحتاج لكتابة نص الرخصة يدويًا — موقع choosealicense.com يساعدك على اختيار الرخصة المناسبة ويوفر النص الكامل جاهزًا للنسخ. MIT هي الأكثر تساهلاً وشيوعًا؛ Apache 2.0 تضيف حماية من براءات الاختراع؛ GPL تفرض أن أي مشروع يستخدم كودك يجب أن يبقى مفتوح المصدر أيضًا.

مثال `README.md` مختصر وفعّال

```Markdown
# textkit

مجموعة أدوات بسيطة وسريعة لمعالجة النصوص بلغة بايثون.

## التثبيت

```bash
pip install textkit
```

26.8 الاستخدام السريع

```Python
from textkit import slugify, truncate

slugify("Hello World!")        # "hello-world"
truncate("نص طويل جداً", 5)    # "نص ط..."
```

26.9 الترخيص

MIT
```

```
---

## 26.8 لمحة عن Deployment: من جهازك إلى الإنترنت

حزمة بايثون (مثل `textkit`) تُستخدم عبر الاستيراد. لكن تطبيقًا مثل واجهة FastAPI التي بنيناها بـ API في الفصل 19 يحتاج إلى شيء مختلف تمامًا: أن يبقى **يعمل باستمرار** على خادم يمكن لأي متصفح أو تطبيق الوصول إليه عبر الإنترنت، 24 ساعة في اليوم. هذا هو **Deployment** (النشر التشغيلي).

### لماذا لا يكفي تشغيل `uvicorn main:app` على جهازك؟

| المشكلة | الشرح |
|---|---|
| جهازك ليس متاحًا دائمًا | إذا أغلقت الحاسوب أو انقطع الإنترنت، يتوقف التطبيق |
| عام ثابت | لا يمكن لأحد خارج شبكتك المحلية الوصول إليه |
| لا يملك عنوان IP | «يعمل عندي» لا يعني أنه سيعمل على خادم بإصدار بايثون أو نظام تشغيل مختلف |
| بيئة غير متطابقة | |

الحل القياسي لمشكلة «البيئة غير المتطابقة» اليوم هو **Docker**.

---

## 26.9 متغيرات البيئة وإدارة الأسرار

**لا تضع المعلومات الحساسة في أي تطبيق حقيقي:** كلمات المرور، مفاتيح API، روابط قواعد البيانات) يجب فهم مبدأ أساسي في Docker، قبل الحديث عن داخل الكود مباشرة.**
```python
# ❌ خطأ فادح وشائع جدًا
DATABASE_URL = "postgresql://admin:my_secret_password@db.example.com/prod"
```
```

> ⚠ **تحذير أمني خطير:** إذا رفعت هذا الكود على GitHub (حتى في مستودع خاص)، فإن كلمة المرور أصبحت جزءًا من سجل Git إلى الأبد — حتى لو حذفتها لاحقًا، تبقى في التاريخ (history) ويمكن لأي شخص الوصول إليها. حوادث تسريب مفاتيح AWS وقواعد بيانات بهذه الطريقة تحدث يوميًا في العالم الحقيقي وتُكلّف شركات ملايين الدولارات.

الحل: متغيرات البيئة (Environment Variables)

نضع القيم الحساسة في ملف `.env` (لا يُرفع أبدًا على Git):

Terminal

```
# .env
DATABASE_URL=postgresql://admin:my_secret_password@db.example.com/prod
SECRET_KEY=a1b2c3d4e5f6
DEBUG=False
```

ونضيفه فورًا إلى `.gitignore`:

```
# .gitignore
.env
__pycache__/
.venv/
dist/
*.egg-info/
```

نقرأ القيم في الكود عبر مكتبة `python-dotenv` :

Terminal

```
uv add python-dotenv
```

Python

```python
from dotenv import load_dotenv
import os

load_dotenv()  # يقرأ ملف .env ويضع قيمه في متغيرات البيئة

database_url = os.environ["DATABASE_URL"]
secret_key = os.environ.get("SECRET_KEY", "default-fallback-key")
```

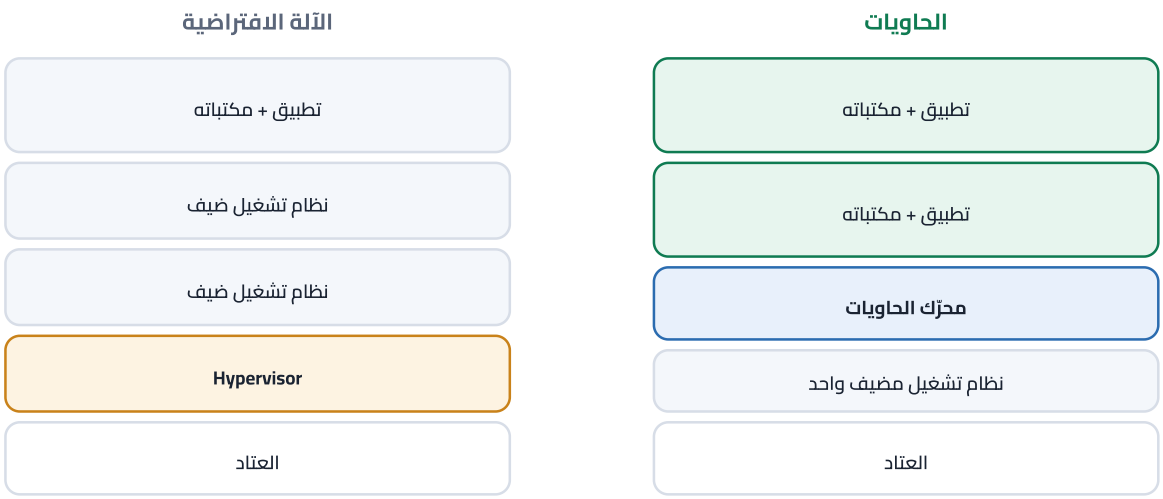
نصيحة الخبير: في مشاريع FastAPI الحديثة، يُفضّل استخدام `pydantic-settings` بدلاً من `os.environ` المباشر — فهي توفر Type Hints وتحقّقًا تلقائيًا من القيم:

Python

```python
from pydantic_settings import BaseSettings

class Settings(BaseSettings):
    database_url: str
    secret_key: str
    debug: bool = False

    class Config:
        env_file = ".env"

settings = Settings()
```

⚠ **خطأ شائع:** رفع ملف `.env` عن طريق الخطأ لأنك أضفته لـ `.gitignore` بعد أن كان قد تمّت إضافته مسبقًا لـ Git. تحقّق دائمًا بالأمر `git status` قبل أول `commit`، وإن حدث الخطأ، استخدم `git rm --cached .env` فورًا ثم غيّر كل المفاتيح المسرّبة فورًا (لا يكفي حذفها من الكود).

26.10 مقدمة إلى Docker: «يعمل على جهازي» تصبح من الماضي



شكل 26.2 — الحاويات مقابل الآلات الافتراضية

المشكلة التي يحلّها Docker

عبارة «يعمل عندي لكن لا يعمل على الخادم» هي من أشهر جمل الإحباط في تاريخ البرمجة. السبب: فروقات دقيقة بين البيئات — إصدار بايثون مختلف، مكتبة نظام مفقودة، متغير بيئة منسي.

Docker يحل هذا بفكرة بسيطة وعبقرية: بدلاً من نشر الكود فقط، ننشر **حاوية (Container)** تحتوي كل شيء يحتاجه التطبيق ليعمل — نظام التشغيل المصغّر، بايثون بالإصدار الصحيح، كل المكتبات، والكود نفسه — كوحدة واحدة متكاملة تعمل **بنفس الطريقة تمامًا** على أي جهاز: حاسوبك، خادم الشركة، أو سحابة Amazon.

مقارنة: الحاويات (Containers) مقابل الآلات الافتراضية (Virtual Machines)

المعيار	Virtual Machine	Docker Container
تحتوي نظام تشغيل كامل؟	نعم (ثقيلة، جيجابايتات)	لا، تشارك نواة النظام المضيف (خفيفة، ميجابايتات)
وقت الإقلاع	دقائق	ثوانٍ
الاستخدام الشائع	عزل كامل بين أنظمة مختلفة	تعبئة تطبيق واحد ببيئته الدقيقة

المفاهيم الأساسية

المصطلح	الشرح
Image (صورة)	«القالب» الثابت الذي يحتوي كل ما يحتاجه التطبيق (كتوصيف وصفة طبخ)
Container (حاوية)	نسخة «حيّة» قيد التشغيل من الصورة (كالطبق المطبوخ فعليًا من الوصفة)
Dockerfile	ملف نصي يصف خطوة بخطوة كيف تُبنى الصورة
Docker Hub	مستودع عام لمشاركة الصور الجاهزة (مثل PyPI لكن للحاويات)

تثبيت Docker

نزّل **Docker Desktop** من docker.com (متوفر لـ Windows وMac وLinux). بعد التثبيت، تحقّق بالأمر:

```
Terminal
```

```
docker --version
```

26.11 كتابة Dockerfile لتطبيق FastAPI

شكل 26.3 — طبقات صورة Docker

لنُعد استخدام مشروع «API إدارة المهام» من الفصل 19 ونُحوّله إلى حاوية جاهزة للنشر.

هيكل المشروع (مُبسّط للتذكير):

```
task_api/
├── pyproject.toml
├── src/
│   └── task_api/
│       └── main.py
├── Dockerfile
└── .dockerignore
```

Dockerfile

```dockerfile
# Dockerfile

# 1. الصورة الأساسية: بايثون 3.14 بنسخة خفيفة (slim)
FROM python:3.14-slim

# 2. تثبيت uv داخل الحاوية
COPY --from=ghcr.io/astral-sh/uv:latest /uv /usr/local/bin/uv

# 3. تحديد مجلد العمل داخل الحاوية
WORKDIR /app

# 4. نسخ ملفات التبعيات أولاً (لتحسين الـ caching)
COPY pyproject.toml uv.lock ./

# 5. تثبيت التبعيات (بدون تثبيت المشروع نفسه بعد)
RUN uv sync --frozen --no-install-project --no-dev

# 6. نسخ باقي كود المشروع
COPY src/ ./src/

# 7. تثبيت المشروع نفسه الآن
RUN uv sync --frozen --no-dev

# 8. فتح المنفذ الذي سيستمع عليه التطبيق
EXPOSE 8000

# 9. أمر التشغيل الافتراضي عند بدء الحاوية
CMD ["uv", "run", "uvicorn", "task_api.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

نصيحة الخبير: لماذا ننسخ `pyproject.toml` قبل `src/`؟

هذا ترتيب مقصود وليس عشوائيًا! Docker يبني الصور طبقة (Layer) فوق طبقة، ويُعيد استخدام الطبقات غير المتغيّرة من عمليات بناء سابقة (caching). إذا غيّرت سطرًا واحدًا في كود `main.py` فقط، لن تُعاد تثبيت كل التبعيات من الصفر طالما لم يتغيّر `pyproject.toml` — هذا يوفّر دقائق ثمينة في كل عملية بناء.

نُنشئ أيضًا `.dockerignore` (تمامًا كمفهوم `.gitignore`، لكن لمنع نسخ ملفات غير ضرورية داخل الصورة):

```
# .dockerignore
.venv/
__pycache__/
*.pyc
.git/
.env
tests/
dist/
```

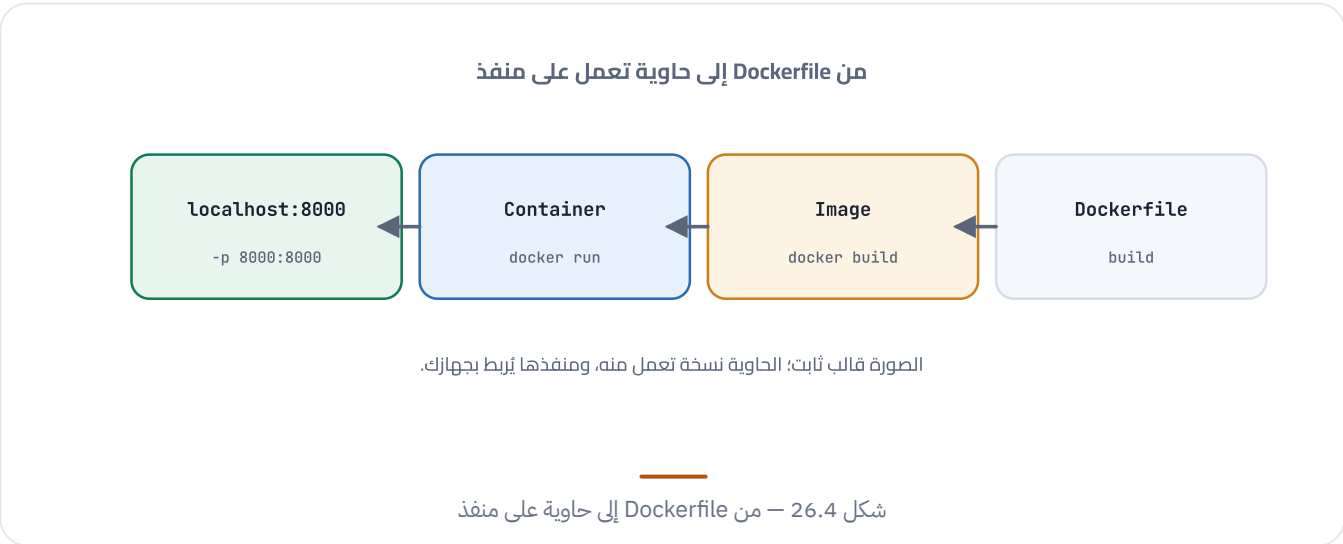
بناء وتشغيل الحاوية

Terminal

```
# بناء الصورة وإعطاؤها اسمًا (tag)
docker build -t task-api:1.0 .

# تشغيل حاوية من الصورة، وربط المنفذ 8000 بجهازك
docker run -p 8000:8000 task-api:1.0
```

الآن يمكنك زيارة `http://localhost:8000/docs` وستجد توثيق FastAPI التفاعلي يعمل — لكن هذه المرة من داخل حاوية Docker معزولة تمامًا عن بقية نظامك!

⚠ **خطأ شائع: نسيان `--host 0.0.0.0`**
إذا شغّلت uvicorn بالإعداد الافتراضي `--host 127.0.0.1` داخل الحاوية، فلن تستطيع الوصول إليه من خارجها إطلاقًا، لأن `127.0.0.1` تعني «استمع فقط من داخل هذا الجهاز نفسه» — والحاوية تعتبر «جهازًا» منفصلاً عن حاسوبك. يجب استخدام `0.0.0.0` للاستماع على كل الواجهات المتاحة.



شكل 26.4 — من Dockerfile إلى حاوية على منفذ

26.12 لمحة سريعة: أين تُشغّل الحاوية فعليًا؟

بعد أن أصبح تطبيقك «حاويًا»، يمكنك تشغيله على أي خدمة سحابية تدعم Docker. من أبسط الخيارات للمبتدئين (توفر خططًا مجانية أو رخيصة للمشاريع الصغيرة):

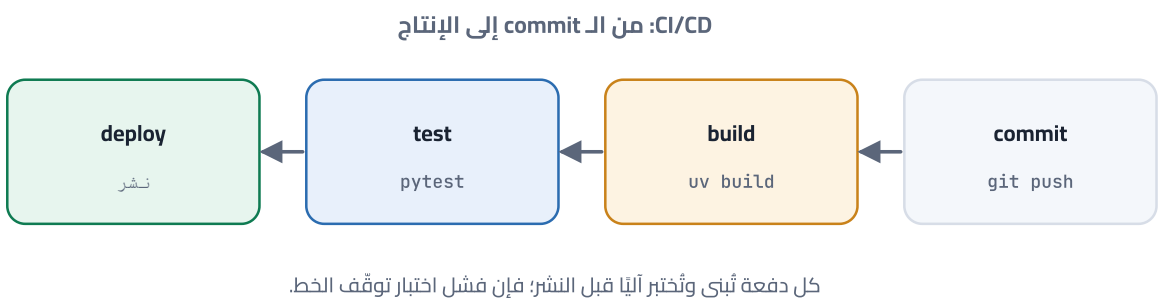
المنصة	الميزة الأبرز
Render	بسيطة جدًا، تربط مستودع GitHub مباشرة وتنشر تلقائيًا
Railway	واجهة سهلة، دعم ممتاز لقواعد البيانات المرفقة
Fly.io	قريبة من المستخدم جغرافيًا (CDN عالمي)، سطر أوامر قوي
AWS / Google Cloud / Azure	أقوى وأكثر مرونة، لكنها أكثر تعقيدًا، مناسبة للمشاريع الكبيرة والشركات

الخطوات العامة (تختلف التفاصيل بين منصة وأخرى) تكون عادة:

1. ربط حساب GitHub بالمنصة
2. اختيار المستودع الذي يحتوي الـ `Dockerfile`
3. ضبط متغيرات البيئة (من قسم 26.8) عبر لوحة تحكم المنصة (لا تُكتب في الكود أبدًا)
4. الضغط على "Deploy" ومراقبة السجلات (Logs)

> **نصيحة الخبير:** لا تحتاج لإتقان كل هذه المنصات. اختر واحدة (ننصح بـ Render للمبتدئين لبساطتها) وتعمّق فيها عبر مشروع تجريبي صغير حقيقي — ستتعلم المفاهيم العامة (متغيرات البيئة، السجلات، إعادة النشر التلقائي) التي تنتقل بسهولة لأي منصة أخرى لاحقًا.

26.13 لمحة عن CI/CD: الأتمتة الكاملة



شكل 26.5 — خط CI/CD من الـ commit إلى الإنتاج

CI/CD (Continuous Integration / Continuous Deployment) تعني: عند كل `git push`، تُنفَّذ تلقائيًا سلسلة من الخطوات — تشغيل الاختبارات، فحص جودة الكود، وربما حتى النشر المباشر — دون تدخل يدوي.

أشهر أداة مجانية ومدمجة مع GitHub هي **GitHub Actions**. نُنشئ ملف `.github/workflows/ci.yml`:

YAML

```yaml
name: CI

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - name: سحب الكود
        uses: actions/checkout@v4

      - name: تثبيت uv
        uses: astral-sh/setup-uv@v3

      - name: تثبيت بايثون 3.14
        run: uv python install 3.14

      - name: تثبيت التبعيات
        run: uv sync --all-extras --dev

      - name: فحص التنسيق بـ ruff
        run: uv run ruff check .

      - name: تشغيل الاختبارات مع التغطية
        run: uv run pytest --cov=src --cov-report=term-missing
```

من الآن فصاعدًا، كل مرة تدفع كودًا (`git push`) إلى GitHub، سيُشغّل هذا الملف تلقائيًا: يُثبّت بايثون والتبعيات، يفحص جودة الكود، ويُشغّل كل الاختبارات. إذا فشل أي شيء، سترى علامة ❌ حمراء على الـ commit مباشرة في GitHub، قبل أن يصل الخطأ لأي مستخدم حقيقي.

> **نصيحة الخبير:** هذه هي بالضبط الممارسة التي تفصل «الهواة» عن «المحترفين» في عالم تطوير البرمجيات. الشركات الكبرى (Google, Microsoft, Meta) تعتمد بالكامل على خطوط أنابيب CI/CD مشابهة، لكن بتعقيد أكبر بكثير. إتقان هذا المفهوم مبكرًا يضعك في مسار احترافي حقيقي.

26.14 مشروع تطبيقي متكامل: من مكتبة محلية إلى منتج منشور بالكامل

الهدف: أخذ مشروع بسيط عبر رحلته الكاملة: كتابة → اختبار → تحزيم → نشر تجريبي → حوسلة (Docker) → أتمتة (CI).

المتطلبات:

1. مكتبة `textkit` (من هذا الفصل) مع بنية كاملة جاهزة للنشر على TestPyPI.
2. تطبيق ويب بسيط بـ FastAPI يستخدم `textkit` كواجهة API (`/slugify` , `/truncate`).
3. `Dockerfile` لحوسلة التطبيق.
4. ملف GitHub Actions لتشغيل الاختبارات تلقائيًا.

الهيكل النهائي الكامل

```
textkit-project/
├── textkit/                    # المكتبة القابلة للنشر
│   ├── pyproject.toml
│   ├── README.md
│   ├── LICENSE
│   ├── src/
│   │   └── textkit/
│   │       ├── __init__.py
│   │       ├── formatters.py
│   │       └── py.typed
│   └── tests/
│       └── test_formatters.py
│
└── textkit_api/                # تطبيق الويب الذي يستهلك المكتبة
    ├── pyproject.toml
    ├── Dockerfile
    ├── .dockerignore
    ├── .github/
    │   └── workflows/
    │       └── ci.yml
    ├── src/
    │   └── textkit_api/
    │       └── main.py
    └── tests/
        └── test_main.py
```

✅ الحل النموذجي الكامل

`textkit/pyproject.toml` (مكتمل كما شرحناه في القسم 26.2، مع الإضافة):

```toml
[project]
name = "textkit"
version = "0.1.0"
description = "مجموعة أدوات بسيطة لمعالجة النصوص"
readme = "README.md"
requires-python = "≥3.12"
license = { text = "MIT" }
authors = [{ name = "اسمك هنا", email = "you@example.com" }]
dependencies = []

[project.optional-dependencies]
dev = ["pytest≥8.0", "pytest-cov≥5.0", "ruff≥0.5"]

[build-system]
requires = ["hatchling"]
build-backend = "hatchling.build"
```

`textkit/src/textkit/formatters.py` و `__init__.py`: كما وردا كاملين في القسمين 26.3 أعلاه.

`textkit/tests/test_formatters.py`: كما ورد كاملاً في القسم 26.3.

`textkit_api/pyproject.toml`:

TOML

```toml
[project]
name = "textkit-api"
version = "0.1.0"
description = "واجهة ويب لمكتبة textkit"
requires-python = "⩾3.12"
dependencies = [
    "fastapi⩾0.115",
    "uvicorn[standard]⩾0.30",
    "textkit @ file://../textkit",
]

[project.optional-dependencies]
dev = ["pytest⩾8.0", "httpx⩾0.27", "ruff⩾0.5"]

[build-system]
requires = ["hatchling"]
build-backend = "hatchling.build"
```

ملاحظة: `textkit @ file://../textkit` تُثبّت المكتبة المحلية مباشرة من مجلدها أثناء التطوير. عند النشر الحقيقي، تُستبدل هذا السطر بـ `"textkit⩾0.1.0"` بعد نشر `textkit` فعليًا على PyPI.

`textkit_api/src/textkit_api/main.py` :

Python

```python
"""واجهة API بسيطة تستخدم مكتبة textkit."""

from fastapi import FastAPI
from pydantic import BaseModel

from textkit import slugify, truncate

app = FastAPI(
    title="Textkit API",
    description="واجهة ويب لأدوات معالجة النصوص",
    version="0.1.0",
)

class TextRequest(BaseModel):
    text: str

class TruncateRequest(BaseModel):
    text: str
    max_length: int = 50

@app.get("/")
def read_root() -> dict[str, str]:
    return {"message": "مرحبًا بك في Textkit API"}

@app.post("/slugify")
def slugify_endpoint(request: TextRequest) -> dict[str, str]:
    return {"result": slugify(request.text)}

@app.post("/truncate")
def truncate_endpoint(request: TruncateRequest) -> dict[str, str]:
    return {"result": truncate(request.text, request.max_length)}
```

`textkit_api/tests/test_main.py` (اختبار تكامل بسيط باستخدام `TestClient`):

Python

```python
from fastapi.testclient import TestClient

from textkit_api.main import app

client = TestClient(app)

def test_read_root() -> None:
    response = client.get("/")
    assert response.status_code == 200
    assert "message" in response.json()

def test_slugify_endpoint() -> None:
    response = client.post("/slugify", json={"text": "Hello World!"})
    assert response.status_code == 200
    assert response.json() == {"result": "hello-world"}

def test_truncate_endpoint() -> None:
    response = client.post(
        "/truncate", json={"text": "Python is amazing", "max_length": 10}
    )
    assert response.status_code == 200
    assert response.json() == {"result": "Python is..."}
```

: `textkit_api/Dockerfile`

Dockerfile

```dockerfile
FROM python:3.14-slim

COPY --from=ghcr.io/astral-sh/uv:latest /uv /usr/local/bin/uv

WORKDIR /app

COPY pyproject.toml uv.lock ./
COPY ../textkit ./textkit

RUN uv sync --frozen --no-dev

COPY src/ ./src/

EXPOSE 8000

CMD ["uv", "run", "uvicorn", "textkit_api.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

: `textkit_api/.dockerignore`

```
.venv/
__pycache__/
*.pyc
.git/
.env
tests/
```

`textkit_api/.github/workflows/ci.yml` :

YAML

```yaml
name: CI

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: astral-sh/setup-uv@v3
      - run: uv python install 3.14
      - run: uv sync --all-extras --dev
      - run: uv run ruff check .
      - run: uv run pytest -v
```

تشغيل المشروع محليًا للتأكد من كل شيء:

Terminal

```
# 1. اختبار المكتبة
cd textkit
uv run pytest --cov=src --cov-report=term-missing

# 2. بناء المكتبة
uv build

# 3. اختبار الـ API
cd ../textkit_api
uv run pytest -v

# 4. تشغيل الـ API محليًا
uv run uvicorn textkit_api.main:app --reload

# 5. بناء وتشغيل حاوية Docker
docker build -t textkit-api:1.0 .
docker run -p 8000:8000 textkit-api:1.0
```

26.15 تمارين

مستوى مبتدئ

1. أنشئ مكتبة بسيطة جديدة باسم `math_helpers` تحتوي دالتين: `fibonacci(n: int) → list[int]` و `is_prime(n: int) → bool`. أنشئ لها `pyproject.toml` كاملاً واكتب لها اختبارات تغطي 100% من الكود.

2. اكتب ملف `.gitignore` مناسب لمشروع بايثون نموذجي (يتضمن `.venv/` , `__pycache__/` , `.env` , `dist/`).

3. اختر رخصة مناسبة لمشروعك الشخصي من choosealicense.com واشرح بجملتين لماذا اخترتها.

🟢 مستوى متوسط

4. خذ مكتبة `math_helpers` من التمرين الأول وابنِها (`uv build`) ثم انشرها على **TestPyPI**. تحقّق من نجاح النشر بتثبيتها في بيئة افتراضية جديدة والاستيراد منها.

5. اكتب `Dockerfile` لتطبيق CLI بسيط (مثل أداة `forganize` من الفصل 23) بحيث يمكن تشغيله كـ:

```
Terminal
```

```
docker run my-cli-tool --help
```

6. أضف ملف GitHub Actions لمشروعك الشخصي (أي مشروع سابق في هذا الكتاب) بحيث تُشغَّل الاختبارات تلقائيًا عند كل `push`.

🟡 تحدي

7. صمّم Dockerfile **متعدد المراحل (Multi-stage build)** لتطبيق FastAPI، بحيث تكون مرحلة البناء منفصلة عن مرحلة التشغيل النهائية، مما يُنتج صورة نهائية أصغر حجمًا (ابحث عن `FROM ... AS builder` كنقطة انطلاق).

8. انشر تطبيق `textkit_api` (من المشروع التطبيقي) فعليًا على منصة مجانية مثل Render أو Railway، واحصل على رابط عام يعمل عليه. وثّق خطواتك في ملف `DEPLOYMENT.md`.

9. أضف خطوة في ملف GitHub Actions تقوم بنشر الحزمة تلقائيًا على PyPI عند إنشاء "Release" جديد على GitHub (ابحث عن `on: release: types: [published]` و `pypa/gh-action-pypi-publish`).

26.16 اختبر فهمك (Quiz)

1. ما الفرق بين sdist و wheel؟

sdist (Source Distribution) هو أرشيف `.tar.gz` يحتوي الكود المصدري كاملاً ويحتاج بناءً عند التثبيت. **wheel** هو صيغة `.whl` جاهزة ومُجمَّعة مسبقًا، تُثبَّت بشكل أسرع وهي الصيغة المفضّلة حاليًا لمعظم الحزم.

2. لماذا لا يُسمح بإعادة استخدام رقم إصدار سبق نشره على PyPI؟

لضمان الاستقرار والأمان: إذا اعتمد مستخدمون على `mylib==1.0.0` واستطعت لاحقًا «استبدال» محتوى هذا الإصدار بكود مختلف، فقد يُكسر تطبيقهم فجأة دون أي تغيير ظاهر من جانبهم، أو قد يُستغل ذلك لحقن كود خبيث (Supply Chain Attack). لذلك PyPI يفرض أن كل إصدار منشور يبقى ثابتًا للأبد.

3. لماذا نضع ملفات `pyproject.toml` قبل نسخ كود المصدر في Dockerfile؟

للاستفادة من آلية الـ layer caching في Docker. إذا لم يتغيّر ملف التبعيات، يُعيد Docker استخدام طبقة التثبيت المخزّنة مسبقًا دون إعادة تثبيت كل المكتبات من جديد، مما يُسرّع عمليات البناء المتكررة بشكل كبير أثناء التطوير.

4. لماذا يجب تجنّب وضع كلمات المرور ومفاتيح API مباشرة في الكود؟

لأن الكود غالبًا يُرفع إلى أنظمة التحكم بالإصدار (مثل Git/GitHub)، وإذا كانت القيم الحساسة مكتوبة مباشرة، فإنها تصبح جزءًا دائمًا من تاريخ المستودع حتى لو حُذفت لاحقًا، مما يُعرّضها للتسريب والاستغلال. الحل القياسي هو تخزينها في متغيرات بيئة أو ملفات `.env` غير مرفوعة إلى Git.

5. ما الفائدة العملية من CI/CD في مشروع صغير يعمل عليه مطور واحد فقط؟

حتى في مشروع فردي، CI/CD يضمن تشغيل الاختبارات تلقائيًا على بيئة نظيفة ومحايدة (وليس فقط جهازك الذي قد يحتوي إعدادات أو ملفات قديمة تُخفي أخطاء)، ويمنع دمج كود معطوب بالخطأ، ويُوثّق تلقائيًا أن كل إصدار مرّ باختبارات ناجحة — وهي عادة احترافية تنمو معك عندما يكبر المشروع أو ينضم إليه مطورون آخرون.

26.17 ملخص الفصل (Cheat Sheet)

Markdown

```
# التحزيم (Packaging)
uv init --lib my_package       # إنشاء مكتبة جديدة بهيكل جاهز للنشر
uv build                       # بناء sdist + wheel في dist/
uv publish --publish-url https://test.pypi.org/legacy/ --token XXX  # نشر تجريبي
uv publish --token XXX         # نشر حقيقي على PyPI

# ملفات أساسية لأي حزمة احترافية
pyproject.toml   → البيانات الوصفية والتبعيات
README.md        → شرح الاستخدام
LICENSE          → الرخصة القانونية (MIT, Apache-2.0, ...)
CHANGELOG.md     → سجل التغييرات
py.typed         → علامة دعم Type Hints (PEP 561)

# الترقيم الدلالي
MAJOR.MINOR.PATCH
  MAJOR → تغيير جذري يكسر التوافق
  MINOR → ميزة جديدة متوافقة
  PATCH → إصلاح خطأ

# متغيرات البيئة
.env             → القيم الحساسة (لا ترفع لـ Git أبدًا!)
.gitignore       → يجب أن يتضمن .env, .venv/, __pycache__/
python-dotenv    → load_dotenv() لقراءة .env في بايثون

# Docker
docker build -t name:tag .     # بناء صورة من Dockerfile
docker run -p 8000:8000 name   # تشغيل حاوية وربط منفذ
FROM python:3.14-slim          # صورة بايثون أساسية خفيفة
EXPOSE 8000                    # توثيق المنفذ المستخدم
CMD [...]                      # أمر التشغيل الافتراضي للحاوية

# CI/CD (GitHub Actions)
.github/workflows/ci.yml       # يُشغَّل تلقائيًا عند push/pull_request
خطوات نموذجية: checkout → تثبيت uv/بايثون → sync → ruff check → pytest
```

26.18 خاتمة: نهاية الكتاب... وبداية رحلتك الحقيقية

لحظة مهمة: لقد وصلنا معًا إلى ختام **المتن الأساسي** لهذا الكتاب. انطلقنا من السؤال الأبسط — «كيف يُفكّر المبرمج؟» — ومررنا بالمتغيرات، النصوص، الحلقات، الدوال، البرمجة الكائنية، معالجة الأخطاء، الأنماط الحديثة في بايثون، ثم فتحنا نوافذ على خمسة مسارات احترافية كاملة: تطوير الويب، تحليل البيانات، الأتمتة، الذكاء الاصطناعي، وأدوات سطر

الأوامر. وأخيرًا، تعلّمنا كيف نكتب كودًا نظيفًا، كيف نختبره بثقة، وكيف نُحوّله إلى منتج حقيقي يصل للعالم.

هذا ليس «نهاية» بقدر ما هو **خط انطلاق**. اللغة التي تعلمتها أداة؛ وما ستبنيه بها الآن هو القصة الحقيقية.

تبقّت لدينا **خمسة ملاحق** ستكون مرجعك السريع في الأيام والأشهر القادمة:

- **الملحق أ:** قاموس المصطلحات البرمجية (عربي-إنجليزي) — لفك شيفرة أي مصطلح تقني تصادفه لاحقًا.
- **الملحق ب:** حلول جميع التمارين في الكتاب كاملة.
- **الملحق ج:** جدول مرجعي سريع (Cheat Sheet) شامل لكل ما تعلمته.
- **الملحق د:** خارطة طريق مفصّلة لما بعد هذا الكتاب — أي مسار تختار، وبأي ترتيب، وبأي مصادر.
- **الملحق هـ:** الأخطاء الشائعة الثلاثين التي يقع فيها كل مبرمج بايثون تقريبًا، مع حلولها.

الـملاحق · 3 فـصول

07

مراجع دائمة

ثلاثة ملاحق للرجوع إليها مرارًا: قاموس المصطلحات، حلول جميع التمارين، وجدول مرجعي سريع يلخّص الكتاب كله.

1 · قاموس المصطلحات البرمجية

2 · حلول جميع التمارين

3 · جدول مرجعي سريع

ملحق · ملاحق · الملاحق

قاموس المصطلحات البرمجية (عربي-إنجليزي)

10 دقيقة قراءة | 19 قسمًا | 0 مثالًا | 1,969 كلمة

في هذا الملحق

لماذا هذا الملحق؟
مفاهيم أساسية في البرمجة
المتغيرات وأنواع البيانات
هياكل البيانات
التحكم في سير البرنامج
الدوال والبرمجة الوظيفية
البرمجة الكائنية (OOP)
معالجة الأخطاء
الملفات وتنسيقات البيانات
البرمجة غير المتزامنة
إدارة المشاريع والأدوات
تطوير الويب (FastAPI)
تحليل البيانات
الأتمتة واستخلاص البيانات من الويب
الذكاء الاصطناعي والتعلّم الآلي
أدوات سطر الأوامر (CLI)
جودة الكود والاختبار
التحزيم والنشر
فهرس أبجدي إنجليزي سريع

لماذا هذا الملحق؟

عبر الفصول الستة والعشرين من هذا الكتاب، مررتَ بعشرات — بل مئات — المصطلحات التقنية. بعضها ترجمناه للعربية لتسهيل الفهم أثناء التعلّم (مثل «المتغيّر» لـ Variable)، لكن الحقيقة التي يجب أن تعرفها الآن هي:

⚠ **حقيقة مهمة يجب استيعابها:** في العالم الحقيقي للبرمجة — التوثيق الرسمي، رسائل الأخطاء، منتديات النقاش مثل Stack Overflow، مقالات المدونات التقنية، ومقابلات العمل — **المصطلحات تُستخدم بالإنجليزية فقط تقريبًا بنسبة 99%**. لن تجد أحدًا يقول «دعنا نستخدم التغليف» بل سيقول "Let's use Encapsulation". لن تقرأ خطأ باسم «خطأ في الفهرس» بل `IndexError`.

لهذا السبب، هذا القاموس ليس مجرد «ترجمة» بل **جسر**: الهدف أن تصل إلى نقطة تقرأ فيها المصطلح الإنجليزي وتفهمه مباشرة دون الحاجة للمرور بالعربية إطلاقًا. استخدم هذا الملحق كمرجع سريع الآن، لكن اجعل هدفك النهائي الاستغناء عنه تدريجيًا.

تم تنظيم المصطلحات حسب الموضوع (مطابقة لأجزاء الكتاب) لتسهيل المراجعة، مع فهرس أبجدي إنجليزي شامل في نهاية الملحق.

مفاهيم أساسية في البرمجة

English	العربية	الشرح المختصر
Programming	البرمجة	كتابة تعليمات يفهمها الحاسوب لتنفيذ مهمة
Programming Language	لغة برمجة	وسيط بين تفكير الإنسان ولغة الآلة (مثل Python)
Algorithm	خوارزمية	سلسلة خطوات منطقية لحل مسألة
Source Code	الكود المصدري	النص الذي يكتبه المبرمج قبل تنفيذه
Compiler	المترجم (المُصرِّف)	برنامج يحوّل الكود المصدري بالكامل إلى لغة آلة قبل التشغيل
Interpreter	المُفسِّر	برنامج ينفّذ الكود سطرًا بسطر مباشرة (طريقة عمل بايثون)
Syntax	الصياغة/بناء الجملة	القواعد النحوية للغة البرمجة
Syntax Error	خطأ صياغي	خطأ في اتباع قواعد كتابة اللغة
Runtime	وقت التشغيل	الفترة التي يُنفَّذ فيها البرنامج فعليًا
Runtime Error	خطأ وقت التشغيل	خطأ يظهر أثناء التنفيذ وليس عند الكتابة
Debugging	تنقيح الأخطاء	عملية إيجاد وإصلاح الأخطاء البرمجية
IDE (Integrated Development Environment)	بيئة تطوير متكاملة	برنامج شامل للكتابة والتشغيل والتنقيح (مثل VS Code)
REPL	بيئة تفاعلية	واجهة لتجربة كود بايثون سطرًا بسطر مباشرة
Comment	تعليق	نص يتجاهله المفسّر، يُستخدم للتوثيق فقط

المتغيرات وأنواع البيانات

English	العربية	الشرح المختصر
Variable	متغيّر	اسم يُشير إلى قيمة مخزّنة في الذاكرة
Value	قيمة	البيانات الفعلية المخزّنة
Data Type	نوع البيانات	تصنيف القيمة (نص، رقم، منطقي...)
String (`str`)	سلسلة نصية	نوع بيانات للنصوص
Integer (`int`)	عدد صحيح	نوع بيانات للأعداد الصحيحة
Float (`float`)	عدد عشري	نوع بيانات للأعداد ذات الفاصلة العشرية
Boolean (`bool`)	منطقي (ثنائي)	نوع بيانات بقيمتين فقط: `True` / `False`
None / Null	لا شيء / عدم وجود قيمة	يمثّل غياب القيمة في بايثون
Type Casting / Conversion	تحويل الأنواع	تحويل قيمة من نوع لآخر (`int("5")`)
Dynamic Typing	الكتابة الديناميكية	تحديد النوع تلقائيًا وقت التشغيل (خاصية بايثون)
Type Hints	تلميحات الأنواع	توضيح نوع متوقع للمتغيرات والدوال دون إلزام
Immutable	غير قابل للتغيير	كائن لا يمكن تعديل محتواه بعد إنشائه (مثل `str` , `tuple`)
Mutable	قابل للتغيير	كائن يمكن تعديل محتواه (مثل `list` , `dict`)
f-string	سلسلة منسّقة	صيغة تضمين متغيرات داخل نص مباشرة

هياكل البيانات

English	العربية	الشرح المختصر
Data Structure	هيكل بيانات	طريقة منظّمة لتخزين مجموعة من القيم
List	قائمة	تسلسل قابل للتعديل من العناصر المرتّبة
Tuple	صَفّ (تَوبِل)	تسلسل غير قابل للتعديل من العناصر
Dictionary (`dict`)	قاموس	تخزين أزواج مفتاح-قيمة

English	العربية	الشرح المختصر
Key	مفتاح	المعرّف الفريد في القاموس
Set	مجموعة	تجميع عناصر فريدة غير مرتّبة
Index	فهرس/مؤشّر	رقم موقع عنصر داخل تسلسل
Slicing	التقطيع	استخراج جزء من تسلسل (`list[1:4]`)
Iterable	قابل للتكرار	كائن يمكن المرور على عناصره واحدًا تلو الآخر
Nested	متداخل	هيكل بيانات داخل هيكل آخر (قائمة داخل قاموس مثلاً)
Comprehension	الصياغة الموجزة	طريقة مختصرة لبناء قائمة/قاموس/مجموعة في سطر واحد

التحكم في سير البرنامج

English	العربية	الشرح المختصر
Control Flow	التحكم بسير التنفيذ	ترتيب تنفيذ التعليمات بناءً على شروط
Conditional Statement	جملة شرطية	تنفيذ كود معيّن بناءً على شرط (`if`)
Boolean Expression	تعبير منطقي	عبارة تُقيَّم إلى `True` أو `False`
Loop	حلقة تكرار	تنفيذ كتلة كود عدة مرات
loop `for`	حلقة `for`	تكرار على عناصر معروفة مسبقًا
loop `while`	حلقة `while`	تكرار يستمر طالما شرط معيّن صحيح
Iteration	دورة تكرار	مرور واحد عبر حلقة
`break`	كسر/إيقاف الحلقة	إنهاء الحلقة فورًا
`continue`	تخطّي الدورة	تخطّي الدورة الحالية والانتقال للتالية

الدوال والبرمجة الوظيفية

English	العربية	الشرح المختصر
Function	دالة	كتلة كود قابلة لإعادة الاستخدام تُنفّذ مهمة محددة
Parameter	مُعامل (بارامتر)	اسم المتغيّر المُعرَّف في تعريف الدالة
Argument	وسيط (آرغيومنت)	القيمة الفعلية المُمرَّرة عند استدعاء الدالة
Return Value	القيمة المُعادة	ما تُرجعه الدالة بعد التنفيذ
Default Argument	وسيط افتراضي	قيمة مُستخدمة إذا لم يُمرَّر وسيط
`*args` / `**kwargs`	وسائط متغيّرة	قبول عدد غير محدد من الوسائط
Scope	النطاق	المنطقة التي يكون فيها المتغيّر مرئيًا وقابلًا للاستخدام
Recursion	الاستدعاء الذاتي (التكرار الذاتي)	دالة تستدعي نفسها
Lambda Function	دالة لامدا	دالة مجهولة الاسم مُعرَّفة في سطر واحد
Higher-Order Function	دالة من الرتبة العليا	دالة تأخذ أو تُعيد دالة أخرى
Decorator	المُزخرِف	دالة تُعدّل سلوك دالة أخرى دون تعديل كودها
Closure	الإغلاق	دالة تتذكر متغيرات من نطاقها الخارجي
Generator	المولِّد	دالة تُنتج قيمًا واحدة تلو الأخرى عبر `yield`
`yield`	إرجاع متسلسل	كلمة مفتاحية لإنتاج قيمة من مولّد دون إنهاء الدالة

البرمجة الكائنية (OOP)

English	العربية	الشرح المختصر
Object-Oriented Programming (OOP)	البرمجة الكائنية	نمط برمجي يُنظّم الكود حول «كائنات»
Class	صنف (فئة)	قالب/مخطط لإنشاء كائنات
Object / Instance	كائن / نسخة	وحدة فعلية مُنشأة من صنف
Attribute	خاصية (سمة)	متغيّر مرتبط بكائن

English	العربية	الشرح المختصر
Method	تابع (دالة عضو)	دالة مرتبطة بصنف
Constructor (`__init__`)	الباني	التابع الذي يُهيّئ الكائن عند إنشائه
`self`	الذات	يُشير إلى الكائن الحالي نفسه داخل التابع
Inheritance	الوراثة	اشتقاق صنف جديد من صنف موجود
Parent / Base Class	الصنف الأب	الصنف الذي يُورَّث منه
Child / Derived Class	الصنف الابن (المشتق)	الصنف الذي يرث من صنف آخر
Polymorphism	تعدد الأشكال	قدرة كائنات مختلفة على الاستجابة لنفس الاستدعاء بطرق مختلفة
Encapsulation	التغليف	إخفاء التفاصيل الداخلية وحماية البيانات
Abstraction	التجريد	إظهار الواجهة الأساسية فقط وإخفاء التعقيد
Magic / Dunder Methods	التوابع السحرية	توابع خاصة محاطة بشرطتين سفليتين (`__str__`, `__eq__`)
Property	خاصية محسوبة	تابع يُستخدم ويُقرأ كأنه خاصية عادية (`@property`)
Dataclass	صنف بيانات	صنف مبسّط للتعامل مع تخزين البيانات فقط

معالجة الأخطاء

English	العربية	الشرح المختصر
Exception	استثناء	خطأ يحدث أثناء التنفيذ ويمكن التعامل معه
`try` / `except`	محاولة / استثناء	بنية للتعامل مع الأخطاء المحتملة
`raise`	إثارة (رفع استثناء)	توليد استثناء يدويًا
`finally`	أخيرًا	كتلة كود تُنفّذ دائمًا بغض النظر عن حدوث خطأ
Custom Exception	استثناء مخصّص	صنف استثناء يُعرّفه المبرمج بنفسه
Traceback	أثر التنفيذ	تفاصيل مسار حدوث الخطأ المعروضة عند التعطّل

الملفات وتنسيقات البيانات

English	العربية	الشرح المختصر
File I/O	إدخال/إخراج الملفات	قراءة وكتابة الملفات
Path	المسار	عنوان موقع ملف أو مجلد
JSON	جيسون	تنسيق تبادل بيانات نصي شائع
CSV	قيم مفصولة بفواصل	تنسيق جداول بيانات بسيط
Serialization	التسلسل (التحويل للتخزين)	تحويل كائن بايثون لصيغة قابلة للتخزين/النقل
Deserialization	فك التسلسل	تحويل البيانات المخزّنة مرة أخرى لكائن بايثون
Encoding	الترميز	طريقة تمثيل النصوص كبايتات (مثل UTF-8)

البرمجة غير المتزامنة

English	العربية	الشرح المختصر
Synchronous	متزامن	تنفيذ تسلسلي، مهمة تلو الأخرى
Asynchronous	غير متزامن	تنفيذ يسمح بالانتظار دون تجميد البرنامج بالكامل
`async` / `await`	غير متزامن / انتظار	كلمات مفتاحية لتعريف وانتظار العمليات غير المتزامنة
Coroutine	روتين مشترك	دالة غير متزامنة قابلة للإيقاف والاستئناف
Concurrency	التزامن	التعامل مع عدة مهام «في نفس الفترة» (ليس بالضرورة بالتوازي الحقيقي)
Parallelism	التوازي	تنفيذ عدة مهام فعليًا في نفس اللحظة (عبر أنوية معالج متعددة)
Event Loop	حلقة الأحداث	الآلية التي تُدير تنفيذ المهام غير المتزامنة

إدارة المشاريع والأدوات

English	العربية	الشرح المختصر
Package	حزمة	مجموعة أكواد بايثون منظّمة وقابلة للتوزيع
Module	وحدة (موديول)	ملف بايثون واحد يحتوي كودًا قابلًا للاستيراد
Library	مكتبة	مجموعة من الوحدات/الحزم لأداء مهام محددة
Dependency	تبعية	مكتبة خارجية يعتمد عليها مشروعك
Virtual Environment	بيئة افتراضية	بيئة معزولة لتثبيت تبعيات مشروع دون التأثير على مشاريع أخرى
Package Manager	مدير الحزم	أداة لتثبيت وإدارة التبعيات (مثل `pip` , `uv`)
Version Control	التحكم بالإصدارات	نظام لتتبع تغييرات الكود عبر الزمن (مثل Git)
Repository (Repo)	مستودع	مكان تخزين مشروع مُتتبَّع بـ Git
Commit	تثبيت (حفظ لقطة)	حفظ لقطة من التغييرات في Git
Branch	فرع	نسخة موازية من المشروع للعمل عليها بمعزل
Merge	الدمج	دمج التغييرات من فرع إلى آخر

تطوير الويب (FastAPI)

English	العربية	الشرح المختصر
API (Application Programming Interface)	واجهة برمجة تطبيقات	نقطة تواصل بين برنامجين
REST / RESTful	ريست	أسلوب معياري لتصميم واجهات API عبر HTTP
Endpoint	نقطة نهاية	عنوان محدد (URL) يستقبل طلبات API
HTTP Method	طريقة HTTP	نوع العملية (`GET` , `POST` , `PUT` , `DELETE`)
Request	طلب	البيانات المُرسَلة من العميل إلى الخادم
Response	استجابة	البيانات المُعادة من الخادم للعميل
Status Code	رمز الحالة	رقم يُشير لنتيجة الطلب (`200` , `404` , `500`)

English	العربية	الشرح المختصر
Server	خادم	البرنامج/الجهاز الذي يستقبل الطلبات ويُعالجها
Client	عميل	الطرف الذي يُرسل الطلب (متصفح، تطبيق)
Middleware	وسيط برمجي	كود يُنفَّذ بين استقبال الطلب وإرسال الاستجابة

تحليل البيانات

English	العربية	الشرح المختصر
DataFrame	إطار بيانات	هيكل بيانات جدولي ثنائي الأبعاد (في pandas)
Series	سلسلة بيانات	عمود بيانات أحادي البُعد (في pandas)
Array	مصفوفة	هيكل بيانات رقمي متعدد الأبعاد (في NumPy)
Vectorization	المعالجة المتجهية	تنفيذ عمليات على مصفوفة كاملة دفعة واحدة دون حلقات صريحة
Missing Value	قيمة مفقودة	بيانات غير متوفرة في مجموعة البيانات
Outlier	قيمة شاذة	قيمة بعيدة بشكل غير طبيعي عن بقية البيانات
Aggregation	التجميع	تلخيص بيانات (مجموع، متوسط، عدد...)
Pivot Table	جدول محوري	إعادة تنظيم بيانات لتلخيصها حسب أبعاد مختلفة
Visualization	التصوير البياني	تمثيل البيانات بصريًا عبر رسوم بيانية

الأتمتة واستخلاص البيانات من الويب

English	العربية	الشرح المختصر
Web Scraping	استخلاص بيانات الويب	استخراج بيانات تلقائيًا من صفحات الويب
HTML	إتش تي إم إل	لغة ترميز بناء صفحات الويب
Tag	وسم	عنصر HTML (مثل `<p>` , `<div>`)
Parsing	التحليل (الفَرز)	استخراج بنية من بيانات غير منظّمة (مثل HTML)
`robots.txt`	ملف الروبوتات	ملف يُحدّد قواعد الزحف المسموحة لموقع
User-Agent	وكيل المستخدم	معلومات تُعرّف البرنامج المُرسل للطلب
Rate Limiting	تقييد المعدّل	تحديد عدد الطلبات المسموحة خلال فترة زمنية
Pagination	التصفّح بالصفحات	تقسيم النتائج إلى عدة صفحات

الذكاء الاصطناعي والتعلّم الآلي

English	العربية	الشرح المختصر
Machine Learning (ML)	تعلّم الآلة	بناء نماذج تتعلّم من البيانات بدل القواعد الصريحة
Supervised Learning	التعلّم بإشراف	التعلّم من بيانات موسومة (لها إجابات معروفة)
Unsupervised Learning	التعلّم دون إشراف	اكتشاف أنماط في بيانات غير موسومة
Regression	الانحدار	التنبؤ بقيمة رقمية مستمرة
Classification	التصنيف	التنبؤ بفئة/تصنيف محدد
Feature	خاصية (ميزة)	متغيّر مُدخل يُستخدم للتنبؤ
Label / Target	الهدف (التصنيف الحقيقي)	القيمة المراد التنبؤ بها
Training	التدريب	عملية «تعلّم» النموذج من البيانات
Model	نموذج	البرنامج الناتج القادر على التنبؤ بعد التدريب
Overfitting	الإفراط في التخصيص	نموذج يحفظ بيانات التدريب لكنه يفشل مع بيانات جديدة

English	العربية	الشرح المختصر
Underfitting	القصور في التعلّم	نموذج بسيط جدًا لا يلتقط الأنماط الحقيقية
Train/Test Split	تقسيم تدريب/اختبار	تقسيم البيانات لتقييم أداء النموذج بصدق

أدوات سطر الأوامر (CLI)

English	العربية	الشرح المختصر
CLI (Command-Line Interface)	واجهة سطر الأوامر	تطبيق يُستخدم عبر الطرفية بدل الواجهة الرسومية
Terminal / Shell	الطرفية	البرنامج الذي يستقبل أوامر النص
Flag / Option	خيار/عَلَم	إعداد اختياري يُمرَّر للأمر (`--verbose`)
Argument (CLI context)	وسيط (سياق CLI)	قيمة إلزامية تُمرَّر للأمر
Exit Code	رمز الخروج	رقم يُشير لنجاح أو فشل تنفيذ الأمر

جودة الكود والاختبار

English	العربية	الشرح المختصر
Clean Code	الكود النظيف	كود واضح، مقروء، وسهل الصيانة
Linter	أداة فحص الأسلوب	تفحص الكود بحثًا عن أخطاء أسلوبية (مثل `ruff`)
Formatter	أداة التنسيق	تُعيد تنسيق الكود تلقائيًا وفق معايير موحّدة (مثل `black`)
Refactoring	إعادة الهيكلة	تحسين بنية الكود دون تغيير سلوكه الخارجي
Unit Test	اختبار وحدة	اختبار أصغر جزء ممكن من الكود بمعزل
Integration Test	اختبار تكامل	اختبار تفاعل عدة أجزاء من النظام معًا
Assertion	تأكيد/تحقّق	عبارة تتحقق من صحة نتيجة متوقعة
Fixture	تجهيزة	بيانات/كائنات مُعدّة مسبقًا للاستخدام في الاختبارات

English	العربية	الشرح المختصر
Mock	محاكاة (كائن وهمي)	كائن وهمي يُحاكي سلوك كائن حقيقي أثناء الاختبار
Code Coverage	تغطية الكود	نسبة الكود التي تُنفَّذها الاختبارات فعليًا
TDD (Test-Driven Development)	التطوير المدفوع بالاختبار	كتابة الاختبار أولاً، ثم الكود الذي يُحقّقه

التحزيم والنشر

English	العربية	الشرح المختصر
Packaging	التحزيم	تجهيز الكود ليكون قابلاً للتوزيع والتثبيت
Build	البناء	تحويل الكود المصدري لصيغة قابلة للتوزيع
Deployment	النشر (التشغيلي)	تشغيل التطبيق على بيئة إنتاجية متاحة للمستخدمين
PyPI	مستودع بايثون الرسمي	المتجر الرسمي لحزم بايثون
Semantic Versioning	الترقيم الدلالي	نظام ترقيم إصدارات موحّد (`MAJOR.MINOR.PATCH`)
Container	حاوية	وحدة معزولة تحتوي تطبيقًا وكل بيئته
Docker Image	صورة دوكر	القالب الثابت المُستخدم لإنشاء الحاويات
Environment Variable	متغيّر بيئة	قيمة تُضبط خارج الكود (مثل كلمات المرور)
CI/CD	التكامل والنشر المستمر	أتمتة اختبار ونشر الكود تلقائيًا عند كل تحديث

فهرس أبجدي إنجليزي سريع

للرجوع السريع عند مصادفة مصطلح أثناء القراءة الخارجية (توثيق، مقالات):

```
Abstraction          → التجريد
Aggregation          → التجميع
API                  → واجهة برمجة تطبيقات
Argument             → وسيط
Array                → مصفوفة
Assertion            → تأكيد
```

```
Async/Await            → غير متزامن/انتظار
Attribute              → خاصية
Boolean                → منطقي
Branch (Git)           → فرع
Build                  → البناء
CI/CD                  → التكامل والنشر المستمر
Class                  → صنف
Classification         → التصنيف
Client                 → عميل
Closure                → الإغلاق
Code Coverage          → تغطية الكود
Commit                 → تثبيت
Comprehension          → الصياغة الموجزة
Concurrency            → التزامن
Constructor            → البانـي
Container              → حاوية
Control Flow           → التحكم بسير التنفيذ
Coroutine              → روتين مشترك
CSV                    → قيم مفصولة بفواصل
DataFrame              → إطار بيانات
Dataclass              → صنف بيانات
Debugging              → تنقيح الأخطاء
Decorator              → المُزخرِف
Dependency             → تبعية
Deployment             → النشر
Deserialization        → فك التسلسل
Dictionary             → قاموس
Docker Image           → صورة دوكر
Encapsulation          → التغليف
Encoding               → الترميز
Endpoint               → نقطة نهاية
Environment Variable   → متغيّر بيئة
Event Loop             → حلقة الأحداث
Exception              → استثناء
Exit Code              → رمز الخروج
f-string               → سلسلة منسّقة
Feature (ML)           → خاصية/ميزة
File I/O               → إدخال/إخراج الملفات
Fixture                → تجهيزة
Flag/Option            → خيار
Float                  → عدد عشري
Formatter              → أداة التنسيق
Function               → دالة
Generator              → المولِّد
Higher-Order Function  → دالة من الرتبة العليا
HTML                   → إتش تي إم إل
HTTP Method            → طريقة HTTP
IDE                    → بيئة تطوير متكاملة
Immutable              → غير قابل للتغيير
Index                  → فهرس
Inheritance            → الوراثة
```

```
Integer                → عدد صحيح
Integration Test       → اختبار تكامل
Interpreter            → المُفسِّر
Iterable               → قابل للتكرار
Iteration              → دورة تكرار
JSON                   → جيسون
Key                    → مفتاح
Label/Target (ML)      → الهدف
Lambda Function        → دالة لامدا
Library                → مكتبة
Linter                 → أداة فحص الأسلوب
List                   → قائمة
Loop                   → حلقة تكرار
Machine Learning       → تعلّم الآلة
Magic/Dunder Methods   → التوابع السحرية
Merge (Git)            → الدمج
Method                 → تابع
Middleware             → وسيط برمجي
Missing Value          → قيمة مفقودة
Mock                   → محاكاة
Model (ML)             → نموذج
Module                 → وحدة
Mutable                → قابل للتغيير
Nested                 → متداخل
None/Null              → لا شيء
Object/Instance        → كائن/نسخة
OOP                    → البرمجة الكائنية
Outlier                → قيمة شاذة
Overfitting            → الإفراط في التخصيص
Package                → حزمة
Package Manager        → مدير الحزم
Packaging              → التحزيم
Pagination             → التصفّح بالصفحات
Parallelism            → التوازي
Parameter              → مُعامل
Parent/Base Class      → الصنف الأب
Parsing                → التحليل
Path                   → المسار
Pivot Table            → جدول محوري
Polymorphism           → تعدد الأشكال
Programming            → البرمجة
Programming Language   → لغة برمجة
Property               → خاصية محسوبة
PyPI                   → مستودع بايثون الرسمي
Rate Limiting          → تقييد المعدّل
Recursion              → الاستدعاء الذاتي
Refactoring            → إعادة الهيكلة
Regression             → الانحدار
Repository             → مستودع
Request                → طلب
Response               → استجابة
```

```
REST/RESTful            → ريست
Return Value            → القيمة المُعادة
robots.txt              → ملف الروبوتات
Runtime                 → وقت التشغيل
Scope                   → النطاق
Semantic Versioning     → الترقيم الدلالي
Serialization           → التسلسل
Series (pandas)         → سلسلة بيانات
Server                  → خادم
Set                     → مجموعة
Slicing                 → التقطيع
Source Code             → الكود المصدري
Status Code             → رمز الحالة
String                  → سلسلة نصية
Supervised Learning     → التعلّم بإشراف
Synchronous             → متزامن
Syntax                  → الصياغة
Syntax Error            → خطأ صياغي
TDD                     → التطوير المدفوع بالاختبار
Terminal/Shell          → الطرفية
Training (ML)           → التدريب
Train/Test Split        → تقسيم تدريب/اختبار
Traceback               → أثر التنفيذ
Tuple                   → صَفّ
Type Casting            → تحويل الأنواع
Type Hints              → تلميحات الأنواع
Underfitting            → القصور في التعلّم
Unit Test               → اختبار وحدة
Unsupervised Learning   → التعلّم دون إشراف
User-Agent              → وكيل المستخدم
Value                   → قيمة
Variable                → متغيّر
Vectorization           → المعالجة المتجهية
Version Control         → التحكم بالإصدارات
Virtual Environment     → بيئة افتراضية
Visualization           → التصوير البياني
Web Scraping            → استخلاص بيانات الويب
```

نصيحة ختامية للملحق: إذا صادفت مصطلحًا تقنيًا جديدًا غير موجود هنا أثناء قراءتك المستقبلية، لا تُترجمه فورًا في ذهنك — بل ابحث عنه مباشرة بالإنجليزية (غالبًا عبر «[المصطلح] python meaning» أو في التوثيق الرسمي). هذه العادة وحدها ستُسرّع من استقلاليتك كمبرمج أكثر من أي قاموس.

ملحق · ملاحق · الملاحق

حلول جميع التمارين

5 دقيقة قراءة | 25 قسمًا | 53 مثالًا | 925 كلمة

في هذا الملحق

ملاحظة قبل البدء
الفصل 3: المتغيرات
الفصل 4: التعامل مع النصوص
الفصل 5: الأرقام والعمليات الحسابية
الفصل 6: اتخاذ القرار (Control Flow)
الفصل 7: التكرار (Loops)
الفصل 8: القوائم (Lists)
الفصل 9: القواميس والمجموعات
الفصل 10: الدوال
الفصل 11: معالجة الأخطاء
الفصل 12: الدوال المتقدمة
الفصل 13 و14: البرمجة الكائنية
الفصل 15: Type Hints
الفصل 16: الملفات وتنسيقات البيانات
الفصل 17: البرمجة غير المتزامنة
الفصل 18: إدارة المشاريع
الفصل 19: FastAPI
الفصل 20: تحليل البيانات
الفصل 21: Web Scraping
الفصل 22: الذكاء الاصطناعي
الفصل 23: أدوات CLI
الفصل 24: Clean Code
الفصل 25: الاختبار (Testing)
الفصل 26: من الكود إلى المنتج
خاتمة الملحق

ملاحظة قبل البدء

هذا الملحق يُقدّم حلولًا نموذجية لتمارين الكتاب، مرتّبة حسب الفصول. تذكّر دائمًا: **«الحل النموذجي» ليس «الحل الوحيد الصحيح»** — في البرمجة، غالبًا توجد عدة طرق صحيحة للوصول لنفس النتيجة. إذا توصّلت لحل مختلف يعمل بشكل صحيح ويتبع الممارسات الجيدة، فهو صحيح تمامًا ولو اختلف عن المعروض هنا.

> **نصيحة قبل القراءة:** حاول حل كل تمرين بنفسك أولًا، مهما استغرق الأمر، قبل النظر إلى الحل. القيمة التعليمية الحقيقية تكمن في الصراع مع المشكلة، وليس في قراءة الحل النهائي.

نظرًا لضخامة عدد التمارين عبر 26 فصلاً، سنعرض حلولًا كاملة وعملية لتمارين مُختارة من كل مستوى في كل فصل، مع التركيز على الأمثلة الأكثر تمثيلاً للمفاهيم الأساسية.

الفصل 3: المتغيرات

✅🟢 تمرين: أنشئ متغيرات لاسمك وعمرك وطولك، واطبعها بجملة واحدة

Python

```
name: str = "سارة"
age: int = 25
height: float = 1.65

print(f"اسمي {name}، عمري {age} سنة، وطولي {height} متر.")
```

✅🟡 تمرين: بدّل قيم متغيرين دون استخدام متغيّر ثالث

Python

```
a: int = 5
b: int = 10

a, b = b, a  # الطريقة البايثونية لتبديل القيم مباشرة

print(f"a = {a}, b = {b}")  # a = 10, b = 5
```

هذه الطريقة ("Tuple Unpacking") تُعدّ الأسلوب المثالي في بايثون، مقارنة بلغات مثل C التي تتطلب متغيرًا ثالثًا مؤقتًا.

✅🔴 تحدي: احسب عمر شخص بالأيام تقريبيًا من عمره بالسنوات

Python

```
age_years: int = 25
days_per_year: float = 365.25  # لمراعاة السنوات الكبيسة تقريبيًا

age_in_days: float = age_years * days_per_year
print(f"العمر التقريبي بالأيام: {age_in_days:.0f} يوم")
```

الفصل 4: التعامل مع النصوص

✅ 🟢 تمرين: اطبع اسمك بأحرف كبيرة وصغيرة

```python
name = "Ahmed"
print(name.upper())   # AHMED
print(name.lower())   # ahmed
```

✅ 🟡 تمرين: تحقق إن كانت كلمة معينة موجودة داخل جملة

```python
sentence = "تعلم بايثون ممتع ومفيد"
word = "ممتع"

if word in sentence:
    print(f"الكلمة '{word}' موجودة في الجملة")
else:
    print(f"الكلمة '{word}' غير موجودة")
```

✅ 🔴 تحدي: اعكس كل كلمة في جملة مع الحفاظ على ترتيب الكلمات

```python
sentence = "Python is powerful"
words = sentence.split()
reversed_words = [word[::-1] for word in words]
result = " ".join(reversed_words)

print(result)  # nohtyP si lufrewop
```

الفصل 5: الأرقام والعمليات الحسابية

✅ 🟢 تمرين: احسب مساحة ومحيط مستطيل

```python
length: float = 8.5
width: float = 4.2

area = length * width
perimeter = 2 * (length + width)

print(f"المساحة: {area} | المحيط: {perimeter}")
```

✅ 🟡 تمرين: حوّل درجة حرارة من فهرنهايت إلى مئوية

```python
fahrenheit: float = 98.6
celsius = (fahrenheit - 32) * 5 / 9

print(f"{fahrenheit}°F = {celsius:.2f}°C")
```

✅ 🔴 تحدي: احسب الفائدة المركبة على مبلغ معين

```python
principal: float = 1000.0
annual_rate: float = 0.05
years: int = 10

final_amount = principal * (1 + annual_rate) ** years
print(f"المبلغ بعد {years} سنوات: {final_amount:.2f}")
```

الفصل 6: اتخاذ القرار (Control Flow)

✅🟢 تمرين: برنامج يحدد إن كان الرقم زوجيًا أم فرديًا

Python

```
number = int(input("أدخل رقمًا: "))

if number % 2 == 0:
    print("الرقم زوجي")
else:
    print("الرقم فردي")
```

✅🟡 تمرين: برنامج تقدير درجة الطالب (A/B/C/D/F)

Python

```
score = int(input("أدخل الدرجة: "))

if score ⩾ 90:
    grade = "A"
elif score ⩾ 80:
    grade = "B"
elif score ⩾ 70:
    grade = "C"
elif score ⩾ 60:
    grade = "D"
else:
    grade = "F"

print(f"التقدير: {grade}")
```

✅🔴 تحدي: برنامج يحدد نوع المثلث (متساوي الأضلاع/متساوي الساقين/مختلف الأضلاع) من أطوال أضلاعه الثلاثة

Python

```
a, b, c = 5, 5, 8

if a == b == c:
    triangle_type = "متساوي الأضلاع"
elif a == b or b == c or a == c:
    triangle_type = "متساوي الساقين"
else:
    triangle_type = "مختلف الأضلاع"

# التحقق من صحة المثلث أولاً
if a + b > c and a + c > b and b + c > a:
    print(f"نوع المثلث: {triangle_type}")
else:
    print("هذه الأطوال لا تُشكّل مثلثًا صحيحًا")
```

الفصل 7: التكرار (Loops)

✅🟢 تمرين: اطبع جدول الضرب لرقم معين

Python

```
number = 7

for i in range(1, 11):
    print(f"{number} × {i} = {number * i}")
```

✅ 🟡 تمرين: احسب مجموع الأعداد الفردية من 1 إلى 100

Python

```
total = sum(n for n in range(1, 101) if n % 2 != 0)
print(f"مجموع الأعداد الفردية: {total}")
```

✅ 🔴 تحدي: اطبع مثلثًا من النجوم بعدد أسطر يحدده المستخدم

Python

```
rows = 5

for i in range(1, rows + 1):
    print("*" * i)

# شكل هرمي متناظر
for i in range(1, rows + 1):
    spaces = " " * (rows - i)
    stars = "*" * (2 * i - 1)
    print(spaces + stars)
```

الفصل 8: القوائم (Lists)

✅ 🟢 تمرين: أنشئ قائمة بأسماء فواكه واطبع كل اسم مع رقمه

Python

```
fruits = ["تفاح", "موز", "برتقال", "عنب"]

for index, fruit in enumerate(fruits, start=1):
    print(f"{index}. {fruit}")
```

✅🟡 تمرين: أزل التكرارات من قائمة مع الحفاظ على الترتيب

Python

```python
numbers = [1, 3, 2, 3, 1, 5, 4, 2]

seen = set()
unique_numbers = []
for n in numbers:
    if n not in seen:
        seen.add(n)
        unique_numbers.append(n)

print(unique_numbers)  # [1, 3, 2, 5, 4]
```

✅🔴 تحدي: دمج قائمتين مرتبتين في قائمة واحدة مرتبة (بدون استخدام sorted مباشرة)

Python

```python
def merge_sorted(list1: list[int], list2: list[int]) -> list[int]:
    merged = []
    i, j = 0, 0

    while i < len(list1) and j < len(list2):
        if list1[i] <= list2[j]:
            merged.append(list1[i])
            i += 1
        else:
            merged.append(list2[j])
            j += 1

    merged.extend(list1[i:])
    merged.extend(list2[j:])
    return merged

result = merge_sorted([1, 3, 5, 7], [2, 4, 6, 8])
print(result)  # [1, 2, 3, 4, 5, 6, 7, 8]
```

الفصل 9: القواميس والمجموعات

✅🟢 تمرين: أنشئ قاموسًا لمعلومات طالب واطبعها

```python
student = {
    "name": "ليلى",
    "age": 20,
    "major": "هندسة برمجيات",
}

for key, value in student.items():
    print(f"{key}: {value}")
```

✅🟡 تمرين: احسب عدد تكرار كل كلمة في نص

```python
text = "python is great python is fun"
words = text.split()

word_count: dict[str, int] = {}
for word in words:
    word_count[word] = word_count.get(word, 0) + 1

print(word_count)  # {'python': 2, 'is': 2, 'great': 1, 'fun': 1}
```

✅ 🔴 تحدي: أوجد العناصر المشتركة والمختلفة بين مجموعتين

Python

```
set_a = {1, 2, 3, 4, 5}
set_b = {4, 5, 6, 7, 8}

common = set_a & set_b
only_in_a = set_a - set_b
only_in_b = set_b - set_a
all_unique = set_a ^ set_b  # الاختلاف المتماثل

print(f"مشترك: {common}")
print(f"فقط في أ: {only_in_a}")
print(f"فقط في ب: {only_in_b}")
print(f"غير مشترك بينهما: {all_unique}")
```

الفصل 10: الدوال

✅ 🟢 تمرين: دالة تحسب مساحة دائرة

Python

```
import math

def circle_area(radius: float) -> float:
    return math.pi * radius ** 2

print(f"{circle_area(5):.2f}")  # 78.54
```

✅🟡 تمرين: دالة تتحقق إن كان رقم أوليًا

Python

```python
def is_prime(n: int) -> bool:
    if n < 2:
        return False
    for i in range(2, int(n ** 0.5) + 1):
        if n % i == 0:
            return False
    return True

primes = [n for n in range(2, 50) if is_prime(n)]
print(primes)
```

✅🔴 تحدي: دالة recursion لحساب متتالية فيبوناتشي مع Memoization

Python

```python
from functools import lru_cache

@lru_cache(maxsize=None)
def fibonacci(n: int) -> int:
    if n <= 1:
        return n
    return fibonacci(n - 1) + fibonacci(n - 2)

result = [fibonacci(i) for i in range(15)]
print(result)
```

بدون `@lru_cache` ، حساب `fibonacci(35)` يستغرق ثواني بسبب إعادة حساب نفس القيم ملايين المرات. الـ memoization تحل هذه المشكلة بتخزين النتائج المحسوبة مسبقًا.

الفصل 11: معالجة الأخطاء

✅🟢 تمرين: تعامل مع خطأ القسمة على صفر

Python

```
def safe_divide(a: float, b: float) -> float | None:
    try:
        return a / b
    except ZeroDivisionError:
        print("خطأ: لا يمكن القسمة على صفر")
        return None

print(safe_divide(10, 2))   # 5.0
print(safe_divide(10, 0))   # خطأ: None ثم..
```

✅🟡 تمرين: تحقق من صحة إدخال المستخدم لرقم باستخدام try/except في حلقة

Python

```
def get_valid_number() -> int:
    while True:
        try:
            return int(input("أدخل رقمًا صحيحًا: "))
        except ValueError:
            print("إدخال غير صالح، حاول مرة أخرى")

# number = get_valid_number()
```

✅ 🔴 تحدي: أنشئ استثناءً مخصصًا لرصيد غير كافٍ في نظام حسابات بنكية

Python

```
class InsufficientFundsError(Exception):
    def __init__(self, balance: float, amount: float) -> None:
        self.balance = balance
        self.amount = amount
        super().__init__(
            f"الرصيد غير كافٍ: رصيد {balance}، المطلوب سحبه {amount}"
        )

def withdraw(balance: float, amount: float) -> float:
    if amount > balance:
        raise InsufficientFundsError(balance, amount)
    return balance - amount

try:
    withdraw(100, 150)
except InsufficientFundsError as e:
    print(f"فشلت العملية: {e}")
```

الفصل 12: الدوال المتقدمة

✅ 🟢 تمرين: استخدم lambda مع sorted لترتيب قائمة أزواج حسب العنصر الثاني

Python

```
pairs = [(1, 5), (2, 2), (3, 8), (4, 1)]
sorted_pairs = sorted(pairs, key=lambda pair: pair[1])
print(sorted_pairs)  # [(4, 1), (2, 2), (1, 5), (3, 8)]
```

✅ 🟡 تمرين: اكتب generator يُنتج الأعداد الأولية بلا حدود

```python
from collections.abc import Iterator

def prime_generator() -> Iterator[int]:
    n = 2
    while True:
        if all(n % i != 0 for i in range(2, int(n ** 0.5) + 1)):
            yield n
        n += 1

gen = prime_generator()
first_10_primes = [next(gen) for _ in range(10)]
print(first_10_primes)
```

✅🔴 تحدي: اكتب decorator يقيس زمن تنفيذ أي دالة

Python

```python
import time
from functools import wraps
from collections.abc import Callable
from typing import TypeVar

T = TypeVar("T")

def timer(func: Callable[..., T]) -> Callable[..., T]:
    @wraps(func)
    def wrapper(*args, **kwargs) -> T:
        start = time.perf_counter()
        result = func(*args, **kwargs)
        elapsed = time.perf_counter() - start
        print(f"{func.__name__} استغرقت {elapsed:.4f} ثانية")
        return result
    return wrapper

@timer
def slow_function() -> None:
    time.sleep(0.5)

slow_function()
```

الفصل 13 و14: البرمجة الكائنية

✅🟢 تمرين: صنف `Car` بخصائص وتابع `honk`

Python

```
class Car:
    def __init__(self, brand: str, model: str, year: int) -> None:
        self.brand = brand
        self.model = model
        self.year = year

    def honk(self) -> str:
        return f"{self.brand} {self.model}: بيب بيب! 🚗"

    def __str__(self) -> str:
        return f"{self.year} {self.brand} {self.model}"

car = Car("Toyota", "Corolla", 2024)
print(car)
print(car.honk())
```

✅🟡 تمرين: وراثة — صنف `ElectricCar` يرث من `Car` ويضيف `battery_range`

Python

```
class ElectricCar(Car):
    def __init__(
        self, brand: str, model: str, year: int, battery_range: int
    ) -> None:
        super().__init__(brand, model, year)
        self.battery_range = battery_range

    def __str__(self) -> str:
        base = super().__str__()
        return f"{base} (مدى البطارية: {self.battery_range} كم)"

tesla = ElectricCar("Tesla", "Model 3", 2024, 500)
print(tesla)
```

✅🔴 تحدي: نظام أشكال هندسية باستخدام Abstract Base Class ومبدأ Polymorphism

Python

```python
from abc import ABC, abstractmethod
import math

class Shape(ABC):
    @abstractmethod
    def area(self) -> float: ...

    @abstractmethod
    def perimeter(self) -> float: ...

class Circle(Shape):
    def __init__(self, radius: float) -> None:
        self.radius = radius

    def area(self) -> float:
        return math.pi * self.radius ** 2

    def perimeter(self) -> float:
        return 2 * math.pi * self.radius

class Rectangle(Shape):
    def __init__(self, width: float, height: float) -> None:
        self.width = width
        self.height = height

    def area(self) -> float:
        return self.width * self.height

    def perimeter(self) -> float:
        return 2 * (self.width + self.height)

shapes: list[Shape] = [Circle(5), Rectangle(4, 6)]

for shape in shapes:
    print(f"{type(shape).__name__}: مساحة={shape.area():.2f}, محيط={shape.perimeter():.2f}")
```

الفصل 15: Type Hints

✅🟡 تمرين: أضف Type Hints كاملة لدالة تُعالج قائمة طلبات

Python

```python
from dataclasses import dataclass

@dataclass
class Order:
    item: str
    quantity: int
    price: float

def total_revenue(orders: list[Order]) -> float:
    return sum(order.quantity * order.price for order in orders)

orders = [Order("20.0 ,3 ,"كتاب"), Order("2.5 ,10 ,"قلم")]
print(total_revenue(orders))  # 85.0
```

✅🔴 تحدي: استخدم Generic لكتابة صنف `Stack` يعمل مع أي نوع بيانات

```python
from typing import Generic, TypeVar

T = TypeVar("T")

class Stack(Generic[T]):
    def __init__(self) -> None:
        self._items: list[T] = []

    def push(self, item: T) -> None:
        self._items.append(item)

    def pop(self) -> T:
        return self._items.pop()

    def is_empty(self) -> bool:
        return len(self._items) == 0

int_stack: Stack[int] = Stack()
int_stack.push(1)
int_stack.push(2)
print(int_stack.pop())  # 2
```

الفصل 16: الملفات وتنسيقات البيانات

✅ 🟡 تمرين: اقرأ ملف CSV واحسب متوسط عمود معين

Python

```python
import csv
from pathlib import Path

csv_path = Path("students.csv")

ages: list[int] = []
with csv_path.open(encoding="utf-8") as f:
    reader = csv.DictReader(f)
    for row in reader:
        ages.append(int(row["age"]))

average_age = sum(ages) / len(ages) if ages else 0
print(f"متوسط الأعمار: {average_age:.1f}")
```

✅🔴 تحدي: احفظ واسترجع كائنات مخصصة من/إلى JSON

Python

```python
import json
from dataclasses import dataclass, asdict
from pathlib import Path

@dataclass
class Contact:
    name: str
    phone: str

contacts = [Contact("0501234567" ,"أحمد"), Contact("0559876543" ,"سارة")]

# الحفظ
data = [asdict(c) for c in contacts]
Path("contacts.json").write_text(
    json.dumps(data, ensure_ascii=False, indent=2), encoding="utf-8"
)

# الاسترجاع
loaded_data = json.loads(Path("contacts.json").read_text(encoding="utf-8"))
loaded_contacts = [Contact(**item) for item in loaded_data]
print(loaded_contacts)
```

الفصل 17: البرمجة غير المتزامنة

✅🟡 تمرين: نفّذ عدة طلبات «وهمية» بالتوازي باستخدام asyncio.gather

Python

```python
import asyncio

async def fetch_data(source: str, delay: float) -> str:
    await asyncio.sleep(delay)
    return f"بيانات من {source}"

async def main() -> None:
    results = await asyncio.gather(
        fetch_data("API-1", 1),
        fetch_data("API-2", 2),
        fetch_data("API-3", 0.5),
    )
    print(results)

asyncio.run(main())
```

✅🔴 تحدي: استخدم asyncio.Semaphore لتحديد عدد العمليات المتزامنة

Python

```python
import asyncio

async def limited_task(semaphore: asyncio.Semaphore, task_id: int) -> None:
    async with semaphore:
        print(f"بدء المهمة {task_id}")
        await asyncio.sleep(1)
        print(f"انتهت المهمة {task_id}")

async def main() -> None:
    semaphore = asyncio.Semaphore(3)  # مهام متزامنة كحد أقصى 3
    tasks = [limited_task(semaphore, i) for i in range(10)]
    await asyncio.gather(*tasks)

asyncio.run(main())
```

الفصل 18: إدارة المشاريع

✅ 🟡 تمرين: أنشئ مشروعًا جديدًا بـ uv وأضف تبعية وفعّل Git

Terminal

```
uv init my_project
cd my_project
uv add requests
git init
git add .
git commit -m "Initial commit"
```

الفصل 19: FastAPI

✅ 🟡 تمرين: أضف endpoint لحذف مهمة بالـ ID

Python

```python
from fastapi import FastAPI, HTTPException

app = FastAPI()
tasks_db: dict[int, dict] = {1: {"title": "تعلم FastAPI"}}

@app.delete("/tasks/{task_id}")
def delete_task(task_id: int) -> dict[str, str]:
    if task_id not in tasks_db:
        raise HTTPException(status_code=404, detail="المهمة غير موجودة")
    del tasks_db[task_id]
    return {"message": "تم الحذف بنجاح"}
```

✅🔴 تحدي: أضف نظام ترقيم صفحات (Pagination) لقائمة المهام

Python

```python
from fastapi import FastAPI, Query

app = FastAPI()
all_tasks = [{"id": i, "title": f"مهمة {i}"} for i in range(1, 101)]

@app.get("/tasks")
def list_tasks(
    page: int = Query(1, ge=1), page_size: int = Query(10, ge=1, le=50)
) -> dict:
    start = (page - 1) * page_size
    end = start + page_size
    return {
        "page": page,
        "page_size": page_size,
        "total": len(all_tasks),
        "items": all_tasks[start:end],
    }
```

الفصل 20: تحليل البيانات

✅🟡 تمرين: استخدم pandas لإيجاد أعلى 5 قيم مبيعات في ملف بيانات

Python

```python
import pandas as pd

df = pd.read_csv("sales.csv")
top_5 = df.nlargest(5, "sales_amount")
print(top_5[["product", "sales_amount"]])
```

✅ 🔴 تحدي: ارسم رسمًا بيانيًا يوضح المبيعات الشهرية مع تمييز أعلى شهر

Python

```python
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("sales.csv", parse_dates=["date"])
monthly = df.groupby(df["date"].dt.to_period("M"))["sales_amount"].sum()

colors = ["orange" if v == monthly.max() else "steelblue" for v in monthly]

plt.figure(figsize=(10, 5))
plt.bar(monthly.index.astype(str), monthly.values, color=colors)
plt.title("المبيعات الشهرية")
plt.xticks(rotation=45)
plt.tight_layout()
plt.savefig("monthly_sales.png")
```

الفصل 21: Web Scraping

✅ 🟡 تمرين: استخرج جميع الروابط من صفحة ويب

Python

```python
import requests
from bs4 import BeautifulSoup

response = requests.get("https://example.com", timeout=10)
response.raise_for_status()
soup = BeautifulSoup(response.text, "html.parser")

links = [a["href"] for a in soup.find_all("a", href=True)]
print(links)
```

✅🔴 تحدي: تعامل مع pagination لاستخراج بيانات من عدة صفحات

Python

```python
import time
import requests
from bs4 import BeautifulSoup

all_titles: list[str] = []

for page in range(1, 4):
    url = f"https://books.toscrape.com/catalogue/page-{page}.html"
    response = requests.get(url, timeout=10)
    if response.status_code ≠ 200:
        break

    soup = BeautifulSoup(response.text, "html.parser")
    titles = [h3.a["title"] for h3 in soup.select("h3")]
    all_titles.extend(titles)
    time.sleep(1)  # احترام الخادم

print(f"إجمالي الكتب المستخرجة: {len(all_titles)}")
```

الفصل 22: الذكاء الاصطناعي

✅🟡 تمرين: درّب نموذج تصنيف بسيط وقيّم دقته

Python

```python
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = KNeighborsClassifier(n_neighbors=3)
model.fit(X_train, y_train)
predictions = model.predict(X_test)

print(f"الدقة: {accuracy_score(y_test, predictions):.2%}")
```

✅🔴 تحدي: قارن أداء 3 نماذج مختلفة باستخدام Pipeline وStandardScaler

Python

```python
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

models = {
    "Logistic Regression": LogisticRegression(),
    "SVM": SVC(),
    "Random Forest": RandomForestClassifier(random_state=42),
}

for name, model in models.items():
    pipeline = Pipeline([("scaler", StandardScaler()), ("model", model)])
    pipeline.fit(X_train, y_train)
    accuracy = accuracy_score(y_test, pipeline.predict(X_test))
    print(f"{name}: {accuracy:.2%}")
```

الفصل 23: أدوات CLI

✅ 🟡 تمرين: أداة click تحسب إحصائيات ملف نصي (عدد الأسطر/الكلمات/الأحرف)

Python

```python
import click
from pathlib import Path

@click.command()
@click.argument("file_path", type=click.Path(exists=True))
def wordcount(file_path: str) -> None:
    """يحسب عدد الأسطر والكلمات والأحرف في ملف نصي."""
    text = Path(file_path).read_text(encoding="utf-8")
    lines = text.splitlines()
    words = text.split()

    click.echo(f"الأسطر: {len(lines)}")
    click.echo(f"الكلمات: {len(words)}")
    click.echo(f"الأحرف: {len(text)}")

if __name__ == "__main__":
    wordcount()
```

✅ 🔴 تحدي: أداة CLI بأوامر فرعية متعددة (مجموعة أوامر click.group)

Python

```python
import click

@click.group()
def cli() -> None:
    """أداة إدارة مهام بسيطة."""
    pass

@cli.command()
@click.argument("title")
def add(title: str) -> None:
    """إضافة مهمة جديدة."""
    click.echo(f"تمت إضافة: {title}")

@cli.command()
def list() -> None:
    """عرض كل المهام."""
    click.echo("عرض المهام.  ")

if __name__ == "__main__":
    cli()
```

الفصل 24: Clean Code

✅🟡 تمرين: أعد هيكلة دالة طويلة تخالف مبدأ المسؤولية الواحدة

Python

```python
# قبل: دالة واحدة تفعل كل شيء
def process_order_bad(order: dict) -> None:
    # تحقق + حساب + طباعة + حفظ، كل هذا في دالة واحدة
    ...

# بعد: تقسيم حسب المسؤولية
def validate_order(order: dict) -> bool:
    return order.get("quantity", 0) > 0

def calculate_total(order: dict) -> float:
    return order["quantity"] * order["price"]

def save_order(order: dict, total: float) -> None:
    print(f"تم حفظ الطلب بمبلغ {total}")

def process_order(order: dict) -> None:
    if not validate_order(order):
        raise ValueError("طلب غير صالح")
    total = calculate_total(order)
    save_order(order, total)
```

الفصل 25: الاختبار (Testing)

✅🟡 تمرين: اكتب اختبارات باستخدام parametrize لدالة is_prime

Python

```python
import pytest

def is_prime(n: int) -> bool:
    if n < 2:
        return False
    return all(n % i != 0 for i in range(2, int(n ** 0.5) + 1))

@pytest.mark.parametrize(
    "number, expected",
    [(2, True), (4, False), (17, True), (1, False), (0, False)],
)
def test_is_prime(number: int, expected: bool) -> None:
    assert is_prime(number) == expected
```

✅🔴 تحدي: استخدم mock لاختبار دالة تعتمد على استدعاء API خارجي

Python

```python
from unittest.mock import patch, MagicMock
import requests

def get_user_name(user_id: int) -> str:
    response = requests.get(f"https://api.example.com/users/{user_id}")
    response.raise_for_status()
    return response.json()["name"]

@patch("requests.get")
def test_get_user_name(mock_get: MagicMock) -> None:
    mock_response = MagicMock()
    mock_response.json.return_value = {"name": "أحمد"}
    mock_get.return_value = mock_response

    result = get_user_name(1)

    assert result == "أحمد"
    mock_get.assert_called_once_with("https://api.example.com/users/1")
```

الفصل 26: من الكود إلى المنتج

✅ 🟢 تمرين: أنشئ ملف .gitignore مناسب لمشروع بايثون

```
.venv/
__pycache__/
*.pyc
.env
dist/
*.egg-info/
.pytest_cache/
.coverage
htmlcov/
.mypy_cache/
.ruff_cache/
```

✅ 🟡 تمرين: أنشئ مكتبة math_helpers كاملة وانشرها على TestPyPI

Python src/math_helpers/__init__.py

```python
from math_helpers.core import is_prime, fibonacci

__all__ = ["is_prime", "fibonacci"]
__version__ = "0.1.0"

# src/math_helpers/core.py
def is_prime(n: int) -> bool:
    if n < 2:
        return False
    return all(n % i != 0 for i in range(2, int(n ** 0.5) + 1))

def fibonacci(n: int) -> list[int]:
    sequence = [0, 1]
    while len(sequence) < n:
        sequence.append(sequence[-1] + sequence[-2])
    return sequence[:n]
```

Terminal

```
uv build
uv publish --publish-url https://test.pypi.org/legacy/ --token XXX
```

خاتمة الملحق

هذا الملحق ليس بديلاً عن محاولتك الشخصية لحل التمارين — بل مرجعًا تعود إليه عند التعثر أو للمقارنة بعد الانتهاء من حلّك الخاص. أفضل طريقة للاستفادة منه: **حاول، تعثّر، فكّر، ثم قارن**. هذه الدورة بالذات هي ما يبني مهارة البرمجة الحقيقية، وليس قراءة الحلول مباشرة.

ملحق · الملاحق

جدول مرجعي سريع (Cheat Sheet) شامل

2 دقيقة قراءة · 22 قسمًا · 25 مثالًا · 363 كلمة

في هذا الملحق

لماذا هذا الملحق؟
أساسيات اللغة
النصوص (Strings)
هياكل البيانات
التحكم بسير التنفيذ
الدوال
معالجة الأخطاء
البرمجة الكائنية
الدوال المتقدمة
Type Hints مرجع سريع
الملفات والبيانات
البرمجة غير المتزامنة
إدارة المشاريع (uv)
FastAPI مرجع سريع
pandas/NumPy مرجع سريع
scikit-learn مرجع سريع
click (CLI) مرجع سريع
جودة الكود
الاختبار (pytest)
التحزيم والنشر
جدول الدوال المدمجة (Built-in Functions) الأكثر استخدامًا
خاتمة الملحق

لماذا هذا الملحق؟

هذا الملحق هو **مرجعك اليومي** بعد إغلاق الكتاب. لا تحتاج لحفظ أي شيء هنا عن ظهر قلب — الهدف أن تعرف «أين أبحث» عندما تنسى صياغة معينة أثناء العمل على مشروع حقيقي. احتفظ بهذا الملحق مفتوحًا بجانبك دائمًا.

أساسيات اللغة

المتغيرات والأنواع

```python
# التعريف مع Type Hints
name: str = "محمد"
age: int = 30
price: float = 19.99
is_active: bool = True
data: None = None

# تحويل الأنواع
int("42")          # 42
float("3.14")      # 3.14
str(100)           # "100"
bool(0)            # False
list("abc")        # ['a', 'b', 'c']

# فحص النوع
type(x)              # النوع الفعلي
isinstance(x, int)   # فحص إن كان من نوع معيّن
```

الأنواع الأساسية

النوع	مثال	قابل للتعديل؟
`int`	`42`	لا
`float`	`3.14`	لا
`str`	`"text"`	لا
`bool`	`False` / `True`	لا
`list`	`[1, 2, 3]`	نعم
`tuple`	`(1, 2, 3)`	لا
`dict`	`{"a": 1}`	نعم
`set`	`{1, 2, 3}`	نعم
`None`	`None`	—

عمليات حسابية ومقارنة

```python
+  -  *  /       # عمليات أساسية (/ تُنتج float دائمًا)
//              # قسمة صحيحة (floor division)
%               # باقي القسمة
**              # قوة (أس)

==   !=  <  >  <= >=     # مقارنة
and  or  not             # منطقية
```

F-strings

```python
name, age = "سارة", 25
f"{name} عمرها {age}"
f"{price:.2f}"           # تنسيق عشري بمنزلتين
f"{number:,}"            # فاصلة آلاف
f"{value:>10}"           # محاذاة يمين بعرض 10
f"{value=}"              # يطبع "value=10" (مفيد للتنقيح)
```

النصوص (Strings)

```python
s = "Hello World"

s.upper()           # "HELLO WORLD"
s.lower()           # "hello world"
s.strip()           # إزالة المسافات من الطرفين
s.replace("a", "b") # استبدال
s.split(" ")        # تقسيم لقائمة
"-".join(["a","b"]) # دمج قائمة لنص
s.startswith("He")  # True
s.endswith("ld")    # True
s.find("World")     # 6 (أو -1 إن لم يوجد)
len(s)              # طول النص
s[0]                # أول حرف
s[-1]               # آخر حرف
s[0:5]              # تقطيع (Slicing)
s[::-1]             # عكس النص
```

هياكل البيانات

القوائم (Lists)

Python

```
lst = [1, 2, 3]

lst.append(4)          # إضافة في النهاية
lst.insert(0, 0)       # إضافة في موضع محدد
lst.remove(2)          # إزالة أول قيمة مطابقة
lst.pop()              # إزالة وإرجاع آخر عنصر
lst.pop(0)             # إزالة وإرجاع عنصر بموضع محدد
lst.sort()             # ترتيب في المكان
sorted(lst)            # ترتيب في نسخة جديدة
lst.reverse()          # عكس في المكان
len(lst)               # الطول
lst[1:3]               # تقطيع
2 in lst               # فحص الوجود

# List Comprehension
squares = [x**2 for x in range(10)]
evens = [x for x in range(20) if x % 2 == 0]
```

القواميس (Dict)

Python

```
d = {"name": "علي", "age": 30}

d["name"]              # قراءة
d["city"] = "الرياض"   # إضافة/تعديل
d.get("key", default)  # قراءة آمنة بقيمة افتراضية
d.keys()               # المفاتيح
d.values()             # القيم
d.items()              # أزواج (مفتاح، قيمة)
del d["age"]           # حذف
"name" in d            # فحص وجود مفتاح

# Dict Comprehension
squares = {x: x**2 for x in range(5)}
```

المجموعات (Set)

Python

```
s = {1, 2, 3}

s.add(4)
s.remove(2)
s1 | s2    # اتحاد
s1 & s2    # تقاطع
s1 - s2    # فرق
s1 ^ s2    # فرق متماثل
```

الصفوف (Tuple)

Python

```
t = (1, 2, 3)
a, b, c = t          # فك التغليف (Unpacking)
```

التحكم بسير التنفيذ

Python

```python
# شرطية
if condition:
    ...
elif other_condition:
    ...
else:
    ...

# التعبير الشرطي (Ternary)
result = "نعم" if condition else "لا"

# حلقة for
for item in iterable:
    ...
for i in range(10):          # 0 → 9
for i, item in enumerate(lst):  # مع الفهرس
for a, b in zip(list1, list2):  # تكرار متوازٍ

# حلقة while
while condition:
    ...

# كلمات تحكم
break       # إيقاف الحلقة
continue    # تخطي الدورة الحالية
pass        # عدم فعل شيء (حجز مكان)
```

الدوال

Python

```python
def greet(name: str, greeting: str = "مرحبًا") -> str:
    """Docstring يشرح الدالة."""
    return f"{greeting}, {name}!"

# وسائط متغيرة
def total(*args: int) -> int:
    return sum(args)

def config(**kwargs: str) -> None:
    for key, value in kwargs.items():
        print(key, value)

# Lambda
square = lambda x: x ** 2

# Type Hints شائعة
def process(items: list[str]) -> dict[str, int]: ...
def maybe(value: int | None = None) -> bool: ...
```

معالجة الأخطاء

Python

```python
try:
    risky_operation()
except ValueError as e:
    print(f"خطأ: {e}")
except (TypeError, KeyError):
    print("أحد هذين الخطأين")
else:
    print("نُفِّذ بدون أخطاء")
finally:
    print("يُنفَّذ دائمًا")

raise ValueError("رسالة الخطأ")

class CustomError(Exception):
    pass
```

الاستثناءات الشائعة

الاستثناء	متى يحدث
`ValueError`	قيمة من النوع الصحيح لكن غير صالحة
`TypeError`	عملية غير مدعومة لنوع البيانات
`KeyError`	مفتاح غير موجود في قاموس
`IndexError`	فهرس خارج نطاق القائمة
`ZeroDivisionError`	قسمة على صفر
`FileNotFoundError`	ملف غير موجود
`AttributeError`	خاصية/تابع غير موجود في الكائن

البرمجة الكائنية

```python
class Animal:
    species_count = 0                # خاصية صنف (Class Attribute)

    def __init__(self, name: str) -> None:
        self.name = name             # خاصية كائن (Instance Attribute)
        Animal.species_count += 1

    def speak(self) -> str:
        return f"{self.name} يصدر صوتًا"

    def __str__(self) -> str:
        return f"Animal({self.name})"

    def __repr__(self) -> str:
        return f"Animal(name={self.name!r})"

    def __eq__(self, other: object) -> bool:
        if not isinstance(other, Animal):
            return NotImplemented
        return self.name == other.name

class Dog(Animal):                   # الوراثة
    def speak(self) -> str:          # تجاوز (Override)
```

```
        return f"{self.name} ينـبح!"

# Dataclass
from dataclasses import dataclass, field

@dataclass
class Point:
    x: float
    y: float
    tags: list[str] = field(default_factory=list)

# Property
class Circle:
    def __init__(self, radius: float) -> None:
        self._radius = radius

    @property
    def area(self) -> float:
        return 3.14159 * self._radius ** 2
```

Magic Methods الشائعة

التابع	يُستدعى عند
`__init__`	إنشاء الكائن
`__str__`	`str(obj)` / `print(obj)`
`__repr__`	التمثيل التقني (في Shell)
`__eq__`	`obj1 == obj2`
`__lt__`	`obj1 < obj2`
`__len__`	`len(obj)`
`__getitem__`	`obj[key]`
`__iter__`	استخدام الكائن في حلقة `for`

الدوال المتقدمة

Python

```python
# Decorator
from functools import wraps

def my_decorator(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        print("قبل التنفيذ")
        result = func(*args, **kwargs)
        print("بعد التنفيذ")
        return result
    return wrapper

@my_decorator
def say_hello():
    print("مرحبًا")

# Generator
def count_up(n: int):
    for i in range(n):
        yield i

# Closure
def make_multiplier(factor: int):
    def multiply(x: int) -> int:
        return x * factor
    return multiply

double = make_multiplier(2)
```

Type Hints مرجع سريع

Python

```python
from typing import TypeVar, Generic
from collections.abc import Callable, Iterator, Iterable

x: int
x: str | None                    # Union (بايثون 3.10+)
x: list[int]
x: dict[str, int]
x: tuple[int, str]
x: tuple[int, ...]               # عدد غير محدد من نفس النوع
x: Callable[[int, int], int]     # دالة تأخذ int,int وتُعيد int
x: Iterator[int]

T = TypeVar("T")
class Box(Generic[T]):
    def __init__(self, item: T) -> None:
        self.item = item
```

الملفات والبيانات

قراءة/كتابة ملفات

Python

```python
from pathlib import Path

p = Path("data.txt")
p.write_text("محتوى", encoding="utf-8")
content = p.read_text(encoding="utf-8")
p.exists()
p.is_file()
p.mkdir(parents=True, exist_ok=True)

with open("file.txt", "r", encoding="utf-8") as f:
    content = f.read()
```

JSON

```python
import json

json.dumps(data, ensure_ascii=False, indent=2)    # كائن → نص JSON
json.loads(json_string)                           # نص JSON → كائن
Path("data.json").write_text(json.dumps(data))
```

CSV

```python
import csv

with open("file.csv", newline="", encoding="utf-8") as f:
    reader = csv.DictReader(f)
    for row in reader:
        print(row)

with open("out.csv", "w", newline="", encoding="utf-8") as f:
    writer = csv.DictWriter(f, fieldnames=["name", "age"])
    writer.writeheader()
    writer.writerow({"name": "علي", "age": 30})
```

البرمجة غير المتزامنة

```python
import asyncio

async def fetch(url: str) -> str:
    await asyncio.sleep(1)
    return f"data from {url}"

async def main() -> None:
    result = await fetch("url")
    results = await asyncio.gather(fetch("a"), fetch("b"))

asyncio.run(main())
```

إدارة المشاريع (uv)

Terminal

```
uv init my_project              # مشروع جديد
uv init --lib my_lib            # مكتبة قابلة للنشر
uv add requests                 # إضافة تبعية
uv add --dev pytest             # تبعية تطوير فقط
uv remove requests              # إزالة تبعية
uv sync                         # مزامنة البيئة مع pyproject.toml
uv run main.py                  # تشغيل سكربت داخل البيئة
uv run pytest                   # تشغيل أمر داخل البيئة
uv build                        # بناء الحزمة (sdist + wheel)
uv publish                      # نشر على PyPI
uv python install 3.14          # تثبيت إصدار بايثون معين
```

Git أساسيات

Terminal

```
git init
git status
git add .
git commit -m "رسالة الالتزام"
git log --oneline
git branch new-feature
git checkout new-feature        # أو: git switch new-feature
git merge new-feature
git push origin main
git pull origin main
git clone <url>
```

FastAPI مرجع سريع

Python

```python
from fastapi import FastAPI, HTTPException, Query
from pydantic import BaseModel

app = FastAPI()

class Item(BaseModel):
    name: str
    price: float

@app.get("/items/{item_id}")
def read_item(item_id: int, q: str | None = Query(None)) -> dict:
    return {"item_id": item_id, "q": q}

@app.post("/items")
def create_item(item: Item) -> Item:
    return item

@app.put("/items/{item_id}")
def update_item(item_id: int, item: Item) -> dict:
    return {"item_id": item_id, **item.model_dump()}

@app.delete("/items/{item_id}")
def delete_item(item_id: int) -> dict:
    if item_id not in db:
        raise HTTPException(status_code=404, detail="غير موجود")
    return {"message": "تم الحذف"}
```

Terminal

```
uv run uvicorn main:app --reload    # تشغيل محلي
# التوثيق التفاعلي: http://localhost:8000/docs
```

pandas/NumPy مرجع سريع

Python

```python
import pandas as pd
import numpy as np

# NumPy
arr = np.array([1, 2, 3])
arr.mean(), arr.sum(), arr.std()

# pandas - القراءة
df = pd.read_csv("file.csv")
df.head()
df.info()
df.describe()

# الاختيار
df["column"]
df.loc[0, "column"]        # بالتسمية
df.iloc[0, 0]               # بالموضع
df[df["age"] > 18]          # فلترة

# المعالجة
df.dropna()
df.fillna(0)
df.groupby("category")["sales"].sum()
df.sort_values("price", ascending=False)
df.merge(other_df, on="id")
pd.pivot_table(df, values="sales", index="region", aggfunc="sum")
```

matplotlib

Python

```python
import matplotlib.pyplot as plt

plt.plot(x, y)
plt.bar(categories, values)
plt.scatter(x, y)
plt.hist(data, bins=20)
plt.title("العنوان")
plt.xlabel("x"); plt.ylabel("y")
plt.savefig("chart.png")
plt.show()
```

scikit-learn مرجع سريع

Python

```python
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, mean_squared_error

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = RandomForestClassifier()
model.fit(X_train, y_train)
predictions = model.predict(X_test)
accuracy_score(y_test, predictions)

pipeline = Pipeline([
    ("scaler", StandardScaler()),
    ("model", LinearRegression()),
])
pipeline.fit(X_train, y_train)

import joblib
joblib.dump(model, "model.pkl")
model = joblib.load("model.pkl")
```

مرجع سريع click (CLI)

Python

```python
import click

@click.command()
@click.argument("name")
@click.option("--greeting", default="مرحبًا", help="نص الترحيب")
@click.option("--loud", is_flag=True, help="بأحرف كبيرة")
def hello(name: str, greeting: str, loud: bool) -> None:
    message = f"{greeting}, {name}!"
    click.echo(message.upper() if loud else message)

@click.group()
def cli(): ...

@cli.command()
def sub_command(): ...

if __name__ == "__main__":
    hello()
```

TOML

```toml
# pyproject.toml - لتحويل الأداة لأمر نظام
[project.scripts]
mytool = "package.cli:cli"
```

جودة الكود

Terminal

```
uv add --dev ruff black mypy
uv run ruff check .          # فحص الأخطاء الأسلوبية
uv run ruff check --fix .    # إصلاح تلقائي
uv run black .               # تنسيق تلقائي
uv run mypy src/             # فحص الأنواع
```

مبادئ سريعة

المبدأ	الفكرة
DRY	Don't Repeat Yourself — لا تكرر الكود
KISS	Keep It Simple, Stupid — ابقَ بسيطًا
YAGNI	You Aren't Gonna Need It — لا تُضف ميزات غير ضرورية الآن
SRP	مسؤولية واحدة لكل دالة/صنف
Guard Clauses	تحقّق من الشروط السلبية أولاً واخرج مبكرًا

الاختبار (pytest)

Python

```python
import pytest

def test_simple():
    assert 1 + 1 == 2

def test_exception():
    with pytest.raises(ValueError):
        int("abc")

def test_float():
    assert 0.1 + 0.2 == pytest.approx(0.3)

@pytest.fixture
def sample_data():
    return {"name": "test"}

def test_with_fixture(sample_data):
    assert sample_data["name"] == "test"

@pytest.mark.parametrize("input,expected", [(1, 1), (2, 4), (3, 9)])
def test_square(input, expected):
    assert input ** 2 == expected
```

Terminal

```
uv run pytest                                # تشغيل كل الاختبارات
uv run pytest -v                             # تفصيلي
uv run pytest tests/test_file.py             # ملف محدد
uv run pytest -k "test_name"                 # حسب الاسم
uv run pytest --cov=src --cov-report=term-missing   # مع التغطية
```

Mocking

Python

```python
from unittest.mock import patch, MagicMock

@patch("module.requests.get")
def test_api_call(mock_get):
    mock_get.return_value.json.return_value = {"key": "value"}
    result = my_function()
    mock_get.assert_called_once()
```

التحزيم والنشر

Terminal

```
uv init --lib my_lib
uv build
uv publish --publish-url https://test.pypi.org/legacy/ --token XXX    # تجريبي
uv publish --token XXX                                                 # حقيقي
```

Dockerfile

```
FROM python:3.14-slim
COPY --from=ghcr.io/astral-sh/uv:latest /uv /usr/local/bin/uv
WORKDIR /app
COPY pyproject.toml uv.lock ./
RUN uv sync --frozen --no-dev
COPY src/ ./src/
EXPOSE 8000
CMD ["uv", "run", "uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

```
Terminal
```

```
docker build -t myapp:1.0 .
docker run -p 8000:8000 myapp:1.0
```

جدول الدوال المدمجة (Built-in Functions) الأكثر استخدامًا

الدالة	الاستخدام
`len()`	طول تسلسل
`range()`	توليد متتالية أرقام
`print()`	طباعة
`input()`	قراءة إدخال من المستخدم
`type()`	نوع الكائن
`isinstance()`	فحص النوع
`enumerate()`	تكرار مع فهرس
`zip()`	دمج تسلسلات متوازية
`sorted()`	ترتيب بنسخة جديدة
`map()`	تطبيق دالة على كل عنصر
`filter()`	تصفية عناصر حسب شرط
`min()` / `max()` / `sum()`	عمليات تجميعية
`any()` / `all()`	فحص منطقي جماعي
`open()`	فتح ملف

خاتمة الملحق

احتفظ بهذا الملحق كمرجع دائم. مع الوقت، ستلاحظ أنك تراجعه أقل فأقل — ليس لأنك نسيت بايثون، بل لأنها أصبحت جزءًا من طريقة تفكيرك.

وصلت إلى آخر صفحة

إن كنت قد قرأت الكتاب كاملًا وحلّلت تمارينه، فأنت لم تعد مبتدئًا. أنت الآن تعرف كيف تفكّر، وكيف تبحث، وكيف تبني — وهذا أهم بكثير من أن تحفظ.

الخطوة التالية ليست كتابًا آخر، بل مشروع. اختر فكرة صغيرة تهمّك شخصيًا، وابنِها من أول سطر إلى آخر. ستكتشف أن 90% مما واجهته في هذا الكتاب يعود إليك، لكن هذه المرة بثقة. وحين تتعثّر — وستتعثّر — عد إلى الملحق ج، ثم إلى التوثيق الرسمي، ثم اسأل. هذه هي الدورة التي لن تتوقف بعد اليوم.